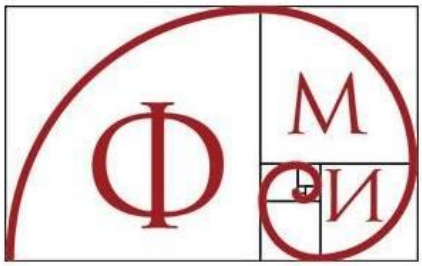
СОФИЙСКИ УНИВЕРСИТЕТ „СВ. КЛИМЕНТ ОХРИДСКИ“

ФАКУЛТЕТ ПО МАТЕМАТИКА И ИНФОРМАТИКА

ЗАПИСКИ ЗА ПОДГОТОВКА ЗА ДЪРЖАВЕН ИЗПИТ ЗА ПРИДОБИВАНЕ НА ОКС „БАКАЛАВЪР“ ПО СПЕЦИАЛНОСТ „СОФТУЕРНО ИНЖЕНЕРСТВО“





Стефан Велев

2024

СЪДЪРЖАНИЕ

Тема **№ 1** – Графи. Дървета. Обхождане на графи. 4

Тема **№ 2** – Булеви функции. Пълнота. 13

Тема **№ 3** – Крайни автомати. Регулярни езици. Теорема на Клини. 20

Тема **№ 4** – Контекстно-свободни граматики и езици. Стекови автомати..... 31

Тема **№ 5** – Компютърни архитектури. Формати на данните. Вътрешна структура на централен процесор – блокове и конвейерна обработка, инструкции. 37

Тема **№ 6** – Структура и йерархия на паметта. Сегментна и странична преадресация. Системи за прекъсване – приоритети и обслужване. 53

Тема **№ 7** – Процеси и комуникация между тях в операционната система. 73

Тема **№ 8** – Компютърни мрежи и протоколи – OSI модел. Протоколи IPv4, IPv6, TCP, HTTP..... 82

Тема **№ 9** – Бази от данни. Релационен модел на данните. 94

Тема **№ 10** – Бази от данни. Нормални форми..... 103

Тема **№ 11** – Процедурно програмиране – основни конструкции. 111

Тема **№ 12** – Процедурно програмиране – указатели, масиви и рекурсия..... 132

Тема **№ 13** – Обектно-ориентирано програмиране. Основни принципи. Класове и обекти. Наследяване и капсулация..... 146

Тема **№ 14** – Обектно-ориентирано програмиране. Подтипов и параметричен полиморфизъм. Множествено наследяване..... 162

Тема **№ 15** – Структури от данни. Стек, опашка, списък, дърво. Основни операции върху тях. Реализация..... 175

Тема **№ 16** – Софтуерно инженерство и неговото място като дял от знанието. Софтуерен процес и модели на софтуерни процеси. Концепция за многократна употреба..... 206

Тема **№ 17** – Модели на разпределени софтуерни архитектури. Среди и протоколи за разпределени приложения. 221

Тема **№ 18** – Модели и методи за проектиране на потребителски интерфейс..... 232

Тема **№ 19** – Управление на качеството на софтуерни приложения. Тестване на софтуер. 240

Тема **№ 20** – Софтуерна архитектура. Проектиране и документиране на софтуерни архитектури. .. 250

Тема **№ 21** – Инженеринг на софтуерните изисквания. Техники за извличане, анализ и валидиране на софтуерните изисквания. Специфициране на изискванията. 262

Тема **№ 22** – Проектиране и интегриране на софтуерни системи..... 276

Тема **№ 23** – Планиране на проекта – същност и основни елементи, обхват на проекта, времеви и финансови ресурси. Дейности по управление и контрол, методи и средства за създаване на план-график на проекта. 281

Тема **№ 24** – Използване на XML за структуриране, валидация, обработка и представяне на документно съдържание. 287

Тема **№ 25** – Теорема на Ферма. Теореми за средните стойности (Рол, Лагранж и Коши). Формула на Тейлър. 303

Тема **№ 26** – Определен интеграл. Дефиниция и свойства. Интегруемост на непрекъснатите функции. Теорема на Нютон-Лайбниц. 311

Тема **№ 27** – Ранг на система вектори. Ранг на матрица. Теорема за равенството на ранг на матрица, ранг на системата от вектор-редове и ранг на системата от вектор-стълбове на тази матрица. Системи линейни уравнения – теорема на Руше, фундаментална система от решения за хомогенна система и теорема за размерността на подпространството от решения на хомогенна система. 320

Тема **№ 28** – Полиноми на една променлива – определение, действия с полиноми и степен на полином. Теорема за делене на полиноми с частно и остатък, правило на Хорнер. Най-голям общ делител на полиноми – тъждество на Безу и алгоритъм на Евклид. Корени на полином и зависимости между корените и коефициентите на полином (формули на Виет). 326

Тема **№ 29** – Дискретни разпределения. Равномерно, биномно, геометрично и Поасоново разпределение. Задачи, в които възникват. Моменти – математическо очакване и дисперсия. 335

За написването на записките са използвани предоставените за ползване материали от съответните университетски курсове, както и основната литература от официалния конспект.

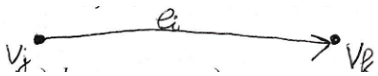
Държавен изпит – спец. „Софтуерно инженерство“

Тема №1 – Графи. Дървета. Обхождане на графи.

Анотация: *Дефиниции за краен ориентиран (мулти)граф и краен неориентиран (мулти)граф. Дефиниции за маршрут (контур) в ориентиран мултиграф и път (цикъл) в неориентиран мултиграф. Свързаност и свързани компоненти на граф. Дефиниция на дърво и кореново дърво. Доказателство, че всяко кореново дърво е дърво и |V|=|E|+1. Покриващо дърво на граф. Обхождане на граф в ширина и дълбочина. Ойлерови обхождания на мултиграф. Теореми за съществуване на Ойлеров цикъл (с доказателство) и Ойлеров път.*

Дефиниция (краен ориентиран мултиграф): Краен ориентиран мултиграф наричаме наредената тройка $G =< V, E, f_G >$, където $V = \{v_1, v_2, \dots, v_n\}$ е крайно множество от елементи, наричани върхове, $E = \{e_1, e_2, \dots, e_m\}$ е крайно множество от елементи, наричани ребра и f_G е функция, наречена свързваща функция $f_G: E \to V \times V$, съпоставяща на всяко ребро наредена двойка от върхове

Прилагателното краен се използва, за да е ясно, че V и E са крайни множества. Обикновено, $V = \{v_1, v_2, \dots, v_n\}$ *и* $E = \{e_1, e_2, \dots, e_m\}$. *Ако* $f(e_i) = (v_j, v_k)$, *то казваме, че реброто* e_i *има за начало* v_j *и край* v_k *и го изобразяваме така:*



Пример: Следният мултиграф $G =< V, E, f_G >$ *дефинираме по следния начин:*

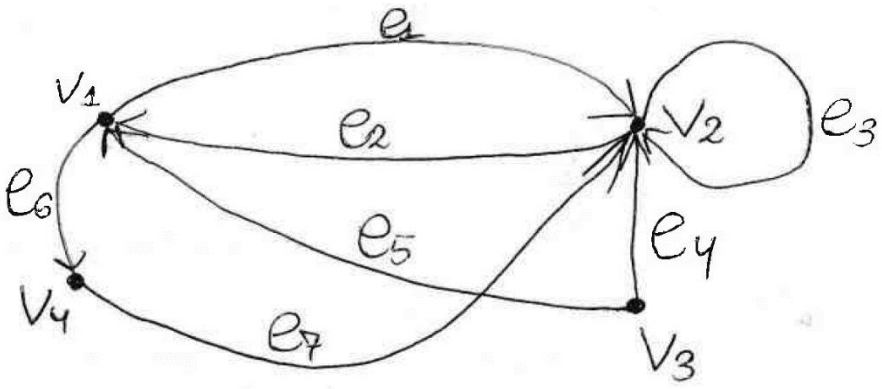
$$V = \{v_1, v_2, v_3, v_4\},$$

$$E = \{e_1, e_2, e_3, e_4, e_5, e_6, e_7\},$$

$$f_G(e_1) = (v_1, v_2), f_G(e_2) = (v_2, v_1), f_G(e_3) = (v_2, v_2), f_G(e_4) = (v_3, v_2),$$

$$f_G(e_5) = (v_3, v_1), f_G(e_6) = (v_1, v_4), f_G(e_7) = (v_4, v_2)$$

и изобразяваме така:



Обратно, по зададена такава диаграма, G е определен напълно с върховете си $\{v_1, v_2, v_3, v_4\}$, *ребрата си* $\{e_1, e_2, e_3, e_4, e_5, e_6, e_7\}$ *и* f_G *със стрелките си и етикети на всяка стрелка.*

Ако $f_G(e_j) = (v_i, v_i)$, *то тази стрелка се нарича примка. На горния пример* e_3 *е примка.*

Дефиниция (краен ориентиран граф): Краен ориентиран граф $G(\mathrm{V}, \mathrm{E}, f_G)$ наричаме краен ориентиран мултиграф $\mathrm{G}(\mathrm{V}, \mathrm{E}, \mathrm{f_G})$, за който f_G е инективна, т.е. $f(e_i) \neq f(e_j), i \neq j$.

Когато $G(V, E, f_G)$ е краен ориентиран граф, то f_G става излишна, защото можем да отъждествим всяко ребро е с (v_i, v_j) при условие, че $f_G(e) = (v_i, v_j)$ (ролята на ребро се играе от наредената двойка (v_i, v_j)).

Посоченият по-горе краен мултиграф, всъщност е краен ориентиран граф. При посоченото отъждествяване, ние можем да означим G само като G=<V, E>, където $E \subseteq V^2$. С други думи, част от наредените двойки са ребра. Така еквивалентно можем да дадем следната дефиниция:

Ориентиран граф е наредена двойка G=<V, E>, където $\mathrm{E} \subseteq \mathrm{V}^2$. Вместо $G =< V, E >$ се използва и $G(V, E)$, като с това се посочват върхове (елементи на V) и ребрата (елементи на E).

Разгледаният по-горе пример на (ориентиран) мултиграф, което се оказа краен ориентиран граф, можем да запишем така:

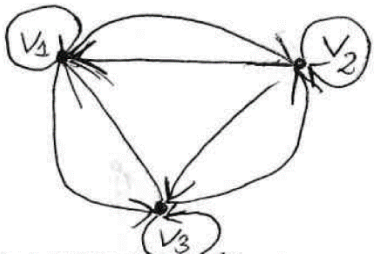
$$V = \{v_1, v_2, v_3, v_4\}, E = \{(v_1, v_2), (v_2, v_1), (v_2, v_2), (v_3, v_2), (v_3, v_1), (v_1, v_4), (v_4, v_2)\}.$$

Дефиниция (краен неориентиран граф): Краен неориентиран граф $G(V, E)$ или просто граф наричаме краен ориентиран граф, за който E, разглеждана като релация във V, е рефлексивна и симетрична.

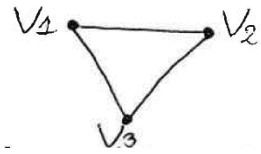
Пример:

$$V = \{v_1, v_2, v_3\}$$

$$E = \{(v_1, v_1), (v_2, v_2), (v_3, v_3), (v_1, v_2), (v_2, v_1), (v_2, v_3), (v_3, v_2), (v_1, v_3), (v_3, v_1)\}$$



Оттук нататък за неориентираните графи ще използваме следната диаграма:



т.е. примките се подразбират и отсечките заместват стрелките в двете посоки. Отсечките се наричат ребра.

Дефиниция (съседни върхове в граф): Нека $G(V, E)$ е граф. Ако $(v_i, v_j) \in E$, то v_i и v_j се наричат съседни (тогава и $(v_j, v_i) \in E$).

Дефиниция (степен на връх в граф): Нека $G(V, E)$ е граф. Степен на един връх $v \in V$ се нарича броят на ребрата (отсечките) с край върха $v \in V$.

Ако графът е ориентиран, казваме, че v_i е баща на v_j ($(v_i, v_j) \in E$).

Върховете от степен 0 наричаме изолирани.

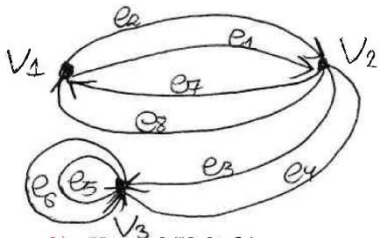
Дефиниция (полустепен на входа/изхода): Нека $G(V, E, f_G)$ е краен мултиграф. Полустепен на входа (на изхода) на v_i се нарича броят на ребрата, които завършват (започват) във v_i.

Дефиниция (подмултиграф, породен от граф): Нека $G(V, E, f_G)$ е краен мултиграф и $V' \subseteq V$. Подмултиграф $G'(V', E', f'_G)$, породен от V' на G се определя по следния начин: $E' = \{e' | \text{съществуват } v' \text{и } v'' \in V', \text{такива че } f_G(v', v'') = e'\}$ и $f'_G(e) = f_G(e)$ за всички $e \in E'$, т.е. $f'_G = f_G|_{E'}$ (f'_G е рестрикцията на f_G върху E').

Дефиниция (изоморфни графи): Нека $G(V, E)$ и $G'(V', E')$ са два (ориентирани) графа. Казваме, че те са изоморфни, ако съществува биекция: $f: V \to V'$, такава че за произволни $u, v \in V$ е изпълнена еквивалентността: $(u, v) \in E \Leftrightarrow (f(u), f(v)) \in E'$.

Дефиниция (матрица на съседство на граф): Нека $G(V, E, f_G)$ е краен мултиграф, като $V = \{v_1, \dots, v_n\}$. Матрицата $M = \|a_{ij}\|_{i,j=1}^{n}$ се нарича матрица на съседства на G, ако: $a_{ij} = \left|\{e | e \in E \text{ и } f_G(e) = (v_i, v_j)\}\right|, i, j = 1, \dots, n$

Пример:



$$\|a_{ij}\|_{i,j=1}^{3} = M = \begin{pmatrix} 0 & 2 & 0 \\ 2 & 0 & 2 \\ 0 & 0 & 2 \end{pmatrix}$$

Дефиниция (маршрут в ориентиран мултиграф): Нека $G(V, E, f_G)$ е краен мултиграф. Маршрут в крайния мултиграф G се нарича редицата: $v_{i_0}, e_{j_0}, v_{i_1}, e_{j_1}, v_{i_2}, \dots, v_{i_{l-1}}, e_{j_{l-1}}, v_{i_l}$, такава, че $f\left(e_{j_{p-1}}\right) = \left(v_{i_{p-1}}, v_{i_p}\right), p = 1, \dots, l$, където l се нарича дължина на маршрута.

Дефиниция (контур в ориентиран мултиграф): Ако $v_{i_l} = v_{i_0}$, то тогава редицата $v_{i_0}, e_{j_0}, v_{i_1}, e_{j_1}, v_{i_2}, \dots, v_{i_{l-1}}, e_{j_{l-1}}, v_{i_l}$ наричаме контур.

Дефиниция (път в неориентиран граф): Нека $G(V, E)$ е даден граф. Път в крайния неориентиран граф G се нарича всяка редица $v_{i_0}, v_{i_1}, \dots, v_{i_l}$, такава че $\left(v_{i_{p-1}}, v_{i_p}\right) \in E, v_{i_{p-1}} \neq v_{i_p}$ и $v_{i_{p-1}} \neq v_{i_{p+1}}, p = 1, \dots, l$.

l се нарича дължина на пътя. v_{i_0} – начало; v_{i_l} – край на пътя.
Освен това в пътя не се разрешават примки

$\left(v_{i_{p-1}} \neq v_{i_p}\right)$ и не се разрешава да се отиде от $v_{i_{p-1}}$ до v_{i_p}

(ако те са различни) и след това да се върне

обратно до $v_{i_{p-1}}$ $(v_{i_{p-1}} \neq v_{i_{p+1}})$.

Дефиниция (цикъл в неориентиран граф): Цикъл в един граф $G(V, E)$ се нарича всеки път с дължина по-голяма или равна на 3, чиито начало и край съвпадат.

Дефиниция (релацията $\mathcal{P}_G$ в граф): За всеки граф $G(V,E)$ определяме релацията $\mathcal{P}_G, \mathcal{P}_G \subseteq V^2$ както следва:

$$\mathcal{P}_G = \{(v,v)|v \in V\} \cup \{(u,v)|\text{съществува път в графа } G \text{ с начало } u \text{ и край } v\}.$$

Теорема: Релацията $\mathcal{P}_G$ във V е релация на еквивалентност. Всеки клас на еквивалентност относно $\mathcal{P}_G$ се нарича свързана компонента на G или компонента на свързаност.

Дефиниция (свързан граф): Графът $G(V,E)$ се нарича свързан, ако за всеки два върха $u, v \in V$ съществува път от u до v, или с други думи, ако $\mathcal{P}_G$ има единствен клас на еквивалентност (единствена компонента на свързаност).

Дефиниция (ацикличен граф): Казваме, че един граф е ацикличен, ако в него няма цикли.

Дефиниция (дърво): Нека $G(V,E)$ е граф. Казваме, че G е дърво, ако в G няма цикли и той е свързан.

Един граф е дърво т.т.к. той е свързан, ацикличен граф.

Дефиниция (кореново дърво – индуктивна):

А.) Нека r е произволен елемент. Тогава $D(\{r\}, \emptyset)$ е дърво с корен r и единствено листо r, без никакви ребра.

Б.) Нека $D(V,E)$ е дърво с корен r и листа $l_1, \dots, l_r$. Нека освен това $u \in V$ и $w \notin V$. Тогава $D'(V',E')$ е дърво, където $V' = V \cup \{w\}$ и $E' = E \cup \{(u,w)\}$. При това ако $u \notin \{l_1, \dots, l_r\}$, то листата на новополученото дърво са $l_1, \dots, l_r, w$, а ако $u = l_i$, то листата на новополученото дърво са $l_1, \dots, l_{i-1}, w, l_{i+1}, \dots, l_r$ (присъединяване на връх).

В.) Други коренови дървета няма.

Теорема: Всяко дърво с корен (кореново дърво) е дърво.

Доказателство: Ще го проведем с индукция относно построението на кореновото дърво:

А.) $D(\{r\}, \emptyset)$ е тривиалното дърво, което е свързан граф, защото съществува тривиалният път от r до r с дължина 0. Освен това в D няма цикли, тъй като няма ребра. Следователно, D е свързан, ацикличен граф, т.е. е дърво.

Б.) Нека $D(V,E)$ е ацикличен свързан граф и $D'(V',E')$ е получен от $D(V,E)$ като $V' = V \cup \{w\}, E' = E \cup \{(u,w)\}$, за някое $u \in V$ и $w \notin V$. Да допуснем, че D' има цикъл. Този цикъл задължително ще съдържа новият връх w, съгласно ИП, тъй като по допускане в D няма връх, който да участва в цикъл. Нека това е цикълът: $w, v_{i_1}, \dots, v_{i_l}, w$. Тогава $v_{i_1} \neq v_{i_l}$ съгласно определението за път. От дефиницията за кореново дърво следва, че $u = v_{i_1}$ или $u = v_{i_l}$. Тогава, ако $u = v_{i_l}$, то (w, v_{i_1}) трябва да принадлежи на E, което е невъзможно. Ако пък $u = v_{i_1}$, то $(w, v_{i_l}) \in E$, което също е невъзможно. Следователно, в $\boldsymbol{D'}$ **няма цикъл**, заради полученото противоречие.

Тъй като D е дърво по ИП, то D е свързан граф, т.е. между всеки два върха съществува път. Следователно, съществува такъв път и в D'. Път от произволен връх $v_i \in V$ до w в D'

получаваме, като към съществуващия (съгласно индукционното предположение) в D път от v_i до u, добавим реброто (u, w), с което w е присъединен към u в D'. Не е възможно w да съвпада с върха, предхождащ u, защото $w \notin V$. Следователно, в D' съществува път между всеки два върха, т.е. $\boldsymbol{D'}$ **е свързан граф**.

Окончателно, получихме, че D' е свързан, ацикличен граф, т.е. D' е дърво, с което теоремата е доказана.

Теорема: Нека $D(V, E)$ е кореново дърво с корен r. Тогава: $|V| = |E| + 1$.

Доказателство: Ще проведем доказателството, използвайки принципа на математическата индукция:

А.) За $D(\{r\}, \emptyset)$ твърдението е очевидно, тъй като $|V| = |\{r\}| = 1$ и $|E| = |\emptyset| = 0$ и следователно $|V| = |E| + 1$ е изпълнено.

Б.) Нека допуснем, че $D(V, E)$ е кореново дърво, такова че $|V| = |E| + 1$ и $D'(V', E')$ е получено от D, като $V' = V \cup \{w\}, E' = E \cup \{(u, w)\}$ за някое $u \in V$ и $w \notin V$. Тогава:

$$|V'| = |V \cup \{w\}| = |V| + |\{w\}| = |V| + 1 = (|E| + 1)_{\text{по ИП}} + 1 = (|E| + |\{(u, w)\}|) + 1 = |E'| + 1,$$

което и трябваше да се докаже. С това теоремата е доказана.

Твърдение: Нека $D(V, E)$ е кореново дърво. Тогава съществува единствен път от корена до всеки връх и единствен път между всеки два върха.

Дефиниция (височина на връх в кореново дърво): *Нека $D(V, E)$ е кореново дърво с корен r. Височина на върха $v \in V$ е 0, ако $v = r$ и е дължината на пътя (той е единствен) от r до v в противен случай.*

Дефиниция (височина на кореново дърво): *Нека $D(V, E)$ е кореново дърво с корен r. Височина на D се нарича максимумът от височините на всички върхове.*

Дефиниция (баща/син в кореново дърво): *Нека $D(V, E)$ е кореново дърво с корен r. Ако $D'(V', E')$ се получава от $D(V, E)$ с добавяне на връх $w \notin V, V = V \cup \{w\}, u \in V$ и $E' = E \cup \{(u, w)\}$, то u се нарича баща на w, а w се нарича син на u.*

Дефиниция (разклоненост на връх в кореново дърво): *Нека $D(V, E)$ е кореново дърво с корен r и $v \in V$. Разклоненост на върха v се нарича броят на синовете на v.*

Дефиниция (разклоненост на кореново дърво): *Нека $D(V, E)$ е кореново дърво. Разклоненост на D се нарича максимумът на разклоненостите на върховете $v \in V$.*

Когато разклонеността на едно дърво е 2, то се нарича двоично.

Теорема: *Нека $D(V, E)$ е кореново дърво с височина h и разклоненост m. Тогава D има най-много m^h листа.*

Дефиниция (покриващо дърво на граф): Нека $G(V, E)$ е граф, а $D(V, E')$ е дърво, $E' \subseteq E$. Тогава D се нарича покриващо дърво на G (за G).

Теорема: Графът $G(V,E)$ има покриващо дърво точно тогава когато той е свързан.

Обхождане на граф в ширина: Основно понятие в тази алгоритмична схема е ниво на обхождане, което представлява подмножество на множеството от върховете на свързания граф $G(V,E)$. В резултат се получава разбиване $\{L_0, L_1, \dots, L_k\}$ на V по следния начин:

Нека е даден свързан граф $G(V,E)$:

1.) Избираме начален връх r на обхождането (r играе ролята на корен на покриващото дърво). Полагаме $L_0 = \{r\}$ и $l := 0$. Обявяваме върха r за „обходен“.
2.) Проверяваме дали $L_0 \cup L_1 \cup \dots \cup L_l = V$. Ако това е така, то спираме и обхождането е завършено. В противен случай преминаваме към 3.).
3.) Нека сме обходили върховете от нивата $L_0, \dots, L_l$. Образуваме си нивото L_{l+1} от всички необходени върхове $v \in V$, които са съседни на някой връх от L_l, т.е.:

$$L_{l+1} = \{v | v \notin L_0 \cup L_1 \cup \dots \cup L_l \text{ и съществува } u \in L_l : (u,v) \in E\}.$$

Обявяваме елементите на L_{l+1} за „обходени“, $l := l+1$ и преминаваме към 2.).

Алгоритъм за построяване на покриващо дърво в ширина на даден граф $G(V,E)$:

Дадено: Свързан граф $G(V,E)$

Резултат: Покриващо дърво $D(V,E')$ на G

Процедура:

1.) Коренът r може да ни бъде даден отнапред или ние го избираме. Полагаме $L_0 := \{r\}$. Образуваме си дървото $D_0(V_0, E_0)$, където $V_0 = \{r\} = L_0$, а $E_0 = \emptyset$. Полагаме $l := 0$.
2.) Проверяваме дали $V_l = V$. Ако това е така, то $D_l(V_l, E_l)$ е търсеното покриващо дърво. В противен случай преминаваме към 3.).
3.) Нека $D_l(V_l, E_l)$ е дървото, получено на $l-$та стъпка и L_l са върховете от $l-$то ниво. Образуваме си:

$$L_{l+1} := \{v | v \notin V_l \text{ и съществува } w \in L_l : (v,w) \in E\} = \{v_1^l, \dots v_{p_l}^l\}.$$

Нека освен това $\{w_1^l, \dots w_{p_l}^l\}$ са такива, че $(v_s^l, w_s^l) \in E$ за $s = 1, \dots, p_l$.
Образуваме си дървото $D_{l+1}(V_{l+1}, E_{l+1}), V_{l+1} = V_l \cup L_{l+1}$ и

$$E_{l+1} = E_l \cup \{(v_1^l, w_1^l), \dots, (v_{p_l}^l, w_{p_l}^l)\}.$$

Полагаме $l := l+1$ и преминаваме към 2.).

Обхождане на граф в дълбочина: Тук основни понятия са текущ връх t и баща на t – $p(t)$. Началният връх r и тук е зададен предварително или ние го определяме. Процедурата е следната:

1.) Вземаме началния връх r и го обявяваме за „обходен“. Полагаме $t := r$ и $p(t)$ е неопределен.
2.) Проверяваме дали има необходен връх, съседен на t.
   a. Ако такъв връх v съществува, тогава правим „стъпка напред“, т.е. $p(v) := t, t := v$ и обявяваме v за „обходен“. Преминаваме към 2.).
   b. Ако няма такъв връх v, то имаме две възможности:

i. Ако $t \neq r, p(t)$ е определен. Полагаме $t \coloneqq p(t)$ и преминаваме към 2.) (правим „стъпка назад“).
ii. Ако $t \coloneqq r$, то обхождането е завършило.

Алгоритъм за построяване на покриващо дърво в дълбочина на даден граф $G(V, E)$:

Дадено: Свързан граф $G(V, E)$

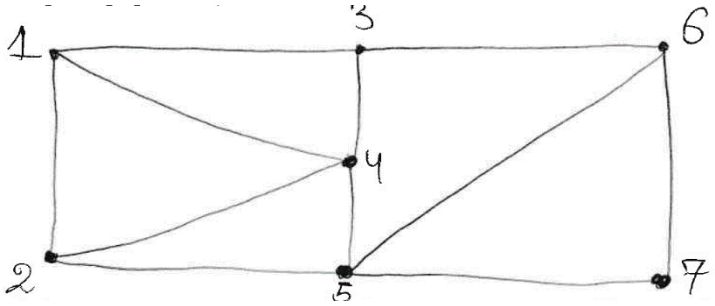
Резултат: Покриващо дърво $D(V, E')$ на G

Процедура:

1.) Нека r е началният връх на обхождане. Образуваме си дървото $D_0(V_0, E_0)$, където $V_0 = \{r\}$ и $E_0 = \emptyset$. Полагаме $t \coloneqq r, i = 0$ и $p(t)$ е неопределен.
2.) Нека $D_i(V_i, E_i)$ е построено. Търсим дали има връх v, такъв че $v \notin V_i$ и $(t, v) \in E$.
   a. Ако има такъв връх, фиксираме един такъв, полагаме: $V_{i+1} = V_i \cup \{v\}$ $E_{i+1} = E_i \cup \{(t, v)\}$, построяваме дървото $D_{i+1}(V_{i+1}, E_{i+1})$ и накрая $p(v) \coloneqq t, t \coloneqq v, i \coloneqq i + 1$ и преминаваме към 2.).
   b. Ако такъв връх няма, тогава проверяваме дали $t \neq r$.
      i. Ако $t \neq r$, то полагаме $t \coloneqq p(t)$ и преминаваме към 2.).
      ii. Ако $t = r$, то търсеното дърво е $D_i(V_i, E_i)$.

Пример за обхождане на граф:

Дадено: Свързан граф $G(V, E)$

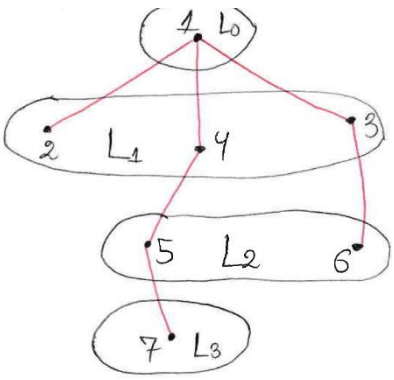


Търси се: Покриващо дърво в ширина = ? Покриващо дърво в дълбочина = ?

Нека избраният начален връх е 1.

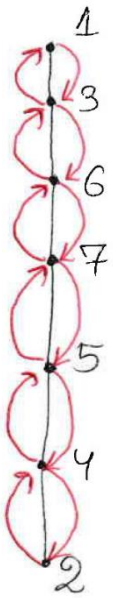
- **Покриващо дърво в ширина:**

$L_0 = \{1\}$
$L_1 = \{2,3,4\}$
$L_2 = \{5,6\}$
$L_3 = \{7\}$



- **Покриващо дърво в дълбочина:**

$t \coloneqq 1$	
$p(v) \coloneqq 1,\ t \coloneqq 3$	стъпка напред
$p(v) \coloneqq 3,\ t \coloneqq 6$	стъпка напред
$p(v) \coloneqq 6,\ t \coloneqq 7$	стъпка напред
$p(v) \coloneqq 7,\ t \coloneqq 5$	стъпка напред
$p(v) \coloneqq 5,\ t \coloneqq 4$	стъпка напред
$p(v) \coloneqq 4,\ t \coloneqq 2$	стъпка напред
$t \neq r,\ t \coloneqq 2$	
$t \neq r,\ t \coloneqq 4$	стъпка назад
$t \neq r,\ t \coloneqq 5$	стъпка назад
$t \neq r,\ t \coloneqq 7$	стъпка назад
$t \neq r,\ t \coloneqq 6$	стъпка назад
$t \neq r,\ t \coloneqq 3$	стъпка назад
$t \neq r,\ t \coloneqq 1$	стъпка назад
$t = r$	край



Ойлерови обхождания на мултиграф

Дефиниция (Ойлеров път в граф): Нека $\mathrm{G(V, E)}$ е свързан граф. Ойлеров път в G се нарича всеки път, който включва всяко ребро точно по веднъж.

Дефиниция (Ойлеров цикъл в граф): Ойлеров цикъл в $\mathrm{G(V, E)}$ се нарича Ойлеров път, чието начало и край съвпадат.

Дефиниция (Ойлеров граф): Един граф $\mathrm{G(V, E)}$ се нарича Ойлеров, ако в G има Ойлеров цикъл.

Теорема за съществуване на Ойлеров цикъл

Теорема (Л. Ойлер): Нека $\mathrm{G(V, E)}$ е свързан мултиграф. Тогава G е Ойлеров точно тогава когато всеки връх на G има четна степен.

Доказателство:

$\Longrightarrow$) Нека най-напред G е Ойлеров граф, т.е. съществува Ойлеров цикъл в G. Тогава очевидно този път минава през всяко ребро точно по веднъж и когато „влезе“ в един връх по този цикъл, той излиза от този връх, следователно ребрата влизащи в този връх са четен брой, като това се отнася за всеки връх. Следователно, всеки връх на G има четна степен.

$\Longleftarrow$) Обратно, нека G е такъв, че всеки връх има четна степен. Ще посочим как да построим Ойлеров цикъл:

1.) Избираме си произволен връх r от G. Обявяваме го за текущ, т.е. $t \coloneqq r$.
2.) Докато е възможно, строим път по следното правило: От върха t, в който се намираме, преминаваме към съседен връх v по необходено ребро. Обявяваме реброто за обходено и $t \coloneqq v$.

Да допуснем, че когато правилото от 2.) не е приложимо, то $t \neq r$. Тогава излиза, че t има нечетна степен, тъй като когато влезем в t, ние сме минали по необходено ребро, с което броят на обходените ребра е нечетен и няма повече необходени ребра. Следователно, този случай $t \neq r$ не е възможен. Следователно, след като правилото в 2.) не е приложимо, то $t = r$. Следователно, след като приложим това правило ще получим цикъл. Сега проверяваме дали всички ребра участват в този цикъл.

3.) Разглеждаме два случая:
   a. Ако в този цикъл участват всички ребра на графа, те участват точно по веднъж по правилото от 2.) и с това сме построили Ойлеров цикъл, т.е. графът е Ойлеров.
   b. Ако в този цикъл не са включени всички ребра, разглеждаме подграфа G' на G, като премахнем всички обходени ребра. Остава граф, който може и да не е свързан. Търсим в построения цикъл връх r', от който излиза поне едно необходено ребро. Ако допуснем, че такъв връх няма, при условието, че не всички ребра са обходени, ще получим противоречие със свързаността на мултиграфа. С избрания връх r', в който влиза (излиза) необходено ребро, прилагаме 2.), за да получим нов цикъл. Ако не сме обходили всички ребра, построяваме нов цикъл, докато изчерпим всички необходени ребра. Най-накрая съединяваме всички цикли в един и получаваме по този начин един общ Ойлеров цикъл, т.е. мултиграфът G е Ойлеров.

С това еквивалентността в двете посоки на твърдението е доказана, с това и теоремата.

Теорема за съществуване на Ойлеров път, който не е Ойлеров цикъл в свързан граф

Теорема (Следствие): Свързаният мултиграф $G(V, E)$ съдържа Ойлеров път, който не е Ойлеров цикъл, тогава и само тогава, когато има точно два върха с нечетна степен.

Държавен изпит – спец. „Софтуерно инженерство“

Тема №2 – Булеви функции. Пълнота.

Анотация: *Дефиниция на булева функция (БФ) и формула над множество БФ. БФ с 1 и 2 променливи. Свойства. Дефиниция на пълно множество БФ. Формулировка и доказателства на теоремата за разбиване на БФ по част от променливите и теоремата на Бул. Теорема на Пост.*

В по-широк смисъл дискретни функции са такива функции $f: A \to B$, *където* A *и* B *са най-много изброими. Ще дадем дефиниция на по-тесния смисъл на понятието дискретна функция.*

Дефиниция (дискретна функция): Нека A е крайно множество с q елемента, като без ограничение на общността ще считаме $A = J_q = \{0,1, \dots, q-1\}$. Всяка функция $f: A^n \to A, n$ е цяло, $n \geq 1$, наричаме дискретна (или $q-$ична) функция.

Нека $|A| = m$. *Броят на различните* $n-$местни *дискретни функции е* $|A|^{|A^n|} = m^{m^n}$.

Дефиниция (булева функция): Булева функция е двоична дискретна функция, при която множеството A е $J_2 = \{0,1\}$, т.е. $\mathcal{F}_2 = \{f: J_2^n \to J_2 | n = 1,2, \dots\}$. Булевите функции на n променливи означаване с $\mathcal{F}_2^n = \{f | f: \{0,1\}^n \to \{0,1\}\}$.

Множеството на всички булеви функции представяме: $\mathcal{F}_2 = \bigcup_{n \in \mathbb{N}} \mathcal{F}_2^n$. *Така* $|\mathcal{F}_2^n| = 2^{2^n}$.

Дефиниция (формула над множество от булеви функции): Нека $X = \{f, x, 0, 1, (,), <$ запетая $>\}$. По нататък ще записваме думите $f\alpha, x\beta$, където $\alpha, \beta \in \{0,1\}^+$ като f_i,x_j, където α е двоичното представяне на числото i, β е двоичното представяне на числото j. Дефинираме индуктивно понятието формула над множеството от функции F:

База: За всяка функция $f_i \in F$ на n променливи, думата $f_i(x_1, x_2, \dots, x_n) \in X^*$ е формула над F

Предположение: Нека $f_i \in F$ е функция на n променливи и $\varphi_1, \varphi_2, \dots, \varphi_n \in X^*$са формули над F или променливи, т.е. от вида x_k.

Стъпка: Тогава думата $f_i(\varphi_1, \varphi_2, \dots, \varphi_n) \in X^*$ е формула над F.

С други думи, формула над множество от булеви функции $F = \{f_i | i \in I\} \subseteq \mathcal{F}_2$ *са всички функции, които се получават от* F *чрез операциите идентитет, композиция и самите функции от* F.

Пример: Ако $f_1(x_1, x_2)$ *и* $f_2(x_1)$ *са функции от* F, *то функциите* $f_1(f_2(x_1), x_2)$ *и* $f_1(f_1(x_3, x_2), f_2(x_1))$ *са формули над* F.

Булеви функции с една променлива: Булевите функции на една променлива са 2^{2^1} и са представени в долната таблица:

x	$f_0(x) = \tilde{0}$	$f_1(x) = I(x)$	$f_2(x) = \overline{x}$	$f_3(x) = \tilde{1}$
0	0	0	1	1
1	0	1	0	1

Имената на функциите са както следва:

- $\boldsymbol{f_0(x) = \widetilde{0}}$: константата нула
- $\boldsymbol{f_3(x) = \widetilde{1}}$: константата единица
- $\boldsymbol{f_1(x) = I(x) = x}$: идентитет
- $\boldsymbol{f_2(x) = \overline{x}}$: отрицание на x

Функциите f_0 и f_3 не зависят от променливата x, затова използваме означения, в които x не участва.

Булеви функции с две променливи: Булевите функции на две променливи са $2^{2^2} = 2^4 = 16$ и са представени в долната таблица:

x	y	f_0	f_1	f_2	f_3	f_4	f_5	f_6	f_7	f_8	f_9	f_{10}	f_{11}	f_{12}	f_{13}	f_{14}	f_{15}
		$\widetilde{0}$	$x \wedge y$	$\overline{x \Rightarrow y}$	x	$\overline{y \Rightarrow x}$	y	$x \oplus y$	$x \vee y$	$x \downarrow y$	$x \Leftrightarrow y$	$\overline{y}$	$y \Rightarrow x$	$\overline{x}$	$x \Rightarrow y$	$x \mid y$	$\widetilde{1}$
0	**0**	0	0	0	0	0	0	0	0	1	1	1	1	1	1	1	1
0	**1**	0	0	0	0	1	1	1	1	0	0	0	0	1	1	1	1
1	**0**	0	0	1	1	0	0	1	1	0	0	1	1	0	0	1	1
1	**1**	0	1	0	1	0	1	0	1	0	1	0	1	0	1	0	1

Имената на функциите са както следва:

- $\boldsymbol{f_0(x,y) = \widetilde{0}}$: константата нула; тук тя е вече функция на две променливи, а самата функция отново не зависи от тези променливи
- $\boldsymbol{f_{15}(x,y) = \widetilde{1}}$: константата единица

Нека $1 \leq k \leq n$. Функцията $I_k^n: \{0,1\}^n \to \{0,1\}$, дефинираме чрез: $I_k^n(x_1, \dots, x_n) = x_k$ и наричаме проектираща функция.

- $\boldsymbol{f_3(x,y) = I_1^2(x,y) = x}$: проектираща функция по x (не зависи от y)
- $\boldsymbol{f_5(x,y) = I_2^2(x,y) = y}$: проектираща функция по y (не зависи от x)
- $\boldsymbol{f_{12}(x,y) = \overline{x}}$: отрицание на x (не зависи от y)
- $\boldsymbol{f_{10}(x,y) = \overline{y}}$: отрицание на y (не зависи от x)

Следващите функции от $\mathcal{F}_2^2$ съществено зависят и от двете си променливи. Ще ги означаваме както следва:

- $\boldsymbol{f_1(x,y) = x \wedge y = xy}$: конюнкция на x и y (логическо *И*); тази функция е частен случай при $q = 2$ на функциите $\min(x_1, x_2)$ и $x_1.x_2 (mod\ 2)$
- $\boldsymbol{f_7(x,y) = x \vee y}$: дизюнкция на x и y (логическо *ИЛИ*); тази функция е частен случай при $q = 2$ на функцията $\max(x_1, x_2)$
- $\boldsymbol{f_6(x,y) = x \oplus y}$: изключващо *ИЛИ* (събиране по модул 2); тази функция е частен случай при $q = 2$ на функцията $x_1 + x_2 (mod\ 2)$
- $\boldsymbol{f_9(x,y) = x \equiv y = x \Leftrightarrow y = \overline{x \oplus y}}$: еквивалентност на x и y
- $\boldsymbol{f_{13}(x,y) = x \Rightarrow y = x \to y}$: импликация от x към y
- $\boldsymbol{f_{11}(x,y) = y \Rightarrow x = y \to x}$: импликация от y към x
- $\boldsymbol{f_{14}(x,y) = x \mid y}$: функция (черта) на Шефер (отрицание на логическо *И*)
- $\boldsymbol{f_8(x,y) = x \downarrow y}$: функция (стрелка) на Пирс (отрицание на логическо *ИЛИ*)

Функциите f_2 и f_4 нямат общоприето инфиксно значение (записване):

- $\boldsymbol{f_2(x,y) = \overline{x \to y} = \overline{x \Rightarrow y}}$: отрицание на импликацията от x към y
- $\boldsymbol{f_4(x,y) = \overline{y \to x} = \overline{y \Rightarrow x}}$: отрицание на импликацията от y към x

Общоприето е отрицанието да има по-висок приоритет спрямо конюнкцията, а последната да има по-висок приоритет от дизюнкцията и събирането по модул 2.

За функции на три и повече променливи инфиксното записване е невъзможно и няма друга подобна форма на запис в таблица. На практика за функциите на повече променливи най-често използваната форма за представяне са формулите над подходящо избрано множество.

Свойства

Теорема: В сила са следните:

1. Комутативни свойства:

$$xy = yx \qquad x \vee y = y \vee x \qquad x \oplus y = y \oplus x \qquad x \Leftrightarrow y = y \Leftrightarrow x$$

2. Асоциативни свойства:

$$(xy)z = x(yz) \qquad (x \vee y) \vee z = x \vee (y \vee z) \qquad (x \oplus y) \oplus z = x \oplus (y \oplus z)$$

3. Дистрибутивни свойства:

$$x(y \vee z) = xy \vee xz \qquad x \vee yz = (x \vee y)(x \vee z) \qquad x(y \oplus z) = xy \oplus xz$$

4. Идемпотентни свойства (идемпотентност):

$$xx = x \qquad x \vee x = x \qquad x \oplus x = \tilde{0}$$

5. Свойства на отрицанието:

$$x\overline{x} = \tilde{0} \qquad x \vee \overline{x} = \tilde{1} \qquad x \oplus \overline{x} = \tilde{1}$$

6. Свойства на константите:

$$x\tilde{0} = \tilde{0} \qquad x \vee \tilde{0} = x \qquad x \oplus \tilde{0} = x$$

$$x\tilde{1} = x \qquad x \vee \tilde{1} = \tilde{1} \qquad x \oplus \tilde{1} = \overline{x}$$

7. Закон на двойното отрицание:

$$\overline{\overline{x}} = x$$

8. Закони на Де Морган:

$$\overline{x \vee y} = \overline{x} \wedge \overline{y}$$

$$\overline{x \wedge y} = \overline{x} \vee \overline{y}$$

9. Слепване: Нека $f \in \mathcal{F}_2$. Тогава е в сила:

$$\boldsymbol{fx \vee f\overline{x} = f(x \vee \overline{x}) = f\widetilde{1} = f}$$

10. Поглъщане: Нека $f, g \in \mathcal{F}_2$. Тогава е в сила:

$$\boldsymbol{f \wedge (f \vee g) = fg \vee f = f\big(g \vee \widetilde{1}\big) = f\widetilde{1} = f}$$

Всяко едно от тези свойства може да се провери като директно сравним стълбовете на функциите, отговарящи на двете формули.

Дефиниция на пълно множество от БФ:

Дефиниция (затваряне на множество от двоични функции): Нека F е произволно множество от двоични функции. Ще дадем индуктивна дефиниция на затваряне $[F]$ на това множество от двоични функции както следва:

А.) Ако $f \in F$, то $f \in [F]$;

Б.) $I_i^n \in [F]$ за всяко $n \geq 1$ и $1 \leq i \leq n$;

В.) Ако $f, g_1, \dots, g_n \in [F]$, то и $f(g_1, \dots, g_n) \in [F]$.

С $[F]$ ще означаваме множеството от всички двоични функции, съпоставени на формулите над F и ще го наричаме затваряне на F (относно суперпозицията).

Дефиниция (пълно множество от БФ): Нека $F \subseteq \mathcal{F}_2$. Ще казваме, че F е пълно точно тогава когато $[F] = \mathcal{F}_2$, т.е. всяка двоична функция може да се получи с прилагането на суперпозиции на елементи на F.

Дефиниция (двоична функция $\boldsymbol{f(x,\sigma) = x^\sigma}$): Двоичната функция $f(x,\sigma) = x^\sigma$ дефинираме:

$$x^\sigma = \begin{cases} x & \text{ако } \sigma = 1 \\ \overline{x} & \text{ако } \sigma = 0 \end{cases}$$

Лема: $x^\sigma = 1 \Leftrightarrow x = \sigma$

Доказателство: Следва от дефиницията на x^σ:

1.) *Нека $\sigma = 1$. Тогава по дефиниция: $x^\sigma = x^1 = x$. За да бъде $x^\sigma = 1$, то $x^\sigma = x^1 = 1$, т.е. $x = 1$. Следователно, $x = \sigma = 1$.*
2.) *Нека $\sigma = 0$. Тогава по дефиниция: $x^\sigma = x^0 = \overline{x}$. За да бъде $x^\sigma = 1$, то $x^\sigma = \overline{x} = \overline{0} = 1$, т.е. $x = 0$. Следователно, $x = \sigma = 0$.*

И в двата случая достигнахме до следствието, че $x = \sigma$ за $x^\sigma = 1$, с което лемата е доказана.

Лема: $x_1^{\sigma_1} \dots x_n^{\sigma_n} = 1$ т.т.к $x_1 = \sigma_1$ и … и $x_n = \sigma_n$ за $x_i \neq x_j$ при $i \neq j$ и $i, j = 1, \dots, n$.

Формули от горния вид ($x_1^{\sigma_1} \dots x_n^{\sigma_n}$) наричаме елементарни конюнкции.

Теорема за разбиване на БФ по част от променливите:

Теорема (разлагане на булева функция по i променливи): Нека са избрани $i, 1 \leq i \leq n$ от променливите на функцията $f(x_1, \dots, x_n) \in \mathcal{F}_2$. Без ограничение на общността, нека това са първите i променливи. Тогава:

$$f(x_1, x_2, \dots, x_n) = \bigvee_{\sigma_1,\dots,\sigma_i \in \{0,1\}} x_1^{\sigma_1} \dots x_i^{\sigma_i} f(\sigma_1, \dots, \sigma_i, x_{i+1}, \dots, x_n)$$

Доказателство: Да означим с $g(x_1, x_2, \dots, x_n)$ функцията, определена от дясната част на равенството. Да пресметнем стойностите на функциите f и g за произволен вектор $(a_1, \dots, a_n) \in J_2^n$. Вляво получаваме $f(a_1, a_2, \dots, a_n)$. От *Лема* следва, че от всички елементарни конюнкции, участващи в дясната част, само една има стойност 1 – тази при която $\sigma_j = a_j$ за $j = 1, \dots, i$. Останалите елементарни конюнкции имат стойност 0 и съответно анулират съответните членове от многократната дизюнкция. Така за стойностите на дясната част получаваме:

$$g(a_1, \dots, a_n) = \bigvee_{\sigma_1,\dots,\sigma_i \in \{0,1\}} a_1^{\sigma_1} \dots a_i^{\sigma_i} . f(\sigma_1, \dots, \sigma_i, a_{i+1}, \dots, a_n) =$$

$$= a_1^{a_1} \dots a_i^{a_i} f(a_1, \dots, a_i, a_{i+1}, \dots, a_n) \vee \tilde{0} =$$

$$= \tilde{1} f(a_1, \dots, a_i, a_{i+1}, \dots, a_n) = f(a_1, \dots, a_n),$$

т.е. $f(a_1, \dots, a_n) = g(a_1, \dots, a_n)$, с което теоремата е доказана.

Теорема на Бул:

Теорема (Дж. Бул): Множеството: $F = \{f_1(x, y) = x.y, f_2(x, y) = x \vee y, f_3(x) = \overline{x}\}$ е пълно.

Доказателство: Ще покажем, че всяка двоична функция може да се представи чрез суперпозиция на функциите $\wedge, \vee, \boxed{}$, т.е. $f \in [\{x \vee y, xy, \overline{x}\}]$:

1.) Нека $f = \tilde{0}$. Тогава $f(x_1, \dots, x_n) = x_1\overline{x_1} = x_1\overline{x_1} \vee x_2\overline{x_2} \vee \dots$. Следователно, f се представя с формула над $\{x \vee y, xy, \overline{x}\}$.

2.) Нека $f \neq \tilde{0}$. Тогава, по горното твърдение:

$$f(x_1, \dots, x_n) = \bigvee_{\sigma_1,\dots,\sigma_n \in \{0,1\}} x_1^{\sigma_1} \dots x_n^{\sigma_n} f(\sigma_1, \dots, \sigma_n) \text{ за } i = n.$$

2.1) Ако $f(\sigma_1, \dots, \sigma_n) = 0$, то съответният член в дясната част се анулира и съгласно свойствата, доказани по-горе, може да не участва във формулата (тъй като е дизюнкция).

2.2) Ако пък $f(\sigma_1, \dots, \sigma_n) = 1$, то: $x_1^{\sigma_1} \dots x_n^{\sigma_n} f(\sigma_1, \dots, \sigma_n) = x_1^{\sigma_1} \dots x_n^{\sigma_n}$. Следователно,

$$f(x_1,\dots,x_n)=\bigvee_{\sigma_1,\dots,\sigma_n\in\{0,1\}} x_1^{\sigma_1}\dots x_n^{\sigma_n} f(\sigma_1,\dots,\sigma_n)=\bigvee_{\substack{\sigma_1,\dots,\sigma_n\in\{0,1\}\\ f(\sigma_1,\dots,\sigma_n)=1}} x_1^{\sigma_1}\dots x_n^{\sigma_n}$$

Да обърнем внимание, че всеки един член на $x_1^{\sigma_1}\dots x_n^{\sigma_n}$ е суперпозиция на $\wedge$ и $\overline{\square}$, защото $x_i^{\sigma_i}=x_i$ или $x_i^{\sigma_i}=\overline{x_i}$.

Окончателно, получихме, че $f(x_1,\dots,x_n)$ е суперпозиция на $\{x\vee y, xy, \overline{x}\}$, с което теоремата е доказана.

В горната формула, когато $f(\sigma_1,\dots,\sigma_n)\neq\tilde{0}$, т.е.

$$f(x_1,\dots,x_n)=\bigvee_{\substack{\sigma_1,\dots,\sigma_n\in\{0,1\}\\ f(\sigma_1,\dots,\sigma_n)=1}} x_1^{\sigma_1}\dots x_n^{\sigma_n}$$

се нарича съвършена дизюнктивна нормална форма, тъй като е израз от вида $\bigvee_{i=1}^{k} f_i$, където f_i са елементарни конюнкции и всички f_i са елементарни конюнкции на всички променливи $x_1,\dots,x_n$.

Теорема на Пост

Дефиниция (булеви функции, запазващи константите): Казваме, че $f(x_1,\dots,x_n)\in\mathcal{F}_2$ запазва константата $\tilde{c}$, ако $f(c,\dots,c)=c$.

Означаваме с T_c множеството от функции, които запазват константата $\tilde{c}$, а с T_c^n тези от тях, които са на n променливи. Получаваме следните две множества от функции (класове):

T_0 – множество от функции $f(x_1,\dots,x_n)\in\mathcal{F}_2$, които запазват нулата, т.е. $f(0,\dots,0)=0$

T_1 – множество от функции $f(x_1,\dots,x_n)\in\mathcal{F}_2$, които запазват единицата, т.е. $f(1,\dots,1)=1$

Дефиниция (двойнствена на f): Нека $f(x_1,\dots,x_n)\in\mathcal{F}_2$. Функцията $f^*(x_1,\dots,x_n)=\overline{f}(\overline{x_1},\dots,\overline{x_n})$ наричаме двойнствена на f.

Дефиниция (самодвойнствена функция): Функцията $f(x_1,\dots,x_n)$ наричаме самодвойнствена, ако $f^*=f$.

Означаваме с S множеството на самодвойнствените функции, а с S^n – тези от тях, които са на n променливи. Получаваме множеството от функции (клас):

S – множество на самодвойнствените функции: $\{f(\alpha)|$за всяко $\alpha\Rightarrow f(\alpha)=f^*(\alpha)\}$

Дефиниция (монотонни булеви функции): За да се дефинират монотонни булеви функции, дефинираме частичната наредба $\mathcal{R}_{\preccurlyeq}$, като ако $\alpha\preccurlyeq\beta$ казваме, че $\alpha(a_1,\dots,a_n)$ предшества $\beta(b_1,\dots,b_n)$ и дефинираме $\alpha\preccurlyeq\beta\Leftrightarrow a_i\leq b_i$ за $\forall i\in\{1,\dots,n\}$. Булевата функция $f(x_1,\dots,x_n)$ наричаме монотонна, ако за $\forall\alpha,\beta\in J_2^n\colon\alpha\preccurlyeq\beta\Rightarrow f(\alpha)\leq f(\beta)$. Означаваме с M множеството на монотонните функции, а с M^n – множеството на тези от тях, които са на n променливи. Получаваме множество от функции (клас):

M – **множество на монотонните функции:** $\{\boldsymbol{f(\alpha)}|$за всяко $\boldsymbol{\alpha \preccurlyeq \beta \Rightarrow f(\alpha) \leq (\beta)}\}$

Дефиниция (полином на Жегалкин): Функция от вида:

$$f(x_1, \dots, x_n) = a_0 \oplus \bigoplus_{1 \leq i \leq n} a_i x_i \oplus \bigoplus_{1 \leq i \leq j \leq n} a_{ij} x_i x_j \oplus \dots \oplus a_{12\dots n} x_1 x_2 \dots x_n,$$

където $a_i \in \{0, 1\}$, наричаме полином на Жегалкин.

Теорема (Ив. Жегалкин): Всяка булева функция може да се представи по единствен начин като полином на Жегалкин.

Дефиниция (линейни булеви функции): Всяка булева функция $f(x_1, \dots, x_n)$, с полином на Жегалкин от вида $a_0 \oplus a_1 x_1 \oplus \dots \oplus a_n x_n$, където $a_i \in \{0,1\}$, наричаме линейна. Означаваме с L множеството на линейните функции, а с L^n – множеството на тези от тях, които са на n променливи. Получаваме множеството от функции (клас):

***L* – множество на линейните функции, т.е. представящи се с полином на Жегалкин от вида $\boldsymbol{a_0 \oplus a_1 x_1 \oplus \dots \oplus a_n x_n}$**

Теорема (Е. Пост – С. Яблонский): Нека $F \subseteq \mathcal{F}_2$. Тогава множеството F е пълно точно тогава когато F не е подмножество на нито един от класовете T_0, T_1, S, M и L.

Държавен изпит – спец. „Софтуерно инженерство“

Тема №3 – Крайни автомати. Регулярни езици. Теорема на Клини.

Анотация: *Детерминирани крайни автомати. Регулярни операции. Недетерминирани крайни автомати. Представяне на всеки недетерминиран краен автомат с детерминиран (с доказателство). Затвореност относно регулярните операции. Теорема на Клини (с доказателство). Лема за покачването (uvw) (с доказателство). Примери за регулярни и нерегулярни езици. Минимизация на състоянията. Теорема на Майхил-Нероуд (с доказателство). Алгоритъм за конструиране на минимален автомат, еквивалентен на даден детерминиран краен автомат.*

Дефиниция (регулярен израз): Нека Σ е азбука. Регулярен израз над Σ се определя индуктивно както следва:

А.) $\emptyset$ и всяка буква от Σ е регулярен израз;

Б.) Ако α и β са регулярни изрази, то $(\alpha\beta), (\alpha \cup \beta), \alpha^*$ са регулярни изрази;

В.) Няма други регулярни изрази, които да не следват от горните две правила.

Дефиниция (регулярен език $\mathcal{L}[\alpha]$, определен от регулярен израз α): Ще дадем индуктивна дефиниция на $\mathcal{L}[\alpha]$ с индукция относно построението на α:

А.) Ако $\alpha = \emptyset$, то $\mathcal{L}[\alpha] = \emptyset$;

Б.) Ако $\alpha = a \in \Sigma$, то $\mathcal{L}[\alpha] = \{a\}$;

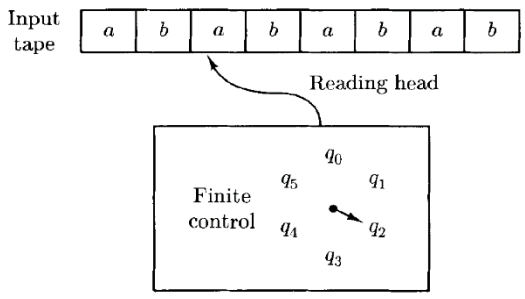
В.) Ако $\alpha = (\alpha_1\alpha_2)$, то $\mathcal{L}[\alpha] = \mathcal{L}[\alpha_1] \circ \mathcal{L}[\alpha_2]$;

Г.) Ако $\alpha = (\alpha_1 \cup \alpha_2)$, то $\mathcal{L}[\alpha] = \mathcal{L}[\alpha_1] \cup \mathcal{L}[\alpha_2]$;

Д.) Ако $\alpha = \alpha_1^*$, то $\mathcal{L}[\alpha] = (\mathcal{L}[\alpha_1])^*$.

Дефиниция (регулярен език): Казваме, че един език L е регулярен, ако съществува регулярен израз α, такъв че $\mathcal{L}[\alpha] = L$.

Много интуитивно, за краен детерминиран автомат (КДА) може да се мисли като една безкрайна лента (потенциално) с клетки, във всяка клетка на която може да се съдържа само по една буква от дадена азбука. Освен това има „машина“ с четяща глава, която е свързана с устройство с крайно много състояния. Устройството може да променя състоянието си в зависимост от това какво е прочело главата и в какво състояние се е намирало устройството до този момент:



безкрайна входна лента

четяща глава

устройство с крайно много състояния за контрол

Дефиниция (краен детерминиран автомат): Краен детерминиран автомат (КДА) се нарича петицата $M = < K, \Sigma, \delta, s, F >$, такава че:

А.) K – крайно множество от състояния;

Б.) Σ – основна азбука, чиито думи ще разпознаваме;

В.) $s \in K$, $s -$ начално състояние;

Г.) $F \subseteq K$; $F -$ множество от заключителни състояния;

Д.) $\delta: K \times \Sigma \to K$; $\delta -$ функция на преходите.

Дефиниция (конфигурация на КДА): Нека $M = < K, \Sigma, \delta, s, F >$ е КДА. Конфигурация за M се нарича всяка двойка $K \times \Sigma^*$, т.е. конфигурация е всяка двойка (q, w), където $q \in K$ и $w \in \Sigma^*$.

Дефиниция (релация $\vdash_M$ за КДА): Нека (q, w) и (q', w') са две конфигурации за $M = < K, \Sigma, \delta, s, F >$. Казваме, че конфигурацията (q, w) се преработва за една стъпка от M в конфигурацията (q', w') и пишем $(q, w) \vdash_M (q', w')$ точно тогава когато съществува $a \in \Sigma$, такава че $w = aw'$ и $\delta(q, a) = q'$.

Дефиниция (приемане на дума): Нека $M =< K, \Sigma, \delta, s, F >$ е КДА. Казваме, че думата $w \in \Sigma^*$ се приема (разпознава) от M точно тогава когато $(s, w) \vdash_M^* (f, \varepsilon)$ и $f \in F$.

Дефиниция (език, разпознаван от КДА): Нека $M =< K, \Sigma, \delta, s, F >$ е КДА. $L(M)$ се нарича език, който се разпознава от M и се означава с:

$$L(M) = \{w | w \in \Sigma^* \text{ и } w \text{ се разпознава от } M\}.$$

Дефиниция (недетерминиран краен автомат): Недетерминиран краен автомат (НКА) се нарича петицата $M = < K, \Sigma, \Delta, s, F >$, такава че:

А.) K – крайно множество от състояния;

Б.) Σ – основна азбука, чиито думи ще разпознаваме;

В.) $s \in K$, $s -$ начално състояние;

Г.) $F \subseteq K$; $F -$ множество от заключителни състояния;

Д.) $\Delta \subseteq K \times (\Sigma \cup \{\varepsilon\}) \times K$; $\Delta -$ релация на преходите; всяка тройка $(q, u, p) \in \Delta$ се нарича правило за преход.

Дефиниция (конфигурация на НКА): Нека $M = < K, \Sigma, \Delta, s, F >$ е НКА. Конфигурация за M се нарича всеки елемент на $K \times \Sigma^*$.

Дефиниция (релация $\vdash_M$ за НКА): Нека (q, w) и (q', w') са две конфигурации за $M = < K, \Sigma, \Delta, s, F >$. Казваме, че конфигурацията (q, w) се преработва за една стъпка от M в конфигурацията (q', w') и пишем $(q, w) \vdash_M (q', w')$ точно тогава когато съществува $u \in \Sigma \cup \{\varepsilon\}$, такава че $w = uw'$ и $(q, u, q') \in \Delta$.

Дефиниция (приемане на дума): Нека $M =< K, \Sigma, \Delta, s, F >$ е НКА. Казваме, че думата $w \in \Sigma^*$ се приема (разпознава) от M точно тогава когато $(s, w) \vdash_M^* (f, \varepsilon)$ и $f \in F$.

Дефиниция (език, разпознаван от НКА): Нека $M =< K, \Sigma, \Delta, s, F >$ е НКА. $L(M)$ се нарича език, който се разпознава от M и се означава с:

$$L(M) = \{w | w \in \Sigma^* \text{ и } w \text{ се разпознава от } M\}.$$

Дефиниция (еквивалентност на автомати): Казваме, че два автомата M_1 и M_2 (независимо дали са КДА или НКА) са еквивалентни точно тогава когато $L(M_1) = L(M_2)$.

Теорема (за представяне на всеки недетерминиран краен автомат с детерминиран краен автомат): За всеки недетерминиран краен автомат съществува еквивалентен на него краен детерминиран автомат.

Доказателство: Нека $N = (Q, \Sigma, \delta, q, F)$ е НКА. Ще докажем, че съществува КДА M, такъв че $L(N) = L(M)$.

Знаем, че НКА N може в общия случай да изпълнява повече от едно изчисление за дадена дума. Идеята на доказателството е, да построим КДА M, който прави всички тези различни изчисления едновременно. По-конкретно, КДА ще има следното свойство: състоянието, в което е M след като е прочел дадена начална част от някаква дума съответства на множеството от състояния, които N може да достигне след като е прочел същата част от дадена входна дума.

Започваме с представянето на преобразуванията в случая, когато N не съдържа ε-преходи. По-нататък, ще разширим идеята на доказателството и до този случай.

Дефинираме КДА $M = (Q', \Sigma, \delta', q', F')$, където:

- множеството Q' от състояния се определя: $Q' = \mathcal{P}(Q)$; $|Q'| = 2^{|Q|}$;
- началното състояние q' се определя: $q' = \{q\}$ (колкото и да бъдат);
- множеството F' от заключителни състояния се определя като множеството от всички елементи R от Q' имащи свойството, че R съдържат поне едно заключително състояние на N, т.е. $F' = \{R \in Q' : R \cap F \neq \emptyset\}$;
- функция на преходите $\delta' : Q' \times \Sigma \to Q'$ се определя както следва: за всяко $R \in Q'$ и за всяко $a \in \Sigma$: $\delta'(R, a) = \bigcup_{r \in R} \delta(r, a)$.

Нека се съсредоточим върху действието на функцията на преходите δ' на M. Първо забелязваме, че тъй като N е НКА, $\delta(r, a)$ е подмножество на Q. Това влече, че $\delta'(R, a)$ е обединението на подмножества на Q и следователно е също подмножество на Q. Следователно, $\delta'(R, a)$ е елемент на $Q' = \mathcal{P}(Q)$.

Множеството $\delta(r, a)$ е еквивалентно на множеството от състояния на НКА N, които може да бъдат достигнати от състоянието r след прочитане на буквата a. Ние взимаме обединението на тези множества $\delta(r, a)$, където r варира по всички елементи от R, достигайки до новото множество $\delta'(R, a)$. Това ново състояние е състоянието, което КДА M достига от състоянието R при прочитане на буквата a. По този начин постигаме идеята зад построението на M, дадена в началото на доказателството.

Сега разглеждаме по-общия случай. С други думи, позволяваме ε-преходи в НКА N. КДА M се определя по същия начин, както по-горе с изключение на началното състояние q' и функцията на преходите δ', които трябва да бъдат модифицирани.

Знаем, че едно изчисление на НКА N се състои от следните етапи:

1. Започни от началното състояние q и направи нула или повече ε-преходи.
2. Прочети един „истински“ символ на Σ и се премести в ново състояние (или остани в текущото).
3. Направи нула или повече ε-преходи.
4. Прочети един „истински“ символ на Σ и се премести в ново състояние (или остани в текущото)
5. Направи нула или повече ε-преходи.
6. … и т.н.

Следователно, в една стъпка КДА M симулира четенето на един „истински“ символ от Σ, следвано от правенето на нула или повече ε-преходи. За да формализираме това, ние се нуждаем от нотацията за ε-затваряне. За всяко състояние r от НКА N, ε-затварянето на r, записано и означено като $C_\varepsilon(r)$, се дефинира като множеството на всички състояния от N, които могат да бъдат достигнати от r чрез извършването на нула или повече ε-преходи.

За всяко състояние R от КДА M ($R \subseteq Q$), определяме: $C_\varepsilon(R) = \bigcup_{r \in R} C_\varepsilon(r)$.

Да се върнем на въпросите – как дефинираме началното състояние q' на КДА M. Преди НКА N да прочете неговия първи „реален“ символ (буква) от Σ, той прави нула или повече ε-преходи. С други думи, в момента, в който N прочита първия символ от Σ, той може да бъде в което и състояние от $C_\varepsilon(q)$. Следователно, определяме q': $q' = C_\varepsilon(q) = C_\varepsilon(\{q\})$.

Сега трябва да определим как дефинираме функцията на преходите δ' на КДА M. Да предположим, че M е в състояние R и прочита буквата a. В този момент, НКА N би бил, в което и да е състояние r от R. Прочитайки буквата a, N може да превключи до което и да е състояние от $\delta(r, a)$ и после да направи нула или повече ε-преходи. Следователно, НКА N може да превключи до което и да е състояние в множеството $C_\varepsilon(\delta(r, a))$. Базирайки се на това, дефинираме $\delta'(R, a)$:$\delta'(R, a) = \bigcup_{r \in R} C_\varepsilon(\delta(r, a))$.

В обобщение, НКА $N = (Q, \Sigma, \delta, s, F)$ се преобразува в КДА $M = (Q', \Sigma, \delta', q', F')$, където:

- $Q' = \mathcal{P}(Q) = 2^Q$
- $q' = C_\varepsilon(\{q\})$
- $F' = \{R \in Q' : R \cap F \neq \emptyset\}$
- $\delta' : Q' \times \Sigma \to Q'$ се определя както следва: за всяко $R \in Q'$ и за всяко $a \in \Sigma$:

$$\delta'(R, a) = \bigcup_{r \in R} C_\varepsilon(\delta(r, a))$$

Остава да покажем, че M е детерминистичен и еквивалентен на N, т.е. че $L(M) = L(N)$. Наблюдението, че M е детерминистичен е праволинейно, тъй като забелязваме, че δ' е едностойностна и добре дефинирана за всяко $R \in Q'$ и $a \in \Sigma$, по начина, по който е конструиран.

За да докажем еквивалентността на двата автомата използваме твърдението, че за всяка дума $w \in \Sigma^*$ и всеки две състояния $p, q \in Q$:

$$p, q \in Q: (q, w) \vdash_N^* (p, \varepsilon) \Leftrightarrow (C_\varepsilon(q), w) \vdash_M^* (P, \varepsilon)$$ за някое множество P, съдържащо p

Нека $w \in \Sigma^*$. Тогава $w \in L(N) \Leftrightarrow (q, w) \vdash_N^* (f, \varepsilon)$ за някое $f \in F$ (по дефиниция) $\Leftrightarrow (C_\varepsilon(q), w) \vdash_M^* (K, \varepsilon)$ за някое K, съдържащо f (от твърдението) $\Leftrightarrow (q', w) \vdash_M^* (K, \varepsilon)$ за някое $K \in F'$. Последното е дефиницията за $w \in L(M)$.

С това теоремата е доказана.

В контекста на регулярни езици регулярни операции наричаме операциите обединение $\cup$*, конкатенация* $\circ$ *и звезда на Клини (итерация)* *.

Теорема: Класът на автоматните езици (езици, разпознавани от краен автомат) е затворен относно операциите:

А.) обединение

Б.) конкатенация

В.) звезда на Клини

Г.) допълнение (като следствие)

Д.) сечение (като следствие)

За всяка от операциите, изброени по-горе, ще построим автомат, който приема език, зададен чрез прилагане на съответната операция върху езиците на крайните автомати A_1 и A_2. Ще посочим построенията им:

А.) **Обединение** – построяваме автомат $A_\cup$ с език равен на $L(A_1) \cup L(A_2)$. За $A_\cup$:

1. Състояния: състоянията на A_1 и A_2 и ново състояние q;
2. Начално състояние: q;
3. Финални състояния: финалните състояния на A_1 и A_2 се запазват. q е финално $\Leftrightarrow$ поне едно от началните състояния на A_1 и A_2 е финално;
4. Преходи: преходите на A_1 и A_2 остават. q копира преходите на началните състояния A_1 и A_2.

Б.) **Конкатенация** – построяваме автомат $A_\circ$ с език равен на $L(A_1) \circ L(A_2)$. За $A_\circ$:

1. Състояния: състоянията на A_1 и A_2;
2. Начално състояние: началното състояние на левия автомат A_1;
3. Финални състояния: финалните състояния на десния автомат A_2. Добавяме финалните състояния на A_1 тогава и само тогава, когато началното състояние на десния автомат е финално;
4. Преходи: преходите на A_1 и A_2 остават. Всяко финално състояние на левия автомат A_1 копира преходите на началното състояние на десния автомат A_2.

В.) **Итерация** – построяваме автомат A^* с език равен на $L(A_1)^*$. За A^*:

1. Състояния: състоянията на A_1 и ново състояние q;
2. Начално състояние: q;
3. Финални състояния: финалните състояния на A_1 и q (за да можем да разпознаем и q);
4. Преходи: преходите в A_1 остават. Всички финални състояния на A^* копират преходите на началното състояние на A_1.

Г.) **Допълнение** – построяваме автомат $\overline{A}$ с език равен на $\Sigma^*\backslash L(A_1)$. За $\overline{A}$:

1. Тотализираме автомата
2. Състояния: състоянията на A_1;
3. Начално състояние: началното състояние на A_1;
4. Финални състояния: състоянията на A_1, които не са финални $(Q\backslash F)$;
5. Преходи: преходите в A_1.

Д.) **Сечение** – $L(A_1) \cap L(A_2) =^{\text{Де Морган}} \overline{\overline{L(A_1)} \cup \overline{L(A_2)}} = \Sigma^*\backslash((\Sigma^*\backslash L(A_1) \cup \Sigma^*\backslash L(A_2))$, т.е. затвореността при операцията сечение следва директно от затвореността при операциите обединение и допълнение.

Теорема (Клини): Един език е регулярен тогава и само тогава, когато се разпознава (приема) от краен автомат, т.е. множествата на регулярните и автоматните езици съвпадат.

Доказателство:

⇒) Знаем, че класът на регулярните езици е най-малкият клас езици, съдържащи празното множество $\emptyset$ и синглетоните a, където a е буква, и затворен относно операциите обединение, конкатенация и звезда на Клини. Очевидно, празното множество и всички букви a се приемат от краен автомат. От предходната теорема знаем, че крайните автоматни езици са затворени относно операциите обединение, конкатенация и звезда на Клини. Следователно, всеки регулярен език се приема от някакъв краен автомат.

⇐) Нека $M = (K, \Sigma, \Delta, s, F)$ е краен автомат (не е необходимо да бъде задължително детерминиран). Ще построим регулярен израз R, такъв че $\mathcal{L}(R) = L(M)$. Ще представим $L(M)$ като обединение на много (но краен брой) прости езици.

Нека $K = \{q_1, \dots, q_n\}$ и $s = q_1$. За всяко $i, j = 1, \dots, n$ и $k = 0, \dots, n$ дефинираме $R(i, j, k)$ като множеството от всички думи в Σ^*, които могат да заведат M от състояние q_i до състояние q_j без да преминават през междинни състояния, номерирани с $k + 1$ или повече (по-големи) – крайните точки q_i и q_j е позволено да бъдат номерирани като по-големи от k. Следователно, $R(i, j, k)$ е множеството на всички думи, спелувани от всички пътеки от q_i до q_j от ранг k. Когато $k = n$ следва, че:

$$R(i, j, n) = \{w \in \Sigma^* : (q_i, w) \vdash_M^* (q_j, \varepsilon)\}.$$

Следователно, $L(M) = \cup\{R(1, j, n) : q_j \in F\}$.

Важният момент тук е, че всички $R(i, j, k)$ са регулярни и следователно, $L(M)$ е регулярен като крайна сума на регулярни езици.

Доказателството, че $R(i, j, k)$ е регулярен е по индукция по k.

За $k = 0, R(i,j,0)$ е или:

(a.) $\{a \in \Sigma \cup \{\varepsilon\}: (q_i, a, q_j) \in \Delta\}$, ако $i \neq j$, или

(b.) $\{\varepsilon\} \cup \{a \in \Sigma \cup \{\varepsilon\}: (q_i, a, q_j) \in \Delta\}$, ако $i = j$.

Всяко от тези множества е крайно и следователно, регулярно.

За индукционна стъпка предполагаме, че $R(i,j,k-1)$ за всяко i,j са били определени като регулярни езици за всички i,j. Тогава всяко множество $R(i,j,k)$ може да бъде дефинирано чрез комбинирането на предишно дефинирани регулярни езици чрез регулярните операции обединение, звезда на Клини и конкатенация както следва:

$$R(i,j,k) = R(i,j,k-1) \cup R(i,k,k-1)R(k,k,k-1)^*R(k,j,k-1).$$

Това равенство показва, че за да достигнем от q_i до q_j, без да преминаваме през състояние по-голямо от k, M може:

(1.) да отиде от q_i до q_j, без да преминава през състояние, номерирано по-голямо от $k-1$ или
(2.) да отиде от q_i до q_k, после от q_k до q_k нула или повече пъти, после от q_k до q_j, във всички случаи без да преминава през някое междинно състояние, номерирано по-голямо от $k-1$.

Следователно, езикът $R(i,j,k)$ е регулярен, използвайки затвореността на регулярните изрази и езици спрямо регулярните операции, за всяко i,j,k, с което доказваме индукцията. С това, теоремата е доказана.

Лема (за покачването $/uvw/$): Нека L е регулярен език. Тогава съществува такова естествено число $n \geq 1$, че за всяка дума $w \in L$, такава че $|w| \geq n$ съществуват думи x, y, z, такива че $w = xyz, y \neq \varepsilon, |xy| \leq n$ и за всяко $i \in \mathbb{N}$ е изпълнено, че $xy^iz \in L$.

Доказателство: Тъй като L е регулярен съществува КДА $M = (K, \Sigma, \delta, s, F)$, който разпознава L. Да положим $n = |K|$ и нека $w \in L, |w| \geq n$. Следователно, ако $q_0 = s$ и $|w| = m \geq n$, то съществува извод $(q_0, w) \vdash_M (q_1, w_1) \vdash_M \ldots \vdash_M (q_m, \varepsilon), q_m \in F$ и $w = a_1 \ldots a_m, a_i \in \Sigma$. Тогава състоянията $q_0, \ldots, q_n$ са $n+1$, а $|K| = n$. Следователно, по принципа на Дирихле съществуват поне две различни i,j, такива че $q_i = q_j$. Да си мислим, че $i < j, 1 \leq i < j \leq n$.

Да означим с $x = a_1 \ldots a_i, y = a_{i+1} \ldots a_j, z = a_{j+1} \ldots a_m$. Очевидно е, че $y \neq \varepsilon\ (i < j), |xy| = |a_1 \ldots a_i a_{i+1} \ldots a_j| = |a_1 \ldots a_j| = j \leq n$. Остава да покажем, че с индукция относно k следва, че $xy^kz \in L$ за произволно естествено k. Да разпишем по-подробно стъпките на извода $(q_0, w) \vdash_M^* (q_m, \varepsilon)$, а именно:

$$(q_0, w) \vdash_M^* (q_i, a_{i+1} \ldots a_m) \vdash_M^* (q_j, a_{j+1} \ldots a_m) \vdash_M^* (q_m, \varepsilon), q_m \in F \text{ и } q_i = q_j.$$

За $q_i = q_j$: $(q_0, xz) = (q_0, a_1 \ldots a_i a_{j+1} \ldots a_m) \vdash_M^* (q_i, a_{j+1} \ldots a_m) = (q_j, a_{j+1} \ldots a_m) \vdash_M^* (q_m, \varepsilon)$,

т.е. $xz = xy^0z \in L$, защото $q_m \in F$.

Допускаме, че твърдението е вярно за някое k. Ще го докажем за $k+1$. Наистина:

$$(q_0, xy^{k+1}z) \vdash_M^* (q_i, y^{k+1}z) \vdash_M^* (q_j, y^k z) = (q_i, y^k z) \vdash_M^* (q_i, y^{k-1}z) \vdash_M^* \ldots \vdash_M^* (q_m, \varepsilon),$$

т.е. $xy^{k+1}z \in L$, защото $q_m \in F$.

С това лемата за покачването (разрастването) за регулярни езици е доказана.

Примери за регулярни и нерегулярни езици

(1.) Примери за нерегулярни езици

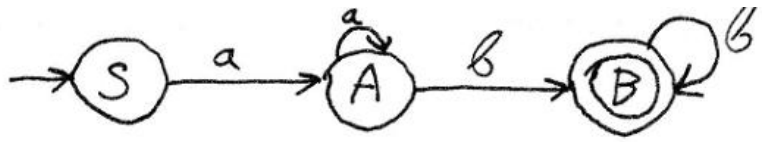
a. Езикът $L = \{a^i b^i : i \geq 0\}$ не е регулярен. Ако беше, тогава от горната теорема ще съществува n, за което $a^n b^n = xyz$, такива че $|xy| \leq n$ и $y \neq \varepsilon$, т.е. $y = a^i, i > 0$, но тогава $xz = a^{n-i}b^n \notin L$, което противоречи на лемата.

С подобни разсъждения можем да покажем, че и следните езици не са регулярни:

b. Езикът $L = \{a^n : n - \text{просто}\}$ не е регулярен.
c. Езикът $L = \{a^n b^m : n < m\}$ не е регулярен.
d. Езикът $L = \{0^{n^2} : n \geq 0\}$ не е регулярен.
e. Езикът $L = \{a^n b^m : n \neq m\}$ не е регулярен.

(2.) Примери за регулярни езици:

a. Езикът $L = \{a^n b^m : n > 0, m > 0\}$ е регулярен. Неговият краен автомат би изглеждал по следния начин:



b. Езикът $L = \{w \in \{a\}^* | w$ има поне едно $a\}$ е регулярен език, т.к. $L = a^* a = a^+$.

Минимизация на състоянията

Задачата, която ще си поставим е да можем по всеки КДА да намерим еквивалентен на него КДА с най-малко състояния, т.е. да решим задачата за минимизация на КДА.

Първата стъпка за минимизация е да проверим дали всички състояния се „използват“, т.е. дали са достижими. Едно състояние q е достижимо за един автомат $M =< K, \Sigma, \Delta, s, F >$, ако съществува дума w, такава че $(s, w) \vdash_M^* (q, \varepsilon)$, т.е. ако има поредица от стрелки, започващи от s и стигащи до q. Всъщност, множеството S от достижими състояния можем да дефинираме:

$S \coloneqq \{s\}$

while съществува състояние $p \in S$ и $a \in \Sigma$, такива че $\delta(p, a) \notin S$

do добави $\delta(p, a)$към S

Или да определим релацията $R = \{(p, q) | \delta(p, a) = q$ за някое $a \in \Sigma\}$. Тогава S е затварянето на множеството $\{s\}$ относно релацията R.

Дефиниция (еквивалентност на думи относно език): Нека L е език в азбуката Σ и $x, y \in \Sigma^*$. Казваме, че x и y са еквивалентни относно L (пишем $x \approx_L y$), ако за всяка дума $z \in \Sigma^*$ е изпълнена еквивалентността: $xz \in L \Leftrightarrow yz \in L$.

Релацията $\approx_L$ е релация на еквивалентност.

Дефиниция (еквивалентност на думи относно автомат): Нека $M = (K, \Sigma, \delta, s, F)$ е КДА. Казваме, че думите x и y са еквивалентни относно M (пишем $x \sim_M y$) точно тогава когато съществува състояние $q \in K$, такова че $(s, x) \vdash_M^* (q, \varepsilon)$ и $(s, y) \vdash_M^* (q, \varepsilon)$.

Релацията $\sim_M$ е релация на еквивалентност с класове на еквивалентност, които зависят от състоянието q. С други думи, всеки клас на еквивалентност е: $E_q = \{x | (s, x) \vdash_M^* (q, \varepsilon)\}$.

Твърдение: За всеки КДА $M = (K, \Sigma, \delta, s, F)$ и за произволни думи $x, y \in \Sigma^*$ е изпълнено: ако $x \sim_M y$, то $x \approx_{L(M)} y$.

Теорема (Майхил-Нероуд): Нека L е регулярен език в азбуката Σ. Тогава съществува КДА M, който разпознава езика L и има точно толкова състояния, колкото са класовете на еквивалентност относно $\approx_L$.

Доказателство:

Нека с $[x]$ да означаваме класа на еквивалентност относно $\approx_L$, който съдържа думата x, за произволно $x \in \Sigma^*$. По дадения език L построяваме КДА $M = (K, \Sigma, \delta, s, F)$, както следва:

- $K = \{[x] | x \in \Sigma^*\}$,
- $s = [\varepsilon]$,
- $F = \{[x] | x \in L\}$,
- $\delta([x], a) = [xa]$ за $x \in \Sigma^*$ и $a \in \Sigma$.

Така построеният автомат се нарича стандартен автомат за езика L. Първото нещо, което трябва да проверим е дали K е крайно множество, защото говорим за КДА. Това е така, защото L е регулярен език и следователно той се разпознава от някой КДА и класовете на еквивалентност относно $\approx_L$ по брой са по-малко, отколкото броя на състоянията на този КДА съгласно теорема. Следователно, K е крайно множество.

След това трябва да отбележим, че функцията δ е добре дефинирана и не зависи от представителя от класа $[x]$, който избираме. Наистина, ако $y \in [x]$, то за всяко $z \in \Sigma^*$е изпълнено, че $yz \in L \Leftrightarrow xz \in L$, т.е. ако вземем произволна буква $a \in \Sigma$ и $z \in \Sigma^*$, то $xaz \in L \Leftrightarrow yaz \in L$. Следователно, $xa \approx_L ya$, т.е. $\delta([x], a) = \delta([y], a)$, ако $x \approx_L y$, т.е. дефиницията е коректна.

Следващата стъпка е да покажем, че $L = L(M)$. За тази цел ще използваме следното помощно твърдение:

За произволни $x, y \in \Sigma^*$ е изпълнено: $([x], y) \vdash_M^* ([xy], \varepsilon)$ (*)

Това ще докажем с индукция относно дължината $|y|$ на думата y:

1. Нека $|y| = 0$, т.е. $y = \varepsilon$. Тогава очевидно (*) е налице.

2. Нека (*) е изпълнено за всички думи $y \in \Sigma^*$ с дължина n и нека $|y| = n+1, y \in \Sigma^*$. Тогава $y = y'a$ за някоя $y', |y'| = n$ и $a \in \Sigma$. Следователно:
$$([x], y) = ([x], y'a) \vdash_M^* ([xy'], a) \vdash_M^* ([xy], \varepsilon),$$
т.е. (*) е налице, с което помощното твърдение е доказано.

Сега, за произволно $x \in \Sigma^*$ имаме $x \in L(M) \Leftrightarrow ([\varepsilon], x) \vdash_M^* ([x], \varepsilon)$ и $[x] \in F \Leftrightarrow x \in L$

(от дефиниция за F).

С това теоремата на Майхил-Нероуд е доказана.

Следствие: Един език L е регулярен точно тогава когато релацията на еквивалентност $\approx_L$ има крайно много класове на еквивалентност, т.е. точно тогава когато $\approx_L$ има краен индекс.

Алгоритъм за конструиране на минимален автомат, еквивалентен на даден краен детерминиран автомат

Нека $M = (K, \Sigma, \delta, s, F)$ е КДА, в който сме махнали всички недостижими състояния. Ще определим релация $A_M \subseteq K \times \Sigma^*$ както следва: $(q, w) \in A_M \Leftrightarrow (q, w) \vdash_M^* (f, \varepsilon)$ за някое $f \in F$. Посредством тази релация A_M ще дефинираме релация на еквивалентност, но този път между състоянията.

Дефиниция (еквивалентност на състоянията на краен автомат): Нека $p, q \in K$. Казваме, че p и q са еквивалентни (пишем $p \equiv q$) за КДА $M = (K, \Sigma, \delta, s, F)$ точно тогава когато за всички думи $z \in \Sigma^*$ е изпълнена еквивалентността: $(q, z) \in A_M \Leftrightarrow (p, z) \in A_M$.

Така две състояния p и q са еквивалентни точно тогава когато E_q и E_p се съдържат в един и същи клас на еквивалентност относно $\sim_M$. За да стигнем до алгоритъма, ние ще определим по-груби релации върху K от релацията $\equiv$, а именно $\equiv_0, \equiv_1 \equiv_2$, ... по следния начин:

q $\equiv_n p$ т.т.к. за произволни $z \in \Sigma^*$, такива че $|z| \leq n$ е изпълнена еквивалентността

$$(q, z) \in A_M \Leftrightarrow (p, z) \in A_M.$$

Най-груба е релацията $\equiv_0$, а след това е $\equiv_1$ и т.н. $\equiv_0$ се състои винаги само от два класа F и $K \backslash F$. Освен това броят на класовете не може да стане повече отколкото броя на състоянията на K. Така че всъщност, ние ще правим релациите по-фини, докато $\equiv_n$ и $\equiv_{n-1}$ съвпадат. Тогава $\equiv_{n-1}$ ще съвпада с $\equiv$. Последният етап от алгоритъма се дава от лемата за едната буква:

Лема (за едната буква): За всеки две състояния $q, p \in K$ и за всяко $n \geq 1$ $p \equiv_n q$ точно тогава когато са изпълнени следните две условия:

1. $p \equiv_{n-1} q$
2. За всички $a \in \Sigma$, $\delta(q, a) \equiv_{n-1} \delta(p, a)$

Тази лема ни дава възможност от $\equiv_{n-1}$ да получаваме $\equiv_n$. Сега вече алгоритъмът за минимизация добива вида:

Започваме с класовете $K \backslash F$ и F за еквивалентността $\equiv_0$. Използвайки лемата за едната буква, от $\equiv_{n-1}$, построяваме $\equiv_n$. По-точно:

Полагаме в началото F и $K \setminus F$ да са класовете на $\equiv_0$:

$repeat\ for\ n \coloneqq 1,2, \ldots$

намерете класовете на еквивалентност на $\equiv_n$ от $\equiv_{n-1}$

$until\ \equiv_n$ и $\equiv_{n-1}$ съвпадат

Държавен изпит – спец. „Софтуерно инженерство“

Тема №4 – Контекстно-свободни граматики и езици. Стекови автомати.

Анотация: *Контекстно-свободни граматики. Дървета за синтактичен анализ. Нормална форма на Чомски. Стекови автомати. Връзка между стековите автомати и контекстно-свободните граматики (доказателство в едната посока по избор). Свойства на затвореност. Лема за покачването (xyuvw) (с доказателство). Примери за езици, които не са контекстно-свободни.*

Контекстно-свободните езици (КСЕ) или както още се срещат контекстно-независимите езици са възникнали като опит да анализираме естествените езици като българския, английския и т.н. Този стремеж е бил продиктуван от стремежа за обработката на тези езици от компютри. Това е необходимо, когато се опитваме да проверим дали някое изречение на български е правилно построено или пък искаме компютъра да генерира правилно построени изречения. Така се стига до КСЕ и контекстно-свободните граматики. Едновременно с това, това да е следващата стъпка към усложняване на разглежданите дотук изчислителни модели като регулярните езици и крайните автомати. И тук ще имаме отново моделиране, както на генератори, така и на разпознаватели. Ще разгледаме разпознавателите, наречени стекови автомати. Ще започнем с по-сложния от регулярните езици вариант на генератори, наречен КСЕ и свързаното с него понятие контекстно-свободни граматики (КСГ). КСГ ще определят правилата, по които ще построяваме съответните смислени изрази (изречения).

Дефиниция (контекстно-свободна граматика): Контекстно-свободна граматика (КСГ) наричаме четворката $G = (V, \Sigma, R, S)$, където

А.) V – фиксирана крайна азбука от символи;

Б.) $\Sigma \subseteq V$ – елементите на Σ се наричат терминални символи или терминали; елементите на $V \backslash \Sigma$ се наричат нетерминали;

В.) R – множество от правила $R \subseteq (V \backslash \Sigma) \times V^*$;

Г.) $S \in V \backslash \Sigma$; S – начален символ (нетерминален).

Ако $(A, u) \in R$, то A е нетерминал, а $u \in V^*$. Често правилото (A, u) ще означаваме $A \to_G u$ (правило от G).

Дефиниция (релацията $\Rightarrow_G$ за контекстно-свободни граматики): Нека $G = (V, \Sigma, R, S)$ е контекстно-свободна граматика. Тогава пишем $u \Rightarrow_G v$ точно тогава когато $u, v \in V^*$ и съществуват $x, y \in V^*$ и $A \in V \backslash \Sigma$ и правило $A \to v'$, такива че $u = xAy, v = xv'y$.

Дефиниция (език, породен от контекстно-свободна граматика): Нека $G = (V, \Sigma, R, S)$ е контекстно-свободна граматика. Тогава с $L(G)$ означаваме множество $L(G) = \{w | w \in \Sigma^* \text{ и } S \Rightarrow_G^* w\}$ и се нарича език, породен от контекстно-свободна граматика G.

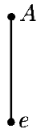
Дефиниция (контекстно-свободен език): Един език L се нарича контекстно-свободен език точно тогава когато $L = L(G)$ за някоя контекстно-свободна граматика $G = (V, \Sigma, R, S)$.

Твърдение: Нека $M = (K, \Sigma, \delta, s, F)$ е КДА. Тогава съществува КСГ $G = (V, \Sigma, R, S)$, такава че $L(M) = L(G)$.

Дефиниция (граматично дърво): Нека $G = (V, \Sigma, R, S)$ е произволна КСГ. Индуктивно определяме понятието граматично дърво:

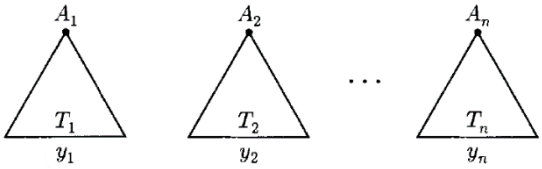
А.) За всяка $a \in \Sigma$, $\circ a$ е граматично дърво с корен a и листа a (тривиалното дърво).

Б.) Ако $A \to \varepsilon$ е правило в G, където $A \in V \backslash \Sigma$, то:

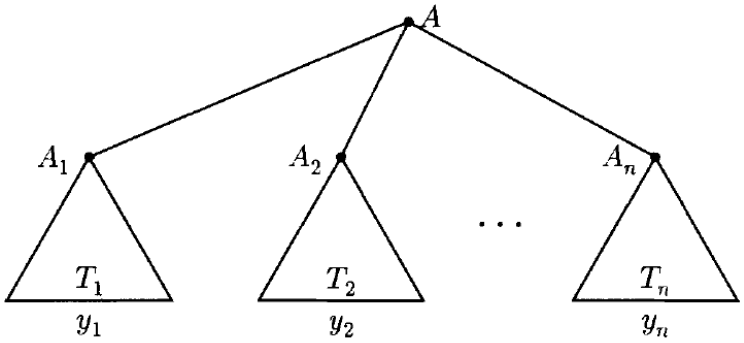


е граматично дърво с корен A и листо ε.

В.) Ако



са граматични дървета с корени $A_1, \ldots, A_n$ и $y_1, \ldots, y_n$ са думи в азбуката Σ, получени съответно от дърветата с корени $A_1, \ldots, A_n$, съответно и $A \to A_1 A_2 \ldots A_n$ е правило в граматиката G, то:



е граматично дърво.

Тук думата y_i се получава от дървото с корен A_i. y_i представлява думата, получена от всички листа на дървото в същия ред, в който се намират в дървото с корен A_i (отляво надясно). В такива случаи казваме, че дървото с корен A_i извежда думата y_i, а окончателното дърво с корен A извежда думата $y_1, \ldots, y_n$.

Г.) Нищо друго не е граматично дърво.

Нормална форма на Чомски

Дефиниция (КСГ в нормална форма на Чомски): Казваме, че една КСГ $G = (V, \Sigma, R, S)$ се намира в нормална форма на Чомски, ако $R \subseteq (V \backslash \Sigma) \times V^2$, т.е. всички правила са от вида: $A \to BC$, където $A \in V \backslash \Sigma$, а $B, C \in V$.

Теорема: За всяка КСГ G съществува КСГ G' в нормална форма на Чомски, такава че

$$L(G') = L(G) \backslash (\Sigma \cup \{\varepsilon\}).$$

С други думи, това, което се поражда в G' и това, което се поражда в G, евентуално се различава с краен брой думи, които са еднобуквени или ε. Тъй като G' е в нормална форма на Чомски, то тя не може да генерира такива символи.

Стекови автомати

Дефиниция (стеков автомат): Стеков автомат се нарича шестицата $M = (K, \Sigma, \Gamma, \Delta, s, F)$, където:

А.) K – крайна азбука от състояния;

Б.) Σ – крайна входна (основна) азбука;

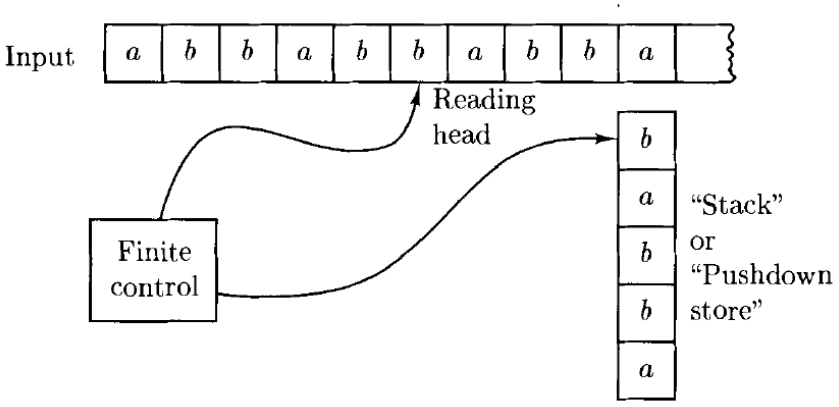
В.) Γ – крайно множество от стекови символи (стекова азбука);

Г.) $s \in K$, $s -$ начален символ;

Д.) $F \subseteq K$; $F -$ множество от заключителните състояния;

Е.) $\Delta \subseteq (K \times (\Sigma \cup \{\varepsilon\}) \times \Gamma^*) \times (K \times \Gamma^*)$; $\Delta -$ релация на преходите, като всеки елемент на Δ се нарича правило (за преход) Δ от M.

Интуитивно, ако четящата глава прочете буквата b върху безкрайната лента и още думата β от стека (четейки от горе надолу) и се намира в състояние q и има правило $((q, b, \beta), (p, \alpha))$ от Δ, то главата върху лентата се премества с една клетка надясно, ако $b \in \Sigma$ и остава неподвижна, ако $b = \varepsilon$. Устройството преминава в състояние p и замества думата β от стека с думата α, така че α да е начало на думата, намираща се в стека, прочетена от горе надолу. Например, ако имаме правило $\big((p, u, a), (q, \varepsilon)\big) \in \Delta$ и $a \in \Gamma$, то изтриваме a от стека, а ако $\big((p, u, \varepsilon), (q, a)\big) \in \Delta$, то добавяме a в стека.



Дефиниция (конфигурация на стеков автомат): Нека $M = < K, \Sigma, \Gamma, \Delta, s, F >$ е стеков автомат. Конфигурация за M се нарича всеки елемент на $K \times \Sigma^* \times \Gamma^*$. Ако $(p, x, \alpha) \in K \times \Sigma^* \times \Gamma^*$, то $p \in K, x \in \Sigma^*, \alpha \in \Gamma^*$.

Дефиниция (релация $\vdash_M$ за стеков автомат): Нека $M = (K, \Sigma, \Gamma, \Delta, s, F)$ е стеков автомат и $(p, x, \alpha), (q, y, \zeta) \in K \times \Sigma^* \times \Gamma^*$, т.е. са две конфигурации. Казваме, че от (p, x, α) за една стъпка се извежда (q, y, ζ) и означаваме $(p, x, \alpha) \vdash_M (q, y, \zeta)$ точно тогава когато съществува правило $\big((p, a, \beta), (q, \gamma)\big) \in \Delta$, такова че $x = ay, \alpha = \beta\eta$ и $\zeta = \gamma\eta$ за някое $\eta \in \Gamma^*$.

С $\vdash_M^*$ означаваме рефлексивното и транзитивно затваряне на релацията $\vdash_M$ в множеството на конфигурациите.

Дефиниция (приемане на дума): Нека $M =< K, \Sigma, \Gamma, \Delta, s, F >$ е стеков автомат. Казваме, че M разпознава думата $w \in \Sigma^*$ точно тогава когато $(s, w, \varepsilon) \vdash_M^* (p, \varepsilon, \varepsilon)$ за някое $p \in F$. Всички конфигурации от вида $(p, \varepsilon, \varepsilon)$, такива че $p \in F$, се наричат още заключителни конфигурации.

Дефиниция (език, разпознаван от стеков автомат): Нека $M =< K, \Sigma, \Gamma, \Delta, s, F >$ е стеков автомат. $L(M)$ се нарича език, който се разпознава от M и се означава с:

$$L(M) = \{w | w \in \Sigma^* \text{ и } w \text{ се разпознава от } M\} = \{w | w \in \Sigma^* \text{ и } (s, w, \varepsilon) \vdash_M^* (f, \varepsilon, \varepsilon) \text{ за } f \in F\}.$$

Връзка между стековите автомати и контекстно-свободните граматики

Дефиниция (език, разпознаван от стеков автомат): Един език L се разпознава от стеков автомат, ако съществува стеков автомат M, такъв че $L = L(M)$.

Теорема: Класът на езиците, разпознавани от стеков автомат, съвпада с класа на контекстно-свободните езици.

Твърдение 1: Всеки контекстно-свободен език се разпознава от стеков автомат.

Твърдение 2: Ако един език се разпознава от стеков автомат, то той е контекстно-свободен.

Доказателство на Твърдение 1:

Нека $G = (V, \Sigma, R, S)$ е КСГ. Ще построим стеков автомат M, такъв че $L(M) = L(G)$.

Нека $M = (\{p, q\}, \Sigma, V, \Delta, \mathrm{p}, \{\mathrm{q}\})$, където Δ се състои от следните правила за преход. Всъщност, ние ще ги опишем като типове правила. Те са три типа:

(1.) $((p, \varepsilon, \varepsilon), (q, S))$ – само едно единствено правило
(2.) $((q, \varepsilon, A), (q, x))$ – за всяко правило $A \to_G x$ от R (много на брой правила; точно толкова на брой, колкото са правилата в R)
(3.) $((q, a, a), (q, \varepsilon))$ – за всяко $a \in \Sigma$ (много на брой правила; точно толкова на брой, колкото са буквите в Σ)

Построеният стеков автомат M в известен смисъл имитира КСГ G в стека. Не съвсем, разбира се, а само частично, докато в началото на стека се появи терминал. После изтрива началото на стека, сравнявайки го с думата, която искаме да разпознаем. Причината е, че няма как да заменим нетерминал в средата на стека.

За да докажем твърдението, използваме следната лема:

Нека $w \in \Sigma^*$ и $\alpha \in \left((V \backslash \Sigma)V^*\right) \cup \{\varepsilon\}$. Тогава:

$$S \overset{L\ *}{\Rightarrow}_G w\alpha \text{ точно тогава когато } (q, w, S) \vdash_M^* (q, \varepsilon, \alpha)$$

С $\overset{L\ *}{\Rightarrow}_G$ означаваме най-ляв извод в G. Използвайки лемата следва:

$$w \in L(G) \Leftrightarrow S \overset{L\ *}{\Rightarrow}_G w \Leftrightarrow (p, w, \varepsilon) \vdash_M^* (q, w, S) \vdash_M^* (q, \varepsilon, \varepsilon) \text{ и } q \in F \text{ (тук } \alpha = \varepsilon) \Leftrightarrow w \in L(M)$$

Следователно, $L(G) = L(M)$.

Свойства на затвореност

Твърдение: Контекстно-свободните езици са затворени относно операциите обединение, конкатенация и звезда на Клини.

Твърдение: Сечението на контекстно-свободен език и регулярен език е контекстно-свободен език.

Твърдение: Множеството на контекстно-свободните езици не е затворено относно операциите сечение и допълнение.

Лема за покачването ($xyuvw$)

Дефиниция ($\Phi(G)$): Нека $G = (V, \Sigma, R, S)$ е КСГ. С $\Phi(G)$ ще означаваме най-големия брой букви на дясната част на правило от граматиката G.

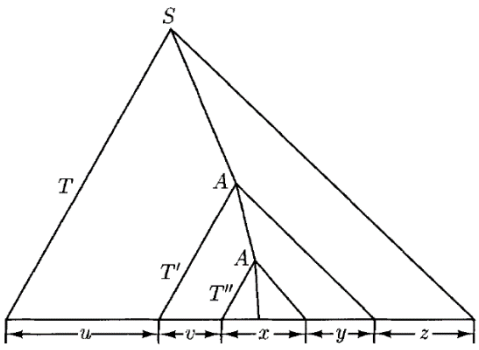
Разглеждаме всички правила от вида $A \to x$, където A е нетерминал, а $x \in V^*$. Намираме максимума на дължините $|x|$ на думите от дясната част на правилата. Това определя $\Phi(G)$. В термините на дърветата това означава разклонеността на граматичното дърво.

Лема: Изводът на всяко граматично дърво на контекстно-свободна граматика G с височина h има дължина най-много $\Phi(G)^h$.

Теорема (лема за покачването $/uvxyz/$ – I-ва формулировка): Нека $G = (V, \Sigma, R, S)$ е КСГ. Тогава за всяка дума $w \in L(G)$, чиято дължина $|w|$ е по-голяма от $\Phi(G)^{|V\setminus\Sigma|}$ може да се представи във вида $w = uvxyz$, така че $vy \neq \varepsilon$ и $uv^nxy^nz \in L(G)$ за всяко естествено число n.

Доказателство:

Нека $w \in L(G)$, такава че $|w| > \Phi(G)^{|V\setminus\Sigma|}$ и нека T е дървото, което извежда думата w и което има най-малко листа. Измежду всички дървета, които извеждат думата w (те са краен брой) има поне едно с най-малко листа. Това означава, че височината на дървото е по-голяма от $|V\setminus\Sigma|$ и значи е с височина поне $|V\setminus\Sigma| + 1$. Нека разгледаме пътя от корена до листото, където се достига тази височина. Този път ще съдържа поне $|V\setminus\Sigma| + 2$ върха, като само последният е терминал, а останалите поне $|V\setminus\Sigma| + 1$ върха са нетерминали. Следователно, има поне два еднакви върха – нетерминали.



Да означим тези върхове с A. Тогава те определят поддърво T' на T с корен A и поддърво T'' на T' (а и на T) с корен другото A, както е показано на фигурата. Тогава T'' поражда думата x, поддума на w, а дървото T' поражда дума vxy и $w = uvxyz$.

Да разгледаме сега поддървото T' на T. Частта (отляво и отдясно на T'') от дървото T' може да бъде повторена многократно, като с това ще се увеличат едновременно v-тата и y-ците, което потвърждава, че за всяко n, $uv^nxy^nz \in L$. Освен това, не е възможно $vy = \varepsilon$, защото в противен случай това дърво няма да е с най-малко листа, а дървото, което ще се състои от премахването на всички ребра и листа между двете A-та, ще остане с едно A и ще бъде с най-малко листа. С това лемата за разрастването е доказана.

Теорема (лема за покачването /$uvxyz$/ – II-ра формулировка): Нека $L \subseteq \Sigma^*$, L е контекстно-свободен език. Тогава съществува естествено число n, такова че за всяка дума $w \in L$, $|w| \geq n$ съществуват думи u, v, x, y, z, такива че $w = uvxyz$ и $|vxy| \leq n, vy \neq \varepsilon$ и за всички $i \in \mathbb{N}$ е изпълнено $uv^i xy^i z \in L$.

Тази формулировка следва директно от доказаната вече формулировка, като положим $n = \Phi(G)^{|V \setminus \Sigma|} + 1$ *и в доказателството на лемата си изберем в дървото* T'' *с корен* A *да няма повтарящи се възли. Това гарантира новото условие* $|vxy| \leq n$.

Примери за езици, които не са контекстно-свободни

(3.) Езикът $L = \{a^n b^n c^n | n \geq 0\}$ не е контекстно-свободен.

Доказателство: Да допуснем, че L е контекстно-свободен език и нека $p > 0$ *е свидетел за това. Нека* $w = a^p b^p c^p$. *Понеже* $w \in L$ *и* $|w| = 3p \geq p$, *то* w *може да се представи като* $w = uvxyz$ *за подходящи* u, v, x, y, z *като* $vy \neq \varepsilon, |vxy| \leq p$ *и за* $\forall i \geq 0 (uv^i xy^i z \in L)$. *Понеже* $vy \neq \varepsilon$, *то поне едно от* v *и* y *не е празно. Възможни са два случая:*

1. *Всички символи във* v *са едни и същи и всички символи в* y *са едни и същи. Тогава поне един от символите* a, b *и* c *не участва нито във* v, *нито в* y. *Така* $w_2 = uv^2 xy^2 z$ *не може да съдържа равен брой* a*-та,* b*-та и* c*-та. Следователно,* $w_2 \notin L$.
2. *Поне едно от* v *или* y *съдържа повече от един тип символи. Тогава* $w_2 = uv^2 xy^2 z$ *няма да съдържа символите в правилния ред (всички* a*-та преди всички* b*-та, преди всички* c*-та). Следователно,* $w_2 \notin L$.

И в двата случая намерихме $i \in \mathbb{N}$, *такова че* $uv^i xy^i z \notin L$. *Следователно, допускането ни, че L е контекстно-свободен е погрешно.*

(4.) Езикът $L = \{a^n b^m c^k | 0 \leq n \leq m \leq k\}$ не е контекстно-свободен.
(5.) Езикът $L = \{0^n | n \text{ е просто}\}$ не е контекстно-свободен.

Доказателство: Да допуснем, че L е КСЕ и нека $p > 0$ *е свидетел за безконтекстността на L. Нека* $q \geq p$ *е просто число (такова винаги съществува). Нека* $w = 0^q$. *Тогава* $w \in L$ *и* $w \geq p$. *Нека* $w = uvxyz$ *е такова представяне на* w, *че* $|vy| > 0, |vxy| \leq q$ *и* $\forall i \geq 0, uv^i xy^i z \in L$. *Тогава при* $i = q + 1$ *имаме, че:*

$$uv^{q+1} xy^{q+1} z = uxz(vy)^{q+1} = 0^{q-|vy|} 0^{(q+1)|vy|} = 0^{q+q|vy|} = 0^{q(1+|vy|)} \in L$$

Следователно, $q(1 + |vy|)$ *е просто. Това обаче е невъзможно, понеже* $q \geq 2$ $(q - \text{просто})$ *и* $1 + |vy| \geq 2$ $(|vy| > 0)$. *Противоречие. Следователно, допускането ни, че L е контекстно-свободен е погрешно.*

(6.) Езикът $L = \{ww | w \in \{0,1\}^*\}$ не е контекстно-свободен.

Държавен изпит – спец. „Софтуерно инженерство“

Тема №5 – Компютърни архитектури. Формати на данните. Вътрешна структура на централен процесор – блокове и конвейерна обработка, инструкции.

Анотация: *Обща структура на компютрите и концептуално изпълнение на инструкциите, запомнена програма.*

Формати на данните
- цели двоични числа;
- двоично-десетични числа;
- двоични числа с плаваща запетая;
- символни данни и кодови таблици.

Вътрешна структура на централен процесор
- регистри;
- аритметико-логическо устройство;
- регистър на състоянието и флагове;
- блок за управление.

Инструкции на централен процесор
- префикси;
- код на операцията;
- местоположение на операндите;
- модели на адресация на операндите;
- аритметико-логически инструкции;
- низови инструкции;
- безусловни и условни преходи;
- управление на програмата.

Основните функции на компютърната система са *информационна обработка*, *съхраняване на информацията*, *информационен трансфер* и *управление*. Компютърната система трябва да осъществява обмен с външния свят. Операционната среда на компютърната система включва устройства, които представляват източници или консуматори на информация. Получаването или предаването на информация от/към компютъра се осъществява чрез т.нар. *периферни устройства*, директно свързани към компютъра, а процесът се нарича вход/изход (Input/Output). Когато информацията се получава от или се предава на отдалечено устройство, процесът се нарича комуникация.

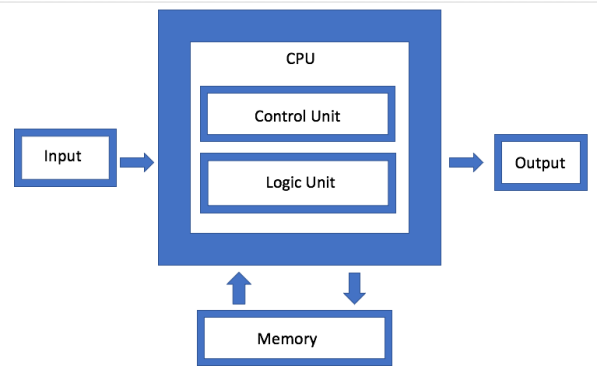
Структурата на компютърната система включва 4 базови градивни модула:

1. **Централен процесор**, осъществяващ *информационната обработка*;
2. **Памет**, осъществяваща *съхраняването на информацията*;
3. **Входно-изходна система**, осъществяваща *информационния обмен* между компютъра и външния свят;
4. **Вътрешни системни връзки**, осъществяващи *комуникациите* между централния процесор, паметта и входно-изходната система

Централният процесор обединява две компоненти: *аритметично-логическото устройство* (АЛУ), отговорно за обработката на информацията, и *управляващото устройство* (УУ), координиращо дейността на компютърните ресурси.

Структурата на компютърната система обхваща базовите градивни модули и начина на тяхното свързване. **Функционирането на компютъра** включва действието на всеки компонент като част от структурата. **Компютърната архитектура** включва структурата на компютъра, модела на изчислителния процес, и атрибутите, които директно влияят върху логическото изпълнение на програмата и системната производителност. Важни архитектурни белези са системата инструкции, дължината на машинната дума, режимите за адресиране на паметта, механизмите за вход-изход и др.

Исторически първа се е появила класическата фон Нойманова архитектура (по името на американския професор John von Neumann). Обобщеният структурен модел на компютър с класическа фон Нойманова архитектура е представен на фигурата:



Той съдържа **централен процесор**, обединяващ АЛУ и УУ, **памет** (главна и вторична) за съхраняване на програмите и данните, и **входно-изходна система**. Изчисленията се осъществяват под управлението на наредена последователност от инструкции (машинни команди), наречена *компютърна програма*. Характерно за фон Нойманова архитектура е чисто последователният характер на изпълнението на програмата, т.е. изпълнението на $i+1$-вата инструкция започва едва след извършването на изпълнението на i-тата инструкция.

Компютър със запаметена програма (stored-program computer) е компютър, при който програмните инструкции са записани в електронна компютърна памет. Компютър с архитектура на фон Нойман съхранява програмните инструкции и данните в една и съща памет докато компютър с Харвардска архитектура има отделни памети за програмите и данните.

Компютърните архитектури се развиват в две основни направления: *еволюционно* и *радикално*. **Еволюционното развитие** на компютърните архитектури се основава на усъвършенстването на класическата фон Нойманова архитектура чрез използване на паралелизъм на всички нива – *на ниво инструкции* (суперскаларни архитектури), *на ниво нишки* (многонишкови архитектури), *на ниво процеси* (мултипроцесори и мултикомпютри). **Радикалното направление** на развитие на компютърните архитектури включва модели на паралелни изчисления, които драстично се различават по механизмите си за управление на изчислителния процес от фон Нойнановата архитектура: *паралелни компютри с управление от потока данни* (data flow), *редукционни архитектури*, *логически паралелни компютри* и др.

Формати на данните

Общите категории данни в компютъра са **числа**, **адреси**, **символи**, **логически данни**. По своята същност адресите са числа и в компютъра те се третират като цели числа без знак. В някои компютри са заложени по-сложни типове данни като например структури от данни или

стринг от символи. Числовите данни в компютрите са три вида: **цели числа** (числа с фиксирана точка), **числа с плаваща точка** или **пакетирани десетични числа**.

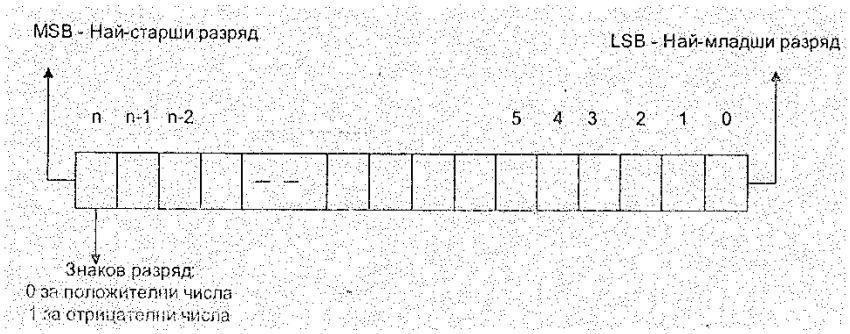
1. Двоично-десетични числа

За приложения с интензивен вход/изход и сравнително малко и прости изчисления се предпочита числата да се съхраняват и обработват в пакетиран десетичен код (packed decimal) или т.нар. двоично-десетични числа. Този код е *по-малко компактен* от директното бинарно представяне, но за сметка на това се *избягват допълнителните разходи за преобразуването*. По същество, най-голямата цифра в десетична бройна система е 9, чието представяне изисква една тетрада (1001). Следователно, за двоичното представяне на десетично число са необходими по една тетрада за всяка негова цифра. Във всяка тетрада се записва двоичният еквивалент на съответната десетична цифра на числото. При преобразуване на дадено число от двоична в десетична бройна система двоичното число се разделя на тетради и за всяка тетрада се записва десетичният еквивалент. Самата идея е сходна на тази при преобразуване от двоична в шестнайсетична бройна система и обратно. *Недостатък на това представяне е, че двоичната аритметика не работи за такъв формат*, което налага необходимостта от специални допълнителни инструкции за работа с такива числа.

Пример: $492 \leftrightarrow 0100\ 1001\ 0010\ (4 \rightarrow 0100, 9 \rightarrow 1001, 2 \rightarrow 0010)$

2. Цели двоични числа

При това представяне обхватът на представяните числа зависи от дължината на машинната дума (разрядността на решетката). При 64-битовите микропроцесори разрядната решетка съдържа 64 бита. Битовете се номерират от дясно на ляво, като най-левият бит е най-старшия и се означава с **MSB** (Most Significant Bit), а най-десният бит е най-младшият и се означава с **LSB** (Least Significant Bit).



Най-левият бит съдържа знака на числото, като положителните числа имат знаков разряд 0, а отрицателните числа – знаков разряд 1. При дължина на разрядната решетка n бита максималното число, което може да бъде представено , е $2^{n-1} - 1$, а най-малкото отрицателно представяно число е -2^{n-1}.

Представянето на данните в прав код има следния вид:

$$A_{\text{ПК}} = \begin{cases} 0, a_{n-1}, a_{n-2} \dots a_3, a_2, a_1, a_0 \text{ за } A \geq 0 \\ 1, a_{n-1}, a_{n-2}, \dots a_3, a_2, a_1, a_0 \text{ за } A < 0 \end{cases}$$

Пример: правият код на +5 е 0,0101, а правият код на -5 е 1,0101.

Недостатъците на правия код са необходимостта от различно третиране на знаковите и цифровите разряди при изпълнение на аритметичните действия и двойнственото представяне на нулата (0,0000000000000000 и 1,0000000000000000).

Представянето на данните в обратен код има следния вид:

$$A_{\text{ОК}} = \begin{cases} 0, a_{n-1}, a_{n-2}, \dots, a_3, a_2, a_1, a_0 \text{ за } A \geq 0 \\ 1, \overline{a_{n-1}}, \overline{a_{n-2}}, \dots, \overline{a_3}, \overline{a_2}, \overline{a_1}, \overline{a_0} \text{ за } A < 0 \end{cases}$$

Пример: обратният код на +5 е 0,0101, а обратният код на -5 е 1,1010.

Представянето на данните в допълнителен код има следния вид:

$$A_{\text{ДК}} = \begin{cases} \text{A}_{\text{ПК}} \text{ за } A \geq 0 \\ A_{\text{ОК}} + 0{,}000..01 \text{ за } A < 0 \end{cases}$$

Пример: допълнителния код на +5 е 0,0101, а допълнителният код на -5 е 1,1011.

Предимствата на допълнителния код са еднозначното представяне на нулата и еднозначното третиране на знаковите и цифровите разряди при аритметичните действия.

Недостатък на представените кодове е трудното идентифициране на препълването на разрядната решетка, когато резултатът от изпълненото действие е по-голям от максимално възможното представяно число за разрядната решетка. За преодоляване на този недостатък се използват т.нар. модифицирани кодове, при които знаковите разряди са два: положителният знак се кодира с две нули (00), а отрицателният знак – с две единици (11). Получаването на 10 или 01 в знаковите разряди на резултата от аритметичното действие показва възникнало препълване на разрядната решетка. Всички аритметични действия над числата с фиксирана точка се извършват в допълнителен код.

Основен недостатък на формата на данните с фиксирана точка е ограниченият обхват на представяните числа. Много малките числа се третират като машинна нула. Представянето на числа извън представения формат изисква мащабиране.

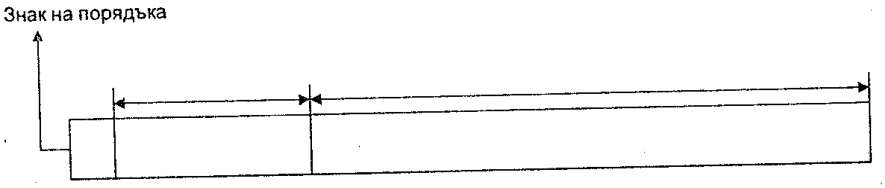
3. Двоични числа с плаваща запетая

Недостатъците на формата с фиксирана точка се преодоляват при формата с плаваща точка. При него представянето на числото има следния формат:

$$A = \pm M * B^{\pm E}$$

където A е представяното число, M – мантиса, E – порядък, B – основа, която не се задава в явен вид, тъй като е една и съща за всички числа. На следващата фигура е показан типичен 32-битов формат с плаваща точка. Най-левият бит съдържа **знака на числото** (0 – за положителни,

1 – за отрицателни числа). Стойността на **порядъка** (exponent) се съдържа в следващите 8 бита. Останалите 23 бита съдържат **мантисата** (fraction).



Числата с плаваща точка могат да имат **различни форми на представяне**. Така например дадено число може да има представянето $0.111*2^5$, $111*2^2$ или $0.0110*2^6$. За да се опрости обработката на числа с плаваща точка, те се представят в **нормализиран вид**. Нормализираното число с плаваща точка има следния вид: $\pm 0.1aaa \ldots a * 2^{\pm E}$, където a са двоични цифри 0 или 1. Това означава, че най-левият бит на мантисата трябва да бъде винаги 1. Поради факта, че този бит винаги има стойност 1 в нормализиран вид, той не се съхранява в явен вид. По този начин 23-битовото поле се използва за съхраняване на 24-битова мантиса с допустими стойности между 0.5 и 1.

В обобщение, възприет е стандартът едно число да се кодира в 32 бита, като полето за порядъка е 8 бита, а останалите 24 бита са за мантиса и знак. Освен това представяне обаче, за по-голяма точност се използват и двойни думи (по дължина) double precision от 64 бита.

Почти при всички съвременни компютри за представянето на данните във формат с плаваща точка се прилага стандартът на Института на електроинженерите и инженерите по електроника *IEEE Standard 754*, който е разработен с цел осигуряването на преносимостта на програмите и стимулирането на разработването на програми за интензивни числени приложения.

Обработка на числа с плаваща точка:

Събирането и изваждането на числа с плаваща точка обхваща следните стъпки:

1. *Изравняване на порядъците (чрез съответното изместване на мантисите);*
2. *Сумиране (изваждане) на мантисите;*
3. *Нормализиране на резултата.*

Умножението (делението) на числа с плаваща точка обхваща следните стъпки:

1. *Умножение (деление) на мантисите;*
2. *Събиране (изваждане) на порядъците;*
3. *Нормализиране на резултата.*

4. Символни данни и кодови таблици

Текстовата форма на данните е много удобна за човек, но не е удобна за системи, които използват бинарно представяне. Затова символите се представят чрез последователности от битове. Най-популярният код за представяне на символни данни е американският стандартен код за информационен обмен **ASCII** (American Standard Code for Information Interchange). При този код всеки символ се представя с последователност от 7 бита, при което е възможно

използването на 128 комбинации. Една част от кодовете се използва за представяне на символите, останалата част се използва за кодиране на управляващи символи, например за управлението на отпечатването на страница или за комуникационни процедури. Кодираните символи се съхраняват и предават в 8-битов формат, като осмият бит може да съдържа 0 или контролен бит за проверка по четност. Друг 8-битов код, който се използва за представяне на символи, е **EBCDIC** (Extended Binary Coded Decimal Interchange Code) на IBM. Друга универсална кодова таблица е **UNICODE**, която е 16-битова. Както ASCII кодът, така и кодът EBCDIC са съвместими с пакетираното десетично кодиране.

5. Логически данни

Информационните единици, които могат да се адресират индивидуално в компютъра като байт, дума, полудума и т.н. се третират като цялостни информационни единици. **Логическите операции** в процесора обаче са **побитови**, т.е. операциите се извършват над битове с еднакви позиции в операндите. Следователно, представянето на логическите данни в компютъра трябва да бъде **бит-ориентирано**, т.е. данните могат да приемат само две стойности: **1** (true) и **0** (false). Едни и същи данни могат да бъдат третирани като числови, логически или символни. Типът на данните се определя от операциите, които се извършват с тях.

Вътрешна структура на централен процесор

Централният процесор е изграден от елементарни електронни устройства, които съхраняват информацията в двоична (бинарна) форма и изпълняват прости логически операции над данните. Логическите променливи са такива променливи, които могат да приемат две стойности: *истина* (true) и *лъжа* (false). Действията с логически променливи са обект на разглеждане в булевата алгебра. В компютъра преобразуването на логическите променливи се осъществява от комбинационни логически схеми. Те представляват многоетажни структури от логически елементи, които осъществяват преобразуването на логическите променливи съобразно логическите функции, които те реализират. Комбинационните логически схеми не съхраняват информация, а само я преобразуват. Съхраняването на информацията се осъществява в електронни елементи памет, наречени *тригери*. Тригерът представлява логическа схема с две устойчиви състояния: логическа 1 и логическа 0. Той съхранява един бит информация. **Регистърът** представлява набор от тригери, броят на които в общия случай се определя от дължината на машинната дума, т.е. от разрядността на микропроцесора. При 64-битовите процесори дължината на регистрите е 64 бита, т.е. те съдържат 64 тригера. Процесорите могат да съдържат регистри с дължина двойна или четворна машинна дума.

Информационните единици в компютрите са както следва:

- *1 байт съдържа 8 бита (1 byte = 8 bits);*
- *1 килобит съдържа 2^{10} бита или 1 Кбита (1 Kbits = 1024 bits = 2^{10} bits);*
- *1 килобайт съдържа 2^{10} байта или 1 Кбайта (1 Kbytes = 1024 bytes = 2^{10} bytes);*
- *1 мегабайт (Mbytes) съдържа 1024 Kbytes;*
- *1 гигабайт (Gbytes) съдържа 1024 Mbytes;*
- *1 терабайт (Tbytes) съдържа 1024 Gbytes.*

Дължината на машинната дума на 64-битовите процесори е 8 байта.

Централният процесор на съвременен компютър включва богат спектър от функционални блокове: шинно интерфейсно устройство, изпълнително устройство, устройство за инструкции, адресно устройство, странично и сегментно устройство, управляващо устройство, кеш за инструкции, кеш за данни.

Изпълнителното устройство включва **аритметично-логическото устройство** (АЛУ), което осъществява аритметичните и логическите операции над данните. Операндите и резултатите от аритметичните и логическите операции се съхраняват в **регистри**, които осигуряват временното им съхраняване за подаване на операндите към АЛУ и поемане на резултата след изпълнението на операцията от АЛУ. *Входните сигнали* за АЛУ са операндите от регистрите и сигнали от управляващото устройство на централния процесор, а *изходните сигнали* са резултатите от изпълнените операции, съхранявани временно в регистри и флагове, отразяващи вида на получените резултати.

Централният процесор функционира в два основни режима: системен режим (S – System) и потребителски режим (U – User). Системният режим (още наречен супервайзор) е привилегирован и при него е активна операционната система. При него всички инструкции от набора инструкции на процесора са разрешени за изпълнение, както и всички ресурси на компютърната система са достъпни. В режим на потребител се изпълняват приложни програми. При него не са разрешени за изпълнение т.нар. привилегировани инструкции, както и не всички ресурси на компютърната система са достъпни.

Централният процесор е отговорен за обработката на данните и за управлението на системата, което определя и основните му компоненти: **АЛУ** и **УУ**. Компонентите на процесора са свързани чрез **вътрешно системна шина**, чрез която се осъществява трансфера на данни и инструкции към и от процесора и се управлява работата на АЛУ. АЛУ получава операндите си и *записва резултатите в регистрите на процесора*. Освен това регистрите се използват за формиране на адреса и съхраняване на информацията, която се получава от или изпраща към паметта. *За управление на системата процесорът използва т.нар. управляващи регистри и регистри за състоянието*.

Централният процесор съдържа **регистри**, които са програмно достъпни (от програмите на машинен език) и които формират т.нар. *програмен модел на процесора*. Тези регистри при съвременните микропроцесори могат да бъдат организирани в следните функционални групи: **универсални регистри (регистри с общо предназначение)**, **регистри за данни**, **адресни регистри**, **регистри за настройка на програмите**, **регистри за числа с плаваща точка**, **регистри за мултимедия**, **регистри за SIMD изчисления**, **флагов регистър**. По-конкретно някои примери за регистри са:

- **PC (Program Counter)** – програмен брояч, съдържащ адреса на следващата инструкция, която трябва да бъде извлечена от паметта
- **CS (Code Segment)** – сегментен регистър на кода
- **DS (Data Segment)** – сегментен регистър за данни
- **SP (Stack Pointer)** – указател на стека, съдържащ адреса на върха на стека, който е организиран в паметта
- **MAR (Memory Adress Register)** – адресен регистър на паметта, който съдържа адреса, по който се извършва обръщането към паметта

Управляващите регистри и регистрите за състоянието се използват от УУ за определяне на режима на работа на процесора и на цялата система, а също така те се използват при изпълнението на привилегировани инструкции от операционната система за управление изпълнението на програмата. Обикновено, управляващите регистри и регистрите за състоянието или поне повечето от тях са прозрачни за потребителя. Някои от тях са достъпни в режим супервайзор.

Регистърът PSW (дума за състоянието на програмата) съдържа информация за състоянието на процесора и обикновено включва и флаговете. Освен аритметичните флагове се съдържат и флаг за разрешаване/забраняване на прекъсванията, за режима на работа на процесора (супервайзор/потребител), текущото ниво на приоритета на обслужваните прекъсвания, ниво на привилегированост на входно-изходните операции, флаг за вложени задачи и др. Процесорите разполагат с множество управляващи регистри CR за задаване на режимите му на работа или индицират състоянието му при изпълнението на дадена задача. Съвременните процесори поддържат многозадачен режим.

Инструкции на централен процесор

Системата инструкции е изключително важен архитектурен белег на всеки процесор. Компютърът изпълнява програма, която представлява наредена последователност от инструкции. Чрез системата инструкции програмистът управлява работата на компютърната система. Компютърните програми се превеждат на **машинен език**, т.е. в **машинни инструкции**. Те имат бинарно представяне, т.е. съдържат само нули и единици. Действието на процесора зависи от инструкцията, която той изпълнява. Множеството машинни инструкции, които даден процесор може да изпълнява, се нарича *система инструкции на процесора*.

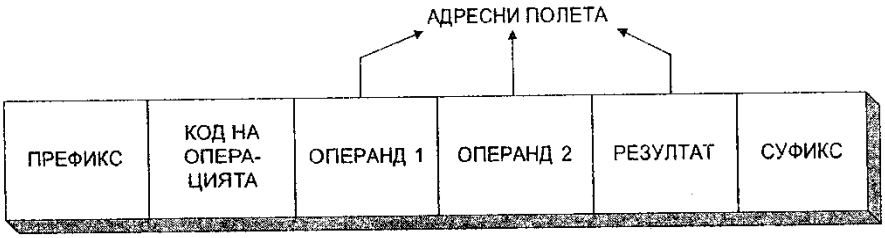
Системата инструкции на процесора трябва да бъде функционално пълна, т.е. всеки оператор от езиците за програмиране на високо ниво да може да бъде реализиран чрез машинните инструкции на процесора. Всяка машинна инструкция трябва да съдържа необходимата информация за процесора за изпълнението на операцията над данните. Форматът на инструкцията дефинира броя на полетата в рамките на инструкцията, тяхното функционално предназначение и дължина.

Форматът на инструкцията съдържа **код на операцията**, наричан още **операционен код** (КОП), **полета за операндите**, **поле за резултата** и евентуално **адресно поле за следващата инструкция**.

Фазите на изпълнение на инструкцията от процесора са следните:

1. *Изчисляване на адреса на инструкцията;*
2. *Извличане на инструкцията от паметта и зареждането ѝ в регистъра на инструкцията (IR);*
3. *Декодиране на операцията;*
4. *Изчисляване на адресите на операндите;*
5. *Четене на операндите от паметта;*
6. *Изпълнение на операцията;*
7. *Изчисляване на адреса на резултата;*
8. *Запис на резултата.*

Форматът на **триадресна компютърна инструкция** е:

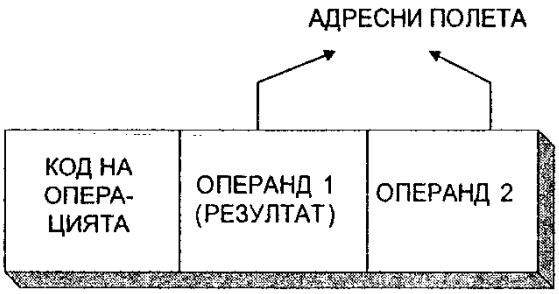


Полетата **префикс** (обикновено с дължина 1 байт) и **суфикс** служат за указване на специални операции. Така например префиксът *ESCAPE* указва операция на данни с формат с плаваща точка, префиксът *LOCK* осигурява ексклузивно използване на общата памет в мултипроцесори и т.н.

При процесорите съществуват два варианта за указване на следващата инструкция – естествен ред на изпълнение на инструкциите и принудителен ред на изпълнение на инструкциите. При естественя ред на изпълнение на инструкциите адресът на текущата изпълнявана от процесора инструкция се съдържа в специален адресен регистър, наречен програмен брояч. След приключването на изпълнението ѝ програмният брояч автоматично увеличава съдържанието си, като по този адрес се извлича следващата инструкция, подлежаща на изпълнение. Следователно при естествен ред на изпълнение на инструкциите не е необходимо да се използва допълнително поле във формата на инструкция за указване на адреса на следващата инструкция. В случаите, когато процесорите използват принудителен ред на изпълнение на инструкциите, последното поле във формата на инструкцията съдържа адреса на следващата инструкция. Съдържанието на това поле се прехвърля в програмния брояч и по този адрес се извлича от паметта следващата инструкция.

Пример: $ADD\ R_1, A$ – съдържанието на клетка от паметта с адрес A трябва да се прибави към съдържанието на регистър R_1

Триадресният формат на инструкциите не се използва в много процесори, тъй като изисква голяма дължина на инструкцията, за да съдържа трите адресни полета. При **двуадресния формат** инструкцията съдържа две адресни полета, като едно от адресните полета трябва да съдържа както адреса на единия операнд, така и адреса на резултата:



Инструкциите с двуадресен формат имат по-малка дължина, но предизвикват известно неудобство. Преди изпълнението на операцията е необходимо единият операнд, на чийто адрес ще бъде записан резултатът, да бъде записан във временна клетка от паметта или временен регистър.

В съвременните процесори се използва и **едноадресният формат на инструкциите**. При този формат вторият адрес трябва да бъде зададен в неявен вид. Обикновено неявно заложеният адрес е на регистър в процесора, наречен *акумулатор*. Акумулаторът съдържа единия адрес на операнда и се използва за запис на резултата. При едноадресен формат на инструкциите дължината на програмите се увеличава, тъй като са необходими допълнителни инструкции, които да заредят операндите в акумулатора.



Броят на адресните полета във формата на инструкцията влияе съществено върху архитектурата на процесора. По-малък брой на адресните в инструкцията водят до по-примитивни инструкции, които от своя страна опростяват структурата на процесора. При едноадресни инструкции програмистът разполага само с един универсален регистър, акумулатора. При инструкции с повече адреси се използват множество универсални регистри. Това дава възможност някои операции да се изпълняват само с регистри. Достъпът до ресурсите е много по-бърз от достъпа до паметта, така че изпълнението се ускорява. За да се постигне по-голяма гъвкавост и да се ускори изчислителния процес съвременните процесори използват както двуадресни, така и триадресни инструкции.

Според това дали **операндите се съдържат в регистри или в клетки в паметта** се различават следните видове операции в компютърната система: **операции регистър-регистър** (R-R), **регистър-памет** (R-M) и **памет-памет** (M-M). Адресът на регистрите е по-къс, тъй като броят на регистрите в процесора не е голям. Обикновено се използват 4 бита за адресиране на 16 регистъра. Инструкциите, които използват за операнди съдържанието на регистри, са по-къси и се изпълняват по-бързо. В този аспект най-бързи са операциите регистър-регистър, а най-бавни са операциите памет-памет, тъй като изискват достъп до паметта. Операциите от типа регистър-памет заемат междинно място по бързодействие. Програмите, съдържащи операции регистър-регистър, са най-дълги, а програмите, съдържащи операции памет-памет, са най-компактни.

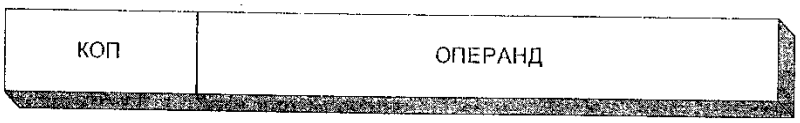
Система инструкции, при която всеки операционен код може да бъде съвместен с произволен режим на адресиране, се нарича ортогонална система инструкции. При повечето процесори обаче различните операционни кодове изискват използването на различни режими на адресиране. Във формата на инструкцията са необходими един или повече бита, задаващи режима на адресиране, който ще бъде използван при изпълнението на инструкцията.

Най-често използваните **режими на адресиране** в съвременните процесори са **непосредствено**, **имплицитно**, **директно**, **индиректно адресиране**, **адресиране на регистри**, **адресиране с изместване** (**адресиране с индексиране**, **релативно адресиране**, **адресиране с базиране**).

При режимите на адресиране се използва **ефективен адрес**. При системи без виртуална памет ефективният адрес представлява адрес в паметта или регистър. При компютърни системи с

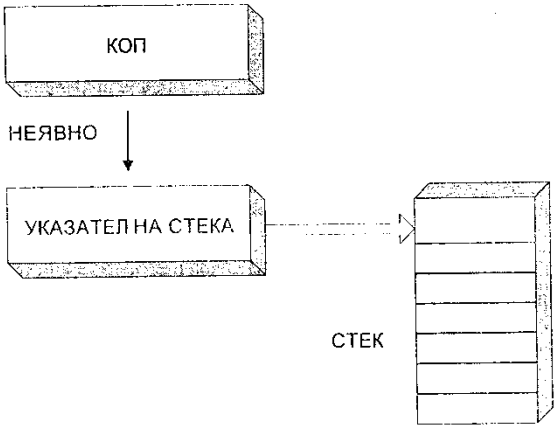
виртуална памет ефективният адрес представлява виртуален адрес или регистър. Преобразуването на виртуалния адрес във физически се реализира прозрачно за програмиста.

При **непосредственото адресиране** адресното поле на инструкцията съдържа стойността на операнда. Числото е представено в допълнителен код, като най-левият бит на операнда съдържа неговия знак.

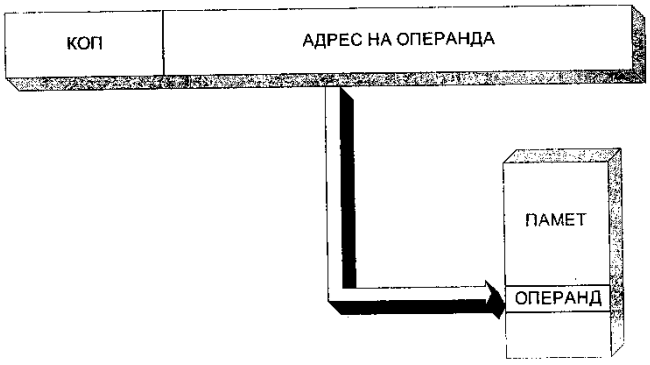


Предимството на непосредственото адресиране е, че се спестява достъп до паметта за четене на операнда. Този режим се използва за задаване на константи или начални стойности на променливи. Негов недостатък е, че дължината на операнда се ограничава от дължината на адресното поле на инструкцията, което при повечето процесори е по-малко от дължината на машинната дума.

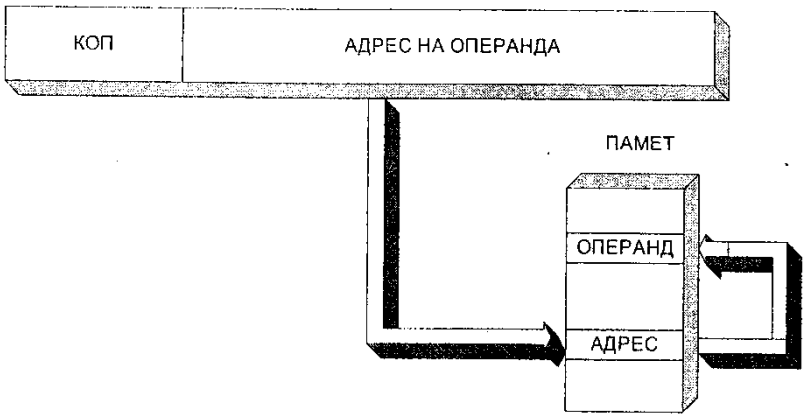
При **имплицитното адресиране** се използва **стек**. Стекът представлява резервиран блок от клетки в паметта. Той представлява опашка с дисциплина на обслужване последен влязъл, пръв излязъл (LIFO – Last In First Out). Данните се записват на върха и се четат пак от върха на стека. Върхът на стека се адресира от **регистър**, който се нарича *указател на стека* (SP – Stack Pointer). При някои процесори двата елемента на върха на стека се съхраняват в регистрите на процесора, така че указателят на стека се обръща към третият елемент на стека.



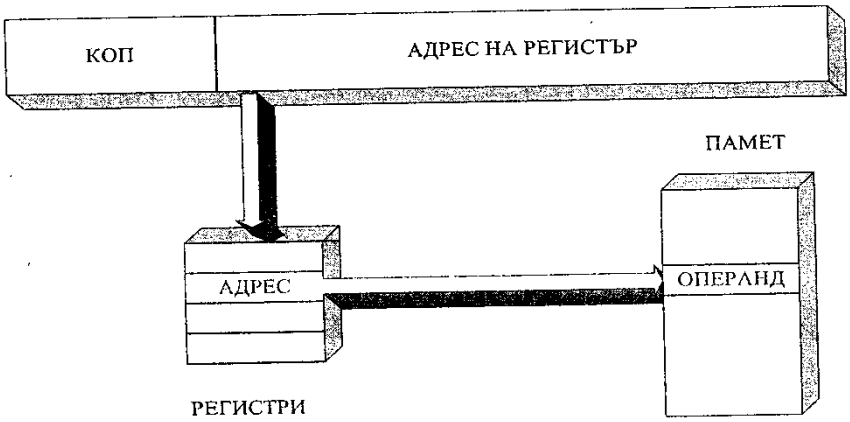
Директното адресиране е най-простият и бърз режим на адресиране на клетки в паметта. При него адресното поле на инструкцията съдържа ефективния адрес на операнда. Поради ограничения размер на адресното поле директната адресация може да се използва за обръщение към ограничена начална област на паметта.



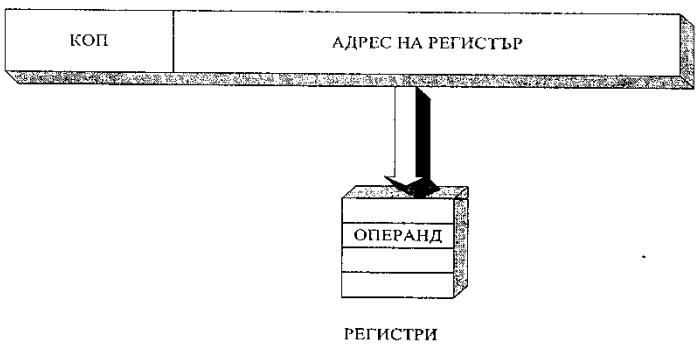
При **индиректната адресация** адресното поле в инструкцията указва клетка в паметта, която съдържа ефективния адрес на операнда. Поради ограничения размер на адресното поле в инструкцията клетките, съдържащи ефективните адреси, се съсредоточават в нулева страница на паметта. Недостатък на индиректното адресиране е необходимостта от два достъпа до паметта за четене на операнда – един за четене на ефективния адрес и един за четене на самия операнд. Предимството е, че се разширява обхватът на адресните клетки памет.



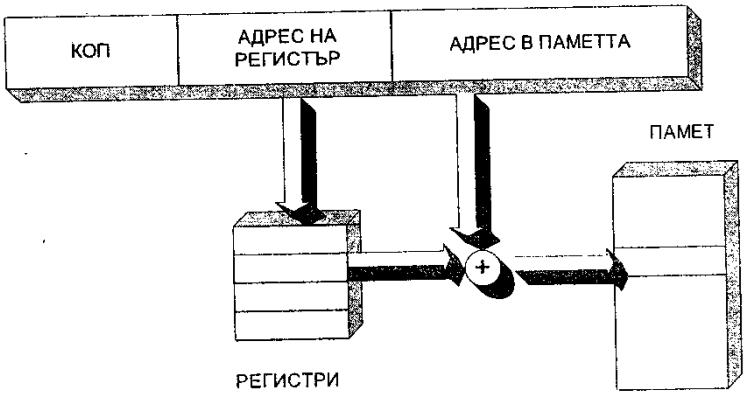
При **индиректното адресиране с регистри** адресното поле на инструкцията съдържа адреса на регистъра, съхраняващ ефективния адрес на операнда. Този режим е по-бърз от индиректното адресиране чрез клетка от паметта, тъй като се спестява един достъп до паметта.



Адресирането на регистри е подобно на директното адресиране с тази разлика, че броят на регистрите в процесора не е голям и адресното поле в инструкцията може да съдържа няколко бита, обикновено 4. Използването на регистри за съхраняване на операциите е ефективно, когато тези операнди участват в множество операции или представляват междинни операции. Съвременните процесори съдържат за тази цел универсални регистри, като определянето на операндите, които ще се съхраняват в регистри, се предоставя на компилатора.



Адресирането с изместване съчетава възможностите на директното адресиране и индиректното адресиране с регистри. Този режим изисква две адресни полета, поне едно от които трябва да бъде зададено в явен вид. Ефективният адрес се формира като сума от адреса, зададен в адресното поле на инструкцията и съдържанието на определен регистър.



При **релативното адресиране** се използва съдържанието на програмния брояч (PC – Program Counter). Ефективният адрес се получава, като адресът на текущата изпълнявана инструкция в програмния брояч се сумира със съдържанието на адресното поле на инструкцията. Следователно, ефективният адрес се определя чрез изместване спрямо адреса на текущата инструкция. Релативното адресиране поддържа локалността на достъпите до паметта.

При **адресирането с базов регистър** ефективният адрес се формира като сума от съдържанието на базовия регистър и съдържанието на адресното поле на инструкцията. Адресирането с базов регистър също поддържа локалността на достъпите до паметта. Този режим ефективно се използва при сегментната организация на паметта. Базовият регистър съдържа началния адрес на сегмента, а адресното поле на инструкцията съдържа изместването спрямо началния адрес, което обикновено е цяло число без знак.

При **адресирането с индексиране** адресното поле в инструкцията съдържа адрес на клетка в паметта, а индексният регистър съдържа положително изместване спрямо този адрес, ефективният адрес се формира като сума от съдържанието на адресното поле на инструкцията и съдържанието на индексния регистър. Този режим на индексиране осигурява ефективен механизъм за реализация на итеративни операции, поради което след всяко обръщане към индексния регистър е необходимо той да бъде инкрементиран (да се увеличи съдържанието му с 1). При процесори, при които това се реализира автоматично, режимът се нарича *автоиндексиране*. Когато модификацията на съдържанието на индексния регистър се осъществява след изпълнението на операцията, режимът се нарича *постиндексиране*, в противен случай – *преиндексиране*.

Броят и видовете операционни кодове в системата инструкции варират за различните процесори, но съществуват общи **функционални групи инструкции**, които се поддържат от всеки процесор. Това са групите за **трансфер на данни**, за **аритметични операции**, за **логически операции**, за **конвертиране на данните**, **вход/изход**, групата инструкции за **управление на системата** и за **предаване на управлението**.

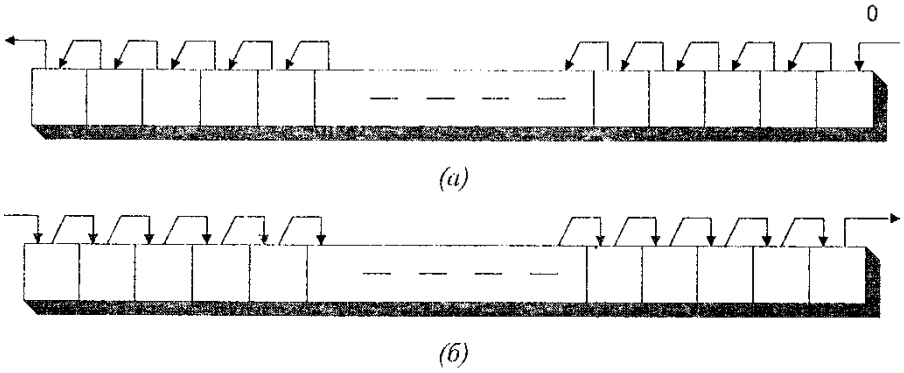
Инструкциите за трансфер на данни. Източниците и дестинациите на трансфера на данни в компютърната система могат да бъдат регистрите на процесора, клетки в паметта, стека. Във всяка инструкция за трансфер на данни трябва да бъдат указани *източникът* и *дестинацията* при трансфера, както и дължината на обменяните данни. Така например **MOV** се използва за

трансфер на данни между регистрите или регистрите и паметта, **PUSH** – за запис в стека, **POP** – за четене от стека и др.

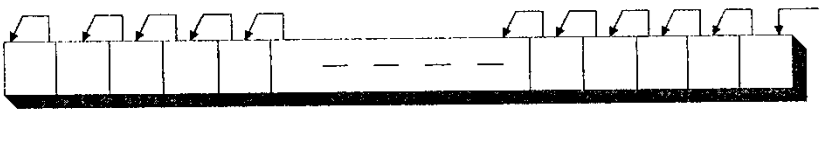
Инструкции за аритметична обработка. Аритметичните операции включват *събиране* **ADD**, *изваждане* **SUB**, *умножение* **MUL**, *деление* **DIV** *на числа с фиксирана точка*, *плаваща точка и пакетирани десетични числа*. Също така се включват и едноместни операции като изчисляване на *абсолютна стойност* на операнда, *промяна на неговия знак*, *увеличаване на операнда с 1* (инкрементиране) и *намаляване на операнда с 1* (декрементиране). Изпълнението на аритметична операция предизвиква трансфер на данни за зареждане на операндите в регистрите на процесора, за да могат да бъдат ползвани от АЛУ, както и запис на резултата от АЛУ в регистър на процесора.

Инструкции за логическа обработка. При тях се изпълняват булеви операции над индивидуалните битове на дума или друга информационна единица. Тази група включва *логическата инверсия* **NOT**, при която се инвертират битовете на операнда, *конюнкция* **AND** (логическо умножение), *дизюнкция* **OR** (логическо сумиране) и *ексклузивна дизюнкция* **XOR**. Конюнкцията е такава логическа операция, при която е достатъчно един от операндите да е 0, за да бъде резултатът 0. Тя се използва за *маскиране на операнда*, т.е. определени битове се селектират, а останалите битове се нулират. Дизюнкцията е такава логическа операция, при която е достатъчно един от операндите да е 1, за да бъде резултатът 1. Ексклузивно логическо сумиране е такава логическа операция, при която при различни операнди резултатът е 1, а при еднакви операнди резултатът е 0. Когато единият операнд съдържа само единици, операцията ексклузивна дизюнкция формира допълнителен код на другия операнд.

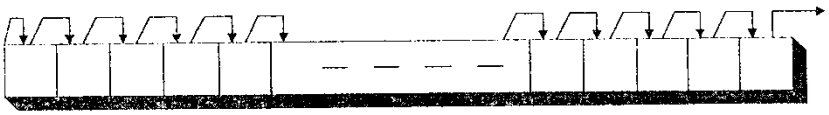
Към тази група инструкции спадат и **инструкциите за аритметично и логическо изместване и ротация**. При логическото изместване битовете на операнда се изместват с една или повече позиции в ляво **SHL** или в дясно **SHR**, при което в единия край битовете се изгубват, а в другия край се вкарват нули. Логическите операции за изместване се използват за изолиране на полета от битове в думата. Излишните битове се изместват извън рамките на разрядната решетка



При **аритметичното изместване данните** се третират като цяло число със знак и знаковият бит не участва в изместването. При аритметичното изместване в дясно **SAR** освободените позиции надясно от знаковия бит приемат неговата стойност. Аритметичното изместване надясно с 1 разряд на числа в допълнителен код е еквивалентно на делене на 2, а аритметичното изместване вляво **SAL** с 1 разряд е еквивалентно на умножение с 2 при положение, че не възниква препълване или незапълване.

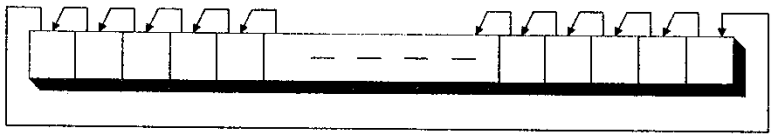


(а)

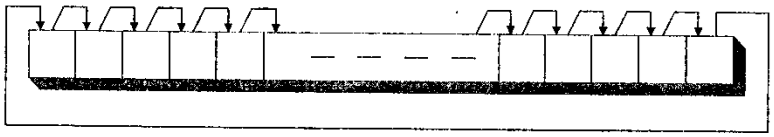


(б)

При **операциите за циклично изместване** (ротация) в ляво **ROL** и в дясно **ROR** се запазват всички битове на операндите:



(а)



(б)

Инструкции за конвертиране на данните служат за форматиране на данните или за промяна на техния формат – например конвертирането на ASCII код в EBCDIC или трансформирането на десетично пакетиран код в двоичен код.

Инструкциите за управление на системата са привилегировани инструкции, които се изпълняват в режим *супервайзор* на централния процесор от операционната система. Такива са инструкциите за четене или зареждане на управляващите регистри, промяна на ключ за защита на паметта и др.

Групата **инструкции за предаване на управлението** променят естествения ред на изпълнение на инструкциите в компютърната програма, като актуализират съдържанието на програмния брояч с адреса на инструкцията, подлежаща на изпълнение. Тази група включва **инструкциите за преход**, **инструкциите за извикване на и връщане от процедура**, **примитиви за цикъл**, **инструкции за програмно прекъсване**. **Инструкциите за безусловен преход** **JMP** съдържат в адресното си поле адреса на следващата инструкция, която трябва да бъде изпълнена. Те предизвикват зареждане на указания адрес на инструкцията в програмния брояч. При **инструкциите за условен преход** се тества кодът на условията **CC** (Condition Code), който включва флаг, отразяващ типа на резултата на определена операция. Аритметичните флагове, които се използват, са *пренос* **C** (carry), *четност* **P** (parity), *междутетраден пренос* **A** (auxiliary carry), *нулев резултат* **Z** (zero), *знак на резултата* **S** (sign), *препълване* **O** (overflow). При положение, че условието е изпълнено, програмният брояч се зарежда с адреса на инструкцията, указана в адресното поле. В противен случай се запазва естественият ред на изпълнението на инструкциите, т.е. изпълнява се следващата инструкция. **Инструкциите за извикване на процедури** *CALL* преустановяват изпълнението на главната програма и стартират изпълнението на процедурата чрез обръщане към началния адрес на процедурата, зададен в адресното поле на инструкцията за извикване. **Инструкциите за връщане от процедура** *RETURN* предизвикват изпълнение на инструкцията от извикващата

програма, непосредствено следваща инструкцията за извикване на процедурата. За да се осигури поддържането на **реентрантни процедури**, при които е възможно в даден момент да има няколко извиквания, е препоръчително използването на стек. При изпълнение на инструкция за извикване на процедура, адресът на връщане се записва на върха на стека. При изпълнение на инструкция за връщане от процедура, адресът на връщане се чете от върха на стека. Друг проблем е предаването на параметрите, което също е препоръчително да бъде осъществено през стека. Информацията, която се записва в стека при извикване на процедура, включва параметрите и адреса на връщане, и се нарича рамка. За поддържане на този механизъм се използват също инструкциите *ENTER* и *LEAVE*. Към тази група спадат и **инструкциите за поддържане на цикъл** *LOOP*, както и **инструкциите за спиране на изпълнението на програмата** *HALT* и *WAIT*.

Инструкциите за символна обработка. Стрингът се дефинира като наредена последователност от символи. Операциите, които се поддържат, са *трансфер на стринг* **MOVS**, *сравнение на стрингове* **CMPS**, *търсене на шаблон*, *търсене на шаблон с канонично заместване*, *конкатенация* (присъединяване на стрингове), *въвеждане на стринг* **INS**, *извеждане на стринг* **OUTS**, както и *префикс* **REP** (няколко разновидности) *за повторение на указаната операция* последователно за всички елементи на стринга.

Инструкции за вход/изход поддържат въвеждането и извеждането на информация. Тази група съдържа *операции за вход* (**IN**) и *операции за изход* (**OUT**).

Държавен изпит – спец. „Софтуерно инженерство“

Тема №6 – Структура и йерархия на паметта. Сегментна и странична преадресация. Системи за прекъсване – приоритети и обслужване.

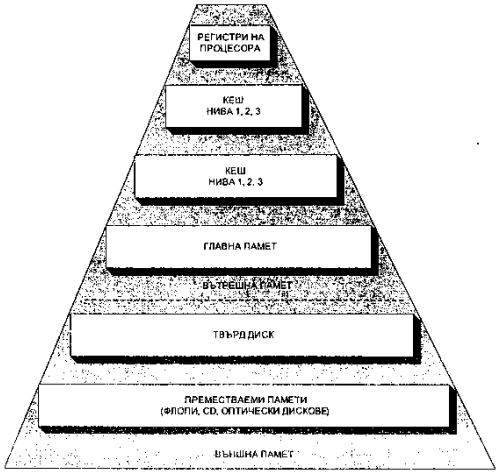
Анотация: *Йерархия на паметта – кеш-памет, оперативна памет и виртуална памет.*

Сегмента преадресация
- сегментен селектор;
- сегментен дескриптор;
- сегментни таблици и регистри.

Странична преадресация
- каталог на страниците;
- описател на страница;
- стратегии на подмяна на страниците.

Прекъсвания
- структура и обработка;
- типове прекъсвания;
- конкурентност и приоритети;
- обслужване на прекъсванията;
- контролери на прекъсванията.

Йерархия на паметта в компютърната система съдържа три нива, които се различават по типа, бързодействието и капацитета на паметта. На най-високото ниво на йерархията на компютърната памет са регистрите на процесора, следвани от кеш на няколко нива (първо ниво L1, второ ниво L2, трето ниво L3). Следва главната (оперативната) памет на компютъра, която представлява динамична полупроводникова памет с произволен достъп DRAM. Изброените нива включват вътрешната (първичната) памет на компютърната система. Следват нивата на външната (вторичната) памет на компютъра, включваща твърдия диск и едно или повече нива с преместваеми памети като гъвкав магнитен диск (флопи диск), компактен диск CD и др. Информацията във външната памет е структурирана във файлове и записи за разлика от думите и индивидуалните байтове в главната памет. Процесорът има директен достъп до вътрешната памет на компютъра, докато достъпът до външната памет се осъществява посредством входно-изходни модули. Външната памет осигурява разширение на главната памет, известно като виртуална памет.



Времето за достъп, както и капацитетът на най-високите нива в йерархията на паметта са най-малки, като по посока на ниските нива стойностите на тези характеристики се повишават, а цената за един бит спада. Оптималният случай е да се използват само най-бързите памети в компютъра, но поради високата им цена се налага да се използват нивата с по-бавните памети, при което съотношението цена/бит става приемливо. Основната идея е да се организира информационния трансфер по такъв начин, че изпълняваната програма и съответните данни да се поддържат в бързите памети на високите нива.

Основна характеристика на компютърната памет е нейния капацитет, отразяващ максималното количество на съхраняваната информация. Капацитетът на вътрешните памети се измерва в байтове или думи, докато капацитетът на външната памет се измерва обикновено в байтове, килобайтове, мегабайтове, гигабайтове и терабайтове. Основната адресируема информационна единица в паметта е думата, но много системи допускат и адресиране на индивидуалните байтове в рамките на думата. Информационната единица за трансфер за вътрешната памет се определя от броя на думите, които едновременно се четат или записват, докато за външната памет информационния обмен използва много по-големи информационни единици, наречени блокове.

Според метода за достъп паметите се разделят на памети с последователен, директен, произволен и асоциативен достъп. Последователният достъп следва модела на магнитната лента, директният достъп – модела на диска. При паметите с произволен достъп RAM (Random Access Memory) индивидуалните думи в паметта директно се четат или записват на съответния адрес. При асоциативния достъп селекцията на информационната единица се осъществява по признак (съдържанието), а не по адреса.

Друга основна характеристика е бързодействието на паметта. Тя се определя от три основни параметъра: време за достъп, цикъл на паметта и скоростта на трансфер.

При паметите с произволен достъп времето за достъп *се дефинира като интервала от време от момента на подаването на адреса към паметта до момента, в който данните са прочетени от или записани в паметта.*

Цикълът на паметта *се дефинира за памети с произволен достъп RAM и включва времето за достъп до паметта плюс допълнителното време, което трябва да измине, преди да се стартира друг достъп. Това допълнително време е необходимо за затихване на предходните процеси и евентуално регенериране на данните.*

Скоростта на трансфер *(пропускателната способност) се дефинира като скоростта, с която информацията се подава от или приема в паметта. За памети с произволен достъп RAM тя е един цикъл.*

При енергозависимите памети информацията се загубва при изключване на захранването. **При енергонезависимите памети** не е необходима електрическа енергия за поддържане на съхранената информация.

За да се осигури висока производителност на компютърната система, информационният обмен между различните нива на йерархията на паметта трябва да се организира по такъв начин, че *процесорът да се обръща най-често към високите (бързите) нива с малко време на достъп и по-рядко към ниските (бавни) нива с голямо време за достъп*. За съжаление обаче малкото

време за достъп на паметта определя и нейната висока цена, а следователно и малък капацитет в компютърната система.

От особено значение за оптималното организиране на информационния обмен между различните йерархични нива на компютърната памет е **принципът на локалността** на достъпите. По време на изпълнението на програмата достъпите на процесора до паметта имат тенденцията да се съсредоточават (клъстерират) в определена област на паметта. В общия случай програмите съдържат цикли и подпрограми. След като веднъж цикълът или подпрограмата са заредени в главната памет, следват повтарящи се достъпи до ограничено множество инструкции. Аналогично, при обработката на масиви от данни или таблици се осъществява достъп до клъстерирано множество от данни. В рамките на малък временен интервал процесорът използва фиксирани клъстери от достъпи до паметта, докато за голям временен интервал тези клъстери се променят.

Организацията на данните в йерархията на паметта поддържа нисък процент на достъпите до ниските нива и висок процент на достъпите до високите нива. В този ред на мисли кешът осигурява междинно ниво, прозрачно за програмиста, за информационния трансфер между регистрите на процесора и главната памет. От друга страна, част от главната памет може да се използва като буфер, съхраняващ временно данните, които подлежат на запис в диска. Тази техника, наричана кеш на диска (обикновено е чисто софтуерна техника), значително подобрява производителността на компютърната система.

Полупроводниковите памети са изградени от запомнящи елементи. Всеки запомнящ елемент има две основни състояния, представящи съответно двоичните стойности 0 и 1. Всеки чип памет съдържа матрица от запомнящи елементи, като информацията е организирана в определен брой думи, всяка от които е с фиксиран брой битове.

В постоянните памети ROM (Read Only Memory) информацията е записана още в процеса на производството и не подлежи на промяна. Поради серийното производство имат ниска цена и освен това са енергонезависими памети. Те се използват за съхраняване на микропрограми, библиотеки с подпрограми за често използвани функции, системни програми, таблици с функции и в общия случай информацията, която не подлежи на промяна в компютъра.

*В случаите, когато са необходими по-малко на брой чипове, се използват **програмируеми постоянни памети PROM** (Programmable ROM). Те също са енергонезависими и записът в тях също се осъществява еднократно, но от потребителя след процеса на производство. Записът на информацията в PROM се осъществява от специални устройства, наречени програматори.*

***Светкавичната (flash) памет** е полупроводникова памет с много висока скорост на препрограмиране. Използва се електрическа технология за изтриване, като цялата флаш памет може да бъде изтрита за много кратко време (няколко секунди). Тя представлява преместваема памет и може да се използва многократно.*

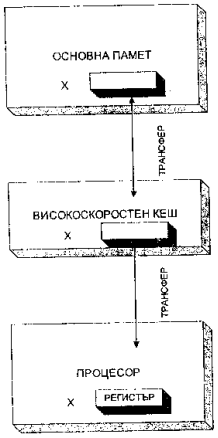
При паметите с произволен достъп RAM (Random Access Memory) индивидуалните думи в паметта директно се четат или записват на съответния адрес. Тя е енергозависима, като при прекъсване на захранването информацията се изгубва. Следователно, паметта RAM може да съхранява временно информацията.

Технологията на RAM може да бъде **статична** или **динамична**. При статичните RAM памети битовете се съхраняват в тригери, т.е. те имат регистрова структура. В общия случай статичните RAM памети са по-бързи от динамичните, но за сметка на това са по-скъпи и имат по-ниска плътност на компонентите (чиповете са с по-ниска степен на интеграция). Динамичните DRAM памети допускат висока плътност на компонентите, което означава, че те осигуряват по-голям капацитет на паметта в рамките на един чип. Освен това имат и по-ниска цена и следователно са предпочитани за имплементиране на главната памет на компютрите.

Динамичните DRAM памети са изградени от запомнящи елементи, съхраняващи информацията под формата на капацитивни заряди. Наличието на капацитивен заряд или отсъствието на такъв се интерпретира като съдържание на бита – съответно двоична 1 или 0. Поради утечка тези капацитивни заряди постепенно се разреждат, така че се налага периодически да се опреснява информацията в паметта. Това опресняване (refresh) се изразява в четене и последващ запис на всяка от позициите в паметта.

Поради изброените предимства динамичната памет DRAM е предпочитана за изграждане на главната памет на компютъра. Селекцията на основна информационна единица в паметта се осъществява посредством селекцията на реда и на колона в квадратния масив от запомнящи елементи. Адресните изводи на чипа памет са мултиплексирани, което означава, че по тях първо се подава адресът на реса, а после – адресът на колоната. Предаването на адреса на реда се индицира с активен сигнал RAS (Row Adress Select), докато предаването на адреса на колоната се индицира с активен сигнал CAS (Colum Adress Select).

Процесорът чете инструкции и операнди от паметта и записва резултатите от изпълнението на инструкциите в нея. Бързодействието на процесора обаче е с един порядък и повече по-голямо от това на паметта. За повишаване на производителността на компютъра се прибавя междинно ниво между процесора и основната памет – **кеш** с малък капацитет и бързодействие, съизмеримо с това на процесора. Думата *cache* означава скрита памет. Функционирането на кеша е прозрачно за потребителя. Инструкциите и данните се прехвърлят от основната памет в кеша, като впоследствие се ползват от процесора. В общия случай програмите и данните се съхраняват в главната памет, така че съдържанието на позициите в кеша са копия на съдържанието на определени клетки от главната памет. Следователно, достъпът до информацията в кеша трябва да се осъществява чрез същите адреси, на които данните са разположени в главната памет. Когато процесорът извършва запис, данните се записват в кеша и едновременно с това в главната памет. Алтернативата е да се запишат данните в кеша, а в главната памет по-късно, когато използваната позиция в кеша трябва да бъде използвана за запис на нови данни, тъй като кешът има ограничен капацитет и може да съдържа копия само на ограничен брой клетки от паметта.

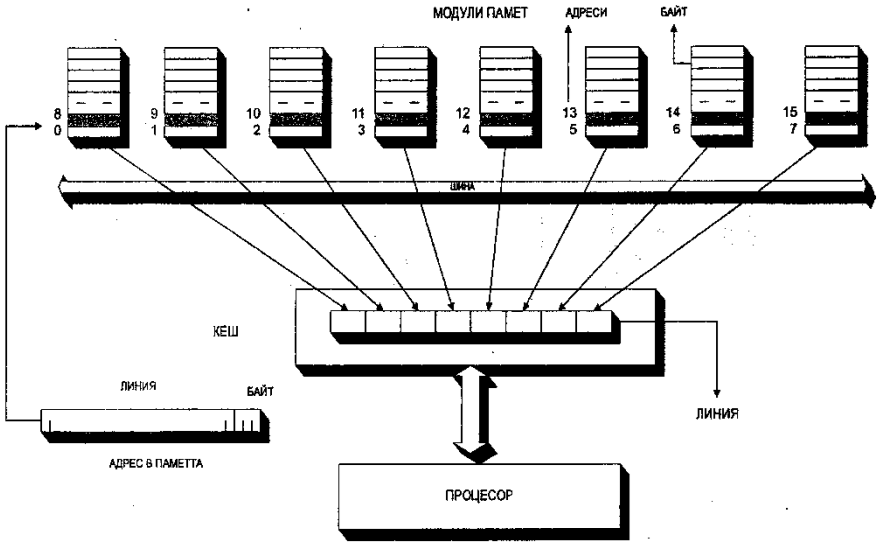


Успехът на използването на кеша зависи от характеристиките на програмата и по-точно от нейната степен на локалност. **Принципът на локалността** се отнася както до обръщението към инструкциите, така и до обръщението към данните и може да се разглежда в два аспекта:

- **Локалност във времето** – при нея има голяма вероятност обръщенията към определени позиции в паметта да се повтарят в близките бъдещи моменти;
- **Локалност в пространството** – следващата позиция в паметта, към която ще се извърши обръщението е най-вероятно да бъде близко до позицията, към която е осъществено последното обръщане.

Локалността по време е актуална при наличието на цикли, които съдържат предимно малко на брой инструкции. Следователно, след като инструкциите в цикъла са заредени веднъж в кеша, те ще бъдат четени многократно, докато се наложи извличането на нови инструкции от паметта (след излизането от цикъла). Аналогична е и ситуацията с данните.

За да се използва локалността в пространството, информационният трансфер между кеша и главната памет включва не дума, а последователност от клетки, наричана **линия** или **блок**. За оптимална производителност линията се предава между главната памет и кеша по широка шина за данни. По този начин се поддържа съответствието между времето за достъп на главната памет и времето за достъп на кеша.



Когато процесорът чете от паметта, първо се извършва обръщение към кеша. Ако търсената информация се съдържа в кеша, то тя се изпраща директно към процесора (попадение – **hit**). Когато търсената информация не се съдържа в кеша (липса – **miss**), се налага обръщение към главната памет за прехвърляне на данните в кеша, след което се налага отново процесорът да се обърне към кеша. Ситуациите при операциите запис в кеша са по-сложни.

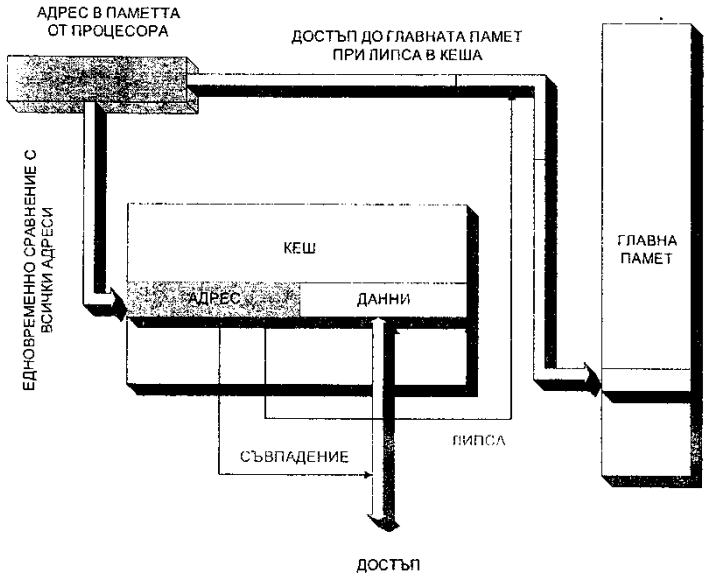
Вероятността необходимата информация да се съдържа в кеша зависи от поведението на програмата и от размера и организацията на кеша.

Практически времето за достъп се измерва в брой (цяло число) машинни цикли, като времетраенето на машинния цикъл е свързано с тактовата честота на системата. Скоростта на попадение се дефинира като отношението на попаденията в кеша към общия брой обръщения към кеша. Скоростта на попаденията определя също вероятността дадена дума да бъде в кеша. Скоростта на възникване на липси в кеша се дефинира като единица минус скоростта на попаденията.

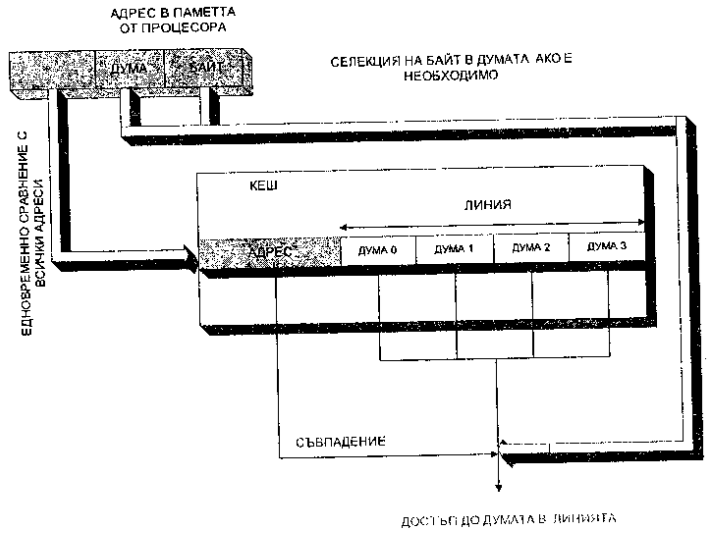
Организация на кеша

Процесорът се обръща към кеша чрез адреса, по който се обръща и към главната памет. Следователно, организацията на кеша трябва да използва този адрес, за да открие данните при попадение или да сигнализира на процесора, че е възникнала липса в кеша.

Кеш с пълна асоциативна организация. Най-логичният начин за организирането на кеша е в него да се съхраняват данните и техният адрес в паметта. В този случай кешът е с асоциативна организация. Такъв кеш се състои от асоциативна памет, съдържаща адреса в паметта и данните, съхраняващи се на този адрес. Адресът, с който се извършва достъпа, се сравнява от вътрешната логика едновременно с всички адреси в кеша, както е показано на фигурата. Ако се открие съвпадение, то съответните данни се четат.

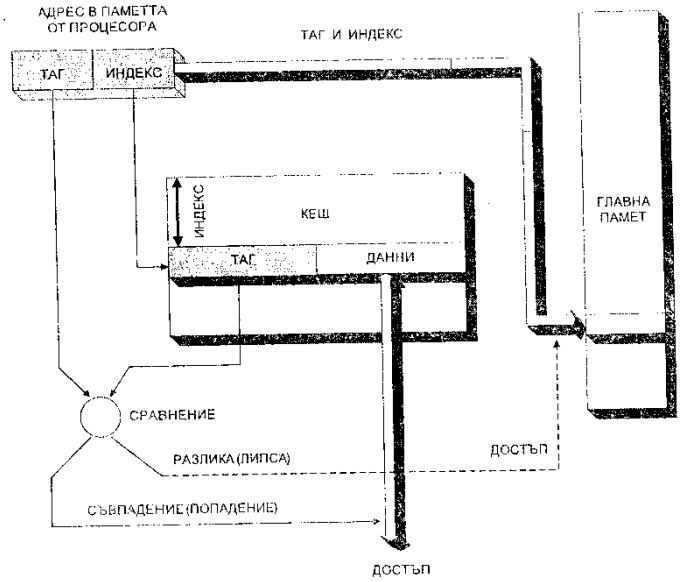


При всички разновидности на организацията на кеша, той съдържа линии с последователността от няколко думи, за да може да се използва локалността в пространството. Цялата линия се обменя между кеша и главната памет в рамките на една транзакция, разбира се ако ширината на шината за данни между кеша и главната памет позволяват това.



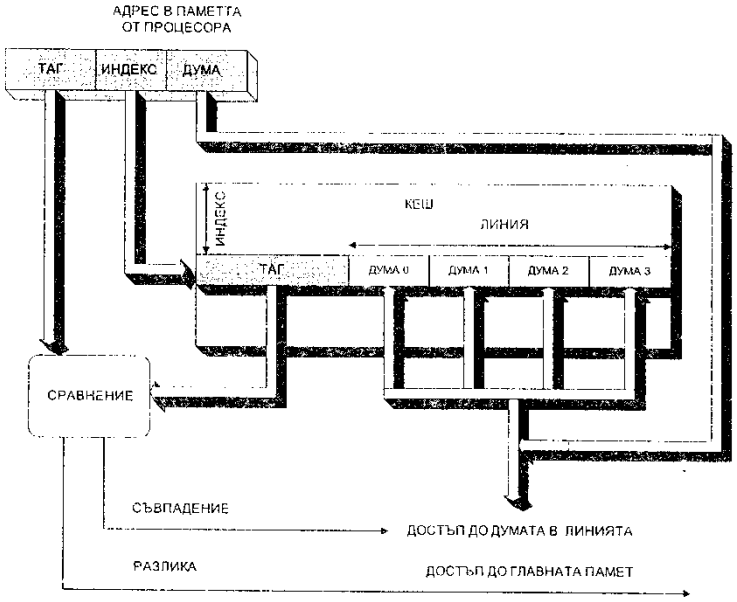
Кешът с пълна асоциативна организация осигурява най-голяма гъвкавост и минимален брой конфликти за фиксиран размер на кеша, но той се отличава с висока цена поради използваната асоциативна памет. Тя изисква алгоритъм за заместване на линия при липса в кеша, който трябва да бъде заложен хардуерно, за да се осигури високо бързодействие. Кешът с пълна асоциативна организация се имплементира само с малък капацитет.

Кеш с директна организация. Той съдържа памет с произволен достъп RAM, като всяка позиция съдържа данни, адресът на които представлява най-младшата част от адреса в главната памет. Линията се избира чрез най-младшата част от адреса в главната памет. Старшата част на адреса се съхранява заедно с данните, за да се завърши идентификацията на данните.



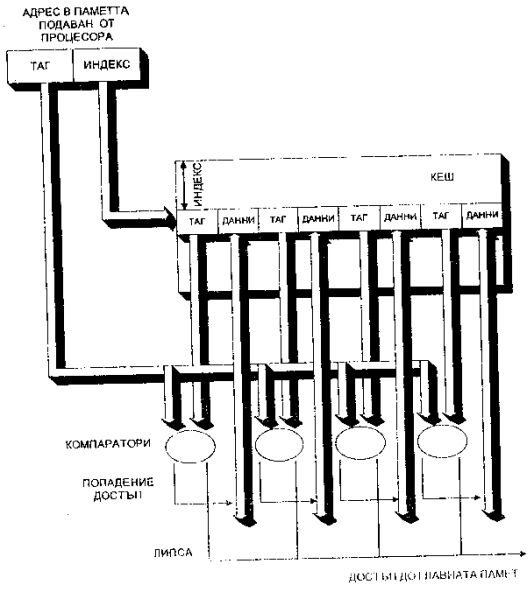
Адресът, подаван от процесора, се разделя на две полета: **таг** и **индекс**. В тага се включват най-старшите битове на адреса, които се съхраняват заедно с данните. Индексът включва младшите битове на адреса, използвани за адресиране на кеша. При обръщение към главната памет индексът се използва за достъп до съответната дума в кеша. След това, тагът, съхраняван заедно с думата, се сравнява с тага на подавания адрес. При съвпадение думата се чете от кеша. При липса (различни тагове) се извършва достъп до главната памет за четене на търсената дума. Тя се зарежда в кеша, след което процесорът я чете от кеша. Възможно е при липса прочетената дума от главната памет да се предава едновременно на кеша и на процесора. Аналогично, при запис на нова дума в кеша тя може да бъде записана едновременно и в главната памет.

Когато в кеша с директната организация е с линия, включваща множество думи, адресът в главната памет включва таг, индекс на линията и номер на думата в рамките на линията. Всички думи в рамките на линията споделят общ таг. Индексът се използва за достъп до съответната позиция в кеша, след което таговете се равняват. При съвпадение съответната дума се избира в линията и се изпраща към процесора. При несъвпадение (липса) се извършва достъп до главната памет.



Кешът с директна организация има проста апаратна реализация и високо бързодействие. Основен недостатък е, че производителността пада значително при серия от последователни обръщения на процесора към клетки с едни и същи индекси.

Кеш с асоциативна организация с множества. Той съдържа ограничен брой линии с еднакви индекси и различни тагове. Затова се счита като компромисно решение между кеша с напълно асоциативна организация и кеша с директна организация. Кешът е разделен на множества от линии. Кеш, който съдържа 4 линии във всяко множество определя асоциативността на кеша или размера на множеството. Всяка линия във всяко множество съдържа таг, който заедно с индекса (номера на множеството) идентифицира линията. Първо, индексът от адреса, подаван от процесора, се използва за достъп до множеството. После всички тагове на избраното множество се сравняват в компаратори с подавания таг. При съвпадение се осъществява достъп до избраната линия, в противен случай (при липса) се извършва достъп до главната памет. В екстремалния случай, когато размерът на множеството е една линия, кешът става с директна организация. В другия екстремален случай, когато кешът съдържа само едно множество, кешът става напълно асоциативен.



Запис в кеша

При операциите *четене* съдържанието на кеша не се променя, така че не съществува опасност от разлики в съдържанието на кеша и на главната памет. При операциите за *запис* в кеша обаче съществува възможност съдържанието на клетки в паметта да се различава от техните копия в кеша, т.е. възниква проблема за кохерентност на кеша. Този проблем е актуален при реализацията на входно-изходен трансфер на съдържанието на клетки от главната памет, както и при мултипроцесори с обща памет.

При механизма със **запис през кеша** (*write-through*) всяка операция за запис в кеша едновременно се повтаря за главната памет, т.е. новите данни се записват едновременно в кеша и в главната памет. Времето за запис в главната памет е значително по-голямо от това в кеша, така че времето за запис ще се определя от времето в главната памет. За да се облекчи този проблем, се използват буфери за запис, които съхраняват временно информацията, за да може да се освободи кешът. Непосредственото записване в паметта на актуализираната информация спестява използването на механизми за разрешаването на проблема за кохерентност на кеша, като гарантира актуалността на информацията за всяко устройство, осигуряващо достъп до главната памет.

При механизма със **запис при връщане на линията от кеша в основната памет** (*write-back*) в главната памет се записват линиите, които подлежат на заместване с нови и в които е извършван запис, т.е. тяхното съдържание е било променено. Заместени линии, чието съдържание не е било актуализирано, не се записват обратно в главната памет. Когато се налага трансфер на нова информация в кеша при липса и кешът е пълен, се налага някои от линиите да бъдат заместени с нови.

Алгоритъмът за замества се реализира изцяло хардуерно и е желателно определянето на линията, която ще бъде заместена, да се вмести в рамките на цикъла на паметта за зареждане на новата линия. Най-често използваните алгоритми за заместване са:

- *избор на произволна линия за заместване;*
- *замества се линия, която е престояла най-дълго в кеша;*
- *замества се линия, която най-дълго време не е била използвана.*

Нива на кеш-паметта

Главната памет на компютъра съхранява програмите и данните. Кешът може да бъде организиран по същия начин, т.е. да съхранява инструкции и данни. В този случай той се нарича *унифициран кеш*. Разделянето на кеша на кеш за инструкции и кеш за данни осигурява следните предимства:

- стратегията за запис се прилага само в кеша за данни (като се приеме, че кодът не е самомодифициращ се, т.е. инструкциите не се променят);
- отделни шини осигуряват свързването на двата кеша с процесора, което дава възможност за едновременен трансфер на инструкции и на данни (от особено голямо значение за процесори с конвейерна обработка);
- двата кеша могат да имат различен капацитет, различна организация и различна дължина на линията.

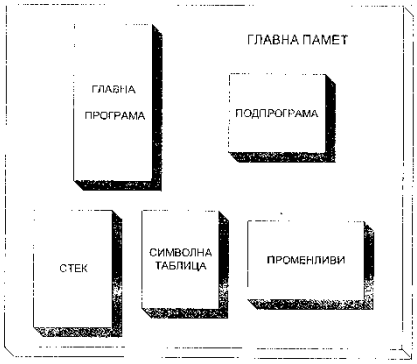
По този начин се изграждат **кешове от първо ниво L1**, които се вграждат в чипа на процесора. Това ограничава капацитета на кеша, но във всички случаи увеличаването на капацитета на кеша намалява неговото бързодействие. Популярен подход е да се осигури втори кеш между главната памет и кеша от първо ниво. Той е със значително по-голям капацитет от кеша на първо ниво. При достъп до паметта процесорът първо се обръща към кеша от първо ниво. При липса процесорът се обръща към кеша от второ ниво. В случай, че отново възникне липса и в кеша от второ ниво, се извършва достъп до главната памет. Съдържанието на клетките в главната памет се предава първо към кеша от второ ниво, и после към кеша от първо ниво. Всяка кеширана клетка на главната памет има две копия – едно в кеша от второ ниво и едно в кеша от първо ниво, т.е. се изпълнява принципът на включването. Най-често използваната стратегия за заместване на линии в кеша и на двете нива е *заместване на линия, която най-дълго не е била използвана* (LRU – Least Recently Used). За запис на информация между двата кеша се използва запис през кеша, така че копията и на двете нива да са еднакви.

Бързодействието на магнитните дискове е значително по-малко (обикновено с няколко порядъка) от бързодействието на главната памет. Разликата се увеличава с напредъка на полупроводниковите технологии, тъй като времето за достъп при диска и скоростта на трансфер се ограничава от механични фактори. Често процесорът не може да продължи изпълнението на текущата програма, докато се реализира трансфер между диска и главната памет, дори когато този режим се осъществява от контролер за директен достъп до паметта без участието на процесора.

За намаление на негативните ефекти от голямото закъснение на дисковата памет се използва концепцията за използване на кеш на диска в интерфейса главна памет/диск.

Сегментиране на паметта

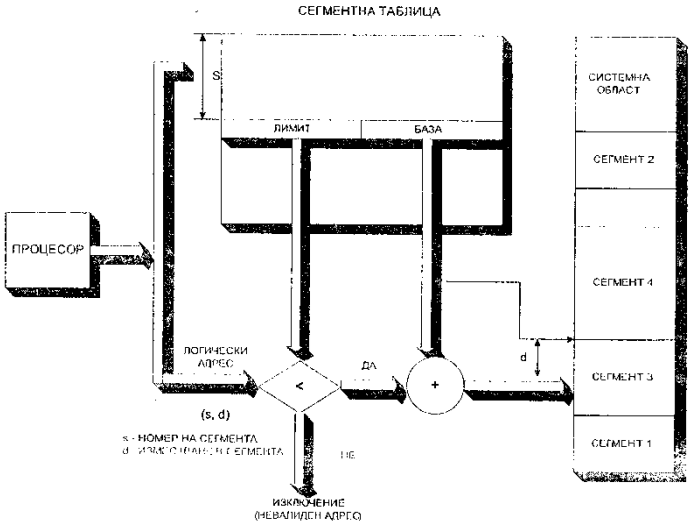
Концептуален модел. Сегментирането е схема за управление на паметта, която е видима за програмиста и му осигурява удобство при организирането на програмата и данните. Сегментирането дава възможност на програмиста да разглежда паметта като колекция от множество адресни пространства или сегменти. Сегментите са дялове от паметта с различен размер и съдържат семантично дефинирана част от програмата. Следователно, логическото адресно пространство може да се разглежда като колекция от сегменти.



Всеки сегмент има *име* и *размер*. Елементите в рамките на сегмента се идентифицират чрез тяхното изместване спрямо началото на сегмента. Всеки адрес задава името на сегмента и изместването в него. Практически в процеса на компилиране на програмата компилаторът автоматично конструира сегментите в зависимост от спецификата на обработваната програма.

За простота сегментите се номерират и обръщението към тях се осъществява чрез техните номера. Така например компилаторът създава отделни сегменти за глобалните променливи, стека за извикване на процедури, кода на всяка процедура и функция и за локалните променливи на всяка процедура и функция.

Очевидно при сегментирането потребителят има възможност да се обръща към обектите на програмата посредством двумерен адрес, въпреки че адресното пространство на компютърната система фактически представлява едномерна (линейна) последователност от думи. Операционната система трябва да трансформира двумерните адреси, зададени от потребителя, в едномерни физически адреси. При сегментирането всеки логически адрес съдържа номер на сегмента s и изместването d в рамките на дефинирания сегмент. Номерът на сегмента се използва като индекс в **таблицата на сегментите**. За всеки сегмент в сегментната таблица се съдържат базата (началния адрес) на сегмента и лимитът (размерът) на сегмента. Изместването d в логическия адрес може да варира от 0 до лимита на сегмента. Физическият адрес при сегментна преадресация в главната памет се формира като към базата на сегмента се прибави изместването.



За да се намалят допълнителните системни разходи за преобразуването на адресите, препоръчително е таблицата на сегментите да се имплементира с **високоскоростни регистри**, към които обръщението се извършва много бързо. За всеки сегмент в таблицата на сегментите се съдържа съответно неговата база, размер и поле за достъп, дефиниращо правата за достъп до сегмента, например право на изпълнение за програмните сегменти и право на четене/запис за сегментите с данни. Използването на асоциативни регистри (кеш) ускорява процеса на преобразуване на адреса.

Сегментирането улеснява механизмите за защита. Сегментите могат да съдържат само инструкции или само данни поради семантичната дефинираност на тяхното съдържание. Програмните сегменти са немодифициращи се, така че тези сегменти могат да бъдат дефинирани като сегменти само за четене или само за изпълнение. Общото ползване на кода и данните също се поддържа особено добре от схемата за сегментиране на главната памет. Общите сегменти се представят в сегментната таблица чрез указатели към едни и същи физически области от паметта. Общото ползване (sharing) се осъществява на сегментно ниво. Броят на общите сегменти не е ограничен. Възможно е също така да се използват общо и само

отделни части от програмата. Сегментирането опростява поддържането на динамични структури от данни, тъй като те се съдържат в отделен сегмент, който операционната система може да разшири или стесни в зависимост от текущите нужди.

Недостатъкът на сегментирането е, че при него възниква външна фрагментация. Тя се дължи на различните размери на сегментите. При връщането на сегментите на диска се освобождават дупки, като след известно време паметта започва да прилича на шахматна дъска, т.е. тя е изпълнена с разпръснати дупки. Зареждането на нов сегмент в дупката може да стане само ако размерът на сегмента е по-малък от сегмента на дупката. При това паметта се изпълва с малки и разпръснати дупки, които не могат да осигурят разполагането на цял сегмент в някоя от тях, тъй като това изисква непрекъсната област.

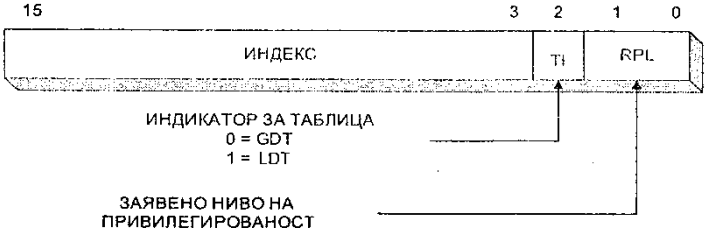
Средствата за управление на паметта при някои архитектури поддържат сегментирането като механизъм за изолиране на индивидуалните модули код, данни и стек на различните програми (или задачи) по такъв начин, че те да бъдат изпълнявани от процесора, без да си пречат. В защитен режим не е възможно да се забрани сегментирането.

Сегментирането осигурява възможност за разделяне на адресното пространство (наречено линейно адресно пространство) на по-малки адресни пространства, наречени сегменти. Ако в даден момент повече от една програма (или задача) се изпълнява от процесора, всяка програма разполага със собствено множество сегменти. Процесорът налага граници между тези сегменти по такъв начин, че изпълнението на една програма да не попречи на останалите, като записва в техните сегменти. Всички сегменти в системата се съдържат в линейното пространство на процесора. Физическото адресно пространство се дефинира като обхвата от адреси, които процесорът може да генерира по адресната шина. То се разпределя между RAM, ROM и входно-изходни адреси. Линейното адресно пространство съдържа всички сегменти и системни таблици.

Сегментният селектор представлява 16-битов идентификатор на сегмента. Той не указва директно сегмента, а вместо това указва дескриптора (описателя) на сегмента, който го дефинира. Сегментният селектор съдържа:

- индекс (битове от 3 до 15), който избира един от многото дескриптора в глобалната дескрипторна таблица GDT или в локалната дескрипторна таблица LDT;
- флаг-идентификатор на таблицата TI – при 0 избира GDT, при 1 – LDT;
- заявено ниво на привилегированост RPL – определя нивото на привилегированост на селектора. То може да бъде в обхвата от 0 до 3, като ниво 0 е най-високото привилегировано ниво.

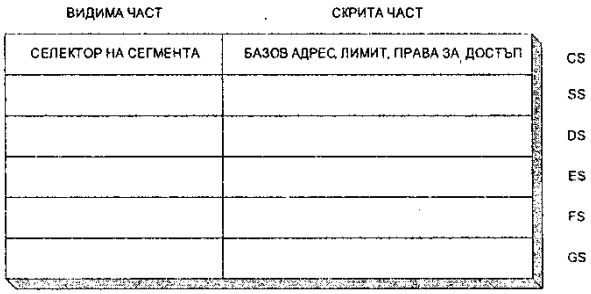
Сегментните селектори са видими за приложния програмист като част от указател, но те се присвояват от свързващия редактор, а не от приложните програмисти.



За да се намали времето за преобразуване на адреса и сложността на кода, процесорът съдържа регистри, които могат да съхраняват до 6 сегментни селектора. За да може да се изпълни една програма, поне регистрите на програмния стек (CS), сегмента за данни (DS) и сегмента на стека (SS) трябва да бъдат заредени с валидни сегментни селектори. Процесорът също осигурява три регистъра за допълнителни сегменти за данни (ES, FS, GS), които могат да бъдат използвани от текущата програма.

За да може програмата да осъществи достъп до сегмент, съответният сегментен селектор трябва да се зареди в един от сегментните регистри. Въпреки че в системата могат да бъдат дефинирани много сегменти, в даден момент могат да бъдат използвани само 6 сегмента. Използването на другите сегменти може да се осъществи, като техните селектори се заредят в сегментните регистри по време на изпълнение на програмата.

Всеки сегментен регистър има „видима“ част и „скрита“ част. Скритата част се нарича още „кеш на дескрипторите“ или „регистър в сянка“.



Когато сегментният селектор се зареди във видимата част на регистъра, процесорът зарежда също скритата част на сегментния регистър с базовия адрес, лимита и информация за управлението на достъпа до сегмента, които се извличат от сегментния дескриптор, указан от селектора. Локалната таблица на дескрипторите LDT съдържа описателите на сегментите, които се използват само от програмата, за която са дефинирани. Глобалната таблица на дескрипторите GDT съдържа описателите на сегментите, които се използват общо от програмите в системата. Информацията, кеширана в сегментния регистър (видима и скрита), дава възможност на процесора да преобразува адресите, без да използва допълнителни цикли по шината за четене на базовия адрес и лимита от дескриптора.

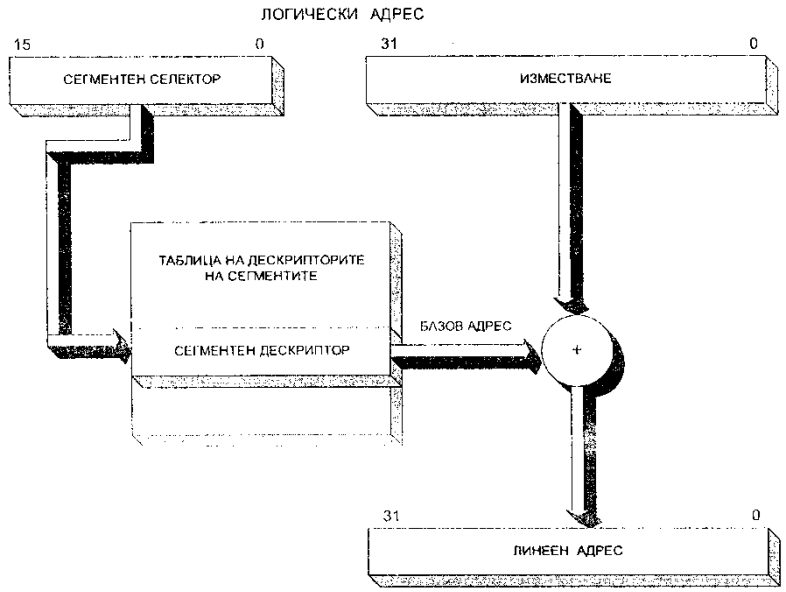
Дескрипторът на сегмента представлява структура данни в глобалната или локалната таблица на дескрипторите, която съдържа информация за размера и местоположението на сегмента, неговия статус и права за достъп. Дескрипторите на сегментите се създават от компилаторите, свързващите редактори, зареждащите програми или от операционната система, но не от приложните програми.

Форматът на дескриптора съдържа следните полета:

- поле на лимита – определя размера на сегмента. Процесорът интерпретира съдържанието на това поле по два начина в зависимост от състоянието на флага за грануларитет G
- поле на базовия адрес – задава адреса на нулевия байт на сегмента в линейното адресно пространство

- поле за типа на сегмента – идентифицира типа на сегмента, разрешените видове за достъп и посоката на нарастване на сегмента (към големите или малките адреси). Кодирането на това поле е различно за дескрипторите на сегментите на кода, данните и системните дескриптори;
- флаг S – показва дали сегментът е системен, сегмент за код или данни;
- поле за ниво на привилегированост DPL – определя нивото на привилегированост на сегмента (от 0 до 3);
- флаг P за наличност – показва дали сегментът присъства в паметта (P = 1) или не присъства в паметта (P = 0);
- поле за параметри по премълчаване DB – този флаг винаги се установява в 1 за 32-битови сегменти за кода и данните и се нулира за 16-битови сегменти за кода и данните;
- флаг за грануляритет G – определя мащаба на полето за лимита на сегмента;
- поле за наличен и резервиран битове AVL

За преобразуването на логическия адрес в линеен процесорът използва изместването в сегментния селектор, за да осъществи достъп до сегментния дескриптор в таблицата на дескрипторите GDT или LDT, проверява правата за достъп и лимита в дескриптора, за да установи дали сегментът е достъпен и изместването е в границите на лимита. Прибавя базовия адрес на сегмента от дескриптора към изместването, за да формира линейни адрес. Ако не се използва странична организация на паметта, линейният адрес съвпада с физическия адрес и процесорът директно го подава по адресната си шина.



При мултисегментния модел всяка програма (или задача) разполага със собствени сегменти и собствена таблица на сегментните регистри. Сегментите могат да се използват само от програмата или да бъдат използвани общо между програмите. Достъпът до сегментите и изпълнителните среди от изпълняваните програми в системата се управлява от хардуера.

Проверките за достъп се използват не само за защита при обръщение към адрес извън лимита на сегмента, а също така и за предотвратяване на забранени операции в определени сегменти. Например, сегментите с кода се дефинират с права само за четене и хардуерът предотвратява запис в този тип сегменти.

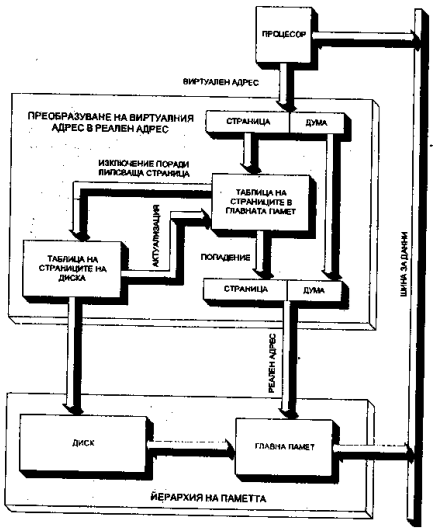
Странична преадресация

През 1962 г. Килбърн от Университета в Манчестър предлага метод за управление на йерархията на компютърната памет като памет с едно ниво, който понастоящем се нарича **страниране** (странична организация) на паметта. Целта на този метод е управлението на главната памет и на дисковата памет да се организира по такъв начин, че за потребителя да изглежда, че в компютъра съществува само RAM памет, откъдето и произлиза терминът памет на едно ниво.

Виртуалната памет осигурява логическо адресно пространство за потребителя, многократно превишаващо физическото адресно пространство на компютърната система (което фактически е скрито за потребителя). За програмиста се създава илюзията, че компютърът разполага с главна памет с изключително голям капацитет. Отделни адреси се използват за виртуалното и реалното адресно пространство. Програмата генерира виртуални адреси, докато реалните адреси в компютърната система са физическите адреси на главната памет.

Както реалното, така и виртуалното адресно пространство са разделени на блокове от думи с фиксиран размер, наречени страници. Всички страници имат един и същи размер, който зависи от компютърната архитектура. Както виртуалният, така и реалният адрес са разделени на две полета – страница и линия (дума в рамките на страницата). Процесорът генерира програмно зависими виртуални адреси, при които се приема, че цялата компютърна памет може да бъде директно адресирана. Във всеки момент всеки виртуален адрес има съответен физически адрес във физическата памет на компютъра (в главната памет или във вторичната /външната/ памет). Съответствието между виртуалните и реалните страници се съдържа в таблицата на страниците. Всеки виртуален адрес, генериран от процесора, се преобразува в реален адрес в главната памет чрез обръщение към таблицата на страниците. Ако в даден момент страницата, към която процесорът се обръща, не присъства в главната памет, възниква изключение поради липсваща страница, при обслужването на което се активира подпрограма, която автоматически зарежда липсващата страница в главната памет и съответно актуализира съдържанието на таблицата на страниците.

На следващата фигура е представен обобщен модел на компютърна система с виртуална памет. На преобразуване подлежи само полето на страницата в адреса, тъй като полетата на линията са еднакви във виртуалния и физическия адрес. Преобразуването на виртуалния адрес във



физически (странична преадресация) се осъществява задължително при всяко обръщане към паметта.

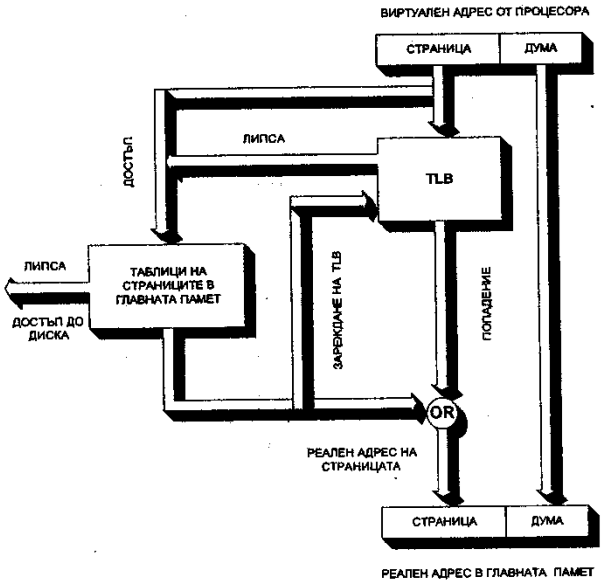
Успехът на странирането на паметта в значителна степен зависи от принципа на локалност (по време и пространство) на изпълняваните програми. Обръщенията към паметта се групират в отделни области, като към някои клетки от тези области се реализират многократни обръщения. В общия случай голяма област се използва за главната програма, една област се използва за стека, съдържащ адресите на връщане и параметрите на процедурите, друга област съхранява локалните променливи, които обикновено се съхраняват заедно, извикваните процедури се съхраняват в отделни области, особено ако се ползват общо с други програми.

Броят на страниците в съвременните компютърни системи е твърде голям, за да може всички страници да се поддържат само в една високоскоростна таблица на страниците. Като се имат предвид характеристиките на изпълняваната програма, съответстващи на принципа на локалността естествено е да се очаква, че определено множество от страници, наречено работно множество, ще бъде многократно използвано до превключването на контекста. Следователно най-вероятно е адресите на работното множество да се налага да бъдат преобразувани от хардуера. Естествено е работното множество като най-вероятното множество страници, към което ще се извърши обръщение, да се поддържа във високоскоростната таблица на страниците, наречена буфер за преобразуване (TLB – Translation Lookaside Buffer), каталожна таблица за преобразуване (DLT – Directory Lookaside Table), или кеш за преобразуване на адреса (ATC – Address Translation Cache).

За съхранение и поддържане на таблиците на страниците се използват асоциативни кешове или асоциативни кешове с множества.

Пълната таблица на страниците, която дава съответствието за всички страници в главната памет, има значително голям размер при 32-битовия адрес на съвременните процесори. За да се преодолее този проблем се използва преобразуване на адресите на две нива. В случая адресът съдържа три полета: каталог, страница, дума. Полето с каталога указва началния адрес на таблицата на страниците. В таблицата на страниците се открива началния адрес на страницата, към който се присъединява адресът на думата за формирането на реалния адрес. Недостатъкът на механизма за преобразуване на две нива е, че изисква повече таблици и две обръщения към таблиците, за да се извлече адресът на страницата. Очевидното му предимство обаче е, че индивидуалните таблици имат много по-малък размер и заемат по-малко памет. Множеството от таблиците, които се били използвани последни, се съхранява в буфера за преобразуване. Системата за преобразуване може да бъде разширена да съдържа повече от две нива, като се формира дървовидна структура от указатели. Такива структури улесняват работата с таблиците и редуцират пълната таблица със страниците, които не се използват.

При липса на страница, към която процесорът се обръща, възниква изключение, като операционната система е отговорна за зареждането на страницата от диска. Местоположението на търсената страница в диска се открива чрез таблицата на страниците в диска. Когато в главната памет няма свободно място за новата страница, е необходимо да се избере някоя от страниците в главната памет за отстраняване. Информационният трансфер между диска и главната памет е относително бавен и се реализира от контролер за директен достъп до паметта или от входно-изходен процесор. За да не престоява, процесорът може да избере друг процес за изпълнение, докато завърши трансфера. Двупосочният информационен трансфер между главната памет и диска (например разменянето на страници) се нарича размяна (swapping).



Популярната стратегия при съвременните процесори за зареждане на страници от диска в главната памет е „при поискване“ (demand paging). Дадена страница се зарежда от диска в главната памет, само ако процесорът се обръща към нея, при което възниква изключение за липсваща страница.

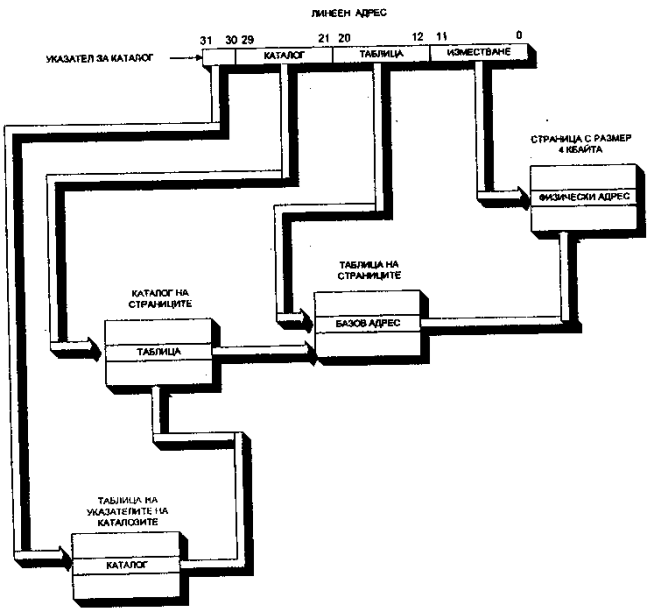
Най-често използваната стратегия за заместване на страници е да се отстранява тази страница, към която най-дълго време не с имало обръщение (LRU – Least Recently Used). При нея се поддържа опашка от страниците и когато възникне обръщение към дадена страница, тя се изважда от опашката и отново се прибавя към края на опашката. При необходимост от заместване на страница се избира страницата в началото на опашката.

В съвременните компютърни системи се използва често т. нар. псевдо-LRU алгоритъм, който апроксимира оригиналната LRU-стратегия. При нея се използват т.нар. битове за използване на страницата. На фиксирани интервали от време, сигнализирани от системния таймер за прекъсване, всички битове за използване на страниците се прочитат от операционната система и след това се нулират. Записаният брой на единичните състояния на бита за използване дава приблизителна представа за честотата на използването на дадена страница в рамките на определен интервал от време. Колкото е по-малък интервалът, толкова приблизителната оценка се доближава до оценката на LRU- алгоритъма

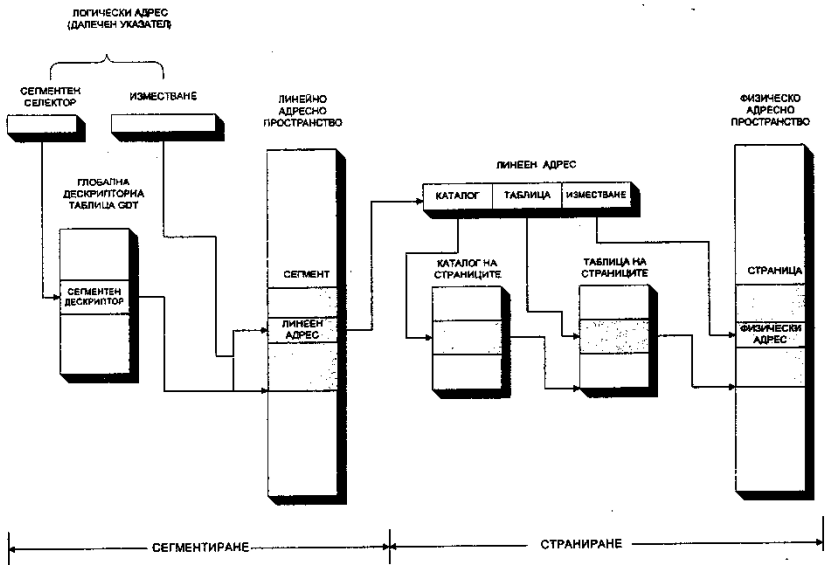
Средствата за управление на паметта при повечето архитектури са разделени на две части: сегментиране и страниране. В защитен режим архитектурата поддържа странична организация на паметта, като се използва механизъм за зареждане на страниците в главната памет при поискване. При него се разчита на допълнителни таблици с указателите на каталозите на таблиците на страниците. Информацията, която се използва от процесора за преобразуването на линейния адрес във физически, се съдържа в 4 структури данни:

- каталог на страниците – може да съдържа до 1024 позиции с каталози на страниците
- таблица на страниците – може да съдържа до 1024 позиции с таблици на страниците
- страница – 4 Kbytes, 2 Kbytes или 4 Mbytes адресно пространство
- таблица с указателите на каталозите, която съдържа 64 позиции с указатели на каталози

На следващата фигура е показана йерархията на **указателите на каталога на страниците**, **каталогът на страниците** и **таблицата на страниците** при преобразуването на линейния адрес във физически адрес. Линейният адрес е разделен на четири секции: таблица на указателите на каталозите, таблица на каталозите, таблица на страниците и изместване в рамките на страницата. Битовете 30 и 31 съдържат изместване за една от четирите позиции в таблиците на указателите на каталозите. Избраната позиция съдържа базовия физически адрес на каталога на страниците. Битове от 21 до 29 съдържат изместване към позиция в избрания каталог. Избраната позиция съдържа базовия физически адрес на таблицата на страниците. Битове от 12 до 20 съдържат изместване към позиция в избраната таблица на страниците. Избраната позиция съдържа базовия адрес физически адрес на страницата във физическата памет. Битовете от 0 до 11 съдържат изместването в рамките на страницата.



Комбинираното използване на сегментиране и странична организация осигурява поддръжка на различни подходи за управлението на паметта. Един от ефективните подходи за комбиниране, който опростява софтуера за управление на паметта, е всеки сегмент да разполага със собствена таблица на страниците.



Преобразуването на адреса при комбиниране на сегмента със странична организация включва два етапа: преобразуване на логическия адрес в линеен адрес (**сегментна преадресация**) и преобразуване на линейния адрес във физически адрес (**странична преадресация**) в главната памет.

Прекъсвания

Изключенията и прекъсванията са събития, които показват, че съществува условие в системата, процесора или в текущата програма или задача, което изисква вниманието на процесора. В резултат изпълнението се предава от текущата програма или задача към специална подпрограма или задача, наричана подпрограма, обслужваща прекъсването или изключението.

Прекъсванията могат да възникнат в произволни моменти от изпълнението на дадена програма в резултат на получавани сигнали от хардуера. Системният хардуер използва прекъсвания, за да обслужва събития, външни за процесора, като например заявки за обслужване на периферни устройства. Софтуерът също може да генерира прекъсвания чрез използването на инструкцията за програмно прекъсване INT n.

Изключения възникват, когато процесорът открие грешка при изпълнението на дадена инструкция, като например деление на нула, липсващи страници в паметта, неразрешен достъп и др. освен грешки в процесора, изключения възникват и при грешки по шината. Прекъсванията и изключенията се обработват прозрачно за приложните програми и операционната система. При получаване на сигнал за прекъсване или откриване на изключение изпълнението на текущата програма временно се преустановява, докато процесорът изпълни подпрограмата, обслужваща прекъсването или изключението. Възстановяването на прекъснатата процедура или задача се осъществява без нарушаване на непрекъснатостта на изпълнението на програмата, освен ако връщането от прекъсване е било възпрепятствано от изключение или прекъсване, предизвикало преустановяване на изпълнението на текущата програма.

На всяко архитектурно дефинирано изключение и всяко условие за прекъсване, изискващо специално обслужване, се присвоява уникален идентификационен номер, наречен **вектор**. Процесорът използва този вектор като индекс в таблицата на прекъсванията за определяне на входната точка на подпрограмата, обслужваща това изключение или прекъсване. Обикновено, първите няколко начални вектора са архитектурно дефинирани изключения и прекъсвания и са резервирани. След това следват позиции, които са предназначени за прекъсвания, дефинирани от потребителя. Тези указатели са разположени на фиксирано място в оперативната памет. Тези прекъсвания обикновено се използват от външните входно-изходни устройства.

Когато възникне прекъсване, централният процесор прекъсва изпълнението на текущата програма и съдържанието на програмния брояч на регистъра на състоянието се запазва в стек, за да може да има вложени прекъсвания. Новото състояние на процесора при влизане в прекъсване се взима от съответния вектор на прекъсване, който се намира по номера на прекъсването. Всяка програма, която обслужва прекъсване, завършва с инструкция за връщане от прекъсване, която от върха на стека прочита предишното състояние на програмния брояч и регистъра на състоянието и ги зарежда в процесора и по този начин изпълнението се връща към прекъснатата програма.

Съществуват различни видове прекъсвания:

- **Прекъсване по машинна грешка** – грешка в апаратурата, те са най-високо приоритетни;
- **Входно-изходни прекъсвания** – те се активират в резултат на изпълнение на входно-изходна операция от външен контролер;
- **Външни прекъсвания** – свързани са с някакъв външен елемент извън процесора и са сходни на входно-изходни прекъсвания; например, натискане на бутон
- **Програмни прекъсвания** – възникват от невъзможност да бъде изпълнена някоя стъпка от цикъла на командата. Практически всяка стъпка може да не бъде изпълнена, затова процесорът преди изпълнение на предстоящата прави проверка дали има условия за нейното изпълнение. Ако не - емулира сигнал за програмно прекъсване
- **Програмно-активирани (SVC – Supervisor Call) прекъсвания** – при тях текущата програма издава специална команда за провеждане на прекъсване.

Маскируеми и немаскируеми прекъсвания (вид програмни прекъсвания)

Друго разделяне на прекъсванията е на маскируеми и немаскируеми. Маскируемите прекъсвания могат да се игнорират, тоест да не се обработват, докато немаскируемите прекъсвания задължително се обработват. Така немаскируемите прекъсвания винаги имат приоритет пред маскируемите. Типично маскируемо изискване е *Overflow* – при събиране на две големи положителни числа се получава малко отрицателно число, но това може да се маскира и програмно да не прекъсне.

За да се отчете важността на събитията, които може да настъпят, се въвежда **приоритет на прекъсванията**. По време на изпълнението си програмата за обработка на едно прекъсване може да бъде прекъсната само от прекъсване с по-висок приоритет. Обикновено прекъсванията с по-малки номера имат по-висок приоритет.

Ако едновременно са възникнали няколко прекъсвания, те се обслужват в ред, определен от техните приоритети. Това често зависи от архитектурата на конкретния процесор. Един примерен ред на приоритети на обслужване е следния:

1. Прекъсвания от инструкция INT и при особени случаи;
2. Прекъсване при стъпков режим (debugging mode);
3. Немаскируеми прекъсвания
4. Маскируеми прекъсвания

Управлението на прекъсванията се извършва от специално логическо устройство, което се нарича **контролер на прекъсванията**. Контролерът на прекъсванията получава заявки за прекъсвания от различните хардуерни устройства посредством линиите за заявки за прекъсвания IRQ (Interrupt Request – заявка за машинно прекъсване). Машинното прекъсване е нужно, за да се избегнат хаотичните заявки към процесора на различни външни устройства, нуждаещи се от управление или обмяна на данни, при което се губи много процесорно време. IRQ позволява бърза реакция при устройства, които се нуждаят от бърза обработка, за да не губят информация. Естествено IRQ има и недостатъци, например дадена програма може да забрани машинните прекъсвания за определено време. Казано по-просто, контролерът на прекъсванията приема заявките за прекъсване на външните устройства, определя приоритетите и след това изпраща заявка за прекъсване към процесора, т.е. контролерът „решава“ кое устройство, кога и колко време ще комуникира с процесора, като ги редува.

Държавен изпит – спец. „Софтуерно инженерство“

Тема №7 – Процеси и комуникация между тях в операционната система.

Анотация: *Процеси и комуникационни канали – основни абстракции, предоставени от операционната система. Неформално определение и функционални изисквания.*

Комуникация между процеси чрез споделена памет. Съревнование за ресурси (race condition). Хардуерна защита на ресурс, синхронизация чрез spinlock.

Синхронизация от високо ниво – семафор. Приспиване и събуждане на процеси (block/wakeup). Реализация чрез семафори на комуникационна тръба (pipe), съхраняваща n елемента.

Процес – определение

Процесът представлява програма в хода на нейното изпълнение, абстрактна машина за обработка на информация. Процесът е реален изчислителен процес, който обработва информацията в реална среда. Той не съществува изолирано, самостоятелно, а си взаимодейства с други обекти в операционната система – с други процеси и файлове. Когато процесите си комуникират, те излизат от локалното си пространство, за да използват споделените ресурси, предоставяни от операционната система – достъп до споделена памет, общи регистри, файлови дескриптори.

Процес е инстанция на програма, която е в момент на изпълнение, в дадена машина. Това са поредица от команди по време на изпълнение. Процесът може да инициира подпроцес, който се нарича *детински процес* (а инициращия процес се нарича негов *родител*). Детинският процес е частична реплика на родителския процес и споделя някои от неговите ресурси. Процесите могат да обменят информация или да синхронизират работата си чрез няколко метода за междупроцесна комуникация (*Inter Process Communication* – IPC). Те могат да комуникират помежду си чрез: споделена памет или чрез обменяне на съобщения. Операционната система може да имплементира и двата метода на комуникация.

Повечето от функционалностите, които една операционна система поддържа са удобства за програмистите. Езиците за програмиране са абстрактни, не се влияят от хардуера, а са езици от високо ниво (C++, Java, Python и др.). Веднъж написана, програмата на такъв език може да бъде превърната в процес като част от операционната система и да съществува дълго време без корекции. Идеята на операционната система е да можем да поддържаме дълготрайността на програмите, защото хардуерът се променя често и бързо, а това не бива да изисква такава промяна и на програмите. Основната цел на процеса е да поддържа тази вечност и абстрактност.

Комуникационни канали – основни абстракции, предоставени от операционната система. Неформално определение и функционални изисквания

Файловете сами по себе си представляват дълго живеещи обособени единици информация. Споменахме вече процесите като работещи програми. **Комуникационните канали** разглеждаме като средства за връзка между процесите и други обекти. Например, при поискване за отваряне на файл, операционната система ни предоставя (отваря) комуникационен канал.

Хардуерните абстракции представляват еднообразни софтуерни модели за различни входно-изходни устройства. **Програмните интерфейси** са свързани с тези хардуерни абстракции дотолкова, доколкото те ни предоставят командни интерпретатори (текстови конзоли, графични потребителски интерфейси) и API за осъществяване на комуникация.

Една друга абстракция е принципът за **изолация**, когато имаме възможност да изолираме и защитим едни от други дейности, ресурси, изчисления. Например, това помага да не си пречат процесите, като ако един прекъсне внезапно или има неочаквано поведение, това да не навреди на останалите процеси като краде чужда памет и др.

За да се поддържа илюзията, че всички процеси работят едновременно, техните изпълнения се делят на малки интервали. За превключване на процесите съществува **хардуерен елемент** (*task scheduler*), който преценява дали даден процес трябва да работи или трябва да се прекрати изпълнението му, ако си е изразходил малкия интервал процесорно време.

Основни комуникационни канали в Linux:

1. **Неименувана тръба (pipe)** се създава чрез системното извикване $pipe(fd[2])$. То връща два файлови дескриптора на мястото на елементите на подадения масив $int\ fd[2]$. $fd[0]$ съхранява файловия дескриптор за четене, а $fd[1]$ съхранява файловия дескриптор за писане. Тъй като тази тръба не е именувана, тя е видима само за процеса, който я е създал, както и от наследниците му (децата му, децата на децата му и т.н.). Тази тръба може да се използва само в системата, т.е. не може да се използва в Интернет. Тя служи за връзка между процес и друг процес, който има роднинска връзка с първоначалния процес. Тъй като тръбата е неименувана, тя не използва пространството на имената.
2. **Именувана тръба (FIFO)** се създава чрез библиотечното извикване $mkfifo(\dots)$. За разлика от неименуваната, този вид тръба е видима за всички процеси в системата и може да бъде ползвана от тях. Тя се използва за осъществяване на комуникация между два процеса, които не са задължително в роднинска връзка. Този вид тръба се обвързва с име, откъдето следва, че тя използва пространството на имената.
3. **Писане във файл и четене от файл** – чрез системното извикване $open(filepath, open_flags, \dots)$ се създава файлов дескриптор, който сочи към единия край на комуникационния канал, който се изгражда в ядрото на операционната система и е за писане, а другият край сочи към файла, който може да се разглежда като много прост абстрактен процес, служещ само за съхраняване на данни. Чрез системните команди $read(int\ fd, void\ *buf, size_t\ cnt)$ и $write(int\ fd, const\ void\ *buf,$ size_t cnt) може да се чете и пише от и във файла. Този вид комуникационен канал използва пространството на имената.
4. **Конекция (socket)** – това е именуван обект, който играе ролята на крайна точка за комуникация между отдалечени обекти и който се използва за адрес при изграждането на връзка с друг процес. Тук има два вида процеси – единият се нарича *сървър* и работи като такъв (ще обслужва други процеси), а другият – *клиент*. И двата вида процеси ще трябва да изпълняват различни поредици от извиквания, за да изградят връзка помежду си. Предимствата на сокетите пред тръбите е, че процесите, с които може да се изгради връзка, могат да не се намират в една и съща система. Сокетът на сървъра използва пространството от имената, тъй като трябва да му бъде зададено име, с което да бъде видим за останалите процеси. Сокетът на клиента обаче не използва пространството на имената, тъй като не се обвързва с име.

Комуникация между процеси чрез споделена памет

Комуникацията между процесите, използващи споделена памет, изисква процесите да споделят някаква обща за тях променлива. Използването на тази споделена променлива напълно зависи от това как програмистът ще имплементира комуникацията. Един от начините за комуникация с помощта на споделена памет може да се представи по следния начин: Да предположим, че процес p_1 и процес p_2 се изпълняват едновременно и те споделят някои ресурси или използват някаква информация от друг процес. p_1 генерира информация за определени изчисления или използвани ресурси и я съхранява като запис в споделена памет. Когато p_2 трябва да използва споделената информация, той ще провери записа, съхранен в споделената памет, и ще вземе под внимание информацията, генерирана от p_1 и ще действа според нея. Процесите могат да използват споделена памет за извличане на информация като запис от друг процес, както и за предоставяне на всякаква специфична информация на други процеси. Проблемите на комуникацията чрез тръби (неименувани /*pipes*/, именувани /*fifos*/) и обмяна на съобщения (*message passing*) е, че информацията трябва да премине през ядрото, докато при комуникацията чрез споделени данни (памет) това не е необходимо.

Съревнование за ресурси

Заради невъзможността да се предвиди работата на процесите във времето и от друга страна нуждата от използването на споделени ресурси е необходимо да се установи начин за синхронизацията на „паралелно“ работещите процеси и начинът, по който те достъпват общи ресурси. В противен случай може да възникне съревнование за ресурси.

Race condition на български може да се преведе като борба или съревнование за ресурси. То възниква, когато два или повече процеса (или нишки) имат достъп до споделени ресурси и се опитват да ги променят едновременно. Алгоритъмът за даване на процесорно време за изпълнение на процес може да скача непредвидимо и по всяко време. Ние не знаем реда, в който процесите ще се опитат да получат достъп до споделените данни (този ред може да се счита за хаотичен) и следователно резултатът от промяната в данните зависи от алгоритъма за планиране на процесите. По този начин два или повече процеса се „съревновават“ за достъп до данните. Този достъп до данни често е свързан и с някаква промяна и това може да доведе до нарушаването им, като в някой момент структурата от данни на общата памет е била нарушена временно от методите ѝ (тоест инвариантът на структурата е бил временно нарушен). Но именно, ако през това време друг метод използва структурата, то ресурсът ще бъде използван докато се е актуализирал – моментите, в които се актуализира, се наричат *критични секции*. Проблемът често възниква, когато един процес или нишка изпълнява операцията „провери и след това действай“ (например „провери“ ако стойността е x и след това „действай“, за да направиш нещо, което зависи от това, че стойността е била такава), а друг процес направи нещо със стойността на x между „провери“ и „действай“ на първоначално споменатия процес:

```
if (x == 5) { // „провери“
    y = x * 5;  // „действай“

}  // но ако някой друг процес или нишка е променила стойността на x между проверката и
//   действието, то y няма да е това, което очакваме, т.е. 25
```

Въпросът е, че y може и да е 25, но може и да бъде всичко (като например парче от string), в зависимост от това дали друг процес е модифицирал в критичен момент и в критична секция на кода. Няма как да знаем дали това се е случило. За да предотвратим появата на борба за ресурси, обикновено поставяме *заключване* около споделените данни, за да гарантираме, че само един процес може да има достъп до тях за определено време. Тоест нещо подобно:

$lock(x)$; // заключи x

$if\ (x == 5)$ {

$y = x * 5$; // сега всеки друг процес или нишка няма да може да модифицира x, докато
// заключването не се отключи и следователно ще знаем, че $y = 25$

}

$unlock(x)$; // отключи x

Без изрично, експлицитно синхронизиране, което да координира достъпа до споделени данни, дадена нишка ще може да промени променливи, които друга нишка е започнала вече да използва и това може да доведе до непредвидими резултати.

Хардуерна защита на ресурс, синхронизация чрез spinlock

Желанието да се синхронизират процесите, използващи споделения ресурс X се свежда до това да се направи така, че винаги само един процес да е в своя критична секция, докато всички други изчакват излизането му от този регион, т.е. никой друг процес не може да е в критичната си секция, използвайки същия споделен ресурс.

Съществуват няколко начина за осъществяването на това, но по същество те се делят на методи за синхронизация от ниско и високо ниво. Еволюционно, първият пример за справяне с този проблем е **забраната на прекъсванията**, докато процес е в своята критична област. Първият процес, който влезе в своя критичен регион, няма да може да бъде прекъснат, докато критичната секция не завърши. Това обаче е приложимо само за еднопроцесорна операционна система.

Повишаването на честотата на процесорите вече е станало невъзможно, поради чисто физически причини. За да се развиват системите, те стават все по-сложни, с все повече ядра и повече от един процесор и тогава имаме реален паралелизъм. Паралелно работещите процесори правят метода с прекъсването неработещ. Може да прекъснем процесите от едно ядро, но процесите от друго ядро може да правят каквото си поискат, в това число и да работят по критични секции. Трябва да пазим някакъв **допълнителен бит** в споделената структура от данни, за да индикираме чрез него за това дали тя се използва в момента от някой друг процес или нишка. Този бит се нарича *lock*, а методът, който използваме, се нарича *spinlock*.

За разсъжденията по-долу се базираме на допускането, че хардуерът може да блокира прекъсвания. Този факт беше достатъчен за еднопроцесорна система. Допускаме още, че хардуерът може да има специални атомарни инструкции, които да променят бит и да го прочитат след това като единна непрекъсната инструкция.

spin_lock: R = test_and_set(lock);

if (R == 1) goto spin_lock;

critical_section $\begin{cases} p_1 \\ \vdots \\ p_k \end{cases}$

spin_unlock: lock = 0;

В момента, в който процес е в критична секция и друг процес изяви желание да я използва, то програмата ще го връща циклично, докато работещият в критичната секция процес не излезе от нея и не промени *lock* бита (макроса) на 0, което ще позволи на втория процес да влезе и да заключи след себе си.

В многопроцесорна система това би трябвало да осигури защита, обаче възникват допълнителни проблеми – ако друга програма подбудена рекурсивно от критичната секция се извика – *lock*-ът вече ще е 1-ца и ще влезе в безкраен цикъл, без да може да излезе, защото излизането и писането на 0 се случва само в края на критичната секция. Да допуснем, че по някакъв начин може да забраним на програмистите рекурсивното извикване в критичната секция. Но същият проблем може да се случи и при прекъсването. Нека обаче се опитаме да забраним прекъсванията:

spin_lock: disable_interrupt;

R = test_and_set(lock);

if (R == 1) goto spin_lock;

critical_section $\begin{cases} p_1 \\ \vdots \\ p_k \end{cases}$

spin_unlock: lock = 0;

enable_interrupt;

Нека допуснем, че два процеса p и q искат да влязат в критичната секция и p е влязъл, а q цикли. Обаче p е забранил прекъсванията и не само p, а и всички, които чакат също са забранили прекъсванията, което е недопустимо, тъй като това е все едно да забраним на периферията да общува с всички тези циклещи процеси, което прави системата неефективна. От друга страна, другите процесори въртят цикли и вършат работа, но и тази работа е неефективна. Затова ще направим още едно подобрение в горния код:

spin_lock: disable_interrupt;

R = test_and_set(lock);

if (R == 0) goto critical_section;

enable_interrupt;

relax;

goto spin_lock;

critical_section $\begin{cases} p_1 \\ : \\ p_k \\ lock = 0 \end{cases}$

enable_interrupt;

Сега вече, ако критичната секция е заета, другите процеси ще циклят, но ще имат възможността да свършат и някоя друга полезна работа през това време.

Синхронизация от високо ниво – семафор

Съществуват и структури от високо ниво, които служат за синхронизация. Те обикновено се основават на принципа за приспиване и събуждане на процес. Ако един процес няма работа, която да извърши в даден момент и очаква външно събитие, то той бива приспан до случването на събитието (за разлика от хардуерния механизъм със *spinlock*, при който реализираме т.нар. *busy waiting*). Когато споделеният ресурс е наличен отново, процесът се събужда и продължава изпълнението си от там, докъдето е стигнал. Най-популярната реализация на тази абстрактна идея е **семафорът**, който е въведен от Едгар Дейкстра през 60-те години на 20-ти век.

Семафорът е абстрактен механизъм за синхронизация на процеси, използващи общи ресурси. Най-грубо казано, той представлява ключалка с брояч, който казва колко процеси (или нишки) могат да се намират едновременно в критичната секция, която пази ключалката. Структурата на данните му е следната:

- **Атрибути:**
  - $cnt : int$ → брояч (показва колко процеса могат да преминат бариерата, за да ползват споделения ресурс). Ако $cnt < 0$, то $|cnt|$ е броят на приспаните процеси;
  - $L : list$ → контейнер/списък, който може да разглеждаме като множество от приспани процеси.

Силен семафор е този, който използва обикновена опашка за множеството L. Слаб семафор е този, при който има някакъв критерий, който се използва за приоритет при обслужването на процес. Слабостта идва от факта, че такъв критерий може да причини така наречената „гладуване“ на процеси (нишки) с най-нисък приоритет, което се счита за проблем със синхронизацията. Очакваното поведение е да нямаме процеси, които да чакат твърде много.

- **Методи:**
  - $init(int\ cnt = 0)$ → инициализатор (конструктор) на семафор, който приема аргумент по подразбиране – цяло число, с което се инициализира броячът;
  - $wait()$ → с този метод намаляваме брояча с 1, т.е. заявяваме, че искаме да ползваме общия ресурс; ако броячът е по-малък или равен на 0, то нишката се блокира (влиза в списъка с приспаните процеси) и чака той да стане положително число;
  - $signal()$ → метод, с който се увеличава броячът с 1 и така се сигнализира от нишката, че е освободила ресурс и е приключила критична секция; при

увеличаване на семафора, една от чакащите нишки бива отблокирана и допусната в своята критична секция.

Трябва да се отбележи, че променливата за брояча на семафора е недостъпна за процеса, който използва семафора, защото не е сигурно, че след четенето, тя няма да се промени от друг процес. Ето защо при извикване на $wait()$ не може да се гарантира, че процесът няма да бъде блокиран. Също така не може да се предвиди дали ще има освободени нишки след извикване на $signal()$. След изпълнението на $signal()$ събуденият процес продължава изпълнението си конкурентно заедно с този, който е направил $signal()$. Добрата операционна система предоставя готови комуникационни канали и възможност за ползване на семафори като допълнителен механизъм. Реалното място, където масово се използват семафорите е в ядрото на операционната система.

Приспиване и събуждане на процеси (block/wakeup)

Процесите се характеризират с някакво текущо състояние, в което се намират. Едно от тези състояние е sleeping/blocked. Спящите процеси от гледна точка на потребителя са работещи програми, които са стигнали до състояние, при което нямат нужда да изчисляват нещо докато не настъпят очаквани от тях събития в системата. Те са в комуникация с друг процес или устройство и очакват да им се подадат данни, но очакваните процеси нямат готовност да им подадат (например поради запушен комуникационен канал или поради бавна работа на другата страна и т.н.). От гледна точка на реализацията може да чакат:

- **I/O** – очаква се извършването на входни-изходни операции;
- **Time** – очаква се да настъпи даден времеви момент;
- **Signal** – очаква се сигнал от друг процес за промяна на състоянието му;
- процесът бива приспан, защото **страницата**, с която иска да работи не е в реалната памет, а е някъде на твърдия диск;
- процесът чака да се **освободи семафор** от предишен ползвател.

Процес не може произволно да реши да се приспи. Трябва да има някаква причина за това.

За да се събуди спящ процес, ядрото изпълнява **алгоритъм за събуждане**. Алгоритъмът освобождава дадена ключалка и събужда всички процеси, които чакат тази ключалка да се освободи. Аналогично процес може да се събуди, ако получи сигнал, че дадена входно-изходна операция е приключила. Ядрото повишава нивото на изпълнение на процесора при събуждане, за да блокира прекъсванията. След това за всеки процес, който е заспал, чакайки събитие, което вече е настъпило, бива вкаран от състояние на спящ в състояние на готов за изпълнение (*sleeping* → *ready to run*). Премахва процеса от свързания списък със спящи процеси, поставя го в свързан списък с процеси, отговарящи на условията за планиране. Събуждането не води до незабавно насрочване на процес. Това само прави процеса допустим за изпълнение (*ready to run*). Кога процесът ще бъде изпълнен, зависи от диспечера на операционната система.

Реализация чрез семафори на комуникационна тръба (pipe), съхраняваща n елемента

Предполагаме, че **тръбата** може да съхранява до n байта, подредени в обикновена опашка. **Тръбата** се използва от няколко паралелно работещи „изпращачи/получатели“ на байтове. **Процесите изпращачи** слагат байтове в края на опашката, а **получателите** – четат байтовете от началото на опашката.

За да реализираме комуникационна тръба ще използваме следните инструменти:

- **Опашка (или масив) Q с n елемента** – тъй като е възможно да имаме бърз и бавен процес е нужно опашката да бъде по-дълга, за да не се приспива по-бързият процес. В началото тази опашка ще е празна;
- ***free_bytes*** – семафор, който ще индикира за свободните байтове в опашката Q;
- ***ready_bytes*** – семафор, който ще индикира колко байта са готови за четене (до колко е запълнена опашката Q) ;
- ***mutex_read*** – мутекс, който ще защитава критичната секция за четене;
- ***mutex_write*** – мутекс, който ще защитава критичната секция за писане.

Инициализация:

free_bytes.init(n)

ready_bytes.init(0)

mutex_read.init(1)

mutex_write.init(1)

READ :

$byte\ b$

p_1

p_2

:

ready_bytes.wait()

mutex_read.wait()

$b = Q.get()$

mutex_read.signal()

free_bytes.signal()

:

WRITE

$byte\ b$

q_1

q_2

:

free_bytes.wait()

mutex_write.wait()

Q.put(b)

mutex_write.signal()

ready_bytes.signal()

Процесът, който чете, първоначално ще проверява дали има готови за четене байтове. Ако няма, той ще се приспи и ще чака да постъпят такива. Ако има, той ще провери дали някое копие на P не чете в момента. Ако да, той отново се приспива. Ако не, той ще прочете байт и ще го запише в променливата b. След това ще сигнализира, че в опашката има още един свободен байт чрез подаване на сигнал на семафора *free_bytes*.

Процесът, който пише, първоначално ще провери дали има свободни байтове в опашката. Ако няма, ще се приспи и ще чака да се освободят байтове. Ако има, ще трябва да провери дали някое друго копие на пишещ процес не пише в момента в опашката. Ако да, то отново процесът ще се приспи. Ако не, ще сложи байта си b в опашката и ще сигнализира, че още един байт е готов за четене, което се осъществява чрез подаване на сигнал на семафора *ready_bytes*.

Решението, предоставено по-горе, което показва как може да се реализира комуникационна тръба чрез семафори, е валидно само за синхронни входно-изходни операции, защото процесите се приспиват, когато има недостиг на байтове.

Държавен изпит – спец. „Софтуерно инженерство“

Тема №8 – Компютърни мрежи и протоколи – OSI модел. Протоколи IPv4, IPv6, TCP, HTTP.

Анотация: *OSI модел – най-обща характеристика на нивата, съпоставяне с модела TCP/IP. IPv4 адресация – класова и безкласова. Основни характеристики на протокол IPv6. TCP – процедура на трикратно договаряне. Хипертекстов протокол HTTP.*

OSI модел – най-обща характеристика на нивата, съпоставяне с модела TCP/IP

Основен принцип в съвременните мрежови архитектури е **принципът за разслояване** на функциите по управление на връзките, като всеки слой ползва услугите, предоставени от по-долните слоеве, без да знае как са реализирани тези услуги. Това е принципът на прозрачност.

Реализацията на слоевете, както и интерфейсите между отделните слоеве не е задължително да са едни и същи на машините в една мрежа – достатъчно е всеки слой n да може да комуникира със съответния слой n по определения протокол и да предоставя съответните услуги на по-горния слой. Данните и контролната информация на слой n се наричат протоколен блок от данни (PDU). За слой n – 1 PDU(n) са си обикновени данни. Чисто потребителските данни се наричат payload.

Списъкът от протоколи, използвани от една система, по един протокол за всеки слой се нарича **протоколен стек**.

В OSI модела базови са три понятия:

- **услуга** – определя функционалността на слоя, без конкретика (не казва как);
- **интерфейс** – определя за по-горележащите слоеве достъпа до услугата;
- **протокол** – определя реализацията на услугата.

Съвременните мрежови архитектури следват принципите на модела **OSI** (Open Systems Interconnection), създаден от Международната асоциация по стандартизация *ISO* (Internationals Standards Organization) за връзка между отворени системи (система, чиито ресурси могат да се използват от другите системи в мрежата).

OSI моделът е абстрактен модел на мрежова архитектура, който описва предназначението на слоевете, но не се обвързва с конкретен набор от протоколи. Поради това OSI моделът се нарича още **еталонен** (опорен) модел и всъщност дава препоръки (reference model).

В еталонния модел има седем слоя – **физически**, **канален**, **мрежов**, **транспортен**, **сесиен**, **представителен**, **приложен**.

Физическият слой (*physical layer*) има за задача да реализира предаването на битове през физическата среда. Основна функция на физическия слой е да управлява кодирането и декодирането на сигналите, представящи двоичните цифри 0 и 1. Той не се интересува от предназначението на битовете.

Физическият слой трябва да осигурява възможност на по-горния слой да активизира, поддържа и прекратява физическите съединения. Обектите на този слой са хардуерните устройства,

реализиращи предаването на 0 и 1 през физическата среда – мрежови карти (NIC), модули, модеми и др.

Основна функция на **каналния слой** (*data-link layer*) е управлението на канала от един възел до друг (точка-точка) според класическия модел, „точка-много точки“ (напр. *frame relay*) или достъп до преносната среда (MAC) в LAN.

Друга възможна функция е (евентуално) откриването и/или коригирането на грешки при предаването на данните. Данните на каналното ниво се обменят на порции (PDU), наречени **кадри** (*frames*), обикновено с дължина от няколко стотин до няколко хиляди байта в зависимост от скоростта на линията. При надеждна комуникация приемникът трябва да уведомява изпращача за всеки успешно получен кадър като му изпраща обратно потвърждаващ кадър.

Форматът на кадрите се определя от избрания протокол на канално ниво. Функциите на каналния слой обикновено се реализират смесено – апаратно и програмно. Колкото повече функции се реализират софтуерно (контролерът е реализиран на дънната платка), толкова е по-ниска производителността.

Мрежовият слой (*network layer*) отговаря за функционирането на комуникационната подмрежа. Пакетите са с фиксирана големина в рамките на една мрежа. При преминаване от една компютърна мрежа в друга е възможно пакетът да се раздели на части – фрагментира, като накрая може да се възстанови (напр. преход LAN-WAN-LAN).

Основна задача на мрежовия слой е маршрутизирането на тези фрагменти, опаковани като **пакети** (PDU за мрежов слой). За системите, реализиращи възлите на комуникационната подмрежа (маршрутизатори – routers), този слой е последен. Функциите на мрежовия слой, както и на по-горните слоеве, се реализират програмно.

Транспортният слой (*transport layer*) осигурява транспортирането на съобщения от източника до получателя. Той е най-ниският слой, който реализира връзка от тип „край-край“ (end to end) между комуникиращите системи.

Изгражда се програмен канал между портовете на приложения, които си „говорят“ през мрежата. Транспортният слой осигурява на приложния слой (и по-горните слоеве) такава степен на надеждност, каквато е необходима.

В транспортния слой на изпращача съобщенията (евентуално могат) се разбиват на **сегменти** (PDU за транспортния слой) и се подават на мрежовия слой, където се опаковат като пакети, а в транспортния слой на получателя разопакованите от мрежовия слой сегменти се реасемблират, т.е. транспортният слой (евентуално може да) отговаря за целостта на обменяните съобщения, което включва откриване на загубени сегменти и тяхното повторно предаване.

Сесийният слой (*session layer*) е отговорен за диалога между две комуникиращи програми. Съобщения се обменят след като двата крайни абоната установят сесия. Сесийният слой осигурява различни режими на диалог: **двупосочен едновременен диалог** (*full duplex* – FD), **двупосочен алтернативен диалог** (*half duplex* – HD), **еднопосочен диалог** (*simplex*). Освен това той предоставя възможност за прекъсване на диалога и последващо възстановяване от

мястото на прекъсването. При липсата на сесиен слой всяко съобщение се предава независимо от другите съобщения.

Представителният слой (*presentation layer*) е най-ниският слой, който разглежда значението на предаваната информация. Първата функция на този слой е да определи **общ синтаксис за предаване на съобщенията**. Втората функция на слоя е да унифицира вътрешната структура на представените данни в съобщенията. По този начин за по-горния приложен слой няма значение дали двете крайни системи използват различни представяния на данните.

UTF-8 (8-bit Unicode Transformation Format) представя всеки символ в *Unicode* стандарт, но е и обратно съвместим с ASCII. По тези причини е предпочитан за email, уеб страници и др. Може да използва криптиране на данните, компресия.

Приложният слой (*application layer)* е най-горният слой, към който се свързват потребителските процеси в двата крайни абоната. Някои потребителски процеси са интерактивни – взаимодействат си в голям период от време с кратки съобщения от тип заявка-отговор (*request-reply*). Други потребителски процеси взаимодействат с малко на брой големи по обем порции от данни.

За двата вида процеси се предвиждат различни протоколи на приложния слой – например протокол **FTP** (*file transfer protocol*) за обмен на цели файлове, протокол **HTTP** (*hyper text transfer protocol*) за обмен на уеб страници и др.

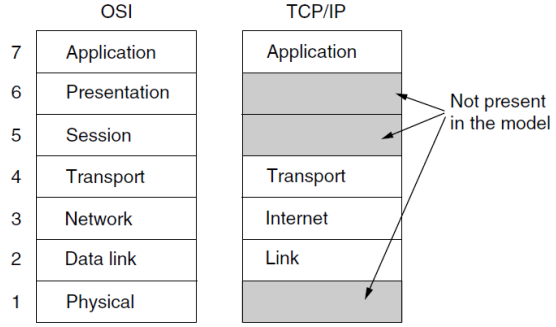
Когато започват да се изграждат реални мрежи, използвайки **OSI модела** и съществуващите протоколи се вижда, че те не отговарят на изискваните спецификации за обслужване.

Моделът TCP/IP е въведен за първи път през 1974 г. в ARPANET – първата компютърна мрежа, която прераства в Интернет. Целта е била да позволи свързването на различни мрежи, да бъде жизнеспособна и гъвкава, да оцелее при всякакви условия.

TCP/IP е мрежа с комутация на пакети, базирана на обслужване с неустановена връзка (connectionless – без предварително уговаряне на параметрите на връзката между източник и приемник). Това е мрежовото ниво, имаме „best effort delivery“ (негарантирана доставка), IP протокол, IP пакети.

Докато комитетите на **ISO** са спорили за своите стандарти, цялата концепция на мрежите се променя и по цял свят се внедрява **TCP/IP**. Днес OSI постепенно се движи към състояние да се окаже на страниците на историята на компютрите.

Общите свойства на OSI и TCP/IP е, че те са единен стек от независими протоколи с подобни функции. Има три основни свойства на **OSI** – дефиниране на услуги, интерфейси и протоколи.



Основно предимство на **OSI** е, че прави разграничението между тези три свойства, докато **TCP/IP** няма точно разграничение между трите. Друго предимство на протоколите в **OSI** е, че те са по-добре обособени, отколкото в **TCP/IP**, т.е. могат да бъдат заменяни по-лесно.

Преди да бъде създадена концепцията за протоколите, **OSI** е бил достатъчно общ модел. Това обаче се изразява в липсата на конкретика, която налага нови подслоеве да бъдат създавани, да бъдат изменяни в зависимост от различията в конкретните мрежи.

Първоначално от създателите на **OSI** се представя идеята, че всяка държава трябва да поддържа по една **OSI** мрежа под управлението на своето правителство. Не е мислено за междумрежово свързване.

От друга страна, при **TCP/IP** първо възникват протоколите, т.е. моделът реално е описание на вече съществуващите протоколи. Те пасват перфектно, без да е необходимо да бъдат напасвани към модела, както при **OSI**, при който първо възниква моделът като идеология.

TCP/IP не е приложим за описание на мрежи, които не поддържат **TCP/IP**. В наши дни обаче, всички производители го поддържат като повечето от тях са се отказали от своите собствени протоколни стекове, т.е. **TCP/IP** става световен мрежов стандарт.

Други разлики са, че на мрежово ниво **TCP/IP** е само connectionless, докато **OSI** може да бъде и connection-oriented. На транспортно ниво обаче **OSI** е само connection-oriented, а **TCP/IP** – и двете (съответно чрез **TCP** и **UDP**).

IPv4 адресация – класова и безкласова

- ***IPv4*** *реализира обмен на информация чрез пакети, които ще наричаме* ***IP дейтаграми****.*
- ***IPv4*** *е протокол за взаимодействие без установяване на логическо съединение.*
- ***IPv4*** *за адресация на възлите в мрежата използва адрес с дължина 4 байта.*
- ***IPv4*** *дейтаграмите имат зададено време на живот в мрежата*
- ***IPv4*** *не гарантира надеждност на доставката на IP дейтаграмите от получателя.*
- ***IPv4*** *няма средства за управление на интензивността на предаването на IP дейтаграмите от изпращача (flow control).*
- ***IPv4*** *не гарантира правилна последователност на IP дейтаграмите от получателя.*

Задачата на протокола IP е да извърши успешно предаване на пакети от източника до получателя, без значение дали те са в една и съща мрежа или в различни мрежи. Дейтаграмното предаване е свързано с това всеки пакет да се предава (транспортира) по мрежата независимо от останалите. Пакетите могат на теория да достигнат 64KB, но за момента не са по-големи от 1500 байта.

Всеки пакет се изпраща самостоятелно, като по пътя може да се фрагментира на по-малки единици. Когато тези единици достигнат до получателя, те се реасемблират от мрежовото ниво за получаване на оригиналния пакет. По-нататък данните от този пакет се подават на транспортното ниво на получателя, което я вмъква в съответния поток от байтове.

IP пакетът във версия 4 е с 32-битови адреси, които се разбиват на 4 части, наричани октети. Пакетът се предава в Big-Endian формат, т.е. от старшите към младшите байтове. IP пакетът се състои от заглавна част и част за данни. Заглавната част е 20B + опции с променлива дължина.

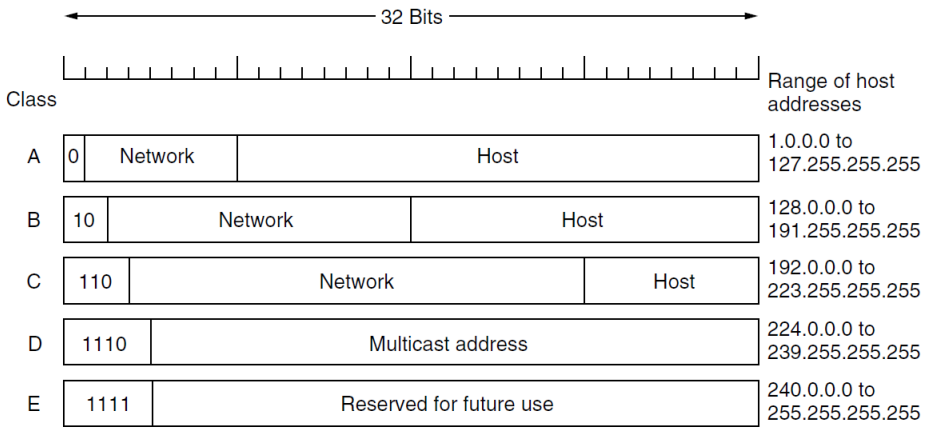
Между физическата линия и хоста се разполага мрежовият интерфейс. IP адресът технически се асоциира с мрежовия интерфейс, а не с хоста или маршрутизатора (рутерът има няколко интерфейса, по един за всяка линия).

За удобство IP адресите се изписват в **точкова десетична нотация**, като всеки от четирите байта се изписва като десетично число от 0 до 255. Най-малкият IP адрес е 0.0.0.0, а най-големият е 255.255.255.255.

Адрес, който съдържа само единици, се интерпретира като broadcast адрес, т.е. адресират се всички хостове в дадена мрежа.

Всеки интерфейс на хост и маршрутизатор в мрежата следва да има уникален IP адрес. Всички IP адреси са 32-битови. Всеки IP адрес се дели на **две части** – *номер на мрежа* и *номер на хост* (*NetID.HostID*). Номерът на мрежата (*prefix*) е в лявата част на адреса, а номерът на хоста е останалата порция от битове в дясната част на адреса.

В зависимост от структурата си IP адресите се делят на следните пет класа:



Битовете в началото на адреса, които определят класа му, се наричат сигнални битове.

- **Клас A**
  - фиксираният префикс е 0, т.е. обхватът на първия октет е 0000 0001 – 0111 1111, т.е. е от 1 – 127.
  - subnet маската на клас A по подразбиране е 255.0.0.0.
  - клас A включва адресите само от 1.0.0.0 до 126.255.255.255, като 127.x.x.x е резервиран за *loopback* адреса, а 0.0.0.0 е нерутируем мета-адрес, който обичайно означава невалиден адрес.
  - брой на мрежите е $2^7 - 2 = 126$ и брой на хостовете е $2^{24} - 2$.
- **Клас B**
  - фиксираният префикс е 10, т.е. обхватът на първия октет е 1000 0000 – 1011 1111, т.е. е от 128 – 191.
  - subnet маската на клас B по подразбиране е 255.255.0.0.
  - клас B включва адресите само от 128.0.0.0 до 191.255.255.255
  - брой на мрежите е 2^{14} и брой на хостовете е $2^{16} - 2$.
- **Клас C**
  - фиксираният префикс е 110, т.е. обхватът на първия октет е 1100 0000 – 1101 1111, т.е. е от 192 – 223

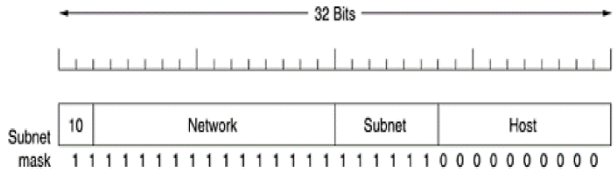
    - subnet маската на клас C по подразбиране е 255.255.255.0
    - клас C включва адреси само от 192.0.0.0 до 223.255.255.255
    - брой на мрежите е 2^{21} и брой на хостовете е $2^8 - 2$.
- **Клас D**
    - фиксираният префикс е 1110, т.е. обхватът на първия октет е 1110 0000 – 1110 1111, т.е. е от 224 – 239
    - клас D е резервиран за multicast и няма subnet маска.
    - щом данните при multicast не са насочени към конкретен хост, то няма нужда от извличане на host адреси
- **Клас E**
    - фиксираният префикс е 1111, т.е. обхватът на първия октет е 1111 0000 – 1111 1111, т.е. е от 240 – 255
    - обхватът на клас E е 240.0.0.0 – 255.255.255.254
    - не разполага със subnet маска
    - резервиран е за експериментални дейности в бъдещето

Всяка мрежа трябва да има уникален номер и всички хостове в дадена мрежа трябва да имат един и същ номер на мрежата. Това води до проблеми при нарастване на броя на мрежите.

Решението на проблема е да се разреши разделянето на една мрежа на подмрежи. За целта полето за мрежов номер се разширява надясно, като се отнемат битове от номера на хоста.

Например, за един адрес от клас B вместо 16 бита за номер на мрежата и 16 бита за номер на хост се използват 22 бита за номер на мрежа, като десните 6 от тях са за номер на подмрежа и 10 бита за номер на хост.

За реализация на подмрежа маршрутизаторите се нуждаят от **подмрежова маска** (*Subnet Mask - SM*), която определя границата между номера на мрежата + номера на подмрежата и номера на хоста. В долния пример имаме мрежовата маска на една разцепена клас B мрежа:



Интернет е сегментирана мрежа. В един мрежов сегмент (една локална мрежа) всички хостове имат IP адреси с еднакъв номер на мрежа (*NetID*) и еднаква маска (*NetMask*).

При разделяне на една мрежа на подмрежи взимаме „назаем“ (*borrow*) битове от хост частта на адресите.

Получава се следното:

$$N\ S\ H$$

Броят на подмрежите е: 2^S

Броят на хостовете в подмрежа ще е: $2^H - 2$ (нулевият адрес остава за номер на подмрежата, а последният – за broadcast)

SM има същия формат като *IPv4* адреса: старшите битове, които не принадлежат на хост частта са равни на 1 и се наричат префикс, а останалите (хост частта) са 0. Възможни са два начина на изписване на мрежов адрес.

Например, следният клас С адрес:

192.168.1.0 255.255.255.0

или

192.168.1.0/24

Второто означение се нарича 24-битов префикс или просто префикс.

Връзката между IP адресацията и маршрутизацията е важна. Всеки маршрутизатор има **таблица с маршрутите** (*Routing Table*), по която определя пътя на пакета. Всеки ред съдържа IP адреса на мрежа/префикс, следващ възел по пътя, изходящ интерфейс и др.

Някои резервирани адреси:

- **0.0.0.0** – маршрут по подразбиране, който се използва при маршрутизация на IP дейтаграмите
- **127.0.0.0** – резервиран за локален IP трафик; присвоява се интерфейс за обратна връзка към хоста – loopback интерфейс
- **127.0.0.1** (както и **127.x.x.x**) – използва се за служебен адрес на самия хост (localhost address)

Така наречените **частни интернет мрежи** (*private networks*) са резервирани за използване от различни организации за реализиране на връзка между компютрите само в рамките на съответната организация. Такива мрежи са **10.x.x.x** от клас А, 16 мрежи от **172.16.x.x до 172.31.x.x** от клас В и 256 мрежи **192.168.x.x** от клас С. Адресите от тези мрежи не се маршрутизират в глобалния Интернет. Най-честото им приложение е при *firewall*.

Големият недостатък на IPv4 е, че половината адреси са от клас А и се разпределят само измежду 127 автономни системи, въпреки че всяка от тях може да съдържа милиони хостове. Всяка мрежа трябва да има уникален номер и всички хостове в дадена мрежа трябва да имат един и същ номер на мрежата. Това води до проблеми при нарастване на броя на мрежите.

С разрастването на Интернет, най-бързо са изчерпани свободните IP адреси на мрежи от клас В, което налага на организации, притежаващи голям брой компютри, да се дават две или повече мрежи от клас С. Това води до увеличаване на размерите на глобалните таблици: от една страна, защото в маршрутизиращите таблици има по няколко записа, които водят до общ маршрутизатор на организацията, а от друга – заради произволното раздаване на номера на мрежи, което налага маршрутизаторите да пазят запис за всяка мрежа, без възможност за агрегиране. За намаляване на този обем се въвежда **безкласова адресация и маршрутизация** – **CIDR** (*Classless Inter-Domain Routing*). CIDR записът представя всеки адрес като 32-битово число, последвано от наклонена черта"/" и броя единици в двоичния запис на subnet маската. Например, 192.0.2.96/28 означава IP address, в който първите 28 бита са за NetID, а Sub Net маската е 255.255.255.240.

Основни характеристики на протокол IPv6

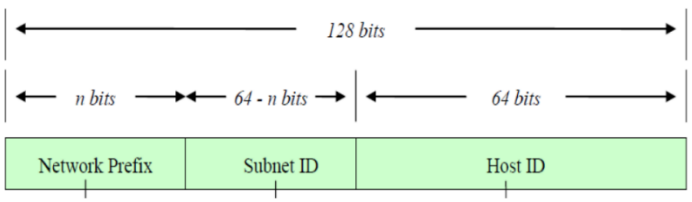
В наши дни **IPv6 протокол** активно се използва от множество мрежи по целия свят, но все още не е получил толкова широко разпространение в Интернет, както IPv4.

Основни характеристики на IPv6 протокола:

1. **Дължината** на IP адреса е увеличена до **16 байта** (128-битови), като така се предоставя на потребителите практически неограничено адресно пространство;
2. **Опростена структура на заглавието** (точно 40 байта), съдържащо само 8 полета (вместо 13 в IPv4 протокола). Последното позволява на маршрутизаторите по-бърза обработка на пакетите, т.е. повишава се тяхната производителност;
3. **Подобрена поддръжка на незадължителни параметри**, понеже в новото заглавие изискваните по-рано полета стават незадължителни, а измененият начин за представяне на незадължителните параметри ускорява обработването на пакетите от маршрутизаторите за сметка на отсъствието на обработване на тези параметри;
4. **Подобрена система за безопасност** – автентикацията и конфиденциалността са основни характеристики на новия IP протокол;
5. Предвидена е **възможност за разширяване на типовете** (класовете) на предоставените услуги, които могат да се появят в резултат на очаквания растеж на мултимедийния трафик. Отделено е по-голямо внимание на типа на предоставените услуги. За тази цел в заглавието на пакета на IPv4 е било предвидено 8 разрядно поле.
6. **Комуникация от край-до-край** – тъй като всеки хост има свой уникален IP адрес, той може да достигне до всеки друг хост без нужда от NAT (Network Address Translation);
7. В IPv6 **фрагментирането** на пакетите става още при **източника**;
8. **Автоматична конфигурация** – IPv6 поддържа stateful и stateless автоматична конфигурация на устройствата. Така неналичието на DHCP (Dynamic Host Configuration Protocol) не спира комуникацията между мрежите;
9. **IPSec** е по-сигурен от IPv4;
10. **Няма broadcast предаване** в IPv6 – не са специфицирани broadcast адреси в IPv6, като се използва multicast за комуникация с много адреси;
11. **Anycast** – много хостове могат да получат един и същ IPv6 адрес, като пакетите се рутират до най-близкия такъв;
12. **Мобилност** – позволява на хостовете да се преместват географски, но да задържат своя IP адрес. Това се постига чрез използване на автоматична конфигурация и допълнителни хедъри;
13. **Плавен преход** от IPv4 към IPv6.

Адресите в IPv6 са 128-битови, като те се разбиват на 8 групи с по 4 шестнадесетични числа, разделени с двоеточие.

Пример: 2001: 0db8: 9095: 02e5: 0216: cbff: frb2: 7474



Един IPv6 адрес се състои от:

- *Мрежов префикс (network prefix) – идентифицира дадена мрежа или специален адрес;*
- *Идентификаторът на подмрежата (subnet ID) – връзка вътре в мрежов обект. Присвоява се от администратора на обекта. Определя на кой мрежов сегмент принадлежи даден хост;*
- *Host ID – идентифицира конкретен възел в мрежата – конкретен негов интерфейс.*

Мрежовият префикс е аналогичен на означението с „/“ в IPv4.

Пример: Мрежата $2001{:}\,0db8{:}\,9095{:}\,02e5{:}\,0216{:}\,cbff{:}\,feb2{:}\,7474/32$

- *Мрежов префикс (Network Prefix):* $2001{:}\,0db8$
- *Идентификатор на подмрежата (Subnet ID):* $9095{:}\,02e5$
- *Идентификатор на хоста (Host ID):* $0216{:}\,cbff{:}\,feb2{:}\,7474$

За да се улесни записването на адреси, съдържащи нули, те се компресират по определени правила. „::“ символът се появява, когато адресът съдържа една или повече 16-битови групи от нули. Този символ се използва за компресиране на водещи или завършващи нули и може да се появи най-много веднъж в адреса.

TCP – процедура на трикратно договаряне

TCP (*Transmission Control Protocol*) е протокол от транспортния слой, който има за цел транспортирането по надежден начин на потока от байтове получен от по-горните слоеве. Това се постига чрез записването на поредни номера в хедърите на сегментите. Така дори и да има повторени, загубени или разбъркани пакети, данните биват сглобявани по коректен начин при получателя.

TCP е протокол, който предоставя надеждно обслужване с установена връзка (*connection-oriented*). Той внася обаче допълнително закъснение заради функциите по надеждност, спазване на реда на подаване на единиците с данни (сегменти) и управление на потока.

Всеки **TCP** сегмент има 20 байта служебна информация, с които описва приложните програми, докато при UDP дейтаграмите са само 8 байта. Типични приложения на TCP са:

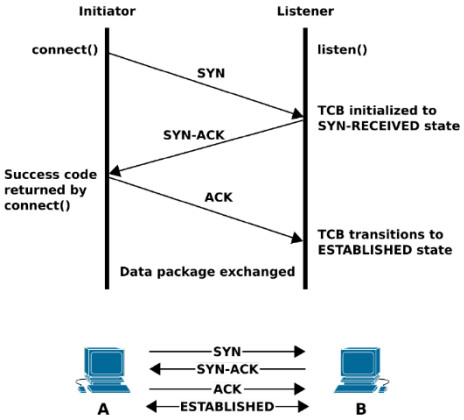
- Уеб браузъри и сървъри
- E-mail
- Обмен на файлове (FTP)

Сред основните характеристики на TCP са:

1. Осигурява **двупосочна дуплексна връзка** (*full-duplex*)
2. Реализира взаимодействие в режим с **установяване на логическо съединение** (*statefulness*)
3. **Управление на потока** (*flow control*) – синхронизацията на скоростта на предаване на данните между изпращача и получателя. Постига се чрез полето *window size*, което определя какво количество данни да бъдат изпратени от изпращача преди да бъде получено потвърждение. Първоначалният *window size* се установява на база MSS (Maximum Segment Size)

4. За **идентификация на партньорите** при взаимодействие на транспортния слой се използват 16 битови номера на **портовете**
5. Поддържат се редица механизми за осигуряване на **надеждно предаване на данните**, но не и за гарантирано време на доставяне.

При транспортния протокол TCP **процесът за договаряне** (установяване на съединение) между две устройства често се нарича **трикратно договаряне** (ръкостискане). При този процес се установява сесия между два възела от мрежата преди да започне изпращането на същинските данни. Използват се съобщения за потвърждаване и отговор – **SYN**, **SYN-ACK**, **ACK сегменти**.



1. **Страната инициатор** на съединението (клиент, възелът A) **изпраща SYN** сегмент (вдигнат бит SYN в полето за флагове на заглавието на TCP), посочвайки **името на домейна** (IPdst) и **номера на порта** на сървъра, към който клиентът A иска да се присъедини, и **началния номер на последователността на клиента**, от който започват сегментите, които той ще изпраща (полето *Sequence Number* в заглавието на TCP).
2. **Сървърът** (възел B) **получава синхронизиращия пакет**. След това той **отговаря** на възела A чрез **изпращането на потвърждаващ** TCP **пакет SYN-ACK**. Сегментът SYN съдържа **своя начален номер на последователността на сървъра**, от който ще започнат сегментите изпратени от получателя. Заедно с вдигнат флаг SYN е вдигнат и флаг ACK с попълнено **полето „номер на потвърждението“,** където се записва **полученият от клиента номер на последователността + 1**.
3. **Клиентът** (възелът A) **получава** този **потвърждаващ пакет** от възела B и трябва да го **потвърди** с **изпращането** и използването на **ACK флага** и с нова стойност в полето за потвърждение (**получения от сървъра номер на последователността + 1**).
4. **Сървърът** (възелът B) **получава** този **ACK пакет**.
5. **TCP връзката** е **установена** и е възможен **двустранен обмен на данни** по установеното съединение.

Хипертекстов протокол HTTP

HTTP (*Hyper Text Transfer Protocol*) е протокол за предаване на хипертекст. Той е символно-ориентиран клиент-сървърен протокол на приложния слой без запазване на състоянието, използван от услугата *World Wide Web*.

При **HTTP протокола** има понятия като *клиент* (обикновено това са уеб браузърите) и *сървър* (това са уеб сървъри). HTTP най-често се използва редом с TCP/IP, но практически може да работи върху всякакъв протокол, който предоставя надежден транспорт. Обикновено HTTP

протоколът работи върху стандартен TCP сокет, отворен от клиента към сървъра. Стандартният порт за HTTP е 80, но може да се използва и всеки друг TCP порт.

Основният **обект за работа** на HTTP е **ресурса**, към който сочи URI (*Uniform Resource Identifier* – уникален идентификатор на ресурса) в заявката на клиента. Основните ресурси се съхраняват във *файлове на сървъра*, но могат да бъдат и други *логически* (напр. каталог на сървъра) или *абстрактни обекти* (напр. ISBN). HTTP протоколът позволява да се зададе начина на представяне (кодиране) на един и същи ресурс според различни параметри: mime-тип, език и други. Така клиентът и уеб сървърът могат да си обменят двоични данни, въпреки че този протокол е текстов.

Структурата на протокола определя, че всяко HTTP съобщение се състои от три части, които са предават в следната последователност:

1. **Стартов ред** (*Starting line*) – определя типа на съобщението;
2. **Заглавия** (*Headers*) – характеризират тялото на съобщението, параметрите на предаването и други сведения;
3. **Тяло на съобщението** (*Message Body*) – конкретните данни на съобщението. То задължително трябва да се отделя от заглавието с празен ред.

Стартовият ред е задължителен елемент, понеже определя типа на заявката/отговора. Заглавията и тялото на съобщението могат да отсъстват. Стартовите редове се различават за заявката и отговора.

Редът на заявката изглежда по следния начин: Метод URI $HTTP$/Версия на протокола

Стартовият ред на отговора на сървъра има следния формат:

$HTTP$/Версия Код на състоянието [Пояснение]

Методът описва вида на HTTP заявката, изпратена от клиента. HTTP дефинира 9 метода, които могат да се използват за извършване на действие от конкретен тип върху заявения ресурс:

1. **GET** – заявява репрезентация на конкретен ресурс. Използва се само за извличане на данни;
2. **HEAD** – същият метод като GET, но без да изисква изпращане на съдържание (response body), т.е. само хедър (*response header*);
3. **POST** – на базата на *request body* променя (добавя) заявения ресурс, ако съществува или го създава в противен случай;
4. **PUT** – заменя всяко представяне на указания адрес с това, което е в *request body*;
5. **PATCH** – частично модифицира заявения ресурс на база на *request body*;
6. **DELETE** – изтрива указания ресурс;
7. **OPTIONS** – връща поддържаните HTTP методи за заявения ресурс;
8. **TRACE** – прави тест на пътя към заявения ресурс, т.е. дали сървърът е получил успешно съобщението.

Наименованията на методите носят само семантичност на заявките. Какво точно ще бъде действието от конкретната заявка, зависи от предоставената имплементация от програмиста.

Request URI определя ресурса, над който ще оперира заявката. Могат да се използват два вида идентификатори: *URI идентификатор* или *релативен път спрямо главната директория на уеб сървъра*. **URI** (*Uniform Resource Identifier*) е идентификатор на ресурс, определен или по местоположение чрез **URL** (*Uniform Resource Locator*, например *http://www.example.com/folder/page.html*), или по име чрез **URN** (*Uniform Resource Name*). **Релативният път спрямо главната директория на уеб сървъра** задава местоположението на ресурс в рамките на текущия уеб сървър. Това е частта от URL-а, която стои след името на хоста (сървъра) в URL идентификатора, например */folder/page.html*.

Държавен изпит – спец. „Софтуерно инженерство“

Тема №9 – Бази от данни. Релационен модел на данните.

Анотация: *Релационен модел на данните: домейн; релация; кортежи; атрибути; схема на релация; схема на релационна база от данни; реализация на релационната база от данни; видове операции върху релационната база от данни; заявки към релационната база от данни. Релационна алгебра: основни (обединение; разлика; декартово произведение; проекция; селекция) и допълнителни (сечение; частно; съединение; естествено съединение) операции.*

Релационен модел на данните: домейн, релация, кортежи, атрибути, схема на релация, схема на релационна база от данни

Всяка база от данни е изградена на базата на модел, който определя логическата ѝ структура и по какъв начин данните се съхраняват, организират и манипулират. Някои от моделите са „същност-отношение“ (Entity-Relationship Model), обектно-ориентиран модел (Object-Oriented Model) и релационен модел (Relational Model).

Базите от данни са създадени да съхраняват данни за голям период от време. Данните в тях се управляват от система за управление на бази от данни (СУБД), като например MySQL, Microsoft SQL Server, Oracle DB, Maria DB и др.

Релационният модел ни дава един-единствен начин за представяне на данните – като двумерни таблици, наречени **релации**. В следващата таблица е представена една примерна релация, наречена **Movies**. Всеки неин ред представя информация за даден филм, всяка нейна колона – свойство на филм. В следващите редове ще въведем терминологията, свързана с релационния модел, като използваме тази релация.

Movies

title	*year*	*length*	*genre*
Gone With the Wind	1939	231	drama
Star Wars	1977	124	sciFi

Колоните на дадена релация се наричат **атрибути**. На примерната релация това са *title*, *year*, *length* и *genre*. **Атрибутите** се поставят на върха на съответната колона. Обикновено, даден атрибут описва значението на стойностите на колоната отдолу. Например, колоната с атрибут *length* съдържа дължината (в минути) на всеки филм.

Името на релация и множеството от атрибутите на релация се нарича **схема на релацията**. Означаваме схемата на дадена релация като запишем името на релацията, следвано от списъка на нейните атрибути в скоби. Следователно, схемата на релацията **Movies** от примера е:

Movies(title, year, length, genre)

Атрибутите от дадена схема на релация са **множество**, а не списък, т.е. няма наредба между тях. Въпреки това, за да можем да говорим за релации ни се налага често да въведем някаква **„стандартна“ наредба** за съответните атрибути. Следователно, всеки път, когато въведем схема на релацията със списък от атрибути като от примера, ще приемаме, че тази подредба е

стандартната подредба, когато се налага да представяме релацията или части от нейните редове.

В релационния модел дадена база от данни се състои от една или повече релации. Множеството от отделните схеми на всяка от тези релации на базата от данни се нарича **схема на релационна база от данни** или просто **схема на базата от данни**.

Редовете на дадена релация, освен заглавния ред, съдържащ имената на атрибутите ѝ, се наричат **кортежи**. **Кортежът** има по един компонент за всеки атрибут на релацията. Например, първият от двата кортежа от примера има следните четири компонента: Gone With the Wind, 1939, 231 и drama за съответните атрибути: *title*, *year*, *length* и *genre*. Когато искаме да запишем даден кортеж отделно, а не като част от релацията, обикновено използваме запетая, за да разделим отделните компоненти и използваме скоби, за да оградим кортежа. Например:

(Gone With the Wind, 1939, 231, drama)

е първият кортеж от примера. Можем да забележим, че когато записваме кортеж в изолация, атрибутите не се записват, така че някаква индикация за релацията, на която кортежът принадлежи, трябва да бъде дадена. Трябва винаги да използваме реда, в който атрибутите са записани последователно в релационната схема.

Релационният модел изисква всеки компонент от даден кортеж да бъде **атомарен**, т.е. да бъде от някой елементарен тип като целочислен (**integer**) или низ (**string**). Не е позволено стойността да бъде структура, множество, списък, масив или всеки друг тип, който може да има стойност, която може да бъде разбивана на по-малки компоненти.

Предполага се, че асоцииран с всеки атрибут на релация е **домейн**, който е някакъв елементарен тип и представлява множеството от допустимите стойности на атрибута. Компонентите на всеки кортеж на дадена релация трябва да имат за всеки компонент, стойност, която принадлежи на домейна от съответната колона. Например, кортежите на примерната релация **Movies** трябва да имат първи компонент, който е низ (**string**), втори и трети компонент, които са целочислени (**integer**) и четвърти компонент, чиято стойност е низ (**string**).

Възможно е домейнът (типът данни) да се включи за всеки атрибут към схемата на релацията. Ще го записваме като добавяме двоеточие и съответния тип след всеки атрибут на релацията. Например, можем да запишем схемата на примерната релация **Movies** така:

Movies(title:string, year:integer, length:integer, genre:string)

Реализация на релационна база от данни

Процесът на проектиране на една релационна база от данни се състои от следните стъпки:

1. Определяне на **данните** (обектите и техните характеристики), които ще бъдат съхранявани
2. Определяне на **връзките** между обектите
3. Определяне на **ключовете** и на свързващите колони
4. Определяне на **ограниченията** върху обектите и връзките между тях
5. Отстраняване на евентуални **недостатъци** (например **излишества**)
6. **Реализиране** на базата от данни

Дефинирането на отделните релационни схеми е твърде свободно, но най-често при създаването им се следват следните правила:

- **Всеки клас** от обекти се представя с **релация**, чиято схема включва всички негови атрибути. Всеки кортеж от релацията представя конкретен обект от класа. **Ключовият атрибут** или списък от **ключовите атрибути** на класа (атрибутите, които еднозначно определят всеки кортеж) се приемат за **ключ на релацията**
- **Връзката** между два или повече класове от обекти се представя чрез **релация**, чиято релационна схема включва **ключовите атрибути** на всеки от тези класове от обекти. Това е само **първоначална стъпка**, която може да се опрости в зависимост от връзката чрез привеждане в нормални форми

Ограниченията в релационния модел могат да бъдат:

- **Ограничение по единствена стойност** – изисква стойността в текущия контекст да бъде уникална. Източник на такива ограничения са ключовете и връзките много-един. Ключовите атрибути са тези, които уникално идентифицират всеки ред от таблицата. В базите от данните всяка релация трябва да има ключ (т.нар. **първичен ключ**);
- **Ограничение по референтна цялост** – изисква стойността, към която се реферира, да съществува. В базите от данни тези ограничения се наричат **външни ключове**;
- **Ограничения по домейн** – изисква стойностите на даден атрибут да са от конкретно изброено множество или в даден интервал;
- **Други ограничения** – междинни проверки, които се съхраняват в базата от данни.

Видове операции върху релационната база от данни. Заявки към релационната база от данни

Както беше посочено, една реална релация не може да бъде статична, а се променя с времето. **Възможни са два вида операции върху релационната база от данни:**

1. Операции, които **променят броя на кортежите или ги видоизменят**. Това са операциите **SELECT**, **INSERT**, **UPDATE**, **DELETE** и те са част от **DML** (*Data Manipulation Language*) – език за манипулиране на данни, който е част от езика **SQL** и който се използва за извличане, вмъкване, обновяване или изтриване на данни (кортежи) от таблиците в СУБД. След тяхното изпълнение промените не се запазват, ако не се направи COMMIT. Обикновено, това се налага само, ако експлицитно е посочено началото на транзакция и се следи изпълнението ѝ. В случая, когато се пишат обикновени заявки, това не се налага.

Примерни заявки:

1) SELECT *
   FROM Movies;
   *Извлича всички кортежи (филми) от релацията **Movies**, като в резултатната таблица ще са всички атрибути на релацията.*

title	*year*	*length*	*genre*
Gone With the Wind	1939	231	drama
Star Wars	1977	124	sciFi

2) SELECT title, year
FROM Movies;
*Извлича всички кортежи (филми) от релацията **Movies**, но в резултатната таблица ще са само атрибутите title и year (**проекция в SQL**).*

title	*year*
Gone With the Wind	1939
Star Wars	1977

3) SELECT title, year
FROM Movies
WHERE year < 2000 AND title LIKE "S%";
*Извлича всички кортежи (филми) от релацията **Movies**, чиито атрибут year е със стойност по-малка от 2000 и атрибут title започва с буквата S (**селекция в SQL**). В резултатната таблица ще присъстват само атрибутите title и year.*

title	*year*
Star Wars	1977

4) INSERT INTO Movies(title, year, length, genre)
VALUES("Death on the Nile", 2021, 127, mystery);
*Добавя нов кортеж (филм) в релацията **Movies** с посочените атрибути. Ако даден атрибут липсва, то неговата стойност се приема за NULL. Актуализираното състояние на релацията **Movies** е следното:*

title	*year*	*length*	*genre*
Gone With the Wind	1939	231	drama
Star Wars	1977	124	sciFi
Death on the Nile	2021	127	mystery

5) UPDATE Movies SET year = 2022
WHERE title LIKE "Death on the Nile";
*Актуализира годината на излизане на филма с име "Death on the Nile", като я променя на 2022. Актуализираното състояние на релацията **Movies** е следното:*

title	*year*	*length*	*genre*
Gone With the Wind	1939	231	drama
Star Wars	1977	124	sciFi
Death on the Nile	2022	127	mystery

6) DELETE FROM Movies
WHERE year < 1960;
*Изтрива всички филми (кортежи), чиято година на излизане е по-малка от 1960 г. Актуализираното състояние на релацията **Movies** е следното:*

title	*year*	*length*	*genre*
Star Wars	1977	124	sciFi
Death on the Nile	2022	127	mystery

2. Операции, които **променят схемата на релацията**. Това са операциите **CREATE**, **DROP**, **ALTER** и те са част от **DDL** (*Data Definition Language*) – език за дефиниране на данни, който е част от езика **SQL** и който се използва за създаване, изтриване или модифициране на таблици и други обекти в базата от данни (изгледи, тригери, процедури, функции и др.). Когато се изпълни една DDL операция, промените се прилагат незабавно в повечето случаи.

Примерни заявки:

1)
```
CREATE TABLE Students(
id INT PRIMARY KEY,
first_name VARCHAR(50),
last_name VARCHAR(50),
age INT
);
```
Създава таблица (релация) със следната схема: ***Students****(id, first_name, last_name, age), ключов атрибут id и атрибут age от целочислени типове и атрибути first_name и last_name с ограничение на дължината до 50 символа. Новата релация* ***Students*** *изглежда по следния начин:*

Students

id	*first_name*	*last_name*	*age*

2)
```
ALTER TABLE Students
ADD address VARCHAR(70);
```
Добавя атрибут address, който е с ограничение до 70 символа, в релацията ***Students****. Актуализираната релация* ***Students*** *изглежда по следния начин:*

Students

id	*first_name*	*last_name*	*age*	*address*

3)
```
ALTER TABLE Students
DROP COLUMN age;
```
Премахва атрибута age от релацията ***Students****. Актуализираната релация* ***Students*** *изглежда по следния начин:*

Students

id	*first_name*	*last_name*	*address*

4)
```
DROP TABLE Students;
```
Изтрива таблицата (релацията) ***Students****.*

Релационна алгебра: основни (обединение, разлика, декартово произведение, проекция, селекция) и допълнителни (сечение, частно, съединение, естествено съединение) операции

През 1970 г., като допълнение на по-рано предложения от него релационен модел, Едгар Код въвежда и операциите върху релации, или т.нар. **релационна алгебра**. Релационната алгебра се състои от оператори и атомарни операнди. Операндите са релации, а операторите – операциите, които могат да бъдат извършвани върху операндите. Резултатът от прилагане на оператор върху релации е също релация. **Петте основни операции** върху релации са: **обединение**, **разлика**, **декартово произведение**, **селекция** и **проекция**.

Операциите в релационната алгебра могат условно да бъдат разделени на четири типа:

- **Множествени операции върху релации** – включват операциите *обединение*, *сечение* и *разлика*;
- **Операции, които премахват част от релацията** – включват операциите *селекция* и *проекция*;
- **Операции, които съединяват кортежите на две релации** – включват операциите *декартово произведение*, *естествено съединение* и *θ-съединение*;
- **Други операции** – включват операциите *преименуване*, *оператор за премахване на дубликати*, *агрегатни оператори*, *оператор за групиране*, *оператор за сортиране* и др.

Обикновено ще наричаме изрази на релационната алгебра като заявки.

Операциите обединение, сечение и разлика се прилага върху релации, които имат еднакви схеми и еднакви домейни за всеки атрибут. Преди да се изчисли резултатът от операцията, колоните на двете релации трябва да бъдат подредени, така че редът на атрибутите да е един и същ. Преименуващият оператор е много подходящ, когато искаме да приложим някой от операторите обединение, сечение и разлика върху две релации, които имат равен брой атрибути с еднакви домейни, но с различни имена.

Нека **R1** и **R2** са две релации.

R1

name	*gender*	*age*
Ivan Ivanov	M	21
Victoriya Petrova	F	22

R2

name	*gender*	*age*
Victoriya Petrova	F	22
Stefan Dimitrov	M	22

1. **Обединението** на двете релации **R1** и **R2** (бележим $R1 \cup R2$) също е релация, която се състои от всички кортежи на релацията R1 и всички кортежи на релацията R2. Ако в R1 и R2 има един и същ кортеж, то той се появява само веднъж в резултатната релация. Операцията е асоциативна и комутативна.

$$\boldsymbol{R1 \cup R2}$$

name	*gender*	*age*
Ivan Ivanov	M	21
Victoriya Petrova	F	22
Stefan Dimitrov	M	22

2. **Сечението** на двете релации **R1** и **R2** (бележим $R1 \cap R2$) също е релация, която се състои от всички общи кортежи на релациите R1 и R2. Ако в R1 и R2 има един и същи кортеж, то той се появява само веднъж в резултатната таблица. Операцията е асоциативна и комутативна.

$$\boldsymbol{R1 \cap R2}$$

name	*gender*	*age*
Victoriya Petrova	F	22

3. **Разликата** на двете релации **R1** и **R2** (бележим $R1 \setminus R2$) също е релация, която се състои от всички кортежи на релацията R1, които не са в релацията R2. Операцията не е комутативна.

$$\boldsymbol{R1 \setminus R2}$$

name	*gender*	*age*
Ivan Ivanov	M	21

4. **Проекцията** на релацията **R1** (бележим $\pi_{A_1,A_2,\ldots,A_n}(R1)$) също е релация, която се състои от подмножество на атрибутите на релацията R1. Проектиращият оператор се използва, когато искаме да създадем нова релация от релацията R1, която използва само част от нейните атрибути.

$$\boldsymbol{\pi_{name}(R1)}$$

name
Ivan Ivanov
Victoriya Petrova

5. **Селекция** от релацията **R1** при условие C (бележим $\sigma_C(R1)$) също е релация, която се състои от подмножество на кортежите на релацията R1. В резултатната селекция попадат само тези кортежи, които отговарят на дадено условие C.

$$\boldsymbol{\sigma_{age \leq 21}(R1)}$$

name	*gender*	*age*
Ivan Ivanov	M	21

6. **Декартовото произведение** на релациите **R1** и **R2** (бележим $R1 \times R2$) също е релация. Тя се състои от кортежи, получени от съединението (слепването) на всеки кортеж от релацията R1 с всеки кортеж от релацията R2. Ако някои от атрибутите на R1 и R2 имат общи имена, тогава е необходимо да ги преименуваме, така че всички имена на атрибути да са единствени. За да няма двусмислие относно атрибут A, който е едновременно и в R1, и в R2, в резултатната релация се използват означенията R1.A и R2.A, за да е определено кой атрибут от коя схема е взет. В общия случай, ако релацията R1 има N1 атрибута и T1 на брой кортежи, а релацията R2 има N2 атрибута и T2 на брой кортежи, то декартовото произведение на релациите R1 и R2 ще има **N1 + N2 атрибута** и **T1 * T2 кортежи**.

S1

major
SE
M

R1 x S1

name	*gender*	*age*	*major*
Ivan Ivanov	M	21	SE
Victoriya Petrova	F	22	SE
Ivan Ivanov	M	21	M
Victoriya Petrova	F	22	M

7. **Естествено съединение** на две релации **R1** и **T** (бележим $R1 \bowtie T$) е нова релация, чиито кортежи са всевъзможни съединения на кортеж от R1 с кортеж от T, които се съгласуват по общите атрибути на релациите R1 и T.

T

age	*major*
21	SE
22	M

R1 ⋈ T

name	*gender*	*age*	*major*
Ivan Ivanov	M	21	SE
Victoriya Petrova	F	22	M

8. **Тита-съединение** на две релации **R1** и **T** при условие C (бележим $R1 \overset{\bowtie}{_C} T$) е нова релация, чиято схема е обединение на схемите на R1 и T, като при това се извършва преименуване на атрибутите, които се дублират. C е условен израз, подобен на условния израз от селекцията, но в него не могат да участват имена на атрибути и на R1, и на T. Кортежите на $R1 \overset{\bowtie}{_C} T$ получаваме по следния начин:
   a. Образуваме релацията $R1 \times T$;
   b. Избираме онези кортежи на $R1 \times T$, които удовлетворяват условието C (след заместване на имената на атрибутите със съответните стойности)

$$\boldsymbol{R1} \overset{\bowtie}{_{\boldsymbol{R1.age = T.age\ AND\ gender = 'M'}}} \boldsymbol{T}$$

name	*gender*	*R1.age*	*T.age*	*major*
Ivan Ivanov	M	21	21	SE

9. **Операторът за частно** на две релации **A** и **B** (бележим $A \div B$ или A/B или $A : B$) може да бъде приложено тогава и само тогава, когато:
   a. Атрибутите на B са изцяло подмножество на атрибутите на A;
   b. Релацията, която се връща от оператора за частно ще има атрибути, представляващи всички атрибути от A без атрибутите на B;
   c. Релацията, която се връща от оператора за частно ще върне тези кортежи от релацията A, които са свързани (съдържат стойностите на атрибутите) с всеки кортеж на B. Казано по друг начин, резултатната релация ще има свойството, че за всяко $t \in A:B$ и всяко $s \in B$, конкатенацията $ts \in A$ (без значение на последователността).

A

x	*y*
A	1
B	2
A	2
D	4

B

y
1
2

A : B

x
A

10. **Преименуване** (бележим $\rho_{R1(A_1,\ldots,A_n)}(R1)$) – тази операция не оказва влияние върху конструкцията на релацията, а само променя имената на таблиците или на техните атрибути. $R1 \coloneqq \rho_{R1(A_1,\ldots,A_n)}(R2)$ превръща R1 в релация с атрибути $A_1, \ldots, A_n$ и същите кортежи като на R2.

$$\boldsymbol{\rho_{R1(name,sex,age)}(R1)}$$

name	*sex*	*age*
Ivan Ivanov	M	21
Victoriya Petrova	F	22

Мултимножествата (bags) са множества, които позволяват повтаряне на кортежи. Съвременните СУБД рядко се базират на чисти множества и позволяват операции над мултимножества, т.е. на множества, в които може да има повтарящи се кортежи. Работата с мултимножества облекчава системата за управление на база от данни, защото резултатната релация не трябва да се сортира и да се премахват дубликати скрито от нас. Повечето оператори се прилагат по същия начин, както при множества с тази разлика, че резултатната релация може да съдържа повтарящи се кортежи.

Други операции в релационната алгебра могат да бъдат:

- **Оператор за премахване на дубликати ($\boldsymbol{\delta}$)** – превръща мултимножества в множества
- **Групиране на кортежи ($\boldsymbol{\Upsilon}$)** – според стойностите по един или повече атрибута
- **Агрегатни оператори** – $\min(\square)$, $\max(\square)$, $\text{avg}(\square)$
- **Оператор за сортиране ($\boldsymbol{\tau}$)** – по един или няколко атрибута
- **Разширена проекция** – аритметични изрази върху проектираните колони
- **Външни съединения** – $R \bowtie S$, $R \bowtie_L S$, $R \bowtie_R S$, $R \bowtie_C S$

Държавен изпит – спец. „Софтуерно инженерство“

Тема №10 – Бази от данни. Нормални форми.

Анотация: *Нормални форми. Проектиране схемите на релационните бази от данни. Аномалии, ограничения, ключове. Функционални зависимости, аксиоми на Армстронг. Първа, втора, трета нормална форма, нормална форма на Бойс-Код. Многозначни зависимости; аксиоми на функционалните и многозначните зависимости; съединение без загуба; четвърта нормална форма.*

Проектиране на схемите на релационните бази от данни

Транслацията на някакъв модел (например същност-отношения, XML и др.) на база от данни до съвкупност от релации, над чиито атрибути са дефинирани зависимости, се нарича **проектиране на схема на релационна база от данни**. Важни цели на този процес са да запази целостта на данните, както и да нормализира получените релации.

Има много начини, по които може да подходим в случаите, когато проектираме схема на релационна база от данни за дадено приложение. За щастие, има добре разработена **теория**, свързана с релационните бази от данни – определянето на зависимости и заключенията от тях за това какво значи добра схема на релационна база от данни и какво може да направим, когато конкретният случай не е такъв.

Нормализацията на релационна база от данни е процесът на реструктуриране на релациите и зависимостите на техните атрибути в нормални форми, които целят минимизация на дублирането на данните, както и запазването на тяхната цялост. Възможни начини за извършването ѝ са синтез, т.е. създаване на нов релационен дизайн, и декомпозиция на вече съществуващ такъв.

Функционални зависимости – аксиоми на функционалните зависимости

Има теория за проектиране на релации, която ни позволява да изследваме дизайна внимателно и да направим подобрения, основавайки се на няколко прости принципа. Теорията започва с това да открием **ограниченията**, които се отнасят за съответната релация. Най-често срещаното ограничение са **функционалните зависимости**, понятие, което генерализира идеята за ключ на релация.

Функционална зависимост (ФЗ) в една релация R наричаме твърдение от следния вид: ако два кортежа имат едни и същи компоненти, отговарящи на атрибутите $A_1, A_2, \dots, A_n$, то те имат и едни и същи компоненти, отговарящи на друг списък от атрибути $B_1, \dots, B_m$ и записваме формално: $A_1A_2 \dots A_n \rightarrow B_1B_2 \dots B_m$ и казваме, че $A_1, \dots, A_n$ функционално определя $B_1, \dots, B_m$.

Ако можем да бъдем сигурни, че всяка инстанция на релация R ще бъде такава, че дадена функционална зависимост е вярна, тогава казваме, че R удовлетворява функционалната зависимост. Само по себе си, това налага вид ограничение над R. Много често дясната страна на дадена функционална зависимост е само един атрибут.

Пример: *Нека да разгледаме следната релация:*

Movies(title, year, length, genre, studioName, starName)

Функционална зависимост за горната релация е:

title year → length genre studioName

Ако два кортежа имат еднакво заглавие и еднаква година на излизане, то тогава те ще имат еднакви стойности и на атрибутите за дължина за филма, жанр и име на филмово студио.

Невярна функционална зависимост за горната релация е:

title year → starName

Не е вярно, че за всеки два филма с едно и също име и година на излизане на екран, ще има и съвпадение на стойността за актьор. Ако това беше вярно, то следва, че за всеки филм в релацията съществува само един актьор, който участва в него, което е невярно твърдение.

Съществуват **правила** (аксиоми), чрез които от даден набор функционални зависимости в релация можем да извличаме нови функционални зависимости в тази релация.

Функционалните зависимости в една релация често могат да бъдат представени по различен начин. По-формално казваме, че множествата S и T от функционални зависимости са **еквивалентни**, ако множеството от екземплярите на релацията, удовлетворяваща S съвпада с множеството от екземплярите на релацията, удовлетворяващи T. По-общо, множеството функционални зависимости S следва от множеството от функционални зависимости T, ако всеки екземпляр на релацията, който удовлетворява T, удовлетворява и S. Естествено, S следва T и T следва S тогава и само тогава, когато S и T са еквивалентни.

Нека е дадена функционална зависимост $A_1A_2 \dots A_n \to B_1B_2 \dots B_m$. Тогава можем да заменим тази функционална зависимост със следните функционални зависимости: $A_1A_2 \dots A_n \to B_i$, $i = 1, \dots, m$. Това правило наричаме **правило за разделяне**.

Обратно, функционалните зависимости $A_1A_2 \dots A_n \to B_i, i = 1, \dots, m$ можем да заменяме с една функционална зависимост $A_1A_2 \dots A_n \to B_1B_2 \dots B_m$. Това правило наричаме **правило за комбиниране**.

Очевидно е, че и в двата случая полученото множество функционални зависимости е еквивалентно на изходното.

Казваме, че функционалната зависимост $A_1A_2 \dots A_n \to B$ е **тривиална**, ако $B = A_i$ за някое $i = 1, \dots, n$. Естествено, тривиалните функционални зависимости са изпълнени във всяка релация. По-общо, казваме, че функционалната зависимост $A_1A_2 \dots A_n \to B_1B_2 \dots B_m$ е **тривиална**, ако $\{B_1, B_2, \dots, B_m\} \subseteq \{A_1, A_2, \dots A_n\}$. Казваме, че функционалната зависимост $A_1A_2 \dots A_n \to B_1B_2 \dots B_m$ е **нетривиална**, ако съществува $i = 1, \dots, m$, такова че $B_i \notin \{A_1, \dots, A_n\}$. Казваме, че функционалната зависимост $A_1A_2 \dots A_n \to B_1B_2 \dots B_m$ е **напълно нетривиална**, ако за всяко $i = 1, \dots, m$, такова че $B_i \notin \{A_1, \dots, A_n\}$.

Пример:

1. *Функционалната зависимост title year → title е тривиална.*
2. *Функционалната зависимост title year → year length е нетривиална.*
3. *Функционалната зависимост title year → length е напълно нетривиална.*

Нека $A_1A_2 \dots A_n \rightarrow B_1B_2 \dots B_m$ е нетривиална функционална зависимост. Тогава можем да премахнем онези $B_i, i = 1, \dots m$, за които B_i съвпада с някое A_j. Получаваме нова функционална зависимост $A_1A_2 \dots A_n \rightarrow C_1C_2 \dots C_k, k < m$, която очевидно е еквивалентна на изходната и е напълно нетривиална. Това правило наричаме **правило за отстраняване на тривиалните зависимости**.

Нека $A_1A_2 \dots A_n \rightarrow B_1B_2 \dots B_m$ и $B_1B_2 \dots B_m \rightarrow C_1C_2 \dots C_k$ са функционални зависимости. Тогава добавяме нова функционална зависимост $A_1A_2 \dots A_n \rightarrow C_1C_2 \dots C_k$. Това правило наричаме **транзитивно правило**. Ясно е, че добавената функционална зависимост следва от изходните функционални зависимости, така че получаваме еквивалентно множество от функционални зависимости.

***Пример**: За релацията Movies са в сила следните две функционални зависимости:*

title year → studioName

studioName → studioAddress

По транзитивното правило получаваме, че е в сила за тази релация и следната функционална зависимост:

title year → studioAddress

Ключове

Ще определим понятието **ключ** в релационния модел. Казваме, че множеството $\{A_1, A_2, \dots, A_n\}$ от един или повече атрибути на релацията R образува **ключ** на R, ако $A_1, A_2, \dots, A_n$ функционално определят всички атрибути на R и $\{A_1, A_2, \dots, A_n\}$ е минимално по включване с това свойство. Ако ключ се състои от един единствен елемент, то не поставяме фигурни скоби.

***Пример**: За релацията Movies забелязваме, че {title, year, starName} е ключ.*

Възможно е една релация да има повече от един ключ. Обикновено, в такава ситуация се избира един ключ, който се обявява за **първичен ключ**. Първичният ключ се използва при съхранение на релацията. Естествено, добре е първичният ключ да съдържа колкото се може по-малко атрибути. За минималност на ключа можем да съдим само ако разполагаме с пълния набор от функционални зависимости между атрибутите.

***Пример**: За релацията Movies:*

Movies(title, year, length, type, studioName, starName)

Суперключ на една релация R се нарича множество от атрибути, което съдържа ключ. С други думи, атрибутите на суперключа трябва функционално да определят всички атрибути на R, но не е задължително суперключът да е минимален. Естествено, всеки ключ е суперключ, но обратното не е вярно.

Аномалии

Проблеми като излишества и др., които се случват, когато се опитаме да съберем твърде много информация в една единствена релация, се наричат **аномалии**. Видовете аномалии, които срещаме, са:

- **Излишества** – повторение на информацията в няколко кортежа, без това да е необходимо
  Пр.: За релацията Movies това са дължината и жанра на филмите.
- **Аномалии при обновяване** – получават се, когато променим информация в един кортеж, но оставим същата информация непроменена в друг. Така се стига до неконсистентност на информацията.
  Пр.: За релацията Movies ако променим дължината на даден филм само в един кортеж.
- **Аномалии при добавяне** – получават се, когато искаме да добавим нова информация в базата данни, но все още нямаме информация за дадена ключова стойност или искаме да я добавим след това
- **Аномалии при изтриване** – получават се, когато се опитаме да изтрием информация от релацията, но това не може да се случи без премахването на друга важна информация
  Пр.: За релацията Movies ако изтрием кортеж за даден филм само защото името на актрисата е грешно и той е единствен за момента за дадения филм, то и последният кортеж за този филм ще изчезне и с това информацията за дължината на филма и неговия жанр.

Общоприет начин за елиминиране на изброените аномалии е **декомпозиция** на релациите – една релация се разбива на две нови. По-формално, нека $R(A_1, A_2, \dots, A_n)$ е релация. Тогава тя се декомпозира на две нови релации $S(B_1, B_2, \dots, B_m)$ и $T(C_1, C_2, \dots, C_k)$, ако:

1. $\{B_1, B_2, \dots, B_m\} \cup \{C_1, C_2, \dots, C_k\} = \{A_1, A_2, \dots, A_n\}$.
2. Кортежите в S са проекции на кортежите на R по $B_1, B_2, \dots, B_m$.
   Това означава, че от всеки кортеж на екземпляр на R избираме само компонентите, които съответстват на $B_1, B_2, \dots, B_m$ и по този начин получаваме кортежите на S. Естествено, от два различни кортежа на R могат да се получат две еднакви проекции, но в екземпляра на S поставяме само едната от тях и по този начин се премахва дублирането на информация.
3. Аналогично, кортежите в T са проекции на кортежите на R по $C_1, C_2, \dots, C_k$.

Ограничения

Ограниченията са механизъм за налагане на цялост върху данните чрез дефиниране на правила, на които трябва да отговарят стойностите на конкретни атрибути. Те не позволяват въвеждането на невалидни данни.

Основните ограничения са:

- **Уникалност на множество от атрибути** – два кортежа не могат да имат едни и същи стойности на атрибутите от множеството. По дефиниция, първичните ключове са уникални.

В **SQL** обичайно се имплементират чрез ключовите думи **UNIQUE** и **PRIMARY KEY**.

- **Задаване на домейн от възможни стойности на множество от атрибути** – задават се чрез условни изрази.
  В **SQL** обичайно се имплементират чрез ключовите думи **CHECK** и **NOT NULL** (не се допуска стойността на атрибут да е с null стойност).
- **Външен ключ** – налага връзка между данните в две таблици. За всяка ненулева стойност на атрибут, който е външен ключ, трябва да съществува ключов атрибут от друга релация, който да съдържа същата стойност.
  В **SQL** обичайно се имплементира чрез ключовата дума **FOREIGN KEY**.

Освен тези ограничения, друг широк клас от ограничения за цялостност въведохме с помощта на понятието за функционална зависимост.

Аксиоми на Армстронг

Нека $\{A_1, A_2, \dots, A_n\}$ е множество от атрибути и S е множество от функционални зависимости. **Покритие** на $\{A_1, A_2, \dots, A_n\}$ относно S е множеството от всички атрибути B, такива че всеки екземпляр на релацията, който удовлетворява S, удовлетворява и функционалната зависимост $A_1A_2 \dots A_n \to B$, т.е. от функционалните зависимости в S следва $A_1A_2 \dots A_n \to B$. Означаваме покритието по следния начин: $\{A_1, A_2, \dots, A_n\}^+$. Естествено, $\{A_1, A_2, \dots, A_n\} \subseteq \{A_1, A_2, \dots, A_n\}^+$, тъй като тривиалните функционални зависимости винаги са изпълнени.

Ще опишем **алгоритъм за изчисляване на покритието** на $\{A_1, \dots, A_n\}$ относно S:

1. Ако е необходимо, модифицираме множеството от функционални зависимости S, така че всяка функционална зависимост да има един атрибут от дясната страна (по правилата, които показахме по-горе).
2. Инициализираме $X = \{A_1, \dots, A_n\}$.
3. Търсим функционална зависимост $B_1B_2 \dots B_m \to C$ в S, такава че $B_1, B_2, \dots, B_m \in X$, но $C \notin X$. Тогава добавяме C към X.
4. Повтаряме стъпка 3., докато вече не е възможно добавянето на атрибути в X. Този момент гарантирано ще настъпи, тъй като X винаги расте, а множеството от атрибути е крайно.
5. Търсеното покритие $\{A_1, A_2, \dots, A_n\}^+$ съвпада с X.

Функционалната зависимост $A_1A_2 \dots A_n \to B_1B_2 \dots B_m$ следва от S тогава и само тогава, когато $\{B_1, B_2, \dots, B_m\} \subseteq \{A_1A_2 \dots A_n\}^+$.

Както вече видяхме, чрез алгоритъма за намиране на покритие винаги можем да определим дали дадена функционална зависимост следва от дадено множество от функционални зависимости.

Аксиомите на Армстронг са правила, чрез които може да се извлече всяка функционална зависимост, която следва от дадено множество функционални зависимости. Те са следните:

1. **Рефлексивност** – ако $\{B_1, B_2, \dots, B_m\} \subseteq \{A_1, A_2, \dots, A_n\}$, то $A_1A_2 \dots A_n \to B_1B_2 \dots B_m$.
2. **Нарастване (разширение)** – ако $A_1A_2 \dots A_n \to B_1B_2 \dots B_m$ и $C_1, C_2, \dots, C_k$ са произволни атрибути, то $A_1A_2 \dots A_nC_1C_2 \dots C_k \to B_1B_2 \dots B_mC_1C_2 \dots C_k$.
3. **Транзитивност** – ако $A_1A_2 \dots A_n \to B_1B_2 \dots B_m$ и $B_1B_2 \dots B_m \to C_1C_2 \dots C_k$, то $A_1A_2 \dots A_n \to C_1C_2 \dots C_k$.

Нормални форми – първа, втора, трета нормална форма, нормална форма на Бойс-Код

Първа нормална форма (1НФ): Казваме, че релацията R се намира в 1НФ, когато всички компоненти в кортежите на R имат атомарен домейн, т.е. атрибутите ѝ са атомарни. Иска се и да няма повтарящи се кортежи, което е основно изискване за една релация.

Един атрибут X от релация R с множество от ФЗ F се нарича **първичен атрибут**, ако влиза в състава на ключа (първичния ключ). В противен случай се нарича **непървичен**.

Втора нормална форма (2НФ): Казваме, че релацията R се намира във 2НФ, когато тя се намира в 1НФ и всеки непървичен атрибут е функционално зависим от атрибутите на първичния ключ, но не и от негово подмножество.

Нека R е релация със съответното множество от ФЗ F, а X и Z са подмножества на множеството от атрибути A. Казва се, че атрибутът Z е **транзитивно зависим** от X, ако съществува такова множество от атрибути Y, $Y \subset A$, $Z \notin XY$, за което са в сила свойствата: $X \rightarrow Y$, $Y \rightarrow Z$.

Трета нормална форма (3НФ): Казваме, че релацията R се намира в 3НФ, когато за всяка нетривиална ФЗ или лявата страна е суперключ, или дясната е първичен атрибут (част от някой ключ на R – незадължително същия). Казано по друг начин: релация, която е в 1НФ и 2НФ и в която нито един първичен атрибут не е транзитивно зависим от първичния ключ.

Нормална форма на Бойс-Код (НФБК): Казваме, че една релация R се намира в НФБК, когато тя се намира в 3НФ и за всяка нетривиална ФЗ, която е в сила в R е изпълнено, че лявата ѝ част е суперключ на R.

Съединение без загуба

Дефиниция (съединение без загуба): Нека $R(A_1, A_2, \dots, A_n), n \in \mathbb{N}$ е релация и R се декомпозира на релациите $R_1, \dots, R_m, m \in \mathbb{N}$. Казваме, че декомпозицията е със съединение без загуба, ако $R = R_1 \bowtie \cdots \bowtie R_m$, където $\bowtie$ е релацията на естествено съединение (natural join) от релационната алгебра.

Твърдение: Декомпозицията на релацията $R(A_1, \dots, A_n)$ на две релации $S(B_1, \dots, B_m)$ и $T(C_1, \dots, C_k)$ за $n, m, k \in \mathbb{N}$ е съединение без загуба тогава и само тогава, когато за R е изпълнена поне една от следните функционални зависимости: $\{B_1, \dots, B_m\} \cap \{C_1, \dots, C_k\} \rightarrow \{B_1, \dots, B_m\}$ или $\{B_1, \dots B_m\} \cap \{C_1, \dots, C_k\} \rightarrow \{C_1, \dots, C_k\}$.

Нека $R(A_1, \dots, A_n), n \in \mathbb{N}$ е релация, за която множеството от ФЗ е F. Нека R се декомпозира на релациите $R_1, \dots, R_m, m \in \mathbb{N}$, чиито множества от ФЗ са съответно $F_1, \dots, F_m$, като те са съответно проекции на F върху $R_1, \dots, R_m$. Казваме, че декомпозицията е със **съединение без загуба на функционалните зависимости**, когато $F_1 \cup \dots \cup F_m = F$.

Алгоритъм за свеждане на релация в НФБК:

Нека R_0 е релация с множество от функционални зависимости S_0. Прилагаме следните стъпки, за да получим декомпозиция, всяка релация от която е в НФБК:

1. Полагаме $R \leftarrow R_0, S \leftarrow S_0$.
2. Проверяваме дали има нарушения на НФБК. Ако не, връщаме $\{R\}$;

3. Избираме една ФЗ от S, която нарушава НФБК – нека тя бъде от вида $X \to Y$. Намираме X^+. Създаваме две релации – $R_1(X^+)$ и $R_2(X \cup (R \backslash X^+))$. С други думи, първата релация съдържа X и всичко, което е функционално определено от тях, а втората релация – X и всичко, което не е функционално определено от X;
4. Проектираме S в R_1 и R_2 – получаваме S_1 и S_2;
5. Рекурсивно прилагаме стъпка 2 за двойките (R_1, S_1), (R_2, S_2). Обединяваме получените множества релации.

В общия случай, **декомпозицията в НФБК не е със съединение без загуба на функционалните зависимости**. Съществува обаче алгоритъм за декомпозиция в 3НФ, който е със съединение без загуба и запазва ФЗ. Този алгоритъм в повечето случаи се справя с излишествата, породени от функционални зависимости.

Многозначни зависимости – аксиоми на многозначните зависимости

Терминът **многозначна зависимост (МЗ)** се използва, когато два атрибута или множество атрибути са независими помежду си. Това състояние обобщава идеята за ФЗ в този смисъл, че всяка функционална зависимост предполага съответната мултифункционална зависимост.

Многозначна зависимост в една релация R наричаме твърдение от следния вид: ако в екземпляр на R, компонентите на кортежите, отговарящи на атрибутите $A_1, A_2, \dots, A_n$ съвпадат, то компонентите на кортежите, съответни на атрибутите $B_1, B_2, \dots, B_m$ са независими от стойностите на всички останали атрибути.

По-формално, нека R е релация с множество от атрибути $\{A_1, A_2, \dots, A_n, B_1, \dots, B_m, C_1, \dots, C_k\}, n, m, k \in \mathbb{N}$. Казваме, че $\{A_1, \dots, A_n\}$ **определя многозначно** $C_1, \dots, C_k$ тогава и само тогава, когато за всеки два кортежа $u, t \in R$, съвпадащи по атрибутите $\mathrm{A}_1, \dots, \mathrm{A}_n$, съществува кортеж $v \in R$, такъв че:

- Съвпада с кортежите u и t по атрибутите $A_1, \dots, A_n$;
- Съвпада с кортежа u по атрибутите $B_1, \dots, B_m$;
- Съвпада с кортежа t по атрибутите $C_1, \dots, C_k$.

Тоест, ако между два случайни кортежа с еднакви стойности по $A_1, \dots, A_n$ разменим стойностите на атрибутите $C_1, \dots, C_k$ ще получим същата релация. Означаваме като $A_1 \dots A_n \twoheadrightarrow C_1 \dots C_k$.

Съществуват няколко **правила** (аксиоми) за МЗ, които са подобни на тези за ФЗ.

1. МЗ $A_1 \dots A_n \twoheadrightarrow B_1 \dots B_m$ в релацията R наричаме **тривиална**, когато $\{B_1, \dots, B_m\} \subseteq \{A_1, \dots, A_n\}$.
2. **Правило за транзитивност**: Ако $A_1 \dots A_n \twoheadrightarrow B_1 \dots B_m$ и $B_1 \dots B_m \twoheadrightarrow C_1 \dots C_k$, то $A_1 \dots A_n \twoheadrightarrow C_1 \dots C_k$.
3. **Правилото за разделяне и комбиниране** не е в сила за МЗ.

Съществуват някои нови правила за МЗ.

4. Всяка **ФЗ е МЗ**, т.е. ако $A_1 \dots A_n \to B_1 \dots B_m$, то $A_1 \dots A_n \twoheadrightarrow B_1 \dots B_m$.
5. **Правило за допълнение:** Ако $A_1 \dots A_n \twoheadrightarrow B_1 \dots B_m$, то $A_1 \dots A_n \twoheadrightarrow C_1 \dots C_k$, където $C_1, \dots, C_k$ е множеството от всички атрибути на R с изключение на $(A_1 \dots A_n \cup B_1 \dots B_m)$.
6. Ако **всички атрибути** на релация R са $\{A_1, \dots, A_n, B_1, \dots, B_m\}$, то $A_1 \dots A_n \twoheadrightarrow B_1 \dots B_m$ е в R.

Четвърта нормална форма

Четвърта нормална форма (4НФ): Казваме, че релацията R е в четвърта нормална форма (4НФ), ако във всяка нетривиална многозначна зависимост $A_1 \dots A_n \twoheadrightarrow B_1 \dots B_m$, $\{A_1, \dots, A_n\}$ е суперключ.

Излишество, което произтича от МЗ, не може да се отстрани чрез привеждане в НФБК. Необходима е по-строга нормална форма, наречена **четвърта нормална форма (4НФ)**, която третира МЗ като ФЗ по отношение на декомпозицията, но не и по отношение на ключовете. В тази нормална форма всички МЗ са елиминирани, както всички ФЗ, които нарушават НФБК. В резултат декомпозираните релации нямат излишък нито от ФЗ, нито от МЗ. В основата си 4НФ е НФБК, но е приложена върху МЗ вместо върху ФЗ.

Алгоритъмът за декомпозиране от НФБК в 4НФ е аналогичен на този за декомпозиране в НФБК, с разликата, че се декомпозира по МЗ. Когато декомпозираме следвайки този алгоритъм е гарантирано, че декомпозицията е съединение без загуба.



Държавен изпит – спец. „Софтуерно инженерство“

Тема №11 – Процедурно програмиране – основни конструкции.

Анотация: *Изложението по въпроса трябва да включва следните по-съществени елементи:*

1. Принципи на структурното програмиране.

2. Управление на изчислителния процес. Основни управляващи конструкции – условни оператори, оператори за цикъл.

3. Променливи – видове: локални променливи, глобални променливи; инициализация на променлива; оператор за присвояване.

4. Функции и процедури. Параметри – видове параметри. Предаване на параметри – по име и по стойност. Типове и проверка за съответствие на тип

5. Символни низове. Представяне в паметта. Основни операции със символни низове.

Принципи на структурното програмиране

Практиката на програмирането е показала, че когато се съставят програми, най-трудното е откриването на грешки. Една от първите причини за това е прекомерното използване на оператор `goto` в програмите. Пръв Е. Дейкстра през 1965 г. защитава тезата срещу оператора `goto` и се счита, че той полага принципите на структурното програмиране. Негова е мисълта, че операторът `goto` може да бъде „изключен“ от езиците за програмиране.

Основната идея на структурното програмиране е ограничаването на допустимите управляващи структури. На теория е доказано, че са достатъчни само два вида. Според теорията, за да се напише една програма са необходими точно три основни компонента – два управляващи и един „изпълнителен“.

При структурното програмиране има **три структури за управление**:

- **Последователно изпълнение** (в езика C++ операторите се изпълняват последователно)
- **Селектиране** (в езика C++ операторът if е основно средство)
- **Повторение** (изключва се `goto`) (в езика C++ основно средство е цикълът while/for)

Структурното програмиране е един подход за създаване на ясни, добре документирани и структурирани програми.

Принципите на структурното програмиране са:

1. Принцип на **визуалност** и **мнемоника** (краснопис в програмирането)
2. Принцип за **програмиране** без `goto`
3. Принцип за **стандартизация** на управлението
4. Програмата да се **проектира предварително**, преди да се кодира
5. Програмата да съдържа **коментари**
6. Разбиване на една програма на **подпрограми** (функции) – стъпка по стъпка, за да се детайлизира алгоритъмът

Други фундаментални принципи на структурното програмиране са **принципа за модулност** и **принципа за абстракция на данните**. Съгласно принципа за модулност, програмата се разделя на подходящи взаимосвързани части, всяка от които се реализира чрез определени средства. Целта е промените в представянето на данните да не променят голям брой от модулите на програмата. Съгласно принципа за абстракция на данните, методите за използване на данните се отделят от методите за тяхното конкретно представяне. Програмите се конструират, така че да работят с абстрактни данни – данни с неуточнено представяне. След това представянето се конкретизира с помощта на множество функции, наречени конструктори, мутатори и функции за достъп (селектори), които реализират абстрактните данни по конкретен начин.

При структурното програмиране в C и C++, основна програмна единица е **функцията** – инструкции, които се характеризират със сигнатура и тяло и които сме написали сами или вземаме наготово от някаква функция. При обектно-ориентираното програмиране в C++, това е класът.

Управление на изчислителния процес. Основни управляващи конструкции – условни оператори, оператори за цикъл

В някои случаи логиката на програмата напълно съответства на физическия ред на операторите, независимо от входните данни. Това не е типичен случай обаче за повечето реални задачи. Обикновено последователността на изпълняваните оператори или т.нар. **управление на програмата**, зависи от входните данни.

Ще разгледаме управляващите оператори в езика C++.

1. Оператор за присвояване

Това е един от най-важните оператори на езика.

Синтаксис:

```
<променлива> = <израз>;
```

- `<променлива>` е идентификатор, дефиниран вече като променлива;
- `<израз>` е израз от тип, съвместим с типа на `<променлива>`;

Семантика:

Намира се стойността на `<израз>`. Ако тя е от тип, различен от типа на `<променлива>`, конвертира се ако е възможно до него и се записва в именуваната с `<променлива>` памет.

Ще отбележим, че в рамките на една функция не са възможни две дефиниции на една и съща променлива, но на една и съща променлива може да ѝ бъдат присвоявани многократно различни стойности.

В езика C++ са въведени някои съкратени форми на оператора за присвояване. Например, заради честото използване на оператора:

```
a = a + 1;
```

той съкратено се означава с:

```
a++;
```

Въведено е също и съкращението `a--` на оператора `a = a - 1;`

Операторът за присвояване `=` е претоварен и с функцията на дясно асоциативна инфиксна аритметично-логическа операция с приоритет по-нисък от този на дизюнкцията `||`. Това позволява на оператора за присвояване

```
x = y;
```

където `x` е променлива, а `y` – израз, да се гледа като на израз от тип – типа на `x` и стойност – стойността на `y`, ако е от типа на `x`, или стойността на `y`, но преобразувана до типа на `x`.

2. Празен оператор

Това е най-простият оператор на C++.

Синтаксис:

```
;
```

Операторът не съдържа никакви символи. Завършва със знака ;.

Семантика:

Не извършва никакви действия. Използва се, когато синтаксисът на някакъв оператор изисква присъствието на поне един оператор, а логиката на програмата не изисква такъв.

3. Блок

Често синтаксисът на някакъв оператор на езика изисква използването на един оператор, а логиката на задачата – редица от оператори. В този случай се налага оформянето на блок.

Синтаксис:

```
{
<оператор1>
…
<операторn>
}
```

Семантика:

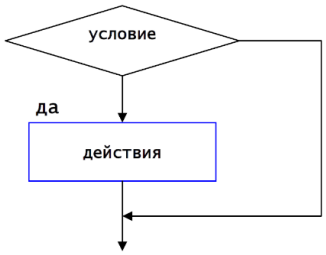
Обединява няколко оператора в един, наречен блок. Може да бъде поставен навсякъде, където по синтаксис стои оператор. Дефинициите в блока се отнасят само за него, т.е. не могат да се използват извън него.

4. Условни оператори

Чрез тези оператори се реализират разклоняващи се изчислителни процеси. Оператор, който дава възможност да се изпълни (или не) един или друг оператор в зависимост от някакво условие, се нарича **условен оператор**. Ще разгледаме следните условни оператори: `if`, `if/else` и `switch`.

a. Условен оператор `if`

Чрез този оператор се реализира разклоняващ се изчислителен процес от следния вид:



Ако указаното условие е в сила, то се изпълняват определени действия, а ако не – тези действия се прескачат. И в двата случая след това се изпълняват общи действия. Условието се задава чрез някакъв булев израз, а действията – чрез оператор.

Синтаксис:

```
if (<условие>) <оператор>
```

където

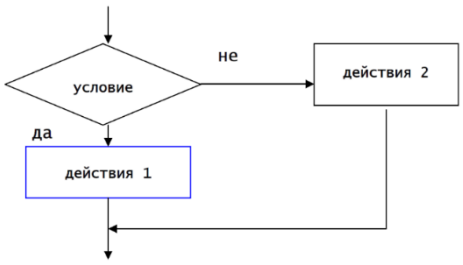
- `if` (ако) е запазена дума;
- `<условие>` е булев израз;
- `<оператор>` е произволен оператор.

Семантика:

Пресмята се стойността на булевия израз, представящ условието. Ако резултатът е `true`, изпълнява се `<оператор>`. В противен случай `<оператор>` не се изпълнява.

b. Оператор `if/else`

Операторът се използва за избор една от две възможни алтернативи в зависимост от стойността на дадено условие. Чрез него се реализира разклоняващ се изчислителен процес от следния вид:



Ако указаното условие е в сила, изпълняват се едни действия, а ако не – други действия. И в двата случая след това се изпълняват общи действия. Условието се задава чрез някакъв булев израз, а действия 1 и 2 – чрез оператори.

Синтаксис:

```
if (<условие>) <оператор1> else <оператор2>
```

където

- `if` (ако) и `else` (иначе) са запазени думи;
- `<оператор1>` и `<оператор2>` са произволни оператори;
- `<условие>` е булев израз.

Семантика:

Пресмята се стойността на булевия израз, представящ условието. Ако резултатът е `true`, изпълнява се `<оператор1>`. В противен случай се изпълнява `<оператор2>`.

Възможно е условните оператори да бъдат **вложени** един в друг. Това може да доведе до двусмислици. В C++ те се решават чрез следното **правило**:

Всяко `else` се съчетава в един условен оператор с най-близкото преди него несъчетано `if`. Текстът се гледа отляво надясно.

c. Оператор `switch`

Често се налага да се избере за изпълнение един измежду множество от варианти.

Синтаксис:

```
switch (<израз>)
{ case <израз1> : <редица_от_оператори1>
  case <израз2> : <редица_от_оператори2>
…
  case <изразn-1> : <редица_от_операториn-1>
  [default : <редица_от_операториn>]
}
```

където

- `switch` (ключ), `case` (случай, избор или вариант) и `default` (по премълчаване) са запазени думи на езика;
- `<израз>` е израз от допустим тип (типовете `bool`, `int` и `char` са допустими, реалните типове `double` и `float` не са допустими). Ще го наричаме още `switch`-израз;

- <израз$_1$>, … , <израз$_{n-1}$> са константни изрази, задължително с различни стойности

Семантика:

Намира се стойността на switch-израза. Получената константа се сравнява последователно със стойностите на етикетите <израз$_1$>, <израз$_2$>, … При съвпадение се изпълняват операторите на съответния вариант и операторите на всички варианти, разположени след него, до срещане на оператор break. В противен случай, ако участва default-вариант, се изпълнява редицата от оператори, която му съответства и в случай, че не участва такъв – не следват никакви действия от оператора switch.

Между фигурните скоби са изброени вариантите на оператора. Всеки вариант (без евентуално един) започва със запазената дума case, следвана от израз (нарича се още case-израз или етикет), който се пресмята по време на компилация. Такива изрази се наричат константни. Те не зависят от входните данни. След константния израз се поставя знакът двоеточие, следван от редица от оператори (оператори на варианта), която може да е празна. Сред вариантите може да има един, който няма case-израз и започва със запазената дума default. Той се изпълнява в случай, че никой от останалите варианти не е бил изпълнен.

Съществува възможност програмистът да съобщи на компилатора, че желае да се изпълни само редицата от оператори на варианта с етикет, съвпадащ със стойността на switch-израза, а не и всички следващи го. Това се реализира чрез използване на оператор break в края на редицата от оператори на варианта. Този оператор предизвиква прекъсване на изпълнението на оператора switch и подаване на управлението на първия оператор след него.

Операторът break принадлежи към групата на т.нар. оператори за преход. Тези оператори предават управлението безусловно в някаква точка на програмата.

Синтаксис:

```
break;
```

Семантика:

Прекратява изпълнението на най-вътрешния съдържащ го оператор switch или оператор за цикъл. Изпълнението на програмата продължава от оператора, следващ (съдържащ) прекъснатия.

5. Оператори за цикъл

Операторите за цикъл се използват за реализиране на **циклични изчислителни процеси**. Изчислителен процес, при който оператор или група оператори се изпълняват многократно за различни стойности на техни параметри, се нарича **цикличен изчислителен процес**.

Съществуват два вида циклични процеси:

- индуктивни
- итеративни

Цикличен изчислителен процес, при който броят на повторенията е известен предварително, се нарича **индуктивен изчислителен процес**.

Цикличен изчислителен процес, при който броят на повторенията не е известен предварително, се нарича **итеративен цикличен процес**. При тези циклични процеси, броят на повторенията зависи от някакво условие.

В езика C++ има **три оператора за цикъл**:

- оператор `for`

  Чрез него могат да бъдат реализирани произволни циклични процеси, но се използва главно за реализиране на индуктивни циклични процеси.

- оператор `while` и `do/while`

  Използват се за реализиране на итеративни изчислителни процеси.

a. Оператор `for`

Използва се основно за реализиране на индуктивни изчислителни процеси.

Синтаксис:

```
for (<инициализация>; <условие>; <корекция>)
     <оператор>
```

където

- `for` (за) е запазена дума
- `<инициализация>` е или точно една дефиниция с инициализация на една или повече променливи, или няколко оператора за присвояване или въвеждане, отделени със `,` и не завършващи с `;`
- `<условие>` е булев израз
- `<корекция>` е един или няколко оператора, не завършващи с `;`. В случай, че са няколко, те се отделят със `,`
- `<оператор>` е точно един произволен оператор. Нарича се *тяло на цикъла*

Семантика:

Изпълнението започва с изпълнение на частта `<инициализация>`. След това се намира стойността на `<условие>`. Ако в резултат се е получило `false`, изпълнението на оператора `for` завършва, без тялото да се е изпълнило нито веднъж. В противен случай, последователно се повтарят следните действия:

- Изпълнение на тялото на цикъла;
- Изпълнение на операторите от частта `<корекция>`;
- Пресмятане стойността на `<условие>`

докато стойността на <условие> е true

Възможно е частта <инициализация>, <условие> и <корекция> поотделно или заедно, да са празни. Разделителите (;) между тях обаче трябва да фигурират. Ако частта <условие> е празна, подразбира се true.

Под област на променлива се разбира мястото в програмата, където променливата може да се използва. Казва се още, където тя е „видима“. Съгласно стандарта *ANSI*, областта на променлива, дефинирана в заглавната част на цикъла for започва от дефиницията ѝ и продължава до края на цикъла.

b. Оператор `while`

Чрез този оператор може да се реализира произволен цикличен процес.

Синтаксис:

```
while (<условие>) <оператор>
```

където

- while (докато) запазена дума
- <условие> е булев израз
- <оператор> е произволен оператор. Той описва действията, които се повтарят и се нарича тяло на цикъла.

Семантика:

Пресмята се стойността на <условие>. Ако тя е false, изпълнението на оператора while завършва без да се изпълни тялото му нито веднъж. В противен случай изпълнението на <оператор> и пресмятането на стойността на <условие> се повтарят докато <условие> е true. В първия момент, когато <условие> стане false, изпълнението на while завършва.

c. Оператор `do/while`

Използва се за реализиране на произволни циклични процеси. Нарича се още **оператор за цикъл с пост-условие**.

Синтаксис:

```
do

<оператор>

while (<условие>);
```

където

- do (прави, повтаряй докато …) и while (докато) са запазени думи на езика
- <оператор> е точно един оператор. Той описва действията, които се повтарят и се нарича тяло на цикъла

- <условие> е булев израз. Нарича се условие за завършване на цикъла. Огражда се в кръгли скоби.

Семантика:

Изпълнява се тялото на цикъла, след което се пресмята стойността на <условие>. Ако то е false, изпълнението на оператора do/while завършва. В противен случай се повтарят действията: изпълнение на тялото и пресмятане стойността на <условие>, докато стойността на <условие> е true. Веднага, след като стойността му стане false, изпълнението на оператора завършва. Дефинициите в тялото не са видими в <условие>.

Тялото на кой да е от операторите за цикъл е произволен оператор. Възможно е да е оператор за цикъл или блок, съдържащ оператор за цикъл. В тези случаи се говори за вложени оператори за цикъл.

При влагането на цикли, а също при използването на блокове, възникват проблеми, свързани с видимостта на дефинираните променливи. Общото правило за дефиниране на променлива е дефиницията ѝ да е възможно най-близко до мястото, където променлива ще се използва най-напред. Областта на една променлива започва от нейната дефиниция и продължава до края на блока, в който променливата е дефинирана.

Променливи – видове: локални променливи, глобални променливи. Инициализация на променлива. Оператор за присвояване

Променливата е място за съхранение на данни, което може да съдържа различни стойности по време на изпълнение на програмата. Означава се чрез редица от букви, цифри и долна черта, започваща с буква или долна черта. Променливите имат три характеристики – **тип**, **име** и **стойност**. Преди да бъдат използвани, трябва да бъдат дефинирани.

C++ е строго типизиран език за програмиране. Всяка променлива има тип, който явно се указва при дефинирането ѝ. Пропускането на типа на променливата води до сериозни грешки.

Синтаксис:

```
<име_на_тип>  <променлива>  [ = <израз> ]
                        {,  <променлива>  [ = <израз> ]  };
```

където

- <име_на_тип> е дума, означаваща име на тип като int, double и др.
- <израз> е правило за получаване на стойност – цяла, реална, знакова и др. тип, съвместим с <име_на_тип>

Семантика:

Дефиницията свързва променливата с множеството от допустимите стойности на типа, от който е променливата или с конкретна стойност от това множество. За целта се отделя определено количество оперативна памет и се именува с името на променливата. Тази памет е с неопределено съдържание или съдържа стойността на указания израз, ако е направена

инициализация. Не се допуска една и съща променлива да има няколко дефиниции в рамките на една и съща функция.

Пример: `double a = 2.3;`

Дефинираме променлива с име `a` от реален тип `double`, като и задаваме начална стойност `2.3`. Казваме още, че инициализираме `a` с `2.3`.

Променливите могат да променят стойностите си многократно, например чрез т.нар. **оператор за присвояване**. В един опростен вариант този оператор има следния вид:

Синтаксис:

`<променлива> = <израз>;`

- `<променлива>` и `<израз>` са от един и същи тип

Семантика:

Пресмята стойността на `<израз>` и я записва в паметта, именувана с променливата от лявата страна на знака за присвояване =.

По-подробно разгледахме оператора за присвояване в миналата секция.

Разгледахме основните **характеристики на променливите** – *име*, *тип данни* и *текуща стойност*. Чрез оператор за деклариране на всяка променлива се определят име и тип на данните. Едновременно с това на променливите може да се присвоят начални стойности, което наричаме **инициализация**. Ще разгледаме още една важна характеристика на променливите, която се отнася до областта от програмния код, в която величината е достъпна и с нея могат да се извършват операции.

Областта, в която една величина е достъпна и с нея може да се извършват операции, се нарича **област на действие (обхват) на величината**. Достъпът до всяка величина може да се извърши само в рамките на нейната област на действие. Извън областта си на действие величините са недостъпни. В зависимост от своята декларация и евентуално от декларациите на други идентификатори величината може да бъде **локална**, **нелокална** или **глобална** за даден блок.

Променлива, декларирана в даден блок, се определя като **локална** за блока. Областта на действие на локалната величина започва от нейната декларация и завършва с края на блока, в който е извършена декларацията ѝ.

Не може да се каже, че една величина е изобщо локална. Тя може да е локална само в рамките на точно определен блок. За друг блок тя може да е достъпна, но не като локална, а като **нелокална**, а за трети блок може да е изобщо недостъпна.

Променлива, декларирана в даден блок A, в който се съдържа друг блок B, е локална за блока A и нелокална за блока B.

В рамките на един и същи блок всички променливи трябва да бъдат с различни имена. Допуска се обаче две променливи от различни блокове да имат еднакви имена, без значение какви са техните типове. Интересен е случаят, когато областта на действие на едната величина е част от

областта на действие на другата. Този специален случай е решен в езика със следното правило: **Локалната декларация има приоритет пред нелокалната декларация**. В подобни случаи се казва, че локалната променлива във вътрешния блок скрива нелокалната променлива, декларирана във външния блок. Казва се, че нелокалната променлива е **невидима** във вътрешния блок.

Променлива, декларирана в даден блок A, който няма обща част с блока B, е локална за блока A и е **недостъпна** в блока B и обратно, променливите, декларирани в блок B, са локални за него и недостъпни в блок A.

Променлива, декларирана извън всички функции в една програма, се нарича **глобална** за програмата. Областта на действие на всяка глобална променлива започва от нейната декларация и завършва с края на файла, в който е декларирана.

Функции и процедури. Параметри – видове параметри. Предаване на параметри – по име и по стойност. Типове и проверка за съответствие на тип

Добавянето на нови оператори и функции в приложенията, реализирани на езика C++, се отъждествява чрез **функциите** (подпрограми, процедури). Те са основни структурни единици, от които се изграждат програмите на езика. Всяка функция се състои от множество от оператори, оформени подходящо, за да се използват като обобщено действие или операция. След като една функция бъде дефинирана, тя може да бъде изпълнявана многократно за различни входни данни.

Програмите на езика C++ се състоят от една или повече функции. Сред тях задължително трябва да има точно една с име **`main`** и наречена **главна функция**. Тя е първата функция, която се изпълнява при стартиране на програмата. Главната функция от своя страна може да се обръща към други функции. Нормалното изпълнение на програмата завършва с изпълнението на главната функция (възможно е изпълнението да завърши принудително с изпълнението на функция, различна от главната).

Използването на функции има следните **предимства**:

- Програмите стават **ясни** и **лесни** за **тестване** и **модифициране**.
- **Избягва се многократното повтаряне** на едни и същи програмни фрагменти. Те се дефинират **еднократно** като функции, след което могат да бъдат изпълнявани произволен брой пъти.
- Постига се **икономия на памет**, тъй като кодът на функцията се съхранява само на едно място в паметта, независимо от броя на нейните изпълнения.

Разпределението на оперативната памет зависи от изчислителната система, от типа на операционната система, а също и от модела памет. Най-общо се състои от: *програмен код*, *област на статичните данни*, *област на динамичните данни* и *програмен стек*.



Програмен код

В тази част е записан изпълнимият код на всички функции, изграждащи потребителската програма.

Област на статичните данни

В нея са записани глобалните обекти на програмата.

Област на динамичните данни

За реализиране на динамични структури от данни (списъци, дървета, графи, …) се използват средства за динамично разпределение на паметта. Чрез тях се заделя и освобождава памет в процеса на изпълнение на програмата, а не преди това (при компилирането ѝ). Тази памет е от областта на динамичните данни.

Програмен стек

Този вид памет съхранява данните на функциите на програмата. Стекът е динамична структура, организирана по правилото „последен влязъл – пръв излязъл“. Той е редица от елементи с пряк достъп до елементите от единия си край, наречен връх. Достъпът се реализира чрез указател. Операцията включване се осъществява само пред елемента от върха, а операцията изключване – за елемента от върха. Елементите на програмния стек са „блокове“ от памет, съхраняващи данни, дефинирани в някаква функция, наречени **стекови рамки**.

Дефиницията на функция се състои от две части – **заглавие** (прототип) и **тяло**.

Синтаксис:

```
[<модификатор>] [<тип_на_функцията>] <име_на_функцията> (<формални_параметри>)
{
	<тяло>
}
```

Модификаторите са спецификатори, които задават препоръка за компилатора (`inline`), класа памет (`extern` или `static`) и др. Ако е пропуснат, подразбира се `extern` (указваме на компилатора, че функцията е декларирана някъде другаде в кода и затова липсва тяло).

Типът на функцията е типа на резултата, който функцията връща. Той може да е **скаларен** или **съставен**. В случай, че типа е съставен се връща указател към него (или първия му елемент, ако е масив). Функциите, които извършват своята работа без да се налага да връщат резултат са от тип `void`. Всяка функция, която връща даден тип може да се пренапише като `void` функция, като данните, които връща се подадат като аргументи с адреси на вече създадени типове в тялото, където се извиква функцията. По този начин може да „връщаме“ повече от един тип данни след изпълнението на функцията.

Името на функцията е произволен идентификатор. Допуска се неоднозначност.

Списъкът от формални параметри (нарича се още **сигнатура**) може да е празен или `void`. В случай, че не е празен, имената на параметрите трябва да са различни. Те заедно с името определят еднозначно функцията. Формалните параметри са: **параметри стойности**, **параметри указатели** и **параметри псевдоними**. Името на параметъра се предшества от тип. Списъкът може да е празен или да съдържа аргументи по подразбиране, които компилаторът автоматично присвоява, ако извикващият функцията не предостави експлицитно стойност за тях. Ако има аргументи по подразбиране, те задължително се слагат в края на списъка.

Тялото на функцията е редица от дефиниции и оператори. То описва алгоритъма, реализиращ функцията. Може да съдържа един или повече оператора `return` – връща резултата на функцията в мястото на извикването.

Функциите могат да се дефинират в произволно място на програмата, но не и в други функции. Преди да се извика една функция, тя трябва да е „позната“ на компилатора. Това става като дефиницията на функцията се постави пред `main` или когато функцията се дефинира на произволно място в частта за дефиниране на функции, а преди дефинициите на функциите се постави само нейната декларация. В нея е възможно имената на параметрите във `<формални_параметри>` да се пропуснат.

Семантика:

Описанието на функция задава параметрите, които носят входа и изхода, типа на резултата, а също и алгоритъма за реализиране на действието, което функцията дефинира. Параметрите стойности най-често задават входа на функцията. Параметрите указатели и псевдоними са входно-изходните параметри за нея. Алгоритъмът се описва в тялото на функцията. Изпълнението на функцията завършва при достигане на края на тялото или след изпълнение на оператор `return [<израз>];`.

Обръщението към функция има следния **синтаксис**:

```
<име_на_функция>(<фактически_параметри>);
```

където `<фактически_параметри>` са толкова на брой, колкото са формалните параметри. Освен по брой, формалните и фактическите параметри трябва да си съответстват по тип, по вид и по смисъл.

Съответствието по тип означава, че типът на i-тия фактически параметър трябва да съвпада (да е съвместим) с типа на i-тия формален параметър. **Съответствието по вид** се състои в следното – ако формалният параметър е от тип указател, то фактическия параметър

задължително е адрес на променлива. Ако пък формалният параметър е от тип псевдоним (тоест адрес на променлива), фактическият параметър задължително е променлива.

Семантика:

Обръщението към функция е унарна операция с най-висок приоритет и с операнд – името на функцията. Последното пък е указател със стойност адреса на мястото в паметта, където е записан програмният код на функцията. Обръщението към функцията предизвиква генериране на нова стекова рамка и се осъществява на следните два етапа:

1. Свързване на формалните с фактическите параметри.
2. Изпълнение на тялото на функцията.

След приключването на изпълнението на функцията, стековата ѝ рамка се зачиства. Трябва да се отбележи, че стековата памет е организирана като стек (откъдето идва и името ѝ). Тоест на дъното на тази памет лежи стековата рамка за функцията `main`, над нея – стековата рамка на функцията, извикана от `main`, над нея – стековата рамка, извикана от предишната функция и т.н. до върха. На върха на стека се намира стековата рамка на последно извиканата функция. Добра практика е една функция да изпълнява точно една задача.

В програмния език C++ съществуват **три вида предаване на аргументи** на функция. Най-простият и лесен начин е предаването на параметри **по стойност**. При предаването по стойност, параметърът, подаден към функцията, се копира локално в нея и се използва копието при работата ѝ. По този начин всяка промяна върху параметъра в рамките на функцията няма да се отрази върху подавания параметър отвън. Този вид предаване на параметри е по-скъп, тъй като трябва да се извършва копие на променлива на локално ниво. Ако променливата заема голямо пространство от паметта, това намалява производителността и отнема допълнителна памет за локалното копие.

За да направим така, че една функция да промени подадените ѝ аргументи и тази промяна да се отрази на фактическата променлива е необходимо да подадем параметрите **по референция** (**псевдоним**). След типа на формалния параметър добавяме запазения символ `&`. Функцията директно работи с променливата, тъй като получава адреса ѝ и „знае“ къде да извършва манипулациите с нея.

Друг начин да се направи така, че промяна на даден параметър във функция да се отрази на външната променлива, подадена като фактически параметър е чрез подаване на параметъра **чрез указател**. Това се извършва чрез поставяне на запазения символ `*` след типа на формалния параметър. Тук обаче функцията очаква да ѝ бъде подаден *адрес* като фактически параметър на мястото на формалния параметър, маркиран със символа `*`.

Символни низове. Представяне в паметта. Основни операции със символни низове

Редица от краен брой символи, заградени в кавички, се нарича **символен низ** или само **низ**. Броят на символите в редицата се нарича **дължина на низа**.

Низ с дължина 0 се нарича **празен низ** (`""`). Низ, който се съдържа в даден низ, се нарича негов **подниз**. **Конкатенация** на два низа е низ, получен като в края на първия низ се запише вторият. Нарича се още **слепване** на низове.

Два символни низа се **сравняват** по следния начин – сравнява се всеки символ от първия низ със символа от съответната позиция на втория низ. Сравнението продължава до намиране на два различни символа или до края на поне един от символните низове. Ако кодът на символ от първия низ е по-малък от кода на съответния символ от втория низ, или първият низ е изчерпен, приема се, че първият низ е по-малък от втория. Ако пък е по-голям или вторият низ е изчерпен – приема се, че първият низ е по-голям от втория. Ако в процеса на сравнение и двата низа едновременно са изчерпени, те са равни. Това сравнение се нарича лексикографско.

В езика C++ може да се използват два вида низове – т.нар. *C низове* и *низове от типа данни string* (*C++ низове*). Първият вид низове са наследство от езика C. Именно с тях ще демонстрираме примерите и операциите по-долу.

В оперативната памет низовете се представят **последователно**. Съществуват два начина за разглеждане на низовете в езика C++ – като **масиви от символи** и като **указатели към тип `char`**. За щастие, те са семантично еквивалентни.

Пример: Дефиницията `char str1[100];` определя променливата `str1` за масив от 100 символа

В езика са дадени средства, реализиращи низа като скаларна структура. За целта низът се разглежда като редица от символи, завършваща с нулевия символ `'\0'`, наречен още знак за край на низ (**терминираща нула**). Тази организация има предимството, че не е необходимо с всеки низ да се пази в променлива дължината му, тъй като знакът за край на низ позволява да се определи краят му.

Типът `char[size]`, където `size` е константен израз от интегрален или изброен тип, може да бъде използван за задаване на тип низ с максимална дължина `size - 1`.

Множеството от стойности на типа низ, зададен чрез `char[size]` се състои от всички низове с дължина 0, 1, 2, …, `size` – 1.

Променлива величина, множеството от допустимите стойности на която съвпада с множеството от стойности на даден тип низ, се нарича **променлива от този тип низ**. Ако редицата от константни изрази съдържа по-малко от `size` израза, то може да не завършва със знака за край на низ. Той автоматично се добавя. А ако съдържа точно `size` константни израза, задължително трябва да завършва със знака за край на низ `'\0'`.

Ако `size` е указано и изброените константни изрази в инициализацията са по-малко от `size`, останалите се инициализират с `'\0'`. На променлива от тип низ не може да бъде присвоявана константа от тип низ. Инициализацията е един начин за свързване на променлива от тип низ с конкретна константа от множеството от стойности на този тип низ. Друг начин предоставят индексираните променливи.

Пример: `q[0] = 'a'; q[1] = 's'; q[2] = 'd';`

Дефиницията на променлива от тип низ не само свързва променливата с множеството от стойности на указания тип, но и отделя определено количество **памет** (обикновено 4B), в която записва адреса на първата индексирана променлива, свързана с променливата от тип низ. Останалите индексирани променливи се разполагат последователно след първата. За всяка

индексирана променлива се отделя по 1B оперативна памет. Съдържанието на отделената за индексираните променливи памет е неопределено освен ако не е зададена дефиниция с инициализация. Тогава в клетките се записват инициализиращите стойности, допълнени със знака за край на низ.

Операции и вградени функции:

1. **Въвеждане на стойност**

Реализира се по стандартен начин чрез оператора `cin`.

Пример:

```
char s[5], t[3];

cin >> s >> t;
```

Настъпва пауза в очакване да се въведат два низа с дължина не по-голяма от 4 в първия и не по-голяма от 2 във втория случай. Водещите интервали, табулации и преминаване на нов ред се пренебрегват. За разделител на низовете се използват интервалът, табулациите и знака за преминаване на нов ред. Знакът за край на низ автоматично с добавя в края на всяка от въведените знакови комбинации. При въвеждане на низовете не се извършва проверка за достигане на указаната горна граница. Това може да доведе до трудно откриваеми грешки.

2. **Извеждане на низ**

Реализира се по стандартен начин чрез оператора `cout`.

Пример:

```
cout << s;
```

Извежда низа, който е стойност на `s`. Не е нужно да се грижим за дължината му. Знакът за край на низ идентифицира края му.

3. **Дължина на низ**

Намира се чрез функцията `strlen`.

Синтаксис:

`strlen(<str>)`

където `<str>` е произволен низ.

Семантика:

Намира дължината на `<str>`.

Пример:

`strlen(”abc”)` → `3`, `strlen(””)` → `0`

За използване на тази функция е необходимо да се включи заглавният файл `string.h`.

4. Конкатенация на низове

Реализира се чрез функцията `strcat`.

Синтаксис:

```
strcat(<var_str>, <str>)
```

където

`<var_str>` е променлива от тип низ, а `<str>` е низ (константа, променлива или по-общо израз)

Семантика:

Конкатенира низа, който е със стойност на `<var_str>` с низа `<str>`. Резултатът от конкатенацията се връща от функцията, а също се съдържа в променливата `<var_str>`.

За използване на тази функция е необходимо да се включи заглавният файл `string.h`.

5. Сравняване на низове

Реализира се чрез функцията `strcmp`.

Синтаксис:

```
strcmp(<str1>, <str2>)
```

където

`<str1>` и `<str2>` са низове (константи, променливи или по-общо изрази).

Семантика:

Низовете `<str1>` и `<str2>` се сравняват лексикографски. Функцията `strcmp` е целочислена. Резултатът от обръщение към нея е цяло число с отрицателна стойност (-1), ако `<str1>` е по-малък от `<str2>`, 0 – ако `<str1>` е равен на `<str2>` и с положителна стойност (1), ако `<str1>` е по-голям от `<str2>`.

За използване на `strcmp` е необходимо да се включи заглавният файл `string.h`.

6. Копиране на низ

Реализира се чрез функцията `strcpy`.

Синтаксис:

```
strcpy(<var_str>, <str>)
```

където

<var_str> е променлива от тип низ, а

<str> е низ (константа, променлива или по-общо израз).

Семантика:

Копира <str> във <var_str>. Ако <str> е по-дълъг от допустимата за <var_str> дължина, възможни са трудно откриваеми грешки. Резултатът от копирането се връща от функцията, а също се съдържа във <var_str>.

За използване на тази функция е необходимо да се включи заглавният ред string.h.

7. Търсене на низ в друг низ

Реализира се чрез функцията strstr.

Синтаксис:

```
strstr(<str1>, <str2>)
```

където

<str1> и <str2> са произволни низове (константи, променливи или по-общо изрази)

Семантика:

Търси <str2> в <str1>. Ако <str2> се съдържа в <str1>, strstr връща подниза на <str1>, започващ от първото срещане на <str2> до края на <str1>. Ако <str2> не се съдържа в <str1>, strstr връща "нулев" указател. Последното означава, че в позиция на условие, функционалното обръщение ще има стойност false, но при опит за извеждане, ще предизвиква грешка.

За използване на тази функция е необходимо да се включи заглавният файл string.h.

8. Преобразуване на низ в цяло число

Реализира се чрез функцията atoi.

Синтаксис:

```
atoi(<str>)
```

където <str> е произволен низ (константа, променлива или по-общо израз от тип низ)

Семантика:

Преобразува символния низ в число от тип int. Водещите интервали, табулации и знака за преминаване на нов ред се игнорират. Символният низ се сканира до първия символ, различен от цифра. Ако низът започва със символ, различен от цифра и знак, функцията връща 0.

За използване на тази функция е необходимо да се включи заглавния файл stdlib.h.

Пример:

```
cout << atoi(”-123a45”); // -123

cout << atoi(”b123a45”); // 0
```

9. Преобразуване на низ в реално число

Реализира се чрез функцията `atof`.

Синтаксис:

```
atof(<str>)
```

където `<str>` е произволен низ (константа, променлива или по-общо израз от тип низ).

Семантика:

Преобразува символния низ в число от тип `double`. Водещите интервали, табулации и знака за преминаване на нов ред се игнорират. Символният низ се сканира до първия символ, различен от цифра. Ако низът започва със символ, различен от цифра, знак или точка, функцията връща 0.

За използване на тази функция е необходимо да се включи заглавният файл `stdlib.h`.

Пример:

```
cout << atof(“-123.35a45”); // -123.35

cout << atof(“.123.34a45”); // 0.123
```

10. Конкатенация на n символа от низ с друг низ

Реализира се чрез функцията `strncat`.

Синтаксис:

```
strncat(<var_str>, <str>, n)
```

където

`<var_str>` е променлива от тип низ, `<str>` е низ (константа, променлива или по-общо израз), а n е цял израз с неотрицателна стойност.

Семантика:

Копира първите n символа от `<str>` в края на низа, който е стойност на `<var_str>`. Копирането завършва, когато са прехвърлени n символа, или е достигнат краят на `<str>`. Резултатът е в променливата `<var_str>`.

За използване на тази функция е необходимо да се включи заглавният файл `string.h`.

11. Копиране на n символа в символен низ

Реализира се чрез функцията `strncpy`.

Синтаксис:

```
strncpy(<var_str>, <str>, n)
```

където

`<var_str>` е променлива от тип низ,

`<str>` е низ (константа, променлива или по-общо израз), а n е цял израз с неотрицателна стойност.

Семантика:

Копира първите n символа на `<str>` във `<var_str>`. Ако `<str>` има по-малко от n символа, `'\0'` се копира дотогава докогато не се запишат n символа. Параметърът `<var_str>` трябва да е от вида `char[n]` и съдържа резултатния низ.

За използването на тази функция е необходимо да се включи заглавният файл `string.h`.

12. Сравняване на n символа на низове

Реализира се чрез функцията `strncmp`.

Синтаксис:

```
strncmp(<str1>, <str2>, n)
```

където

`<str1>` и `<str2>` са низове (константи, променливи или по-общо изрази), а n е цял израз с неотрицателна стойност.

Семантика:

Сравнява първите n символа на `<str1>` със символите от съответната позиция на `<str2>`. Сравнението продължава до намиране на два различни символа или до края на един от символните низове. Резултатът от функцията `strncmp` е цяло число с отрицателна стойност, ако `<str1>` е по-малък от `<str2>`, 0 – ако `<str1>` е равен на `<str2>` и с положителна стойност, ако `<str2>` е по-голям от `<str1>`.

За използване на `strncmp` е необходимо да се включи заглавният файл `string.h`.

13. Търсене на символ в низ

Реализира се чрез функцията `strchr`.

Синтаксис:

```
strchr(<str>, <expr>)
```

където

`<str>` е произволен низ, а `<expr>` е израз от интегрален или изброен тип с положителна стойност, означаващ ASCII код на символ.

Семантика:

Търси първото срещане на символа, чийто ASCII код е равен на стойността на `<expr>`. Ако символът се среща, функцията връща подниза на `<str>`, започващ от първото срещане на символа и продължаващ до края му. Ако символът не се среща – връща „нулев“ указател. Последното означава, че в позиция на условие, функционалното обръщение ще има стойност `false`, но при опит за извеждане, ще предизвика грешка.

За използване на тази функция е необходимо да се включи заглавният файл `string.h`.

14. Търсене на първата разлика

Реализира се чрез функцията `strspn`.

Синтаксис:

```
strspn(<str1>, <str2>)
```

където

`<str1>` и `<str2>` са произволни низове (константи, променливи или по-общо изрази).

Семантика:

Проверява до коя позиция `<str1>` и `<str2>` съвпадат. Връща дължината на префикса до първия различен символ.

За използването на тази функция е необходимо да се включи заглавният файл `string.h`.

Пример:

```
char a[10] = ”asdndf”, b[15] = ”asdsdfdhf”;

cout << strspn(a, b) << ”\n”; // 3
```

Държавен изпит – спец. „Софтуерно инженерство“

Тема №12 – Процедурно програмиране – указатели, масиви и рекурсия.

Анотация: *Изложението по въпроса трябва да включва следните по-съществени елементи:*

1. Указатели и указателна аритметика.

2. Едномерни и многомерни масиви. Основни операции с масиви – индексиране. Сортиране и търсене в едномерен масив – основни алгоритми.

3. Рекурсия – пряка и косвена рекурсия, линейна и разклонена рекурсия.

Указатели и указателна аритметика

Променлива – това е място за съхранение на данни, което може да съдържа различни стойности. Идентифицира се с дадено от потребителя име (идентификатор). Има си и тип. Дефинира се като се указват задължително типът и името ѝ. Типът определя броя на байтовете, в които ще се съхранява променливата, а също и множеството от операциите, които могат да се изпълняват над нея. Освен това, с променливата е свързана и стойност – неопределена или константа от типа, от който е тя. Нарича се още *rvalue*. Мястото в паметта, в което е записана *rvalue*, се нарича адрес на променливата или *lvalue*. По-точно адресът е адресът на първия байт от множеството байтове, отделени за променливата.

Пример: Фрагментът `int i = 1024;` дефинира променлива с име `i` и тип `int`. Стойността ѝ (*rvalue*) е 1024. `i` именува място от паметта (*lvalue*) с размери 4 байта, като lvalue е адреса на първия байт.

Намирането на адреса на дефинирана променлива става чрез унарния префиксен дясно асоциативен оператор `&` (амперсанд). Приоритетът му е същия като на унарните оператори `+`, `-`, `!`, `++`, `--` и др.

Синтаксис:

`&<променлива>`

където `<променлива>` е вече дефинирана променлива.

Семантика:

Намира адреса на `<променлива>`.

Пример: `&i` е адреса на променливата `i` и може да се изведе чрез оператора `cout << &i;`

Операторът `&` не може да се прилага върху константи и изрази. Не е възможно също прилагането му и върху променливи от тип масив, тъй като те имат и смисъла на константни указатели.

Адресите могат да се присвояват на специален тип променливи, наречени **променливи от тип указател** или само **указатели**.

Нека `T` е име или дефиниция на тип. За типа `T`, `T*` е тип, наречен указател към `T`. `T` се нарича **указван тип** или **тип на указателя**.

Множеството от стойности се състои от адресите на данните от тип `T`, дефинирани в програмата, преди използването на `T*`. Те са константите на типа `T*`. Освен тях съществува специална константа с име **`nullptr`**, наречена **нулев указател**. Тя може да бъде свързвана с всеки указател, независимо от неговия тип. Тази константа се интерпретира като „сочи към никъде“, а в позиция на предикат е `false`.

Променлива величина, множеството от допустимите стойности на която съвпада с множеството от стойности на типа `T*`, се нарича променлива от тип `T*` или **променлива от тип указател** към тип `T`. Дефинира се по стандартния начин:

```
T* <променлива> [= <стойност>]; |

T *<променлива> [= <стойност>];
```

където

- `T` е име или дефиниция на тип
- `<променлива>` е идентификатор
- `<стойност>` е шестнадесетично цяло число, представляващо адрес на данна от тип `T` или `nullptr`

Дефиницията на променлива от тип указател предизвиква в оперативната памет да се определят 4B (за 32-битова машина) или 8B (за 64-битова машина), в които се записва някакъв адрес от множеството от стойности на съответния тип, ако дефиницията е с инициализация и неопределено или `nullptr`, ако дефиницията не е с инициализация. Този адрес е стойността на променливата от тип указател, а записаното на този адрес е съдържанието ѝ.

Извличането на съдържанието на указател се осъществява чрез префиксния, дясно асоциативен унарен оператор `*`.

Синтаксис:

```
*<променлива_от_тип_указател>
```

Семантика:

Извлича стойността на адреса, записан в `<променлива_от_тип_указател>`, т.е. съдържанието на `<променлива_от_тип_указател>`.

Пример:

```
int i = 12;

int* p = &i;

cout << *p << "\n"; // 12
```

Освен, че намира съдържанието на променлива от тип указател, обръщението *<променлива_от_тип_указател> е данна от тип T (променлива или константа). Всички операции, допустими за типа T, са допустими и за нея. Следователно, от примера *p е цяла променлива. След изпълнение на оператора за присвояване:

```
*p = 20;
```

стойността на i се променя на 20.

Променливите от тип указател могат да участват като операнди в следните аритметични и логически операции +, –, ++, --, ==, !=, >, >=, < и <=. Изпълнението на аритметични операции върху указатели е свързано с някои особености, заради което аритметиката с указатели се нарича още адресна (указателна) аритметика. Особеността се изразява в т.нар. **мащабиране**.

Пример:

```
int *p;

double *q;

…

p = p + 1;

q = q + 1;
```

Операторът p = p + 1; свързва p не със стойността на p, увеличена с 1, а с p + 1 * 4, където 4 е броя на байтовете, необходими за записване на данна от тип int (p е указател към int). Аналогично, q = q + 1; увеличава стойността на q не с 1, а с 8, тъй като q е указател към double (8 байта са необходими за записване на данна от този тип).

Общото правило е следното: Ако p е указател от тип T*, p + i е съкратен запис на p + i * sizeof(T), където sizeof(T) е функция, която намира броя на байтовете, необходими за записване на данна от тип T.

Не е възможно въвеждане на данни от тип указател чрез оператора cin. Свързването на указател със стойност става чрез инициализация или оператора за присвояване.

Извеждане на указател става по стандартния начин – чрез оператора cout.

Типът, който се задава в дефиницията на променлива от тип указател, е информация за компилатора относно начина, по който да се интерпретира съдържанието на указателя. В контекста на горния пример *p са четири байта, които ще се интерпретират като цяло число от тип int. Аналогично, *q са осем байта, които ще се интерпретират като реално число от тип double.

В някои случаи е важна стойността на променливата от тип указател (адресът), а не нейното съдържание. Тогава тя се дефинира като указател към тип void. Този тип указатели са предвидени с цел една и съща променлива указател да може в различни моменти да сочи към

данни от различен тип. В този случай, при опит да се използва съдържанието на променливата от тип указател, ще се предизвика грешка. Съдържанието на променлива указател към тип `void` може да се извлече само след привеждане на типа на указателя (`void*`) до типа на съдържанието. Това може да се осъществи чрез операторите за преобразуване на типовете.

В C++ е възможно да се дефинират указатели, които са константи, а също и указатели, които сочат към константи. И в двата случая се използва запазената дума **`const`**, която се поставя пред съответните елементи от дефинициите на указателите. Стойността на елемента, дефиниран като `const` (указателя или обекта, към който сочи) не може да бъде променяна.

Пример:

```
char i = 'S';  // const applies to the item to its left (if there
               // is not, the item to its right)

const char* ptr1 = &i; // ptr1++ allowed

                       // *ptr1 = 'V' not allowed

char const* ptr2 = &i; // ptr2++ allowed

                       // *ptr2 = 'V' not allowed

char* const ptr3 = &i; // ptr3++ not allowed

                       // *ptr3 = 'V' allowed

const char* const ptr4 = &i; // ptr4++ not allowed

                             // *ptr4 = 'V' not allowed
```

В C++ има интересна и полезна връзка между **указателите и масивите**. Тя се състои в това, че имената на масивите са указатели към техните „първи“ елементи. Това позволява указателите да се разглеждат като алтернативен начин за обхождане на елементите на даден масив.

Например, ако имаме масив `arr`, то `arr[0]` и `*arr` имат една и съща стойност равна на първия елемент от масива. Аналогично, може да достъпим и n-тия елемент от масива: `arr[n - 1] = *(arr + n - 1)` (тъй като елементите в масива са разположени последователно в паметта).

Има обаче една особеност. Имената на масивите са константни указатели. Заради това, някои от аритметичните операции, приложими над указатели, не могат да се приложат над масиви. Такива са `++`, `--` и присвояване на стойност.

Използването на указатели е по-бърз начин за достъп до елементите на масива и заради това се предпочита. **Индексираните променливи** правят кода по-ясен и разбираем. В процеса на компилация всички конструкции от вида `a[i]` се преобразуват в `*(a + i)`, т.е. операторът за индексиране `[…]` се обработва от компилатора чрез **адресна аритметика**.

Името на двумерен масив е константен указател към първия елемент на едномерен масив от константни указатели. Ще изясним с пример случая:

```
int a[10][20];
```

Променливата `a` е константен указател към първия елемент на едномерния масив `a[0], …, a[9]`, като всяко `a[i]` е константен указател към `a[i][0] (i = 0, 1, …, 9)`.

Тогава

```
**a == a[0][0]

a[0] == *a      a[1] == a[0] + 1     …     a[9] == a[0] + 9
```

т.е.

```
a[i] == a[0] + i == *a + i
```

Като използваме, че операторът за индексиране е ляво асоциативен, получаваме:

`a[i][j] == (*a + i) [j] == *((*a + i) + j)`

Низовете са масиви от символи. Името на променлива от тип низ е константен указател, както и името на всеки друг масив. Така всичко, което казахме за връзката между масив и указател, е в сила и за низ указател.

Ако низът е зададен чрез указател към `char`, този указател не е константен. Това е предимство, но от друга страна не може да се използва във всякакви ситуации, като например при вход на низ.

*Чрез **псевдонимите** се задават алтернативни имена на обекти в общия смисъл на думата (променливи, константи и др.). Нека `T` е име на тип. Типът `T&` е тип псевдоним на `T`. `T` се нарича **базов тип** на типа псевдоним.*

***Дефиницията** на променлива от тип псевдоним е следната:*

`T& <var> = <defined_var_of_T>;` |

`T &<var> = <defined_var_of_T>;`

където

- *`T` е име на тип*
- *`<defined_var_of_T>` е име на вече дефинирана променлива от тип `T`. Нарича се **инициализатор***

Дефинициите задължително са с инициализация – променлива от същия тип като на базовия тип на типа псевдоним. Освен това, след инициализацията, променливата псевдоним не може да се променя като ѝ се присвоява нова променлива или чрез повторна дефиниция. Затова тя е „най-константната“ променлива, която може да съществува.

Дефиницията на ***променлива от тип псевдоним*** *свързва променливата псевдоним с инициализатора и всички операции и вградени функции, които могат да се прилагат над инициализатора, могат да се прилагат и над псевдонима и обратно.*

Възможно е типът на инициализатора да е различен от този на псевдонима. В този случай се създава нова, наречена ***временна****, променлива от типа на псевдонима, която се инициализира до типа на псевдонима.*

В C++ е възможно да се дефинират ***псевдоними****, които са* ***константни****. За целта се използва запазената дума* `const`*, която се поставя пред дефиницията на променливата от тип псевдоним. По такъв начин псевдонимът не може да променя стойността си, но ако е псевдоним на променлива, промяната на стойността му може да стане чрез промяна на променливата. По този начин е възможно константен псевдоним на променлива да защитава промяната на стойността на променливата чрез псевдонима.*

Едномерни и многомерни масиви. Основни операции с масиви – индексиране. Сортиране и търсене в едномерен масив – основни алгоритми

Под **структура от данни** се разбира организирана информация, която може да бъде описана, създадена и обработена с помощта на програма. Структури от данни, елементите на които са редици от елементи, се наричат **съставни**.

Структури от данни, за които операциите включване и изключване на елемент не са допустими, се наричат **статични**, в противен случай – **динамични**.

Масивът е крайна редица от фиксиран брой елементи от един и същи тип. Към всеки елемент от редицата е възможен пряк достъп, който се осъществява чрез индекс. Операциите **включване** и **изключване** в масива са **недопустими**, т.е. масивът е статична структура от данни.

Елементите на масива се записват **последователно в паметта** на компютъра, като за всеки елемент на редицата се отделя определено количество памет.

В езика C++ структурата масив се реализира чрез **типа масив**. Индексите на масива започват от 0. Типът масив се определя чрез задаване на типа и броя на елементите на редицата, определяща масив. Нека `T` е име или дефиниция на произволен тип, различен от псевдоним, `void` или функционален. За типа `T` и константния израз от интегрален или изброен тип с положителна стойност `size`, `T[size]` е тип масив от `size` елемента от тип `T`. Елементите се индексират от `0` до `size - 1`. `T` се нарича **базов тип** за типа масив, а `size` – горна граница.

Множеството от стойности на типа `T[size]` се състои от всички редици от по `size` елемента, които са произволни константи от тип `T`. Елементите от множеството от стойности на даден тип масив са константите на този тип масив.

Променлива величина, множеството от допустимите стойности на която съвпада с множеството от стойности на даден тип масив, се нарича **променлива от дадения тип масив**. Понякога ще я наричаме само масив. Когато имаме дефиниция на масив с инициализация, е възможно `size` да се пропусне. Тогава за стойност на `size` се подразбира броят на

константните изрази, изброени при инициализацията. Ако `size` е указано и изброените константни изрази в инициализацията са по-малко от `size`, останалите се инициализират с 0.

Пример:

```
int a[5];

double r[] = {0, 1, 2, 3};
```

Инициализацията е един начин за свързване на променлива от тип масив с конкретна константа от множеството от стойности на този тип масив. Друг начин предоставят т.нар. **индексирани променливи**. С всяка променлива от тип масив е свързан набор от индексирани променливи.

Пример:

```
a[0], a[1], a[2], a[3], a[4]

r[0], r[1], r[2], r[3]
```

Дефиницията на променлива от тип масив не само свързва променливата с множеството от стойности на указания тип, но и отделя определено количество памет (обикновено 4B), в която записва адреса в паметта на първата индексирана променлива на масива. Останалите индексирани променливи се разполагат последователно след първата. За всяка индексирана променлива се отделя по толкова памет, колкото базовият тип изисква. Съдържанието на отделената за индексираните променливи памет е неопределено освен ако не е зададена дефиниция с инициализация. Тогава в клетките се записват инициализиращите стойности.

Не са възможни операции над масиви като цяло, но всички операции и вградени функции, които базовият тип допуска, са възможни за индексираните променливи, свързани с масива.

Разглежданите досега масиви се наричат **едномерни**. Те реализират крайни редици от елементи от скаларен тип. Възможно е обаче типът на елементите да е масив. В този случай се говори за **многомерни масиви**.

Масив, базовият тип на който е едномерен масив, се нарича **двумерен**. Масив, базовият тип на който е двумерен масив, се нарича **тримерен** и т.н. На практика се използват масиви с размерност най-много 3.

Нека `T` е име или дефиниция на произволен тип, различен от псевдоним, `void` и функционален, $size_1$, …, $size_n$ ($n > 1$ е дадено цяло число) са константни изрази от интегрален или изброен тип с положителни стойности. `T[`$size_1$`][`$size_2$`]…[`$size_n$`]` е тип `n`-мерен масив от тип `T`. `T` се нарича базов тип за типа масив.

Пример:

`int [5][3]` е двумерен масив от тип `int`;

`double [4][5][3]` е тримерен масив от тип `double`;

Множеството от стойности на типа `T[size1][size2]…[sizen]` се състои от всички редици от по `size1` елемента, които са произволни константи от тип `T[size2]…[sizen]`. Достъпът до елементите на редиците е пряк и се осъществява с помощта на индекс, като достъпът до първия елемент се осъществява с индекс със стойност 0, до последния – с индекс със стойност `size1 - 1`, а до всеки от останалите елементи – с индекс със стойност с 1 по-голяма от тази на индекса на предишния елемент. Елементите от множеството от стойности на даден тип многомерен масив са константите на този масив.

При дефиницията с инициализация е възможно `size1` да се пропусне. Тогава за стойност на `size1` се подразбира броят на редиците от константи на най-външно ниво, изброени при инициализация.

Пример: Дефиницията

```
int s[][2][3] = {{{1, 2, 3}, {4, 5, 6}},
                 {{7, 8, 9}, {10, 11, 12}},
                 {{13, 14, 15}, {16, 17, 18}}};
```

е еквивалентна на

```
int s[3][2][3] = {{{1, 2, 3}, {4, 5, 6}},
                  {{7, 8, 9}, {10, 11, 12}},
                  {{13, 14, 15}, {16, 17, 18}}};
```

Вложените фигурни скоби не са задължителни.

Инициализацията е един начин за свързване на променлива от тип масив с конкретна константа от множеството от стойности на този тип масив. Друг начин предоставят т.нар. **индексирани променливи**. С всяка променлива от тип масив е свързан набор от индексирани променливи.

Дефиницията на променлива от тип многомерен масив не само свързва променливата с множеството от стойности на указания тип, но и отделя 4B памет, в която записва адреса на първата индексирана променлива на масива. Останалите индексирани променливи се разполагат последователно след първата по по-бързото нарастване на по-далечните си индекси. За всяка индексирана променлива се отделя толкова памет, колкото базовият тип изисква.

Освен масивите, които разгледахме досега, съществуват и **динамично заделени масиви**, които се намират на различно място в паметта. Когато една C++ програма се стартира, компилаторът автоматично заделя памет за кода на програмата, друга памет за глобални променливи и стекова памет, в която се съхранява реда на извикване на функции и съответните им аргументи и локални променливи. Останалата част от паметта е **heap**-а и е достъпна за всички потребители на системата.

В **heap**-а може да заделяме памет чрез оператора **`new`**, който след заделяне на памет връща указател, сочещ към първия елемент, за който е заделил памет. Пример: `int *arr = new`

`int[10]`. Тази команда заделя `10 * 4` байта от **heap** паметта (достатъчна за `10` числа от тип `int`) и връща указател, който сочи към първото число от масива. За масивите заделени в **heap**-а и изобщо за всички обекти създадени с оператора `new`, трябва да се погрижим, като освободим паметта, когато решим, че вече няма да ни е необходима и е удачен момент да заделим процесорно време за тази операция по освобождаване на паметта. Това става с оператора `delete` за една данна или `delete[]` за масив от данни. Например `delete[] arr;` След освобождаването на паметта, указателят `arr` ще сочи към `nullptr`.

Променливата от тип двумерен масив е указател към първия ред на матрицата. Заделянето на двумерен масив в **heap** паметта става по следния начин: `T** matrix = new T[10][10];` Но за да освободим тази памет е необходимо да обходим по всички редове на матрицата и да изтрием масивите, които ги репрезентират, след което да изтрием масива, който съхранява указателите към тези редове:

```
for (int i = 0; i < 10; i++) {

    delete[] matrix[i];

}

delete[] matrix;
```

Масивите от ASCII символи също могат да бъдат заделяни и в **heap** паметта като например `char* arr = new char[n].`

Съществуват множество **алгоритми за сортиране** на едномерни масиви. В най-общ случай, те се делят на „бавни“ (със сложност **O(n^2)**) и „бързи“ (със сложност **O(nlogn)**). Трите основни „бавни“ алгоритъма са **сортиране по метода на мехурчето** (*bubble sort*), **сортиране с пряка селекция** (*selection sort*) и **сортиране с вмъкване** (*insertion sort*). “Бързите” алгоритми включват **сортиране със сливане** (*merge sort*) и **бързо сортиране** (*quick sort*). Ще ги демонстрираме в средата на езика C++.

Метод на мехурчето (Bubble Sort) – O(n^2), In-Place, Stable

Основна идея *– започвайки последователно от началния елемент до крайния, сравняваме* ***всеки елемент със следващия*** *като ги* ***разменяме****, ако не са подредени. По този начин на всяка стъпка изкарваме най-големия в края, на правилното за него място. Повтаряйки тази процедура, толкова пъти, колкото е дължината на масива, постигаме правилно подреждане на целия масив.*

```
1. void bubbleSort(int arr[], int n) {
2.     for (int i = 0; i < n - 1; i++) {
3.         for (int j = 0; j < n - i - 1; j++) {
4.             if (arr[j] > arr[j + 1]) {
5.                 swap(arr[j], arr[j+1]);
6.             }
7.         }
8.     }
9. }
```

Метод на пряката селекция (Selection Sort) – O(n^2)

Основна идея *– намалява броя на разменянията в сравнение с метода на мехурчето. Търсим* ***най-малкия*** *(най-големия) елемент в масива и* ***директно го поставяме*** *на първо (последно) място. След това следващия по големина и отново го поставяме на място и т.н. докато всички са подредени.*

```
 1. void selectionSort(int arr[], int n) {
 2.     for (int j = 0; j < n - 1; j++) {
 3.         int iMin = j;
 4.
 5.         for (int i = j + 1; i < n; i++) {
 6.             if (arr[i] < arr[iMin]) {
 7.                 iMin = i;
 8.             }
 9.         }
10.         if (j != iMin) {
11.             swap(arr[j], arr[iMin]);
12.         }
13.     }
14. }
```

Сортиране чрез вмъкване (Insertion Sort) – O(n^2)

Основна идея *– сортиране постепенно на все по-голяма част от масива, като обхождайки несортираната част,* ***всеки*** *един* ***елемент*** *го* ***поставяме*** *в сортираната част с* ***връщане назад*** *и сравняване като намираме правилното за него място, с което сортираният масив остава сортиран. Най-бързият при почти сортиран масив или масив с малко числа.*

```
 1. void insertionSort(int arr[], int n) {
 2.     for (int i = 1; i < n; i++) {
 3.         int curValue = arr[i];
 4.         int j = i - 1;
 5.
 6.         while (j >= 0 && arr[j] > curValue) {
 7.             arr[j + 1] = arr[j];
 8.             j--;
 9.         }
10.
11.         arr[j + 1] = curValue;
12.     }
13. }
```

Сортиране чрез сливане (Merge Sort) – сложност по памет O(n)

Основна идея *– ако имаме* ***два сортирани масива****, то със линейна сложност може да ги* ***влеем*** *в един масив. Тогава, ако искаме да разделим масива, който искаме да сортираме, на по-малки масиви и на всяка стъпка сливаме два по-малки масива в един голям, то за log(N) стъпки ще* ***слеем*** *всички масиви до един масив, като всяка от стъпките е била линейна.*

```
 1. void mergeMade(int arr[], int l, int mid, int r) {
 2.     vector<int> left;
 3.     vector<int> right;
 4.
 5.     for (int i = l; i <= mid; i++) {
 6.         left.push_back(arr[i]);
 7.     }
 8.
 9.     for (int i = mid + 1; i <= r; i++) {
10.         right.push_back(arr[i]);
11.     }
12.
13.     int i = 0;
14.     int j = 0;
```

```
    int index = l;

    while (i < left.size() && j < right.size()) {
        if (left[i] < right[j]) {
            arr[index++] = left[i++];
        }
        else {
            arr[index++] = right[j++];
        }
    }

    while (i < left.size()) {
        arr[index++] = left[i++];
    }

    while (j < right.size()) {
        arr[index++] = right[j++];
    }
}

void mergeSort(int arr[], int l, int r) {
    if (r - l <= 0) {
        return;
    }
    int mid = l + (r - l) / 2;
    mergeSort(arr, l, mid);
    mergeSort(arr, mid + 1, r);

    mergeMade(arr, l, mid, r);
}
```

Бързо сортиране (Quick Sort) – в най-лошия случай $O(n^2)$ (когато масивът е сортиран наобратно)

Основна идея *– ако вземем едно* ***произволно число*** *от масива, то с линейна сложност може да* ***прехвърлим*** *всички по-малки числа от масива да са* ***в ляво*** *на числото, а всички по-големи –* ***в дясно****. При бързото сортиране избираме число от масива, прехвърляме по-малките от ляво, по-големите от дясно и след това изпълняваме същата процедура за лявата и дясната половина.*

```
int partitionMade(int arr[], int l, int r) {
    int pivot = arr[r];

    int i = l;

    for (int j = l; j < r; j++) {
        if (arr[j] <= pivot) {
            swap(arr[i], arr[j]);
            i++;
        }
    }

    swap(arr[i], arr[r]);
    return i;
}

void quickSort(int arr[], int l, int r) {
    if (l < r) {
        int pivot = partitionMade(arr, l, r);
        quickSort(arr, l, pivot - 1);
        quickSort(arr, pivot + 1, r);
    }
}
```

Сред основните **алгоритми за търсене** в едномерен масив са **линейното търсене** (*linear search* – със сложност **O(n)**), **двоичното търсене** (*binary search* – със сложност **O(logn)**), **троичното търсене** (*ternary search* – със сложност **O(logn)**), **търсене със скоци** (*jump search* – със сложност **O($\sqrt{n}$)**) и др. Ще демонстрираме част от тях в средата на езика C++.

Линейно търсене (Linear Search) – за произволен масив

```
1.     bool linearSearch(int[] arr, int n, int x) {
2.         for (int i = 0; i < n; i++) {
3.             if (arr[i] == x) {
4.                 return true;
5.             }
6.         }
7.         return false;
8.     }
```

Двоично търсене (Binary Search) – предварително условие е масивът да е сортиран

```
 1. int binarySearch(vector<int>& arr, int x, int left, int right) {
 2.     while (left <= right) {
 3.         int middle = left + (right - left) / 2;
 4.
 5.         if (arr[middle] == x)
 6.             return middle;
 7.
 8.         if (arr[middle] < x)
 9.             left = middle + 1;
10.         else
11.             right = middle - 1;
12.     }
13.
14.     //Element is not found
15.     return -1;
16. }
```

Троично търсене (Ternary Search) – предварително условие е масивът да е сортиран

```
 1.     bool ternarySearch(int[] arr, int l, int r, int x) {
 2.         if (r < l) {
 3.             return false;
 4.         }
 5.         int m1 = (2 * l + r) / 3;
 6.         int m2 = (l + 2 * r) / 3;
 7.         if (arr[m1] == x || arr[m2] == x) {
 8.             return true;
 9.         }
10.         if (arr[m1] > x) {
11.             return ternarySearch(arr, 0, m1 - 1, x);
12.         }
13.         if (arr[m2] > x) {
14.             return ternarySearch(arr, m1 + 1, m2 - 1, x);
15.         }
16.         return ternarySearch(arr, m2 + 1, r, x);
17.     }
```

Рекурсия – пряка и косвена рекурсия, линейна и разклонена рекурсия

Ако в дефиницията на някаква функция се използва самата функция, дефиницията на функцията е **рекурсивна**.

Примери:

Ако n е произволно естествено число, следната дефиниция на функцията факториел е рекурсивна:

$$n! = \begin{cases} 1, & n = 0 \\ n.(n-1)!, & n > 0 \end{cases}$$

*Условието при $n = 0$ не съдържа обръщение към функцията факториел и се нарича **гранично**.*

Редицата от числата на Фибоначи 0, 1, 1, 2, 3, 5, 8, 13, 21, 34, ... може да се дефинира рекурсивно по следния начин:

$$fib(n) = \begin{cases} 0, & n = 0 \\ 1, & n = 1 \\ fib(n-1) + fib(n-2), & n > 1 \end{cases}$$

В този случай имаме две гранични условия – при $n = 0$ и при $n = 1$.

Рекурсивната дефиниция на функция може да се използва за намиране стойността на функцията за даден допустим аргумент.

Известно е, че в тялото на всяка функция може да бъде извикана друга функция, която е дефинирана или е декларирана до момента на извикването ѝ. Освен това, в C++ е вграден т.нар. **механизъм на рекурсия** – разрешено е функция да вика в тялото си самата себе си.

Функция, която се обръща пряко или косвено към себе си, се нарича **рекурсивна**. Програма, съдържаща рекурсивна функция, се нарича **рекурсивна**. Общата препоръка обаче е, че ако за решаването на някаква задача може да се използва итеративен алгоритъм, то по-добре е той да се реализира. **Не се препоръчва** винаги използването на рекурсия, тъй като това води до загуба на памет (опасност от препълване на стековата памет) и време. Съществуват обаче задачи, които се решават трудно, ако не се използва рекурсия.

Съществува теорема, че **всяка рекурсивна функция може да се пренапише като итеративна**. При всяко рекурсивно обръщение в програмния стек се разпределя памет за параметрите и локалните променливи на функцията.

Етапи от изпълнението на рекурсията:

- **Разгъване** – когато се извикват вложените функции
- **Свиване** – когато вложените функции една по една връщат резултатите си

Характеристики и необходими условия на рекурсията:

- **Дълбочина** (характеристика) – броя на рекурсивните извиквания на функцията
- **Дъно** (необходимо условие) – необходимо условие за достигане на край на рекурсията

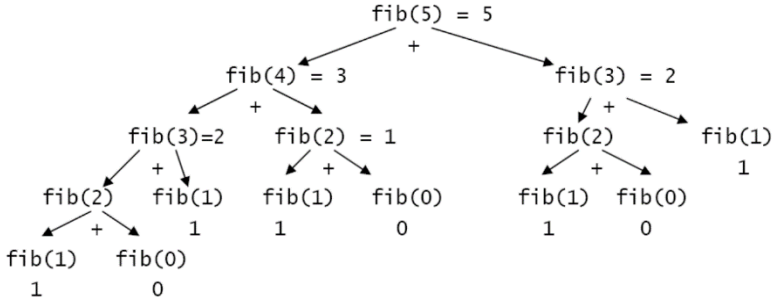
Рекурсията може да се раздели на няколко вида:

- **Пряка** – когато една функция извиква самата себе си
- **Косвена** (непряка, верижна) – когато няколко на брой функции се извикват и извикванията образуват цикъл
- **Линейна** – когато имаме едно обръщение към функцията в рамките на тялото ѝ
- **Разклонена** (дървовидна) – когато имаме повече от едно обръщение към функцията в рамките на тялото ѝ

Примери:

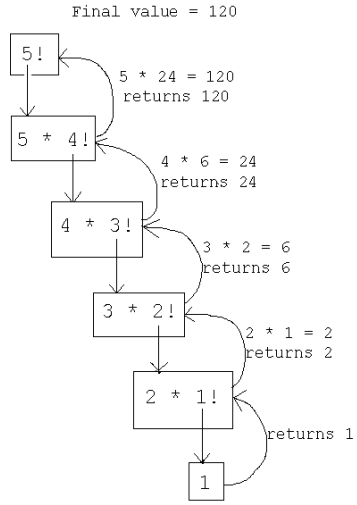
1. ***Разклонена рекурсия*** *– пресмятане на числата на Фибоначи*

```
1. unsigned int fibo(unsigned int n) {
2.  return n >= 2? fibo(n-1) + fibo(n-2):1;
3. }
```



2. ***Пряка и линейна рекурсия*** *– пресмятане на факториел на число*

```
1. long long factorial(int n) {
2.  if (n < 2) {
3.  return 0;
4.  }
5.
6.  return n * factorial(n - 1);
7. }
```



3. ***Непряка рекурсия*** *– примерна структура*

```
 1. fun1()
 2. {
 3. //  some code
 4. fun2()
 5. }
 6. fun2()
 7. {
 8. // some code
 9. fun3()
10. // some code
11. }
12. fun3()
13. {
14. // some code
15. fun1()
16. }
```

Държавен изпит – спец. „Софтуерно инженерство“

Тема №13 – Обектно-ориентирано програмиране. Основни принципи. Класове и обекти. Наследяване и капсулация.

Анотация: *Изложението по въпроса трябва да включва следните по-съществени елементи:*

1. Абстракция със структури от данни. Класове и обекти. Декларация на клас и декларация на обект. Основни видове конструктори. Управление на динамичната памет и ресурсите (“RAII”). Методи – декларация, предаване на параметри, връщане на резултат.

2. Наследяване. Производни и вложени класове. Достъп до наследените компоненти. Капсулация и скриване на информацията. Статични полета и методи.

Абстракция със структури от данни

Записът е съставна статична структура от данни, която се определя като крайна редица от фиксиран брой елементи, които могат да са от различни типове. Достъпът до всеки елемент от редицата е пряк и се осъществява чрез име, наречено поле на записа. Полетата на записа се представят последователно в паметта.

Примери:

1. Данните за студент от една група (име, адрес, факултетен номер, оценки по изучаваните предмети) могат да се зададат чрез запис с четири полета.
2. Данните за книга от библиотека (заглавие, автор, година на издаване, издателство, цена) могат да се зададат чрез запис с пет полета.
3. Комплексно число може да се зададе чрез запис с две реални полета.

В езика C++ записите се реализират чрез структури. Една структура се определя чрез имената и типовете на съставящите я полета:

```
struct <име_на_структурата>

     { <дефиниция_на_полета>;

     };
```

Структурите, дефинирани по този начин, могат да се използват като типове данни. Имената на полетата в рамките на една дефиниция на структура трябва да са различни идентификатори. Допуска се влагане на структури, т.е. поле на структура да е структура. Не е възможно обаче поле на структура да е от тип, съвпадащ с името на структурата. Допустимо е обаче поле с тип указател към структурата. За да е възможно две дефиниции на структури да се обръщат една към друга, е необходимо пред дефинициите им да се постави декларацията на втората структура.

Дефиницията на структура не предизвиква отделянето на памет за съхраняване на компонентите ѝ. Може да се постави извън функция, в началото на функция или в началото на блок. Местоположението на дефиницията определя областта на името на структурата – съответно за всички функции след дефиницията, в рамките на функцията и в рамките на блока.

Най-често дефиницията се задава пред първата функция на програмата и така става достъпна за всички функции.

При дефинирането на променлива от тип структурата не е задължително използването на ключовата дума **`struct`**. Ако са записани по-малко изрази от броя на полетата, компилаторът допълва останалите с нулевите стойности за съответния тип на поле.

Достъпът до полетата на структура е пряк. Един начин за неговото осъществяване е чрез променлива от тип структурата, като променливата и името на полета на структурата се разделят с оператора точка. Получените конструкции са променливи от типа на полето и се наричат полета на променливата от тип структура или член-данни на структурата, свързани с променливата.

Освен това, дефинирането на променлива от тип структура предизвиква отделяне на определено количество памет за всяко поле на променливата. Последното се определя от типа на полето. Полетата се разполагат последователно в паметта. Обикновено всяко поле се разполага от началото на машинна дума. Полетата, които не изискват цяло число на брой машинни думи, не използват напълно отредената им памет. Този начин за подреждане на полетата на структурата се нарича изравняване на границата на машинна дума. Две структури, дефинирани по един и същи начин, са различни.

Операциите над структури зависят от релацията на езика. По стандарт за всяка реализация са определени следните операции и вградени функции.

1. **Над полетата на променливи от тип структура** – всяко поле на променлива от тип структура е от някакъв тип. Всички операции и вградени функции, допустими над данните от този тип, са допустими и за полето на променливата.
2. **Над променливи от тип структура** – възможно е на променлива от тип структура да се присвои стойността на вече инициализирана променлива от същия тип структура или стойността на израз от същия тип. Възможно е формален параметър на функция, а също резултатът от изпълнението ѝ, да са структури. Структури с големи размери обикновено се предават чрез указатели или псевдоними на структури. Така се спестяват ресурси. Освен това, тези начини за предаване са по-сигурни.

Указателите към структури се дефинират по общоприетия начин.

Пример:

```
student st1;

student *pst = &st1;
```

В резултат за променливата указател се отделят 4B оперативна памет, в които се записва адресът на `st1`. Достъпът до полетата на променлива от тип структура чрез указател към нея се осъществява чрез обръщението: `(*<променлива_указател >).<име_на_поле>`, което е еквивалентно на `<променлива_указател>-><има_на_поле>`.

Пример:

```
pst -> facnom
```

За работа със структури от данни се използва подходът **абстракция със структури от данни**. При този подход методите за използване на данните са разделени от методите за тяхното представяне. Програмите се конструират така, че да работят с „абстрактни данни“ – данни с неуточнено представяне. След това представянето се конкретизира с помощта на множество функции, наречени конструктори, селектори и предикати, които реализират „абстрактните данни“ по конкретен начин.

Възниква въпросът дали може да използваме дефинирана структура като тип данни. Отговорът е не, защото при типовете данни представянето на данните е скрито за потребителя. На последния са известни множеството от стойности и операциите и вградените функции, допустими за типа.

Етикетите `private` и `public` се наричат спецификатори за достъп. Всички член-данни, следващи спецификатора за достъп `private`, са достъпни само за член-функциите от дефиницията на структурата. Всички член-данни и член-функции, следващи спецификатора за достъп `public`, са достъпни за всяка функция, която е в областта на структурата. Ако спецификаторите за достъп са пропуснати, всички членове са `public`. Един и същ спецификатор за достъп може да се използва повече от веднъж в една и съща дефиниция на структурата.

Ако заменим запазената дума `struct` с `class`, програмата не променя поведението си. Тези идеи са в основата на нов подход за програмиране – обектно-ориентирания подход.

Основни принципи на обектно-ориентираното програмиране

Обектно-ориентираното програмиране се основава на идеята за обект. Обектът е единица, която притежава данни и има поведение (извършва операции). Данните се наричат свойства на обекта, а операциите включват достъп и модифициране на тези свойства, както и други функции, които изобразяват поведението на обекта.

Основните принципи на обектно-ориентираното програмиране са:

1. **Енкапсулация** – обектите скриват вътрешното си състояние (данните, от които се състоят, връзките между тях и как те си взаимодействат). Само те могат да го модифицират директно. Клиентският код има достъп до вътрешното състояние на обекта само през специфични методи на самия обект. В C++ това се осъществява чрез групирането на данни и функции в клас или структура и с модификаторите за достъп `private`, `protected` и `public`.
2. **Абстракция** – обектите показват на външния свят само интерфейс, но не и допълнителни детайли за начина им на функциониране. Промяната в даден обект е локална и не се отразява върху обектите, които го използват. Това прави кода по-лесен за поддръжка. В C++ принципът на абстракция се осъществява чрез модификаторите за достъп, както и чрез специални header файлове, интерфейси и др.
3. **Наследяване** – клас може да наследява друг клас. Класовете, които се наследяват, се наричат базови, а класовете, които ги наследяват – дериватни, наследници. Дериватните класове ще наследят всички данни и функционалност на базовите класове, но достъпът им до тях ще зависи от спецификатора за достъп при наследяването и модификаторите за достъп в базовия клас. Дериватните класове не се ограничават до член данните и методите на базовия клас – те могат да дефинират свои данни, както и поведение. Най-

големите предимства на наследяването са преизползването на код (да се преизползват вече съществуващи функционалности) и създаването на йерархия по естествен начин. В C++ има механизми за единично и множествено наследяване. С множественото наследяване трябва да се подхожда внимателно.

4. **Полиморфизъм** – означава, че един и същ обект може да се държи различно в различни сценарии. Обектите в една йерархия на наследяване споделят някакво поведение (на най-базовия клас), което може да се извика или пък да се извика предефинираното поведение на класа наследник. В C++ полиморфизмът се осъществява по два начина:
    a. ***По време на компилация*** – написване на функции с едни и същи имена, но с различен брой аргументи или различни типове аргументи; пренаписване на оператори
    b. ***По време на изпълнение*** – пренаписване на функции от базовия клас в класа наследник; виртуални функции – функции, дефинициите на които се оставят да бъдат дефинирани от обектите наследници

Класове. Декларация на клас

Класовете са нови типове от данни, дефинирани от потребителя. Те могат да обогатяват възможностите на вече съществуващ тип или да представят напълно нов тип. Класовете са подобни на структурите, дори може да се каже, че в някои отношения са почти идентични. В C++ класът може да се разглежда като структура, на която са наложени някои ограничения по отношение на правата на достъп.

Основен принцип на процедурното програмиране е модулният. Програмата се разделя на „подходящи“ взаимосвързани части (функции, модули), всяка от които се реализира чрез определени средства. Важен е обаче начинът, по който да стане определянето на частите и връзките помежду им. Целта е следващи промени в представянето на данните да не променят голям брой от модулите на програмата. Разсъждения в тази посока довеждат до подхода абстракция със структури от данни, който разгледахме. При него методите за използване на данните се разделят от методите за тяхното конкретно представяне. Програмите се конструират, така че да работят с „абстрактни данни“ – данни с неуточнено представяне. След това представянето се конкретизира с помощта на множество функции, наречени конструктори, мутатори и функции за достъп, които реализират „абстрактните данни“ по конкретен начин.

Езикът C++ дефинира структурите и класовете почти идентично. Съществената разлика е свързана със спецификаторите на достъп. По подразбиране, членовете на структура има `public` (публичен) достъп, а членовете на клас – `private` (частен) достъп. Препоръчва се структурите да се използват само когато се реализират свойства, които са прости и включват малки „натоварвания“, по-точно, когато структурата от данни е идентична с интерфейса си. В останалите случаи е за предпочитане да се използват класове.

Класовете осигуряват механизми за създаване на напълно нови типове от данни, които могат да бъдат интегрирани в езика, а също за обогатяване възможностите на вече съществуващи типове. Дефинирането на един клас се състои от две части: декларация на класа и дефиниция на неговите член-функции (методи).

Декларацията на клас се състои от заглавие и тяло. Заглавието започва със запазената дума `class`, следвана от името на класа. Тялото е заградено във фигурни скоби. След скобите стои

знака ; или списък от имена на обекти. В тялото на класа са декларирани членовете на класа (член-данни и член-функции) със съответните им нива на достъп.

За имената на класовете важат същите правила, които се прилагат за имената на всички останали типове и променливи. Също като при структурите името на класа може да бъде пропуснато.

Имената на членовете на класа са локални за класа, т.е. в различни класове в рамките на една програма могат да се дефинират членове с еднакви имена. Член-данни от един и същи тип могат да се изредят, разделени със запетая и предшествани от типа им.

Пример:

```
class Point {

 private:

 double x, y;

 public:

 Point(double, double); // конструктор

 void read() // мутатор

 int get_x() const; // селектор, функция за достъп

 int get_y() const; // селектор, функция за достъп

 void print() const; // селектор, функция за достъп

}p = Point(2, 7), q(-2, 3), r = q;
```

Препоръчва се член-данните да се декларират в **нарастващ ред по броя на байтовете**, необходими за представянето им в паметта. Така за повечето реализации се получава оптимални изравняване до дума.

Типът на член-данна на клас не може да съвпада с името на класа, но типът на член-функция на клас може да съвпада с името на класа.

В тялото някои декларации на членове могат да бъдат предшествани от спецификаторите на достъп `private`, `public` или `protected`. Областта на един спецификатор на достъп започва от спецификатора и продължава до следващия спецификатор. Подразбиращ се спецификатор на достъп е `private`. Един и същ спецификатор на достъп може да се използва повече от веднъж в декларацията на клас. Препоръчва се, ако секция `public` съществува, да бъде първа в декларацията, а секцията private да бъде последна в тялото на класа.

Достъпът до членовете на класовете може да се разгледа на следните две нива:

1. **По отношение на член-функциите в класа** е в сила, че те имат достъп до всички членове на класа. При това не е необходимо тези компоненти да се предават като параметри. Този режим на достъп се нарича режим на пряк достъп.

2. **По отношение на функциите, които са външни за класа**, режимът на достъп се определя от начина на деклариране на членовете. Членовете на даден клас, декларирани като **`private`** са видими само в рамките на класа. Външните функции нямат достъп до тях. По подразбиране членовете на класовете са `private`. Чрез използването на членове, обявени като `private`, се постига скриване на членовете за външната на класа среда. Процесът на скриване се нарича още **капсулиране на информацията**. Членовете на клас, които трябва да бъдат видими извън класа (да бъдат достъпни за функции, които не са методи на дадения клас) трябва да бъдат декларирани като **`public`** (декларирани след запазената дума `public`). Модификаторът **`protected`** забранява външен достъп до данните/методите, декларирани в него, но те могат да бъдат достъпени от дериватните му класове (неговите наследници).

Когато един клас бъде дефиниран, никаква памет не се заделя за този клас. Това става тогава, когато се създават обекти.

Методи – декларация, предаване на параметри, връщане на резултат

След декларирането на клас трябва да се дефинират неговите методи. Дефинициите са аналогични на дефинициите на функции, но името на функцията се предшества от името на класа, на който принадлежи метода, следвано от оператора за принадлежност `::` (нарича се още оператор за област на действие). Такива имена се наричат пълни.

Дефиницията на метод има следната структура:

```
[<тип>] <име_на_клас>::<име_на_функция>(<параметри>) [const]
{
    <тяло>
}
```

Дефиницията на конструктор не започва с `<тип>`. Запазената дума `const` присъства в дефиницията на функция за достъп. Ако се пренебрегне това правило, ще се създават класове, които няма да могат да се използват от други програмисти.

Обикновено, дефинициите на методите са разположени веднага след декларирането на класа, на който те са членове. Възможно е обаче дефинициите на методите на един клас да бъдат част от декларациите на този клас, т.е. в декларациите на член-функциите на класа могат да се зададат не само прототипите им, но и техните тела. В този случай обаче член-функциите се третират като вградени (`inline`) функции.

Ще отбележим също, че в тялото на дефиницията на член-функция явно не се използва обектът, върху който тя ще се приложи. Той участва неявно – чрез член-данните на класа. Заради това се нарича неявен параметър, а член-данните – абстрактни данни. Параметри, които участват явно в дефиницията на член-функция се наричат явни. Всяка член-функция има точно едни неявен параметър и нула или повече явни.

За разлика от функциите, класовете могат да се декларират на различни нива в програмата: **глобално** (ниво функция) и **локално** (вътре във функция или в тялото на клас). Областта на глобално деклариран клас започва от декларацията и продължава до края на програмата. Ако клас е деклариран във функция, всички негови член-функции трябва да са вградени (`inline`). В противен случай, ще се получат функции, дефинирани във функция, което не е възможно. Областта на клас, дефиниран във функция, е функцията. Обектите на такъв клас са видими само в тялото на функцията. Възможно е използването на обекти с еднакви имена. Важи правилото, че локалният обект скрива нелокалния. Не е възможно в тялото на локално дефиниран клас да се използва функцията, в която класът е дефиниран.

Предаването на параметри може да става както при функциите – по стойност, чрез указател и псевдоним. При предаването по стойност се създава копие. За да се отразят евентуални модификации, предаването на параметъра трябва да става чрез указатели или псевдоними (по референция). При предаване на параметър чрез указател се очаква подаването на адреса на променливата, която ще се предава.

Обекти. Декларация на обект

След като даден клас е дефиниран, могат да бъдат създавани негови екземпляри, които се наричат **обекти**. Връзката между клас и обект в езика C++ е подобна на връзката между тип данни и променлива, но за разлика от обикновените променливи, обектите се състоят от множество компоненти (член-данни и член-функции).

Когато за даден клас явно са дефинирани конструктори, при всяко дефиниране на обект на класа те автоматично се извикват с цел да се инициализира обектът. Ако дефиницията е без явна инициализация, дефинираният обект се инициализира според дефиницията на конструктора по подразбиране, ако такъв е определен.

Когато за даден клас явно не е дефиниран конструктор, реализацията автоматично създава такъв. Последният се нарича подразбиращ се конструктор. Подразбиращият се конструктор изпълнява редица действия като заделяне на памет за обектите, инициализиране на някои системни променливи и др.

Дефиницията на клас не заделя памет за него. Памет се заделя едва при дефинирането на обект от класа. Достъпът до компонентите на обектите (ако е възможно) се осъществява чрез задаване на името на обекта и името на член-данната или метода, разделени с точка. Изключение от това правило са конструкторите.

При създаването на обекти на един клас кодът на методите на този клас не се копира във всеки обект, а се намира на едно място в паметта. Естествено възниква въпросът по какъв начин методите на един клас „разбират“ за кой обект на този клас са били извикани. Отговорът на този въпрос дава указателят `this`. Всяка член-функция на клас поддържа допълнителен формален параметър – указател с име `this` и от тип `<има_на_клас>*`. За да разберем точно как става това, ще разгледаме работата на компилатора на C++. Тя се извършва на две стъпки:

1. Всяка член-функция на даден клас се транслира в обикновена функция с уникално име и един допълнителен параметър – указателят `this`.
2. Всяко обръщение към член-функция се транслира в съответствие с правилото от 1.

Обекти от един и същи клас могат да се присвояват един на друг. Присвояването може да е на ниво инициализация. При присвояването се копират всички член-данни на обекта.

Основни видове конструктори

Създаването на обекти е свързано със заделяне на памет, запомняне на текущо състояние, задаване на начални стойности и други дейности, които заедно се наричат инициализация на обекта. В езика C++ тези дейности се изпълняват от специален вид член-функции на класовете – **конструкторите**.

Дефиницията на конструктор изглежда по следния начин:

```
<име_на_клас>::<име_на_клас>(<параметри>): <член_данна>(<израз>), …
{
    <тяло>
}
```

Конструкторът е член-функция, която притежава повечето характеристики на другите член-функции, но има и редица особености като:

- Името на конструктора съвпада с името на класа
- Типът на резултата е указателят this и явно не се указва
- Изпълнява се автоматично при създаване на обекти
- Не може да се извиква явно

Освен това в един клас може явно да не е дефиниран конструктор, но може да са дефинирани и няколко конструктора. В заглавието на конструктора е възможно да се свърже член-данна с инициализираща стойност (чрез **инициализиращ списък**). Смисълът на обобщената синтактична конструкция е, че инициализацията на член-данните в заглавието предшества изпълнението на тялото на конструктора. Това я прави изключително полезна, тъй като когато член-данни на класа са обекти, в дефиницията на конструктора на класа се използват конструкторите на класовете, от които са обектите (член-данни). Преди да започне изпълнението на конструктора, автоматично се извикват конструкторите по подразбиране на всички член-данни, които са обекти. Веднага след това тези член-данни се инициализират с обектите, резултат от изпълнението на извиквания конструктор. Това двойно извикване на конструктори намалява ефективността на програмата.

В един клас може да са дефинирани **няколко конструктора**. Всички те имат едно име (името на класа), но трябва да се различават по броя и/или типа на параметрите си. Наричат се предефинирани конструктори. При създаването на обект на класа се изпълнява само един от тях. Определя се съгласно критерия за най-добро съвпадение.

1. Конструктор по подразбиране (default constructor)

Генерира се автоматично от компилатора, когато не са дефинирани други конструктори. Не приема аргументи и единствено инициализира всички полета със стойностите им по подразбиране или със специфично избрани стойности, ако конструкторът по подразбиране е предефиниран. Извиква се, когато обект се създава без скоби.

```
class A {

A() {

//logic of the constructor

}

// other logic of the class

};

A a; // default constructor called
```

2. **Конструктор с параметри (parametrized constructor)**

Това е конструктор, който приема параметри. Дефинира се от програмиста, който решава и как точно да използва подадените му като аргументи параметри, за да инициализира с тях инстанция на клас. Може да има произволен брой параметризирани конструктори. Някои от параметрите, които даденият конструктор приема, може да са подразбиращи се (с дефинирана предварително от програмиста стойност, в случай, че параметърът отсъства). В такъв случай е задължително те да са последни в списъка с формални параметри.

При наличието на поне един конструктор в дефиницията на класа, конструкторът по подразбиране бива предефиниран и повече не може да се използва. Ако все пак искаме да можем да генерираме обекти използвайки конструктор по подразбиране, трябва да добавим експлицитно и конструктор по подразбиране, дори и тялото му да е празно.

3. **Копиращ конструктор (copy constructor)**

Това е специален параметризиран конструктор, който като единствен параметър получава псевдоним от тип **<име_на_клас> const &** на обект от същия клас. Работата му е да клонира подадения обект. Компилаторът създава копиращ конструктор по подразбиране. При наличието на динамично заделяне на памет за член-данни на класа, този конструктор трябва да бъде предефиниран ръчно от програмиста.

```
class A {

     A(const A& a) {

          // logic of the copy constructor

     }

     // other logic of the class

};

A a;

A b = a; // copy constructor called
```

Управление на динамичната памет и ресурсите (RAII)

Динамичните данни са такива обекти, чийто брой не е известен в момента на проектирането на програмата. Те се създават и разрушават по време на изпълнението на програмата. След разрушаването им, заетата от тях памет се освобождава и може да се използва отново. Така паметта се използва по-ефективно. Динамичните обекти се съхраняват в динамичната памет (heap). Създаването и разрушаването на динамични обекти в C++ се осъществява съответно чрез операторите `new` и `delete`. Извикването на `new` заделя в heap-а необходимата памет и връща указател към първия байт от структурата ѝ. Този указател може да се съхрани и да се пази, докато е необходимо. Освобождаването на паметта от heap-а става посредством оператора `delete`. На всяко заделяне на памет с `new` трябва да съответства изтриването ѝ с `delete` (в противен случай ще има memory leak), за да може тази памет да се използва за други цели. Това се налага, тъй като динамичната памет не се изчиства автоматично, а в C++ липсва система за самоизчистване на паметта, която е динамично заделена и няма откъде да се адресира (garbage collector).

Операторът `delete` трябва да се използва само за освобождаване на динамична памет, заделена с `new`. В противен случай, действието му е непредсказуемо. `delete` може да се прилага и към указател със стойност 0 (`nullptr`), защото той не адресира нищо и фактически не заема динамична памет. Динамичната памет не е неограничена. Ако тя бъде изчерпана по време на изпълнение на програмата, операторът `new` ще върне `nullptr`.

Разрушаването на обекти на класове в някои случаи е свързано с извършването на определени действия, които се наричат заключителни. Пример за такива действия са освобождаване на динамична памет и възстановяване на състояние на програма. Заключителните действия имат ефект, противоположен на инициализацията. За да не се специфицира всеки път една и съща последователност от заключителни действия за всеки обект на даден клас, се използва **деструкторът**, който се извиква автоматично при разрушаването на даден обект. Това разрушение може да настъпи поради две причини:

1. Извиква се оператор `delete`
2. Излиза се от блока, в който е бил създаден обект от класа

В един клас може да присъства точно един явно дефиниран деструктор. Неговата дефиниция изглежда по следния начин:

```
~<class_name>() { … }
```

Дефинирането на деструктор невинаги е наложително. Член-данните на даден обект автоматично се разрушават при разрушаването на самият обект. Но, когато някои член-данни са обвързани с динамичната памет, използването на деструктор е задължително, тъй като трябва да се освободи заделената динамична памет. Ако в класът явно не е указан деструктор, то самият компилатор предоставя деструктор по подразбиране (но той не решава проблемите с динамично заделените обекти).

Една от основните задачи на всяка нетривиална програма е да управлява ресурси, а фундаментална техника за управление на тези ресурси в C++ е тази, означавана с идиома **„Придобиването на ресурс е инициализация“ – Resource Acquisition Is Initialization (RAII)**. В основата ѝ стои идеята, че *ресурсите се придобиват в конструктори и се*

освобождават от деструктори. Това автоматизира освобождаването, тъй като разположените в стека обекти (локални обекти) се разрушават автоматично, когато се напусне областта им на действие. Така RAII подобрява надеждността на кода в две посоки – освобождава програмиста от задължението да помни, че трябва да освободи ресурсите и гарантира освобождаване на ресурса при възникване на изключение.

Една програма може да управлява **различни видове ресурси** – динамична памет, файлови манипулатори, буфери, сокети, мутекси, манипулатори на нишки и др. Най-общо казано ресурс е „всичко, което програмата придобива от друга част на системата и трябва (изрично или неявно) да освободи и върне на неговия собственик, след като го е използвала“. Паметта е важен пример за ресурс. Ако ресурсите не се освобождават правилно, това ще се отрази на производителността на системата и в крайна сметка, след по-продължително използване, системата ще блокира поради липса на свободна памет.

Ако се налага дадено действие да се извършва при всеки възможен изход от блок, естественият подход е то да се постави в деструктора на локален обект. Деструктор за обект се извиква, когато този обект се унищожава, независимо дали това става изрично с оператор `delete` или мълчаливо, когато обектът излиза от обхват (out of scope). Това дава възможност да се изчистят всички притежавани от него ресурси и предотвратява изтичането на памет или друг вид ресурс (resource leaks). Обектът придобива ресурси, най-често памет, при своята инициализация (в рамките на конструктора си) и след това освобождава тези ресурси в деструктора. Това е изключително мощна техника и улеснява много писането на стабилен код. Няма значение как обектът излиза от обхват – при достигане края на блока, в който е дефиниран; при изпълнение на оператори като `break`, `continue` или `goto`; когато с ранен `return` се прекратява изпълнение на функция; при възникване на изключение. Както и да се унищожава обектът, деструкторът ще се изпълни. Това означава, че не е необходимо кодът да се претрупва с обръщения към „почистващи“ функции. Достатъчно е да се дефинира клас, който ще управлява ресурса и в неговия деструктор да се извикат необходимите почистващи функции. Управлението се осъществява чрез обект на класа. Това гарантира, че ресурсът ще бъде коректно освободен, когато обектът бъде разрушен. Такива обекти са известни с термина RAII обекти, а класовете, чиито инстанции са те – RAII класове или resource management classes.

Наследяване

Производните класове и наследяването са една от най-важните характеристики на обектно-ориентираното програмиране (ООП). Чрез механизма за наследяване от съществуващ клас се създава нов клас. Класът, от който се създава, се нарича **базов** (основен) клас, а този, който е създаден – **производен** (дериватен). Понятията основен и производен клас са относителни, тъй като производен клас може да е основен за други класове, а основен – да е производен от други основни класове. Производният клас може да наследи компонентите на един или няколко базови класа. В първия случай наследяването се нарича **единично** (просто), а във втория – **множествено**.

Дефинирането на производни класове е еквивалентно на конструирането на йерархии от класове. Ако множество от класове имат общи данни и методи, тези общи части могат да се обособят като основни класове, а всяка от останалите части да се дефинира като производен клас на съответния основен клас. Така се прави икономия на памет, тъй като се избягва многократното описание на едни и същи програмни фрагменти.

Производни и вложени класове

При конструирането на **производни класове** е достатъчно да се разполага само с обектните модули на основните класове, а не с техния програмен код. Това позволява да бъдат създавани библиотеки от класове, които да бъдат използвани при създаването на производни класове.

Подобно на обикновените, производните класове се дефинират като се декларира класът и се дефинират неговите методи:

```
 class     <име_на_производен_клас>     :     [<атрибут_за_област>]
<име_на_базов_клас>, …

{

     <декларация_на_компоненти>

};
```

Пред всяко име на базов клас може да се постави запазената дума `public`, `private` или `protected`. Нарича се атрибут за област, тъй като определя областта на наследените членове. Употребата на атрибутите за област е различна от тази за обновяване на секции в тялото на класа. Ако атрибут за област е пропуснат, подразбира се `private`.

Пример:

```
class der : public base1, base2, protected base3

{

     …

};
```

Декларациите на компонентите на производен клас, а също дефинициите на неговите методи не се различават от съответните при обикновените класове. Производният клас може да дефинира допълнително свои член-данни и методи, аналогични на тези на основния клас, а също и нови. Дефинираните в производния клас данни и методи се наричат **собствени**. Производният клас е дефиниран след като вече е дефиниран базовият клас, от който той произлиза. Чрез него се разширява декларацията на съществуващ клас. Разширяемостта на класовете е една от важните характеристики на ООП.

Вложените класове са класове, дефинирани в рамките на друг клас. Те могат да бъдат използвани, когато има тясна връзка между класовете и единият клас има нужда от достъп до членовете или методите на другия. Нека разгледаме пример:

```
#include <iostream>

// Външен клас

class Outer {

public:
```

```
    // Вложен клас
    class Inner {
    public:
        void display() {
            std::cout << "This is the inner class." << std::endl;
        }
    };
};

int main() {
    Outer::Inner innerObj; // Създаване на обект от вложения клас
    innerObj.display(); // Извикване на метод от вложения клас
    return 0;
}
```

Когато говорим за връзката между производните и вложените класове от гледна точка на композицията и наследяването, трябва да разгледаме как тези концепции се използват за построяване на връзки между класовете в програмата.

Наследяването е механизъм, при който един клас (наследникът) придобива свойствата и методите на друг клас (родителя), който се нарича базов клас. В случая с производните класове, новият клас разширява функционалността на базовия клас чрез добавяне на допълнителни членове и методи или чрез промяна на вече съществуващите. Този вид връзка представлява "е-наследяване", където производният клас "е" базовият клас.

Композицията е механизъм, при който един клас съдържа обекти от друг клас като членове. В случая с вложените класове, външният клас съдържа обект от вложения клас. Тази връзка се нарича "**има-обект**" (*HAS-A*). Външният клас "има" обект от вложения клас като член.

Взаимоотношението между наследяването и композицията е важно и се използва в проектирането на софтуерни системи. В общи линии, ако има ясна йерархия на класовете, където един клас представлява разширение или специализация на друг, наследяването е подходящ избор. С други думи, ако класът "**е**" (*IS-A*) нещо друго (например класът Dog "is an" Animal), то наследяването е подходящо.

От друга страна, ако връзката между двата класа е по-слаба или ако единият клас е част от другия, композицията е по-подходящ избор. Например, ако класът "използва" друг клас като част от своята реализация, композицията е предпочитаната връзка.

Достъп до наследените компоненти. Капсулация и скриване на информацията

В рамките на един клас без наследяване, `protected` частта има аналогична роля като тази на `private` частта. До компоненти от тип `protected` имат пряк достъп само член-функции и приятелски функции на класа.

Атрибутът за област на базовия клас в декларацията на производния клас (`public`, `private` или `protected`) управлява механизма на наследяване и определя какъв да бъде режимът на достъп до наследените членове. Таблицата показва наследяванията на компоненти на основен клас в зависимост от атрибута за област:

Атрибут за област	**Компонента на основен клас, определена като:**	**Наследява се като:**

`public`	`private`
	`public`
	`protected`
		`private`
		`public`
		`protected`

`private`	`private`
	`public`
	`protected`
		`private`
		`private`
		`private`

`protected`	`private`
	`public`
	`protected`
		`private`
		`protected`
		`protected`

Наследените компоненти обаче се различават от декларираните в производния клас по **правата за достъп**. **Производният клас има пряк достъп до компонентите, декларирани като `public` и `protected`, но няма пряк достъп до декларираните като `private` в базовия клас.** Достъпът до `private` компонентите на базовия клас се извършва чрез неговия интерфейс. По този начин се получава **скриване на информация**.

Следващата таблица показва **прекия достъп** (ПД) на член-функции и на обекти на производния клас и **външния достъп** (ВД) на производния клас до компонентите на базовия клас:

Компонента на базовия клас	Производен клас с атрибут `public`		Производен клас с атрибут `private`		Производен клас с атрибут `protected`	
	ПД	**ВД**	**ПД**	**ВД**	**ПД**	**ВД**
`public`	Да	Да	Да	Не	Да	Не
`protected`	Да	Не	Да	Не	Да	Не
`private`	Не	Не	Не	Не	Не	Не

Статични полета и методи

Обикновено, всеки обект на клас има свое собствено копие на всички данни – елементи на класа. В определени случаи обаче е нужно да се използва **само едно копие** на променлива съвместно от всички обекти на класа. За тези и за други цели се използват **статични** променливи в класа, които съдържат информация, която може да се използва от всички обекти на целия клас. Декларирането на **статичните полета** започва с ключовата дума **`static`**.

Препоръчва се с цел икономия на памет, ако е достатъчно единствено копие на данните, да се използват **статични полета**. Статичните полета могат да бъдат `public`, `private` или `protected`. На тях могат да се задават начални стойности само веднъж в областта на действие на файла. Достъпът до публичните статични полета на класа е възможен посредством всеки обект на класа или посредством името на класа. Статичните полета на клас са достъпни (съществуват) дори когато не съществуват обекти от този клас. За осигуряване на достъп до `private` и `protected` статични полета трябва да се предвиди публична статична функция.

Една функция може да бъде декларирана като статична, ако тя няма достъп до нестатични елементи на класа. За разлика от нестатичните методи, в статичните не може да се използва указател `this`, тъй като статичните полета и методи съществуват независимо от обектите на класа. Следователно, статичните методи не могат да бъдат декларирани като `const`.

Пример:

```
class Employee {

 public:

 Employee(const char *, const char *);

 ~Employee();

 const char *getFirstName() const;

 const char *getLastName() const;

 static int getCount(); // връща броя на създадените обекти
```

```
 private:
 char *firstName;
 char *lastName;
 static int count;
};
int Employee::count = 0;
int Employee::getCount() {
 return count;
}
```

Държавен изпит – спец. „Софтуерно инженерство“

Тема №14 – Обектно-ориентирано програмиране. Подтипов и параметричен полиморфизъм. Множествено наследяване.

Анотация: *Изложението по въпроса трябва да включва следните по-съществени елементи:*

1. Виртуални функции и подтипов полиморфизъм. Динамично свързване. Абстрактни методи и класове. Масиви от обекти и от указатели към обекти.

2. Параметричен полиморфизъм. Шаблони на функция и на клас

3. Множествено наследяване.

Множествено наследяване

При **простото наследяване** всеки производен клас се поражда само от един базов клас. Йерархията при простото наследяване може да се изобрази чрез дърво. При **множественото наследяване** производният клас се поражда от два или повече базови класа. Йерархията при множественото наследяване може да се изобрази чрез ориентиран ацикличен граф. Йерархията на наследяването може да бъде с произволна дълбочина в границите на физическите ограничения за конкретната система.

Пример:

```
class Derived : public Base1, public Base2 { … };
```

В примера класът `Derived` се поражда от два класа `Base1` и `Base2` чрез множествено наследяване – то се указва с `:` след името на класа и следващ списък на базовите класове, разделени със запетаи. Конструкторът на `Derived` извиква явно конструкторите на всеки от своите базови класове `Base1` и `Base2` с използване на списък от инициализатори на елементи. Конструкторите на базовите класове се извикват в този ред, в който е определено наследяването, а не в този, в който конструкторите са описани.

Редът на извикване на конструкторите е следният:

1. **Конструкторите на базовите класове по реда на тяхното задаване** не в инициализиращия списък на конструктора, а **в декларацията на производния клас**.
2. **Конструкторите на класовете**, чиито обекти са **членове на производния клас**. Редът на извикване съответства на реда на деклариране на тези членове в тялото на производния клас.
3. **Конструкторът на производния клас**.

Всеки **деструктор** се грижи само за собствените си компоненти. Извикването им става в **обратен ред** – първо се извиква деструкторът на производния клас, след това в обратен ред, се извикват деструкторите на класовете на обектите – членове на производния клас и най-накрая на основните му класове, отново в обратен ред на реда на извикване на техните конструктори.

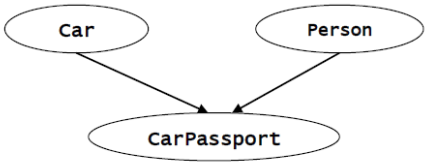
Чрез обект на производния клас могат да се викат всички негови `public` компоненти – собствени и наследени. Ако в базовите класове са дефинирани компоненти с еднакви имена, необходимо е да се използва пълното име на компонентата, т.е.

<име_на_клас>::<компонента>. Ако всички основни класове basei на производния клас der имат член-функция f() и член-данна x с еднакви имена, различаването им в производния клас става чрез пълните им имена:

base1::f(), base2::f(), …

base1::x, base2::x, …

Пример (голяма четворка при множествено наследяване):

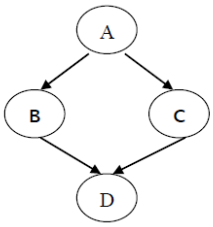



```
#include <iostream>
#include <string>
using namespace std;
class Car
{
public:
     Car(char * = "", unsigned = 0, unsigned = 0);
     ~Car();
     Car(const Car&);
     Car& operator=(const Car&);
     void display() const
private:
     char *mark;
     unsigned year;
     unsigned reg_numb
};
Car::Car(char *m, unsigned y, unsigned r_n)
{
     mark = new char[strlen(m)+1];
     strcpy(mark, m);
     year = y;
     reg_numb = r_n;
}
Car::~Car()
{
     cout << "~Car()\n";
     delete mark;
}
Car::Car(const Car& c)
{
     mark = new char[strlen(c.mark)+1];
     strcpy(mark, c.mark);
     year = c.year;
     reg_numb = c.reg_numb;
}
Car& Car::operator=(const Car& c)
{
     if (this != &c)
     {
       delete mark;
       mark = new char[strlen(c.mark)+1];
       strcpy(mark, c.mark);
       year = c.year;
       reg_numb = c.reg_numb;
     }
     return *this;
}
void Car::display() const
{
     cout << "Mark: " << mark << endl;
     cout << "Year: " << year << endl;
```

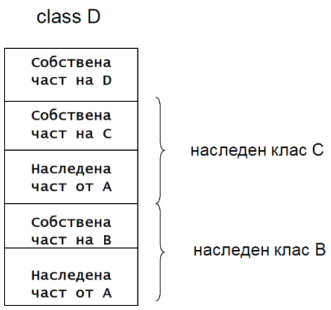
```
52.      cout << "Reg. Number: " << reg_numb << endl;
53. }
54. class Person
55. {
56. public:
57.      Person(char * = "", char * = "");
58.      Person(const Person&);
59.      Person& operator=(const Person& p);
60.      void display() const;
61.      ~Person();
62. private:
63.      char * name;
64.      char * egn;
65. };
66. Person::Person(char *str, char *num)
67. {
68.      name = new char[strlen(str)+1];
69.      strcpy(name, str);
70.      egn = new char[11];
71.      strcpy(egn, num);
72. }
73. Person::Person(const Person& p)
74. {
75.      name = new char[strlen(p.name)+1];
76.      strcpy(name, p.name);
77.      egn = new char[11];
78.      strcpy(egn, p.egn);
79. }
80. Person& Person::operator=(const Person& p)
81. {
82.      if (this != &p)
83.      {
84.        delete name;
85.        delete egn;
86.        name = new char[strlen(p.name)+1];
87.        strcpy(name, p.name);
88.        egn = new char[11];
89.        strcpy(egn, p.egn);
90.      }
91.      return *this;
92. }
93. void Person::display() const
94. {
95.      cout << "Ime: " << name << endl;
96.      cout << "EGN: " << egn << endl;
97. }
98. Person::~Person()
99. {
100.      cout << "~Person()\n";
101.      delete name;
102.      delete egn;
103. }
104. class CarPassport: public Car, public Person
105. {
106. public:
107.      CarPassport(char* = "", unsigned = 0, unsigned = 0,
108.                                  char* = "", char* = "");
109.      ~CarPassport();
110.      CarPassport(const CarPassport&);
111.      CarPassport& operator=(const CarPassport&);
112.      void display() const;
113. };
114. CarPassport::CarPassport(char *mark, unsigned year,
115.                          unsigned reg_numb, char* name, char *egn) :
116.                                  Car(mark, year, reg_numb), Person(name, egn)
117. {
118. }
119. CarPassport::~CarPassport()
120. {
121.      cout << "~CarPassport()\n";
122. }
123. CarPassport::CarPassport(const CarPassport& cp) :
```

```
124.                                          Car(cp), Person(cp)
125. {
126. }
127. CarPassport& CarPassport::operator=(const CarPassport& cp)
128. {
129.      if (this != &cp)
130.      {
131.        Car::operator =(cp);
132.        Person::operator =(cp);
133.      }
134.      return *this;
135. }
136. void CarPassport::display() const
137. {
138.      Car::display();
139.      Person::display();
140. }
141. void main()
142. {
143.      CarPassport x("FORD FIESTA", 2000, 2295,
144.                                    "Vassil Todorov", "8012174586");
145.      x.display();
146.      CarPassport y("LADA", 1900, 8817,
147.                                    "Sonia Todorova", "8203314576");
148.      y = x;
149.      y.display();
150. }
```

Стандартният механизъм за наследяване дефинира йерархия, която се представя чрез **дърво**. В този случай, при всяко срещане на основен клас се създава копие на неговите член-данни. **Виртуалните основни класове** са механизми за отменяне на стандартния наследствен механизъм. При реализиране на йерархии на класове с множествено наследяване е възможно един производен клас да наследи многократно даден базов клас. Например, ако класът `A` има два производни класа `B` и `C`, които са базови за класа `D`.



Като производни на класа `A`, класовете `B` и `C` наследяват неговите компоненти. От друга страна класовете `B` и `C` са базови за класа `D`. Следователно, класът `D` ще наследи два пъти компонентите на класа `A`, веднъж чрез класа `B` и още веднъж – чрез класа на `C`. За член-функциите двойното наследяване не е от значение, тъй като за всяка член-функция се съхранява само едно копие. Член-променливите обаче се дублират и обект на класа `D` ще наследи двукратно всяка член-променлива, дефинирана в класа `A`. Класът `D` в паметта ще има вида:



Този пример илюстрира един от недостатъците на многократното наследяване на клас – **неефективността от поддържането на множество копия** на наследени компоненти. Друг недостатък е **нееднозначността**, която се получава. Този проблем е известен още и като **диамантен проблем** (*diamond problem*).

Многократното наследяване на клас води от една страна до затруднен достъп до многократно наследените членове, а от друга до поддържане на множество копия на член-данните на многократно наследения клас, което не е ефективно.

Преодоляването на недостатъците на многократното наследяване на клас се осъществява чрез използването на т.нар. **виртуални основни класове**. Чрез тях се дава възможност да се „поделят“ основни класове. Когато един клас е виртуален, независимо от участието му в няколко списъка на основни класове, се създава само едно негово копие. В нашия случай, ако класът `A` се определи като виртуален за класовете `B` и `C`, класът `D` ще съдържа само един „поделен“ основен клас `A`.

Декларацията на основен клас като виртуален се осъществява като в декларацията на производния клас заедно с името и атрибута за област на основния клас се укаже и запазената дума `virtual`. В нашия случай, казваме, че `B` и `C` наследяват класа `A` виртуално. Виртуалното наследяване на класа `A` от класовете `B` и `C` оказва влияние само на производните на `B` и `C` класове. То не променя поведението на самите класове `B` и `C`. Няма значение редът на `virtual` и атрибута за област пред класа `A`.

Дефинирането и използването на виртуални класове има редица особености. Една от тях касае дефиницията и използването на конструкторите на наследените класове. Правилото за извикване на конструктори с параметри на виртуални класове може да се изкаже така: **конструкторите с параметри на виртуални класове трябва да се извикват от конструкторите на всички класове, които са техни наследници, а не само от конструкторите на преките им наследници**. С други думи, производният клас е отговорен за инициализирането на класовете, от които произлиза, както и на всички виртуални основни класове. Ако в инициализиращия списък на конструктора на производен клас няма обръщение към конструктор с параметър на виртуалния клас, използва се неговия подразбиращ се конструктор, ако такъв съществува или се съобщава за отсъствието на подходящ конструктор.

Пример:

```
D(int a, int b, int c, int d): A(a), B(a, b), C(c, d) {…}
```

Друга особеност е промяната на реда на инициализиране. Инициализирането на виртуални основни класове предхожда инициализирането на другите основни класове в декларацията на производния клас. Ако производен клас наследява основен и виртуален клас, **конструкторът на виртуалния клас се извиква първи**. При няколко виртуални класа извикването на конструкторите става по реда им в декларацията на производния клас.

Виртуални функции и подтип полиморфизъм. Динамично свързване

С помощта на **виртуални функции** и **полиморфизъм** могат да се разработват и реализират програми за обобщена обработка на обекти от всички съществуващи в йерархията класове като обекти от базовия клас. Ако по време на разработката на такава програма трябва да се добави нов клас в йерархията, това може да стане с незначителни изменения на самата програма.

Единствените части на програмата, които трябва да се модифицират, са частите, изискващи непосредствено познаване на конкретния клас, добавян в йерархията.

В случая на обикновени функции за разпознаването на функцията се използва механизъм, който се изразява в следното: по време на компилация се сравняват формалните с фактическите параметри в обръщението и по правилото за най-доброто съвпадане се избира необходимата функция. След заместване на формалните с фактическите параметри се изпълнява тялото на функцията.

При член-функциите на йерархията от класове, конфликтът между имената на наследените и собствените методи от един и същ тип и с едни и същи параметри се разрешава също по време на компилация чрез правилото на локалния приоритет и чрез явно посочване на класа, към който принадлежи методът.

В тези два случая тъй като процесът на реализиране на обръщението към функцията приключва по време на компилация и не може да бъде променян по време на изпълнение на програмата се казва, че има **статично разрешаване на връзката** или **статично свързване**. При статичното свързване се нуждаем често от явни преобразувания. По време на създаването на класа трябва да се предвидят възможните обекти, чрез които ще се викат член-функциите му. При сложни йерархии от класове това е не само трудно, но и понякога невъзможно.

Езикът C++ поддържа още един механизъм, прилаган върху специален вид член-функции, наречен **късно** или **динамично свързване**. При него изборът на функцията, която трябва да се изпълни, става по време на изпълнение на програмата.

Динамичното свързване капсулира детайлите в реализацията на йерархията. При него не се налага проверка на типа. Текстовете на програмите се опростяват, а промени се налагат много по-рядко. Разширяването на йерархията не създава проблеми. Това обаче е с цената на усложняване на кода и забавяне на процеса на изпълнение на програмата.

Наличието на двата механизма на свързване – статично и динамично, дава възможност на програмиста да се възползва от положителните им страни.

Прилагането на механизма на динамичното свързване се осъществява върху специални член-функции на класове, наречени виртуални член-функции или само виртуални функции. Виртуалните методи се декларират чрез поставяне на запазената дума **`virtual`** пред декларацията им:

```
virtual <тип_на_резултата> <име_на_метод>(<параметри>);
```

```
 1. #include <iostream>
 2. using namespace std;
 3. class Point2
 4. {
 5. public:
 6.       Point2(int a = 0, int b = 0) : x(a), y(b) {}
 7.       virtual void print() const
 8.       {
 9.         cout << x << ", " << y;
10.       }
11. private:
12.       int x;
13.       int y;
14. };
15. class Point3 : public Point2
```

```
16. {
17. public:
18.       Point3(int a = 0, int b = 0, int c = 0) : Point2(a, b), z(c) {}
19.       virtual void print() const
20.       {
21.         Point2::print();
22.         cout << ", " << z << endl;
23.       }
24. private:
25.       int z;
26. };
27. class ColPoint3 : public Point3
28. {
29. public:
30.       ColPoint3(int a = 0, int b = 0, int c = 0, int col = 0) :
31.                                                   Point3(a, b, c), color(col) {}
32.       virtual void print() const
33.       {
34.         Point3::print();
35.         cout << "color: " << color << endl;
36.       }
37. private:
38.       int color;
39. };
40. void main()
41. {
42.       Point2 p2(5, 10);
43.       Point3 p3(2, 4, 6);
44.       ColPoint3 p4(12, 24, 36, 11);
45.       Point2 *ptr1 = &p3; // атрибутът на Point2 е public
46.       ptr1->print();
47.       cout << endl;
48.       Point2 *ptr2 = &p4; // атрибутът на Point2 е public
49.       ptr2->print();
50.       cout << endl;
51. }
```

Резултат:

2, 4, 6

12, 24, 36

color: 11

Декларирането на член-функцията `print()` като виртуална е причина обръщенията `ptr1->print();` и `ptr2->print();` да определят функцията, която ще бъде извикана едва при изпълнението на програмата. Определянето е в зависимост от типа на обекта, към който сочи указателят, а не от класа към който е указателят. В случая, указателят `ptr1` е към класа `Point2`, но сочи обекта `p3`, който е от класа `Point3`. Затова обръщението `ptr1->print();` изпълнява `Point3::print()`. Указателят `ptr2` е към класа `Point2`, но сочи обекта `p4`, който е от класа `ColPoint3`. Затова обръщението `ptr2->print();` изпълнява `ColPoint3::print()`.

Ще отбележим няколко особености:

1. Само член-функции на класове могат да се декларират като виртуални. По технически съображения конструкторите не могат да се декларират като виртуални.
2. Ако в даден клас е декларирана виртуална функция, декларираните член-функции със същия прототип (име, параметри и тип на върнатата стойност) в производните на класа класове също са виртуални дори ако запазената дума `virtual` бъде пропусната.

3. Ако в производен клас е дефинирана функция със същото име като определена вече в основен клас като виртуална член-функция, но с други параметри и/или тип, то това ще е друга функция, която може да бъде или да не бъде декларирана като виртуална.
4. Ако в производен клас е дефинирана виртуална функция със същия прототип като на невиртуална функция на основен клас, то те се интерпретират като различни функции.
5. Възможно е виртуална функция да се дефинира извън клас. Тогава заглавието ѝ не започва със запазената дума `virtual`, т.е. запазената дума `virtual` може да се среща само в тялото на класа.
6. Виртуалните функции се наследяват като другите компоненти на класа.
7. Основният клас, в който член-функция е обявена за виртуална, трябва да е с атрибут `public` в производните от него класове.
8. Виртуалните функции се извикват чрез указател към или псевдоним на обект от някакъв клас.
9. Виртуалната функция, която в действителност се изпълнява, зависи от типа на аргумента.
10. Виртуалните функции не могат да бъдат декларирани като приятели на други класове.

Сред предимствата на виртуалните функции са:

1. Производният клас наследява всяка виртуална функция на базовия клас, за която няма собствена дефиниция.

От тук следва, че не е задължително виртуалните функции да се декларират във всеки клас от йерархията. Ако виртуална функция е дефинирана в базов клас и логиката на производния клас не изисква нейното предефиниране, декларацията ѝ може да се пропусне. Когато бъде извикана виртуална функция за обект от даден клас, тя се търси първо в него. Ако не е дефинирана в класа, търсенето продължава в базовия клас и нагоре по йерархията.

2. Реализират се полиморфни действия.

Полиморфизмът е важна характеристика на ООП. Изразява се в това, че едни и същи действия (в общия смисъл) се реализират по различен начин в зависимост от обектите, върху които се прилагат, т.е. действията са полиморфни (с много форми).

Полиморфизмът е свойство на член-функциите на обектите и в езика C++ се реализира чрез виртуални функции. Това е т.нар. **подтипов полиморфизъм**, за който е характерно, че имаме една и съща сигнатура (прототип) за всички подтипове на даден тип.

За да се реализира полиморфно действие, класовете върху които то ще се прилага, трябва да имат **общ родител** или **прародител**, т.е. да бъдат производни на един и същи клас. В този клас трябва да бъде дефиниран виртуален метод, съответстващ на полиморфното действие. Във всеки от производните класове този метод може да бъде предефиниран съобразно особеностите на този клас.

Активирането на полиморфното действие става чрез **указател към базовия клас**, на който могат да се присвоят адресите на обекти на който и да е от производните класове от йерархията. Ще бъде изпълнен методът на съответния обект, т.е. в зависимост от обекта, към който сочи указателят, ще бъде изпълняван един или друг метод.

Абстрактни методи и класове. Масиви от обекти и от указатели към обекти

Ако класовете, в които трябва да се дефинират виртуални методи, нямат общ родител, такъв може да бъде създаден изкуствено чрез дефиниране на т.нар. **абстрактен клас**.

Преди да разгледаме **абстрактните класове**, ще се спрем на още един важен въпрос – **достъпът до виртуална функция**. Всяка член-функция на клас, в който е дефинирана виртуална функция, има пряк достъп до виртуална функция, т.е. на локално ниво достъпът се определя по традиционните правила. На глобално ниво достъпът е малко по-различен.

Достъпът до виртуална функция на глобално ниво зависи от секцията (в която е дефинирана тя) на класа, към който сочи указателят, чрез който се активира функцията (типа на указателя). Съществуват три случая, при които обръщението към виртуална функция се решава статично (по време на компилация):

1. Виртуалната функция се извиква чрез обект на класа, в който е дефинирана.
2. Виртуалната функция се активира чрез указател към или псевдоним на обект, но явно чрез операцията `::` е посочена конкретна функция.
3. Виртуалната функция се активира в тялото на конструктор или деструктор на основен клас.

При използване на полиморфизма за обработка на динамично разпределени обекти от йерархията на класовете, може да се появи проблем. Ако обектът се унищожава с явно използване на операцията `delete` над указател от базовия клас към обекта, то се извиква деструктора на базовия клас на дадения обект. Това става независимо от типа на обекта, към който сочи указателят от базовия клас и независимо от факта, че деструкторите на всеки клас имат различни имена.

Съществува просто решение на този проблем – обявяването на **деструктора на базовия клас като виртуален**. Това автоматично ще доведе до там, че всички деструктори на производните класове ще станат виртуални дори ако те имат имена, различни от името на деструктора на базовия клас.

В този случай, ако обекта в йерархията е унищожен с явно използване на операцията `delete`, приложена към указателя от базовия клас към обекта от производния клас, то ще се извиква деструктора от съответния клас. Когато производният клас е унищожен, частта от базовия клас, съдържаща се в производния клас също се унищожава. Деструкторът от базовия клас автоматично се изпълнява след деструктора на производния клас.

Възможно е виртуална член-функция да има само декларация, а не дефиниция. Такава виртуална член-функция се нарича **чисто виртуална (абстрактна)**. За да се определи една виртуална функция като чисто виртуална, се използва следния синтаксис:

```
virtual <тип> <име_на_функция>(<параметри>) = 0;
```

Клас, в който е декларирана поне една чисто виртуална функция, се нарича абстрактен. **Абстрактните класове** се характеризират със следните свойства:

1. Обекти от тези класове не могат да се създават, но е възможно да се дефинират указатели към такива класове

2. Чисто виртуалните функции задължително трябва да бъдат дефинирани в производните класове или да бъдат обявени за чисто виртуални. В последния случай класът наследник също е абстрактен.

Абстрактните класове са предназначени да служат за базови на други класове. Чрез тях могат да се обединят в обща структура различни йерархии. Абстрактен клас, който няма член-данни и всички член-функции са чисто виртуални, се нарича **интерфейс**.

Ако наследник на виртуален клас не реализира никоя чисто виртуална функция, то той също остава абстрактен. Абстрактните класове могат да имат конструктори и деструктори, но те винаги се извикват косвено от наследник. Може да имаме указатели или псевдоними към абстрактни класове. Абстрактните класове не могат да бъдат параметри на шаблон или тип на връщан резултат.

Полиморфизмът, с помощта на абстрактните класове, позволява създаването на класове с различна логическа структура. Пример за такава структура е списък, елементите на който са от различен тип – стек, дърво, опашка и други контейнери. Такива структури се наричат **хетерогенни** или **полиморфни**. Логическата структура на класа, представляващ хетерогенна конструкция, се реализира отделно от обектите, които се включват в него. Връзката между тях се осъществява чрез указатели към контейнери, които съхраняват обекти от различни класове.

Хетерогенните контейнери са контейнери, които съдържат обекти от различен тип. Масивите могат да играят такава роля. Реализират се чрез използване на **указател към полиморфен тип**. Не може да се използва директно обект или псевдоним. Над хетерогенните контейнери могат да се изпълняват масовите операции от общия интерфейс.

Параметричен полиморфизъм. Шаблони на функция и на клас

Параметричен полиморфизъм е вид полиморфизъм, при който имаме една и съща реализация независимо от типовете. Представлява механизъм за изграждане на отделни модули, които са с общо предназначение и поради това могат да бъдат използвани в различен контекст от различни клиенти. Модулите с общо предназначение могат да представят структури от данни или алгоритми.

Модулите с общо предназначение се наричат **шаблони** (*templates*). Шаблонът е програмен код, който съдържа части с общо предназначение. Той показва на компилатора как заместващи тези части с общо предназначение с конкретен код (обикновено конкретен тип) да построи конкретен изпълним програмен код.

В C++ могат да се използва **шаблонни класове** (*class templates*) и функции. Това са параметризирани класове, чийто параметър е някакъв тип.

Шаблонните класове дават възможност да се дефинира тип данни (под формата на клас), в който част от информацията за типа е неуточнена нарочно, с цел да бъде специфицирана по-късно. Ето защо те понякога се наричат параметризирани типове. При създаването на различни конкретни варианти на параметризирания клас тази информация за типа може да бъде попълнена по различен начин (по подобие на механизма за предаване на параметри на функции). По този начин на базата на шаблона могат да се генерират различни конкретни типове данни.

Чрез шаблонните класове можем да проектираме един клас, който може да оперира с данни от различни типове, вместо да създаваме отделни класове за всеки един от тези типове.

Шаблонните класове могат да са производни или да се наследяват, както от шаблонен, така и от обикновен клас. Ако обикновен клас наследява шаблонен клас, то в неговата декларация параметризираните типове трябва да получат конкретни стойности.

Броят на параметрите на шаблон на клас е произволен. Параметрите могат да участват на произволни места в дефиницията на шаблона. Освен това е възможно някои или всички параметри на шаблона да са подразбиращи се. Това се осъществява чрез добавяне на инициализацията = след името на параметъра. В този случай, при пропускане на параметър, се използва подразбиращият се тип.

Синтаксис:

```
template<[списък_параметри]> декларация_на_клас
```

`списък_параметри` е списък от типове (от вида `class` идентификатор или обикновени параметри, които се използват в тялото на шаблона)

Пример:

```
template <class T, class S = int>

class Example {

 public:

     T func1(T x, S y);

     S func2(T x, S y);

private:

     T a;

     S b;

};
```

Шаблоните на класове не са истински класове, а **описания**, които се използват от компилатора за създаване на конкретни (шаблонни) класове. Наричат се още **специализации на шаблона на класа**.

Шаблоните на класове не се компилират. Затова шаблоните на класовете заедно с дефинициите на член-функциите (те са шаблони на функции) се слагат в заглавни файлове. При всяко използване на шаблон с различни параметри се **генерира нов шаблонен клас**. Компилират се само член-функциите, които се използват от съответния шаблонен клас. Именно затова може да не разберем, че има грешка в член-функция на шаблон, докато не я използваме.

Ако искаме да дефинираме различна имплементация за шаблона, когато е зададен специфичен параметър, може да декларираме **специализация на шаблона**.

Пример:

```
// template specialization

#include <iostream>

using namespace std;

// class template:

template <class T>

class mycontainer {

     T element;

public:

     mycontainer(T arg) { element = arg; }

     T increase() { return ++element; }

};

// class template specialization:

template <>

class mycontainer <char> {

     char element;

public:

     mycontainer(char arg) { element = arg; }

     char uppercase()

     {

          if ((element >= 'a') && (element <= 'z‘))

               element += 'A' - 'a‘;

     return element;

     }

};

int main() {
```

```
    mycontainer<int> myint(7);

    mycontainer<char> mychar('j');

    cout << myint.increase() << endl;

    cout << mychar.uppercase() << endl;

    return 0;

}
```

При наличие на шаблони на класове възниква необходимостта и от **шаблони на функции**, използващи шаблони на класове. Например, искаме да реализираме сортиране на елементите на шаблон на стек. Налага се да дефинираме шаблони на функциите за намиране на минимален елемент на стек с произволен тип на елементите и сортиране на елементите на стек с произволен тип на елементите.

```
template < class <параметър> {,class <параметър>}>

<тип_на_функция> <име_на_функция>(<формални_параметри>) {<тяло>}
```

Вместо ключовата дума `class` може да се използва `typename`.

Използването на дефинираните шаблони на функции става чрез обръщение към "обобщената" функция, която шаблонът дефинира, с параметри от конкретен тип. Компилаторът генерира т. нар. **шаблонна функция**, като замества параметрите на шаблона с типовете на съответните фактически параметри. При това заместване не се извършват преобразувания на типове.

Пример:

```
template <class T>

T max(T a, T b) {

    return a > b ? a : b;

}

template <class T>

void swap(T& a, T& b) {

    T tmp = a; a = b; b = tmp;

}
```

Държавен изпит – спец. „Софтуерно инженерство“

Тема №15 – Структури от данни. Стек, опашка, списък, дърво. Основни операции върху тях. Реализация.

Анотация: *Изложението по въпроса трябва да включва следните по-съществени елементи:*

1. Структури от данни – дефиниране на понятието.

2. Списък. Логическо описание. Списък с една и две връзки. Характеристики на реализациите с една и две връзки. Сложност на операциите по добавяне, премахване и намиране на елемент. Дефиниране на клас за списък, използващ една от реализациите.

3. Стек. Логическо описание. Характеристики на статичната, динамичната и свързаната реализация. Сложност на операциите по добавяне и премахване на елемент. Дефиниране на клас за стек, използващ една от реализациите.

4. Опашка. Логическо описание. Характеристики на статичната, динамичната и свързаната реализация. Сложност на операциите по добавяне и премахване на елемент. Дефиниране на клас за опашка, използващ една от реализациите.

5. Дървовидни структури от данни – кореново дърво и двоично кореново дърво. Логическо описание. Начини за представяне в паметта. Дефиниране на клас, реализиращ кореново дърво или двоично кореново дърво.

6. Двоично кореново дърво за търсене. Логическо описание. Начини за представяне в паметта. Сложност на операциите по добавяне, премахване и търсене на елемент. Дефиниране на клас реализиращ двоично кореново дърво за търсене.

Забележка: *За изпита ще бъдат избрани две от посочените структури, които да бъдат описани.*

Структури от данни – дефиниране на понятието

Типовете данни (ТД) са инструмент за класификация на данните, характеризиращ се с *множество от стойности* и *операции над стойностите*.

Най-общо в C++ типовете данни (ТД) биват разделени на **две основни групи**:

Примитивни типове данни:

- Булев (`bool`)
- Целочислен (`short`, `int`, `long`, `unsigned`)
- Числа с плаваща запетая (`float`, `double`)
- Символен (`char`)
- Указател (`*`)
- Псевдоним (`&`)

Съставни типове данни:

- Масив (`[]`)
- Структура / Запис (`struct`)
- Клас (`class`)

Структурите от данни (СД) са схеми за организация на даден вид данни в паметта на компютъра, обикновено с цел ефективност. Всеки съставен тип данни в частност може да се разглежда като структура от данни. Структурите от данни често се реализират чрез потребителски дефинирани типове данни (`class`).

Пример: `int[]` *е тип данни масив от елементи от тип* `int` *и може да се разглежда като структура от данни, която представлява редица от цели числа, разположени последователно в паметта.*

Абстрактните типове данни (АТД) са формален модел на типовете данни и структурите от данни. Те включват: *множество от стойности*, *описание на имената и вида на операциите*, *описание на поведението и свойствата на операциите*. Не налагат конкретна организация на паметта (за разлика от *структурите от данни*) и не налагат конкретно представяне със средствата на някакъв език (за разлика от *типовете данни*). Абстрактните типове данни допускат една или повече реализации.

Съществуват **два основни вида описания** на структурите от данни (СД):

Логическо описание (АТД):

- Същност и предназначение
- Компоненти
- Операции
- Свойства на операциите

Физическо описание:

- Организация на паметта
- Представяне с един или повече типове данни
- Реализация на операциите

Видове структури от данни:

- **според вида на компонентите си:**
  - хомогенни (масив)
  - хетерогенни (структура)
- **според способността за промяна на размера:**
  - статични (статичен масив)
  - динамични (разширяващ се масив)
- **според връзките между данните:**
  - линейни (свързан списък)
  - разклонени (дърво)
  - мрежови (граф)

Списък

Списъкът е хомогенна линейна структура с последователен достъп до елементите.

Логическо описание:

Свързаният списък е крайна **редица** от елементи от един и същи тип. Операциите **включване** и **изключване** са допустими в произволно място в редицата. Възможен е пряк достъп до елемента в единия край на редицата, наречен **начало на списъка**, и последователен до всеки от останалите елементи.

Основни операции:

`create()` – създаване на празен списък

`empty()` – проверка за празен списък

`insert(x, p)` – включване на елемент x на дадена позиция p

`delete(p)` – изключване на елемент на дадена позиция p

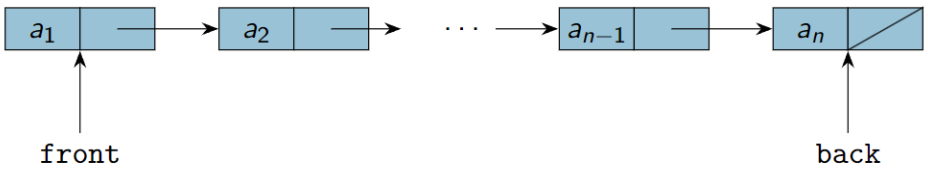
`get(p)` – достъп до елемент на дадена позиция p

Физическо представяне:

Има **два основни начина** за представяне на свързания списък в паметта на компютъра – **свързано представяне с една връзка** и **свързано представяне с две връзки**. Има и последователно представяне (чрез масив от структури), което не се използва заради добре развитите средства за динамично разпределение на паметта.

Свързано представяне с една връзка – едносвързано представяне:

При реализирането на операциите **включване**, **изключване** и **обхождане** се въвеждат указател към края и указател към текущ елемент на списъка.

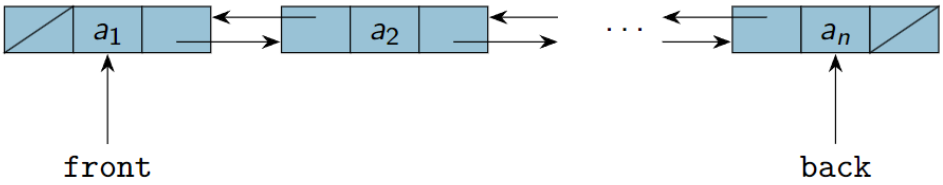



```
struct SinglyLinkedListNode {
     int data;
     SinglyLinkedListNode* next;
};
```

```
class SinglyLinkedList {
     public:
          SinglyLinkedListNode *head;
          SinglyLinkedListNode *tail;
};
```

Свързано представяне с две връзки – двусвързано представяне:

При реализирането на операциите **включване**, **изключване** и **обхождане** се въвеждат тройни кутии с едно информационни и две свързващи полета, съдържащи елемента и адресите на предшестващия и следващия елемент на свързания списък.




```
struct DoublyLinkedListNode {
     int data;
     DoublyLinkedListNode* next;
     DoublyLinkedListNode* prev;
};


class DoublyLinkedList {
     public:
          DoublyLinkedListNode *head;
          DoublyLinkedListNode *tail;
};
```

Свързаният списък е **линейна структура от данни**, която се състои от **върхове** (или наричани още **възли**) и ограничени **връзки между тях**. В **едносвързания списък** всеки връх пази указател към непосредствено следващия го връх, а в **двусвързания** – всеки връх има и указател и към предходния връх. Обикновено, списъкът пази указател освен към **първия връх** (наричан още **глава**) и към **последния възел** (наричан още **опашка**).

По този начин ще може да осигурим **константна сложност** $O(\mathbf{1})$ за **добавяне** и **изтриване** на **върхове от главата** и **добавяне към опашката** на списъка. Ако е **двусвързан списък**, то с константна сложност ще е и **изтриването в края** на списъка.

Всички останали операции като **търсене**, **вмъкване** или **изтриване на позиция** имат **линейна сложност** по броя на върховете в списъка $O(\boldsymbol{n})$ (тъй като в най-лошия случай ще трябва да се обходи целия списък).

В случай, че имаме **указател към даден връх от списъка**, ще имаме възможност да **добавим** или **изтрием** с **константна сложност** следващия елемент след този, към който указателят, който имаме, сочи или следващия, текущия и предходния при двусвързан списък. **Конкатениране** с оригинал на **друг списък** също е с **константна сложност**.

Реализация на клас за списък с едносвързано представяне:

```
 1. #include <iostream>
 2. #include <stdexcept>
 3. using namespace std;
 4.
 5. struct Node {
 6.     int data;
 7.     Node* next = nullptr;
 8.
 9.     Node(int data, Node* next = nullptr) {
10.         this->data = data;
11.         this->next = next;
12.     }
13. };
14.
15. class SinglyLinkedList {
16. private:
17.     Node* head = nullptr;
18.     Node* tail = nullptr;
19. public:
20.     SinglyLinkedList() = default;
21.
22.     void freeMemory() {
23.         Node* i = head;
24.         while (i != nullptr) {
25.             Node* next = i->next;
26.             delete i;
27.             i = next;
28.         }
29.
30.         head = nullptr;
31.         tail = nullptr;
32.     }
33.
34.     ~SinglyLinkedList() {
35.         freeMemory();
36.     }
37.
38.     void copyLinkedList(const SinglyLinkedList& rhs) {
39.         Node* i = rhs.head;
40.         while (i != nullptr) {
41.             push_back(i->data);
42.             i = i->next;
43.         }
44.     }
45.
46.     SinglyLinkedList(const SinglyLinkedList& rhs) {
47.         copyLinkedList(rhs);
48.     }
```

```
49.
50.     SinglyLinkedList& operator=(const SinglyLinkedList& rhs) {
51.         if (this != &rhs) {
52.             freeMemory();
53.             copyLinkedList(rhs);
54.         }
55.         return *this;
56.     }
57.
58.     void push_front(int number) {
59.         Node* newHead = new Node(number, head);
60.         head = newHead;
61.         if (tail == nullptr) {
62.             tail = head;
63.         }
64.     }
65.
66.     void push_back(int number) {
67.         if (head == nullptr) {
68.             head = new Node(number);
69.             tail = head;
70.         }
71.         else {
72.             tail->next = new Node(number);
73.             tail = tail->next;
74.         }
75.     }
76.
77.     void insert(int number, int position) {
78.         if (position == 0) {
79.             push_front(number);
80.             return;
81.         }
82.
83.         if (head == nullptr) {
84.             throw std::out_of_range("Invalid insert index, list is empty");
85.         }
86.
87.         Node* current = head;
88.         Node* previous = nullptr;
89.         for (int i = 0; i < position; i++) {
90.             if (current != nullptr) {
91.                 previous = current;
92.                 current = current->next;
93.             }
94.             else {
95.                 throw std::out_of_range("Cannot insert in invalid position");
96.             }
97.         }
98.
99.         previous->next = new Node(number, current);
100.        if (tail == previous) {
101.            tail = previous->next;
102.        }
103.    }
104.
105.    void pop_front() {
106.        if (head != nullptr) {
107.            Node* oldHead = head;
108.            if (tail == head) {
109.                tail = nullptr;
110.            }
111.            head = head->next;
112.            delete oldHead;
113.        }
114.        else {
115.            throw std::out_of_range("List is empty, cannot pop_front()");
116.        }
```

```
117.     }
118.
119.     void pop_back() {
120.         if (head != nullptr) {
121.             if (head == tail) {
122.                 delete head;
123.                 head = nullptr;
124.                 tail = nullptr;
125.             }
126.
127.             // Find the node pointing to the tail
128.             Node* i = head;
129.             while (i->next != nullptr && i->next != tail) {
130.                 i = i->next;
131.             }
132.
133.             // Delete the tail and point tail to the current node (i)
134.             delete i->next;
135.             i->next = nullptr;
136.             tail = i;
137.         }
138.         else {
139.             throw std::out_of_range("List is empty, cannot pop_front()");
140.         }
141.     }
142.
143.     void remove(int position) {
144.         if (position == 0) {
145.             pop_front();
146.             return;
147.         }
148.
149.         if (head == nullptr) {
150.             throw std::out_of_range("Invalid remove index, list is empty");
151.         }
152.
153.         Node* current = head;
154.         Node* previous = nullptr;
155.         for (int i = 0; i < position; i++) {
156.             if (current->next != nullptr) {
157.                 previous = current;
158.                 current = current->next;
159.             }
160.             else {
161.                 throw std::out_of_range("Cannot insert in invalid position");
162.             }
163.         }
164.
165.         previous->next = current->next;
166.         if (tail == current) {
167.             tail = previous;
168.         }
169.         delete current;
170.     }
171.
172.     int peek_front() {
173.         if (head != nullptr) {
174.             return head->data;
175.         }
176.         else {
177.             throw std::out_of_range("List is empty, cannot peek_front()");
178.         }
179.     }
180.
181.     int peek_back() {
182.         if (tail != nullptr) {
183.             return tail->data;
184.         }
```

```
185.         else {
186.             throw std::out_of_range("List is empty, cannot peek_back()");
187.         }
188.     }
189.
190.     int get(int position) {
191.         if (head == nullptr) {
192.             throw std::out_of_range("List is empty, cannot get from it");
193.         }
194.
195.         Node* current = head;
196.         for (int i = 0; i < position; i++) {
197.             if (current->next != nullptr) {
198.                 current = current->next;
199.             }
200.             else {
201.                 throw std::out_of_range("Cannot get from list, no such position");
202.             }
203.         }
204.
205.         return current->data;
206.     }
207.
208.     void print() const {
209.         Node* i = head;
210.         while (i != nullptr) {
211.             cout << i->data << " ";
212.             i = i->next;
213.         }
214.         cout << "\n";
215.     }
216. };
217.
218. int main() {
219.     SinglyLinkedList l;
220.     l.insert(0, 0);
221.     l.insert(1, 1);
222.     l.insert(2, 2);
223.     l.insert(3, 3);
224.     l.print();
225. }
```

Стек – LIFO

Стекът е хомогенна линейна структура с организация „последен влязъл – пръв излязъл“ (LIFO).

Основни операции:

`create()` – създаване на празен стек

`empty()` – проверка за празнота на стек

`push(x)` – включване на елемент на стек

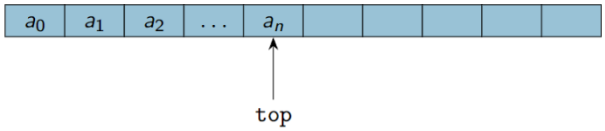
`pop()` – изключване на елемент от стек

`peek()` – връщане на последен елемент на стек

Стекът е **линеен списък**, в който всички операции (включване, изключване и т.н.) се извършват **само в единия му край**, т.е. само върху първия или само върху последния елемент. Прието е елементът, върху който се извършват операциите, да се нарича **връх на стека**.

Думата *стек* се превежда на български като *купчина*. Аналогията е очевидна – ако разгледаме купчина от вестници в кашон, лесно можем да вземем вестника от върха на купчината или да добавим нов най-отгоре. Казва се още, че стекът се подчинява на дисциплината **LIFO** (Last-In-First-Out).

Последователна (статична) реализация:



Структурата стек се реализира **статично** чрез **масив**. Въвежда се единствен индекс, който сочи адреса на върха на стека. Така при операцията включване елементът ще се записва на адреса, сочен от индекса, след което индексът ще се увеличава с единица. Съответно, при изключване първо ще се извлича елементът, след което индексът ще се намалява с единица. При това представяне са необходими проверки за броя на елементите в масива, за да не се получи достъп до памет извън масива.

При тази реализация с фиксиран масив **сложността по време** на операциите за добавяне на елемент (`push(x)`) и за премахване на елемент (`pop()`) е **константна** – $O(1)$.

Свързана (динамична) реализация:

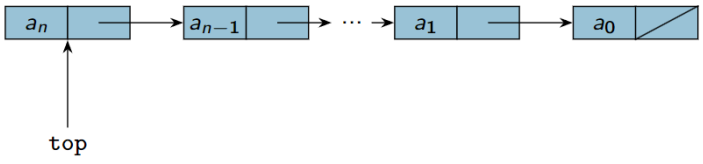
Характеристики при реализиране на **стек с динамичен масив** (масив с променлива дължина):

- заделяме първоначално масив с фиксирана дължина
- ако масивът се запълни – заделяме нов масив с по-голяма дължина и копираме стария в него
- ако стекът остане с по-малко от определен брой елементи, създаваме нов с по-малка големина и копираме от стария
- ефективното решение е когато масивът е пълен, да удвояваме по размер, а когато масивът е пълен на ¼ да намаляваме на половина

При тази реализация с **динамичен масив** (напр. `std::vector`) сложността по време на операциите за добавяне на елемент (`push(x)`) и за премахване на елемент (`pop()`) е **амортизирана константна** – $O(1^*)$, защото има случаи, когато е необходимо **копиране** на елементи и съответно **последователно обхождане**.

Характеристики при реализиране на **стек със свързан списък**:

- представяме стека като „верига“ от **двойни кутийки** – по аналогия със структурата свързан списък



```
struct StackElement {

      int data;

      StackElement* next;

};
```

- операцията за **включване на елемент** на върха (`push(x)`) се реализира по аналогия на операцията `push_front(x)` за свързан списък
- операцията за **изключване на елемент** от върха (`pop()`) се реализира по аналогия на операцията `delete_front()` за свързан списък

При тази реализация със свързан списък (напр. `std::list`) **сложността по време на операциите** за добавяне на елемент (`push(x)`) и за премахване на елемент (`pop()`) е **гарантирана константна** – $O(1)$.

Реализация на клас за стек със свързано представяне

Забележка: За реализацията може да се използва и структурата за свързан списък в `STL` `std::list` и да се добави необходимия `include`.

```
 1. #include <iostream>
 2. #include <stdexcept>
 3.
 4. using namespace std;
 5.
 6. struct Node {
 7.     int data;
 8.     Node* next = nullptr;
 9.
10.     Node(int data, Node* next = nullptr) {
11.         this->data = data;
12.         this->next = next;
13.     }
14. };
15.
16. class Stack {
17. private:
18.     Node* top = nullptr;
19. public:
20.     Stack() { }
21.
22.     void copyStackRecursive(Node* current) {
23.         if (current != nullptr) {
24.             copyStackRecursive(current->next);
25.             push(current->data);
26.         }
27.     }
28.
29.     void copyStack(const Stack& rhs) {
30.         copyStackRecursive(rhs.top);
31.     }
32.
33.     void freeMemory() {
34.         while (top != nullptr) {
35.             Node* next = top->next;
36.             delete top;
37.             top = next;
38.         }
```

```
39.     }
40.
41.     Stack(const Stack& rhs) {
42.         copyStack(rhs);
43.     }
44.
45.     Stack& operator=(const Stack& rhs) {
46.         if (this != &rhs) {
47.             freeMemory();
48.             copyStack(rhs);
49.         }
50.         return *this;
51.     }
52.
53.     ~Stack() {
54.         freeMemory();
55.     }
56.
57.     void push(int number) {
58.         if (isEmpty()) {
59.             top = new Node(number);
60.         }
61.         else {
62.             Node* newTop = new Node(number, top);
63.             top = newTop;
64.         }
65.     }
66.
67.     void pop() {
68.         if (!isEmpty()) {
69.             Node* oldTop = top;
70.             top = top->next;
71.             delete oldTop;
72.         }
73.         else {
74.             throw std::underflow_error("Stack is empty");
75.         }
76.     }
77.
78.     int peek() const {
79.         if (!isEmpty()) {
80.             return top->data;
81.         }
82.         else {
83.             throw std::underflow_error("Stack is empty");
84.         }
85.     }
86.
87.     bool isEmpty() const {
88.         return top == nullptr;
89.     }
90. };
91.
92. int main() {
93.     Stack s;
94.     s.push(1);
95.     s.push(2);
96.     s.push(3);
97.
98.     while (!s.isEmpty()) {
99.         cout << s.peek();
100.        s.pop();
101.    }
102. }
```

Опашка – FIFO

Опашката е хомогенна линейна структура с организация „пръв влязъл – пръв излязъл“ (FIFO).

Логическо описание:

Опашката е крайна **редица** от елементи от един и същи тип. Операцията **включване** е допустима за елементите от единия край на редицата, който се нарича **край на опашката**, а операцията **изключване** на елемент – само за елементите от другия край на редицата, който се нарича **начало на опашката**. Възможен е достъп само до елемента, намиращ се в началото на опашката, като достъпът до него е пряк. При тази организация на логическите операции, първият включен елемент се изключва първи. Затова опашката се определя още като структура от данни „**пръв влязъл – пръв излязъл**“.

Основни операции:

`create()` – създаване на празна опашка

`empty()` – проверка за празнота на опашката

`enqueue(x)` – включване на елемент в края на опашката

`dequeue()` – изключване на елемент от началото на опашката

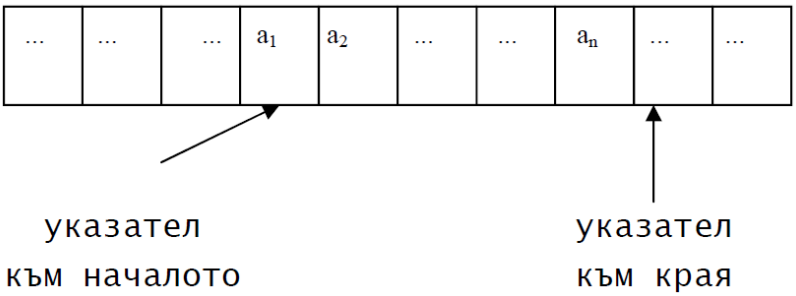
`head()` – достъп до първия елемент

Физическо представяне:

*Използват се **два основни начина** за физическо представяне – **последователно** и **свързано**.*

Последователно (статично) представяне:

При това представяне се запазва **блок от паметта**, вътре в който опашката да расте и да се съкращава. Ако редицата от елементи от един и същи тип a_1, a_2, …, a_n е опашка с начало a_1, последователното представяне има вида:



Включване на елемент в опашката се осъществява чрез поместването му в последователни адреси в неизползваната част веднага след края на опашката. При изчерпване на масива, ако има освободена памет в началото му, включването се извършва там (цикличен масив).

При тази реализация **сложността на операциите** по **включване** и **изключване** на елемент е **константа** – $O(1)$. Важно е да се отбележи обаче, че размерът на заделената памет е **фиксиран** в началото, което **ограничава** размера на опашката.

Свързано (динамично) представяне:

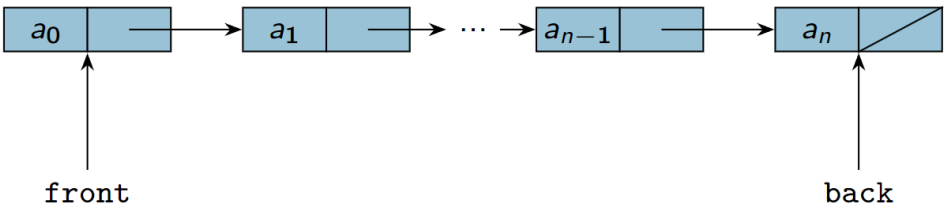
Характеристики при реализиране на **опашка с динамичен масив** (масив с променлива дължина):

- заделяме първоначално масив с фиксирана дължина
- ако масивът се запълни – заделяме нов масив с по-голяма дължина и копираме стария в него
- ако масивът остане запълнен на по-малко от ¼, то създаваме нов наполовина по-малък и копираме в него

При тази реализация с **динамичен масив** (напр. `std::vector`) **сложността по време** на операциите за **добавяне на елемент** (`enqueue(x)`) и за **премахване на елемент** (`dequeue()`) е **амортизирана константна** – $O(1^*)$, защото има случаи, когато е необходимо **копиране** на елементи и съответно **последователно обхождане**.

Характеристики при реализиране на **опашка със свързан списък**:

- представяме опашката като „верига“ от **двойни кутийки** – по аналогия със структурата свързан списък




```
struct QueueElement {

      int data;

      QueueElement* next;

};
```

- операцията за **включване на елемент** към опашката (`enqueue(x)`) се реализира по аналогия на операцията `push_back(x)` за свързан списък
- операцията за **изключване на елемент** от опашката (`dequeue()`) се реализира по аналогия на операцията `delete_front()` за свързан списък

При тази реализация със свързан списък (напр. `std::list`) **сложността по време** на операциите за **добавяне на елемент** (`enqueue(x)`) и за **премахване на елемент** (`dequeue()`) е **гарантирана константна** – $O(1)$.

Реализация на клас за опашка със свързано представяне

Забележка: За реализацията може да се използва и структурата за свързан списък в STL `std::list` и да се добави необходимия `include`.

```
 1. #include <iostream>
 2. #include <stdexcept>
 3.
 4. using namespace std;
 5.
 6. struct Node {
 7.     int data;
 8.     Node* next = nullptr;
 9.
10.     Node(int data, Node* previous = nullptr) {
11.         this->data = data;
12.         this->next = previous;
13.     }
14. };
15.
16. class Queue {
17. private:
18.     Node* front = nullptr;
19.     Node* back = nullptr;
20. public:
21.     Queue() = default;
22.
23.     void copyQueue(const Queue& rhs) {
24.         Node* i = rhs.front;
25.         while (i != nullptr) {
26.             push(i->data);
27.             i = i->next;
28.         }
29.     }
30.
31.     void freeMemory() {
32.         while (front != nullptr) {
33.             Node* next = front->next;
34.             delete front;
35.             front = next;
36.         }
37.     }
38.
39.     Queue(const Queue& rhs) {
40.         copyQueue(rhs);
41.     }
42.
43.     Queue& operator=(const Queue& rhs) {
44.         if (this != &rhs) {
45.             freeMemory();
46.             copyQueue(rhs);
47.         }
48.         return *this;
49.     }
50.
51.     ~Queue() {
52.          freeMemory();
53.     }
54.
55.     void push(int number) {
56.         if (isEmpty()) {
57.             front = new Node(number);
58.             back = front;
59.         }
60.         else {
61.             back->next = new Node(number);
```

```
62.             back = back->next;
63.         }
64.     }
65.
66.     void pop() {
67.         if (!isEmpty()) {
68.             Node* oldFront = front;
69.             front = front->next;
70.             delete oldFront;
71.         }
72.         else {
73.             throw std::underflow_error("Queue is empty");
74.         }
75.     }
76.
77.     int peek() const {
78.         if (!isEmpty()) {
79.             return front->data;
80.         }
81.         else {
82.             throw std::underflow_error("Queue is empty");
83.         }
84.     }
85.
86.     bool isEmpty() const {
87.         return front == nullptr;
88.     }
89. };
90.
91. int main() {
92.     Queue s;
93.     s.push(1);
94.     s.push(2);
95.     s.push(3);
96.
97.     while (!s.isEmpty()) {
98.         cout << s.peek();
99.         s.pop();
100.    }
101. }
```

Дървовидни структури от данни – кореново дърво и двоично кореново дърво

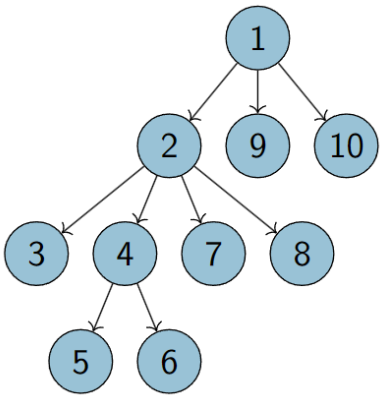
Кореновото дърво е **йерархична структура**, в която на всеки елемент е съпоставено множество от подчинени елементи.

Логическо описание:

Кореново дърво е **списък** $(X, T_1, \dots, T_n)$, където:

- X е данна (корен)
- $T_1, \dots, T_n$ са коренови дървета (поддървета), $n \geq 0$

Често отъждествяваме кореновите дървета с корените им (възли).



Основни операции:

`create(x)` – създаване на дърво с корен `x`

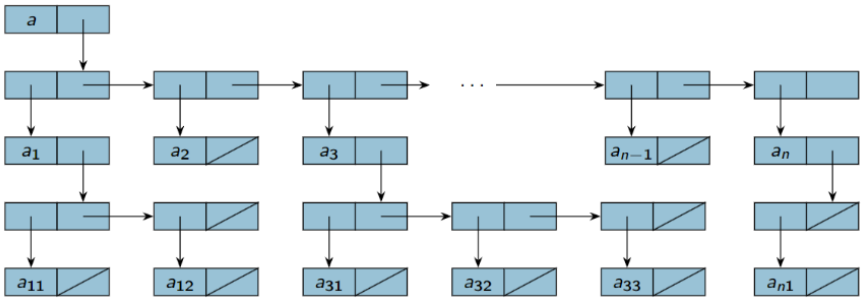
`addSubtree(t)` – добавяне на поддърво `t`

`root()` – достъп до корена

`subtrees()` – достъп до поддърветата

Съществуват **два вида представяния** в паметта на кореновото дърво:

- **Свързано представяне** – свързана структура от възли, където всеки възел се състои от стойността на корена и свързан списък от възли, представящи поддърветa (деца)




```
struct Node {

    int data;

    vector<Node*> children;

};
```

- **Последователно представяне** – последователност от тройки (корен, най-ляво дете, десен брат), представени в три масива

	0	1	2	3	4	5	6	7	8	...
корен	a	a_1	$a_{1,1}$	a_2	$a_{1,2}$	a_3	$a_{3,1}$	$a_{3,2}$	$a_{3,3}$	...
най-ляво дете	1	2	-1	-1	-1	6	-1	-1	-1	...
десен брат	-1	3	4	5	-1	9	7	8	-1	...

Други дефиниции:

- **дете** – коренов възел на поддърво
- **родител** – възел, сред чиито деца е коренът
- **листо** – възел без деца
- **път** – редица от възли, в която всеки следващ е дете на предходния
- **разстояние** – дължина на пътя от корена до възел
- **ниво** – множество от възлите на еднакво разстояние
- **височина** (дълбочина) – максималното разстояние в дървото
- **широчина** (разклоненост) – максималният брой деца на възел

Схеми за обхождане:

- **Префиксно обхождане** – първо посещаваме корена и след това обхождаме децата

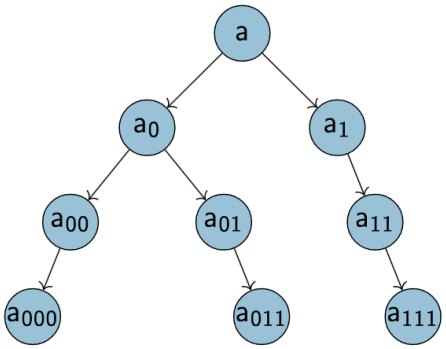
***Пример**: За примера на кореново дърво от по-горе: 1, 2, 3, 4, 5, 6, 7, 8, 9, 10*

- **Постфиксно обхождане** – първо обхождаме децата и накрая посещаваме корена

***Пример:** За примера на кореново дърво от по-горе: 3, 5, 6, 4, 7, 8, 2, 9, 10, 1*

Двоично дърво

Логическо описание:

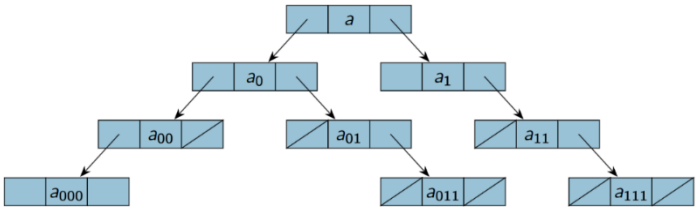


Двоично дърво е структура от данни, която е или **празна** (празното дърво), или е **наредената тройка** (X, L, R), където:

- X е данна (корен)
- L е ляво поддърво
- R е дясно поддърво

Съществуват **два вида представяния** в паметта на двоичното кореново дърво:

- **Свързано представяне** – реализира се чрез указател към кутия с три полета: информационно, съдържащо **стойността на корена** и две адресни, съдържащи представянията на **лявото поддърво** и **дясното поддърво** съответно.




```
struct Node {

      int data;

      Node* left;

      Node* right;

};
```

- **Последователно (верижно) представяне** – при това представяне се използват **три масива** – за елементите на дървото, за адресите на лявото и за адресите на дясното поддърво. Ролята на адреси се изпълнява от индекси. i-тият елемент на масива „корен“ съдържа стойността на връх на двоичното дърво, i-тият елемент на масива „ляво дете“ съдържа адреса на лявото поддърво на поддървото с корен i-тия елемент, а i-тият

елемент на масива „дясно дете“ – адреса на дясното му поддърво. Има също така и указател, който съдържа адреса на корена.

	0	1	2	3	4	5	6	7	8	...
корен	a	a_0	a_1	a_{00}	a_{01}	a_{11}	a_{000}	a_{011}	a_{111}	...
ляво дете	1	3	-1	6	-1	-1	-1	-1	-1	...
дясно дете	2	4	5	-1	7	8	-1	-1	-1	...

Основни операции:

- **Достъп до връх** – възможен е пряк достъп до корена и непряк достъп до всеки от останалите върхове на двоичното дърво
- **Включване и изключване на връх** – включването и изключването са възможни в произволно място на двоичното дърво. Резултатът е отново двоично дърво

Обхождане на двоично дърво:

Обхождането на двоично дърво дава метод, позволяващ да се осъществи достъп до всеки връх на дървото по един единствен път. Осъществява се чрез изпълнение на следните три действия в някаква последователност – **обработка на корена**, **обхождане на лявото поддърво**, **обхождане на дясното поддърво**, т.е. процесът на обхождане е рекурсивен.

Съществуват **шест различни начина** за обхождане на двоично дърво:

- **Префиксни** – низходящо обхождане
  - КЛД
  - КДЛ
- **Инфиксни** - смесено обхождане
  - ЛКД
  - ДКЛ
- **Постфиксни** – възходящо обхождане
  - ЛДК
  - ДЛК

Горните видове обхождания са варианти на т.нар. **търсене в дълбочина** (*depth-first search – DFS*). Съществува и друг метод за обхождане на дървета наречен **търсене в широчина** (*breadth-first search – BFS*). При него обхождането се извършва по **нива**. За целта се използва **опашка**, в която се съхраняват върховете последователно от корена, последователно по нивата.

Клас, реализиращ двоично кореново дърво

```
 1. #include<iostream>
 2.
 3. using namespace std;
 4.
 5. struct Node {
 6.     int root;
 7.     Node* left;
 8.     Node* right;
 9.
10.     Node(int root, Node* left = nullptr, Node* right = nullptr) {
11.             this->root = root;
12.             this->left = left;
13.             this->right = right;
14.     }
```

```
15. };
16.
17. class Tree {
18. private:
19.     Node* root;
20.
21.     void deleteTree(Node* root) {
22.               if (root) {
23.                         deleteTree(root->left);
24.                         deleteTree(root->right);
25.                         delete root;
26.                         root = nullptr;
27.               }
28.     }
29.
30.     void copyTree(const Tree & t) {
31.               copy(root, t.root);
32.     }
33.
34.     void copy(Node* pos, Node* r) {
35.               pos = nullptr;
36.               if (r) {
37.                         pos = new Node(r->root);
38.                         copy(pos->left, r->left);
39.                         copy(pos->right, r->right);
40.               }
41.     }
42.
43.     void printInorder(const Node* p) const {
44.               if (p) {
45.                         printInorder(p->left);
46.                         cout << p->root << " ";
47.                         printInorder(p->right);
48.               }
49.     }
50.
51.     void createTree(Node*& pos) {
52.               int x;
53.               char c;
54.               cin >> x;
55.
56.               pos = new Node(x);
57.
58.               cout << "Left Tree of: " << x << " y/n? ";
59.
60.               cin >> c;
61.               if (c == 'y') createTree(pos->left);
62.
63.               cout << "Right Tree of: " << x << " y/n? ";
64.
65.               cin >> c;
66.               if (c == 'y') createTree(pos->right);
67.     }
68.
69. public:
70.     Tree() {
71.               root = nullptr;
72.     }
73.
74.     ~Tree() {
75.               deleteTree(root);
76.     }
77.
78.     Tree(const Tree & t) {
79.               copyTree(t);
80.     }
81.
```

```
82.     Tree& operator=(const Tree & t) {
83.             if (this != &t) {
84.                     deleteTree(root);
85.                     copyTree(t);
86.             }
87.
88.             return *this;
89.     }
90.
91.     bool empty() const {
92.             return root == nullptr;
93.     }
94.
95.     int rootTree() const {
96.             return root->root;
97.     }
98.
99.     void print() const {
100.            printInorder(root);
101.            cout << endl;
102.    }
103.
104.    void create() {
105.            deleteTree(root);
106.            createTree(root);
107.    }
108. };
109.
110. int main() {
111.    Tree t;
112.    t.create();
113.    t.print();
114. }
```

Двоично кореново дърво за търсене

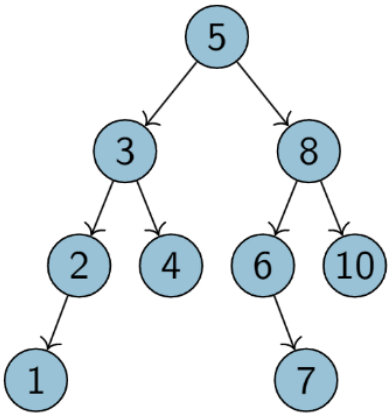
Логическо описание:

Празното двоично дърво е двоично кореново дърво за търсене. Непразно двоично дърво, върховете на лявото поддърво на което са **по-малки от корена**, а върховете на дясното поддърво са **по-големи от корена** и лявото и дясното поддърво са **двоични коренови дървета за търсене**, се нарича двоично кореново дърво за търсене.

За всеки връх е изпълнено следното:

- Или няма ляво дете, или то, а и всички негови деца, са по-малки от текущия връх;
- Или няма дясно дете, или то, а и всички негови деца, са по-големи от текущия връх;
- Всеки връх има най-много две деца.

Пример:



Създаденото по този начин двоично наредено дърво притежава следното **свойство** – обхождането на такова дърво по **метода ЛКД сортира във възходящ ред** елементите от върховете на дървото, а обхождането му по **метода ДКЛ сортира в низходящ ред** елементите от върховете на дървото.

Основни операции:

`search(x)` – търси елемент в дървото

`insert(x)` – добавя елемент в дървото

`remove(x)` – изтрива елемент от дървото

`print()` – принтира дървото в специфичен ред

При **добавяне на елемент** се сравнява с всеки елемент на дървото и му се намира правилното място винаги като листо. При **изтриване на елемент** алгоритъмът е следният:

- Намираме възела за изтриване
- Ако е листо (т.е. няма деца) – изтриваме
- Ако има само един наследник, поставяме наследника на негово място
- Ако има двама наследника (т.е. две деца), намираме следващото по големина число в дървото, разменяме го с елемента за изтриване и премахваме елемента

Сложността по време на операциите по **добавяне**, **премахване** и **търсене на елемент** е **логаритмична –** $O(\log n)$ в **средния случай**. В **най-лошия случай** тя става **линейна** – $O(n)$. Това е така, защото в тази ситуация двоичното дърво за търсене е придобило формата на **свързан списък**, където обхождането винаги е **последователно**.

Начини за представяне в паметта:

- **Последователно** – елементите се съхраняват в **масив** и връзката между елементите зависи от техните индекси в масива. Кореновият елемент е разположен на индекс 0 и за всеки елемент на индекс i, неговото ляво дете се намира на индекс $2i + 1$, а неговото дясно дете – на индекс $2i + 2$. Недостатък на това представяне е, че в масива остават празни клетки поради липса на съответно дете в дървото, които трябва да се индикират по някакъв начин. Също така при добавяне и изтриване на елементи може да се наложи копиране и преместване на елементи, което не е ефективно.

- **Свързано** – реализира се чрез указател към кутия с три полета: информационно, съдържащо стойността на корена, и две адресни, съдържащи представянията на лявото поддърво (ЛПД) и дясното поддърво (ДПД) съответно.

```
struct Node {

      int data;

      Node* left;

      Node* right;

};
```

Клас, реализиращ двоично кореново дърво за търсене

```
 1. #include <iostream>
 2. #include <queue>
 3.
 4. using namespace std;
 5.
 6. struct Node {
 7.     int key;
 8.     Node* left;
 9.     Node* right;
10.
11.     Node(int key, Node* left = nullptr, Node* right = nullptr) {
12.         this->key = key;
13.         this->left = left;
14.         this->right = right;
15.     }
16. };
17.
18. class BST {
19. private:
20.     Node* root;
21.
22.     Node* insert(int value, Node* current) {
23.         if (current == nullptr) {
24.             return new Node(value);
25.         }
26.
27.         if (value < current->key) {
28.             current->left = insert(value, current->left);
29.         }
30.         else if (value > current->key) {
31.             current->right = insert(value, current->right);
32.         }
33.
34.         return current;
35.     }
36.
37.     void inorder(Node* current) const {
38.         if (current != nullptr) {
39.             inorder(current->left);
40.             cout << current->key << " ";
41.             inorder(current->right);
42.         }
43.     }
44.
45.     Node* minValueNode(Node* current) {
46.         Node* temp = current;
47.         while (temp != nullptr && temp->left != nullptr) {
```

```
48.             temp = temp->left;
49.         }
50.         return temp;
51.     }
52.
53.     Node* deleteNode(Node* current, int value) {
54.         if (current == nullptr) {
55.             return current;
56.         }
57.
58.         if (value < current->key) {
59.             current->left = deleteNode(current->left, value);
60.         }
61.         else if (value > current->key) {
62.             current->right = deleteNode(current->right, value);
63.         }
64.         else {
65.             if (current->left == nullptr) {
66.                 Node* temp = current->right;
67.                 delete current;
68.                 return temp;
69.             }
70.             else if (current->right == nullptr) {
71.                 Node* temp = current->left;
72.                 delete current;
73.                 return temp;
74.             }
75.             else {
76.                 Node* temp = minValueNode(current->right);
77.                 current->key = temp->key;
78.                 current->right = deleteNode(current->right, temp->key);
79.             }
80.         }
81.
82.         return current;
83.     }
84.
85.     Node* copyTree(Node* current) {
86.         if (current == nullptr) {
87.             return nullptr;
88.         }
89.
90.         Node* newNode = new Node(current->key);
91.         newNode->left = copyTree(current->left);
92.         newNode->right = copyTree(current->right);
93.         return newNode;
94.     }
95.
96.     void clearTree(Node* current) {
97.         if (current != nullptr) {
98.             clearTree(current->left);
99.             clearTree(current->right);
100.            delete current;
101.        }
102.    }
103. public:
104.    BST() : root(nullptr) {}
105.
106.    BST(const BST& other) {
107.        root = copyTree(other.root);
108.    }
109.
110.    BST& operator=(const BST& other) {
111.        if (this != &other) {
112.            clearTree(root);
113.            root = copyTree(other.root);
114.        }
115.
```

```
116.         return *this;
117.     }
118.
119.     ~BST() {
120.         clearTree(root);
121.     }
122.
123.     void insert(int value) {
124.         root = insert(value, root);
125.     }
126.
127.     void print() const {
128.         inorder(root);
129.         cout << endl;
130.     }
131.
132.     void remove(int value) {
133.         root = deleteNode(root, value);
134.     }
135.
136.     void breadthFirstSearch() const {
137.         if (root == nullptr) return;
138.
139.         queue<Node*> q;
140.         q.push(root);
141.
142.         while (!q.empty()) {
143.             Node* current = q.front();
144.             q.pop();
145.             cout << current->key << " ";
146.             if (current->left != nullptr) q.push(current->left);
147.             if (current->right != nullptr) q.push(current->right);
148.         }
149.         cout << endl;
150.     }
151. };
152.
153. int main() {
154.     BST bst;
155.
156.     bst.insert(1);
157.     bst.insert(2);
158.     bst.insert(3);
159.     bst.insert(0);
160.     bst.insert(-6);
161.     bst.insert(6);
162.
163.     cout << "Inorder traversal: ";
164.     bst.print();
165.
166.     cout << "Breadth-first search: ";
167.     bst.breadthFirstSearch();
168.
169.     return 0;
170. }
```

Реализация на клас за стек със статично представяне

```
 1. #include <iostream>
 2.
 3. using namespace std;
 4.
 5. int const MAX = 100;
 6.
 7. class Stack {
 8. private:
 9.       int arr[MAX] = { 0 };
10.       int top;
11.
12.       bool full() const {
13.                 return top == MAX - 1;
14.       }
15.
16. public:
17.       Stack() {
18.                 top = -1;
19.       }
20.
21.       bool empty() const {
22.                 return top == -1;
23.       }
24.
25.       int peek() const {
26.                 if (empty()) {
27.                           cout << "Stack is empty: cannot peek()\n";
28.                           return 0;
29.                 }
30.
31.                 return arr[top];
32.       }
33.
34.       void push(int x) {
35.                 if (full()) {
36.                           cout << "Stack is full: cannot push(x)\n";
37.                 }
38.                 else {
39.                           arr[++top] = x;
40.                 }
41.       }
42.
43.       int pop() {
44.                 if (empty()) {
45.                           cout << "Stack is empty: cannot pop()\n";
46.                           return 0;
47.                 }
48.                 return arr[top--];
49.       }
50.
51. };
52.
53. int main() {
54.       Stack s;
55.
56.       s.push(1);
57.       s.push(2);
58.       s.push(3);
59.
60.       while (!s.empty()) {
61.                 cout << s.peek() << "\n";
62.                 s.pop();
63.       }
64. }
```

Реализация на клас за стек с динамично представяне

```
#include <iostream>

using namespace std;

class DynamicStack {
private:
    int * arr;
    int capacity;
    int size;

    void resize(int newCapacity) {
        int * temp = new int[size];
        for (int i = 0; i < size; i++) {
            temp[i] = arr[i];
        }

        delete[] arr;
        arr = new int[newCapacity];

        for (int i = 0; i < size; i++) {
            arr[i] = temp[i];
        }

        delete[] temp;
    }

public:
    DynamicStack() {
        arr = new int[16];
        size = 0;
        capacity = 16;
    }

    void push(int value) {
        if (size >= (2 * capacity) / 3) {
            resize(capacity * 2);
        }

        arr[size] = value;
        size++;
    }

    void pop() {
        if (empty()) {
            return;
        }

        size--;

        if (size <= capacity / 3) {
            resize(min(capacity * 2, 16));
        }
    }

    int top() const {
        return arr[size - 1];
    }

    int peek() const {
        if (empty()) {
            cout << "Stack is empty: cannot peek()\n";
        }
        else {
            return arr[size - 1];
        }
```

```
66.     }
67. 
68.     bool empty() const {
69.         return size == 0;
70.     }
71. };
72. 
73. int main() {
74.     DynamicStack s;
75. 
76.     s.push(1);
77.     s.push(2);
78.     s.push(3);
79. 
80.     while (!s.empty()) {
81.         cout << s.peek();
82.         s.pop();
83.     }
84. 
85.     return 0;
86. }
```

Реализация на клас за опашка със статично представяне

```
 1. #include <iostream>
 2.
 3. using namespace std;
 4.
 5. int const MAX_SIZE = 100;
 6.
 7. class StaticQueue {
 8. private:
 9.     int front;
10.     int rear;
11.     int arr[MAX_SIZE] = { 0 };
12.
13. public:
14.     StaticQueue() : front(-1), rear(-1) {}
15.
16.     bool isEmpty() const {
17.         return front == -1 || front > rear;
18.     }
19.
20.     bool isFull() const {
21.         return rear == MAX_SIZE - 1;
22.     }
23.
24.     int peek() {
25.         if (isEmpty()) {
26.             cout << "Queue is empty" << endl;
27.             return -1;
28.         }
29.
30.         return arr[front];
31.     }
32.
33.     void enqueue(int x) {
34.         if (isFull()) {
35.             cout << "Queue is full" << endl;
36.             return;
37.         }
38.
39.         if (isEmpty()) {
40.             front = 0;
41.         }
42.
43.         rear++;
44.         arr[rear] = x;
45.     }
46.
47.     int dequeue() {
48.
49.         if (isEmpty()) {
50.             cout << "Queue is empty" << endl;
51.             return -1;
52.         }
53.
54.         int ans = arr[front];
55.         front++;
56.
57.         if (isEmpty()) {
58.             front = rear = -1;
59.         }
60.
61.         return ans;
62.     }
63.
64.     void print() const {
65.         if (isEmpty()) {
```

```
            cout << "Queue is empty" << endl;
            return;
        }
        cout << "Queue: ";
        for (int i = front; i <= rear; i++) {
            cout << arr[i] << " ";
        }

        cout << endl;
    }
};

int main()
{
    StaticQueue q;

    q.enqueue(1);
    q.enqueue(2);
    q.enqueue(3);

    q.print();

    return 0;
}
```

Реализация на клас за опашка с динамично представяне

```
 1. #include <iostream>
 2.
 3. using namespace std;
 4.
 5. class DynamicQueue {
 6. private:
 7.     int* arr;
 8.     int capacity;
 9.     int left;
10.     int right;
11.
12.     void resize(int newCapacity) {
13.         int size = right - left + 1;
14.         int* temp = new int[size];
15.         for (int i = 0; i < size; i++) {
16.             temp[i] = arr[left + i];
17.         }
18.
19.         delete[] arr;
20.         arr = new int[newCapacity];
21.
22.         for (int i = 0; i < size; i++) {
23.             arr[i] = temp[i];
24.         }
25.
26.         left = 0;
27.         right = size - 1;
28.
29.         delete[] temp;
30.     }
31.
32.     void shift() {
33.         int size = right - left + 1;
34.
35.         for (int i = 0; i < size; i++) {
36.             arr[i] = arr[left + i];
37.         }
38.
39.         left = 0;
40.         right = size - 1;
41.     }
42.
43. public:
44.     DynamicQueue() {
45.         arr = new int[16];
46.         left = 0;
47.         right = -1;
48.         capacity = 16;
49.     }
50.
51.     void push(int x) {
52.         int size = right - left + 1;
53.
54.         if (size >= (2 * capacity) / 3) {
55.             resize(capacity * 2);
56.         }
57.         else if (right + 1 == capacity) {
58.             shift();
59.         }
60.
61.         right++;
62.         arr[right] = x;
63.     }
64.
65.     void pop() {
```

```
        if (empty()) {
            cout << "Queue is empty: cannot pop()" << endl;
            return;
        }

        left++;

        int size = right - left + 1;
        if (size <= capacity / 3 && capacity != 16) {
            resize(capacity / 2);
        }
    }

    int peek() const {
        if (empty()) {
            cout << "Queue is empty: cannot peek()" << endl;
            return 0;
        }

        return arr[left];
    }

    bool empty() const{
        return left > right;
    }
};

int main() {
    DynamicQueue q;

    q.push(1);
    q.push(2);
    q.push(3);

    while (!q.empty()) {
        cout << q.peek();
        q.pop();
    }

    return 0;
}
```

Държавен изпит – спец. „Софтуерно инженерство“

Тема №16 – Софтуерно инженерство и неговото място като дял от знанието. Софтуерен процес и модели на софтуерни процеси. Концепция за многократна употреба.

Анотация: *Изложението по въпроса трябва да включва следните по-съществени елементи:*

1. Софтуерното инженерство – какво е софтуер, видове софтуер, същност и обхват на софтуерното инженерство.

2. Софтуерен процес – фази и основни дейности, видове модели и езици за моделиране, шаблони за описание.

3. Сравнителен анализ на описателни модели на софтуерен процес – модел на водопада, прототипен модел, модел на бързата разработка, спираловиден модел и др.

4. Концепцията за многократна употреба.

Софтуерното инженерство – какво е софтуер, видове софтуер, същност и обхват на софтуерното инженерство

Софтуерът и **софтуерните технологии** стават все по-важни за икономиката и обществото. Те могат да се намерят навсякъде (*pervasive*) – от персоналните устройства като компютри, мобилни телефони, *mp3 players* до системите, които се използват за телекомуникационни, енергийни и финансови мрежи. Секторът на софтуер и софтуерни услуги играе ключова роля в информационното общество.

Повече от всякога днес организациите се стремят да доставят софтуерните продукти и услуги на пазара или на клиента **по-добре, по-бързо** и на **по-ниска цена**. **Процесът**, **хората** и **технологията** могат да се разглеждат като една свързана **триада**. Освен като един от възлите в тази триада процесът може да се разглежда и като лепило, което държи заедно триадата. Процесът, хората и технологията са решаващите фактори, които определят цената на продукта, времето за изпълнение и качеството.



Съществуват различни **интуитивни** и **формални дефиниции** на понятието **софтуер**. Много често то се идентифицира с компютърна програма. Това е твърде опростен и ограничен поглед. Според формулираното от професионалната американска асоциация на електронните инженери **IEEE** (IEEE 1993) определение, **софтуерът представлява**:

- **Инструкции** (компютърни програми), които при изпълнение осигуряват желаните характеристики, функциониране и производителност;
- **Структури от данни**, които дават възможност на програмите адекватно да обработват информация;
- **Документи**, които описват работата и използването на програмите.

Софтуерът има **двойнствена роля** – на продукт и на средство за доставяне на продукт. Продукт наричаме софтуер, който е предназначен да се продава на клиент/потребител. Могат да се разграничат два типа софтуерни продукти:

- **Общи** (generic) продукти – самостоятелни продукти/системи, които се продават на пазара и са предназначени за всеки потребител, който иска и може да ги купи

Примери: *системи за управление на бази данни, документообработващи системи, пакети за рисуване, софтуер за управление на проекти и др.*

- **Поръчани** (customized) продукти – продукти/системи, които са възложени от отделен клиент и се разработват специално за него

Примери: *системи, които поддържат определен бизнес процес, системи за планиране на ресурсите, системи за управление на електронни устройства и др.*

Основната разлика между двата типа софтуерни продукти е в това **кой определя** изискванията към продукта – в първия случай това е самата организация, а във втория – външните клиенти.

Характеристики на софтуера:

- **Логическа единица** (за разлика от хардуера, който е физическа единица)
- Софтуерът се **разработва** (developed), а хардуерът се произвежда (manufactured)
- Софтуерът **не се износва**
- Софтуерът е **сложен**, **абстрактен** – голям брой изисквания, липса на взаимно разбиране

Независимо дали софтуерът е с общо предназначение или е поръчан от конкретен клиент, могат да се идентифицират **следните категории софтуер**:

1. **Системен софтуер** (system software) – съвкупност от програми, предназначени за **обслужване на други програми**. Характеризира се с интензивна комуникация с **хардуера**, интензивно използване от много потребители, съвместно използване на ресурси и сложно управление на процеси, сложни структури от данни и многобройни външни интерфейси. Има за цел да улесни работата на хардуера

Примери: *операционни системи, драйвери за устройства, компилатори, програмни средства, асемблери и др.*

2. **Приложен софтуер** (application software) – самостоятелни програми, които решават специфична **нужда** на бизнеса или потребители

Примери: *комуникация посредством компютър (електронна поща, уеб браузър), мултимедиен софтуер (графични програми, компютърни игри), аналитичен софтуер (статистически пакети), софтуер за съвместна работа (blog, wiki), бизнес софтуер (счетоводен софтуер, софтуер за управление на проекти, софтуер за управление и планиране на ресурсите) и др.*

3. **Научен софтуер** (scientific software) – обслужват нуждите на отделни дялове от **фундаменталните науки** – астрономия, молекулярна биология, метеорология и др.

Примери: *софтуерни приложения за симулации на системи, автоматизирано проектиране, интерактивни приложения и др.*

4. **Вграден софтуер** (embedded software) – намира се **в дадена система** и се използва за реализиране и управление на определени функции вместо крайният потребител или системата

Примери: *мобилните телефони, продукти за автоматизиране на дома (климатици, термостати), медицински устройства, автомобили и др.*

5. **Продуктова линия** (product line software) – съвкупност от софтуерни системи, които притежават **общи черти**, задоволяващи нуждите на **определен търговски сегмент** или предопределена функция. Ключовото тук е използването на **общи основни компоненти**, което намалява цената на разработката
6. **Уеб приложения** (web applications) – в най-простата си форма са множество свързани файлове **хипертекст**, които представят информация, използвайки **текст** и ограничени графики. Съвременните уеб приложения обикновено са **многослойни** и могат да се използват от множество потребители едновременно. В допълнение се интегрират с **бази данни** и други приложения.
7. **Изкуствен интелект** (artificial intelligence software) – използва **нечислови алгоритми** за решаване на сложни проблеми, не подлежащи на изчисления или директен анализ

Примери: *роботика, експертни системи, разпознаване на образи, звук, невронни мрежи и др.*

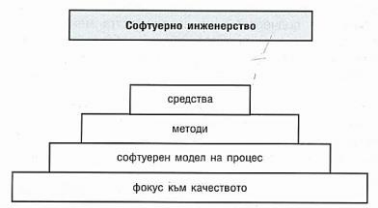
8. **Отворен код** (open source software) – всяка от изброените категории може да е отворен код. Потребителят е свободен да взема **решения** за използването на софтуерния код. В допълнение условията за разпространение на софтуера позволяват създаване на нов софтуер, **произлязъл** от отворения код
9. **Наследствен софтуер** (legacy software) – **съществуващ софтуер**, който е разработен преди няколко десетилетия и **продължава да се използва**, като непрекъснато е бил променян, за да удовлетвори промените в бизнес изискванията и платформите (хардуерни и софтуерни). Ключовите характеристики на наследения софтуер са **дълъг живот** и критична **важност** за бизнеса и често, за съжаление, **лошо качество**, поради множеството промени.

Съществуват много различни **дефиниции на софтуерното инженерство**. Според тази на професионалната американска асоциация на електронните инженери IEEE (IEEE 93):

Софтуерното инженерство е: (1) прилагането на **систематичен**, **дисциплиниран**, **количествено измерим** подход към **разработването**, **функционирането** и **поддръжката** на софтуера;
(2) **Изследване** на подходите в (1).

Софтуерното инженерство, както и останалите инженерни дисциплини, се занимава с **разбирането** и **анализирането** на даден проблем (*анализ* – разбиване на подпроблеми) и **решаването** на този проблем (*синтез* – обединяване на решенията на отделните подпроблеми). По-конкретно софтуерното инженерство е **дисциплина, която се занимава с всички аспекти на проектирането и разработването на висококачествен софтуер**. Софтуерните инженери трябва да прилагат систематичен и организиран подход в работата си и да използват подходящи инструменти и техники, зависещи от решавания проблем, ограниченията върху разработката и наличните ресурси.

Софтуерното инженерство може да се разглежда като **технология на нива**, която обхваща **процес**, **методи** и **средства**.



Софтуерното инженерство трябва да се базира на общо съгласие в рамките на организацията по отношение на **качеството**. Стабилната основа, която поддържа цялото софтуерно инженерство, е концентрирането към **качество**. Съществуват различни разбирания и гледни точки към качеството. Според едно от описанията съществуват **пет гледни точки**:

- **Абстрактен поглед** – качеството се разпознава, но не може да се дефинира
- **Потребителска гледна точка** – качеството е съответствие на поставената цел
- **Гледна точка на разработчиците** – качеството е придържането/съответствието на спецификацията
- **Гледна точка на продукта** – характеристики, които са свързани с продукта – надеждност, възможност за поддръжка и др.
- **Гледна точка на цената/стойността** – качеството е свързано със сумата, която клиентът е готов да плати

Важно е да се разграничат качество на продукт и качество на процес.

Софтуерният процес може да се разглежда като:

- **последователност от предвидими стъпки** – план, който трябва да се следва, за да се създаде продукт или система с високо качество и в срок
- **рамка за задачите**, които е необходимо да се изпълнят, за да се изгради висококачествен софтуер

Софтуерните методи представляват структуриран подход към разработването на софтуер, чиято цел е да улесни създаването на **рентабилен софтуер с високо качество**. Те отговарят на въпроса как технически да се изгради софтуерът. Първите методи са били предимно функционално ориентирани, а впоследствие те се допълват с обектно-ориентирани методи. Методите би трябвало да включват **следните компоненти**:

- описания на отделни модели на системата
- правила
- препоръки
- упътване за процеса

Софтуерните средства осигуряват **автоматична** или **полуавтоматична поддръжка** на софтуерните процеси и методи. Целта им е да **улеснят** дейностите, които се извършват в рамките на процеса на разработка. В обхвата на понятието **CASE** (Computer-Aided Software Engineering) ще включваме различни типове програми, които се използват да поддържат дейностите от софтуерния процес. Разделят се на две групи:

- **Upper-CASE средства** – поддържат началните фази – **анализ** и **проектиране**
- **Lower-CASE средства** – поддържат фазите на **имплементация** и **тестване**

Софтуерен процес – фази и основни дейности, видове модели и езици за моделиране, шаблони за описание

Процесът е последователност от стъпки, включващи дейности, ограничения и ресурси, които осигуряват постигането на някакъв вид резултат. **Процесът** е повече от процедура. Той е съвкупност от процедури, организирани така, че да се изграждат продукти, удовлетворяващи определени цели и стандарти.

Характеристики на софтуерния процес:

- Процесът **описва всички важни основни дейности**
- Процесът **използва ресурси**, които най-често са **ограничени**
- Процесът създава **междинни и крайни работни продукти**
- Процесът може да бъде **композиран от подпроцеси** с йерархии и връзки между тях
- Процесът представлява **последователност от дейности**
- Съществуват **входни и изходни критерии** за всяка дейност – начало и край на дейност
- Съществуват **ограничения** за всяка дейност

Основни цели на софтуерния процес:

- **Ефикасност** (effectiveness)
- **Възможност за поддръжка** (maintainability)
- **Предвидимост/предсказуемост** (predictability)
- **Повторяемост** (repeatability)
- **Качество** (quality)
- **Усъвършенстване** (improvement)
- **Проследяване** (tracking)

Софтуерният процес може да се разглежда като рамка за задачите, които е необходимо да се изпълнят, за да се изгради висококачествен софтуер. Пресман дефинира рамка на софтуерен процес и съответни дейности в нея. Всяка дейност на рамката се състои от действия. Действието е съвкупност от свързани задачи, които произвеждат работен продукт. Всяко действие се състои от отделни работни задачи, които изпълняват някаква част от цялата работа на действието.

Според Пресман съществуват **пет основни рамкови дейности** (basic activities), които са приложими за повечето софтуерни проекти. Тези дейности се извършват при разработването на всякакви софтуерни приложения. Във всеки отделен случай детайлите на софтуерния процес може да са различни, но основните дейности винаги присъстват. Другата категория дейности, които съпътстват основните дейности, са т.нар. допълнителни дейности (umbrella activities).

Основни дейности – приложими за всички софтуерни проекти:

1. **Комуникация** (Communication) – интензивно взаимодействие и сътрудничество с клиента. Събиране и разбиране на изисквания за функционалността и на ограниченията върху работата и разработката на продукта. Включва всички фази по инженеринг на изискванията (requirements engineering)

2. **Планиране** (Planning) – създава се план за бъдеща работа по разработка на софтуера. Описват се рисковете, необходимите ресурси, работните продукти, които ще се произвеждат и времевия график
3. **Моделиране** (Modelling) – обхваща създаването на модели, които са на различно ниво на абстракция (анализ) и създаването на дизайн, архитектурата, интерфейса и на ниво компоненти, които заедно ще удовлетворят идентифицираните изисквания (проектиране)
4. **Конструиране** (Construction) – включва генерирането на код (ръчно или автоматично) и последователно тестване – на самостоятелни компоненти, на модулите, на подсистемите и на цялата система, а също и потребителско тестване
5. **Внедряване** (Deployment) – софтуерът се предоставя на клиента и той го оценява. Дават се евентуално забележки и препоръки за подобрение

Допълнителни дейности – приложими по време на целия софтуерен процес:

- **Следене и управление на софтуерния проект** (Software product tracking and control)
- **Управление на риска** (Risk management)
- **Осигуряване на качеството** (Software quality assurance)
- **Формални технически прегледи** (Formal technical reviews)
- **Измерване** (Measurement)
- **Управление на софтуерната конфигурация** (Software configuration management)
- **Управление на повторното използване** (Reusability management)
- **Подготовка и генериране на работни продукти** (Work product preparation and production)

Тъй като софтуерният процес е сложен, възниква нуждата от моделиране с цел разбиране и прилагане на процеса. Моделът представлява абстракция, а не реалност. Той моделира реалността.

Моделът на процес представлява опростено описание на начина на разработване на софтуера, представено от определена гледна точка. В зависимост от гледната точка могат да се разграничат следните модели на процеси:

- **Модел на потока от дейности** (Workflow model) – описва **последователността от дейности**, които се извършват в процеса на разработка. Отделните дейности се описват с техните входни и изходни параметри и зависимости една от друга. Обикновено дейностите в този модел се извършват от хора. Това е един от най-разпространените типове модели на процес
- **Модел на потока от данни** (Dataflow model) – описва **множеството от дейности, които се извършват, от гледна точка на преобразуване на данни**. Например какви дейности се извършват при преобразуване на дизайна на софтуера в програмен код. Дейностите могат да се извършват от хора или компютри
- **Модел на роля/действие** (Role/Action model) – описва **отделните роли на хората, които участват в процеса на разработване на софтуер**, както и дейностите, които всяка от тези роли трябва да извършва

Софтуерните модели на процеси могат да са **описателни** или **предписателни**. Описателните модели описват историята на това как е протекло разработването на отделна софтуерна

система. Предписателните модели описват как би трябвало да се разработи нова софтуерна система.

Езиците и формализмите за моделиране на процес се използват за представяне по прецизен и изчерпателен начин на следните характеристики на софтуерния процес:

- Дейностите, които трябва да се извършват, за да се постигнат целите на процеса
- Ролите на хората, участващи в процеса
- Структурата и същността на артефактите, които се създават и поддържат
- Средствата, които се използват

Класификация на езиците за моделиране спрямо парадигмата:

- **Дескриптивни** (логически) – използват правила или тригери
- **Мрежово базирани** – представят процесите като мрежи
- **Императивни** – базирани на езиците за програмиране

Класификация на езиците за моделиране спрямо тяхната организация:

- **Entity-Relation** – същности и връзки между тях
- **Role-Interaction** – роли и техните взаимоотношения
- **Object-Oriented** – основната единица е обект, който има данни и функционалност

Класификация на езиците за моделиране спрямо тяхната формалност:

- **Формални** – формален синтаксис и семантика; правила за употреба; конструкции
- **Полуформални** – графично представяне с формален синтаксис; без формална семантика
- **Неформални** – няма строго дефинирани правила; разчита се на опита и употребата им

Шаблонът е описание на **общо решение на общ проблем или въпрос**, на базата на което може да се извлече детайлно решение на специфичен проблем. Съществуват редица шаблони, свързани с разработката на софтуер – шаблони на анализа, шаблони на проектирането, организационни шаблони, шаблони на процес и др.

Шаблон на процес е шаблон, който описва доказан, успешен подход или последователност от действия за разработване на софтуер. Описва какво трябва да се направи, а не точни детайли как трябва да се направи. Шаблоните могат да бъдат дефинирани на различни нива на абстракция.

Съществуващите типове шаблони на процеси в нарастваща степен на абстрактност са:

1. **Шаблон за задача на процес** (Task process patterns) – описва отделни стъпки за извършване на действие или задача, която е част от процеса
2. **Шаблон за етап на процес** (Stage process patterns) – дефинира стъпките, които се извършват итеративно в рамките на една базова дейност за процеса
3. **Шаблон за фаза на процес** (Phase process patterns) – дефинира последователността и взаимодействието между базови дейности, които се случват в рамките на процеса

Всеки шаблон на процес трябва да съдържа следните елементи:

1. **Начален контекст** – описание на условията, които трябва да са налице
2. **Проблем** – описание на проблема, който трябва да се реши от шаблона
3. **Решение** – какво точно прави шаблонът с цел да реши проблема
4. **Краен (резултатен) контекст** – условията, които трябва да са налице в резултат на успешното прилагане на шаблона
5. **Свързани шаблони** – шаблоните, които са директно свързани с този шаблон
6. **Начини на употреба** – описание на случаите, в които шаблонът е приложим

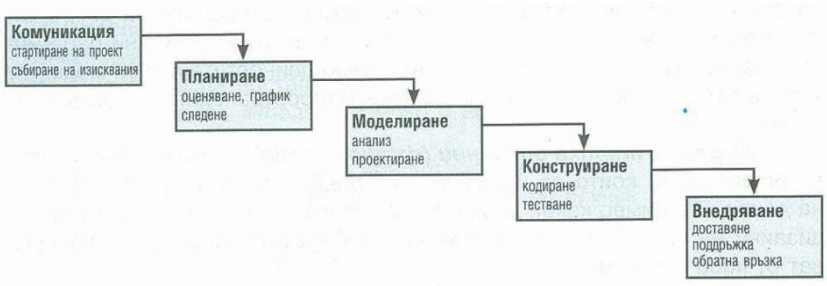
Сравнителен анализ на описателни модели на софтуерен процес – модел на водопада, прототипен модел, модел на бързата разработка, спираловиден модел и др.

1. Модел на водопада (каскаден модел)

➢ **Същност**

Моделът на водопада е най-старият метод на структурирано разработване на софтуер. Той осигурява теоретичната база за останалите модели на процеси и все още намира приложение в практиката. Предлага систематизиран, последователен подход към разработването на софтуер, който включва следните **основни дейности**:

*a. **Събиране на софтуерните изисквания***
*b. **Оценяване, изготвяне на график, проследяване***
*c. **Анализ и проектиране***
*d. **Генериране на код и тестване***
*e. **Доставяне и поддръжка***



➢ **Характеристики**

❖ **Ясно разграничен процес**, който е лесен за разбиране
❖ Всяка стъпка от модела завършва със създаване на множество от документи – например документ с изискванията, документ с дизайна, програмен код, документи за тестването
❖ Всяка дейност трябва да бъде **напълно завършена**, преди да се премине към следващата, а това става като се одобри множество от документи
❖ Ясно са дефинирани **входовете и изходите на дейностите**, както и интерфейсите между отделните стъпки
❖ Ясно са дефинирани **ролите** на разработчиците на софтуер

➢ **Проблеми**

▪ Реалните проекти **рядко следват последователния поток** на разработване, който се предлага от модела. Критики:
  - Моделът налага по-скоро структура на управление на проект за разработване на софтуер, отколкото да дава насоки как да се извършват отделните дейности

- Моделът е произлязъл от областта на хардуера и не отчита същността на софтуера като творчески процес на решаване на проблем (с итерации и връщане назад)

- Трудно е за потребителя да **дефинира всичките си изисквания в началото**
- Клиентът трябва да е **търпелив**, защото не вижда междинен резултат
- Разделянето на проекта на отделни етапи **не е гъвкаво**
- Трудно е да се реагира на променящите се изисквания на клиента

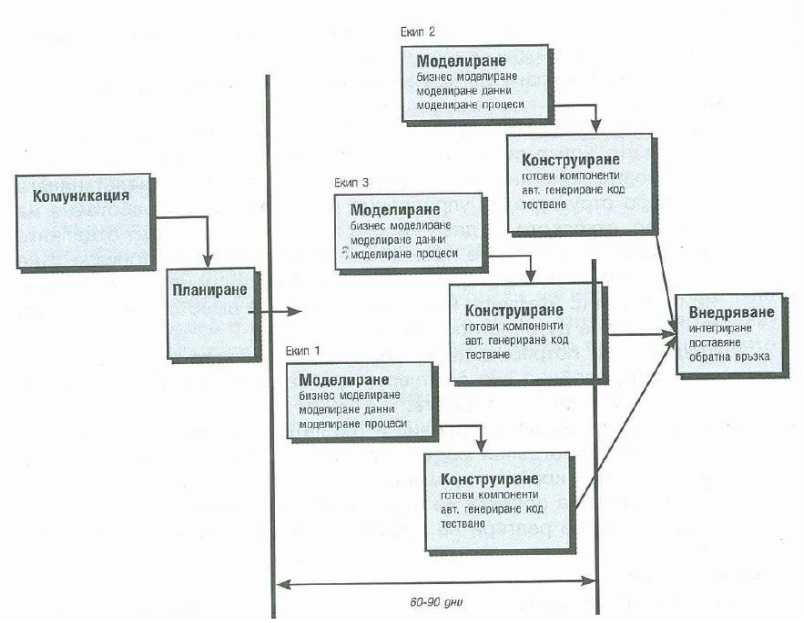
➢ **Използване на модела**

✓ Когато изискванията са **осъзнати** и **ясно формулирани** в началото
✓ Когато проектите са **ясно организирани** в смисъл на ясно дефинирани роли
✓ При **повторяеми проекти** или големи проекти, за които времето и бюджетът не са критични

2. Модел на бързата разработка (Rapid Application Development – RAD)

➢ **Същност**

Целта на този модел е **кратък цикъл на разработка**. Нарича се още „висикоскоростна“ адаптация на модела на водопада. За постигането на целта се разчита на използването на различни средства за бърза разработка. Под бърза разработка се има предвид период от 60-90 дена.



➢ **Характеристики**

❖ **Комуникация**
❖ **Планиране** – няколко софтуерни екипа работят паралелно върху планирането на различни функционалности от крайния продукт, разделени в модули
❖ **Моделиране** – всеки екип паралелно с останалите извършва отделно моделиране на своя модул

- Бизнес моделиране (Business modelling)
- Моделиране на данните (Data modelling)
- Моделиране на процес (Process modelling)

- ❖ **Конструиране** – най-често се използват предварително съществуващи софтуерни компоненти, които се интегрират, и средства за автоматично генериране на код
- ❖ **Внедряване**
  - ➢ **Проблеми**
- ▪ За **големи приложения**, подлежащи на разделяне на модули, този модел изисква значителни човешки ресурси, които формират различни екипи за паралелна работа
- ▪ Когато функционалността на софтуерната система не може да бъде **подходящо разделена** в отделни модули
- ▪ Когато е важна **високата производителност** на софтуерното приложение – множество интерфейси между отделните модули
- ▪ Когато за разработката на приложението се разчита на все още **нови и недостатъчно усвоени технологии**
  - ➢ **Използване на модела**
- ✓ При проекти, чиято функционалност може да се разпредели в **отделни модули**, така че те да могат да се реализират в рамките на период от 60-90 дни
- ✓ Когато организацията разполага и може да задели **достатъчно разработчици**, които да се разпределят в паралелни екипи за бързата разработка

3. Фазови (еволюционни) модели

- ➢ **Същност**

Постъпковото разработване на софтуер не доставя системата като едно цяло, а вместо това процесът на разработка и доставянето са **разделени на стъпки**, като всяка стъпка доставя само част от цялата функционалност.

На идентифицираните потребителски изисквания се присвояват **приоритети** и тези с по-висок приоритет се реализират в първите стъпки.

След като започне разработката на една стъпка, изискванията не се променят, но изискванията за по-късните стъпки, по които още не е започната работа, могат да се променят.

Моделът на инкременталното разработване комбинира елементи на модела на водопада, но приложени на отделни стъпки. При него се доставят на клиента последователност от версии на продукта, наречени инкременти. Всяка следваща версия съдържа допълнителна функционалност.

За разлика от постъпковия модел **итеративният модел** на процес още в самото начало доставя цялостната софтуерна система, макар и част от функционалността да е в примитивна форма.

На всяка следваща итерация **постепенно се доставя все по-завършена версия на софтуера** в смисъл на подобрено качество. За разлика от постъпковия модел тук при всяка итерация не се добавя нова функционалност, а само се усъвършенства вече съществуваща.

На практика много организации прилагат **комбинация от постъпков и итеративен модел** на процес.

➢ **Проблеми**

- Необходимостта от **активно участие на клиентите** по време на изпълнение на проекта може да доведе до закъснения
- **Уменията за комуникация и координация** са от особено голямо значение при разработката и ако не са на достатъчно добро ниво, водят до проблеми
- **Неформалните заявки за подобрения** след завършването на всяка стъпка могат да доведат до объркване
- Този модел може да доведе до т.нар. **„scope creep“** – бавно и постепенно разширение на обхвата на приложението, без процесът да е сходящ

➢ **Предимства**

- Клиентът може да **използва системата, преди да е готов** целият продукт
- Първите стъпки могат да служат като **прототип**, за да помогнат за извличане и изясняване на изискванията към следващите стъпки
- **По-малък риск от неуспех** на целия проект
- Функционалностите от цялата система, които **са с най-висок приоритет, са тествани най-много**, тъй като са част от системата от самото начало и на всяка следваща стъпка се тестват отново

➢ **Използване на модела**

✓ Когато организацията **няма достатъчно човешки ресурс** за цялостната реализация в определен срок – плаващ брой разработчици в зависимост от нуждите

✓ Когато с итерациите могат да се управляват **технологичните рискове** – изместване на итерации, изискващи определени умения с нови технологии, за по-късен етап

4. Прототипен модел

➢ **Същност**

Според Съмървил използването на **прототип** може да се раздели на два типа – **еволюционен прототип** (да достави работеща система на крайния потребител) и **хвърлен** (throw-away) **прототип** (да подпомогне специфицирането на изискванията към софтуера).

Основната цел на **прототипния модел** е изграждане на прототип, който да извлече или валидира системните изисквания. Освен това прототипът осигурява допълнителна информация, въз основа на която могат да се оценят рисковете на процеса.

Основните дейности са същите като описаните. Специфичното е, че след комуникацията се извършва бързо планиране, проектиране и конструиране на прототип, който отразява най-неясните изисквания към софтуера. След доставянето на клиента и получаването на оценка от него прототипът се променя, като вече е направил крачка към по-добро изясняване на изискванията. Тези итерации на промяна продължават, докато заинтересованите лица постигнат общо съгласие по изискванията и дизайна на софтуера.

➢ **Проблеми**

- Не е подходящ за използване при разработване на софтуерни системи, където **проблемът е добре разбран** и интерфейсът е ясен и прост
- Може да използва **значителни ресурси**, а като резултат да не успее да удовлетвори очакванията

- Може да доведе до **лошо проектирана система**, ако самият той стане част от крайния продукт

➢ **Използване на модела**

✓ В проекти, където не са достатъчно **ясни потребителските изисквания** и дизайнът на софтуерната система

✓ Както самостоятелно, така и в комбинация с други модели на процеси – модел на водопада, спираловиден модел, постъпков модел и т.н.

5. Спираловиден модел

➢ **Същност**

Спираловидният модел е еволюционен модел на софтуерен процес, който съчетава прототипния модел и модела на водопада. Движещият фактор е **анализ на риска** и това е неговата специфична характеристика. Основните характеристики са, че е **итеративен/цикличен подход** и има множество от **точки на прогреса** (anchor point milestones).

Обикновено този модел се представя със **спирала**, като всяко пълно завъртане по спиралата представлява една стъпка от разработката. Освен това спираловидният модел може да включва в себе си други модели.

При всяко завъртане по спиралата, независимо каква е целта му, се преминава през следните **сектори**:

a. **Установяване на целите** – определят се целите, алтернативите и ограниченията на текущата фаза от разработката
b. **Оценка на рисковете и намаляването им** – идентифицират се и се анализират потенциалните рискове и в допълнение се предприемат действия за намаляването или елиминирането им
c. **Разработване и валидиране** – избира се модел за разработване на текущата фаза
d. **Планиране** – преглежда се и се анализира текущото състояние и се планира следващото завъртане по спиралата

➢ **Проблеми**

- Може да се окаже **трудно да се убедят клиентите**, че процесът на разработка е контролируем, а не е безкраен цикъл
- Изисква се **участието на разработчици с компетентност за оценка на рисковете**
- Ако не се **идентифицира и открие някой основен риск**, това може да доведе до неуспех

➢ **Използване на модела**

✓ При разработването на **големи (large-scale) софтуерни системи**

✓ Може да се адаптира и да се прилага през **целия жизнен цикъл на софтуера**:

- Проекти за **изграждане на концепция** за софтуера
- Проекти за **разработване на нов продукт**
- Проекти за **подобрение на продукт**

6. Метод на формалната трансформация

➢ **Същност**

Основава се на **математическо трансформиране** на спецификацията на системните изисквания до изпълнима програма. При трансформирането трябва да се запази коректността и ясно да се покаже, че изпълнимата програма съответства на спецификацията. Спецификацията на софтуерните изисквания се усъвършенства в детайлна формална спецификация, изразена с математическа нотация. Дейностите моделиране и конструиране са заменени с разработване и прилагане на **трансформации**.

➢ **Проблеми**

- Разработването на формални модели е **скъп и бавен процес**
- Необходими са **разработчици със специализирани умения**, както и обучение за това как да се прилага формалната трансформация
- Поради **сложността си** изготвените модели трудно могат да се използват
- Някои от аспектите на софтуерна система е **трудно да се специфицират формално** – например потребителският интерфейс

➢ **Използване на модела**

✓ При разработването на софтуерни системи, които са свързани с поддръжката на **човешки живот** и за които трябва да са гарантирани **сигурността** и **отказоустойчивостта** на софтуера преди да започне реалното му използване

Концепцията за многократна употреба

Днес мнозинството от софтуерните инженери се занимават с промяна на вече съществуващ софтуер. Следователно, софтуерните системи би трябвало да се разработват така, че да **улесняват промените**, с други думи, да може лесно да се отстранява и да се добавя функционалност.

Докато гъвкавите методологии предлагат един подход за решаване на проблема с по-бързо доставяне на софтуер за пазара или клиента, то **компонентно-базираното разработване на софтуер** се основава на друг подход, а именно *интегриране на вече съществуващи софтуерни компоненти в цялостна система*; изграждане на компоненти с идеята да бъдат повторно използвани в различни приложения и разграничаване на разработването на компонент от разработването на система от компоненти.

В контекста на създаване на софтуер **многократната употреба** е всяка процедура, която създава (или помага при създаване) на софтуерна система чрез повторно използване на нещо от предишни разработвания на системи.

Повторното използване е подход за разработване, който се опитва да извлече максимална полза от повторно използване на съществуващ софтуер. Софтуерните единици могат да бъдат с различен размер – от цялостни приложни системи, през отделни компоненти, до обекти и функции. Повторното използване на софтуер се допълва от повторно използване на концепции.

Според Съмървил **потенциалните предимства** на повторното използване са следните:

➢ Повишена отказоустойчивост
➢ Намален риск при процеса на разработване
➢ Ефективно използване на квалифицирани софтуерни инженери

- Съвместимост със стандарти
- Ускорено разработване

Съответно **проблемите** при повторното използване са следните:

- Повишена цена за поддръжка (съпровождане)
- Липса на софтуерни средства, подпомагащи повторното използване
- Синдромът „не е измислено тук“
- Създаване и поддържане на библиотека от компоненти – изисква ресурси
- Намиране, разбиране и адаптиране на повторно използваеми компоненти

Съществуват два концептуални подхода за многократна употреба на софтуер:

- **Базиран на съществуващи програмни единици**
  - **COTS (Commercial Off-The-Shelf) продукти**

COTS системите обикновено са цели приложения, които предлагат **API (Application Programming Interface – приложно програмен интерфейс)**. Приложими са в широк кръг от области. Ключовото тук е да се намери този COTS продукт, който предлага най-подходящата функционалност и да се реши въпроса с комуникацията между модулите – всеки модул има собствен формат и структура на данните. Също така, обикновено чрез COTS продуктите се получава повече функционалност от реално необходимата и понякога трябва да се добави обвивен **wrapper** модул, който да предотврати достъпа до ненужната функционалност.

  - **Компонентно-базирано разработване**

Компонентите са по-общо понятие от обектите – един компонент може да е изграден един или няколко обекта. Чрез компонентите се постига по-високо ниво на абстракция. Софтуерният компонент е градивна единица на софтуерните системи. Компонентите могат да осъществяват връзка помежду си единствено чрез входно-изходни точки, наречени интерфейси.

В най-общ смисъл различаваме **два вида компоненти**:

- **Входен (requires) интерфейс** – дефинира какво трябва да се предостави на компонента, така че той да се изпълни според спецификацията
- **Изходен (provides) интерфейс** – дефинира услугата и резултата от нея, които компонентът предоставя, в следствие на изпълнението му при условие, че са удовлетворени изискванията на входния интерфейс

Примери: *EJB, Corba Component Model и др.*

  - **Аспектно-ориентирано разработване**

Засяга и цели да постигне т.нар. **разделяне на функциите** (separation of concerns). Припокриващата се функционалност се разработва като аспекти, които динамично се вмъкват в програмата.

  - **Архитектура, ориентирана към услуги**

- **Базиран на създаване на програмни единици**
  - **Шаблони на проектирането (design patterns)**

Съществуват редица техники за подпомагане на многократната употреба. Най-популярните от тях са т.нар. **шаблони на проектирането** (design patterns). Те са абстрактни и целят да могат да се прилагат в множество различни ситуации, без да зависят от конкретен програмен език. Най-често стъпват на свойствата на класовете наследяване и полиморфизъм. В зависимост от предназначението си, различаваме следните **видове шаблони за проектиране**:

- **Създаващи шаблони на проектирането (creational design patterns)** – използват се за създаване на обекти
- **Структурни шаблони на проектирането (structural design patterns)** – използват се за улесняване на интеграцията на класове/обекти
- **Поведенчески шаблони на проектирането (behavioural design patterns)** – предоставят методи за комуникация между обектите
  - **Програмна генерация**

Програмните генератори използват вече дефинирани шаблони и алгоритми. Те са вградени в генератора и могат да се параметризират. Програмата се генерира автоматично. Програмната генерация е възможна, когато може да се дефинира съответствие между понятията от приложната област и програмния код. Използва се специфичен (за проблемната област) език. Програмната генерация е **ефективен метод за разработка**, но **приложим в ограничен набор от приложни области**.

Държавен изпит – спец. „Софтуерно инженерство“

Тема №17 – Модели на разпределени софтуерни архитектури. Среди и протоколи за разпределени приложения.

Анотация: *Изложението по въпроса трябва да включва следните по-съществени елементи:*

1. Параметри на паралелната и разпределена обработка, метрика, методи за анализ.

2. Модели на разпределените софтуерни архитектури – процедурни, обектни, потокови, контекстни, йерархични, асинхронни и интерактивни модели на софтуерната архитектура. Структури, организация, компоненти, приложение.

3. Организация на разпределените приложения – клиент-сървер, двуслойни, трислойни и n-слойни модели. p2p. Сървери за приложения и Web-сървери. Метасистеми и грид. Сервизно-, моделно- и аспектно-ориентирани архитектури. Софтуерни агенти.

Примерна задача: *Анализ на изчислителната сложност, ускорението, ефективността и цената на обработка на базов или еталонен алгоритъм за паралелна или разпределена обработка.*

Параметри на паралелната и разпределената обработка, метрика, методи за анализ

Конкурентното програмиране се разделя на **паралелно** и **разпределено програмиране**. Паралелната обработка наричаме обработка на данни, която се извършва от няколко процеса или нишки, които общо наричаме работници (workers). Те са разположени на една машина, докато при разпределената обработка процесите са разположени на няколко машини, които общо ще наричаме изчислителни възли. При паралелната обработка целта е ускорение (speed), а при разпределената – технологична.

Параметър	Паралелно програмиране	Разпределено програмиране
Вид инфраструктура	мултипроцесор	мултикомпютър
Брой машини (възли)	1	$p_{HW}; 10^1, 10^2$
Модел, вид декомпозиция	SPMD /Single Program, Multiple Data/ декомпозиция по данни, /една програма, която стартираме върху различни данни/	MPMD /Multiple Program, Multiple Data/ декомпозиция по код, по управление /няколко различни програми, които имат различни роли/
Обмен на данни	обща памет (променливи) и механизъм за синхронизация	предаване на съобщения чрез middleware

Цел	$S(p)$ – ускорение /speed/	технологична
Примери за синхронни задачи	WaTor, nBody	Client-Server, p2p, шаблони, някои конвейери
Примери за асинхронни задачи	фрактали (Mandelbrot)	пакетна обработка, някои конвейери

1. Паралелизъм – p

Паралелизмът обикновено го бележим с p и с него означаваме максималния брой операции, които могат да се изпълнят паралелно при обработката на алгоритъма. Това обикновено се свързва с броя на работниците, които работят по време на изпълнение на дадена програма.

Паралелизмът се дели на два вида – **хардуерен паралелизъм** (машинен паралелизъм) и **софтуерен паралелизъм** (паралелизъм на приложението). Софтуерният паралелизъм е характеристика на програмите – колко работници се стартират. Хардуерният паралелизъм зависи от броя процесори (ядра), върху които се изпълнява задачата. Ускорението от софтуерния паралелизъм е ограничено от хардуерния – не може да очакваме 4 пъти ускорение от 4 нишки, ако машината има по-малко от 4 ядра дори и ако на едно ядро се стартират няколко нишки.

2. Грануларност – $g = \frac{N}{p} \geq 1$

Грануларност има и при паралелната обработка, и при разпределената обработка.

При **паралелната обработка грануларността** е степента на декомпозиция на данните, т.е. на колко подзадачи се разбива цялата задача. Различават се три вида грануларност, които се разграничават не по фиксирани точни диапазони, а по ефекта, който постигат:

- **едра грануларност** – $g \in [10^0, 10^1]$ – малък брой подзадачи; малко време за разпределение на задачите на работниците, за сметка на лошо балансиране в средния случай (тъй като някои работници получават по-сложни задачи от други)
- **фина грануларност** – $g > 10^{1.5}$ – голям брой подзадачи; задачите са достатъчно голям брой, за да се счита, че всички работници ще получат по равен брой задачи в средния случай, но за сметка на много време по разпределението им
- **средна грануларност** – $g \in [10^1, 10^{1.5}]$ – компромис между горните два вида; това е златната среда, към която алгоритъмът за паралелна обработка трябва да се стреми

Пример: *Имаме p работници. Имаме за задача матрица, по която трябва да сметнем нещо. Матрицата трябва да поделим на равни части – p на брой, като разлика от порядъка на един елемент не е проблем. Това е работа с най-едра грануларност. Ако я разделим на $2p, 3p, \ldots$ говорим за все по-фина грануларност. Максималната фина грануларност е да разглеждаме всеки елемент на матрицата като едно задание. По-фина грануларност ни трябва за да правим балансиране. Колкото по-фина е грануларността, толкова повече се влошава динамичното балансиране.*

Другият вид **грануларност при разпределената обработка** е по код за разпределените софтуерни архитектури. Това е относителната тежест на отделните атомарни услуги – с колко услуги на брой ще създадем едно сървърно приложение. Повече на брой (по-къси) означава по-фина грануларност.

3. Балансиране

Балансирането е такова разделение на подзадачите, при което всеки процес да получи равен на брой задачи с относително еднаква трудност в цялост. Доброто балансиране ще гарантира, че подзадачите ще завършват приблизително по едно и също време. Това е желана цел, тъй като времето за работа на един паралелен алгоритъм е равно на времето за работа на най-бавния работник в него, т.е. не се очаква един работник да приключи работа много по-рано от друг.

Балансирането може да се раздели по няколко признака:

- **Време на осъществяване**
  - **Статично балансиране** – подзадачите се дефинират като предварителна операция на алгоритъма, след което стартира паралелизма; ясно е коя задача от кой работник ще се извършва; в резултат нямаме наличие на тясно място (bottleneck), нямаме централизирана опашка; има скалируемост
  - **Динамично балансиране** – при постъпване на нова подзадача, динамично се определя кой работник да я поеме; разпределението на задачите става по време на изпълнение и няма детерминистичност по отношение на броя изпълнение задачи;
- **Структура на разбиване**
  - **Централизирано** – топология звезда
  - **Разпределено** – например верига, решетка, хиперкуб и др.
  - **Йерархично** – клъстери, дърво

4. Синхронизация

Синхронизацията описва обема на информация, който процесите използват, за да си комуникират помежду си по време на изпълнение и чрез какви механизми се осъществява тази комуникация. В това отношение разглеждаме три вида алгоритми:

- **Асинхронни** – не обменят данни между различните си работници; никой от процесите не знае за съществуването на останалите процеси
- **Локално синхронни** – всеки работник си има дефинирани съседи, с които обменя информация; приема информация от едни и предава на други, като тези от които приема и тези, на които предава, са точно дефинирани
- **Глобално синхронни** – всеки работник може да си комуникира с всеки останал (работниците образуват пълен граф)

5. Ускорение

Ускорението се пресмята по формулата $S_p = \frac{T_1}{T_p}$, където T_1 е времето за последователна обработка, а T_p е времето за паралелна обработка при паралелизъм p. Поради наличието на комуникационни и синхронизационни закъснения, обикновено $S_p \in (1, p)$, но съществуват и

изключения. При оценка или измерване на ускорението се приема, че всички процесори са с идентична производителност.

6. Ефективност

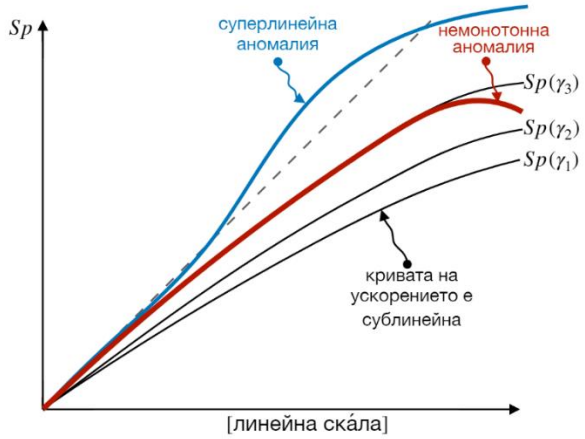
Ефективността се пресмята по формулата $E_p = \frac{S_p}{p} = \frac{T_1}{pT_p} < 100\%$. Представлява нормализираното ускорение и характеризира частта от общото време за паралелна обработка, през която процесорните елементи се използват ефективно.

7. Суперлинейна аномалия

Този тип аномалия имаме, когато $S_p > p$. Причината за суперлинейната аномалия е такава фрагментация (декомпозиция на данните), която позволява обработката на алгоритъма ни да се изпълни без скритите операции по подготовка на кеша. Тази аномалия се дължи на управлението на кеша. При пускане на повече нишки може да се включат повече хардуерни ядра, като всяко от тях има собствен $L1$ кеш. Допълнително, ако областите от данни се припокриват между нишки, то дадено парче от данни се зарежда само веднъж от по-бавната оперативна памет и след това се достъпва бързо от споделения $L2$ кеш, т.е. размерът на отделното задание се е напаснал точно в пространството на кеша.

8. Немонотонна (нелинейна) аномалия

Този тип аномалия имаме, когато $S_{p+1} < S_p$ за някое p. Тоест програмата започва да се забавя при увеличаване на паралелизма. Сред възможните причини за това може да е надвишаване на хардуерния паралелизъм.



9. Закон на Амдал (Amdahl)

При наличие на две интензивности (темпове) на обработка на даден проблем – високопаралелна R_h и нископаралелна R_l, които са в съотношение $f:(1-f)$ про брой на генерирани резултати (междинни и крайни), общата интензивност на обработката е:

$$R(f) = \frac{1}{\frac{f}{R_h} + \frac{(1-f)}{R_l}}$$

Следователно, за $f \to 1, R(f) \to R_h$ и при $f \to 0, R(f) \to R_l$.

Модели на разпределените софтуерни архитектури – процедурни, обектни, потокови, контекстни, йерархични, асинхронни и интерактивни модели на софтуерната архитектура. Структури, организация, компоненти и приложение

Софтуерната архитектура дефинира какви са съставните компоненти и процеси на една софтуерна система, как те са структурирани и си взаимодействат. Моделите на софтуерни архитектури се представят чрез диаграми, описващи дизайна на системата. Под дизайн се разбира съвкупността от декомпозицията на съставните компоненти, функционалната им композиция, прилагания архитектурен стил и качествените (нефункционалните) атрибути на услугата – QoS.

Процедурните архитектурни модели се базират на разпределена обработка чрез *отдалечено извикване на процедури* (Remote Procedure Call – RPC) – неспециализирано, но прилагано главно за клиент-сървър. Представлява извикване на процедура на един възел от друг възел, като това е прозрачно за извикващия възел, т.е. е като локална за него процедура. Това е синхронен комуникационен модел, тъй като е с отлагане на викащия процес.

Обектно-ориентираните архитектурни модели се основават на разделянето на отговорностите на системата в индивидуални повторно използваеми и независими обекти, които си сътрудничат. Основните принципи на тези модели са:

- **Абстракция** – опростяване на решението на реален проблем чрез дефиниране на подходящи класове
- **Композиция** – изграждането на обекти от други обекти
- **Наследственост** – автоматичното присвояване на функционалностите на едни обекти на други обекти и тяхното разширение и промяна
- **Капсулиране** – скриването на ненужните за потребителя детайли по отношение на строежа на дадени обекти
- **Полиморфизъм** – възможност на даден обект да се държи като друг, запазвайки логиката на собственото си поведение

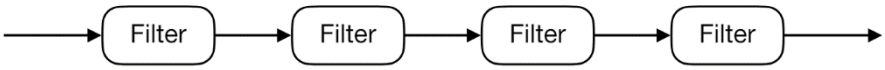
По отношение на комуникацията чрез обектен модел в клиент-сървър архитектури се използва *отдалечено извикване на методи* (Remote Method Invocation – RMI). Разпределените обекти поддържат изпълним интерфейс на отдалечена машина, т.е. достъпът до техните методи може да се реализира чрез отдалечения интерфейс.

Потоковите архитектурни модели разглеждат системата като последователност от трансформации на постъпваща като от линеен поток информация. Всеки компонент трансформира входните си данни в изходни. Топологията на пренос на данните между тях се задава експлицитно чрез блок-диаграми. Модулите поддържат само интерфейс по данни, но не и контролен, като те не се адресират директно, а чрез предаваните данни. Системата се декомпозира на функционални модули или подсистеми. Това само по себе си представлява паралелизъм по управление, като аналогия с механизма на конвейерите.

Можем да разграничим няколко вида потокови архитектури в зависимост от това кога и как се извършва обработката на данните от всеки модул:

- **Пакетна обработка (Batch Sequential)** – модулите се извикват в последователен ред един след друг, като изходът от един модул се предава като вход на следващия; пакетите от данните, които се обработват от всеки модул, са записани под формата на временни

файлове, които служат за комуникация между два последователни в изпълнението си модули; ключовото е, че обработката от следващия модул може да е **отложена** във времето; най-често този модел се използва за асинхронна обработка на данни, като се цели оптимално използване на ресурси за сметка на паралелизъм и интерактивност, които са ограничени и дори липсват



- **Филтрирани канали (Pipe & Filter)** – обработката се случва в **онлайн** (интерактивен) режим; всеки процес (отговарящ на модул) е активен и е свързан директно със съседите си; всеки процес започва да предава изходните данни към следващия филтър преди да е свършил с обработката на всички данни; този начин на обработка позволява паралелизъм (**конвейеризация**)
- **Контролни процеси (Process Control)** – като Pipe & Filter, но с добавени ограничения върху **реалното време за изпълнение** (realtime constraints); прилага се най-вече при вградените системи (BAC); управлението е на базата на сканиране на променливи на средата и управляващо въздействие чрез контролирани актуатори

Контекстните (Data Centric) архитектурни модели се характеризират с централизирано хранилище за данните, откъдето те са достъпни за всички останали компоненти на системата. Декомпозицията на модулите се състои от модул за управление на достъпа до данните и агенти, които извършват операции върху тях. Интерфейсът между агентите и данните може да бъде явен (напр. RPC или RMI) или имплицитен (напр. транзактивен). Оригиналният модел не предвижда комуникация между агентите.

Съществуват два основни контекстни модела:

- **Хранилище (Repository)** – агентите са активни, т.е. те инициират взаимодействието

Примери: *релационни СУБД, CORBA, UDDI, Web-услуги*

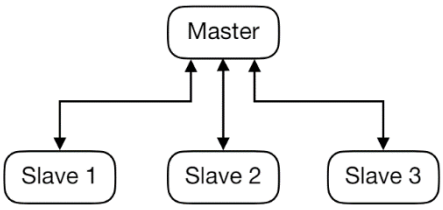
- **Черна дъска (Black board)** – инициативата е на модула с данните, а агентите се абонират за събития (event listeners); събитие настъпва при промяна в данните (publish-subscribe)

Примери: *AI-разпределени приложения, охранителни системи за разпознаване на звук и образ, експертни системи, системи за управление на бизнес ресурси – складове, транспорт*

Йерархичните архитектурни модели декомпозират системата на йерархични модули, като функциите се групират по йерархичен принцип на няколко нива. Координацията обикновено е между модули от различни нива (вертикална свързаност) и се базира на явни (т.е. „заявка-отговор“) обръщения. Ниските нива функционират като услуги към непосредствените по-високи нива. Пълна прозрачност между нивата се постига при запазване на свързващите интерфейси, но имплементацията на услугите може да еволюира. Този архитектурен модел е характерен за много операционни системи (UNIX, MS .NET), както и протоколни стекове (TCP/IP, OSI).

Ще разгледаме два основни типа йерархични модели:

- **Master/Slaves** – представлява разбиване на главна програма и подпрограми; главната програма разпределя работата на подчинените и след това е възможно да ги координира; това е вариант на архитектурата с подпрограми, който е специализиран към поддържане на допълнителни нефункционални изисквания:
  - **Отказоустойчивост (fault-tolerance) и надеждност** – постига се като главната програма проверява резултата от подчинените, както и статуса им
  - **Балансиране за ускорено изпълнение на заявките –** постига се чрез балансиране на натоварването между подчинените за по-добро ускорение



- **Слоести архитектури** – при този модел имаме последователност от слоеве, като всеки предоставя услуги на по-долните; използван при много операционни системи като UNIX, GNU и Windows; също така протоколните стекове OSI и TCP/IP са слоести архитектури; целта на този тип архитектура е бързо проектиране на (преносими) приложения чрез използване на компонентни имплементации

Асинхронните модели на разпределени софтуерни архитектури се базират на неявни (implicit) асинхронни обръщения между обслужващите процеси, т.е. обмяна на съобщения (message passing).

Асинхронният обмен може да бъде:

- **Тип Производител/Консуматор (Producer/Consumer**) – обменът на данни е 1:1; има следните разновидности:
  - **Интерактивен (online)** – няма буфериране и двата процеса трябва да са активни, но не блокират изчакващо в точката на обмен
  - **Отложен (offline)** – обменът на съобщенията се опростява посредством процес-буфер, който служи като пощенска кутия; това позволява консуматорът да не е активен по време на изпращане на съобщението, както и обратно
- **Тематичен: Издател/Абонати (Publisher/Subscriber)** – обменът на данни е 1:много; има следните разновидности:
  - **topic-based** – съобщенията се изпращат до теми (topics), които са именувани като канали; издателят е отговорен за дефиниране на темите, а абонатите получават от всички теми, за които са се абонирали
  - **context-based** – съобщенията се изпращат до абонат, ако атрибутите на съдържанието отговарят на ограниченията, дефинирани от абоната; абонатът е отговорен за класификация на съобщенията

Интерактивните модели на разпределени софтуерни архитектури поддържат интензивен потребителски интерфейс. За целта декомпозицията на системата е на три функционални модула:

- **Модул за представяне (изглед)** – имплементира потребителския интерфейс; използва се за представяне (в това число графично или мултимедийно) на изходните данни и намеса на потребителите в обработката (т.е. вход за данни и контрол)
- **Модул на данни** – поддържа на модела на данните заедно с базова функционалност за обработка върху тях
- **Модул за управление** – отговаря за системни комуникации, управление на процесите, инициализиране и конфигуриране на модули данни, както и управление на изгледи я

Горните компоненти се имплементират от модела **MVC** (**Model-View-Controller**). **PAC** (**Presentation-Abstraction-Control**) е MVC за многоагентна система, например ако има много източници на данни. При него се поддържа агентен обмен на съобщения. Представлява йерархично (разслоено) и разпределено управление, при което системата се формира от набор коопериращи агенти на три нива – базово ниво, състоящо се от агенти, работещи върху общи данни и бизнес логика, средно ниво, състоящо се от агенти координатори на изгледите и ниво на изгледите за обработка на локални данни. Всеки агент интегрира P, A и C компоненти. При MVC агентите са равнопоставени.

Организация на разпределените приложения – клиент-сървър, двуслойни, трислойни и n-слойни модели. p2p.

При наличие на множество потребители, като при системите за масово обслужване, най-популярният вид декомпозиция е MPMD/MU.

Клиент-сървър е класически MPMD модел за адаптиране на ограничената клиентска инфраструктура за изпълнение на сложни и съставни услуги. Процесите се класифицират като два основни вида:

- **Сървър** – реактивен процес, който имплементира приложната логика и модела на данните под формата на конкретен набор от услуги
- **Клиент** – активен интерфейсен процес, който използва услугите на сървъра

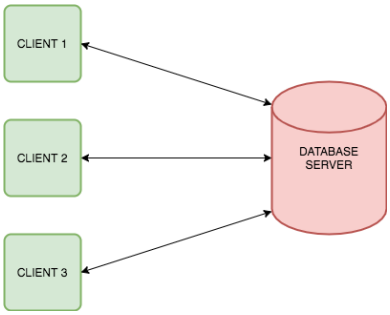
Сървърите са специално проектирани устройства, проектирани да обработват определен товар. Те са свързани с опорни мрежи (backbone). Клиентските устройства (клиентска инфраструктура) са унифицирани (напр. браузърен достъп).

Има **два основни вида клиенти** – **тънки** (thin), които делегират цялата работа на сървъра, и **богати** (rich), които изпълняват част от задачата.

По отношение на сървърите съществуват основно **два вида сървъри** по отношение на изпълнението на асинхронните клиентски заявки:

- **Итеративни сървъри** – сървърният процес е един за всички клиенти и обслужва последователно клиентските заявки, като обикновено функционалността изисква връщане на резултат към клиента; при необходимост сървърът буферира (т.е. съхранява в опашка) постъпващите асинхронни клиентски заявки, докато завърши изпълнението на текущата заявка, което е съществен недостатък
- **Конкурентни сървъри** – сървърният процес приема заявки от множество клиенти и ги предава веднага за изпълнение на друга нишка (или процес), която той стартира специално за съответната заявка

Двуслойните архитектури са подходящи за бързо проектиране на клиент-сървър приложения с ограничена функционалност и без високи нефункционални изисквания. По отношение на производителност двуслойните приложения са ефективни за групови приложения типично в интранет и до няколко десетки потребители/клиенти. Мащабируемостта на такива приложения е ограничена. По функционалност те са приложими предимно за СУБД-базирани приложения, което е свързано с ограничена преносимост при смяна на сървърите.



Трислойният модел въвежда междинен слой (middleware) между клиентския потребителски интерфейс и сървърния контролер на базата от данни. Предимствата на приложението на междинен слой са свързани с по-голямата асинхронност и независимост между клиентския и сървърния процес, ако се поддържа буфериране на съобщенията заявки и на резултата. Същевременно междинният слой дава възможност за планиране на сървърното обслужване на клиентските заявки по време, приоритет, ресурсна обезпеченост и др. Затова трислойните приложения са ефективни и при групово обслужване на хиляди клиентски процеси. Позволяват по-голяма преносимост и модулност.

Трислойната йерархия на модела клиент-сървър включва:

- **Слой на потребителския интерфейс** – обслужва предимно графични и други интерфейси
- **Слой на обработката** – състои се предимно от програмната имплементация на обслужващите алгоритми
- **Слой на данните** – обслужва достъпа до ползваните данни

Трите слоя освен като *клиент-мидълуеър-сървър* се означават също и като *представителен, логически и контекстен.*

Многослойна (n-tier) софтуерна архитектура представлява разделяне на логическия междинен слой по отношение на функции, технологии или друг признак. Можем да си представим, че слоевете са вертикално подредени и компонентите на даден слой са клиенти на сървъри от съседния долен слой.

При n-слойните архитектури се срещат многослойни сървъри, където сървърната част е декомпозирана на поне два слоя, които се класифицират като:

- **Междинни слоеве** – намират се между интерфейсния и вътрешния слой; имплементира бизнес логиката
- **Вътрешен слой** – обикновено най-долния слой; за достъпа до данни или комуникации

Брокерът е инстанция на междинния слой, която поддържа достъпа до и обработката на множество услуги или алтернативни услуги или външни услуги. Позволява и услугите да се класифицират по своите функционални и нефункционални атрибути.

Peer-to-peer (p2p) архитектурата се разгръща като наслоена мрежа (overlay network) върху съществуващата клиентска част от мрежова инфраструктура и поддържа нейерархичен модел на обмен върху йерархичния IP. Всеки участник е еднакво привилегирован и заделя част от ресурсите си, която да се използва от останалите участници в мрежата без нуждата от каквато и да е централна координация. За разлика от класическата архитектура клиент/сървър, при p2p всеки участник изпълнява функцията и на клиент, и на сървър за другите участници. p2p се реализира чрез децентрализирана топология, самоорганизация, инцидентна (ad hoc) и динамична свързаност и наличност на възлите и процесите в тях, скалируемост, отказоустойчивост (fault tolerance), анонимност на споделянето и специална юридическа регулация.

Типовете наслоени мрежи (overlay networks) за безсървърно масово обслужване са:

- **Неструктурирани** – топологията на свързване е произволна, както и производителността на комуникационните канали; често се случва възли да се включват и отпадат; в резултат, общата производителност е също произволна
- **Структурирани** – топологията на свързване е изрично зададена, което дава гаранции и за бързодействието; съществуват няколко топологии, като типично са кръг, хорда, дърво, верига
- **Хибридни/йерархични** – ниските нива са структурирани, като всяко ниско ниво играе като супервъзел в неструктурирана мрежа

Сървъри за приложения и Web-сървъри. Метасистеми и грид. Сервизно-, моделно- и аспектно-ориентирани архитектури. Софтуерни агенти

Сървърите за приложения обслужват интерфейсните клиенти обикновено върху HTTP – като уеб сървъри, но се различават от типичните уеб сървъри по преобладаващата генерация на динамично съдържание и интензивни операции върху контекста в базите от данни. Сървърите за приложения са достъпни чрез платформено-независими интерфейси (API). Клиентите се проектират само като интерфейс към вградените интерфейсни инструкции, интерпретирани от сървъра.

Уеб сървърите са специализиран вариант на сървъри за приложения, които обслужват HTTP заявки от специализирани клиенти – уеб браузъри и връщат HTTP отговор. Те обслужват стандартно порт 80 на сървърната платформа, а за защитен обмен с HTTPS заявки портът е 443. Извлеченото или генерирано съдържание е предназначено за интерпретация от браузър, така че обикновено е текст на HTML, но може да съдържа изображения или друг документален общ тип файл, дефиниран съгласно MIME. Уеб сървърите записват последователните несвързани заявки на един клиент в log (дневник).

Метасистемите осигуряват достъп до хетерогенни разпределени ресурси. Те са най-ранната форма на обединяване на ресурси за изпълнение на комплексни задачи или за оптимизиране на приложението на дадени разпределени ресурси. Типичният модел на обслужване с метасистема е оторизираният потребител да изпрати прозрачно своето задание към виртуална разпределена система, която от потребителска гледна точка е виртуален уникомпютър. Основният модел на обслужване е пакетната обработка с планиране на заданията според критерии за оптимално натоварване на ресурсите (възлите). В резултат потребителят обикновено може да зададе срок за изпълнение на своето задание. Това е най-ранният, най-зрелият и най-лесен (от гледна точка на системно осигуряване) модел за имплементация на

разпределено обслужване. Метасистемите се прилагат предимно в интранет системи, отколкото в голяма мрежа като WAN.

Пример: *Condor – свободно разпространявана система, разработена от Университета на Уисконсин, приложима не само за академични, но и бизнес организации*

Гридът представлява разслоена програмна среда за достъп до услуги, базирани върху разпределени информационни, програмни, апаратни и комуникационни ресурси, които координирано (т.е. с договорени компоненти) са споделени между няколко организации, федерирани в обща виртуална организация.

При **сервизно-ориентираната архитектура** (**Service Oriented Architecture, SOA**) бизнес и системните функции се поддържат от дискретни изолирани процеси, дефинирани съгласно съответния платформен стандарт за услуги – най-често уеб услуги, но и др. Услугите в SOA са с функционална спецификация – т.е. покриват функция или група от функции прозрачно от инфраструктурата; грануларността им не се стандартизира – приложими са стандартни и комплексни услуги. Услугите поддържат изисквания за публичност и откриваемост (advertising), която обикновено се базира на публични директории на услугите – по този начин приложенията стават напълно прозрачни от конкретната грид инфраструктура. Обменът е чрез стандартни протоколи – SOAP и WSDL.

Моделно-базираните архитектури (**Model Driven Architecture, MDA**) са подход на проектиране на разпределената софтуерна архитектура, съобразен с принципите на моделното проектиране. На свой ред MDA представлява допълнение или средство за автоматизирано проектиране на приложения, използващо някои от обичайните архитектурни подходи като ОО, COTS продукти или SOA, но по начин, съобразен с изискванията на софтуерното инженерство. Детайлността на модела предопределя използването на автоматизирано проектиране – например широко се прилага формално специфициране с UML. Моделите допускат прилагането на индустриални стандарти, както и въвеждане на стандартизация в моделирането. При моделно базираните архитектури вместо анализ за грануларност и функционална декомпозиция на конкретно информационно приложение може да приложим шаблон, чрез който ползваме частично готови решения, а освен това отделните услуги се проектират от звена или разработчици със специализирана компетентност. Автоматичната генерация на код при тях може да се съчетае и с прилагане на различни други проектни подходи – най-често с използване на програмни шаблони (design patterns).

Аспектно-базираните архитектури (**Aspect Oriented System Architecture, AOSA**) са структурирани в съответствие с принципите на аспектно базираното проектиране. Базират се на т.нар. **аспекти**. Аспекти на разпределеното приложение са общи функционалности, които се прилагат от няколко от специализираните бизнес модули – например аспектите дневник (log), оторизиране, защита и други такива. Аспектно базираните архитектури се фокусират върху идентифицирането, локализацията и спецификацията на аспектите в явен вид на етапа на проектиране на софтуерната архитектура.

Софтуерните агенти (**software agents**) са модел и технология за развитие на информационни услуги. По същество те са специализирана софтуерна компонента, която разполага с автономност, осигурявайки оперативно съвместим интерфейс на произволна система, държейки се като човек в осъществяването на дадена задача. Изграждат се на базата на специализирани среда за поддържане на агенти – агентна платформа, която предоставя основните свойства на агентите като междинни приложно ориентирани системни услуги. Основните свойства на процесите агенти са комуникативност, реактивност и автономност.

Държавен изпит – спец. „Софтуерно инженерство“

Тема №18 – Модели и методи за проектиране на потребителски интерфейс.

Анотация: *Изложението по въпроса трябва да включва следните по-съществени елементи:*

1. Основни модели и методи при създаване на потребителски интерфейс: подходи, основни процеси, анализ на задачите, специфициране на взаимодействията, основни техники и инструментални средства.

2. Проектиране на графичен интерфейс: интерактивни стилове и техники, отчитане на психологичните особености на потребителите, концептуални модели и метафори, методи и средства за реализация.

3. Разработка на използваем графичен интерфейс: техники базирани на експерименти.

4. Разработка на мултимедиен графичен интерфейс: проектиране на цветове, звуци, текст, графика, анимация.

5. Особености при създаване на интегриран интерфейс: методи за моделиране насочени към крайния потребител, екранен дизайн, обработка на взаимодействията, интерактивни методи за проектиране, прототипиране.

Основни модели и методи при създаване на потребителски интерфейс: подходи, основни процеси, анализ на задачите, специфициране на взаимодействията, основни техники и инструментални средства

Проектирането на човеко-машинен интерфейс е интердисциплинарна дисциплина, която се занимава с проектиране, създаване и оценяване на интерактивни компютърни системи, предназначени за използване от хора, изучавайки всички основни процеси и явления, влияещи на тези дейности. Основен фокус се поставя върху диалога и начина на взаимодействие, а не върху самите входно-изходни устройства, което означава да се изследват и човешките фактори – възприятие, психологически основни на диалога и др.

Потребителският интерфейс е средство, чрез което потребителят и системата, в частност компютърната система, си взаимодействат помежду си. Нарича се още **човеко-машинен интерфейс**. Включва в себе си както хардуерни, така и софтуерни компоненти. Добрият потребителски интерфейс позволява на потребителя да взаимодейства със софтуера и хардуера по естествен и интуитивен начин.

Основните модели при създаването на потребителски интерфейс са познавателни и са:

- **GOMS (Goals, Operators, Methods, and Selection)** – семейство от модели за предсказване на човешкото поведение, което може да се използва за повишаване на ефективността на взаимодействие между човек и компютър чрез идентифициране и елиминиране на ненужни и неефикасни човешки дейности. С GOMS се моделират задачи, ориентирани към постигане на цел, като те трябва да са обикновени познавателни дейности. Моделират се както последователни, така и паралелни задачи
- **KLM-GOMS (Keystroke-Level-Model-GOMS)** – предсказва времето за изпълнение на различни дейности от човек чрез просто множество оператори за физически и умствени действия като натискане на клавиш, преместване на мишка, разбиране на ситуация и време за избор на действие. Всеки KLM оператор е свързан с време за изпълнение на

базата на емпирични данни. За изчисляване общото време на действие се сумират времената на отделните прости оператори. Всяко действие, свързано с вземане на решение, се моделира чрез последователности от прости познавателни стъпки, които се изпълняват една след друга. Тези последователни стъпки представляват елементарни познавателни дейности, които не могат да бъдат декомпозирани на още по-прости оператори

- **Методи за анализ на действията (Task Analysis Methods)**

Основните методи за създаване на интерфейси са:

- **Сценарии/истории (storyboarding)** – среда; последователност; удовлетвореност
- **Хартиени прототипи** – интерактивен макет на хартия; естествено взаимодействие; човек симулира работата на компютъра; слаба прецизност на изглед и възприятие; прецизно в дълбочина
- **Компютърни прототипи** – интерактивна софтуерна симулация; висока прецизност в показване и възприемане; ниска прецизност в дълбочина
- **Смесени прототипи**

Основните процеси при създаване на потребителски интерфейс могат да се обобщят в следните дейности:

1. **Идентифициране на нуждите и определяне на изискванията** – тази дейност изисква анализ на задачите, като се отчита контекста на всяка задача. На всяка стъпка се идентифицира нужната информация, както и поредността и зависимостта между стъпките и задачите. Анализират се основните роли и се дефинира базово множество от сценарии
2. **Създаване на множество от алтернативни интерфейсни решения** – зависи от творческите решения и изобретателността на проектанта, като позволява прилагане на алтернативни начини на взаимодействие. Резултатите са силно зависими от предишен опит и намиране на сходни ситуации
3. **Изграждане на интерактивни интерфейсни решения** – включва използването на познавателни методи, които анализират човешкото поведение и помагат за увеличаването на ефективността при взаимодействието между човек и компютър
4. **Оценяване на интерфейсните решения** – на тази стъпка е необходима оценка от потребители, експерти, използване на прототипи и симулационни модели. Отчита се безопасност, полезност, ефективност, ефикасност

Анализът на задачите включва:

- Анализ и описание как хората изпълняват своята работа/дейности
- Определяне какво хората искат от интерфейса и как ще го използват
- Фокус върху ресурсите от средата на работа, а не от компютърния интерфейс
- Моделиране чрез дейности, обекти/ресурси, връзки и релации

Основните методи за анализ на задачите (task analysis methods) включват:

- **Task Analysis** – простите действия се разглеждат като черни кутии
- **Cognitive Task Analysis** – анализира действия, изискващи значителна умствена дейност от потребителите, като взимане на решение, решаване на проблеми, работа с паметта, внимание и преценка. **Cognitive Walkthrough** е един от начините да проверим дали е използваем проектирания от нас интерфейс чрез дефиниране на контекста (определяне на потребители, задачи, описание) и организиране на екип (за анализ и изготвяне на сценарий)

- **Hierarchical Task Analysis** – всички действия са представени като йерархия, като така по-сложните се представят като съвкупност от по-прости

Основни техники за събиране на необходими данни са:

- **Въпросници** – за извличане на специфична информация от потребителите
- **Интервюта** – за установяване удобствата и предизвикателствата при използване на интерфейса
- **Фокус групи и работилници**
- **Наблюдения** – прекарване на известно време с потребителите с цел наблюдение
- **Автоматично събрани компютърни данни**

Взаимодействието между потребителя и интерфейса може да бъде специфицирано чрез средства като:

- BNF (Backus Naur Form) граматики и диаграми
- Многозначни граматики
- Дървета от диалогови кутии
- Избор на менюта
- Диаграми за преход
- Графи на състоянията

Категориите софтуерни средства за спецификация на потребителските взаимодействия се категоризират на:

- **Средства с общо предназначение** – PowerPoint, Visio
- **Средства за спецификация на софтуерни системи** – базирани на UML и подобни
- **Специфични системи за интерфейси** – LucidChart
- **Системи за прототипиране** – MyBalsamiq

Основни инструментални средства за конструиране на интерфейс:

- Визуални графични среди за програмиране
- Визуални интерфейсни езиково-независими библиотеки
- Графични редактори
- Специализирани софтуерни системи за проектиране

Сред основните характеристики на средствата за създаване на интерфейси са, че създаваните елементи от потребителския интерфейс са независими от приложните програми. Базирани са основно на метода за бързо създаване на прототипи. Съществуват явни средства и начини за свързване на интерфейса към останалата част на софтуерната система.

Проектиране на графичен интерфейс: интерактивни стилове и техники, отчитане на психологичните особености на потребителите, концептуални модели и метафори, методи и средства за реализация

Графичният потребителски интерфейс (Graphical User Interface – GUI) предоставя на потребителя набор от различни графични средства и компоненти за взаимодействие със системата. Различните компоненти на графичния интерфейс представят различни аспекти от взаимодействието между потребителя и системата. Изграждането на графичния интерфейс се базира на редица правила и принципи.

Процесът на дизайн на потребителски интерфейс се състои от следните дейности:

1. Анализ на потребностите
2. Анализ на аудиторията
3. Анализ на задачата
4. Софтуерни изисквания
5. Дизайн на потребителския интерфейс
6. Имплементация
7. Тестване

Сред основните принципи при дизайн на графичен интерфейс са:

- **Единство** – език, подбор и групиране на елементи, стил и визуално оформление, функционална последователност
- **Стандарти и познати модели** – следване на добре установени, доказани практики, стандарти, конвенции и познати модели
- **Визуална йерархия** – организиране на сходни функционалности в групи; най-важните елементи на по-високо ниво; показване само на основните функционалности на основния екран; сходно оформление на елементи с еднаква важност
- **Превенция срещу грешки** – ясни и подробни инструкции, които предотвратяват появата на грешки; ясни съобщения при грешка
- **Обратна връзка** – ясна, информативна; визуални сигнали; предоставяне на помощ
- **Простота** – без ненужни елементи; ясна и проста комуникация
- **Достъпност** – минимален общ брой на действията, необходими на потребителя за постигането на дадена цел; правилото с „три щраквания на мишката“; достъп за потребителите с различни способности; подобряване на достъпността

Формите на взаимодействие на потребителя с графичния потребителски интерфейс могат да се класифицират във следните интерактивни стилове на взаимодействие:

1. **Директно манипулиране (direct manipulation)** – потребителят директно работи с обектите на екрана
2. **Избор от меню (menu selection)** – потребителят избира команда от списък с възможности наречен меню
3. **Попълване на форми (form fill-in)** – потребителят попълва данни във форма
4. **Команден език (command language**) – потребителят въвежда команди и техни параметри, за да укаже на системата какво да извърши
5. **Естествен език (natural language)** – потребителят използва естествен език

Психологичните особености на потребители са от голямо значение за проектирането на графични интерфейси. В тази връзка идва и понятието **концептуален модел**, което се дефинира като „описание на високо ниво за това как системата е организирана и оперира“. То определя какво ни е нужно, за да разберем как да взаимодействаме с дадено нещо. Казано накратко, концептуалният модел предоставя работеща стратегия и платформа за общи концепции и техните взаимоотношения.

Сред основните средства и начини за прилагане на принципите на графичния дизайн са:

- **Визуално оформление (графичен дизайн)**
  - Пространство и отношения между елементи на страницата
  - Използване на цвят, контраст, текстура
  - Типография

- **Интерфейсни метафори**
  - Сравнения и асоциации, чрез които непознати за човек идеи се описват чрез неща от реалния свят, с цел по-лесното им разбиране
  - Връзка между интерфейса на приложението и реалния живот
  - Правят интерфейса по-интуитивен и лесен за разбиране
  - Използват се икони и други елементи, напомнящи обекти от реалния живот
- **Мнемоника и асоциации**
  - Техника за запаметяване, свързана със запомняне на максимален обем информация чрез създаване на изкуствени асоциации
  - Основава се на схващането, че ако два елемента се срещнат заедно, всяка следваща среща на единия ще предполага връзка с другия
  - Това означава, че ако възприемем два обекта като свързани, ще имаме нужда да си спомним само единия, за да възпроизведем в паметта си и другия

Разработка на използваем графичен интерфейс: техники базирани на експерименти

За да е използваем един графичен интерфейс, то е необходимо да си отговорим на няколко въпроса:

- Колко добре продуктът изпълнява функциите си? (**ефективност**)
- Доколко продуктът подпомага потребителя в изпълнението на неговите задачи? (**експедитивност**)
- Какви грешки са възможни при използване на продукта и какви мерки са налични за възстановяване от тях? (**безопасност**)
- Продуктът предоставя ли нужната функционалност? (**удобство**)
- Колко лесно е да се запомни как се работи с продукта, след като потребителя се научи? (**лесен за научаване, запомняне, възприемчивост**)

Всички тези измерения трябва да бъдат взети предвид при определяне степента на използваемост на даден графичен интерфейс. По отношение на техниките за оценка на постигнатото, широко разпространени са тези, **базирани на експерименти** с типични за дадената система потребители в специализирани лаборатории. Участниците са представители на планираните потребители на системата със същите опит в използването на ИТ, опит в решаване на задачи, мотивация, образование, разбиране на естествения език, използван в интерфейса.

Видео заснемането на участниците е удобно за по-късен анализ и показване на проблемите на дизайнерите и ръководството. Участниците се приканват да коментират на глас, докато работят с интерфейса. Друг начин за събиране на информация за оценка е директното наблюдение на потребители на живо. Често срещани е използването на въпросници и анкети в края на експеримента, които позволяват да се събере информация за това какво мислят потребителите за интерфейса.

Разработка на мултимедиен графичен интерфейс: проектиране на цветове, звуци, текст, графика, анимация

Мултимедийният графичен интерфейс съчетава в себе си цветове, звуци, текст, графика, анимация и др. **Цветовете** привличат вниманието. Цветът е важен елемент на богатата визуална обратна връзка и постоянното му използване за подчертаване на важна информация осигурява важен канал на комуникация. Цветовете подобряват навигацията и скоростта на

търсене, както и взаимовръзките. Те трябва да се прилагат последователно без изменящо се значение. При проектирането на графичен интерфейс е по-добре да се избягва използването на цвят като единствен начин за предаване на информация, тъй каро хората възприемат цвета по-различен начин.

Има няколко начина за представяне на цвят, по-известни като **цветови модели**. Техен интензитет наричаме нюанса, с който всеки от основните цветове участва в изграждането на нов цвят. Основните цветови модели са:

- **RGB** – червено (red), зелено (green), синьо (blue), където интензивността на всеки цвят се задава с число $n \in [0,255]$; използва се най-вече в компютърните монитори и телевизори
- **CMYK** – циан (cyan), магента (magenta), жълто (yellow), черно (black), като интензивността им се измерва в проценти; използва се най-вече при принтиране

Цветовите схеми са систематичен подход за комбиниране на цветове в зависимост от разположението им на цветовия кръг. Те могат да бъдат монохромни (monochromatic), противоположни (complementary), триади (triads), тетри (square), аналогични (analogous) и др.

Звукът има различни проявления – като отделни звуци, говор и като музика. Интегрира се в софтуерни системи чрез т.нар. звукови формати като *.mp3, *.wav, *.wma и др. Придава емоционални въздействия, обогатява с ефекти елементите на мултимедията и дава възможност за комуникация с потребителя. Той е основна част от мултимедията.

Текстът е един от най-лесните начини, по който бързо може да се предаде подробна и обемиста информация. По отношение на проектирането на текст съществуват следните препоръки:

- Да се осигури висок контраст на текста във фона и да не се използват конфликтни цветове, които могат да влошат четимостта
- Да се изберат подходящи шрифтове и размер на шрифта. Размер, който е по-малък от 10 пункта, се чете трудно. За сбит текст като заглавие или кратка инструкция, е подходящ ясно очертан несерифен шрифт (напр. Arial), а за параграфи е възможно използването на serif (напр. Times New Roman)

Сред по-известните текстови кодирания са:

- **ASCII** – представяне на всяка буква чрез 1 байт
- **UTF-8** – стандарт за символно кодиране; съдържа много повече символи от ASCII; кодирането е с променлива ширина и използва 8-битови кодови единици; всеки символ заема различен брой байтове (от 1 до 4 байта), като латинските букви например заемат по 1 байт, а тези на кирилица – по 2 байта

Основните типове **компютърна графика** (изображения) се определят според типа на построяване на изображението и се категоризират на:

- **Растерна графика** – изображението е съставено от точки (малки квадратчета), оцветени по различен начин, наречени пиксели (съкратено от picture elements). Всички те са подредени в правоъгълна решетка, наречена растер, откъдето произхожда и наименованието на този тип графика. Пример за растерни изображения са изображенията, получени чрез сканиране и дигитален фотоапарат и видеокамера. Сред основните им предимства е, че конвертирането от един формат в друг става лесно, а сред основните им недостатъци е т.нар. пикселизация – при увеличаване на размера, качеството на изображението се нарушава, а отделните пиксели стават все по-ярко

изразени. Растерната графика намира широко приложение в обработката на готови цифрови изображения. Известни файлови формати са *.jpg, *.png, *.gif и др.

- **Векторна графика** – изображението е съставено от множество обекти, изградени от вектори. Основният елемент изграждащ обектите е права или крива линия. Тя се представя с помощта на формула, а не като съвкупност от точки, както е при растерната графика. Визуализирането на даден обект на екрана става като програмата първо изчисли по формулата неговия вид и след това го представя. Много често векторната графика се нарича обектно ориентирана графика, тъй като в процеса на работа по-често се работи със самите обекти, а не толкова с линиите, от които са изградени. Основното предимство на този тип графика е, че при промяна на размера на изображението неговото качество се запазва. Причината за това е, че при извършването на такова действие формулите, на базата на които се изгражда изображението, се преизчисляват автоматично с нови параметри. Сред недостатъците е необходимостта от специални програми за визуализиране и редактиране. Основното има приложение е за създаване за изображения, а не толкова за тяхната обработка. Известни файлови формати са *.svg, *.eps, *.wmf и др.
- **Фрактална графика** – изображението се конструира от уравнения или система от уравнения. При промяна на коефициентите на уравнението се променя видът на изображението. Основен принцип е наследяването

Анимацията представлява движещи се образи, които показват състоянието на обекти под формата на графики в различни моменти от времето. Често се използват за визуализиране на неща, които се илюстрират трудно по друг начин. Широко използвани файлови формати за анимация са *.svg, *.gif, *.mp4 и др.

Особености при създаване на интегриран интерфейс: методи за моделиране, насочени към крайния потребител, екранен дизайн, обработка на взаимодействията, интерактивни методи за проектиране, прототипиране

При създаването на **интегриран интерфейс** основно внимание трябва да се обръща на потребителите и техните нужди, а не просто технологиите. Именно затова трябва да се използват **методи, ориентирани към събиране на информация от крайния потребител**:

- **Етнография** – наблюдение на потребителите, докато извършват ежедневните си дейности
- **Метод на свързаността** – интегрира етнографския подход с обектно-ориентирания анализ от софтуерното инженерство
- **Метод на контекстния дизайн** – цели да улесни събирането и интерпретацията на информацията от дадена област, така че да се изпълни подходящ за нея софтуерен продукт
- **Метод на участието** – за разлика от контекстния метод, потребителите участват активно в разработката, като част от екипа на дизайнерите

При разглеждане на **екранния дизайн** основна роля заема дизайна на иконите върху екранните интерфейси:

- Избраната икона трябва да е ясно видима и лесно да се разграничава, когато около нея има неизбрани икони
- Всяка икона трябва да е лесно различима

- Всички икони трябва да образуват хармонично семейство от сходни, но различими обекти
- Избягване на излишните детайли при съставянето на иконите

Един добре изграден интерфейс трябва да поддържа **обработка на взаимодействията** с потребителя, като предлага:

- Ясна и подробна информация, но не повече от нужната
- Еднозначни и лесни указания
- Смислени и информативни съобщения за грешки
- Лесен достъп до допълнителна информация и указания

Интерактивните методи включват:

- Разбиране на нивото (опита) на потребителите
- Определяне на задачите – провеждане на наблюдения и разговори
- Избиране на стил на взаимодействие – инструкции, диалог, манипулиране, откриване
- Освобождаване на потребителя
- Съчетаване на автоматичност и управление на потребителя – адаптивни интерфейси

Софтуерното прототипиране е динамично симулиране. То представлява модел, създаден за да се демонстрира новият продукт. Потребителите могат да изпробват операции чрез взаимодействие със системата в „реално време“, но обикновено функционалността е само частична или симулативна. Прототипът може да послужи и за оценяването на интерфейса. Процесът на прототипиране може да е на два етапа – в началото могат да се използват хартиени прототипи, а след това проектът се уточнява и се разработват все по-сложни автоматизирани прототипи.

Видове прототипиране:

- **Хартиено прототипиране** – за възможно най-ранна обратна връзка
- **Компютърен прототип** – висока прецизност в показване и възприемане
- **Бързо прототипиране** – създаване на бързи представи за потребителския интерфейс на системата чрез методи като мозъчна атака, създаване на прототипи, тестване с краен потребител, оценяване и подобряване на системата на всеки етап на разработката
- **Пълно прототипиране** – функционалност на завършения продукт
- **Хоризонтално прототипиране** – потребителски интерфейс, но без функционалност, т.е. много широчина и малко дълбочина; клиент без сървър
- **Вертикално прототипиране** – цялата функционалност от ниско и високо ниво що се отнася до една ограничена част от системата, т.е. малко широчина и много дълбочина; една част от интерфейса се разработва напълно
- **Прототипиране „Магьосник от Оз“** – софтуерна симулация с помощта на човек

Държавен изпит – спец. „Софтуерно инженерство“

Тема №19 – Управление на качеството на софтуерни приложения. Тестване на софтуер.

Анотация: *Изложението по въпроса трябва да включва следните по-съществени елементи:*

1. Осигуряване на качеството – качество на софтуера, алтернативи за осигуряване на качество.

2. Тестови дейности, управление и автоматизация

3. Видове тестване - тестване с контролни списъци, тестване с класове на еквивалентност, разделяне на входния домейн и тестване на границите.

4. Нива на тестване и приложение на техниките за тестване.

5. Измерване. Метрики – характеристики и класификация. Метрики за качество на софтуерен продукт и софтуерен процес.

Примерна задача: *Дефиниране на тестови сценарии чрез прилагане на конкретна техника за тестване.*

Осигуряване на качеството – качество на софтуера, алтернативи за осигуряване на качеството

Според формулираното от ***професионалната американска асоциация на електронните инженери IEEE*** определение, **софтуерът (software)** е:

- Инструкции (компютърни програми), които при изпълнение осигуряват желаните характеристики, функциониране и производителност
- Структури от данни, които дават възможност на програмите адекватно да обработват информация
- Документи, които описват работата и използването на програмите

Според ***професионалната американска асоциация на електронните инженери IEEE*** определение, **качество на софтуера (software quality)** е:

- Степента, до която система, компонент или софтуерен процес отговаря на специфицираните изисквания
- Степента, до която система, компонент или софтуерен процес удовлетворява нуждите или очакванията на заинтересованите лица

Осигуряване на качеството (Quality Assurance) е множество от активности, планирани с цел да открият, документират и анализират и коригират дефекти в процеси и да управляват процедурите за промяната, която се налага като резултат от тези дейности. Проследява се дали се прилагат приетите процеси за изграждане на софтуерно решение – фирмени и изисквани от клиента, в описаната последователност и взаимосвързаност.

Контрол на качеството (Quality Control) е множество от активности, планирани с цел да открият, документират, анализират и да коригират дефекти в продуктите или крайните решения (за софтуерни услуги) и да управляват процедурите за промяната, която се налага като резултат от тези дейности.

Двете понятия – осигуряване на качеството (Quality Assurance) и контрол на качеството (Quality Control) – са в **контраст** един с друг, защото имат различни цели. Контролът на

качеството се извършва след разработката, но преди продуктът да се достави на клиента, т.е. дейностите по контрол на качеството са само част от общия набор от дейности по осигуряване на качеството.

Целите за осигуряване на качеството се категоризират в две отношения:

- Софтуерна разработка (ориентирана към процеси)
- Софтуерна поддръжка (ориентирана към продукти)

Тестови дейности, управление и автоматизация

Основните тестови дейности са три:

1. **Тестово планиране и подготовка**
2. **Изпълнение на тестове и наблюдение**
3. **Анализ и проследяване**

Най-ключова и детайлна е **фазата на тестовото планиране и подготовка**. Ще разгледаме в детайлен план представени по-горе тестовите дейности:

1. **Планиране на тестването** – дефиниране на тестовата цел, фокусирайки се върху изправността на продукта като изискване за качество; решения, които се вземат при определяне на тестовата стратегия; планиране на ресурси и персонал; избор на формални методи и техники за тестване
   1.1. **Дейности по конструиране на тестов модел** – идентифициране на източник на информация и събиране на данни; анализ, конструиране и валидиране на модела
   1.2. **Генериране на тестови сценарии от модела** – необходима информация: времева информация, определяща точка на стартиране и точка на приключване, вход към софтуерния обект, очакван резултат
      1.2.1. ***Тестов сценарий (test case)*** *– колекция от елементи и свързана с тях информация, осигуряващи изпълнението на тест или тестова серия*
      1.2.2. ***Тестова серия (test run)*** *– динамична единица от специфични тестови данни в общата тестова последователност върху избран тестов обект; често в литературата тестовата серия се нарича опит (изпълнение на няколко теста)*
   1.3. Създаване и управление на тестови пакети
      1.3.1. ***Тестов пакет (test suite)*** *– колекция от отделни тестови сценарии, които се стартират в тестова последователност, докато не се удостовери определен критерий за спиране*
      1.3.2. Създаване на тестов пакет – конструиране, подреждане, преизползване
      1.3.3. Управление на тестови пакети – споделена база от данни (тестери, сценарии)
   1.4. Подготовка на тестова процедура
      1.4.1. ***Тестова процедура (test procedure)*** *– определя последователността и превключването на тестовите сценарии*
      1.4.2. Съображения за избор на тестова процедура – зависимости, дефекти и др.
      1.4.3. Дефиниране на преход между тестовите сценарии
      1.4.4. Определяне на роли и отговорности за изпълнението на тестовите сценарии
      1.4.5. Планиране на необходимото време и ресурси

2. **Изпълнение на тестовете и наблюдение**
   - **2.1. Заделяне на време и ресурси**
   - **2.2. Стартиране, изпълнение и събиране на информация от тестовете**
   - **2.3. Проверка на тестовите резултати чрез тестови оракули**
3. **Анализ и проследяване**
   - 3.1. **Анализ на индивидуалните тестови сценарии** – проверка на резултата и идентифициране на повредите
   - **3.2. Анализ на общите тестови резултати**
     - *3.2.1. Анализ на надеждността при тестване, базирано на употреба*
     - *3.2.2. Анализ на покритието при тестване, базирано на покритие*
     - *3.2.3. Общ анализ на дефектите*

По отношение на управлението ролята на изпълнителите на тестовите сценарии зависи от вида тестване. При **неформалното тестване** продуктът се стартира и се наблюдава директно поведението му като се изискват само минимални познания за продукта. При **формалното тестване** е задължително включването на множество заинтересовани лица. Изискват се специалисти с високи технически умения и опит.

Съществуват следните **видове организация и управление на екипа за тестване**:

- **Вертикален модел на организация** – организацията е около продукта, като една или повече тестови задачи се асоциират с определен изпълнител; например един или повече екипи може да изпълнява всички различни типове тестове за даден продукт (от тестване на ниво единица до тестване за приемане на системата)
- **Хоризонтален модел на организация** – за големи организации, като един тестов екип изпълнява един тип тестване за всички продукти; например различни продукти могат да споделят един и същи екип по системно тестване
- **Смесен модел на организация**
  - Ниските нива на тестване се изпълняват от екипите, отговарящи за съответните проекти
  - Системното тестване се споделя между подобни проекти
  - Съществува обща централизирана поддръжка на проекта

Външни участници в тестовия процес могат да бъдат потребители на системата или софтуерни и хардуерни среди, които си взаимодействат със системата. Могат да участват и външни организации, които да извършат независима верификация и валидация.

Целта на **тестовата автоматизация** е да освободи тестерите от досадни и повтарящи се задачи и да повиши тестовата производителност. Важни въпроси, свързани с автоматизацията са:

- *възможността за спецификация на специфични нужди*
- *избор на налични софтуерни инструменти*
- *възможност и цена за конструиране на собствен продукт*
- *наличие на обучение за използване на инструментите и необходими усилия и време*
- *крайна цена на автоматизацията*
- *влияние върху ресурси, график и управление на проекта*

Тестова дейност	Възможност за автоматизация
Тестово планиране и подготовка	
Цялостно планиране на тестовия процес	Ниска

Създаване на формални модели	Средна
Подготовка на тестови сценарии	Средна
Генериране на тестови сценарии	Висока
Планиране на тестовата процедура	Ниска
Изпълнение на тестове	
Изпълнение на тестове	Висока
Анализ и проследяване	
Анализ на надеждността	Висока
Анализ на тестовото покритие	Висока
Действия за подобряване на продукта	Ниска

Видове тестване – тестване с контролни списъци, тестване с класове на еквивалентност, разделяне на входния домейн и тестване на границите

Най-простата форма на тестване е **„ad hoc“ (случайно) тестването**, което представлява стартиране на софтуера, наблюдение на поведението и идентифициране на специфични проблеми. Често обаче този вид тестване е труден за проследяване в случай на големи софтуерни системи.

За систематизиране на „ad hoc“ тестването възниква идеята за създаване на **контролни списъци** за проследяване на изтестваните елементи. Така възниква идеята за **тестването с контролни списъци**. Тази техника може да бъде използвана както за функционално тестване (black-box testing), така и за структурно тестване (white-box testing). В резултат на това често срещани списъци са функционални контролни списъци (с различно ниво на абстракция и грануларност) и контролни списъци със системни елементи (с различно ниво на грануларност: подсистеми, модули, изрази, елементи с данни и др.) Условието за спиране на тестването е изчерпване на елементите в списъка.

Съществуват следните типове контролни списъци:

- **Базови списъци** – включват набор от елементи, които трябва да се тестват
- **Йерархични списъци** – списъците са разделени на нива, като елементите от по-горните нива съдържат списъци от по-ниските нива
- **Комбинирани (многомерни) списъци** – списъците са многомерни, като всеки контролен списък се обхожда за всички елементи от останалите контролни списъци
- **Смесени списъци** – съчетават йерархичните и комбинираните контролни списъци

При контролните списъци се срещат следните проблеми и ограничения:

- Трудности при покриване на всички функционални или структурни компоненти от различни гледни точки и различни нива на грануларност
- Опасност от припокриване на елементи в различните контролни списъци
- Трудности при описание на възможни взаимодействия между компоненти на системата

Тестването с класове на еквивалентност се основава на простото предположение, че различните неща, които бихме искали да тестваме, могат да бъдат разделени на подгрупи от класове на еквивалентност. Предимствата на тестването с класове на еквивалентност е, че те осигуряват по-добро покритие чрез изчерпателност и липса на препокриване, т.е. се намалява значително броят на необходимите тестове, покриващи различните сценарии.

Пример 1: Система с булеви входни променливи *– броят на входните комбинации е* 2^n *за* n *на брой променливи*

Пример 2: Система, изчисляваща квадратно уравнение *– разделяне на входния домейн и дефиниране на тестови сценарии посредством използването на зависимостта между корена* r *и дискриминантата* d

TC	**Условие** $\mathbf{d = b^2 - 4ac}$	**Вход**		
		a	**b**	**c**
1	$d > 0$	1	2	-1
2	$d = 0$	1	2	1
3	$d < 0$	1	2	3

Домейните аналогично наподобяват контролен списък, а неговите елементи са поддомейните.

Формално, **разбиване на множество** $\boldsymbol{S}$ е разделяне на множество на колекции от подмножества $G_1, G_2, \dots, G_n$, които удовлетворяват следните свойства:

- Подмножествата $G_1, \dots, G_n$ са **взаимно изключващи се**, т.е. не споделят общи елементи

$$\forall i, j, i \neq j \Rightarrow G_i \cap G_j = \emptyset$$

- Подмножествата $G_1, \dots, G_n$ са **взаимно изчерпателни**, т.е. покриват всички елементи на оригиналното множество

$$\bigcup_{i=1}^{n} G_i = S$$

Всяко от тези подмножества в едно разбиване се нарича **клас на еквивалентност**, като отделните елементи на тези подмножества са еднакви от гледна точка на специфично свойство или релация, използвани за дефинирането му. Релациите между елементите в класовете на еквивалентност са рефлексивни, симетрични и транзитивни.

Тестването с класове на еквивалентност може да се раздели на следните стъпки:

1. Дефиниране на класове на еквивалентност
2. Избор на един тестов сценарий за всеки клас на еквивалентност
3. Постигане на пълно покритие на класовете на еквивалентност

Типовете разделяне на класове на еквивалентност са:

- **Базирано на софтуерни елементи** – външни функции или системни компоненти
  Пример: $x \in S$

- **Базирано на определени свойства, релации, логически условия** – директно използване на логически предикати, сравнение на числови стойности
  Пример: *AND, OR, NOT;* $\{x: x < 0\}$, $\{x: x \geq 0\}$
- **Комбинации от представените типове**
  Пример*:* $(x \in I) \wedge (x \geq 0) \wedge (x < 21)$

Дърветата и таблиците за вземане на решения могат да представят последователност от решения, които се взимат по време на изпълнение на програмата. Могат да се разгледат като йерархичен контролен списък. Прилагат се за тестване, базирано на решения и предикати.

Тестването с разделяне на входния домейн и тестване на границите са следваща стъпка по отношение на софтуерното тестване. Генериране на тестови сценарии посредством присвояване на специфични стойности на входните променливи се прави чрез анализ на входния домейн. Тестване с разделяне на входния домейн наричаме тестване с покриване на малък брой входни ситуации посредством систематичен избор на определени входни стойности. Характеризира се с:

- **Тестване на входно/изходните зависимости** – осигуряване на стойности за всички входни променливи
- **Изходните променливи не се специфицират експлицитно** – допуска се, че има начин да се направи проверка за очаквания изход от всеки вход, ако се налага,
- Прилага се както за **функционално**, така и за **структурно тестване**

Основните стъпки при тестването с разделяне на домейна са:

1. **Идентифициране** на входната променлива, входния вектор, входното пространство и дефиниране на входния домейн, базирани на спецификации или имплементационни детайли за програмата
2. **Разделяне** на входния домейн на поддомейни
3. **Анализ на поддомейните** с цел определяне на границите им по всички измерения
4. Избор на всички тестови **точки** (сценарии), които покриват поддомейните
5. **Тестване с избраните тестови**, проверка на резултатите, решаване на проблеми и анализиране на резултатите

Проблемите при тестването с разделяне на входния домейн са:

- **Неопределеност на даден вход** – тестваната програма не обработва някои входни стойности

Причина: *липса на изчислителни процедури за определени поддомейни от общия вход домейн („дупки“ във входния домейн)*

- **Противоречивост за даден вход** – повреда в системата или производство на различни изходи при един и същи вход

Причина: *дефиниране на изчислителни процедури за поддомейни, които се припокриват*

Проблемите при определянето на границите са:

- **Проблем със затвореността на границите** – отворена граница се специфицира или реализира като затворена
- **Изместване на границата** – разминаване между очакваната и действително специфицирана или реализирана граница
- **Изкривяване на граница** – промяна на някои от параметрите на границата
- **Липсваща граница** – сливане на два различни поддомейна в общ поддомейн

- **Излишна граница** – некоректно разделяне допълнително на поддомейн

Стратегии, които решават част или всички от горните проблеми са:

- **Тестване на екстремните точки** – $min_i, max_i, under_i, over_i, interior$

Брой тестови сценарии: $4^n + 1$

- **Weak *N* x 1** – избор на N "ON" точки и 1 "OFF" точка

Брой тестови точки: $(n + 1)\ x\ b + 1$ *за всеки домейн с* b *на брой граници*

- **Weak 1 x 1** – избор на 1 "ON" точка за всяка граница и 1 "OFF" точка

Брой тестови сценарии: $2b + 1$ *за всеки домейн с* b *на брой граници*

Съществуват и други техники за тестване, които се използват за моделиране на по-сложни връзки като **тестване с машина на крайните състояния (автомат)**, **тестване на управляващия поток, данновите зависимости и взаимодействията** и др.

Нива на тестване и приложение на техниките за тестване

Ще разгледаме **нивата на тестване** по обикновено обичайния ред от долу нагоре на V-модела и техниките за тестване, които могат да се приложат на тези нива:

1. **Тестване на ниво програмна единица (Unit testing)** – тестване на малки програмни единици, най-често от програмистите, които ги реализират. Използва предимно техники за тестване по метода на бялата кутия с ниво на покритие като критерий за спиране. Възможно е и прилагане на техники за тестване по метода на черната кутия, но се използва рядко поради наличието на информация за начина, по който програмната единица е реализирана.

Най-често използваните техники за тестване при тестването на ниво единица са:

1.1. **"Ad-hoc" тестване или неформално дебъгване** – изпълнение в изолация; използване на специфична тестова среда (testbed) при наличие на зависимост от друга програмна единица

1.2. Разделяне на входния домейн и тестване на границите

1.3. **Тестване на управляващия поток (CFT) и тестване на данновия поток (DFT)** – типично структурно базирани

2. **Компонентно тестване (Component testing)** – тестване на софтуерни компоненти от малка група разработчици. Компонентното тестване при структурното програмиране наподобява тестването на ниво програмна единица, тъй като там понятието за компонент не е силно изразено. В обектно-ориентираните системи ролята на компоненти играят самите обекти и класове, които се тестват като едно цяло.

Поради енкапсулацията по-резонно е да доминира **тестването по метода на черната кутия** и тестването, базирано на употреба, тъй като скриването на информация е един от принципите на обектно-ориентираното програмиране, като той не налага да се знаят подробности за конкретната имплементация. Въпреки това съществува и възможност за **тестване по метода на бялата кутия** със същите структурни техники, които можеха да се приложат при тестването на ниво програмна единица.

3. **Интеграционно тестване (Integration testing)** – тестване на интерфейсите и взаимодействието на компонентите при интегрирането им. Тук отделните компоненти се разглеждат като атомарни единици или като черни кутии, от които само поведението е

важно за нас. От друга страна, интерес за нас представляват връзките между тях, което е причина и за по-честото използване на **техники за тестване по метода на бялата кутия**. Управлението на изпълнението може да се моделира с **краен автомат**, което позволява използването на **техниките, базирани на този модел (тестване на управляващия поток и тестване на данновите зависимости)**. Обикновено този вид тестване и останалите след него се изпълняват от професионални тестери. Възможно е и прилагане на техники за тестване по метода на черната кутия, разглеждайки системата като едно цяло, аналогично на системното тестване. В някои организации интеграционното тестване се счита като част от системното тестване.

4. **Системно тестване (System testing)** – тестване на външните системни операции като цяло, обикновено от гледна точка на клиента. Тетовия екип е необходимо да знае всичко за продуктовите функции, приложния домейн, пазарния сегмент и потребителските очаквания.

Прилагат се следните техники за тестване:

4.1. **Функционални контролни списъци от високо ниво** – тестване, базирано на покритие; статистическо тестване, базирано на употреба
4.2. **Техники за тестване с краен автомат** – отделните състояния представляват главните функции, които са очаквани и видими за крайния потребител, т.е. крайния автомат се използва като метод за тестване по метода на черната кутия
4.3. **Статистическо тестване, базирано на употреба** – чрез оперативен профил или модел на Марков

5. **Тестване за приемане на системата (Acceptance testing)** – тества се готовността на продукта за доставяне до крайните потребители. Разглеждат се въпроси, свързани с надеждността и нивото на поддръжка на продукта. Понякога тестването за приемане на системата може да се разглежда като част от системното тестване.

Прилагат се следните техники за тестове:

5.1. Статистическо тестване, базирано на употреба
5.2. Приложение на оперативните профили на Муса и Марков

6. **Бета тестване (Beta testing)** – тестване при контролирана и ограничена доставка на продукта до крайните потребители. Прилага се основно за приложения с широко разпространение, при които е невъзможно статистическо тестване, базирано на употреба. Обвързано е с висока цена, тъй като предварителната подготовка отнема значителни ресурси.

Всички **горни тестови подфази са основни** и следват относително **хронологичен ред**. Съществуват обаче някои **други специализирани задачи по отношение на тестването**, които нямат определено, фиксирано място в хронологията:

- **Тестване за диагностика на дефекти (Defect diagnosis testing)** – диагностициране на проблеми, получени при работа на крайните потребители със софтуера или при бета тестване. Реализира се чрез изпълнение на последователност от свързани тестове, като се прилага основно при инспекция на качеството.
- **Тестване, базирано на дефекти (Defect-based testing)** – тестване, при което се използват открити или потенциални дефекти. Сред стратегии за тестване са инжектиране на дефекти и тестване на мутации.

- **Регресионно тестване (Regression testing)** – проверява дали поддържаните софтуерни функции са засегнати от новите версии на продукта. Фокусира се основно върху интеграционното тестване при свързването на стари и нови компоненти.
- **Тестване за постигане на допълнителни цели (Testing to achieve other goals/objectives)** – тестване на производителността (Performance testing), стрес тестване (Stress testing), тестване на използваемостта (Usability testing)
- **Динамични анализи и техники, свързани с тях (Dynamic analyses and other related techniques for QA)** – симулация (Simulation), прототипиране (Prototyping), анализ с дърво на събитията (Event-tree analysis)

Необходима е **интеграция** по отношение на техниките за тестване, тъй като сами по себе си те имат различна ефективност по отношение на различни проблеми, но заедно могат да доведат до повишена гъвкавост и по-ниска цена.

Измерване. Метрики – характеристики и класификация. Метрики за качество на софтуерен продукт и софтуерен процес

Измерване наричаме процес, при който в съответствие с определени правила, на характеристики на изследвания обект се съпоставят стойности.

Мярка наричаме числото, съпоставено при измерването. Тя представлява различните състояния на измерваната характеристика.

Метрика на дадена характеристика наричаме процедура, която съпоставя нейните състояния на съответните им мерки. Понякога се използва и като синоним на мярка.

Различните **нива на измерване** могат да се представят чрез четири вида скали:

- **Номинална (Nominal) скала** – изброимо множество от категории/стойности, които се присвояват, без да се вземат предвид количествени съотношения; категориите трябва да бъдат взаимно изключващи се (mutually exclusive) и взаимно изчерпателни (jointly exhaustive)
- **Порядкова (Ordinal) скала** – наредено множество от категории, позволяващо подреждане на стойностите на оценъчните елементи чрез сравняването им с определени гранични стойности; асиметрична и транзитивна; нямаме информация за степента на разлика между елементите
- **Интервална (Interval) скала** – числови стойности, като разликата между всеки две последователни е една и съща; могат да се прилагат някои математически операции (събиране, изваждане) върху данните от скалата; могат да се превръщат мерки за един и същи атрибут от една към друга интервална скала
- **Относителна (Ratio) скала** – както интервалната, но освен че разликата между две последователни стойности е една и съща, има и ясно дефинирана мярка 0; всички математически операции могат да се прилагат върху данните от скалата

Важни **свойства на измерването** са:

- **Обективност** – получените мерки не зависят от субекта, извършващ измерването
- **Надеждност** – еднакви резултати при повтаряне на измерването при еднакви условия
- **Валидност** – получените мерки отразяват реално свойствата на измервания обект
- **Точност** – мярката има необходимата различаваща способност

Дейности при софтуерното измерване са:

1. Формулиране на метрична система

2. Събиране на данни
3. Анализиране
4. Интерпретиране
5. Осъществяване на обратна връзка

Характеристики на метриките са:

- **Общи изисквания**
  - **Уместност (relevance)** – измерва атрибут от съществено значение
  - **Валидност (validity)** – измерва искания атрибут
  - **Надеждност (reliability)** – еднакви резултати при еднакви условия
  - **Изчерпателност (comprehensiveness)** – приложимост в различни ситуации
  - **Взаимно изключваща се (mutually exclusive)** – не измерва вече измерен атрибут
- **Оперативни изисквания**
  - **Леснота и простота**
  - **Не се изисква независимо събиране на данни**
  - **Неподатлива на предубедени намеси от заинтересовани страни**

Класификация на софтуерните метрики в зависимост от:

- **Предназначението** – оценяване на разходите и усилията за разработване на софтуера, производителността на разработчиците, качеството на софтуера, неговата надеждност
- **Целта на прилагане на метриката** – за характеризиране, оценяване, прогнозиране, подобряване
- **Начина на получаване на информацията** – регистрационен подход, измерителен подход, възприятен подход, изчислителен метод
- **Типа на изследвания обект**
  - **Метрики на продукта** – свързани са с характеристики като размер, сложност, дизайн, бързодействие и др.
  - **Метрики на процеса** – служат за оценка на процеса на разработка
  - **Метрики на проекта** – свързани са с въпросите, засягащи частта, включваща дисциплината *Управление на проекти* – финансовите резултати, времевите резултати, продуктивността на екипа

Метриките за качество на софтуерен процес са подкатегория на метриките на процес и се делят на две категории сами по себе си:

1. **Метрики за честота на грешките (error density)** – размер на софтуера, брой грешки
2. **Метрики за сериозност на грешките (error severity)** – средна сериозност на грешките в кода, средна сериозност на грешките в разработката

Метриките за качество на софтуерен продукт основно се делят на:

- **Средно време до отказ (MTTF)** – особено важно за системи, от които се изисква висока степен на надеждност
- **Процент на грешки (defect rate)** – определя се на базата на допуснатите грешки спрямо възможността за допускането им, изразени например в брой редове от кода (Lines Of Code – LOC), брой функционални точки (functional points)
- **Проблеми, идентифицирани от потребителите (customer problems)** – най-често се използва „Проблеми за месеци“ (Problems per User Month – PUM)
- **Удовлетвореност на потребителите (customer satisfaction)** – измерва се с анкети на определени периоди след внедряването на софтуерното решение; петстепенна скала – много удовлетворен, удовлетворен, неутрален, неудовлетворен, силно неудовлетворен

Държавен изпит – спец. „Софтуерно инженерство“

Тема №20 – Софтуерна архитектура. Проектиране и документиране на софтуерни архитектури.

Анотация: *Изложението по въпроса трябва да включва следните по-съществени елементи:*

1. Дефиниция на софтуерна архитектура. Структури и изгледи (structures and views) на архитектурата.

2. Изисквания към качеството (нефункционални изисквания) на системата.

3. Проектиране на софтуерната архитектура. Процес за проектиране. Избор на подходящи структури. Последователност на създаване на архитектурата. Тактики (архитектурни решения) за постигане на желаните качествени показатели.

4. Архитектурни стилове.

5. Документиране на софтуерната архитектура. Предназначение на документацията. Основен принцип на документиране. Съдържание на документацията. Структура на документацията.

Дефиниция на софтуерна архитектура. Структури и изгледи (structures and views) на архитектурата

Обикновено **софтуерната архитектура** се създава като първа стъпка по време на проектирането, като целта е да се гарантира наличието на дадени качества в системата. Детайли като алгоритми, представяне на данни, реализация и др. не са предмет на софтуерната архитектура. Предмет на софтуерната архитектура е поведението и връзките между различни елементи, разглеждани като „черни кутии“.

Съгласно Software Engineering Institute архитектура на дадена софтуерна системата е съвкупност от структури, показващи различните софтуерни елементи на системата, външно видимите им свойства и връзките между тях.

Съгласно дефиницията става ясно, че системите имат повече от една структура. Нито една от тях самостоятелно не представлява архитектурата на системата.

Софтуерната архитектура обхваща **системната архитектура** (инфраструктура) и **архитектурата на приложението** (организация, подсистема, компонент).

Структура на архитектурата (structure) наричаме съвкупност от софтуерни елементи, техните външно видими свойства и връзките между тях. Външно видимите свойства и връзки задаваме чрез интерфейси, представени тук от високо ниво на абстракция.

Изглед на архитектурата (view) наричаме конкретно документирано представяне на дадена структура.

Двете понятия в голяма степен са взаимнозаменяеми.

Архитектурните структури се делят най-общо казано на три групи:

1. **Модулни структури**

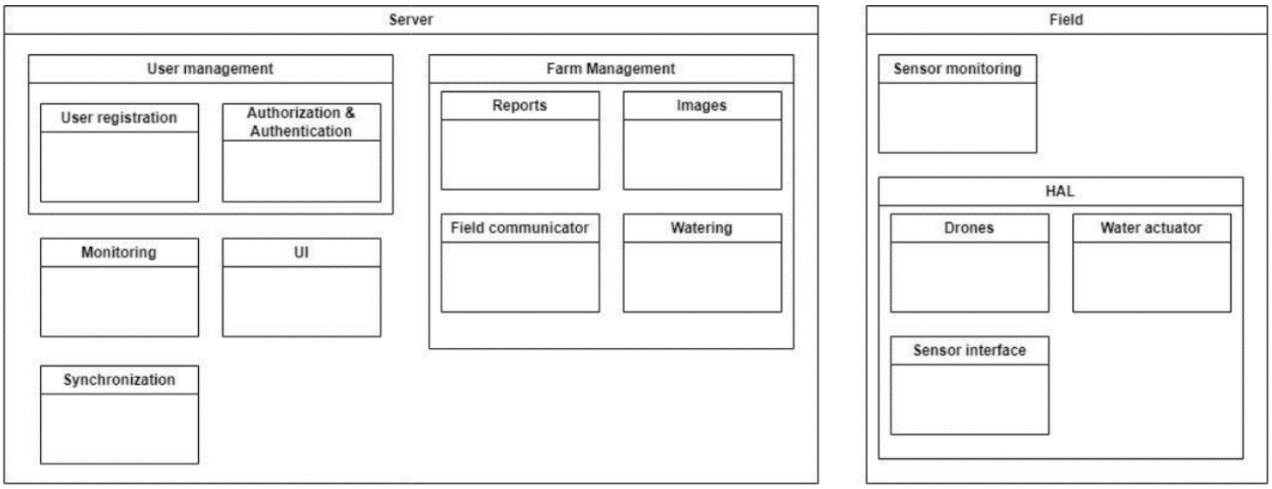
Елементите в модулните структури са **модули** – единици работа за изпълнение. Модулите предлагат поглед, ориентиран към реализацията на системата, без значение какво става по време на изпълнението.

Ще разгледаме няколко типа модулни структури:

1.1. Декомпозиция на модулите

При тази структура връзките между модулите са от вида **„X е подмодул на У“** и модулите биват рекурсивно **разбивани на по-прости единици**, докато станат лесни за разбиране. Декомпозицията на модулите обуславя в голяма степен възможността за лесна промяна, като обособява логически свързани функционалности на едно място. Много често служи и като основа на разпределението на работата между екипите на изпълнителя.

Пример: декомпозиция на модулите от архитектурата на софтуерна система за управление и оптимизиране на дейности в земеделското стопанство



1.2. Употреба на модулите

При този вид структура връзките между модулите са от вида **„X използва У“**. Структурата за употребата на модули обуславя възможността за лесно добавяне на нова функционалност, обособяване на [в голяма степен] самостоятелни подмножества от функционалност, както и позволява последователна разработка, много важна и мощна техника за работа. Ако има нужда от по-детайлно описание, може връзките да са насочени към конкретен интерфейс или ресурс на модула.

1.3. Структура на слоевете

Частен случай на „употреба на модулите“. Модулите са разделени на **слоеве**, като модулите от слой N може да ползват услугите само на модули от слой N-1. Слоевете скриват детайлите относно работата си от следващия слой, позволявайки лесна смяна на даден слой. Не е прието слоеве да се прескачат. Структурата позволява без особени сътресения да бъде подменен цял един слой (например да се смени СУБД).

1.4. Йерархия на класовете

В терминологията на ООП, модулите се наричат **„класове“**, а в настоящата структура, връзките между класовете са от вида **„класът X наследява класа У“** и **„обектът X е инстанция на класа У“**. Тази структура обосновава наследяването – защо подобни функционалности са обособени в супер-класове или пък защо са дефинирани под-класове за обслужване на параметризирани различия.

2. Структури на процесите

Елементите са **процеси** (или нишки), изпълнявани в системата (компоненти) и **комуникационни, синхронизационни или блокиращи операции** между тях (конектори). Връзките между тях (attachments) показват как компонентите и конекторите се отнасят помежду си.

Структурата е полезна, тъй като има отношение по въпросите на бързодействието по време на изпълнението и високата надеждност.

2.1. Структура на потока на данните

3. Структури на разположението

Структурите на разположението показват връзката между софтуерните елементи и елементите на околната среда, в която се намира системата по време на разработката или по време на изпълнението.

Ще разгледаме няколко типа структури на разположението:

3.1. Структура на внедряването (Deployment structure)

Показва как софтуера се разполага върху **хардуера** и **комуникационното оборудване**. Елементите са процеси, хардуерни устройства и комуникационни канали. Връзките са например **„внедрен върху“** или **„мигрира върху“** – показвайки върху кое устройство е разположен даден софтуерен елемент. Тази структура може да се използва за поглед върху производителността, сигурността и други метрики на дадена система.

3.2. Файлова структура

Показва кой модул къде се помещава във **файловата структура** по време на различните фази на реализация. Структурата е критична за управлението на дейностите по разработка и за създаването и поддържането на обкръжение за build-ове.

3.3. Разпределение на работата

Показва кой модул **от кой екип** се реализира. **Елементите** са **модули** и **екипи**. Екипите често не са списък от хора, а по-скоро виртуална група хора с подходящ опит, знания и умения. Структурата помага и за това дадени общи функционалности да бъдат обособени и разработени от един екип, вместо всеки сам да си ги прави.

Кои структури да използваме **зависи от системата**, както и от архитектурата. Добра практика е да се използва **поне по една структура** от всяка от гореизброените групи структури.

4 + 1 модел на софтуерната архитектура:

1) **Логически изглед** – показва основните абстракции в системата като обекти, класове и компоненти
2) **Изглед на процесите** – показва системата като съвкупност от взаимодействащи си процеси по време на изпълнение
3) **Изглед на кода** – показва как отделните елементи на системата се разполагат във файлове код
4) **Физически изглед** – показва как софтуерните компоненти са разпределени между хардуерните възли в системата

+1) **Съответните сценарии на употреба**

Изисквания към качеството (нефункционални изисквания) на системата

За проектиране на архитектурата е достатъчно да се започне с няколко най-важни изисквания. Най-важните изисквания ги наричаме **архитектурни драйвери**. Съществуват две големи изисквания към софтуерните системи – **функционални** и **нефункционални изисквания**.

Функционалните изисквания определят какво трябва да прави софтуерната система. **Качествените изисквания** определят как софтуерната система да работи. На практика качествата поставят **ограничения** върху начина, по който системата ще се изпълнява.

Качеството е субективно понятие – различните заинтересовани лица имат различни възприятия за него. **Бизнес целите** определят качествата, които трябва да бъдат вградени в архитектурата на системата.

Качествата се разделят на следните три основни групи:

- **Технологични качества** – например надеждност, изменяемост, производителност, сигурност, изпитаемост, използваемост и др.
- **Бизнес качества** – например време за пускане на продукта на пазара и др.
- **Архитектурни качества** – присъщи на самата архитектура като например идейна цялост (влияят косвено върху всички останали качества)

Софтуерната архитектура се изгражда на база качествените изисквания, като от тях се създават съответните структури, на които се вменява функционалност. Качествените характеристики трябва да се имат предвид както по време на проектирането, така и по време на разработката и внедряването. Тъй като някои от тях си противоречат, то задача на софтуерния архитект да прецени кои от тях са по-важни спрямо други, т.е. представляват архитектурни драйвери.

Изискванията за качество трябва да се формализират от архитекта посредством т.нар. **сценарии за качество**, за да бъдат те поставени на обективна основа. Сценариите демонстрират какво е качество в рамките на създаваната система, като дават на архитекта и на заинтересованите лица еднозначна основа за оценка на дизайна.

Сценарият за качество е специфично изискване към поведението на системата в дадена ситуация, в светлината на дадено качество. Те играят същата роля за дефиниране на нефункционалните изисквания, каквато роля играят сценариите за употреба (use cases) за

дефиниция на функционалните изисквания. **Всеки сценарий описва някаква случка и се характеризира със следните шест компонента (ВИОКРК):**

1. **Въздействие (Stimulus)** – състояние/събитие, което подлежи на обработка
2. **Източник (Source of Stimulus)** – обект (човек, система или нещо друго), който генерира въздействието
3. **Обект (Artifact)** – система или конкретна нейна част, върху която се случва въздействието
4. **Контекст (Environment)** – условията, при които се намира обекта по време на обработката на въздействието
5. **Резултат (Response)** – действията, предприети от обекта при случването на въздействието
6. **Количествени параметри (Response Measure)** – резултатът трябва да подлежи на някакви количествени измервания, така че да позволи проверката дали сценарият се изпълнява съгласно изискванията

Пример: *По време на работа на системата (**контекст**), потребителят (**източник**) прави статистика за продажбите за месеца (**въздействие**) и системата (**обект**) обработва заявката и връща pdf файл (**резултат**) за не повече от 20 секунди (**количествени параметри**).*

Някои нефункционални изисквания:

- **Изправност (dependability)** – занимава се с отказите, сривовете в системата и свързаните с това последствия
- **Изменяемост (modifiability)** – себестойността на промените
- **Производителност (performance)** – времето, за което системата реагира на възникващите събития
- **Сигурност** – мярка за способността на системата да устоява на опити за неразрешена употреба, без това да пречи на легитимните потребители
- **Изпитаемост** – лекотата, с която софтуерът може да бъде накаран да покаже дефектите си посредством извършване на различни тестове
- **Използваемост** – колко лесно потребителят успява да свърши дадена задача и каква подкрепа му оказва системата в това му начинание
- **Време за пускане на продукта на пазара**
- **Себестойност и печалба**
- **Предвидено време за живот на системата**

Проектиране на софтуерната архитектура. Процес за проектиране. Избор на подходящи структури. Последователност на създаване на архитектурата. Тактики (архитектурни решения) за постигане на желаните качествени показатели

Процесът на проектиране представлява циклично изпълнение на последователност от стъпки, в следствие на което се изгражда дадена софтуерна архитектура. В началото се избират целите (изискванията) с най-висок приоритет, които се превръщат в use case или качествени сценарии. Това са така наречените **архитектурни драйвери**, като се избират не повече от 10. Самият процес е известен като **Architecture Driven Design (ADD)**.

След като драйверите бъдат избрани се пристъпва към **проектиране**. По време на проектиране могат да се появят други проблеми за разрешаване, които да доведат до промяна на драйверите

и оттам идва цикличността на процеса. **За да удовлетворим тези изисквания, ние прилагаме определени архитектурни тактики и стилове.**

ADD е подход за проектиране, в който основна роля играят качествените свойства (атрибути). Това е рекурсивен процес на дефиниране на архитектурата, като на всяка стъпка се използват тактики и архитектурни модели за постигане на желаните качествени свойства. В следствие на приложението на ADD се получават първите няколко нива на модулната декомпозиция (**подходящо начало, с което да започнем**) и на други структури в зависимост от случая. Това е първото проявление на архитектурата и като такова е на достатъчно високо ниво без излишни детайли.

ADD приема следните входни данни:

- Функционални изисквания, като сценарии за употреба (use cases)
- Функционални ограничения (constraints)
- Качествени сценарии

ADD включва следните стъпки:

1. **Избира се модул за декомпозиция**
2. **Модулът се детайлизира:**
   *2.1. Избират се **архитектурни драйвери**, които се отнасят за модула*
   *2.2. Избира се **архитектурния модел на модула** чрез избор и конфигурация на тактики за осъществяване на драйверите*
   *2.3. Създават се **подмодулите** спрямо избрания модел*
   *2.4. Приписва се **функционалност на подмодулите***
   *2.5. Създават се **други структури за подмодулите** (например на процесите)*
   *2.6. Дефиниция на **интерфейсите на подмодулите***
   *2.7. Проверява се декомпозицията **дали отговаря на изискванията***
3. **Това се повтаря за всички модули, които се нуждаят от по-нататъшната декомпозиция**

Тактиката е архитектурно решение, чрез което се контролира резултата на даден сценарий за качество. Наборът от конкретни тактики се нарича архитектурна стратегия. Конфигурация от тактики оформя **архитектурен модел**.

1. Тактики за изправност (dependability)

1.1. Тактики за изправност – откриване на откази

*1.1.1. **Ping/echo** – компонент А пуска сигнал до компонент Б и очаква отговор в рамките на определен интервал от време*

*1.1.2. **Heartbeat, Keepalive** – компонент периодично изпраща сигнал, който друг компонент очаква*

*1.1.3. **Изключения (Exceptions)** – обработват се изключения, които се генерират, когато се стигне до определено състояние на отказ*

1.2. Тактики за изправност – отстраняване на откази

*1.2.1. **Активен излишък (Active redundancy, hot restart)** – активно дублиране и поддържане в едно състояние на важните компоненти. Downtime за милисекунди*

*1.2.2. **Пасивен излишък (Passive redundancy, warm restart)** – активният компонент реагира на събитията и синхронизира резервните. Downtime за секунда до часове*

*1.2.3. **Резерва (Spare)** – поддръжка на резервни изчислителни мощности. Downtime за минута до часове*

1.2.4. ***Разнородност*** *– отказите в софтуера обикновено се предизвикват от грешки при проектирането*

1.2.4.1. Разнородност в проектирането

1.2.4.1.1. ***Recovery Blocks*** *– разработват се няколко алтернативни модула на програмата*

1.2.4.1.2. ***N-version programming (Voting)*** *– на различни процесори работят еквивалентни процеси, като всички те получават един и същи вход и генерират един и същи резултат, който се изпраща на т.нар. voter, който решава крайния резултат от изпълнението.*

1.2.4.2. Разнородност по време

1.2.4.2.1. ***Извеждане от употреба (Removal from service)*** *– премахва се даден компонент от системата, за да се избегнат очаквани сривове. Например reboot за предпазване от memory leaks*

1.2.4.2.2. ***Следене на процесите (Process monitoring)*** *– специален процес следи други процеси, които ако откажат се преинициализират*

1.3. Тактики за изправност – при повторно въвеждане в употреба

1.3.1. ***Паралелна работа (Shadow mode)*** *– компонент, който е бил повреден, се оставя да поработи за известно време в парален с другите компоненти преди да бъде въведен обратно*

1.3.2. Ре-синхронизация на състоянието (State resynchronization)

1.3.3. ***Контролни точки и rollback*** *– при срив системата се възобновява (rollback) до последното запомнено консистентно състояние (checkpoint)*

2. Тактики за производителност (performance)

2.1. Тактики за производителност – намаляване на изискванията

2.1.1. Увеличаване на производителността на изчисленията

2.1.2. Премахване на overhead от ненужни изчисления

2.1.3. Промяна на периода на настъпване на периодични събития

2.1.4. ***Промяна на тактовата честота (rate limiting)*** *– пропускаме само част от събитията, когато не можем да променим периода на идването им*

2.1.5. Ограничаване на времето на изпълнение

2.1.6. Опашка с краен размер

2.2. Тактики за производителност – управление на ресурсите

2.2.1. Паралелна обработка на заявките

2.2.2. ***Излишък на данни/процеси*** *– cache, load balancing и др.*

2.2.3. ***Включване на допълнителни ресурси*** *– повече и по-мощен хардуер*

2.3. Тактики за производителност – арбитраж на ресурсите

Когато има недостиг на ресурси (т.е. спор за тях), трябва да има институция, която да решава (т.е. да извършва арбитраж) кое събитие да се обработи с предимство. Това се нарича scheduling.

2.3.1. ***FIFO*** *– заявките са равноправни и се обработват подред*

2.3.2. ***Фиксиран приоритет*** *– на различните заявки се присвоява различен фиксиран приоритет. Пристигащите заявки се обработват по реда на техния приоритет*

3. Тактики за изменяемост (modifiability)

3.1. Тактики за изменяемост – локализиране на промените – намалява броя на директно засегнатите модули

3.1.1. Поддръжка на семантична свързаност

3.1.2. Очакване на промените

3.1.3. Ограничаване на възможните опции

3.2. Тактики за изменяемост – предотвратяване на ефекта на вълната – модификациите да бъдат ограничени само до директно засегнатите модули

3.2.1. Скриване на информация

3.2.2. Ограничаване на комуникацията

3.2.3. Поддръжка на съществуващи интерфейси

3.2.4. Използване на посредник

3.2.5. Name server

3.2.6. Object request broker

3.3. Тактики за изменяемост – отлагане на свързването при внедряване

3.3.1. Включване/изключване/замяна на компоненти, както по време на изпълнението, така и по време на зареждането

3.3.2. Конфигурационни файлове, в които се задават стойности на различни параметри

3.3.3. Дефиниране и придържане към протоколи, които позволяват промяна по време на изпълнение

4. Тактики за сигурност (security)

4.1. Тактики за сигурност – устояване на атаки

4.1.1. Автентикация на потребителите – дали е този, за когото се представя

4.1.2. Оторизация на потребителите – има ли достъп до определени ресурси

4.1.3. Конфиденциалност на данните – чрез криптиране

4.1.4. Интегритет – механизми на излишък (checksums, hash algorithms)

4.1.5. Ограничаване на експозицията – на местата, от които можем да бъдем атакувани

4.1.6. Ограничаване на достъпа – firewall, demilitarized zone, вкл. и физическия достъп

4.2. **Тактики за сигурност – възстановяване след атака** – като тактиките за изправност

4.2.1. Поддръжка на audit trail – копие на всяка транзакция

5. Тактики за изпитаемост (testability)

5.1. Запис и възпроизвеждане на информацията, която минава през даден интерфейс

5.2. **Разделяне на интерфейса от реализацията** – позволява замяна (mock) на реализацията за тестови цели

5.3. **Специализиран интерфейс за тестване** – предоставяне на различни метаданни

5.4. **Вградени модули за мониторинг** – подаване на информация като метрики за състоянието към интерфейс

6. Тактики за използваемост (usability)

6.1. **Тактики за използваемост – по време на изпълнението** – насочени към крайния потребител

6.1.1. Моделиране на задачата

6.1.2. Моделиране на потребителя

6.1.3. Моделиране на системата

6.2. **Тактики за използваемост – свързани с UI** – насочени към разработчика на интерфейса

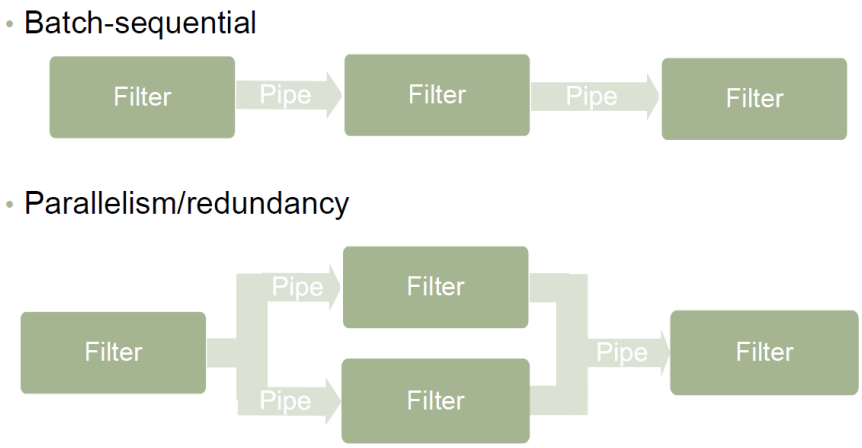
Архитектурни стилове

Архитектурният стил дефинира семейство от системи чрез използването на шаблон за структурна организация. **Представляват често срещани архитектурни решения на проблеми, с които е възможно да се сблъскаме. Стиловете** обуславят речника на компонентите и конекторите, използвани в тях, както и ограниченията в начина на употребата им, например топологията и семантиката на изпълнението им.

1. Pipe-and-filter

Модулите се делят на **филтри**, **канали**, **източник на данни** и **консуматор на данни**. Каналите са потоци от данни изпълняващи ролята на конектори между филтрите. Филтрите са модули, трансформиращи входните си данни, които могат да работят паралелно и независимо едни от други, като всеки филтър започва да предава изходните данни към следващия филтър преди да е свършил с обработката на всичките данни. Този начин за обработка на данни позволява паралелизъм (конвейеризация).

Филтрите нямат никаква информация за съседите си, което прави архитектурата гъвкава и лесна за разбиране. От друга страна, производителността страда заради парсването на данни от всеки филтър.



2. Shared data style

Активно използвана в системи, където компонентите трябва да предават **големи количества информация**. Споделените данни могат да се разглеждат като конектори между компонентите.

Вариации:

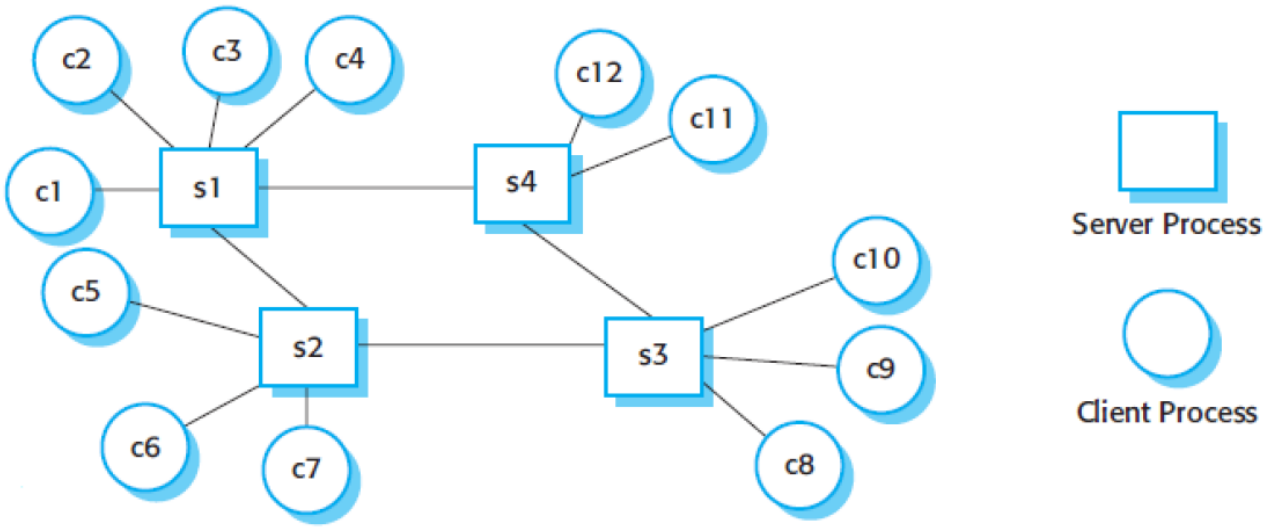
2.1. **Blackboard** – когато някакви данни са изпратени до конектора на споделената памет, всички компоненти трябва да бъдат информирани за това. С други думи, споделените данни са активен агент
2.2. **Repository** – споделените данни са пасивни. Никакви уведомления не се изпращат до компонентите

Предимствата на архитектурния стил са, че нови компоненти лесно могат да се добавят, всички компоненти могат да работят паралелно, високо ефективна, сигурна и др. Сред **недостатъците** ѝ са, че е трудно приложима в разпределена среда, споделените данни трябва да поддържат унифициран модел на данните, промените в модела могат да доведат до ненужни разходи, проблем при твърде много клиенти и др.

3. Client-Server style

Системата е построена като **набор от сървъри**, предлагащи услуги, и клиенти, които използват тези услуги. Клиентите могат да бъдат „тънки“ или „дебели“ – дебелите имплементират част от функционалността, а тънките – единствено потребителския интерфейс.

Предимствата на този стил са централизираните данни и сигурността, а **недостатъците** – риск от претоварване, както и нужда от резерви в случай на server failure.



4. Layered style

Тук компонентите са **вертикално разделени на слоеве**, като всеки слой може да предлага услуги на слоя над себе си и да използва услугите на слоя под себе си. Може да разгледаме този стил като разширение на клиент сървър архитектурата – всеки слой е клиент за слоя под себе си и сървър за слоя над себе си.

Предимствата включват абстракция и лесна изменяемост, докато **недостатъците** включват компромиси с производителността поради стриктните правила на комуникация, както и по-сложен дизайн процес.

5. Model-View-Controller

Състои се от:

- **Модел** - изпраща данни към изгледа, приема заявки за промяна на данните и управлява операциите с тях;
- **Изглед** – управлява презентацията на данните пред потребителите;
- **Контролер** – отговаря за взаимодействието с потребителя – натискане на клавиш, кликване и т.н., като изпраща заявки към модела и изгледа за необходимите действия

Този стил прави системата гъвкава и разширяема, но добавя много сложност към архитектурата както и може да доведе до компромиси с производителността.

6. Implicit Invocations style (publish-subscribe, event-based, message passing)

При този стил компонентите си комуникират чрез т.нар. „**събития**“. Тези събития може да съдържат както контролни съобщения, така и данни. Компонентите при такава архитектура работят паралелно, а конектора между тях е шина, по която се предават събитията.

Предимствата включват сигурност, както и гъвкавост (loose coupling на компонентите). **Недостатъците** са компромис с надеждността (ако шината прекъсне), както и несигурност (ако няма компонент, който да отговори на дадено събитие).

Документиране на софтуерната архитектура. Предназначение на документацията. Основен принцип на документиране. Съдържание на документацията. Структура на документацията

Дори перфектната архитектура е безполезна, ако не е документирана правилно. **Документацията на архитектурата** зависи от това кой ще я използва. Трябва да е достатъчно абстрактна, така че да бъде разбрана от нови служители. От друга страна трябва да е достатъчно детайла, за да послужи като основа на проектирането.

За някои **документацията** е предписание, а за други – описание. Всичко това означава, че един прост документ няма да стигне, за да удовлетвори изискванията на всички заинтересовани лица. Най-често се създава набор от документи с обогатено съдържание, което указва къде каква информация се съдържа.

Състава на набора от архитектурни документи до голяма степен се определя от това кой ще ги чете. Основен принцип е, че по време на документирането архитектът трябва да може да се постави на мястото на четящия.

Концепцията за структури и перспективи позволява да се дефинира **основният принцип на документирането на софтуерната архитектура**:

Документирането на софтуерната архитектура е въпрос на документиране на всички съставляващи я структури поотделно и последващо добавяне на документация, която се отнася за няколко структури.

Основният принцип е полезен, тъй като позволява **разделянето на проблема** на по-малки такива:

- Избор на подходящите структури
- Документирането им (създаване на съответните перспективи)
- Добавяне на допълнителна информация, която се отнася за повече от една структура. Тя обединява отделните изгледи в единна документация.

Различните перспективи преследват различни цели и имат различно предназначение. Кои **структури ще бъдат документирани** зависи от това кой ще чете документацията. Алгоритъмът за избор на структури е следният:

1. Създава се таблица с разпределение на приоритетите на различните заинтересовани лица по различни изгледи
2. Комбинират се перспективите, за да се намали броят им (например декомпозиция + слоеве, процеси + внедряване и др.)
3. Задават се приоритети на различните перспективи

Няма изграден индустриален стандарт за съдържанието на документацията на дадена структура. От съществено значение е да има някакъв стандарт (**шаблон**), т.е. да има последователност при създаването на документацията. Разглеждаме **7-елементно съдържание**, доказано в практиката:

1. **Първично представяне** – обикновено графично, често UML, само с първостепенна информация
2. **Описание на елементите и връзките** – детайлно описание на елементите и връзките, както и други второстепенни детайли. Следва да се опише смисъла и ролята на всеки един елемент и връзка, както и поведенията и интерфейсите на елементите
3. **Описание на обкръжението** – съдържа информация за това как елементите от документираната структура си взаимодействат с обкръжението – други системи, интерфейси, протоколи и др.
4. **Описание на възможните вариации** – често архитектурата е изградена така, че да позволява варианти за някои от детайлите, като конкретния вариант може да бъде избран на по-късен етап. Обикновено се дават като ограничителни условия и изисквания. Трябва да бъдат описани вариантите и условията;
5. **Архитектурна обосновка** – обяснява на заинтересованите лица защо структурата е проектирана по начина, по който е описана. Целта е да се дадат убедителни аргументи, че архитектурният проект е издържан. Включва обосновка на взетите решения, алтернативи, резултати, предположения
6. **Терминологичен речник** – кратко описание на използваната стандартна и нововъведена терминология
7. **Допълнителна информация** – всичко останало, като административна информация, описание на съдържанието, каталог на структурите, шаблон за описание на структурите, кратко описание на системата, карта на съответствието между структурите, списък на софтуерните елементи, разширен терминологичен речник

Държавен изпит – спец. „Софтуерно инженерство“

Тема №21 – Инженеринг на софтуерните изисквания. Техники за извличане, анализ и валидиране на софтуерните изисквания. Специфициране на изискванията.

Анотация: *Изложението по въпроса трябва да включва следните по-съществени елементи:*

1. Цел, задачи на инженеринга на софтуерните изисквания.

2. Видове изисквания - класификация

3. Същност на отделните етапи (дейности) на инженеринга на изискванията.

4. Техники за извличане, анализ и валидиране на изискванията, прилагани в отделните етапи на инженеринга на изискванията.

5. Начини за специфициране на изискванията. Видове модели в зависимост от перспективата на системата: модел на потока на данните, модели на поведението, семантични модели, обектно-ориентирани модели, формални модели.

Цел, задачи на инженеринга на софтуерните изисквания

Според **IEEE** (Institute of Electronical and Electronics Engineers) **софтуерно изискване** е:

1) **Условие или способност на софтуера**, необходимо на **потребителя**, за да реши проблем или да постигне определена цел.
2) **Условие или способност**, което трябва да има **системата** или **компонент** на системата, за да удовлетвори договор, стандарт, спецификация или друг формално наложен документ.
3) **Документално представяне** на условието или способността от т. 1) и т. 2).

С други думи, софтуерно изискване е описание на услугите/функционалностите и на ограниченията на системата, документирано представяне на нужда, способност или качество на софтуерна система. Изискванията се дефинират по време на процес, наречен **инженеринг на изискванията**.

Процесът на инженеринг на изискванията е систематичен процес на идентифициране, анализиране, документиране и проверка на функционалностите (услугите) и ограниченията на даден софтуер.

Класическото понятие **„инженеринг на изискванията“** е прилагането на систематичен, дисциплиниран, количествено измерим подход към спецификацията и управлението на изискванията.

Целта на инженеринга на софтуерните изисквания е достигането до пълни и недвусмислени изисквания преди етапа на проектирането. Спецификацията на идеалните изисквания трябва да описва бъдещата система (или промяна в системата) толкова пълно и точно, че разработчиците да могат да внедрят системата, без да искат обратна информация.

Инженеринга на изискванията е в ролята на **мост** между нуждите на клиентите, ползвателите и останалите заинтересовани от работата на създаваната софтуерна система и техническите възможност, предлагани от (интензивните) софтуерни технологии.

Процесът на инженеринг на изискванията е процес на проектиране, т.е. творчески процес, за който е необходима инженерна преценка. Като **вход** на самия процес се включват: *съществуваща информация за системата*, *нуждите на заинтересованите лица*, *организационни стандарти*, *регулации*, *информация за домейна*. Като **изход** на самия процес се включват: *съгласувани изисквания*, *системни спецификации*, *системни модели*.

Основните задачи на процеса на инженеринг на изискванията се обособят в следните **дейности**:

1. **Извличане на изискванията (Requirement elicitation)** – изискванията се откриват чрез консултации със заинтересованите страни и други техники, знания и информация
2. **Анализ на изискванията и преговори на изискванията (Requirements analysis and negotiation)** – изискванията се анализират, конфликтите се решават чрез преговори
3. **Специфициране (документиране) на изискванията (Requirements documentation)** – оформя се документ с изискванията
4. **Валидиране на изискванията (Requirements validation)** – документът на изискванията се проверява за съгласуваност и пълнота
5. **Управление на изискванията (Requirements management)** – успореден с всички по-горе описани процеси и управлява промените в изискванията

Видове изисквания – класификация

I. Класификация според нивото на детайлност на описанието (вертикална класификация)

1. Изисквания на клиента, бизнес изисквания, потребителски изисквания (Somerville) (User requirements)

Бизнес изискванията представляват целите на системата на високо ниво, записани на високо ниво на детайлност според вижданията на организацията или клиента, който е поръчал системата. Те дефинират визията и обхвата на системата.

Бизнес изискванията описват защо организацията реализира системата и целите, които организацията се стреми да постигне. Необходимо е да се разграничат два съществено различни типа:

- **Същински бизнес изисквания** – представят резултатите, стоки или услуги, от разработката на системата. Същинските бизнес изисквания са концептуални и съществуват в бизнес среда. Потребителските изисквания описват целите на потребителите или задачите, които потребители трябва да могат да извършват с продукта. Потребителските случаи описват какво ще може да извършва потребителят със системата.
- **Изисквания на продукта** – определят се от гледна точка на продукта, определен от заинтересованите лица. Това е един от начините за изпълнението на бизнес изискванията. Те доставят стойност само ако удовлетворяват същинските бизнес изисквания.

2. **Системни изисквания (System requirements)**

Системните изисквания представляват детайлно описани на услугите и ограниченията, което се характеризира с пълнота и консистентност. Описват изисквания за софтуерния продукт като система и са основа за сключване на договор. Обикновено съдържат структурни модели.

3. **Спецификация на софтуерните изисквания (Software design specification)**

II. Хоризонтална класификация на софтуерните изисквания

1. Функционални изисквания

Функционалните изисквания описват какви **услуги** или **функционалности** трябва да предоставя системата на потребителите или на други системи. В някои случаи, функционалните изисквания указват какво не трябва да прави системата. Едно функционално изискване описва необходимото **поведение** от гледна точка на нужните дейности. Дефинират границите на решенията за даден проблем.

Функционалните изисквания се документират чрез три допълващи се, но частично препокриващи се **перспективи**:

- Перспектива на данните
- Функционална перспектива
- Перспектива на поведението

Описанието не трябва да бъде твърде общо, за да се избегне неяснота, но и не трябва да бъде твърде специфично, за да се избегнат твърде обемисти и трудно четими документи.

2. Нефункционални изисквания

Нефункционалните изисквания са тези изисквания, които не се отнасят до функционалността на системата. Те поставят **ограничения** върху софтуерния продукт и процеса на разработката му. Уточняват **външните ограничения**, с които трябва да е съобразен продукта. Отнасят до **системата като цяло**, а не до някоя отделна функционалност.

Типовете нефункционални изисквания са:

- **Изисквания към продукта (качествени изисквания)**

Изискването за качество описва някаква **качествена характеристика**, която трябва системата да притежава като поведение. Например, кратко време за отговор, висока надеждност и др.

Изискването за качество дефинира свойство на цялата система. Типовете качествени свойства се разделят на такива с **основно значение за потребителите** (*наличност*, *ефективност*, *надеждност и др.*) и такива с **основно значение за разработчиците** (*възможност за поддържане, обновяване, повторно използване, лесно тестване*).

Качествените системни изисквания са:

- **Наличност** – процент от времето, през което системата е налична и работи изцяло
- **Ефективност** – мярка за това колко добре системата използва хардуерните си ресурси
- **Гъвкавост** – колко усилия са необходими, за да се разшири системата с нови възможности
- **Цялостност** – колко добре системата е защитена срещу непозволен достъп, нарушаване на поверителността на данните, загуба на информация и заразяване с вреден софтуер
- **Съвместимост** – колко лесно системата може да обменя данни с други системи
- **Надеждност** – способността на една система или компонент да извършва своите функции при указаните условия за определен период от време
- **Устойчивост/Здравина** – способността една система да продължи да функционира, когато трябва да се справи с определени входни данни, нарушения в свързаните системи или непредвидени условия за работа
- **Използваемост** – усилията, които се изискват от потребителите, за да подготвят входни данни, да работят със системата и да интерпретират изходните данни
- **Възможност за поддържане и обновяване** – доколко е лесно да се поправи недостатък или да се направи промяна в системата
- **Преносимост** – усилията за извършване на миграция на система или компонент от една операционна среда в друга
- **Възможност за повторно използване** – доколко е възможно един компонент да се използва в други системи, за които не е бил разработван първоначално
- **Възможност за тестване** – степента на лесната, с която софтуерните компоненти или цялата система могат да се тестват за откриване на дефекти или недостатъци

- **Организационни изисквания (изисквания на процеса на разработка)**

Ограниченията представляват организационно, технологично или нормативно изискване, което определя начина, по който ще бъде разработена системата. **Организационните изисквания** са изисквания, които са ограничения върху процеса на разработка, зависещи от съществуващи политики и установени практики в организациите на разработчиците и на клиентите.

Видовете организационни изисквания са:

- **Методи и стандарти**, които трябва да се следват
- **Изисквания за имплементация** – използван език за програмиране, метод за проектиране, технология за разработка и др.
- **Изисквания по доставянето** – срокове и отчети, които трябва да бъдат доставени

- **Външни изисквания**

Външните изисквания са резултат на **външни** за системата и за процеса на създаването ѝ фактори. Те могат да са наложени както върху продукта, така и върху процеса на разработване. Това са неща като необходимостта да работи с други системи, съобразяване с правила, закони и корпоративни политики.

3. Изисквания, произтичащи от приложната област

Изискванията, произтичащи от приложната област, произлизат като резултат от областта на приложение, а не от изискванията на клиента. Тези изисквания могат да бъдат сами по себе си **функционални** и **нефункционални изисквания**. Могат да произлизат от основни природни закони.

Същност на отделните етапи (дейности) на инженеринга на изискванията

I. Извличане на изискванията

Извличане на изискванията е процесът на търсене, откриване и обобщаване на софтуерните изисквания от наличните източници. Включва работа, с която да може да се проучи приложната област и внимателно да се анализират изискванията на всички заинтересовани страни.

Дейностите по идентифициране на изискванията включват следните четири измерения:

- **Разбиране на приложната област** – знания за областта, в която се прилага системата
- **Разбиране на проблема** – детайлите за специфичния клиентски проблем, предоставени от различни хора
- **Разбиране на бизнеса** – как си взаимодействат системите и как спомагат за общите бизнес цели
- **Разбиране на нуждите и ограниченията на заинтересованите лица** – имат различни изисквания и могат да ги изразяват по различни начини

II. Анализ на изискванията

Целта на анализа на изискванията е да открие проблеми като незавършеност или несъгласуваност в идентифицираните изисквания, както и да се обсъдят и разрешат със заинтересованите лица в процеса на преговорите. Той е зависим от опита и експертизата на хората, които са включени в процеса на анализ.

Проверки на анализа:

- **Проверка за необходимост** – анализира се необходимостта на изискването, за да не се допуска изискване, което не допринася за постигането на бизнес целите
- **Проверка за съгласуваност и завършеност** – изискванията не трябва да си противоречат, но и никои от необходимите услуги и ограничения не трябва да бъдат пропуснати
- **Проверка за осъществимост** – изискванията се проверяват за това дали са изпълними в контекста на бюджета и плана за разработването на системата

III. Специфициране на изискванията

Специфициране на изискванията е процесът на записване на потребителските и системните изисквания в документ на изискванията. Изискванията на потребителите трябва да бъдат разбираеми от крайните потребители и клиентите, които нямат техническа подготовка. Системните изисквания са по-подробни изисквания и могат да включват повече техническа информация. Системните изисквания могат да бъдат част от договор за разработване на системата. Затова е важно те да са възможно най-пълни.

Независимо от използвания език за програмиране и метод на работа, **три аспекта** трябва да бъдат документирани:

- **Аспект на функционалност** – описва се с трите **перспективи** на системата
  - **Данни** – как се използват и как са структурирани
  - **Функции** – условия за ползване, начини на обработка на входните данни и очаквани резултати
  - **Поведение** – следи се динамиката на проявление
- **Аспект на качеството на софтуерната система**
  - Начини на представяне на системата – обем на данните, скорост, време за реакция, производителност и др.
  - Специфициране на стойности на метрики – използваемост, надеждност и др.
- **Аспект на ограниченията**
  - Технически – протоколи, интерфейси, среда за разработка
  - Правни – закони, стандарти, нормативи
  - Културни, етични
  - Околна среда – физически закони
  - Решения/ограничения, изисквания от ключови заинтересовани лица

IV. Валидиране на изискванията

Валидирането на изискванията е процес на проверка на това дали изискванията дефинират системата точно така, както клиентът е пожелал. Това е важен процес, защото поправка на изискванията след доставянето на софтуера би струвала стотици пъти повече от поправка на имплементационна грешка по време на разработка.

Ако **анализът на изискванията** работи със сурови изисквания, директно извлечени от заинтересованите лица, то **валидирането на изискванията** работи с крайния чернови вариант на формулираните изисквания, т.е. с вече с договаряни и съгласувани изисквания.

Основните проверки по време на етапа за валидиране на изискванията са проверка за валидност, консистентност, пълнота, реалистичност, проверимост. Документът на изискванията е входна информация за процеса на валидирането.

V. Управление на изискванията

Управлението на изискванията е процес на управление на промените на изискванията на системата. Главните задачи на управлението на изискванията са:

- Управление на промените в уговорените изисквания
- Управление на връзките между изискванията
- Управление на зависимостите между документа на изискванията и други документи, създадени в процеса на инженеринг на изискванията

Изискванията не могат да бъдат управлявани ефективно, без да има **проследимост** на изискванията.

Техники за извличане, анализ и валидиране на изискванията, прилагани в отделните етапи на инженеринга на изискванията

I. Техники за извличане на изискванията

1. Сценарии и потребителски случаи

Сценариите са примери от реалния живот за това как може да се използва системата. Те са точно определена последователност на взаимодействие между актьора и системата. Обикновено са кратки (между 3 и 7 стъпки). Описват положителни и отрицателни взаимодействия.

Предимства:

- много са **естествени** и обикновено заинтересованите лица ги използват **спонтанно**
- кратките сценарии са много добри за **бързо илюстриране** на специфични взаимодействия

Недостатъци:

- липса на **структура**
- трудно се проверяват за **пълнота**

Потребителските случаи са техника на структурно представяне, базирана на сценарии, която идентифицира актьорите в едно взаимодействие и която описва самото взаимодействие. Наборът от всички потребителски случаи за една система описва всички възможни взаимодействия със системата.

За илюстриране със средствата на **UML** се използват диаграма на потребителските случаи, допълнена с диаграма на последователността и диаграма на дейностите.

2. Интервюта, въпросници, анкети

Интервюирането е основен източник на информация за изискванията. То изисква търпение, изслушване, разбиране и задаване на разнопосочни въпроси. **Интервютата** могат да бъдат **затворени** (предварително подготвени въпроси) и **отворени** (няма предварително подготвени въпроси). Интервюиращите трябва да бъдат непредубедени и да не провеждат интервюто с предварително формирани идеи за това какво се изисква.

Интервюто се фокусира върху въпроси за:

- Бизнес случай за проекта
- Функционални изисквания за проекта
- Рискове
- Ограничения
- Заинтересовани лица

3. Мозъчна атака

Също позната като **brainstorming**, това е техника, която представлява работа в **екип** и предизвиква откриването на знания от всеки участник, което помага при разглеждането на алтернативи и правене на правилни избори. Помага за разглеждане на проблема от различни гледни точки.

4. **Работни срещи**

Аналитикът **събира (основни) заинтересовани страни**, за да анализира системата и да разработи решението. Изискванията се идентифицират и дискутират съвместно. В идеалния случай се провеждат в контролирана среда чрез обмен на знания. Ръководителят на срещата поддържа фокусирана дискусия, а протоколчик я документира.

5. **Soft Systems Method**

Това не е техника за детайлно идентифициране на изискванията. Тя е по-скоро начин да се **разбере** даден проблем и неговия **контекст в организацията**. Обръща се внимание на това, че софтуерните системи са вградени в един по-широк човешки и организационен контекст. Правят се редица оценки, описание, модели, сравнения, препоръки.

6. **Наблюдение и социален анализ**

Етнографът прекарва известно време в **наблюдение** на хората по време на работа и така си изгражда представа за това как се извършват дадените процеси. **Етнографията** може да се използва за извличане на социални изисквания и някои организационни такива. Тя е хубаво да се комбинира с други техники, като например интервюта и прототипиране.

7. **Повторно използване на изискванията**

Повторното използване включва вземане на изискванията, които са били разработени за една система и използването им за друга. Спестява **време** и **усилия**, тъй като изискванията вече са били анализирани и валидирани в други системи. Към тази техника трябва да се подхожда с внимание, тъй като може да доведе до проблеми в някои непредвидени ситуации.

8. **Потребителска история (User story)**

Потребителските истории представляват кратки, прости общи описания на функция, разказани от конкретна гледна точка. Запазва фокуса върху потребителя и напомня, че има действителна цел, която трябва да бъде постигната чрез прилагане на функционалност.

Кратки са и имат следния формат: *Като [потребител] искам [да извършва това действие] така, че да [постигна тази цел].*

9. **Прототипиране**

Прототипът е първоначална версия на системата, която може да се използва за експериментиране. **Експериментирането** подпомага изясняването и допълването на изискванията. Може да прерасне в цялостен краен продукт в определени условия. **Прототипирането** може да се прилага при два процеса на инженеринга на изискванията – **извличане на изискванията** (потребителите могат да експериментират със системата и да посочат нейните силни и слаби страни) и **валидиране на изискванията** (лесно може да се коригира/оптимизира първоначалния вид на системата).

Бързото разработване на прототипа е от изключителна важност. Съществуват два вида прототипи – **throw-away прототип**, който подпомага специфицирането на изискванията, и **еволюционен прототип**, с който не се предоставя детайлна спецификация, но изисква прецизност в изработката като целта му е бързо да се представи работеща система на клиента.

Подходи за бързо прототипиране:

- Прототипиране на хартия
- Wizard of Oz прототипиране
- Автоматизирано прототипиране

II. Техники за анализ на изискванията

1. Списък (Checklist)

Списък от ключови и достатъчно общи въпроси за оценка на всяко едни изискване. В тях се проверява дали изискванията са:

- Неделими (Cohesive)
- Пълни
- Съгласувани
- Без връзки (атомарни)
- Проследими
- Текущи
- Осъществими
- Недвусмислени
- Задължителни
- Проверими

Добрите изисквания са **SMART(T)** изисквания:

- **Specific** (Специфични)
- **Measurable** (Измерими)
- **Attainable** (Достижими)
- **Realistic** (Реалистични)
- **Traceable** (Проследими)
- **Testable** (Възможност за тестване)

2. Взаимодействия между изискванията

Много важна цел на анализа на изискванията е да се открият **взаимодействията** между изискванията и да се посочат конфликтите и припокриванията между тях. Това се прави чрез матрица на зависимостите и взаимодействията.

Техника за приоритизиране на изискванията е използването на **MoSCoW**. Тя представлява съвместно споразумение със заинтересованите лица за важността върху доставянето на всяко едно от изискванията.

3. Договаряне на изискванията

Договарянето на изискванията е процес на обсъждане на **конфликтите** между изискванията и постигане на споразумение, с което всички заинтересовани лица са съгласни. Разногласията за изискванията са неизбежни при наличието на много заинтересовани лица. При планирането на процеса на инженеринг на изискванията е важно да се остави достатъчно време за преговори.

Последователността на стъпките е следната:

- Обсъждане на изискванията
- Приоритизиране на изискванията
- Споразумение на изискванията

III. Валидиране на изискванията

1. Преглед на изискванията

Прегледът на изискванията е най-често използваната техника за валидиране на изискванията. Формулира се специална група, която да:

a. се запознае с изискванията и да ги анализира
b. формулира забелязаните проблеми
c. дискутира възможните действия за справяне с проблемите

Разходът за прегледа на изискванията би могъл да бъде намален, ако се направи предварителна проверка от един човек.

Дейностите по прегледа включват:

- **Планиране** – определя се екип, ръководите и протоколчик; избира се време и място за срещите
- **Разпространение на документи** – до участниците в екипа
- **Подготовка за прегледа** – всеки участник прочита документа и набелязва открити конфликти, пропуски, отклонения от стандартите и др.
- **Среща по прегледа** – дискутират се коментари и проблеми от отделни участници на екипа и се дефинира наборът от възможности за справяне с проблемите
- **Проследяване на действията** – ръководителят на екипа проверява дали формулираните действия са изпълнени
- **Ревизиране на документа** – документът се редактира, за да се отразят съгласуваните действия

2. Прототипиране

Прототипирането като техника за валидиране на изискванията се прилага само ако има вече разработен прототип за целите на извличане на изискванията.

Прототипите имат за цел да демонстрират изискванията и да помогнат на заинтересованите лица да открият проблеми. Те трябва да бъдат достатъчно пълни и ефикасни, за да се оцени използването на исканата система. Валидирането може да започне с непълен прототип, но задължително системата трябва да бъде допълнена и довършена по време на валидирането.

Действията при прототипирането включват успоредни процеси и са:

- Избор на подходящ екип за тестване на прототипа
- Разработване на тестови сценарии
- Изпълнение на сценариите
- Документиране на проблемите

3. **Валидиране на модел**

Цел на валидирането:

- Проверява дали моделът е **логичен**, съдържа **цялата необходима информация** и няма конфликти
- Проверява дали всеки модел точно представя **реалните изисквания** на клиента
- Проверява дали моделите за една система са **вътрешно и външно съвместими**

Някои проверки са възможни с автоматични софтуерни инструменти – CASE инструменти.

4. **Тестване на изискванията**

За всяко изискване трябва да може да бъдат съставени **тестове за проверка дали изискването е добре дефинирано**. Невъзможността да се създаде тест означава, че има липсваща или неясна информация в описанието на изискванията. Всяко функционално изискване в документа трябва да бъде анализирано с подходящ тест.

Един **формуляр за тест на изискванията** би включвал информация за:

- Идентификатор на изискването
- Свързани изисквания
- Описание на теста
- Проблеми
- Коментари и препоръки

Начини за специфициране на изискванията. Видове модели в зависимост от перспективата на системата: модел на потока на данните, модели на поведението, семантични модели, обектно-ориентирани модели, формални модели

Начините за спецификация на изискванията могат да се разделят на следните категории:

- **Неформално**
  - Естествен език
- **Полуформално**
  - Структурни модели
  - Модели на взаимодействието
- **Формално**
  - Математически концепции

Документирането на естествен език означава, че изискванията са написани като естествени езикови изкази. Всяко изречение трябва да изразява само едно изискване. Този начин сам по себе си е изразителен, интуитивен и универсален, както и разбираем за клиентите. Проблемите при него обаче са липсата на яснота и на точност, смесване и сливане на изискванията.

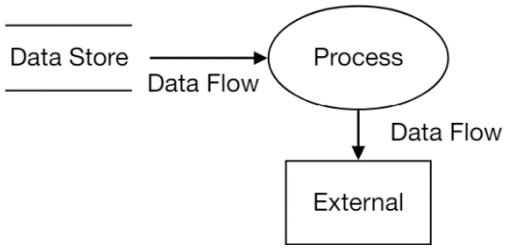
Следващата стъпка е **документиране с използване на модели на системата**. Моделът е абстракция, опростено изображение на обекта на изследване. Моделите се използват главно да представят функционалните изисквания, докато нефункционалните се изразяват на естествен език.

В зависимост от **трите перспективи** на софтуерната система (функции, поведение и данни) се оформят различни категории модели – **модели на функциите**, **процесите**, **модели на поведението** и **модели на данните**.

Модел на потока на данните (Data Flow Model – DFM) представя потока на информацията в рамките на системата, както и между системата и средата. Той не показва последователност на поведение и на управляваща информация. Подходът на потока на данни е типичен за метода на структурен анализ. Най-високото ниво на диаграмата на потока на данни е **контекстната диаграма**.

Диаграмата на потока на данните графично представя:

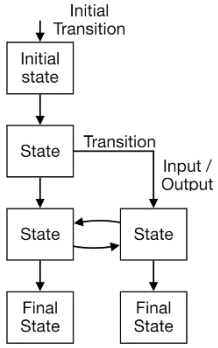
- **Външни за процеса елементи (External Entity)** – външен източник или получател на данните; обикновено извън обхвата на изучаваната област
- **Процеси (функции) (Process)** – трансформира или манипулира данни
- **Поток на данните (Data Flow)** – пренася данни между източник и процес, процес и получател или между два процеса
- **Хранилища за данни (Data Store)** – хранилище на данни
- **Конекторни символи (Connector Symbols)** – използват се в случай, че диаграмата се представя на няколко страници



Модел на поведението на системата дава т.нар. **State Transition Diagram (STD)**, която показва динамиката в поведението на една система – как системата реагира на последователност от външни събития и как независими компоненти координират работата си.

Основни компоненти на диаграмата са:

- **Състояние (State)**
- **Действие/събитие (Event)** – външно или вътрешно, времево
- **Преходи между състоянията**



Моделът на данните е начин да представим структурата на данните. Моделът на данните се използва често заедно с DFD.

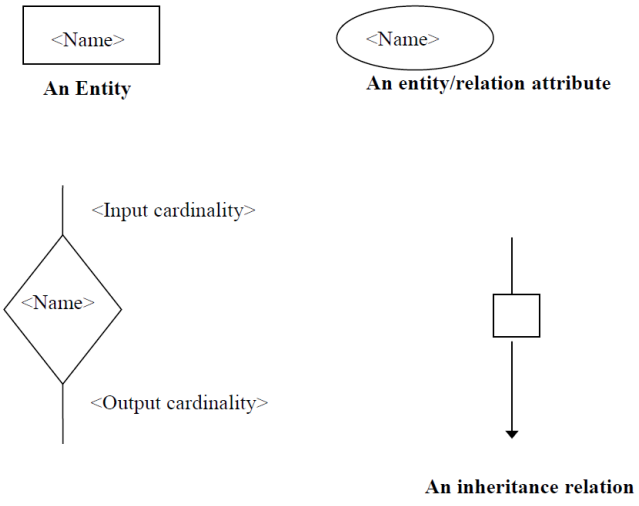
Видове модели на данните:

- **Релационен модел** – задава данните като набор от таблици с общи ключове
- **Семантичен модел** – представят субектите в база данни, техните атрибути и връзки
- **Обектно-ориентиран модел** – представя едновременно данните и тяхната обработка, както и класификация и връзки на субектите в системата

Семантичният модел е модел на данните, който представя субектите в база данни, техните атрибути и връзки. Описва се чрез **Entity Relationship Diagram (ERD)**, която представя логическия вид на данните и връзките между тях.

Основните компоненти на ERD:

- **Същност (Entity)** – обект или дейност
- **Атрибути (Attributes)**
- **Връзки (Relationships)** – с елементи:
  - *Кардиналност* – (1:1), (1:n), (n:1), (n:m)
  - *Опционалност* – задължителна и незадължителна
  - *Зависимост* – рекурсивна, подтип и надтип



Обектно-ориентираните модели представят едновременно данните и тяхната обработка, както и класификация и връзки на субектите на системата. Основната идея е идеята за обект. Обектно-ориентираният анализ, проектиране и разработка е общоприет метод в компютърните науки.

Основни концепции на обектно-ориентираните модели:

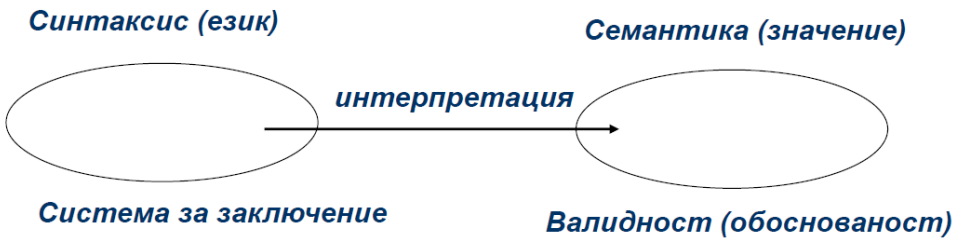
- **Обект** – нещо реално или абстрактно, за което запазваме данни и описваме операции за манипулиране на тези данни
- **Клас** – описание на множество от обекти, които споделят общи атрибути и операции (услуги). Обектът е инстанция на клас

- **Операции** – действия за манипулиране на данните на обект – четене, запис
- **Метод** – реализация на дадена операция от поведението на обекта. Специфицира се пътят, по който операциите се имплементират
- **Капсулиране на данни за обект** – пакетиране на данни и операции, което предпазва данните на обекта от неразрешен достъп
- **Наследяване** – при таксономия на класовете, обекти на по-ниско ниво в йерархията наследяват атрибутите и поведението на техните родители
- **Агрегиране** – обекти са съставени от други обекти
- **Съобщения** – начин, по който обектите си комуникират. Когато обект получи съобщение, той изпълнява съответен метод

Формалните модели включват математически синтаксис и семантика. Те осигуряват високо ниво на точност на системата, която напълно да покрие спецификацията, но не могат да гарантират абсолютна точност. **Формалните методи** са ниво на абстракция, което позволява да се създаде качествен софтуер.

Основни компоненти:

- **Синтаксис** – специфична нотация за представяне на спецификацията
- **Семантика** – дефинира съвкупността от обекти, които ще се използват, за да се опише системата
- **Връзки** – дефинират правилата, които показват кои обекти правилно удовлетворяват спецификацията



Държавен изпит – спец. „Софтуерно инженерство“

Тема №22 – Проектиране и интегриране на софтуерни системи.

Анотация: *Изложението по въпроса трябва да включва следните по-съществени елементи:*

1. Характеристики на разпределените софтуерни системи – дефиниции, видове системи и тенденции.

2. Междупроцесна комуникация – отдалечено извикване, мултикаст.

3. Разпределени обекти и компоненти.

4. Уеб услуги – дефиниции, шаблони за комуникация. Стандарти за уеб услуги – SOAP, UDDI, WSDL.

Характеристики на разпределените софтуерни системи – дефиниции, видове системи и тенденции

В наши дни мрежите от компютри са навсякъде. Всички тези мрежи могат да се обединят под името **разпределени системи**. В своята книга *Distributed Systems: Concepts and Design* **G. Coulouris** дефинира **разпределената система като такава, в която хардуерните или софтуерните компоненти, разположени в компютри (възли), свързани в мрежа, комуникират един с друг и координират своите действия само чрез размяна на съобщения**. Според една друга дефиниция на **A. Tanenbaum** **разпределената система е колекция от независими компютри, които изглеждат на потребителя като една кохерентна (цялостна) система**.

В резултат от дефинициите следват **три важни характеристики** на разпределените системи:

1. **Конкурентност** – задължително условие, превърнало се в норма
2. **Липса на глобален часовник** – единственият начин за комуникация е чрез размяна на съобщения
3. **Независими повреди** – всеки компонент на системата може да се повреди независимо от останалите

Примери на разпределени системи в наши дни могат да се намерят в много услуги, които ползваме ежедневно като например в *Интернет среда и WWW*, *уеб търсенето*, *онлайн игрите*, *имейл*, *социални мрежи*, *електронна търговия* и др.

В книгата на **G. Coulouris** се говори за **три вида разпределени системи**, оформящи различните поколения разпределени системи:

1. **Ранни разпределени системи (early distributed systems)** – състоят се от между 10 и 100 възела, свързани обикновено в локална мрежа; поддръжка на малко услуги; хомогенни с малка степен на разширяемост и отвореност
2. **Интернет-скалируеми разпределени системи (Internet-scale distributed systems)** – разширяемо множество от възли, свързани чрез мрежа от мрежи (Интернет); високо ниво на хетерогенност; отворени стандарти и наличие на междинен слой / платформа (middleware)
3. **Съвременни разпределени системи (contemporary distributed systems)** – мобилност (смартфони, лаптопи); наличие на вградени системи; облачни изчисления

Разпределените системи претърпяват период на **значителни промени**, които се дължат на **съвременните тенденции**, сред които са:

1. Появата на **всеобхватни (pervasive) мрежови технологии**
2. Интеграцията на **мобилни, преносими изчислителни устройства** (смартфони, таблети, умни часовници и др.) в разпределените системи
3. Голямото търсене и нужда от **мултимедийни услуги**
4. Разглеждането на разпределените системи като **услуга (удобство), а не като притежание (не е задължително наличието на оборудване**) – облачни изчисления

Междупроцесна комуникация – отдалечено извикване, мултикаст

Съществуват три типа комуникационни парадигми:

1. **Междупроцесна комуникация (Interprocess Communication)** – комуникация от ниско ниво между процеси в разпределени системи (примитивни за размяна на съобщения, директен достъп до API на Интернет протоколите, мултикаст комуникация)
2. **Отдалечено извикване (Remote Invocation)** – най-разпространената комуникационна парадигма в разпределените системи; двупосочна размяна
   2.1. Request-reply protocols (RRP) – отдалечено извикване на операции
   2.2. Remote procedure calls (RPC) – отдалечено извикване на процедури
   2.3. Remote method invocations (RMI) – отдалечено извикване на методи
3. **Индиректна комуникация (Indirect Communication)** – групова комуникация, опашки от съобщения, системи за публикуване и абониране и др.

Размяната на съобщения между двойка процеси (IPC) може да се поддържа от две операции за комуникация чрез съобщения – send и receive. За осъществяване на комуникация един процес изпраща съобщение (последователност от байтове) до определена дестинация и друг процес, намиращ се на дестинацията, получава съобщението. Комуникацията може да бъде **синхронна** и **асинхронна**. При **синхронната комуникация** изпращащият и получаващият процес синхронизират на всяко съобщение. В този случай операциите *send* и *receive* са блокиращи операции. При **асинхронната комуникация** операцията *send* е неблокираща. Операцията *receive* може да бъде както блокираща, така и неблокираща. Дестинацията на съобщенията винаги е във формат: *(Internet address, local port)*. Клиентските програми реферират към дадена услуга по име и използват сървър на имената (*name server*), за да преведе техните имена към интернет адреси по време на изпълнение.

Други **важни характеристики** на IPC са **надеждността** и **подредбата**. По отношение на **надеждността** е важно съобщенията да бъдат доставени дори и част от пакетите да се изгубят по пътя, както и целостта, т.е. съобщенията да пристигнат некорумпирани и без дублиране. Що се отнася до **наредбата**, тук влиза подредбата при пристигане дали съвпада с тази при изпращане и има ли възможност да се осъществи тази подредба чрез някакви механизми.

Съществуват редица **протоколи за междупроцесна комуникация**. Един възможен механизъм е комуникацията чрез **UDP дейтаграми**, която е по-проста и в някои случаи по-бърза, но не гарантира надеждност и подредба. Друга възможност е **потоковата TCP комуникация**, при която имаме по-голяма надеждност, поддържане на сесия, както и подредба, за разлика от UDP, но за сметка на това имаме по-ниска скорост. На базата на TCP конекция стъпват протоколи като **HTTP**, **FTP**, **Telnet**, **SMTP** и др.

Размяната на съобщения по двойки не е най-добрият модел за комуникация от страна на един процес към много други процеси. **Мултикаст операция** е операция, която изпраща едно съобщение от един процес към всеки един член на група от процеси. Най-простият мултикаст протокол не може да ни даде гаранция за доставката на съобщенията и за тяхната подредба.

IP multicast се изгражда на основата на протокола **IP (Internet Protocol)**. В този случай изпращащият процес не е наясно с идентичността на отделните получаващи процеси, както и за техния брой. Членството към такава група е динамично, т.е. присъединяването и напускането е позволено по всяко време. Нямаме гаранция за пристигане на съобщенията, както и гаранция за подредба. За **IP multicast** е характерно **свързаност във времето**, но **липса на такава по отношение на пространството**, т.е. не е необходимо изпращача да идентифицира получателя.

Отдалеченото извикване (Remote Invocation) разглежда три парадигми за разпределено програмиране. **Протоколите заявка-отговор** предоставят олекотена и много малка поддръжка за клиент-сървър изчисления. Този тип протоколи обикновено се използват най-вече във **вградените системи**, където закъснението, предизвикано от комуникационните механизми, трябва да бъде силно ограничено.

Remote Procedure Call (RPC) се базира на предоставянето на интерфейси, специфициращи процедури, предлагани от друг процес. По този начин се скрива реализацията на другия модул. Интерфейсите се дефинират посредством **Interface Definition Language (IDL)**, който е проектиран с цел да позволи извикването на процедура от един език в друг език.

Семантиките на RPC са:

1. **maybe** – отдалечената процедура се изпълнява веднъж или въобще не се изпълнява
2. **at-least-once** – клиентът получава резултат и знае, че процедурата се изпълнява поне веднъж или получава информация за липса на резултат
3. **at-most-once** – клиентът получава резултат и знае, че процедурата се е изпълнила най-много веднъж, или получава информация за липса на резултат

Remote Method Invocation (RMI) е обектно-ориентирана парадигма за отдалечено извикване на методи. Приликите с **RPC** са това, че и двете поддържат програмиране с интерфейси, базират се на протокол заявка-отговор и поддържат идентично ниво на прозрачност. Разликите идват от това, че RMI използва **обектно-ориентирана парадигма** и дава възможност за **предаване на параметри като референции към обекти**.

Реализацията на RMI предоставя **компоненти** (*proxy*, *skeleton*, *dispatcher* и др.), които скриват детайлите по маршализиране, размяната на съобщения и пряката работа с отдалечени обекти на клиента и сървъра.

Разпределени обекти и компоненти

Ключовата характеристика на **разпределените обекти** е, че те позволяват да се адаптира **обектно-ориентиран програмен модел** за разработката на разпределена система и така скриват сложността от разпределеното програмиране. В този метод комуникиращите помежду си единици са **обектите**. Комуникацията между обектите се осъществява най-често посредством **отдалечено извикване на методи (RMI)** или алтернативна парадигма (разпределени събития).

Преимуществата на подхода са:

1. **Енкапсулацията**
2. **Абстракция на данни** и **скриване на реализация**
3. Възможност за реализиране на **динамични и разширяеми решения**

Съществуват **редица платформени решения**, базирани на разпределени обекти като например **Java RMI** (използва се само при разработване на Java приложения), **CORBA** (предлага независимо от програмния език решение) и др.

Сред недостатъците на този подход са, че:

1. Интерфейсите на обектите **не описват от какво зависи имплементацията** им, което ги прави трудни за разработка и поддържане
2. Програмистите трябва да са наясно с **детайли от платформата**, т.е. от по-ниско ниво
3. Програмистите трябва да са наясно с **нефункционални системни услуги** по отношение на **сигурност**, **транзакции**, **репликация** и др.
4. Програмистите трябва **ръчно да внедряват всеки един обект**, което води до допускане на грешки

Горните четири недостатъци са довели до появата на **разпределените компоненти** и **компонентно-базираните подходи** за разработване на разпределени системи. **Софтуерен компонент** наричаме композитен модул със специфициран под формата на договор интерфейс и наличие единствено на експлицитни контекстни зависимости. Елементите на договора са два вида – **множество от осигурени интерфейси (provided interfaces)** и **множество от необходими интерфейси (required interfaces)**.

Централна роля при този подход заема понятието **контейнер**, който осигурява управляема хостваща среда за компонентите от страна на сървъра. Той гарантира **разделяне на приложната логика на компонента от функциите на разпределената среда**. Скрива директния достъп до компонента и предприема подходящи действия, за да осигури **поддръжка** на разпределеното приложение.

Примери за платформени решения, базирани на разпределени компоненти, са **Enterprise Java Beans (EJB)** и **CORBA Component Model (CCM)**.

Уеб услуги – дефиниции, шаблони за комуникация. Стандарти за уеб услуги – SOAP, UDDI, WSDL

Интерфейс на уеб услуга наричаме колекция от операции, които могат да бъдат използвани от клиенти през Интернет. Самата **уеб услуга** дефинираме като **софтуерно приложение**, което се **идентифицира по URI** и което се състои от **интерфейси на уеб услуги** като **XML артефакти**. Операциите може да се имплементират от различни ресурси – програми, обекти, бази от данни и др. Уеб услугите могат да бъдат управлявани от уеб сървър.

Ключова характеристика на повечето уеб услуги е, че те могат да обработват **XML-форматирани SOAP съобщения** или като алтернатива да се използва **REST подхода**. Всяка уеб услуга разчита на свое описание, за да се справи с характеристиките на съобщенията, които получава. Целта на уеб услугите е да се отдели изпълнението на разпределени изчисления от начина, по който те са имплементирани.

Шаблоните за комуникация, които се срещат по отношение на уеб услугите, са:

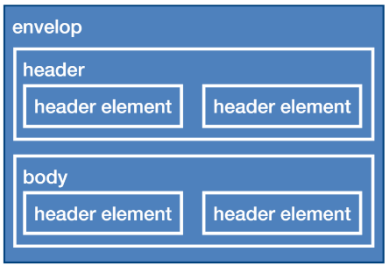
1. **Асинхронна комуникация** – обмяна на съобщения, при която изпращането на заявка не изисква незабавен отговор
2. **Синхронна комуникация** – обмяна на съобщения, при която се изисква клиентът да блокира до получаване на отговор; използват се протоколи от тип заявка-отговор
3. **Комуникация, базирана на събития** – получаване на съобщения при настъпване на определено събитие (с предварително абониране)

При всички случаи обаче имаме независимост на уеб услугите от програмната среда.

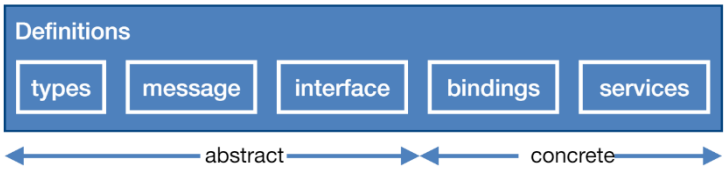
SOAP (Simple Object Access Protocol) е **протокол**, който дефинира **схема за използване на XML при представянето на заявки и отговори**, както и **схема за обмяна на документи**. Той позволява **синхронна и асинхронна комуникация** през Интернет. Оригинално **SOAP** е бил базиран на **HTTP**, но настоящата версия може да използва редица транспортни протоколи като **SMTP**, **TCP** или **UDP**.

Спецификацията на SOAP дефинира:

- начина за **представяне на съобщенията в XML**
- начина, по който **двойка съобщения могат да се комбинират** за комуникация от тип заявка-отговор
- правилата за **обработка на XML елементите в съобщенията** от получателите
- начина за **използване на HTTP и SMTP за комуникация** със SOAP съобщения



WSDL (Web Service Description Language) се използва за **описание на уеб услуги**. Той дефинира **XML схема за представяне на компонентите** от описанието на услугата – *definitions*, *types*, *message*, *interface*, *bindings*, *services* и др. Състои се от **абстрактна част** и **конкретна част**.



UDDI (Universal Description Discovery and Integration) е **директорийна услуга**, която е предназначена за **описание на организации и предлаганите от тях услуги**. Дефинира **стандарт за представяне на данновите структури в регистъра**. **UDDI API** се състои от няколко **интерфейса** – за търсене и получаване на информация, за нотификация, за публикуване и обновяване. **Структурата на UDDI регистъра** се състои от един или няколко сървъра, всеки от които притежава копие на данните.

Държавен изпит – спец. „Софтуерно инженерство“

Тема №23 – Планиране на проекта – същност и основни елементи, обхват на проекта, времеви и финансови ресурси. Дейности по управление и контрол, методи и средства за създаване на план-график на проекта.

Анотация: *Изложението по въпроса трябва да включва следните по-съществени елементи:*

1. Планиране на проекта – същност и основни елементи на плана на проекта

2. Планиране на обхвата на проекта. Определяне на структура на работа (WBS). План на контролните точки.

3. Планиране на време за изпълнение на задачите, бюджет и необходими ресурси, дейности по управление и контрол

4. Създаване на план-график на проекта. Методи и средства за създаване на план-график. Метод на критичния път. Метод PERT. GANTT диаграми.

Планиране на проекта – същност и основни елементи на плана на проекта

Проектът е временно начинание, което се изпълнява от една или повече организации, за да се реши определена необходимост или проблем и да се реализира подобрение на дейността или промяна. **Целта** на всеки проект е да се произведе уникален продукт или услуга.

Основните фази на жизнения цикъл на проекта са:

1. Инициализация на проекта
2. Планиране на проекта
3. Изпълнение и контрол на целия проект
4. Завършване на проекта

Преди да се започне същинското планиране на проекта, трябва да се отговори на следните **основни въпроси**:

- Какво трябва да се направи?
- Как трябва да се направи?
- Кога трябва да бъде направено?
- Колко ще струва и какъв ще е бюджетът на проекта?
- Кой ще го направи?
- Какви ресурси са необходими за проекта и кога?
- Какво качество се очаква от проекта?
- Какви са рисковете на проекта?
- Какви ще са основните комуникации по проекта?

На част от тези въпроси вече е отговорено, но по време на планирането тези отговори се детайлизират още повече. **Процесът на планиране** не започва отначало, а продължава детайлизацията на вече получените резултати и оценки и ги превръща в работещ план на проекта.

Процесът на планиране се разделя на четири основни групи дейности:

1. **Определяне на съдържанието на плана за управление на проекта**
2. **Определяне на структурата на работа по проекта (Work Breakdown Structure – WBS)**
3. **Планиране на времето за изпълнение на задачите** – създаване на план-график на проекта
4. **Планиране на ресурсите, разходите и бюджета на проекта**

Планирането на проекта продължава и детайлизира направеното по време на инициализацията на проекта и отразява определени факти и действия, които се извършват в процеса на изпълнение на проекта.

Основни елементи на плана на проекта са:

1. **Цел на плана** – описва целта на плана на проекта
2. **Бизнес цели на проекта** – намаляване на разходи, брой грешки, време и др.
3. **Основни резултати, очаквани от проекта** – връзка с бизнес целите
4. **Обхват и мисия** – изясняват се изискванията
5. **План-график на проекта** – дефинират се контролни точки
6. **Разходи и ползи** – описват се всички разходи по проекта
7. **Подход за управление на риска** – включва описание на идентифицираните рискове
8. **Списък на доставките по проекта (deliverables list)** – основните формални доставки
9. **Подход за управление на качеството** – стратегия за гарантиране на качеството
10. **Изисквания към ресурсите, ролите и отговорностите** – човешки ресурси, знания, опит
11. **Управление на комуникациите** – описва вътрешните и външните комуникации в проекта
12. **Оценка на работата по проекта** – за подобряване на процесите

Планиране на обхвата на проекта. Определяне на структура на работа (WBS). План на контролните точки

Обхватът е определен от стратегическите цели на проекта и може да се отнася до:

- **Обхват на продукта (product scope)** – описание на характеристиките и функциите на продукта или услугата
- **Обхват на проекта (project scope)** – работата, която трябва да се извърши за създаване на продукта или услугата

Планирането на обхвата на проекта има за цел създаването на писмен план за обхвата на проекта и на неговия продукт, служещ за бъдещата му реализация. Основните процеси от управлението на обхвата са:

1. **Планиране на обхвата и мисията на проекта** – подготвя се план за управление на обхвата и мисията на проекта, който документира как те да се определят, верифицират и контролират и как да се създаде структура по задачи
2. **Определяне на обхвата и мисията на проекта** – разработване на детайлно описание на обхвата и мисията на проекта като основа за бъдещи решения
3. **Създаване на структура по задачи (WBS)** – разделяне на съставни по-малки и по-лесни за управление части на основните резултати и работа по проекта
4. **Верифициране на обхвата и мисията на проекта** – формализиране на одобряването на готовите проектни резултати
5. **Контрол на обхвата и мисията на проекта** – контролиране на промени в обхвата и мисията на проекта

Планирането на обхвата на проекта започва от фазата на събиране на изискванията. Събирането на изискванията определяме като процес на определяне и документиране на нуждите на заинтересованите страни за постигане на целите на проекта. Изискванията стават основата на структурата на работата по проекта (WBS).

Изискванията се разделят на **две категории**:

- **Изисквания към проекта** – бизнес изисквания, изисквания за управление на проекти, изисквания за доставка и др.
- **Изисквания към продукта** – информация за техническите изисквания, изисквания за сигурност, изисквания за производителност и др.

Съществуват следните **техники за събиране на изискванията**:

1. **Интервюта** – директен разговор
2. **Фокус групи** – събиране на заинтересовани лица
3. **Ръководени семинари** – фокусирани сесии
4. **Групови креативни техники** – мозъчна атака, делфи техника, картографиране на идеи
5. **Техники за групово вземане на решения** – единодушие (консенсус), мнозинство (обикновено – 50% + 1, квалифицирано – 75% + 1), плурализъм, диктатура
6. **Въпросници и анкети** – подходящи за статистически анализ
7. **Наблюдения** – при трудност при формулирането на изискванията
8. **Прототипи** – ранна обратна връзка

Дефинирането на обхвата на проекта е процес на разработване на подробно описание на проекта и продукта. По време на планирането обхватът на проекта се определя и описва с по-голяма специфичност, тъй като е известна повече информация за проекта.

Съществуват следните **техники за дефиниране на обхвата**:

1. **Експертна преценка** – използва се за анализ на информацията, необходима за разработване на описанието на обхвата на проекта
2. **Анализ на продукта** – за проекти, където се доставя продукт
3. **Идентифициране на алтернативи**
4. **Ръководени семинари**

Създаването на структура на работата (WBS) е процес на разделяне на резултатите от проекта (deliverables) и работата по проекта на по-малки, по-лесно управляеми компоненти. Йерархичната структура на работата е свързана с нейното декомпозиране на компоненти, които трябва да бъдат изпълнени от екипа за изпълнение на целите на проекта и създаване на необходимите резултати, като всяко ниво по-навътре представя все по-детайлна дефиниция на проектната работа. Планираната работа се съдържа в компонентите на структурата на работата (WBS) от най-ниско ниво, наречени *работни пакети*.

Декомпозицията става чрез подразделяне на резултатите на проекта (project deliverables) на по-малки, по-лесно управляеми компоненти, докато работата и резултатите бъдат определени до нивото на работен пакет. Нивото на детайлност на работните пакети варира в зависимост от размера и сложността на проекта.

Работните пакети са основа за планиране и контрол на разходите и времето. За всеки работен пакет се извършва остойностяване и оценка на времетраенето. Сумираните очаквани разходи за всеки пакет плюс останалите непреки разходи и разходи за управление формират предварителния бюджет на целия проект. Този бюджет и общият календарен план за изпълнение на дейностите във времето са еталон, спрямо който впоследствие се извършва оценката за производителността и изпълнението на плана

Структурата на работните пакети е основа за проектния контрол. По време на изпълнение на проекта се извършва постоянно сравнение и оценка на действително извършените работи и календарния план. Аналогични сравнения се извършват за планираните разходи и извършените действителни разходи.

Описанието на работен пакет обикновено може да включва:

- Общо описание на извършваните дейности
- Предшестващи дейности – като условие за започване
- Ръководител и екип
- Спецификации на продукта
- Подизпълнители и доставчици – ако има такива
- Оценка за ресурсите
- Общи разходи
- Календарния план и контролните точки
- Специфични резултати

Контролна точка (milestone) е точка във времето, която е ключова или събитие от средна важност от жизнения цикъл на проекта, предварително заложено в плана на проекта. Ясно определените контролни точки през жизнения цикъл на проекта са много важни, защото в тези моменти различните заинтересовани лица се срещат и обсъждат прогреса на проекта и планираните дейности. Целта на тези събития е не само да се демонстрира напредъка на проекта, но и да се постигне:

- Синхрон между очакванията на заинтересованите лица и постигнатото от проектния екип относно изискванията, проектирането, дизайна и планирането (служат за валидация)
- Одобрение (или искане за преработване) на междинни резултати
- Определяне на възникналия риск, нови ситуации и условия
- Разглеждане и одобрение на следващите действия по плана

Контролните точки трябва да имат добре дефинирани очаквания и да представят конкретни резултати. Това не изключва повторното договаряне на целите на контролните точки, след като проекта е получил одобрение за промени на изискванията, проектирането / дизайна и планирането.

Контролните точки са три вида:

1. **Основни контролни точки** – в края на всяка фаза от жизнения цикъл на проекта
2. **Второстепенни контролни точки** – при завършване на сложна дейност с определен краен резултат
3. **Контролни точки, отчитащи състоянието** – например преди проверка или в края на годината

Последователността при планиране на контролните точки е следната:

1. Постига се общо съгласие за крайната цел
2. Генерират се варианти на контролни точки чрез „мозъчна атака“
3. Обсъждат се алтернативите – обмисляне, добавяне и редактиране
4. Генерират се предложения за пътища на резултатите – експерименти
5. Изчертават се логическите зависимости като се започва от крайната цел
6. Построява се окончателна схема на плана

Планиране на време за изпълнение на задачите, бюджет и необходими ресурси, дейности по управление и контрол

Планирането на времето за изпълнение на задачите се състои от следните етапи:

1. **Дефиниране на дейностите** – процес по откриване на конкретни дейности, които да се извършват за доставяне на крайните резултати (project deliverables)
2. **Определяне на последователност на действията (създаване на мрежова диаграма)** – процес по откриване на взаимовръзките между дейностите
3. **Оценка на ресурсите за дейностите**
4. **Оценка на продължителността на дейностите**
5. **Разработване на план-график** – на база на всичко проучено от по-ранни етапи
6. **Контролиране на графика** – мониторинг на развитието на проекта спрямо графика и извършване на промени при необходимост

За да се планират разходите и да се направи бюджетът на един проект е необходимо да се определят какви **ресурси** (човешки и материални) са на разположение за работа по неговите задачи, работата и свързаните с нея разходи за всяка задача и общите разходи за целия проект.

Стъпки за определяне на ресурсите, необходими за изпълнение на всяка задача:

1. **За всяка задача се определят необходимите ресурси:**
   1.1. ***Човешки ресурси (труд)*** *– хората от екипа на проекта с техните умения*
   1.2. ***Материални ресурси*** *– софтуер и хардуер за изграждането на технологичната среда*
   1.3. ***За подизпълнител***
   1.4. ***Индиректни (режийни) разходи и др.***
2. **Определя се количеството на ресурсите, необходими за изпълнение на задачата в определеното време**
3. **Определя се кога и колко време ресурсите са на разположение на проекта**

Разходите се състоят от цената на ресурсите. Всички разходи се сумират и определят **бюджета на проекта**. Той е разпределен по различните задачи и продукти/резултати, за да се създаде основа за измерване и оценка на проекта според неговите разходи. Бюджетът на проекта се оценява на базата на определен период. Нуждае се от одобрение.

Целта на **дейностите по управление и контрол** е минимизирането на разликата между резултатите, които са планирани и обещани в проекта и резултатите, които реално са постигнати от проекта. В това отношение се осъществяват различни видове анализи.

Основен предмет на контрола на проекта са:

- Обхватът на проекта
- План-графикът на проекта
- Разходите (бюджета) на проекта
- Качеството на проекта

Създаване на план-график на проекта. Методи и средства за създаване на план-график. Метод на критичния път. Метод PERT. GANTT диаграми

Създаването на план-график включва пет основни стъпки, които се изпълняват в итерация до постигане на оптимален резултат:

1. **Определяне на предварително времетраене на задачите** – при идеални условия с неограничени ресурси и пълно натоварване
2. **Определяне на предварителен (идеален) график според последователността на задачите**
3. **Идентификация на ресурсите и времето, през което те ще работят по задачите** – определяне на работата по изпълнение на всяка задача
4. **Определяне на реалното време (duration) на задачите**
5. **Създаване/промяна на основния план-график**

Сред **методите и средствата за създаване на план-график** са:

1. Метод на критичния път (Critical Path Method – CPM)

Критичен път (critical path) наричаме последователността от задачи, които определят цялостното времетраене на проекта (project duration). Това са задачите с минимално плаващо време (не могат да се отлагат). В мрежовата диаграма с последователността на задачите това е най-дългият път (последователност) от задачи в проекта.

Методът на критичния път калкулира еднозначно следните дати за всяка задача:

- Ранен старт (Early Start – ES)
- Късен старт (Late Start – LS)
- Ранен край (Early Finish – EF)
- Късен край (Late Finish – LF)

на базата на последователна мрежова логика и определеното време (duration). Той се фокусира върху калкулацията на плаващото време (float), за да определи кои задачи имат най-малка гъвкавост по отношение на графика. Плаващото време е разликата между реалното време за изпълнение на една задача и планираното време за нейното изпълнение. Задачите на критичния път обикновено имат минимални или въобще нямат плаващо време.

2. Метод PERT (Program Evaluation and Review Technique)

PERT е метод за оценка, който използва средни претеглени стойности за времетраенето на задачите, за да изчисли техните времена. Изчислява се по следната формула:

$$\frac{\text{Оптимистично време} + 4 * \text{Най} - \text{вероятно време} + \text{Песимистично време}}{6}$$

Различава се от метода на критичния път по това, че методът на критичният път използва най-вероятното време.

3. GANTT диаграми (Gantt Charts)

При **диаграмите на Гант** времетраенето на задачите се изобразява с правоъгълници във времето за изпълнение на проекта. Това е най-използваният и лесно разбираем метод, особено за определяне на статуса на проекта и неговия прогрес. Лесно представя план-графика и зависимостите между задачите.

Държавен изпит – спец. „Софтуерно инженерство“

Тема №24 – Използване на XML за структуриране, валидация, обработка и представяне на документно съдържание.

Анотация: *Изложението по въпроса трябва да включва следните по-съществени елементи:*

1. Добре-структуриран XML – основни концепции, XML йерархии, синтактични правила. XML пространства от имена.

2. XML валидация чрез Document Type Definitions (DTD) – цели на валидирането, DTD структура, синтаксис.

3. XML валидация чрез XML Schema – спецификации, типове данни, фасети, структури. Сравнение с DTD.

4. Използване на XSLT (eXtensible StyleSheet Language Transformations) и XPath за алокиране, манипулиране и представяне на XML съдържание.

5. Използване на DOM (Document Object Model) и SAX (Simple API for XML) за обработка на XML документи – основни интерфейси на DOM и SAX и начини за използването им. Сравнение между DOM и SAX.

Добре-структуриран XML

XML (eXtensible Markup Language) е свободно разпространяван разширяем метаезик за описателно маркиране на текстови документи, утвърден от W3C. Данните в **XML** документ са заобиколени от тагове за маркиране, които описват самите данни. В **XML** те не са ограничени, както в HTML, т.е. **XML** позволява на потребителя да създава нови тагове.

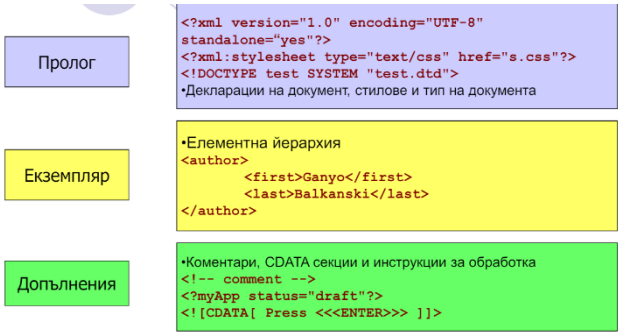
Основните характеристики на един документ са:

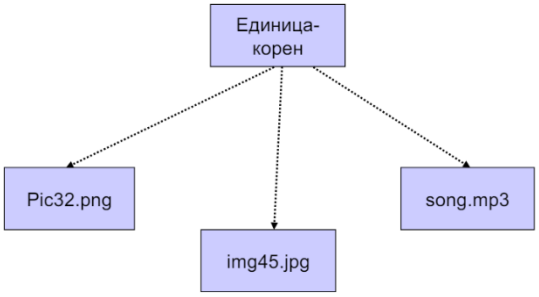
1. **Съдържание** – информация, съдържаща текст, графика, аудио или видео
2. **Структура** – подялба и последователност на информационните части
3. **Оформление** – онагледяване на съдържанието и структурата на документа
4. **Метаданни** – описание семантиката на съдържанието

XML заедно със своите стилови езици е ориентиран към всички основни характеристики на документа, които са изброени по-горе. **Съдържанието** може да не бъде само текстово, а и двоично, към което трябва да бъде реферирано. **Визуалното оформление** може да бъде постигнато с помощта на стилови трансформации посредством XSLT или чрез използване на каскадни стилови множества (CSS). Много XML приложения се използват за дефинирането на семантични **метаданни** за даден домейн, което е неотменна черта при прехода към семантичен Уеб.

Ефективната комуникация между хората разчита на организиране на идеи и факти в смислени информационни структури. XML предлага мощни средства за представянето на такива **структури** в йерархична форма, удобна за ползване както от хора, така и от софтуерни приложения. Организацията се представя в логическа структура на документа, като най-важните компоненти на тази структура са XML елементите. **Логическата структура** на XML документ може да се представи като планарна и дървовидна йерархия от елементи. Елементите формират йерархията, като са вложени в корена на документа по коректен начин без припокриване. Освен елементи, други компоненти на логическата структура са декларации, коментари, CDATA секции и инструкции за обработка. Така логическата структура се състои

от пролог, екземпляр и допълнение. За разлика от логическата структура на XML документа, физическата структура може да се разпростира върху повече от един файл. Типичен пример за това е включването на бинарно съдържание в XML документ като графика или аудио- и видеозаписи. Тъй като логическата структура е ориентирана към представяне на текстово съдържание, няма възможност в нея да се включват мултимедийни ресурси. Това обаче е възможно да стане във **физическата структура**, която е изградена на базата на информационни единици (entities). Всеки XML документ включва основно текстово съдържание, обособено като документна единица в корена на документа (единица корен). В рамките на тази единица може да се реферират неограничен брой текстови или бинарни информационни единици, външни за единицата корен на документа.





XML спецификацията на W3C описва каква трябва да бъде структурата на XML документите според формална XML граматика. Всеки софтуерен модел, предназначен само за четене на XML документи, се нарича **XML процесор (парсер)**. XML документът се подава на XML процесора под формата на текстови файлове с произволно име и разширение *.xml*. XML процесорът чете маркирания текст, разделя маркирането от текстовото съдържание и проверява дали текстът е в съответствие с правилата, определени за всички XML документи в XML спецификациите. Ако документът е конструиран или оформен така, като предписват тези правила, то той се нарича **добре конструиран** или **добре оформен XML документ** (**well-formed XML document**). Проверката за добро конструиране на документа се прилага само за маркирания текст, а не за външно текстово съдържание, реферирано в маркирания текст, както и не за текстово съдържание, за което е посочено изрично да бъде пропуснато при обработката на XML документа.

Всеки XML документ е изграден от следните компоненти:

- **Маркер (таг)** – обхваща името на елемент, оградено със символите “<” и “>” от двете му страни

Пример: **<elementname>** → отварящ таг **</elementname>** → затварящ таг

- **Елемент** – включва отварящия и затварящия таг, като между тях има някакво съдържание. Типът на съдържанието е Parsed Character DATA (PCDATA). Освен текст, съдържанието може да бъде и друг елемент, тъй като те могат да бъдат влагани. Важно е, че е възможно да има само един елемент корен.

Пример: **<elementname>**Content of the element**</elementname>**

Пример: **<author>**

<first>Agatha**</first>**

<last>Christie**</last>**

```
</author>
```

- **Атрибут** – допълнителна информация за елемент. Задава се като наредена двойка име – стойност. Стойността на атрибута се огражда от единични или двойни кавички. Един елемент може да има множество атрибути, като между тях няма фиксирана подредба.

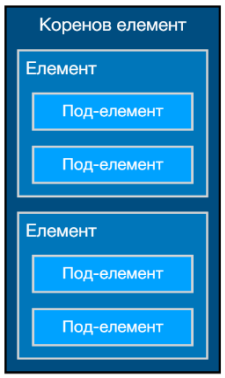
Пример:

```
<author gender="female">
    <first>Agatha</first>
    <last>Christie</last>
</author>
```

Елементната структура на един XML документ представлява **йерархия**. Тя може да се представи по два начина: **планарно** и **дървовидно**.

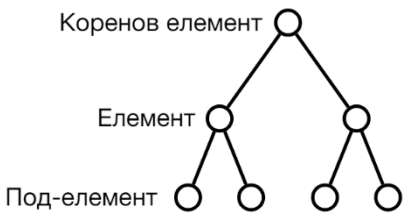
Планарната йерархия на един XML документ представлява вертикално представяне на подредбата и релациите на елементите. Тя използва влагане за представяне на това, че вложеният елемент е дете на елемента, в който е вложен. Това може да стане например чрез вложени една в друга правоъгълници. Коректността изключва припокриване на елементи, което може да се получи, ако елемент наследник бъде завършен със затварящ таг след завършването на елемента родител. Друг случай на припокриване имаме, когато един елемент е без затварящ таг.

Пример:



Дървовидната йерархия на един XML документ използва дървовидната структура за по-добро представяне на релацията „родител-дете“, както и за по-добрата визуализация на онези елементи, които се намират на едно и също ниво спрямо влагането. Корен на дървото е задължително само един елемент, след което се разделя на елементите, които се намират на следващото ниво и т.н. Следващото ниво се състои от елементите, които се намират в елементите под корена и т.н. до листата, които представляват най-вложените елементи в дадена йерархия и в тях няма други вложени елементи.

Пример:



Според XML спецификацията на W3C един добре конструиран (well-formed) XML документ трябва да отговаря на следните **правила за коректни елементи**:

1. На всеки отварящ таг трябва да съответства затварящ таг със същото име.
2. Трябва да съществува точно един-единствен елемент на най-горно ниво (root), който да съдържа всички останали елементи.
3. Редът на влагане на елементите не трябва да бъде нарушаван, т.е. да няма припокриване на елементи в документната йерархия.
4. Имената и съдържанието на елементите са чувствителни към регистъра (case-sensitive).
5. XML елементите трябва да имат имена съгласно принципите за конвенция за именоване:
   - ***5.1.*** *Имената трябва да започват с малка или главна буква или „_“.*
   - ***5.2.*** *По-нататък в имената, освен букви и „_“, могат да се използват цифри, „-“ и „.“.*
   - ***5.3.*** *Имената не могат да започват с думите XML, xml, Xml, xmL и т.н.*
   - ***5.4.*** *Празни пространства се допускат само след имената на елементите, т.е. преди знака „>“.*
   - ***5.5.*** *Използването в имената на символа за двоеточие „:“ е разрешено, но е резервирано за отделяне на името на пространството от имена от локалното име на тага.*
   - ***5.6.*** *В XML имената не могат да бъдат използвани други символи освен букви (вкл. и от азбуки, различни от латинската), цифри, празно пространство, „_“, „-“, „:“ и „.“.*
   - ***5.7.*** *В XML имената празното пространство може да бъде интервал, нов ред, или табулация, и е допустимо да се зададе само преди затварящия разделител „>“.*
   - ***5.8.*** *Използването на символите „<“ и „>“ е резервирано за разделители.*
   - ***5.9.*** *Символите „&“ и „;“ се използват за указване на реферирane към съдържание посредством единица (entity) – наричана още семантична единица или същност.*

Според XML спецификацията на W3C един добре конструиран (well-formed) XML документ трябва да отговаря на следните **правила за коректни атрибути**:

1. Всеки атрибут трябва да има стойност, дори и тя да бъде празен стринг.
2. За разделител между два и повече атрибута се използва празно пространство, а не запетая.
3. Възможно е едновременното използване на кавички и апострофи за ограждане на стойностите на атрибутите на един елемент. Не е допустимо обаче, ограждането на стойност на един атрибут с кавички и апостроф или обратното. Възможно е използването на кавички, ако в стойността на атрибута има използвани един или няколко апострофа.
4. Дублиращи се атрибути (в същия елемент) не са разрешени в добре оформен XML документ.
5. Атрибутите могат да следват произволен ред на подреждане в елемента. Това е разрешено да бъде така, защото те описват характеристики на елемента, които не са структурирани и нямат релация за подреждане.

Други XML конструкции са:

1. **Коментарите** – за анотиране на съдържанието, с цел анотиращата информация да остане налична не за обработващите документа програми, а за създателя на документа или други негови потребители

Пример: `<!-- this is a comment -->`

2. **CDATA секции** – съдържанието се игнорира от парсера и се подава към приложението така, както е зададено

Пример:

```
<![CDATA[
This text is ignored by the XML parser.
]]>
```

3. **Инструкции за обработка** – представляват начин за предаване на скрита информация към дадено приложение

Пример: **<?xml:stylesheet type=”text/css” href=”sample.css” ?>**

4. **XML декларация** – специална директива, посочваща версията на езика и опционално може да посочи и схемата за кодиране на съдържанието на документа (encoding)

Пример: **<?xml version=”1.0” encoding=”UTF-8”?>**

Използването на **еднакви имена** на XML елементи в различен контекст, който не е явно указан, може да доведе до проблеми с дублирани имена. Такива колизии могат да възникнат при:

- Смесване в един XML документ на маркирано съдържание от източници с различен контекст
- При работа с един и същ XML документ в различни обкръжения, където имената на елементите се интерпретират по различен начин

Пример: **<title>** → за указване на звание, титла, но и за означаване на заглавие в ***XHTML***

Това налага употребата на **пространства от имена** за елементите и атрибутите. Пространство от имена е абстрактна категория за група от имена. В рамките на един XML документ дадено пространство от имена може да се означава с префикс (освен, ако не е пространството по подразбиране). За осигуряване на единствеността на префиксите, означаващи пространства от имена, се използва само URL. Чрез URL се задава уникален адрес на Уеб ресурс и начин за достъп до него. Не е задължително условие означеният чрез URL ресурс да съществува в Интернет. Префиксът **xmlns** е име на пространство от имена, специално създадено за описание на пространствата от имена на XML проектантите. XML пространството от имена се отнася само за йерархията от елементи, за която е дефинирано, а не за всички оставащи до края на документа елементи. Възможно е и задаването на пространства от имена по подразбиране с цел избягване посочването на префикс за всеки елемент в йерархията. Важно е, че в един XML документ може да се използва само едно пространство от имена по подразбиране в даден участък на документната йерархия.

Пример за употреба на пространствата от имена:

```
<?xml version='1.0'?>
<wellFormed    xmlns:ref1="http://www.w3.org"
               xmlns= "http://www.w3.org"
               xmlns:ref2="http://www.hmmm.bg">
 <good a="1" b="2"/>
 <good a="1" ref1:a="2"/>
 <good ref1:a="1" ref2:a="2"/>
</wellFormed>
```

XML валидация чрез Document Type Definitions (DTD)

DTD (Document Type Definition) и **XML Schema** или **XSD (XML Schema Definition)** задават правила, съгласно които се определят имената на елементите и атрибутите, тяхната последователност, честота на срещане и др. Те осигуряват начин за проверка на неговата структура и съдържание, т.е. за валидация.

Валидацията е отнемащ време процес, но често спестява много проблеми на външните приложения и се извършва от специализиран процесор (валидиращ парсер). Всеки валиден документ е добре конструиран, но обратното не е задължително вярно.

Целите на валидирането са:

1. DTD описанието е споделено от валидиращия парсер, т.е. използва се многократно само едно описание
2. Външните приложения не извършват валидация, а използват оптимизиран за целта парсер
3. Подобрява се сигурността на работа на тези приложения – проблемите във валидните са значително по-малко в сравнение с останалите

Синтаксисът на DTD е базиран на описанието на регулярни граматики и е формално дефиниран. Файлът, който съдържа формалната дефиниция на определен тип на документ, трябва да има разширение ***.dtd*** и да бъде прикрепен по определен начин към XML документа. Освен като отделен документ, DTD дефинициите могат и да се вграждат в XML документа. Възможна е и комбинация от двата подхода.

Най-общо структурата на едно DTD описание е следната:

- **DOCTYPE** – информира парсера, че с документа има асоцииран DTD. Задължително се поставя в началото на документа
- **Тяло** – съдържа декларациите на елементи, атрибути, единици, нотации и др.

Пример:

```
<!DOCTYPE recipe[
.....
.....
.....
]>
```

Дефиниране на елементи `<!ELEMENT ... >`

- Използват се за декларация на нови елементи и модела на тяхното съдържание
- Състои се от три части – ключовата дума ELEMENT, името на елемента и модела на съдържанието
- Съдържанието може да бъде:
  - **Елементно** – съдържа друго елементи, които трябва да имат дефиниция в DTD

Пример: `<!ELEMENT elementName (child1, child2, child3)>`

  - **Смесено** – в тази декларация ключовият елемент #PCDATA трябва да предхожда всички други елементи. Различните възможности за съдържание се отделят чрез знака „|“

Пример: `<!ELEMENT elementName (#PCDATA|child1|child2)>`

- **Празно** – указва се чрез ключовата дума EMPTY

Пример: **`<!ELEMENT elementName EMPTY>`**

- **Произволно** – указва се чрез ключовата дума ANY

Пример: **`<!ELEMENT elementName ANY>`**

- В дефинициите на съдържанието може да използваме и **индикатори** за брой срещания:
  - **+** → указва, че даденият елемент трябва да се среща 1 или повече пъти
  - ***** → указва, че даденият елемент трябва да се среща 0 или повече пъти
  - **?** → указва, че даденият елемент трябва да се среща нула или само веднъж

Пример: **`<!ELEMENT elementName (child1*, child2?, child3)>`**

Дефиниране на атрибути `<!ATTLIST … >`

- За всеки един DTD елемент може да бъде дефиниран атрибут или списък от атрибути.
- Декларацията на атрибут се задава с ключовата дума ATTLIST и се състои от три основни части:
  - Ключова дума ATTLIST
  - Името на елемента, на когото тези атрибути принадлежат
  - Списък с декларациите на атрибутите – за всеки атрибут задължително се задават име, тип и стойност
- Тип на атрибут:
  - **CDATA** – произволна текстова информация
  - **Именована група** – една от изброените в скоби стойности
  - **ID** – атрибутите от този тип трябва да имат уникална стойност
  - **IDREF/IDREFS** – стойността трябва да съвпада със стойност на атрибут от тип ID
  - **NMTOKEN/NMTOKENS** – валидно XML име
  - **ENTITY/ENTITIES** – стойността на атрибут от този тип трябва да е име на ENTITY
  - **NOTATION** – определя различни от XML типове данни
- Стойност на атрибут:
  - **Стойност по подразбиране** – текстът се загражда в кавички
  - **#IMPLIED** – атрибутът не е задължително да присъства в XML документа
  - **#REQUIRED** – атрибутът задължително трябва да присъства в XML документа
  - **#FIXED** – стойността на атрибута трябва задължително да съвпада с текста, ограден в кавичките (само една разрешена стойност)

Пример: **`<!ATTLIST list version CDATA #FIXED "1.0">`**

Дефиниране на единици `<!ENTITY … >`

- Единиците се дефинират чрез специална декларация (ENTITY) и се използват през референция, която я указва
- В един XML документ ENTITY референциите се използват като указател към текст или външен ресурс
- Типове единици:
  - **Общи (General)** – декларирани в DTD, реферирани в XML документите
  - **Параметрични** – декларирани в DTD, реферирани единствено в декларациите на DTD

- **Вътрешни (Internal)** – съхраняват се в XML документа и представляват текстово съдържание; реферират текст, който е дефиниран в DTD документа
- **Външни (External)** – съхраняват се извън XML документа и представляват или текстово, или двоично съдържание
- **Неанализирани (външни единици, съдържащи не- XML съдържание)** – служат за вграждане на не-XML данни (например графика и др.); всяка неанализирана единица има асоциирана нотация, определена по име; нотацията информира приложението за формата на данните на единицата, така че съдържанието да може да бъде обработено адекватно

Дефиниране на нотация `<!NOTATION ... >`

- Нотациите са инструкции, които дават информация на XML парсера как да обработи не-XML данни
- Съответният формат се асоциира с външна програма, която парсерът подава на приложението, както и информация къде да я намери
- Нотацията се използва, за да специфицира данни като например файлове от тип ***image/gif***, ***image/jpeg*** и др.

Пример:

```
<!NOTATION jpeg SYSTEM "image/jpeg">
<!ENTITY turing_getting_off_bus
                SYSTEM "http://www.tur.org.uk/aa.jpg" NDATA jpeg>
<!ELEMENT image EMPTY>
<!ATTLIST image source ENTITY #REQUIRED>
```

XML валидация чрез XML Schema

Въпреки че определянето на типа на документа може да се използва за валидация на документа спрямо основните структурни правила, много приложения се нуждаят от по-мощен и по-изразителен език за проверка на валидността. **XML schema** на практика е XML документ, който описва структурата на друг XML документ. Тя е наследник на DTD, но предоставя много по-богати възможности за по-прецизна спецификация на XML структура. Има множество XML schema езици като най-разпространените са **XML Schema**, или наречен още **XSD (XML Schema Definition)**.

XML Schema е препоръка на W3C от 2004 г. Спецификацията е представена в следните части:

XML Schema Part 0: Primer – ненормативна въвеждаща част, целта на която е да предостави лесно четимо описание и ориентирана към бързото разбиране как да се създават схеми чрез XML Schema

XML Schema Part 1: Structures – дефинира структурите и ограниченията, които се прилагат в XML Schema

XML Schema Part 2: Datatypes – специфицира какви типове данни могат да се използват за елементи и атрибути

Част 1 на спецификацията на XML Schema дефинира следните структурни компоненти:

1. **Дефиниции на типове** – **`<simpleType>`, `<complexType>`**

2. **Декларации на атрибути** – **<attribute>**
3. **Декларации на елементи** – **<element>**
4. **Декларации на групи от атрибути** – **<attributeGroup>**
5. **Дефиниции на моделиращи групи** – **<group>**
6. **Декларации на нотации** – **<notation>**
7. **Анотации** – **<annotation>**

Декларациите (на елементи и атрибути) се използват за онези компоненти, които ще се използват в рамките на XML документа, а **дефинициите** (на нов тип, групи) – за онези компоненти, които ще се използват само в рамките на схемата.

Съществуват **два вида декларации** – **локални** и **глобални**. Глобалните са тези, които са директни деца на **<xs:schema>**, а локалните имат друг родителски елемент.

Основните елементи, които се споменават и в спецификацията на XML Schema са:

1. **<xs:schema>** – коренът на всеки схема документ
   1.1. **targetNamespace** – атрибут на корена, който указва пространството от имена, за което тази схема е предназначена
2. **<xs:element>** – чрез него се декларират елементи, които ще бъдат използвани в XML документа
3. **<xs:attribute>** – използва се за декларация на атрибути; най-често се използва в **<attributeGroup>** и **<complexType>**
4. <**xs:simpleType>** – дефинира елемент от прост тип; елементите от прост тип в една XML Schema са елементите, които са от даден предефиниран тип и нямат атрибути и поделементи; самото създаване на нов прост тип е на базата на съществуващите, като се обединяват различни типове или се добавят ограничения върху тях
5. **<xs:complexType>** – дефинира елемент от сложен тип; за целта съответния сложен тип трябва да бъде дефиниран и именован извън елемента или да бъде специфициран като поделемент. Сложните елементи са няколко вида:
   5.1. Празни елементи – без съдържание, поделементи и атрибути
   5.2. Елементи с атрибути – имат атрибути и могат да имат текстово съдържание
   5.3. Елементи, съдържащи поделементи – имат поделементи и могат да имат също и текстово съдържание, но нямат атрибути
   5.4. Елементи с текстово съдържание и атрибути – имат атрибути, поделементи, текстово съдържание
6. **<xs:attributeGroup>** – позволява декларацията на група от атрибути; ако се декларира на глобално ниво трябва да има име
7. **<xs:group>** – позволява да се групират елементи и така получената група да се преизползва на различни места. В една група могат да присъстват единствено елементи
8. **<xs:all>** – съдържа декларации на елементи, които трябва да присъстват в XML документа, независимо в какъв ред
9. **<xs:choice>** – съдържа декларации на елементи, от които точно един може да присъства в XML документа
10. **<xs:sequence>** – съдържа декларации на елементи, които трябва да присъстват в XML документа в точно указания ред (последователно)

Има две основни групи типове данни:

- **Примитивни (primitive) типове** – служат за основа при дефинирането на всички останали типове – **`string`**, **`boolean`**, **`float`**, **`double`**, **`duration`**, **`dateTime`**, **`time`**, **`date`**, **`gYearMonth`**, **`gYear`**, **`gMonthDay`**, **`gDay`**, **`gMonth`**, **`hexBinary`**, **`base64Binary`**, **`anyURI`**, **`QName`**, **`NOTATION`**
- **Вградени (built-in), производни типове** – използвани са други примитивни и производни типове – **`token`**, **`ID`**, **`IDREF`**, **`ENTITY`**, **`int`**, **`short`**, **`byte`**, **`long`**

Фасетът е определящ аспект на пространството от стойности за даден тип. Фасетите на даден тип данни служат да се разграничат онези аспекти на този тип, които се различават от другите типове данни.

Съществуват два вида фасети:

- **Фундаментални фасети** – дефинират типа
  - **Equal** – важи за всички видове данни
  - **Order** – за някои типове
  - **Bounds** – за долна и горна граница
  - **Cardinality** – крайност или безкрайност
  - **Numeric** – дали е число или не
- **Ограничаващи фасети** – ограничават стойността на типа
  - **`minInclusive/minExclusive`** – задава, че стойността на елемента трябва да по-голяма или равна / по-голяма от дефинираната тук
  - **`maxInclusive/maxExclusive`** – задава, че стойността на елемента трябва да е по-малка или равна / по-малка от дефинираната тук
  - **`length`** – задава колко на брой знаци може да съдържа елементът
  - **`minLength/maxLength`** – задава минималнкия/максималния брой знаци, които може да съдържа елементът
  - **`pattern`** – стойността на елемента трябва да отговаря на даден регулярен израз
  - **`enumeration`** – изброява стойностите, които елементът може да приема
  - **`fractionDigits`** – задава броят на цифрите от дясно на десетичната запетая в едно число с плаваща запетая
  - **`totalDigits`** – задава минималния брой цифри, които може да съдържа елементът
  - **`whiteSpace`** – задава как да се обработват празните пространства

DTD и XML Schema се използват с едни и също предназначение. Поради това е важно да се отбележат разликите между двата езика за дефиниране на XML структура. Разликите са следните:

- XML Schema използва XML-базиран синтаксис, докато DTD има специфичен синтаксис, повлиян от SGML DTD.
- DTD използва по-стегнат и компактен синтаксис в сравнение с XML Schema.
- XML Schema дава възможност да се създават типове данни, които след това да се използват при специфицирането на елементи и атрибути. Върху тях могат да бъдат специфицирани различни ограничения
- С DTD за разлика от XML Schema честотата на срещане на даден елемент не може да се зададе с произволно цяло число, а може да приема само стойностите – нула, едно или безкрайност

- В DTD даден елемент не може да бъде асоцииран с избираем списък от стойности, а е възможно само за атрибутите. В XML Schema това е възможно, като се дефинира такъв тип данни и след това съответният елемент се асоциира с него
- XML Schema поддържа пространства от имена, докато DTD не осигурява такава поддръжка
- В DTD моделът на съдържание на един елемент се определя изцяло от неговото име и така не може да се използва в различен контекст в зависимост от това къде се намира. В XML Schema всеки поделемент на даден елемент може да се специфицира локално и така отделните поделементи с едно и също име ще имат различен модел на съдържание

Случаите, в които **DTD е по-ефективно** да бъде използван са:

- Ако е необходимо да бъде предефинирана дефиниция на даден елемент в XML документа (например с **`INCLUDE`** и **`IGNORE`**)
- Когато съдържанието на елементите е най-вече текстово и не се налага дефинирането на собствени типове и налагането на ограничения
- Ако е от значение големината на файла, дефиниращ структурата на XML документа

Използване на XSLT и XPath за алокиране, манипулиране и представяне на XML съдържание

XSLT (eXtensible Stylesheet Language for Transformation) е език за трансформиране на XML документи с помощта на различни стилове и функции. Най-често се използва за конвертиране на документ от XML формат към документ в HTML формат, обикновен текст или пък например друг XML документ. Полезен е, когато искаме да разделим презентационния слой на едно приложение от модела на данните му.

Всеки XSLT документ има ***.xsl*** разширение. Той трябва да бъде добре конструиран XML документ, който започва с началните тагове:

```
<?xml version="1.0"?>

<xsl:stylesheet     version="1.0"
                xmlns:xsl="http://www.w3.org/1999/XSL/Transform">
```

Задаването на XSL трансформация XML документ става посредством:

```
<?xml-stylesheet type=”text/xsl” href =”stylesheet.xsl”?>
```

В XSLT има дефинирани 34 елемента, част от които са следните:

1. **`<xsl:stylesheet>`** – задава обхват на XSLT документа и осигурява конфигуриране на документи
2. **`<xsl:template>`** – специфицира кои елементи трябва да бъдат обхванати от обработката; дефинира какво ще бъде добавено в изходния документ
   - **2.1.** атрибут **`match`** – задава образец за съответствие на част от йерархията на началното дърво, който представлява XPath израз
   - **2.2.** атрибут **`name`** – задава QName с цел извикване на такъв именован шаблон по име чрез **`call-template`**
   - **2.3.** атрибут **`priority`** – задава числов приоритет, който да се ползва за разрешаване на конфликтите при вземане на решение кой от няколко възможни шаблона да се използва за даден възел от дървото (стойността му по подразбиране е 0.5)

2.4. атрибут **mode** – дава възможност даден възел да бъде обработван многократно, с различен краен резултат в зависимост от стойността на mode

3. **<xsl:apply-templates>** – взима решение кои елементи ще бъдат обработени; активира рекурсивна обработка на всички наследници на контекстния възел
4. **<xsl:call-template>** – предизвиква изпълнение на именован шаблон
5. **<xsl:variable>** – за деклариране на променлива; достъпва се чрез символа „$“, последван от името на променливата; стойността на променливата може да се използва и в елементи от резултатното дърво; по този начин могат да се добавят прости константи
6. **<xsl:with-param>** – подаване на стойност към елемент параметър в шаблон **(<xsl:param>)**
7. **<xsl:value-of>** – изчислява израз и добавя резултата в изхода
8. **<xsl:for-each>** – осигурява групова обработка на елементи по еднотипен начин
9. **<xsl:if>** – прилага се само ако условието му е истина; не предоставя else клауза
   9.1. атрибут **test** получава стойност XPath израз
10. **<xsl:choose>** – задава многократни условни тестове; използва се заедно с **<xsl:when>** и **<xsl:otherwise>** (случай по подразбиране, когато никой от случаите в when не са изпълнени)
11. **<xsl:sort>** – използва се като дъщерен елемент на **<xsl:apply-templates>** или **<xsl:for-each>** елементите
    11.1. Атрибут **select** – XPath израз реферящ възел(и) за сортиране
    11.2. Атрибут **data-type** – начин на тълкуване на стойностите за възлите (обикновено се използва текст или число)
    11.3. Атрибут **order** – ред на сортиране (възходящ или низходящ)
12. **<xsl:output>** – позволява контрол над изходния документ; разполага се веднага след **<xsl:stylesheet>**
13. **<xsl:element>** – осигурява начин за създаване на елементи
14. **<xsl:copy>** – копира контекстния възел без атрибутите и наследниците му
15. **<xsl:copy-of>** – копира фрагмента от входното XML дърво без загуба на атрибути и съдържани поделементи (deep copy)
16. **<xsl:attribute>** – осигурява начин за създаване на атрибути
17. **<xsl:attribute-set>** – използва се за създаване на група от атрибути за по-нататъшна употреба
18. **<xsl:include>** – извършва вмъкване на XSLT документ в друг XSLT документ; осигурява изграждане на модули
19. **<xsl:import>** – ако импортираният шаблон е в конфликт с някой от шаблоните в основния XSLT документ, то той получава по-нисък приоритет; приложим е при многократна обработка на възли
20. **<xsl:function>** – дефиниране на функция (от **XSLT 2.0**)

XSLT предоставя възможност за структурни промени на входното съдържание, многократно преизползване на елементно или атрибутно съдържание на друго място в документа и др. Ако браузърът поддържа XSLT, то тази технология може да се ползва за трансформиране на документа до XHTML в самия браузър.

XPath е декларативен език с не-XML синтаксис, който се използва за адресиране и манипулиране на секции от XML документ. В **XPath 1.0** модела повечето съставни части от XML документа са представени като възли, свързани с определени отношения. Коренът на XPath 1.0 дървото представлява самия документ, а не коренът на документа и се бележи с „/“.

XPath процесорът изчислява XPath израз, който задава път от дадена начална точка в дървото. Чрез изчислението на този път като последователност от свързани възли, процесорът се „придвижва“ по дървото, моделиращо документа, до указания от израза възел. Този възел, в който за текущия момент се намира XPath процесорът, се нарича контекстен възел и се бележи с „.“.

Видовете XPath възли са:

1. **Възел-корен на дървото** – сочещ към корена на документа
2. **Възел-елемент** – представя елемент
3. **Възел-текст** – представя текстово съдържание на елемент
4. **Възел-атрибут** – представя атрибут
5. **Възел-инструкция за обработка**
6. **Възел-декларация на CDATA секция (област от данни)**
7. **Възел-коментар**

Възлите в XPath дървото са свързани с отношения от два типа:

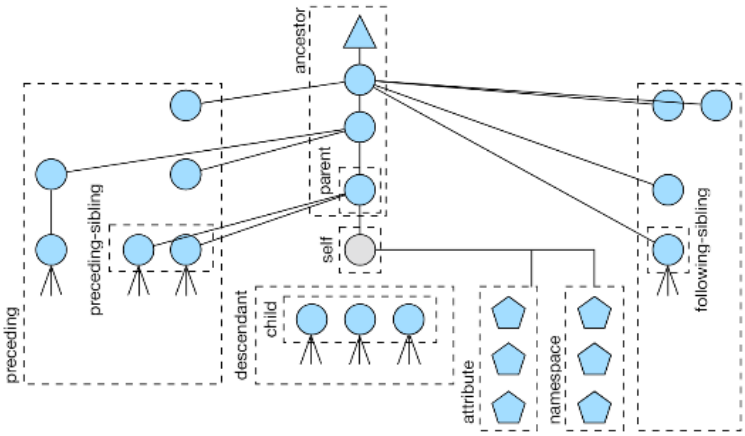
1. **Отношения на свързване на възел с подреден списък от възли-наследници**
   - **1.1.** Наследници на даден елемент
   - **1.2.** Текстови възли
   - **1.3.** Инструкции за обработка
   - **1.4.** Коментари
2. **Отношения на свързване на възел с неподредено множество от други върхове**
   - **2.1.** Възлите за атрибути
   - **2.2.** CDATA секциите

Повечето XPath изрази връщат **множество от върхове**. Тези изрази се наричат местоположения. В XPath съществуват **два вида пътища**: относителни и абсолютни. Един **относителен път** на местоположение се състои от една или повече стъпки на местоположение, разделени със знака „/“. Те могат да започват с низа „./“, който ако е пропуснат се подразбира. Тоест един относителен път започва от контекстния възел. Един **абсолютен път** на местоположение се състои от знака „/“, последван от относителен път. Тоест винаги започва от корена на дървото.

Ос в XPath е определен път в дървото на документа, използващ характерна връзка между възлите. Спецификация XPath 1.0 дефинира **13 вида оси** с особен характер на връзката, както следва:

1. **Ос на предшествениците (ancestor axis)** – избира предшествениците на контекстния възел в обратен ред на срещането им
2. **Ос на предшествениците и на самия възел (ancestor-or-self axis)** – избира контекстния възел и неговите предшественици в обратен ред на появата им в документа
3. **Ос на директните наследници / ос на децата (child axis)** – селектира преките наследници на контекстния възел
4. **Ос на атрибутите (attribute axis)** – задава всички атрибути на контекстния възел
5. **Ос на наследниците (потомците) (descendant axis)** – връща всички наследници (преки и непреки) на контекстния възел по ред на появата им в документа. Атрибутите и секциите за данни не се разглеждат като наследници на елементен възел
6. **Ос на наследниците и на самия възел (descendant-or-self axis)** – връща резултата на оста на наследниците, заедно с контекстния възел

7. **Ос на следващите възли (following axis)** – селектира всички следващи възли след контекстния възел, без наследниците и атрибутите му и без пространствата от имена
8. **Ос на следващите възли и на самия възел (following-sibling axis)** – избира всички братя и сестри на контекстния възел, следващи вдясно от него
9. **Ос на пространствата от имена (namespace axis)** – не се препоръчва в XPath 2.0
10. **Ос на родителя (parent axis)** – връща родителя на контекстния възел
11. **Ос на предходните възли (preceding axis)** – селектира всички предходни възли спрямо контекстния възел, без наследниците и атрибутите му и без пространствата от имена
12. **Ос на предходните възли и на самия възел (preceding-sibling axis)** – избира всички предшестващи братя и сестри на контекстния възел вляво от него
13. **Ос на самия възел / собствена ос (self axis)** – връща контекстния възел



За работата с **относителни и абсолютни пътища** съществуват два вида синтаксис – **разширен** и **кратък**. Форматът на разширения синтаксис е следният:

```
axis :: node-test [predicate] … [predicate]
```

Разширеният синтаксис не е практически толкова удобен за работа и затова в XPath по-често използва краткият синтаксис, който наподобява адресирането на директории на файлови системи. Краткият синтаксис използва означения като „@“ за атрибут и „*“ за избор на всички възли. Родителят на контекстния възел се адресира чрез „..“, а текущия – чрез „.“ – точно както в пътищата при файловите системи.

Използване на DOM (Document Object Model) и SAX (Simple API for XML) за обработка на XML документи – основни интерфейси на DOM и SAX и начини за използването им. Сравнение между DOM и SAX

Document Object Model (DOM) дефинира логическо дърво, представляващо анализирания XML документ. Представлява комплект от приложни интерфейси за четене на XML файл в паметта и съхраняването му като дървовидна структура. Абстрактните API дават възможност за изграждане, достъп и манипулиране на структурата и съдържанието на XML и HTML документи.

DOM е W3C стандарт, който дефинира стандартен начин за работа с документи като XML и HTML. Той представлява обектно-ориентиран подход. XML DOM дефинира обекти и свойства за всички XML елементи, както и интерфейс за достъп до тях. Най-честата употреба на XML DOM е за манипулирането на съдържанието на XML документи – може да се създава ново съдържание, да се навигира, да се модифицира или пък да се добавят или изтриват части от XML документи. В DOM един документ се представя като йерархия (дърво) от **`Node`** обекти.

Спецификацията на DOM се състои от 4 нива, като всяко надгражда предходното. Спецификацията включва основна задължителна част от интерфейси, която трябва да бъде налична във всички имплементации. Тя се нарича Core DOM и се състои от Ниво 2 и Ниво 3.

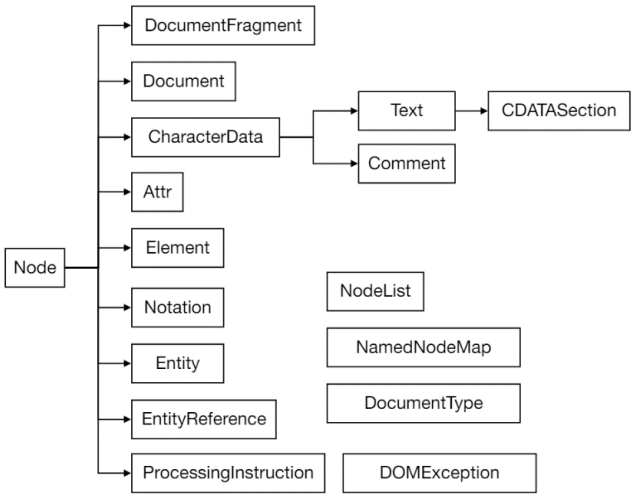
Основните типове DOM интерфейси се състоят както от базови интерфейси, задължителни за всички имплементации на DOM, така и от разширени интерфейси, необходими за DOM имплементации, работещи с XML документи.

Базовите интерфейси са:

1. **`Node`** – представлява един връх в документното дърво; базов тип данни на DOM
2. **`Document`** – представя целия XML документ; често се означава като DOM дърво; съдържа и методи фабрики за създаване на останалите обекти
3. **`DocumentFragment`** – фрагмент от Document може да бъде временно съхранен в такъв тип възел
4. **`NodeList`** – съдържа колекция от наредени **`Node`** обекти
5. **`Element`** – представя елемент; не може да бъде нищо друго освен елемент
6. **`NamedNodeMap`** – дефинира неподредена колекция от възли
7. **`Attr`** – интерфейс към обекти, съдържащи атрибутни данни; свойства *name* и *value* връщат съответно името на атрибута и на неговата стойност; атрибутите не са пряка част от дървовидната структура на документа
8. **`CharacterData`** – осигурява свойства и методи за работа с текст
9. **`Text`** – представя съдържанието на атрибут или елемент; прилага се специфично за PCDATA
10. **`Comment`** – представя коментари; автоматично форматиране на коментарите с разделители
11. **`DOMException`** – включва различни видове изключения, свързани с DOM обработката
12. **`DOMImplementation`** – за определяне на нивото на поддръжка в DOM парсер

Разширените интерфейси са:

1. **`CDATASection`** – представя CDATA секция (немаркиран текст)
2. **`DocumentType`** – списък от предефинирани свойства на документа като информация за съдържанието на DTD и др.
3. **`Notation`** – всяка декларация на нотация в DTD е представена чрез такъв тип възел
4. **`Entity`** – представя единица (parsed/unparsed) в XML документ
5. **`EntityReference`** – DOM включва интерфейси за работа с референции към единици
6. **`ProcessingInstruction`** – представя декларация за инструкция за обработка



Simple API for XML (SAX) дефинира манипулатори (handlers), съдържащи методи за разбор на XML документа (push). **SAX е event-driven API**, който дефинира манипулатори (handlers), съдържащи методи за разбор (push) на XML документа. Разработен е за по-ефикасен анализ на големи XML документи (не се налага всеки път да прочита целите документи). Целта му е да реши проблема на DOM – създаването на масивно дърво на документа в паметта, преди да започнем работа с него. По този начин се пести както памет, така и време. Той осъществява това подобрение с управлението на събития. XML документът се изпраща до **SAX парсера**, след което документът се прочита ред по ред, като парсерът известява за грешки. След това имплементациите на интерфейсните методи обработват събитията. Самият SAX парсер използва **callback механизъм** за известяване на приложението. По този начин може да напишем код за обработката на всяко събитие.

Сред основните SAX интерфейси са:

1. **`DocumentHandler`** – за управление на събитията за съдържанието на документа в SAX 1.; заместен от **`ContentHandler`** в SAX 2
2. **`ContentHandler`** – използва се за получаване на основни събития от парсера; надгражда **`DocumentHandler`**, като добавя поддръжка и за пространствата от имена; предоставя имплементация; в нея можем да пренапишем онези методи, които обработват важни за нас събития
3. **`XMLReader`** – интерфейс за четене на XML документ с използване на callbacks; това разрешава приложението да зададе и провери свойства за парсера, да регистрира обработчици на събития за обработка на документа и да инициира самото парсване
4. **`XMLFilterImpl`** – имплементация на **`ContentHandler`**, която стои между **`XMLReader`** и обработчиците на събития в приложението; предава събитията на обработчиците без промяна; неговите подкласове могат да пренапишат специфични методи
5. **`XMLReaderAdapter`** – обвива XMLReader и го прави като Parser от SAX1
6. **`ErrorHandler`** – осигурява информация за възникнали грешки; три нива на изключения:
    a. **`warning`** – известява приложението с предупреждение
    b. **`error`** – известява приложението за грешка (не блокира парсването)
    c. **`fatalError`** – известява приложението за фатална грешка (блокира парсването)
7. **`DTDHandler`** – предоставя методи за известяване за DTD събития
8. **`EntityResolver`** – определя поведението на SAX парсера при преобразуване на външни референции в рамките на DTD дефиницията

В обобщение, споменахме, че **DOM разглежда XML като логическо дърво**, докато **SAX е базиран на събития**. Когато ги сравняваме, то можем да кажем, че е по-добре да използваме **SAX при по-големи документи** и ако **извличаме само някои елементи**. **DOM е по-добре да използваме при случаи на обработка на много елементи** и **структурни промени** в XML документа, както и при **многократен достъп** до него.

Държавен изпит – спец. „Софтуерно инженерство“

Тема №25 – Теорема на Ферма. Теореми за средните стойности (Рол, Лагранж и Коши). Формула на Тейлър.

Анотация: *Да се дефинира понятието* ***локален екстремум*** *на функция на една променлива. Да се формулира и докаже* ***необходимо условие за локален екстремум*** *за диференцируеми функции (****теорема на Ферма****).*

Да се докажат следните теореми, формулирани общо за по-кратко.

Нека функцията f е непрекъсната в затворения интервал $[a,b]$ и притежава производна поне в отворения интервал (a,b). Да се докаже, че:

*а) ако $f(a)=f(b)$, то съществува $c\in(a,b)$ такова, че $f'(c)=0$ (****Рол****);*

*б) съществува $c\in(a,b)$ такова, че $f(b)-f(a)=f'(c)(b-a)$ (****Лагранж****);*

в) ако функцията g е непрекъсната в затворения интервал $[a,b]$ и притежава производна поне в отворения интервал (a,b), като $g'(x)\neq 0, x\in(a,b)$, то съществува $c\in(a,b)$ такова, че

$\frac{f(b)-f(a)}{g(b)-g(a)}=\frac{f'(c)}{g'(c)}$ *(****Коши****).*

По отношение на твърдението във в) да се докаже, че при направените в него предположения имаме $g(a)\neq g(b)$.

За установяването на теоремата на Рол може да се използва без доказателство теоремата на Вайерщрас, според която всяка непрекъсната функция върху краен затворен интервал достига своите максимум и минимум.

Да се изведе ***формулата на Тейлър*** *с остатъчен член във формата на Лагранж.*

Локални екстремуми

Дефиниция (производна): Казваме, че функцията $f(x)$ е диференцируема в дадена вътрешна точка x_0 от своята дефиниционна област, когато съществува границата:

$$\lim_{x\to x_0}\frac{f(x)-f(x_0)}{x-x_0}.$$

Тази граница се нарича **производна на функцията** $f(x)$ в точката x_0 и се бележи с $f'(x_0)$.

Аналогично може да запишем производната $f'(x)$ като $\lim_{\Delta x\to 0}\frac{\Delta f(x)}{\Delta x}$, където сме положили $\Delta x = x-x_0$ и $\Delta f(x)=f(x_0+\Delta x)-f(x_0)$.

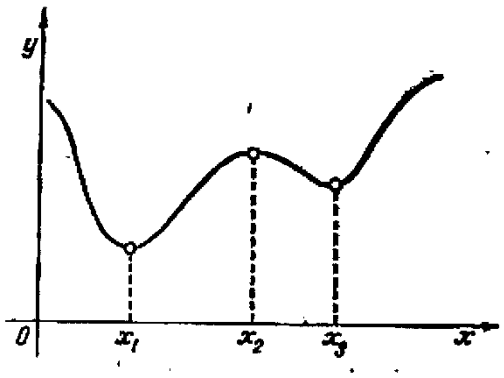
Понятията локален максимум и локален минимум се срещат при много въпроси от анализа и неговите приложения.

Дефиниция (локален максимум): Казваме, че функцията $f(x)$ има локален максимум в някоя вътрешна точка x_0 от своята дефиниционна област, когато съществува такава околност $(x_0-\delta, x_0+\delta)$ на x_0, съдържаща се в дефиниционната област на $f(x)$, че за всяко x от тази околност да имаме $f(x)\leq f(x_0)$.

Дефиниция (локален минимум): Казваме, че функцията $f(x)$ има локален минимум в някоя вътрешна точка x_0 от своята дефиниционна област, когато съществува такава околност $(x_0 - \delta, x_0 + \delta)$ на x_0, съдържаща се в дефиниционната област на $f(x)$, че за всяко x от тази околност да имаме $f(x) \geq f(x_0)$.

Локалните максимуми и локалните минимуми ще наричаме с общото име **локални екстремуми**.

Пример: *На фигурата е показана графиката на една функция, която има локален максимум в точката x_2 и два локални минимума – в точките x_1 и x_3.*



Нека отбележим, че ако една функция $f(x)$ има локален максимум в дадена точка x_0, то стойността ѝ в тази точка е максимална в сравнение със стойностите, които тя приема в точките от някоя околност на x_0, но не непременно в сравнение с всички нейни функционални стойности. Казано по друг начин, един локален максимум не е непременно най-голямата стойност на разглежданата функция. Аналогичен е разсъждението и за понятието локален минимум.

Теорема на Ферма

Теорема на Ферма: Ако функцията $f(x)$ е диференцируема в една вътрешна точка x_0 от своята дефиниционна област и ако тя има локален екстремум в тази точка, то $f'(x_0) = 0$.

Доказателство: Да разгледаме случая, когато $f(x)$ има локален максимум в точката x_0 (случаят, когато тя има локален минимум, се разглежда аналогично). Тогава ще бъде изпълнено неравенството: $f(x) \leq f(x_0)$ за всяко x от някоя околност $(x_0 - \delta, x_0 + \delta)$ на точката x_0. Както знаем, дясната и лявата производна в точката x_0 се дефинират с равенствата:

$$f'_+(x_0) = \lim_{\substack{x \to x_0, \\ x > x_0}} \frac{f(x) - f(x_0)}{x - x_0}, \; f'_-(x_0) = \lim_{\substack{x \to x_0, \\ x < x_0}} \frac{f(x) - f(x_0)}{x - x_0}.$$

Ако разгледаме частното:

$$\frac{f(x) - f(x_0)}{x - x_0}$$

за стойности на x, принадлежащи на интервала $(x_0 - \delta, x_0 + \delta)$ и същевременно по-големи от x_0, ще видим, че числителят е отрицателен или нула (поради горното неравенство за локален максимум), а знаменателят е положителен (от допускането). Оттук, заключваме, че $f'_+(x_0) \leq 0$.

Да разгледаме сега частното:

$$\frac{f(x) - f(x_0)}{x - x_0}$$

за стойности на x, принадлежащи на интервала $(x_0 - \delta, x_0 + \delta)$ и същевременно по-малки от x_0, ще видим, че числителят отново е отрицателен или нула (поради горното неравенство за локален максимум), но сега и знаменателят е отрицателен (от допускането). Оттук, заключваме, че $f'_-(x_0) \geq 0$.

По условие функцията $f(x)$ е диференцируема в точката x_0. Следователно, $f'_+(x_0) = f'_-(x_0) = f'(x_0)$. И така имаме едновременно $f'(x_0) \leq 0$ и $f'(x_0) \geq 0$, откъдето получаваме окончателно, че $f'(x_0) = 0$, което и искахме да докажем. ■

Теоремата на Ферма може да се формулира и още така: Ако функцията $f(x)$ е диференцируема в точката x_0, то за да притежава тя локален екстремум в тази точка, необходимо е производната ѝ $f'(x_0)$ да бъде равна на нула. Именно затова, тя играе ролята на **необходимо условие за локален екстремум** на диференцируеми функции.

Когато искаме да намерим локалните екстремуми на някоя диференцируема функция, ние обикновено най-напред намираме нейната производна и търсим онези точки, в които тази производна е нула. Това са единствените точки, в които е възможно да имаме екстремуми. Анулирането на първата производна в една точка обаче не е още достатъчно условие за съществуването на локален екстремум.

Теорема на Рол

Теорема за ограниченост: Ако една функция $f(x)$ е непрекъсната в крайния и затворен интервал $[a, b]$, то тя е ограничена в този интервал.

Теорема на Вайершрас: Ако една функция $f(x)$ е непрекъсната в крайния и затворен интервал $[a, b]$, то тя притежава една най-голяма стойност и една най-малка стойност.

Теорема на Рол: Ако функцията $f(x)$ е непрекъсната в крайния и затворен интервал $[a, b]$ и е диференцируема в отворения интервал (a, b) и ако освен това $f(a) = f(b)$, то съществува поне една точка c, намираща се между a и b, за която $f'(c) = 0$.

Доказателство: Тъй като функцията $f(x)$ е непрекъсната в един краен и затворен интервал, то тя е ограничена съгласно теоремата за ограниченост. Да означим съответно с L и l точната ѝ горна и точната ѝ долна граница в интервала $[a, b]$.

Ако $l = L$, то поради неравенствата $l \leq f(x) \leq L$, изпълнени за всяко x от интервала $[a, b]$, функцията $f(x)$ ще бъде константа в този интервал. Тогава нейната производна е нула в целия интервал (a, b) и теоремата е доказана.

Остава да разгледаме случая, когато $l < L$. Съгласно теоремата на Вайерщрас съществуват две точки x_1 и x_2 от интервала $[a, b]$, такива че $f(x_1) = l$ и $f(x_2) = L$. Поне една от тези две точки е вътрешна за интервала $[a, b]$. И наистина, ако допуснем противното, т.е. ако имаме $x_1 = a, x_2 = b$ или пък $x_1 = b, x_2 = a$, то от условието $f(a) = f(b)$ бихме получили $l = L$, което е в противоречие с разглеждания случай.

Нека x_1 е вътрешна точка за интервала $[a, b]$. Но $f(x_1) = l$. Тогава функцията $f(x)$ ще има очевидно локален минимум в точката x_1, поради което съгласно теоремата на Ферма, ще имаме $f'(x_1) = 0$. Ако пък x_1 е крайна точка за интервала $[a, b]$, то точката x_2 ще бъде вътрешна. В такъв случай това ще бъде една точка на локален максимум и следователно пак по теоремата на Ферма ще имаме $f'(x_2) = 0$. И така във всички случаи съществува точка c между a и b, за която $f'(c) = 0$. ■

Теорема за крайните нараствания (на Лагранж)

Теорема за крайните нараствания (Лагранж): Ако функцията $f(x)$ е непрекъсната в крайния и затворен интервал $[a, b]$ и е диференцируема в отворения интервал (a, b), то съществува поне една точка c между a и b, за която $f(b) - f(a) = f'(c)(b - a)$.

Доказателство: Да въведем помощната функция

$$\varphi(x) = f(x) - \frac{f(b)-f(a)}{b-a}(x - a).$$

Тя също е непрекъсната в интервала $[a, b]$ и диференцируема в интервала (a, b). Лесно се пресмята при това, че:

$$\varphi(a) = f(a) - \frac{f(b) - f(a)}{b - a}(a - a) = f(a),$$

$$\varphi(b) = f(b) - \frac{f(b) - f(a)}{b - a}(b - a) = f(b) - f(b) + f(a) = f(a).$$

И така имаме, че $\varphi(a) = \varphi(b)$. Значи функцията $\varphi(x)$ удовлетворява всички условия на теоремата на Рол и следователно ще съществува поне една точка c между a и b, за която $\varphi'(c) = 0$. Но:

$$\varphi'(x) = f'(x) - \frac{f(b) - f(a)}{b - a}.$$

Оттук получаваме:

$$\varphi'(c) = f'(c) - \frac{f(b) - f(a)}{b - a} = 0,$$

или

$$f'(c) = \frac{f(b)-f(a)}{b-a} \Leftrightarrow f(b) - f(a) = f'(c)(b - a),$$

с което теоремата е доказана. ■

Следствие 1 (основна теорема на интегралното смятане): Ако функцията $f(x)$ има производна, равна на нула във всички точки на един интервал D, то тя е константа в този интервал.

Следствие 2: Ако функцията $f(x)$ е диференцируема в един интервал D и ако $f'(x) \geq 0$ за всяко x от D, то $f(x)$ е растяща в този интервал. Ако $f'(x) > 0$ за всяка вътрешна точка x от D, то $f(x)$ е строго растяща.

Обобщена теорема за крайните нараствания (на Коши)

Теорема на Коши (обобщена теорема за крайните нараствания): Ако функциите $f(x)$ и $g(x)$ са непрекъснати в крайния и затворен интервал $[a, b]$ и диференцируеми в отворения интервал (a, b) и ако $g'(x) \neq 0$ за всяко x от (a, b), то съществува поне една точка c, намираща се между a и b, за която е изпълнено равенството

$$\frac{f'(c)}{g'(c)} = \frac{f(b) - f(a)}{g(b) - g(a)}.$$

Доказателство: Преди всичко нека отбележим, че знаменателят $g(b) - g(a)$ на частното, което е в дясната страна на равенството е сигурно различен от нула, т.е. $g(a) \neq g(b)$. Наистина, ако допуснем, че $g(a) = g(b)$, то функцията $g(x)$ би удовлетворявала условията на теоремата на Рол. Тогава би съществувала някоя точка от отворения интервал (a, b), за която производната $g'(x)$ би била равна на нула, а това противоречи на условието на теоремата.

Да преминем сега към самото доказателство на теоремата, което по своята идея не се различава много от доказателството на теоремата за крайните нараствания. Да си образуваме помощната функция

$$\varphi(x) = f(x) - \frac{f(b) - f(a)}{g(b) - g(a)}[g(x) - g(a)].$$

Вижда се, че тя е непрекъсната в интервала $[a, b]$ и диференцируема в интервала (a, b). Освен това имаме

$$\varphi(a) = f(a) - \frac{f(b) - f(a)}{g(b) - g(a)}[g(a) - g(a)] = f(a),$$

$$\varphi(b) = f(b) - \frac{f(b) - f(a)}{g(b) - g(a)}[g(b) - g(a)] = f(a).$$

И така получаваме, че $\varphi(a) = \varphi(b)$, т.е. функцията $\varphi(x)$ удовлетворява условията на теоремата на Рол. Тогава ще съществува някаква точка c от отворения интервал (a, b), за която $\varphi'(c) = 0$. Но

$$\varphi'(x) = f'(x) - \frac{f(b) - f(a)}{g(b) - g(a)}g'(x),$$

откъдето

$$\varphi'(c) = f'(c) - \frac{f(b)-f(a)}{g(b)-g(a)}g'(c) = 0,$$

или получаваме, че

$$\frac{f'(c)}{g'(c)} = \frac{f(b)-f(a)}{g(b)-g(a)},$$

което искахме да докажем. ■

Забележка: *Теоремата за крайните нараствания може да се получи като частен случай на теоремата на Коши. Наистина нека $f(x)$ е една функция, непрекъсната в интервала $[a,b]$ и диференцируема в интервала (a,b). Да разглеждаме освен това и функцията $g(x) = x$. Като приложим теоремата на Коши към тези две функции и вземем предвид, че $g(a) = a, g(b) = b$, както и това, че $g'(x) = 1$ за всяко x, заключваме, че съществува точка c от интервала (a,b), за която имаме $\frac{f'(c)}{1} = \frac{f(b)-f(a)}{b-a}$.*

Формула на Тейлър

Нека вземем един полином от n-та степен:

$$f(x) = a_n x^n + a_{n-1}x^{n-1} + \cdots + a_2 x^2 + a_1 x + a_0. \qquad (1)$$

Ако диференцираме равенството (1) n пъти, ще получим последователно

$$\begin{cases} f'(x) = na_n x^{n-1} + (n-1)a_{n-1}x^{n-2} + \cdots + 2a_2 x + 1.\,a_1, \\ f''(x) = n(n-1)a_n x^{n-2} + (n-1)(n-2)a_{n-1}x^{n-3} + \cdots + 2.1.\,a_2, \\ \cdots\cdots\cdots\cdots\cdots\cdots\cdots\cdots\cdots\cdots\cdots\cdots\cdots\cdots\cdots\cdots\cdots\cdots\cdots\cdots\cdots\cdots\cdots\cdots\cdots\cdots\cdots \\ f^{(n-1)}(x) = n(n-1)\ldots 3.2.\,a_n x + (n-1)(n-2)\ldots 2.1.\,a_{n-1}, \\ f^{(n)}(x) = n(n-1)\ldots 3.2.1.\,a_n. \end{cases} \qquad (2)$$

Като заместим в равенствата (1) и (2) x с 0, ще получим

$$f(0) = a_0,$$

$$f'(0) = 1!\,a_1,$$

$$f''(0) = 2!\,a_2,$$

$$\ldots$$

$$f^{(n-1)}(0) = (n-1)!\,a_{n-1},$$

$$f^{(n)}(0) = n!\,a_n.$$

Виждаме, че коефициентите на полинома $f(x)$ се изразяват чрез стойностите на $f(x)$ и неговите производни в точката 0. Тогава равенството (1) може да бъде записано и по следния начин:

$$f(x) = f(0) + \frac{f'(0)}{1!}x + \frac{f''(0)}{2!}x^2 + \cdots + \frac{f^{(n)}(0)}{n!}x^n. \qquad (3)$$

Можем да обобщим тази формула, като приемем произволна точка a да играе ролята на точката 0. За целта да положим $x = a + h$ и $f(a + h) = \varphi(h)$. Функцията

$$\varphi(h) = a_n(a + h)^n + a_{n-1}(a + h)^{n-1} + \cdots + a_1(a + h) + a_0$$

е очевидно полином от n-та степен на променливата h и следователно съгласно формула (3) ще имаме

$$\varphi(h) = \varphi(0) + \frac{\varphi'(0)}{1!}h + \frac{\varphi''(0)}{2!}h^2 + \cdots + \frac{\varphi^{(n)}(0)}{n!}h^n. \qquad (4)$$

От друга страна,

$$\varphi(h) = f(a + h),$$

$$\varphi'(h) = f'(a + h),$$

$$\varphi''(h) = f''(a + h),$$

...

$$\varphi^{(n)}(h) = f^{(n)}(a + h)$$

и следователно

$$\varphi(0) = f(a),$$

$$\varphi'(0) = f'(a),$$

$$\varphi''(0) = f''(a),$$

...

$$\varphi^{(n)}(0) = f^{(n)}(a).$$

Тогава равенството (4) се написва така:

$$f(a + h) = f(a) + \frac{f'(a)}{1!}h + \frac{f''(a)}{2!}h^2 + \cdots + \frac{f^{(n)}(a)}{n!}h^n, \qquad (5)$$

или, което е еквивалентно на:

$$f(x) = f(a) + \frac{f'(a)}{1!}(x - a) + \frac{f''(a)}{2!}(x - a)^2 + \cdots + \frac{f^{(n)}(a)}{n!}(x - a)^n. \qquad (6)$$

Равенството (5), както и равносилното на него равенство (6), се нарича **формула на Тейлър**. Тази формула може да бъде видоизменена по такъв начин, че да запази своята сила не само за полиноми, но и за твърде широка категория от функции.

Теорема на Тейлър: Да предположим, че функцията $f(x)$ притежава първа, втора и т.н. до $(n+1)$-ва производна в някоя околност $(a-\delta, a+\delta)$ на една точка a (тази околност може в частност да съвпада с цялата реална права). Ако x е една точка от тази околност, то валидно е равенството

$$f(x) = f(a) + \frac{f'(a)}{1!}(x-a) + \cdots + \frac{f^{(n)}(a)}{n!}(x-a)^n + \frac{f^{(n+1)}(c)}{(n+1)!}(x-a)^{n+1}, \qquad (7)$$

където c е точка, намираща се между a и x.

Равенството (7) се нарича **обща формула на Тейлър**. Последното събираемо в дясната страна

$$R_n(x) = \frac{f^{(n+1)}(c)}{(n+1)!}(x-a)^{n+1}$$

се нарича **остатъчен член във формата на Лагранж**.

Държавен изпит – спец. „Софтуерно инженерство“

Тема №26 – Определен интеграл. Дефиниция и свойства. Интегруемост на непрекъснатите функции. Теорема на Нютон-Лайбниц.

Анотация: *Да се дефинират последователно: разбиване на интервал, големи и малки суми на Дарбу. Да се установи, че при добавяне на нови точки в разбиването на интервала, големите суми на Дарбу не нарастват, а малките не намаляват (желателно е да се направи чертеж). Да се дефинира риманов интеграл чрез подхода на Дарбу.*

Да се докаже, че дадена функция е интегруема по Риман тогава и само тогава, когато за всяко $\varepsilon > 0$ *съществуват голяма сума на Дарбу* S *и малка сума на Дарбу* s *такива, че* $S - s < \varepsilon$. *Като се използва тази теорема и теоремата на Кантор (без доказателство), според която всяка непрекъсната функция в краен и затворен интервал е равномерно непрекъсната, да се докаже, че всяка непрекъсната функция в краен и затворен интервал е интегруема по Риман.*

Да се изброят (без доказателство) основните свойства на римановия интеграл. Да се докаже, че ако f *е непрекъсната в* $[a, b]$, *то съществува* $c \in [a, b]$ *такова, че*

$$\int_a^b f(x)dx = f(c)(b - a).$$

За установяването на това твърдение да се приложат (без доказателство) свойството за интегриране на неравенства и теоремата, че всяка непрекъсната функция в $[a, b]$ *приема всички стойности между максимума и минимума си.*

Да се докаже теоремата на Нютон-Лайбниц, т.е. ако f *е непрекъсната в* $[a, b]$, *то за всяко* $x \in [a, b]$ *е изпълнено*

$$\frac{d}{dx}\int_a^x f(t)dt = f(x).$$

Да се покаже как теоремата се използва за изчисляване на определени интеграли.

Разбиване на интервал, големи и малки суми на Дарбу

Историческите стимули за въвеждането на определения интеграл са свързани главно със стремежа да се придаде строг смисъл на понятието лице на фигура. Като отправна точка може да се приеме лицето на правоъгълник, което е равно на произведението на дължините на страните му. Ние ще се опитаме да сведем лицето на произволна фигура към сума от лицата на подходящи правоъгълници.

Дефиниция (разбиване на интервал): Разбиване на интервала $[a, b]$ ще наричаме всяка крайна система от точки $x_0, x_1, \dots, x_n$ такива, че $a = x_0 < x_1 < \cdots < x_n = b$.

Разбиванията *ще означаваме с* τ, *а точките, определящи разбиването, ще наричаме* ***делящи точки***.

Дефиниция (диаметър на разбиване): Под диаметър на разбиването τ ще разбираме числото:

$$d(\tau) = \max_{1 \le i \le n}(x_i - x_{i-1}).$$

Дефиниция (големи и малки суми на Дарбу): Нека предположим, че функцията $f(x)$ е ограничена в интервала $[a, b]$ и нека е зададено разбиване τ: $a = x_0 < x_1 < \cdots < x_n = b$ на този интервал. Да означим с M_i, съответно с m_i, точната горна и точната долна граница на стойностите на $f(x)$ в интервала $[x_{i-1}, x_i]$. Да разгледаме изразите:

$$S_\tau = \sum_{i=1}^{n} M_i(x_i - x_{i-1}),$$

$$s_\tau = \sum_{i=1}^{n} m_i(x_i - x_{i-1}),$$

които ще наричаме съответно голяма и малка сума на Дарбу, съответстващи на разбиването τ.

На всяко разбиване τ съответстват точно една голяма и една малка сума на Дарбу (за разлика от римановите суми, които са безкрайно много).

Свойства на сумите на Дарбу

Дефиниция (релацията „следва“): Ще казваме, че разбиването τ' следва разбиването τ (ще записваме $\tau' \succ \tau$), ако делящите точки на τ са делящи точки и на τ'.

Свойство 1: Ако τ' следва τ, то са в сила неравенствата:

$$s_\tau \le s_{\tau'},$$

$$S_\tau \ge S_{\tau'}.$$

Доказателство: Трябва да докажем, че при добавянето на нови делящи точки малките суми на Дарбу не намаляват, а големите – не нарастват или казано по друг начин малките суми растат, а големите намаляват. Достатъчно е да се разгледа случаят, когато τ' се получава от τ чрез добавяне на една деляща точка. Нека означим с $x_0, x_1, \ldots, x_n$ делящите точки на τ и нека τ' да съдържа освен тях и още една деляща точка x', лежаща в интервала (x_{i-1}, x_i).

Да означим с M' и M'' точната горна граница на $f(x)$ съответно в интервалите $[x_{i-1}, x']$ и $[x', x_i]$. Очевидно числата M' и M'' не надминават M_i. Да сравним изразите за S_τ и $S_{\tau'}$. Ясно е, че $S_{\tau'}$ се получава от S_τ, като събираемото $M_i(x_i - x_{i-1})$ се замени с две нови събираеми $M'(x' - x_{i-1}) + M''(x_i - x')$. Имаме обаче:

$$M'(x' - x_{i-1}) + M''(x_i - x') \le M_i(x' - x_{i-1}) + M_i(x_i - x') = M_i(x_i - x_{i-1})$$

и следователно $S_\tau \ge S_{\tau'}$. Неравенството за малките суми на Дарбу се доказва аналогично. ■

Свойство 2: Всяка малка сума на Дарбу не надминава коя да е голяма сума.

От свойство 2 следва, че големите и малките суми на Дарбу на дадена функция образуват ограничени множества.

Определение на Дарбу за интеграл

Дефиниция (долен и горен интеграл): Да означим с $\underline{I}$ точната горна граница на всички малки суми на Дарбу, а с $\overline{I}$ – точната долна граница на всички големи суми на Дарбу за функцията f в интервала $[a, b]$:

$$\underline{I} = \sup_{\tau} s_\tau, \quad \overline{I} = \inf_{\tau} S_\tau$$

по всевъзможните разбивания τ на интервала $[a, b]$. Числата $\underline{I}$ и $\overline{I}$ се наричат съответно долен и горен интеграл от f в $[a, b]$.

От **свойство 2** очевидно следва неравенството:

$$\underline{I} \leq \overline{I}$$

Дефиниция (на Дарбу за интеграл): Ограничената функция f се нарича интегруема, ако горният и долният и долният и интеграл съвпадат помежду си. Тяхната обща стойност се нарича определен интеграл от f в интервала $[a, b]$.

Критерий на Дарбу: Функцията $f(x)$ е интегруема в $[a, b]$ тогава и само тогава, когато за всяко $\varepsilon > 0$ съществуват разбивания τ_1, τ_2 на интервала $[a, b]$ такива, че:

$$S_{\tau_1} - s_{\tau_2} < \varepsilon.$$

Доказателство:

⇐) Ако горното условие е изпълнено, то от неравенствата:

$$s_{\tau_2} \leq \underline{I} \leq \overline{I} \leq S_{\tau_1}$$

следва, че $\overline{I} - \underline{I} < \varepsilon$ за всяко $\varepsilon > 0$. Следователно, $\overline{I} = \underline{I}$, т.е. $f(x)$ е интегруема в $[a, b]$.

⇒) Обратно, нека f е интегруема и $\varepsilon > 0$. От дефинициите за точна долна и точна горна граница следва, че могат да се намерят разбивания τ_1, τ_2 така, че:

$$s_{\tau_2} > \underline{I} - \frac{\varepsilon}{2}, \quad S_{\tau_1} < \overline{I} + \frac{\varepsilon}{2},$$

откъдето, използвайки, че $\underline{I} = \overline{I}$, получаваме:

$$s_{\tau_2} + \frac{\varepsilon}{2} > \underline{I}, \;\; S_{\tau_1} - \frac{\varepsilon}{2} < \overline{I} \quad \Leftrightarrow \quad s_{\tau_2} + \frac{\varepsilon}{2} > S_{\tau_1} - \frac{\varepsilon}{2} \Leftrightarrow S_{\tau_1} - s_{\tau_2} < \varepsilon. \qquad \blacksquare$$

Забележка: *Лесно се вижда, че можем да смятаме, че* $\tau_1 = \tau_2$. *Наистина, ако* τ *следва* τ_1 *и* τ_2, *то* $S_\tau - s_\tau \leq S_{\tau_1} - s_{\tau_2}$. *В този случай неравенството от условието се записва във вида:*

$$\sum_{i=1}^{n}(M_i - m_i)(x_i - x_{i-1}) < \varepsilon.$$

Разликата $M_i - m_i$ *е точно осцилацията* $\omega_i = \omega(f, [x_{i-1}, x_i])$ *на функцията* f *в интервала* $[x_{i-1}, x_i]$, *което ни позволява да запишем неравенството и по следния начин:*

$$\sum_{i=1}^{n}\omega_i(x_i - x_{i-1}) < \varepsilon$$

Следствие: Сума и произведение на интегруеми функции също е интегруема функция.

Теорема: Всяка функция, непрекъсната в крайния и затворен интервал $[a, b]$, е интегруема в него.

Доказателство: Както знаем, от непрекъснатостта на функцията $f(x)$ в крайния и затворен интервал $[a, b]$ следва, че тя е ограничена, както и равномерно непрекъсната (теорема на Кантор). Нека $\underline{I}$ и $\overline{I}$ са съответно долният и горният интеграл на функцията $f(x)$ в интервала $[a, b]$. Трябва да докажем, че те са равни помежду си.

Да допуснем противното. Вече видяхме, че $\underline{I} < \overline{I}$. Нека в такъв случай числото $\varepsilon = \frac{\overline{I} - \underline{I}}{b-a}$, което ще бъде положително. Ще използваме теоремата за осцилациите, според която можем да разделим интервала $[a, b]$ на краен брой подинтервали по такъв начин, че във всеки от тях осцилацията на $f(x)$ да бъде по-малка от ε. Нека тези подинтервали са:

$$[x_0, x_1], [x_1, x_2], \ldots, [x_{n-1}, x_n],$$

където $x_0 = a$ и $x_n = b$. На това разделяне на интервала $[a, b]$ на подинтервали ще отговарят една малка и една голяма сума на Дарбу. Това са сумите:

$$s = \sum_{i=1}^{n} m_i(x_i - x_{i-1}) \text{ и } S = \sum_{i=1}^{n} M_i(x_i - x_{i-1}),$$

където с m_i и M_i са означени съответно точната долна и точната горна граница на $f(x)$ в интервала $[x_{i-1}, x_i]$. От дефиницията на долния и горния интеграл следва, че:

$$s \leq \underline{I}, \quad \overline{I} \leq S.$$

От тези пък неравенства получаваме:

$$\overline{I} - \underline{I} \leq S - s \qquad (1)$$

Като извадим почленно голямата и малка сума на Дарбу получаваме:

$$S - s = \sum_{i=1}^{n} M_i(x_i - x_{i-1}) - \sum_{i=1}^{n} m_i(x_i - x_{i-1}) = \sum_{i=1}^{n} (M_i - m_i)(x_i - x_{i-1}).$$

Но разликата $M_i - m_i$ не е нищо друго освен осцилацията на $f(x)$ в интервала $[x_{i-1}, x_i]$. Следователно, ще имаме:

$$S - s < \varepsilon \sum_{i=1}^{n} (x_i - x_{i-1}) = \varepsilon(b - a).$$

Като си спомним как бяхме избрали ε, последното неравенство добива вида:

$$S - s < \frac{\overline{I} - \underline{I}}{b - a}(b - a) = \overline{I} - \underline{I} \qquad (2)$$

Но неравенства (1) и (2) си противоречат. Следователно, първоначалното ни допускане е погрешно. И така имаме, че $\underline{I} = \overline{I}$, т.е. $f(x)$ е интегруема в интервала $[a, b]$, с което теоремата е доказана. ■

Основни свойства на определените интеграли

1. **Линейност:** Нека функциите $f(x)$ и $g(x)$ са интегруеми в интервала $[a, b]$ и $\lambda \in \mathbb{R}$. Тогава $f(x) + g(x)$ и $\lambda f(x)$ също са интегруеми и са в сила равенствата:

$$\int_a^b \big(f(x) + g(x)\big)dx = \int_a^b f(x)dx + \int_a^b g(x)dx\,,$$

$$\int_a^b \lambda f(x)dx = \lambda \int_a^b f(x)dx$$

2. **Позитивност:** Ако $f(x)$ е неотрицателна интегруема функция, то

$$\int_a^b f(x)dx \geq 0$$

Следствие (интегриране на неравенства): Нека $f(x)$ и $g(x)$ са интегруеми в $[a, b]$ и $f(x) \leq g(x)$ за всяко $x \in [a, b]$. Тогава:

$$\int_a^b f(x)dx \leq \int_a^b g(x)dx$$

3. Ако функцията $f(x)$ е интегруема в $[a,b]$, то $|f(x)|$ е също интегруема в този интервал и е в сила неравенството:

$$|\int_a^b f(x)dx| \leq \int_a^b |f(x)|dx$$

4. **Адитивност:** Нека $c \in (a,b)$. Тогава f е интегруема в $[a,b]$ тогава и само тогава, когато f е интегруема във всеки от интервалите $[a,c]$ и $[c,b]$. При това:

$$\int_a^b f(x)dx = \int_a^c f(x)dx + \int_c^b f(x)dx$$

всеки път, когато трите интеграла съществуват.

Конвенция: Досега разглеждахме определени интеграли, за които долната граница е по-малка от горната. Нека сега да премахнем това ограничение и $a > b$. Дефинираме:

$$\int_a^b f(x)dx := -\int_b^a f(x)dx,$$

$$\int_a^a f(x)dx := 0$$

5. Ако $f(x)$ е интегруема в интервала $[a,b]$ и ако m и M са съответно една нейна долна и една нейна горна граница в този интервал, то:

$$m(b-a) \leq \int_a^b f(x)dx \leq M(b-a)$$

6. **Теорема за средните стойности:** Ако функцията $f(x)$ е непрекъсната в крайния и затворен интервал $[a,b]$, то съществува поне една точка c в този интервал, за която е изпълнено равенството:

$$\int_a^b f(x)dx = f(c)(b-a).$$

Доказателство: Да означим с m и M съответно точната долна и точната горна граница на $f(x)$ в интервала $[a,b]$ (от теорема за ограниченост). От неравенствата:

$$m \leq f(x) \leq M$$

следва както знаем от свойството за интегриране на неравенства, че:

$$m(b-a) \leq \int_a^b f(x)dx \leq M(b-a)$$

Оттук получаваме:

$$m \leq \frac{\int_a^b f(x)dx}{b-a} \leq M$$

Поради непрекъснатостта на $f(x)$ съществуват от теоремата на Вайерщрас две точки x_1, x_2 от интервала $[a, b]$, за които:

$$f(x_1) = m, \;\; f(x_2) = M$$

Можем да считаме, че тези две точки са различни. Наистина, ако $x_1 = x_2$, то бихме имали $m = M$, откъдето би следвало, че $f(x)$ е константа. Този случай обаче може да се изключи от разглеждане, тъй като теоремата тогава е очевидна. И така точките x_1 и x_2 определят един интервал. По отношение на този интервал ще приложим към функцията $f(x)$ теоремата за междинните стойности (теорема на Болцано), в следствие на която всяка непрекъсната функция в $[a, b]$ приема всички стойности между максимума и минимума си. Като вземем предвид горните равенства и неравенства, ще заключим, че съществува поне една точка c, намираща се между x_1 и x_2, за която е изпълнено равенството:

$$f(c) = \frac{\int_a^b f(x)dx}{b-a}$$

Оттук се получава равенството:

$$\int_a^b f(x)dx = f(c)(b-a),$$

с което теоремата е доказана. ■

Теорема на Нютон-Лайбниц

Теорема (Нютон-Лайбниц): Ако функцията $f(x)$ е непрекъсната в един интервал D, то функцията:

$$F(x) = \int_a^x f(t)dt$$

е диференцируема в този интервал и за всяко x от D е изпълнено равенството:

$$F'(x) = f(x).$$

Казано по друг начин, $F(x)$ е примитивна функция на $f(x)$ в интервала D.

Доказателство: Нека x е произволна точка от интервала D. Ако $x+h$ е друга точка от този интервал, то ще имаме:

$$\frac{F(x+h)-F(x)}{h}=\frac{1}{h}\left[\int_a^{x+h} f(t)dt-\int_a^{x} f(t)dt\right]=$$

$$=\frac{1}{h}\left[\int_a^{x} f(t)dt+\int_x^{x+h} f(t)dt-\int_a^{x} f(t)dt\right]=\frac{1}{h}\int_x^{x+h} f(t)dt$$

Ако приложим към последния интеграл теоремата за средните стойности, ще получим:

$$\int_x^{x+h} f(t)dt=f(c)(x+h-x)\Leftrightarrow f(c)=\frac{\int_x^{x+h} f(t)dt}{h}=\frac{F(x+h)-F(x)}{h}$$

където c е някаква точка, намираща се между x и $x+h$.

Ако оставим h да клони към 0, то точката c ще клони към x. Ето защо като вземем предвид непрекъснатостта на $f(x)$ в точката x, ще получим за $F'(x)$:

$$F'(x)=\lim_{h\to 0}\frac{F(x+h)-F(x)}{h}=\lim_{h\to 0} f(c)=f(x),$$

с което теоремата е доказана. ■

Формула на Нютон-Лайбниц: Нека $f(x)$ е непрекъсната в $[a,b]$ и $\Phi(x)$ е произволна нейна примитивна. Тогава е изпълнена формулата:

$$\int_a^{b} f(x)dx=\Phi(b)-\Phi(a)$$

Доказателство: *Нека F е примитивната на f, дефинирана в теоремата на Нютон-Лайбниц. Очевидно тя удовлетворява равенството $F(a)=0$. Както знаем, разликата на всеки две примитивни на дадена функция е константа и следователно:*

$$F(x)=\Phi(x)+C$$

Като положим тук $x=a$, получаваме:

$$F(a)=0=\Phi(a)+C\Leftrightarrow C=-\Phi(a)$$

Оттук имаме

$$\int_a^{b} f(x)dx=F(b)=\Phi(b)-\Phi(a)$$

Примери за изчисляване на определени интеграли

$$\mathbf{1}.\quad \int_0^1 \frac{dx}{1+x^2} = ?$$

Първо трябва да намерим една примитивна $\Phi(x)$ на функцията $f(x) = \frac{1}{1+x^2}$. За целта трябва първо да пресметнем неопределения интеграл:

$$\int \frac{dx}{1+x^2} = arctg\ x + C$$

т.е. избираме например $\Phi(x) = arctg\ x$.

Следователно:

$$\int_0^1 \frac{dx}{1+x^2} = \Phi(1) - \Phi(0) = arctg\ x|_0^1 = arctg\ 1 - arctg\ 0 = \frac{\pi}{4}$$

$$\mathbf{2}.\quad \int_1^2 x^3 dx = ?$$

$$\int x^3 dx = \frac{x^4}{4} + C$$

$$\Phi(x) = \frac{x^4}{4}$$

$$\int_1^2 x^3 dx = \Phi(2) - \Phi(1) = \frac{2^4}{4} - \frac{1^4}{4} = \frac{16}{4} - \frac{1}{4} = \frac{15}{4}$$

Държавен изпит – спец. „Софтуерно инженерство“

Тема №27 – Ранг на система вектори. Ранг на матрица. Теорема за равенството на ранг на матрица, ранг на системата от вектор-редове и ранг на системата от вектор-стълбове на тази матрица. Системи линейни уравнения – теорема на Руше, фундаментална система от решения за хомогенна система и теорема за размерността на подпространството от решения на хомогенна система.

Анотация: *Във въпроса се включва:*

- Ранг на система вектори – определение и връзката му с размерността на линейната обвивка; Ранг на матрица – определение; Теорема за равенството на ранг на матрица с ранг на системата от вектор редове и ранг на системата от вектор стълбове на тази матрица (с доказателство);

- Системи линейни уравнения – определение за съвместима и несъвместима система, Теорема на Руше (с доказателство);

- Хомогенни системи – доказателство, че множеството от решения е подпространство, определение за фундаментална система от решения, теорема за размерността на подпространството от решения на хомогенна система (само формулировка).

Ранг на матрица – определение

Дефиниция (минор от *k*-ти ред на матрица): Нека $A \in F_{m\times n}$ и нека $1 \leq k \leq \min(m,n)$. Избираме произволни k реда и произволни k стълба от A и образуваме детерминанта от k-ти ред от общите елементи, т.е.:

$$\begin{vmatrix} a_{i_1 j_1} & a_{i_1 j_2} & \dots & a_{i_1 j_k} \\ \vdots & \vdots & \vdots & \vdots \\ \vdots & \vdots & \vdots & \vdots \\ a_{i_k j_1} & a_{i_k j_2} & \dots & a_{i_k j_k} \end{vmatrix} \qquad \begin{matrix} 1 \leq i_1 < i_2 < \dots < i_k \leq m \\ 1 \leq j_1 < j_2 < \dots < j_k \leq n \end{matrix}$$

Казваме, че тази детерминанта е минор от k-ти ред на A.

Пример:

$$A = \begin{pmatrix} \vec{\mathbf{2}} & \mathbf{9} & \check{\mathbf{6}} & \mathbf{4} & \check{\mathbf{3}} & \check{\mathbf{2}} \\ 1 & 0 & \mathbf{7} & 8 & \mathbf{2} & \mathbf{3} \\ \overrightarrow{\mathbf{-5}} & \mathbf{-6} & \mathbf{1} & \mathbf{4} & \mathbf{3} & \mathbf{8} \\ \vec{\mathbf{7}} & \mathbf{6} & \mathbf{8} & \mathbf{2} & \mathbf{1} & \mathbf{0} \end{pmatrix}_{4\times 6}$$

Минор от I-ви ред: $|a_{33}| = |1| = 1$

Минор от II-ри ред:

$$\begin{vmatrix} a_{12} & a_{13} \\ a_{32} & a_{33} \end{vmatrix} = \begin{vmatrix} 9 & 6 \\ -6 & 1 \end{vmatrix} = 9 + 36 = 45$$

Минор от III-ти ред:

$$\begin{vmatrix} a_{13} & a_{15} & a_{16} \\ a_{33} & a_{35} & a_{36} \\ a_{43} & a_{45} & a_{46} \end{vmatrix} = \begin{vmatrix} 6 & 3 & 2 \\ 1 & 3 & 8 \\ 8 & 1 & 0 \end{vmatrix} = 64.3 + 2 - 48 - 48 = 194 - 96 = 98$$

Дефиниция (ранг на матрица): Нека $A \in F_{m\times n}$. Казваме, че матрицата A има ранг $r = r(A)$, ако съществува ненулев минор от r-ти ред и всички минори от по-висок ред са нулеви.

По дефиниция: $r(\mathbb{O}) = 0$.

Забележка*: Ако всички минори от ред $r+1$ са равни на нула, то и всички минори от ред, по-голям от $r+1$ също са равни на нула.*

Забележка*: От теоремата за транспонирана детерминанта следва, че: $r(A^t) = r(A)$.*

Ранг на система вектори – определение и връзката му с размерността на линейната обвивка

Нека V е линейно пространство над числово поле F и $a_1, \dots, a_k$ е система вектори от V.

Дефиниция (ранг на система вектори): Казваме, че $\{a_i\}_1^k$ има ранг $r = r(\{a_i\}_1^k) = r(a_1, \dots, a_k) \leq k$, ако съществуват r на брой линейно-независими вектора и всеки друг вектор е тяхна линейна комбинация, т.е. без ограничение на общността $a_1, \dots, a_r \to$ ЛНЗ и $a_j \in l(a_1, \dots, a_r), j = \overline{r+1, k}$.

Казваме още, че сме избрали една максимална линейно независима подсистема $a_1, \dots, a_r$ на системата $a_1, \dots, a_k$.

Връзка с размерността на линейната обвивка:

$$r(a_1, \dots, a_k) = r = r(a_1, \dots, a_r) = \dim l(a_1, \dots, a_k)$$

Теорема за равенството на ранг на матрица с ранг на системата от вектор редове и ранг на системата от вектор стълбове на тази матрица

Нека $A \in F_{m\times n}$ и да означим с $a_1, \dots, a_m$ вектор-редовете на матрицата A, т.е. $a_1, \dots, a_m \in F^n$, а с $b_1, \dots, b_n$ – вектор-стълбовете на матрицата A, т.е. $b_1, \dots, b_n \in F^m$.

$$A = \begin{pmatrix} \underline{b_1} & \underline{b_2} & \dots & \underline{b_n} & \\ a_{11} & a_{12} & \dots & a_{1n} & |a_1 \\ a_{21} & a_{22} & \dots & a_{2n} & |a_2 \\ \vdots & \vdots & \vdots & \vdots & \vdots \\ a_{m1} & a_{m2} & \dots & a_{mn} & |a_m \end{pmatrix}$$

Теорема: В сила са следните равенства:

$$r(A) = r(a_1, \dots, a_m) = r(b_1, \dots, b_n) = r$$

Доказателство: Тъй като от теоремата за транспонирана детерминанта следва, че $r(A) = r(A^t) = r$, то достатъчно е да проверим, че $r(A) = r(b_1, \dots, b_n) = r$.

Нека $r(A) = r$ и без ограничение на общността първите r реда и r стълба да образуват ненулев минор от r-ти ред, т.е. от:

$$A = (a_{ij})_{m\times n} \to D = \begin{pmatrix} \underline{d_1} & \underline{d_2} & \dots & \underline{d_r} \\ a_{11} & a_{12} & \dots & a_{1r} \\ \vdots & \vdots & \vdots & \vdots \\ a_{r1} & a_{r2} & \dots & a_{rr} \end{pmatrix}, \det D = \Delta, \ \ \Delta \neq 0$$

Трябва да докажем, че $r(b_1, \dots, b_n) = r$.

Ще покажем, че векторите $b_1, \dots, b_r$ (първите r стълба на A) са линейно-независима система. Нека да допуснем противното, т.е. че $b_1, \dots, b_r$ са линейно-зависима система. В такъв случай вектор-стълбовете $d_1, \dots, d_r$ на D са също линейно-зависими, т.е. $\det D = \Delta = 0$. Противоречие с факта, че $\det D = \Delta \neq 0$. Следователно, $\{b_j\}_1^r$ образуват линейно-независима система от вектори.

Остава да докажем, че всеки стълб b_l $(r+1 \leq l \leq n)$ е линейна комбинация на векторите $b_1, \dots, b_r$, т.е. $b_l \in l(b_1, \dots, b_r)$.

Нека да образуваме матрицата D_i от $(r+1)$-ви ред като избираме произволно l $(r+1 \leq l \leq n)$ и произволно i $(1 \leq i \leq m)$:

$$D_i = \begin{pmatrix} a_{11} & a_{12} & \dots & a_{1r} & |a_{1l} \\ a_{21} & a_{22} & \dots & a_{2r} & |a_{2l} \\ \dots & \dots & \dots & \dots & |\dots \\ a_{r1} & a_{r2} & \dots & a_{rr} & |a_{rl} \\ \overline{a_{i1}} & \overline{a_{i2}} & \overline{\dots} & \overline{a_{ir}} & a_{il} \end{pmatrix}_{(r+1)\times(r+1)}$$

- Ако $1 \leq i \leq r$, то $\det D_i = 0$, тъй като в $\det D_i$ има два равни реда (от свойство на det)
- Ако $r+1 \leq i \leq m$, то $\det D_i = 0$, тъй като $r(A) = r$ (по дефиниция) и сме образували минор от ред по-голям от r

Следователно, $\det D_i = 0$. Развиваме тази детерминанта по $(r+1)$-ви ред, т.е.:

$0 = \det D_i = a_{i1}.A_1 + a_{i2}.A_2 + \dots + a_{ir}.A_r + a_{il}.A_l$, където:

$$A_l = (-1)^{r+1+r+1}.\det D = (-1)^{2(r+1)}.\Delta = \Delta \neq 0$$

Ще покажем, че A_j $(j = \overline{1,r})$ не зависят от избора на реда i, $i = \overline{1,m}$:

$$A_j = (-1)^{(r+1)+j}.\begin{vmatrix} a_{11} & \dots & a_{1j-1} & a_{1j+1} & \dots & a_{1r} & a_{1l} \\ a_{21} & \dots & a_{2j-1} & a_{2j+1} & \dots & a_{2r} & a_{2l} \\ \dots & \dots & \dots & \dots & \dots & \dots & \dots \\ a_{r1} & \dots & a_{rj-1} & a_{rj+1} & \dots & a_{rr} & a_{rl} \end{vmatrix}_{r\times r}$$

$$a_{il} = -\underbrace{\frac{A_1}{\Delta}}_{\beta_1} a_{i1} - \underbrace{\frac{A_2}{\Delta}}_{\beta_2} a_{i2} - \dots - \underbrace{\frac{A_r}{\Delta}}_{\beta_r} a_{ir}, \ \ i = \overline{1,m}$$

От горния израз за адюнгираните количества A_j видяхме, че те не зависят от i, т.е. и коефициентите $\beta_1, \beta_2, \dots, \beta_r$ не зависят от i. Така всеки елемент a_{il} на l-тия стълб b_l е линейна комбинация на съответните елементи на стълбовете $b_1, \dots, b_r$ с едни и същи коефициенти за всяко $i = \overline{1,m}$. Следователно, b_l е линейна комбинация на $b_1, \dots, b_r$ със същите коефициенти:

$$b_l = \beta_1 b_1 + \beta_2 b_2 + \cdots + \beta_r b_r, \quad b_l \in l(b_1, \dots, b_r), r+1 \leq l \leq n$$

Следователно, $r(b_1, \dots, b_n) = r$ и една максимална линейно-независима подсистема е $b_1, \dots, b_r$. Следователно, $r(A) = r = r(b_1, \dots, b_n) = r(A^t) = r(a_1, \dots, a_m)$, с което теоремата е доказана.

■

Следствие: Ако A е квадратна матрица от ред n, то $\det A = 0$ тогава и само тогава, когато редовете (стълбовете) на A са линейно-зависими.

Системи линейни уравнения – определение за съвместима и несъвместима система

Нека е дадена произволна система линейни уравнения с m уравнения и n неизвестни, т.е.:

$$(*) \quad \begin{vmatrix} a_{11}x_1 + \cdots + a_{1n}x_n = b_1 \\ a_{21}x_1 + \cdots + a_{2n}x_n = b2 \\ \dots \\ a_{m1}x_1 + \cdots + a_{mn}x_n = b_m \end{vmatrix}$$

$$A = (a_{ij})_{m \times n} \rightarrow \text{матрица на } (*)$$

$$\overline{A} = (a_{ij}|b) \rightarrow \text{разширена матрица на } (*)$$

Дефиниция (несъвместима система): Казваме, че системата е несъвместима, ако няма решение, т.е. $0x_1 + \cdots + 0x_n = c \neq 0$.

Дефиниция (съвместима система): Казваме, че системата е съвместима, ако съществува нейно решение:

- Системата е съвместима и определена, ако съществува единствено решение
- Системата е съвместима и неопределена, ако съществуват безброй много решения (повече от едно)

Системи линейни уравнения – Теорема на Руше

Критерий за съвместимост на системата линейни уравнения $(*)$ дава теоремата на Руше:

Теорема (Руше): Една система линейни уравнения $(*)$ е съвместима тогава и само тогава, когато $r(A) = r(\overline{A})$.

Доказателство: Да означим с $b_1, \dots, b_n$ вектор-стълбовете на матрицата A, а с b – стълба от свободните членове. Имаме, че:

$$r(A) = r(b_1, \dots, b_n) \leq r(b_1, \dots, b_n; b) = r(\overline{A})$$

Ако $r(A) = r(\overline{A}) \Leftrightarrow b \in l(b_1, \dots, b_n) \Leftrightarrow \exists \alpha_i\ i = \overline{1,n} : \alpha_1 b_1 + \alpha_2 b_2 + \dots + \alpha_n b_n = b \Leftrightarrow$
$(\alpha_1, \dots, \alpha_n)$ е решение на $(*)$.

Извод: *Една система е съвместима и определена, ако:* $r(A) = r(\overline{A}) = r = n =$ брой неизвестни *и решението се намира с формули на Крамер. Една система е съвместима и неопределена, ако* $r(A) = r(\overline{A}) = r < n$ *и решенията са резултат от заместване на свободните неизвестни с подходящи стойности и пресмятане на свързаните неизвестни с формули на Крамер, за да се достигне до решенията на системата* $(*)$.

Пример: $(\underbrace{x_1 = 5p + 7r, x_2 = q + 4s, \dots, x_r = t + s}_{\text{свързани неизвестни}}, \underbrace{x_{r+1} = p, x_{r+2} = q, \dots, x_n = t}_{\text{свободни неизвестни}})$

Хомогенни системи – доказателство, че множеството от решения е подпространство

Хомогенна система е система, в която свободните членове са равни на нула.

Нека е дадена системата:

$$(**)\ |Ax = \mathbb{0}, A \in F_{m\times n}, x^t = (x_1, \dots, x_n), \overline{A} = (A|\mathbb{0}) \Rightarrow r(A) = r(\overline{A})$$

Една хомогенна система линейни уравнения е винаги съвместима и едно нейно решение е $\mathbb{0}$ $(A\mathbb{0} = \mathbb{0})$.

Ако $r(A) = r(\overline{A}) = m = n \Leftrightarrow \det A \neq 0 \Rightarrow (**)$ има единствено решение $\mathbb{0}$.

$$\left|\begin{matrix} x_1 = 0 \\ x_2 = 0 \\ \vdots \\ x_n = 0 \end{matrix}\right. \Leftrightarrow |Ex = \mathbb{0}$$

Ако $r(A) = r(\overline{A}) = r < n \Rightarrow \det A = 0$.

Без ограничение на общността приемаме, че:

$$D = \begin{pmatrix} a_{11} & \dots & a_{1r} \\ \dots & \dots & \dots \\ a_{r1} & \dots & a_{rr} \end{pmatrix}, \det D \neq 0$$

$(**) \Leftrightarrow$ ще има и ненулеви решения

$$\left|\begin{matrix} a_{11}x_1 + a_{12}x_2 + \dots + a_{1r}x_r = -a_{1r+1}x_{r+1} - \dots - a_{1n}x_n \\ \dots \\ a_{r1}x_1 + a_{r2}x_2 + \dots + a_{rr}x_r = -a_{rr+1}x_{r+1} - \dots - a_{rn}x_n \end{matrix}\right.$$

Твърдение: Множеството от решения на една хомогенна система линейни уравнения е подпространство на $V = F^n$, т.е. $(**)\ U: |Ax = \mathbb{0}, U \leq V$

Доказателство:

$\forall w = (w_1, \dots, w_n)$ е решение на $(**)$

$\forall u = (u_1, \dots, u_n)$ е решение на $(**)$

$\forall \alpha, \beta \in F \quad ? \alpha u + \beta w$ е също решение на $(**) \Rightarrow U \leq V$

u – решение на $(**) : a_{11}u_1 + a_{12}u_2 + \dots + a_{1n}u_n = 0$

w – решение на $(**) : a_{11}w_1 + a_{12}w_2 + \dots + a_{1n}w_n = 0$

Тогава $\alpha u + \beta w : a_{11}(\alpha u_1 + \beta w_1) + a_{12}(\alpha u_2 + \beta w_2) + \dots + a_{1n}(\alpha u_n + \beta w_n) =^F \alpha(a_{11}u_1 + \dots + a_{1n}u_n) + \beta(a_{11}w_1 + \dots + a_{1n}w_n) = \alpha.0 + \beta.0 = 0 \Rightarrow \alpha u + \beta w$ е решение на $(**) \Rightarrow U \leq V$ ■

Хомогенни системи – определение за фундаментална система от решения

Дефиниция (фундаментална система от решения): Нека $U: |Ax = \mathbb{0}$. Всеки базис на U наричаме една фундаментална система от решения на хомогенната система от линейни уравнения $(**)$.

Така всяка фундаментална система от решения се състои от линейно-независими вектори от F^n, които са решения на хомогенната система и всяко друго решение е тяхна линейна комбинация.

Хомогенни системи – теорема за размерността на подпространството от решения на хомогенна система

Теорема: Нека U е пространството от решения на една хомогенна система линейни уравнения $Ax = \mathbb{0}$ и нека $r(A) = r$. Тогава $U: |Ax = \mathbb{0}$ има $\dim U = n - r$, т.е. всяка фундаментална система решения на U се състои от $n - r$ на брой решения.

Теорема: Всяко подпространство U на линейното пространство $V = F^n$ е пространството от решенията на подходяща хомогенна система линейни уравнения с n неизвестни.

Държавен изпит – спец. „Софтуерно инженерство“

Тема №28 – Полиноми на една променлива – определение, действия с полиноми и степен на полином. Теорема за делене на полиноми с частно и остатък, правило на Хорнер. Най-голям общ делител на полиноми – тъждество на Безу и алгоритъм на Евклид. Корени на полином и зависимости между корените и коефициентите на полином (формули на Виет).

Анотация: *Във въпроса се включва:*

- Определение за полиноми на една променлива с коефициенти над поле, действия с полиномите, степен на полином и твърдение за степента на сумата и на произведението на полиноми (с доказателство);

- Теорема за деление с частно и остатък при полиноми (с доказателство), правило (схема) на Хорнер;

- Най-голям общ делител (НОД) на полиноми с коефициенти от поле – определение, теорема за съществуване на НОД на полиноми и тъждество на Безу (с доказателство), алгоритъм на Евклид за намираме на НОД (с доказателство, че се получава НОД на полиномите);

- Корени на полином – определение, принцип за сравняване на коефициентите (само формулировка), формули на Виет за полином от степен n.

Полиноми на една променлива с коефициенти над поле

Нека A е комутативен пръстен с единица / област / F – поле. Нека B е множеството от всички безкрайни редици с елементи от A, в които имаме краен брой ненулеви елементи:

$$B = \{f = (a_0, a_1, a_2, \dots, a_n, \dots) | a_i \in A\}$$

Елементите (безкрайните редици) на така дефинираното множество B ще наричаме **полиноми**.

Дефинираме действия събиране („+“) и умножение („·“) по следния начин:

$\boldsymbol{f = (a_0, a_1, a_2, \dots, a_n, \dots), g = (b_0, b_1, b_2, \dots, b_m, \dots)}$**, където б. о. о.** $\boldsymbol{m < n}$**, то:**

$$\boldsymbol{f + g := (a_0 + b_0, a_1 + b_1, a_2 + b_2, \dots, a_m + b_m, a_{m+1}, \dots, a_n, \dots)}$$

$$\boldsymbol{fg := (c_0, c_1, c_2, \dots, c_k, \dots)}, \text{където } \boldsymbol{c_s = \sum_{i+j=s} a_i b_j}, \text{ т. е. } \boldsymbol{c_k = (a_0 b_0, a_1 b_0 + a_0 b_1, \dots)}$$

Проверява се, че $(B, +, \cdot)$ се получава комутативен пръстен с единица.

$$(-f) = (-a_0, -a_1, -a_2, \dots, -a_n, \dots), 0 - \text{нула}, 1 = (1,0, \dots, 0) - \text{единица}$$

Две безкрайни редици (полинома) f и g са **равни**, т.е. $f = g \Leftrightarrow a_0 = b_0, a_1 = b_1, \dots, a_n = b_n$ и всички останали елементи са нули.

Ако A е ***комутативен пръстен с единица***, то се получава, че и B е ***комутативен пръстен с единица***. Още повече, ако A е било ***област*** (т.е. A е комутативен пръстен с единица, в който няма делители на нулата), то се оказва, че и B е ***област***. За A – поле, това не е вярно.

Константните полиноми, т.е. редиците: $a = (a, 0, \dots, 0, \dots) \cong a \in A \subseteq B$, т.е. всички елементи от A могат да се разглеждат като елементи от B.

Означаваме с:

$$x = (0,1,0, \dots) \in B$$

$$x^2 = (0,1,0, \dots, 0, \dots)(0,1,0, \dots, 0, \dots) = (0,0,1,0, \dots, 0, \dots)$$

$$x^3 = (0,1,0, \dots, 0, \dots)(0,0,1,0, \dots, 0, \dots) = (0,0,0,1,0, \dots, 0, \dots)$$

$$x^i = \left(0,0, \dots, \underbrace{1}_{(i+1)-\text{място}}, 0, \dots\right)$$

$$ax = (a, 0, \dots)(0,1,0, \dots) = (0, a, 0, \dots)$$

$$f = (a_0, a_1, \dots, a_n, \dots) = (a_0, 0, \dots) + (0, a_1, 0, \dots) + \cdots + (0, \dots, a_n, \dots)$$
$$= a_0 + a_1 x + a_2 x^2 + \cdots a_n x^n$$

Дефиниция (полином на една променлива с коефициенти над поле): Нека $B = \{f = a_0 + a_1 x + a_2 x^2 + \cdots + a_n x^n | a_i \in A, x - \text{променлива}\}$. f наричаме полином на променливата x с коефициенти от A.

Друг удобен запис: $f = a_0 x^n + a_1 x^{n-1} + \cdots + a_{n-1} x + a_n \in B$

$B = A[x], f \in A[x]$ – *полиномиален пръстен на променливата* x *с коефициенти от* A

a_0 – *старши коефициент*, a_n – *свободен коефициент*, a_i – *коефициенти*, x – *променлива*

n – **степен** на f и означаваме $\mathbf{deg}\, \boldsymbol{f} \coloneqq \boldsymbol{n}$; $a \neq 0, a \in A$ – *константни полиноми* $\deg a = 0$

$\deg 0 \coloneqq -\infty \rightarrow$ *нулев полином*

Твърдение (за степента на сумата и произведението на полиноми): Нека $f, g \in A[x]$ и $\deg f = n$ и $\deg g = m$. Тогава:

а.) $\mathbf{deg}(\boldsymbol{f} + \boldsymbol{g}) \leq \mathbf{max}(\mathbf{deg}\, \boldsymbol{f}, \mathbf{deg}\, \boldsymbol{g})$

б.) $\mathbf{deg}(\boldsymbol{fg}) = \mathbf{deg}\, \boldsymbol{f} + \mathbf{deg}\, \boldsymbol{g}$ **, само ако** $\boldsymbol{A}$ **е област, т.е. и** $\boldsymbol{A}[\boldsymbol{x}]$ **е област**

Доказателство:

Нека $f(x) = a_0 + a_1 x + a_2 x^2 + \cdots + a_n x^n, a_n \neq 0$, тъй като $\deg f = n$.

Нека $g(x) = b_0 + b_1 x + b_2 x^2 + \cdots + b_m x^m, b_m \neq 0$, тъй като $\deg g = m$.

а.) $\boldsymbol{f(x) + g(x) = a_0 + b_0 + a_1 x + b_1 x + \cdots + a_n x^n + b_m x^m}$**.**

1 сл.) Ако $m < n$, то $\deg(f + g) = n$, тъй като n остава най-високата степен в сумата

2 сл.) Ако $n < m$, то $\deg(f + g) = m$, тъй като m остава най-високата степен в сумата

3 сл.) Ако $n = m$, то $\deg(f+g) = n = m$, тъй като най-високите степени са едни и същи

Следователно, $\deg(f+g) = \begin{cases} n, & \text{ако } m < n \\ m, & \text{ако } n \leq m \end{cases} = \max(n,m) = \max(\deg f, \deg g)$.

б.) $\boldsymbol{f(x).g(x) = a_0b_0 + a_0b_1x + a_1b_0x + \cdots + a_nb_mx^{n+m}}$**.**

Понеже A е област, то нямаме делители на нулата, т.е. $a_nb_m \neq 0$.

Следователно, $\deg(fg) = n + m = \deg f + \deg g$. ■

Функция определена от полином:

$\forall f \in A[x]$ *определя функция:*

$$f: \begin{cases} A \to A \\ a \to f(a) = a_0a^n + a_1a^{n-1} + \cdots + a_{n-1}a + a_n \end{cases}$$

$$(f+g)(a) = f(a) + g(a)$$

$$(fg)(a) = f(a)g(a)$$

$f = g$ като полиноми $\Rightarrow f = g$ като функции, но обратното в общия случай не е вярно

Теорема за деление с частно и остатък при полиноми

Нека F е поле и $F[x]$ е полиномиалният пръстен на променливата x с коефициенти от F. В този случай, $F[x]$ е област.

$$f(x) = a_0x^n + a_1x^{n-1} + \cdots + a_{n-1}x + a_n, a_0 \neq 0$$

Теорема (за деление с частно и остатък при полиноми): Нека $f, g \in F[x], g \neq 0$. Тогава съществуват единствени полиноми q (наречен ***частно***) и r (наречен ***остатък***), $q, r \in F[x]$, такива че са в сила:

1. $\boldsymbol{f = gq + r}$
2. $\boldsymbol{\deg r < \deg g}$

Доказателство:

I. Съществуване

$$f = a_0x^n + a_1x^{n-1} + \cdots + a_{n-1}x + a_n, a_0 \neq 0$$

$$0 \neq g = b_0x^m + b_1x^{m-1} + \cdots + b_{m-1}x + b_m, b_0 \neq 0$$

$g = a \in F, a \neq 0, f = g\left(\underbrace{a^{-1}f}_{q}\right) + \underbrace{0}_{r}$, $\deg r < \deg g$, т.е. $-\infty < 0$

Когато $\deg g \geq 1$ правим индукция по $\deg f = n$.

Ако $m > n$, то $f = g.\underbrace{0}_{q} + \underbrace{f}_{r}$, $\deg r < \deg g$, т.е. $n < m$.

Нека $n \geq m > 0$.

$\boldsymbol{f = a_0x^n + \cdots}$ $\quad$ $\boldsymbol{|\underline{g = b_0x^m + \cdots}}$

$\boldsymbol{\bar{}\ \underline{a_0x^n + \cdots}}$ $\quad$ $\boldsymbol{q = b_0^{-1}a_0x^{n-m}}$

$\boldsymbol{f_1}$, $\quad \boldsymbol{\deg f_1 < n}$

Извършихме следното: $f - a_0b_0^{-1}x^{n-m}g = f_1, \deg f_1 < n$ и по ИП за f_1 съществуват полиноми q_1 и $r_1, q_1, r_1 \in F[x]$, такива че:

$$f_1 = q_1g + r_1, \deg r_1 < \deg g$$

$$\Rightarrow f = f_1 + a_0b_0^{-1}x^{n-m}g = q_1g + r_1 + a_0b_0^{-1}x^{n-m}g = \underbrace{(q_1 + a_0b_0^{-1}x^{n-m})}_{q}g + \underbrace{r_1}_{r}$$

Получихме, че $f = gq + r, \deg r < \deg g$, което и искахме да докажем.

II. Единственост

Да допуснем противното, т.е. че $f = gq_1 + r_1 = gq_2 + r_2$, където $\deg r_1 < \deg g, \deg r_2 < \deg g$.

След групиране получаваме, че: $g(q_1 - q_2) = r_2 - r_1$ в $F[x]$ $\quad (*)$.

Използваме твърдението за степента на произведението на полиноми (тъй като сме в поле):

$$\deg g(q_1 - q_2) = \deg g + \deg(q_1 - q_2) > \deg g > \deg(r_1 - r_2)$$

$$(\deg r_1 < \deg g, \deg r_2 < \deg g, \text{т.е. } \deg(r_1 - r_2) < \deg g)$$

Получихме противоречие с факта, че имаме равенството в $(*)$, а степените на лявата и дясната страна са различни (степента на лявата е строго по-голяма от степента на дясната). Следователно, съществуват единствени q и r, такива че $f = gq + r$ и $\deg r < \deg g$. ■

Правило (схема) на Хорнер

Твърдение (схема на Хорнер): Нека $f = a_0x^n + a_1x^{n-1} + \cdots + a_{n-1}x + a_n \in F[x]$,

$g = x - \alpha \in F[x]$. Нека $f = gq + r, q = b_0x^{n-1} + b_1x^{n-2} + \cdots + b_{n-2}x + b_{n-1}$.

$\deg r < \deg g = 1 \Rightarrow \deg r \leq 0 \Rightarrow r = const \in F$

Тогава са в сила следните равенства:

	$\boldsymbol{a_0}$	$\boldsymbol{a_1}$	$\boldsymbol{a_2}$	$\boldsymbol{\ldots}$	$\boldsymbol{a_{n-1}}$	$\boldsymbol{a_n}$
$\boldsymbol{\alpha}$	b_0	b_1	b_2	$\ldots$	b_{n-1}	r

като сравняваме полиномите

тоест $f = gq + r \Leftrightarrow (\ \ )x^k = (\ \ )x^k, \forall k = \overline{0,n}$

$$\boldsymbol{b_0 = a_0}$$

$$\boldsymbol{b_1 = a_1 + \alpha b_0}$$

$$\boldsymbol{b_2 = a_2 + \alpha b_1}$$

… … … … … … … … …

$$\boldsymbol{b_{n-1} = a_{n-1} + \alpha b_{n-2}}$$

$$\boldsymbol{r = a_n + \alpha b_{n-1}}$$

Следствие (принцип за сравняване на коефициентите): Нека A е област и полиномите $f, g \in A[x]$ и $\deg f \leq n$ и $\deg g \leq n$ и съществуват $(n+1)$ на брой два по два различни елементи $\beta_1, \beta_2, \ldots \beta_{n+1}$ $(\beta_j \in A, \beta_i \neq \beta_j, i \neq j)$, такива че $f(\beta_j) = g(\beta_j)$. Тогава $f = g$.

Най-голям общ делител на полиноми с коефициенти от поле

Нека F е поле, а $F[x]$ е област, т.е. нямаме делители на нулата.

Дефиниция (делимост на полиноми): Нека $f, g \in F[x], g \neq 0$. Казваме, че „g дели f“ и означаваме g/f $(g|f)$, ако съществува $h \in F[x]$, такъв че $f = gh$. В противен случай, $g \nmid f$.

Свойства:

1. $bf|af, b \in F^*, F^* = F\backslash\{0\}, a \in F$
2. $f|g$ *и* $g|f$, *то* $g = cf, c \in F^*$

Казваме, че f *е* ***унитарен полином****, ако старшият му коефициент на* $f = a_0 = 1$.

В частност, ако f *и* g *са унитарни и* $f|g$ *и* $g|f$, *то* $f = g$.

3. $f|g \Rightarrow af|gb, a \in F^*, a, b \in F$
4. $f|g$ *и* $g|h \Rightarrow f|h$ ***(транзитивност)***
5. $g|f_1, g|f_2, \ldots, g|f_k \Rightarrow$ *то* $g|f_1h_1 + f_2h_2 + \cdots + f_kh_k, h_i \in F[x]$
6. $g|f_1 + f_2$ *и* $g|f_1 \Rightarrow g|f_2$

В частност, ако $f_1 + f_2 = 0$ *и* $g|f_1$, *то* $g|f_2$.

Дефиниция (най-голям общ делител на полиноми): Нека $f, g \in F[x], g \neq 0$. Казваме, че един полином $d \in F[x]$ е най-голям общ делител (НОД) на f и g и означаваме $d = (f, g)$, ако за d са изпълнени следните две условия:

1. $\boldsymbol{d|f}$ **и** $\boldsymbol{d|g}$
2. $\boldsymbol{d_1|f}$ **и** $\boldsymbol{d_1|g}$**, то** $\boldsymbol{d_1|d}$

По-общо, ако $f_1, \ldots, f_k \in F[x], f_1 \neq 0 \Rightarrow$ НОД на $f_1, \ldots, f_k = (f_1, \ldots, f_k) = ((f_1, \ldots, f_{k-1}), f_k)$

Дефиниция (взаимно прости полиноми): Казваме, че f и g са взаимно прости, ако $(f,g) = 1$ ($(f,g) = a \in F^*$).

Важно: Ако $a \in F^*$ и $d = (f,g) \Rightarrow ad = (f,g)$

НОД(f,g) е определен с точност до ненулева константа от F.

Дефиниция (идеал): Нека A е комутативен пръстен с единица и I е подмножество на A. Казваме, че I е идеал на пръстена A и означаваме $I \trianglelefteq A$, ако са изпълнени следните две условия:

1. $\forall a,b \in I \Rightarrow a - b \in I; a + (-b)$
2. $\forall a \in I$ и $\forall r \in A \Rightarrow ar = ra \in I$

Дефиниция (главен идеал): Главен идеал, породен от елемента $b \in A$ наричаме $I = < b > = (b) = \{br | r \in A\} \trianglelefteq A$.

Теорема: Нека F е поле. Тогава всеки идеал I във $F[x]$ е главен идеал.

Доказателство: Нека $I = \{0\}$. Тогава е ясно, че I е главен идеал. Нека $I \neq \{0\}$. Тогава нека вземем ненулев полином g, който е от минимална степен от I. Ще докажем, че $< g > = I$.

1. $I \subseteq < g >$) Нека вземем произволен полином $f \in I$. Ще докажем, че $f \in < g >$. От теоремата за деление с частност и остатък можем да представим f по следния начин: $f = qg + r$, където $\deg r < \deg g$. Но това означава, че $r = f - qg$ и $r \in I$. Нека $r \neq 0$. Тогава получаваме противоречие с избора на g да бъде от минимална степен от I, тъй като получихме, че $\deg r < \deg g$. Следователно, $r = 0$. Тогава: $f = gq$, т.е. $f \in < g >$, $\forall f \in I$. Следователно, $I \subseteq < g >$.
2. $< g > \subseteq I$) Нека вземем произволно $s \in < g >$. Тогава $s = qg$, където $g \in I$. Следователно, $s \in I$.

От 1. и 2. получаваме, че $I = < g >$, което искахме да докажем. ■

Теорема (за съществуване на НОД на полиноми): Нека $f,g \in F[x], g \neq 0$. Тогава съществува $d = (f,g)$.

I-во доказателство: Нека I е идеала, породен от полиномите f и g, т.е. $I =< f,g > = \{uf + vg | u,v \in F[x]\}$. Тъй като F е поле, то всеки идеал във $F[x]$ е главен (от теорема), т.е. I е главен. Доказва се, че полиномът d, който поражда I е $d = (f,g)$ и в частност съществуват полиноми $u,v \in F[x]$, такива че $\boldsymbol{uf + vg = d}$ (**тъждество на Безу**).

1. $f,g \in I \Rightarrow f = f_1 d, g = g_1 d$ за някакви $f_1, g_1 \in F[x]$. Това обаче означава, че $d|f$ и $d|g$.
2. $d \in I \Rightarrow d = uf + vg$ за някакви $u,v \in F[x]$. Сега нека $d_1|f$ и $d_1|g$. Тогава от свойствата на делимостта следва, че $d_1|uf + vg$ за всякакви $u,v \in F[x]$. Следователно, $d_1|d$.

От горните две свойства следва, че d е НОД на f и g, т.е. $d = (f,g)$.

II-ро доказателство:

Алгоритъм на Евклид

Нека $\boldsymbol{f = gq + r, \deg r < \deg g}$.

Ако $r = 0 \Rightarrow (f, g) = g$, иначе

$\boldsymbol{| \; g = rq_1 + r_1, \deg r_1 < \deg r}$

$\boldsymbol{| \; r = r_1 q_2 + r_2, \deg r_2 < \deg r_1 < \deg r}$

… …

$\boldsymbol{| \; r_{k-2} = r_{k-1} q_k + r_k, \deg r_k < \deg r_{k-1}}$

$\boldsymbol{| \; r_{k-1} = r_k q_{k+1}, r_{k+1} = 0,}$

$\boldsymbol{r_k}$ е последният ненулев остатък, който е търсеният НОД с точност до константа

$\boldsymbol{| \; r_k = da, d = (f, g), a \in F^*}$

1) от дефиницията на НОД се доказва тръгвайки от долу нагоре, т.е. $d|r_k$, но и $r_k|r_{k-1} \Rightarrow d|r_{k-1}$ и т.н. чрез заместване на остатъците получаваме, че $d|g$ и $d|f$.

2) от дефиницията на НОД се доказва тръгвайки от горе надолу, т.е. нека $d_1|f$ и $d_1|g \Rightarrow d_1|r$ и т.н. стигаме до $d_1|r_{k-2}$ и $d_1|r_{k-1}$ значи $d_1|r_k$ и $d_1|d$.

$d = r_k = r_{k-2} - r_{k-1}q_k = \cdots = f(\underbrace{\ldots}_{u}) + g(\underbrace{\ldots}_{v}) \Rightarrow$ получаваме тъждеството на Безу $d = uf + vg$

Корени на полином

Нека F е поле и $f(x) \in F[x]$ е полином с коефициенти над това поле.

Дефиниция (корен на полином): Нека $f(x) \in F[x]$ и $\alpha \in K \geq F$. Казваме, че α е корен на f, ако $f(\alpha) = (x - \alpha)g = 0, g \in K[x]$.

Теорема: Нека $f \in F[x], \deg f > 0$. Тогава съществува разширение $K \geq F$, в което f има корен.

Следствие: Нека $f \in F[x], \deg f = n > 0$. Тогава съществува разширение L на полето F, в което f се разлага на линейни множители: $f = a_0(x - \alpha_1) \ldots (x - \alpha_n)$, т.е. всички корени на f са в това разширение.

Дефиниция (поле на разлагане): Нека $f \in F[x], \deg f > 0$ и L е най-малкото разширение на полето F, в което се съдържат всички корени на полинома f и F, т.е. $f = a_0(x - \alpha_1) \ldots (x - \alpha_n) \in L[x]$. Казваме, че L е поле на разлагане на полинома f над полето F.

Теорема (единственост на полето на разлагане на полином с коефициенти от поле): Нека $f \in F[x], \deg f > 0$. Нека L_1 и L_2 са две полета на разлагане на f над полето F. Тогава $L_1 \cong L_2$, т.е. с точност до изоморфизъм полето на разлагане на полинома f над полето F е единствено.

Формули на Виет

$f = a_0x^n + a_1x^{n-1} + \cdots + a_n \in F[x], a_0 \neq 0$ и нека L е полето на разлагане на f над полето F, т.е. в L f има вида $f = a_0(x-\alpha_1)\ldots(x-\alpha_n), \alpha_i \in L, i = \overline{1,n}$ са корените на f. Тогава $f \in L[x]$:

$$\boldsymbol{f = a_0x^n + a_1x^{n-1} + \cdots + a_n = a_0(x-\alpha_1)\ldots(x-\alpha_n), a_0 \neq 0}$$

В сила са следните формули на Виет: **брой събираеми в σ_i: $\binom{n}{i}$**

$$\binom{n}{1}|\sigma_1 = \alpha_1 + \alpha_2 + \cdots + \alpha_n = -\frac{a_1}{a_0}$$

$$\binom{n}{2}|\sigma_2 = \alpha_1\alpha_2 + \alpha_1\alpha_3 + \cdots + \alpha_{n-1}\alpha_n = \frac{a_2}{a_0}$$

$$\binom{n}{3}|\sigma_3 = \alpha_1\alpha_2\alpha_3 + \cdots + \alpha_{n-2}\alpha_{n-1}\alpha_n = (-1)^3\frac{a_3}{a_0}$$

$$\ldots\ldots\ldots\ldots\ldots\ldots\ldots\ldots\ldots\ldots\ldots\ldots\ldots\ldots\ldots\ldots$$

$$\binom{n}{n}|\sigma_n = \alpha_1\alpha_2\ldots\alpha_n = (-1)^n\frac{a_n}{a_0}$$

Следствие: Нека е дадено поле L, в което $\alpha_1, \ldots, \alpha_n \in L$ и образуваме $\sigma_1, \ldots, \sigma_n$. То тогава $\alpha_1, \ldots, \alpha_n$ са корени на полинома:

$$f = x^n - \sigma_1x^{n-1} + \sigma_2x^{n-2} + \cdots + (-1)^n\sigma_n \in L[x]$$

Формули на Виет за полином от трета степен:

$$f = ax^3 + bx^2 + cx + d, \qquad \alpha_1, \alpha_2, \alpha_3 \to \text{корени}, L[x]$$

$$\sigma_1 = \alpha_1 + \alpha_2 + \alpha_3 = -\frac{b}{a}$$

$$\sigma_2 = \alpha_1\alpha_2 + \alpha_1\alpha_3 + \alpha_2\alpha_3 = \frac{c}{a}$$

$$\sigma_3 = \alpha_1\alpha_2\alpha_3 = -\frac{d}{a}$$

Формули на Виет за полином от четвърта степен:

$$f = ax^4 + bx^3 + cx^2 + dx + m, \quad \alpha_1, \alpha_2, \alpha_3, \alpha_4 \to \text{корени}, L[x]$$

$$\sigma_1 = \alpha_1 + \alpha_2 + \alpha_3 + \alpha_4 = -\frac{b}{a}$$

$$\sigma_2 = \alpha_1\alpha_2 + \alpha_1\alpha_3 + \alpha_1\alpha_4 + \alpha_2\alpha_3 + \alpha_2\alpha_4 + \alpha_3\alpha_4 = \frac{c}{a}$$

$$\sigma_3 = \alpha_1\alpha_2\alpha_3 + \alpha_1\alpha_2\alpha_4 + \alpha_1\alpha_3\alpha_4 + \alpha_2\alpha_3\alpha_4 = -\frac{d}{a}$$

$$\sigma_4 = \alpha_1\alpha_2\alpha_3\alpha_4 = \frac{m}{a}$$

Дефиниция (k-кратен корен на полином): Нека $f \in F[x]$ и L е разширение на F, в което $\alpha \in L$. Казваме, че α е k-кратен корен на $f(x)$, ако: $f(x) = (x-\alpha)^k g, g \in L[x], g(\alpha) \neq 0$. Ако $k = 1$, то α е еднократен (прост) корен на f.

Нека $f^{(0)} = f$ и $f', f'', \dots, f^{(n)}$ са формалните производни на полином $f(x) \in F[x]$.

Теорема (критерий за кратност на корен в поле с характеристика 0): *Нека F е поле и $char\ F = 0$ и $f \in F, L \geq F$ и $\alpha \in L$. То α е k-кратен корен на $f(x) \Leftrightarrow f(\alpha) = f'(\alpha) = \dots = f^{(k-1)}(\alpha) = 0, f^{(k)}(\alpha) \neq 0$.*

Теорема (общ критерий): *Един полином $f \in F[x]$ има кратен корен $\alpha \in L \geq F \Leftrightarrow L$ е общ корен на $f(x)$ и $f'(x)$.*

Държавен изпит – спец. „Софтуерно инженерство“

Тема №29 – Дискретни разпределения. Равномерно, биномно, геометрично и Поасоново разпределение. Задачи, в които възникват. Моменти – математическо очакване и дисперсия.

Анотация: *На изпита комисията дава две разпределения, върху които се развива въпросът. Дефиниция на дискретно вероятностно разпределение на случайна величина. Свойства на вероятностите (неотрицателност и нормираност, монотонност и адитивност). За всяко от дадените две разпределения да се посочи пример, при който то възниква. Да се пресметне математическото очакване и дисперсията на всяко от тези разпределения. При пресмятанията може да се използва пораждаща функция или пораждаща моментите функция, но тя трябва да се дефинира и да се покажат основните ѝ свойства (без доказателство).*

Дискретно вероятностно разпределение на случайна величина

Дефиниция (случаен експеримент /опит/): Случаен експеримент е опит, при който изходът не е предефиниран. Такъв възможен изход бележим с ω и наричаме елементарно събитие. Множеството от всички възможни изходи означаваме с Ω и наричаме съвкупност от елементарни събития.

Дефиниция (събитие): Нека Ω е съвкупност от елементарни събития. Тогава $A \subseteq \Omega$ наричаме събитие.

Дефиниция (σ-алгебра): Нека Ω е съвкупност от елементарни събития и $\mathcal{A}$ е колекция от подмножества на Ω (т.е. събития). Казваме, че $\mathcal{A}$ е σ-алгебра, ако:

1. $\Omega \in \mathcal{A} (\emptyset \in \mathcal{A})$
2. $A \in \mathcal{A} \Rightarrow A^C \in \mathcal{A}$
3. Ако $A_i \in \mathcal{A}, i \geq 1$, то $\bigcup_{i=1}^{\infty} A_i \in \mathcal{A}$

Дефиниция (вероятност): Нека $\mathcal{A}$ е σ-алгебра върху множество от елементарни събития Ω. Тогава изображението $\mathbb{P}: \mathcal{A} \to [0,1]$ се нарича вероятност (вероятностна функция), ако:

1. $\mathbb{P}(\Omega) = 1$ (**нормираност**)
2. Ако $A \in \mathcal{A}$, то $\mathbb{P}(A^C) = 1 - \mathbb{P}(A)$ (**допълнителност**)
3. Ако $A_i, i \geq 1$ са елементи на $\mathcal{A}$ и $A_i \cap A_j = \emptyset$ за $i \neq j$, то:

$$\mathbb{P}\left(\bigcup_{i=1}^{\infty} A_i\right) = \sum_{i=1}^{\infty} \mathbb{P}(A_i) \ \ (\sigma - \text{адитивност})$$

В този смисъл вероятността е мярка, тъй като това е най-класическото свойство на мярката.

Теорема: Нека $\mathcal{A}$ е σ-алгебра и $\mathbb{P}: \mathcal{A} \to [0,1]$ е вероятностна функция. Тогава за $A, B \in \mathcal{A}$ са изпълнени следните свойства:

1. $\mathbb{P}(\emptyset) = 0$
2. $\mathbb{P}(A) \geq 0$ за всяко $A \in \mathcal{A}$ (**неотрицателност**)
3. $\mathbb{P}(A) = \mathbb{P}(A \cap B) + \mathbb{P}(A \cap B^C)$ за всяко $A, B \in \mathcal{A}$

4. $\mathbb{P}(A) \leq \mathbb{P}(B)$, ако $A \subseteq B \in \mathcal{A}$ (**монотонност**)
5. $\mathbb{P}(A \cup B) = \mathbb{P}(A) + \mathbb{P}(B)$, когато $A \cap B = \emptyset$ (**адитивност**)
6. $\mathbb{P}(A \cup B) = \mathbb{P}(A) + \mathbb{P}(B) - \mathbb{P}(A \cap B)$ (**силна адитивност**)
7. Ако $A_1, A_2, \ldots \in \mathcal{A}$, то:
$$\mathbb{P}\left(\bigcup_{i=1}^{\infty} A_i\right) \leq \sum_{i=1}^{\infty} \mathbb{P}(A_i) \text{ (полуадитивност)}$$
8. Ако $A_j \in \mathcal{A}$ за $j \geq 1$ и $A_j \supseteq A_{j+1}$ за всяко $j \geq 1$, то:
$$\mathbb{P}\left(\bigcap_{j=1}^{\infty} A_j\right) = \lim_{j \to \infty} \mathbb{P}(A_j) \text{ (непрекъснатост)}$$

Дефиниция (вероятностно пространство): Нека Ω е пространство от елементарни събития, $\mathcal{A}$ е σ-алгебра и $\mathbb{P}: \mathcal{A} \to [0.1]$ е вероятностна функция. Тогава $(\Omega, \mathcal{A}, \mathbb{P})$ се нарича вероятностно пространство.

Дефиниция (случайна величина): Нека $V = (\Omega, \mathcal{A}, \mathbb{P})$ е вероятностно пространство. Тогава $X: \Omega \to \mathbb{R}$ е случайна величина, ако за $\forall a < b, a, b \in \mathbb{R}$ е вярно, че: $X^{-1}((a, b)) = \{\omega \in \Omega: a < X(\omega) < b\} = \{a < X < b\} \in \mathcal{A}$, т.е. трябва да имаме възможност да кажем каква е вероятността X да е между a и b за всяко a и b.

Дефиниция (индикаторна функция): Нека Ω е множество от елементарни събития и $H \subseteq \Omega$. Тогава $1_{H(\omega)}$ се нарича индикаторна функция, ако:
$$1_{H(\omega)} = \begin{cases} 1, \text{ако } \omega \in H \\ 0, \text{ако } \omega \in H^C \end{cases}$$

Лема: Нека $V = (\Omega, \mathcal{A}, \mathbb{P})$ е вероятностно пространство и нека $A \in \mathcal{A}$. Тогава 1_A е случайна величина.

Въвеждаме следните означения:
$$H = \begin{cases} H_1, \ldots, H_n: \coprod_{j=1}^{n} H_j = \Omega: H_j \in \mathcal{A}; 1 \leq j \leq n \\ H_1, \ldots, \text{където } H_j \in \mathcal{A}, H_i \cap H_j = \emptyset, i \neq j, \coprod_{j=1}^{\infty} H_j = \Omega \end{cases} \to \text{пълна група от събития във } V$$
$$\overline{x} = \begin{cases} (x_1, \ldots, x_n) \\ (x_1, x_2, \ldots) \end{cases} \to \text{редица от реални числа}$$

Дефиниция (дискретна случайна величина): Нека $V = (\Omega, \mathcal{A}, \mathbb{P})$ е вероятностно пространство и H е пълна група от събития в него и $\overline{x}$ е редица от реални числа. Тогава:
$$X(\omega) = \sum_{j=1}^{n} x_j 1_{H_j(\omega)} \ \left(X(\omega) = \sum_{j=1}^{\infty} x_j 1_{H_j(\omega)}\right)$$
се нарича дискретна случайна величина.

Дефиниция (разпределение на дискретна случайна величина): Нека X е дискретна случайна величина. Тогава таблицата:

$\boldsymbol{X}$	$\boldsymbol{x_1}$	$\boldsymbol{x_2}$	$\boldsymbol{\dots}$	$\boldsymbol{x_k}$	$\boldsymbol{\dots}$
$\mathbb{P}$	p_1	p_2	$\dots$	p_k	$\dots$

където $\mathbb{P}(X = x_i) = p_i = P(H_i)$ и $\sum_i p_i = 1, p_i \geq 0$ се нарича разпределение на X.

Дефиниция (математическо очакване на ДСВ): Нека X е дискретна случайна величина. Тогава:

$$\mathbb{E}X \coloneqq \sum_j x_j p_j$$

се нарича математическо очакване на X, ако:

$$\sum_j |x_j| p_j < \infty.$$

Дефиниция (дисперсия на ДСВ): Ако X е дискретна случайна величина, то:

$$\mathbb{D}X \coloneqq \sum_j \left(x_j - \mathbb{E}X\right)^2 p_j$$

е дисперсията на X, ако $\mathbb{D}X < \infty, \mathbb{E}X < \infty$.

$\mathbb{D}X = \mathbb{E}(X - \mathbb{E}X)^2 = \mathbb{E}X^2 - (\mathbb{E}X)^2$

Пораждаща функция

Дефиниция (пораждаща функция): Нека X е случайна величина във вероятностно пространство $V = (\Omega, \mathcal{A}, \mathbb{P})$ като $X: \Omega \to \mathbb{N} = \{0,1,2, \dots\}$. Тогава:

$$g_X(s) \coloneqq \mathbb{E}s^X = \sum_{n=0}^{+\infty} s^n \mathbb{P}(X = n) \text{ за } |s| \leq 1,$$

се нарича пораждаща функция на X.

Основни свойства на пораждащата функция:

1. $\mathbb{E}X = g_X'(1)$
2. $\mathbb{D}X = g_X''(1) + g_X'(1) - (g_X'(1))^2$
3. $\mathbb{P}(X = k) = \frac{g_X^{(k)}(0)}{k!}$
4. Нека X и Y са независими случайни величини и $X \in \mathbb{N}_0^+$ и $Y \in \mathbb{N}_0^+$. Тогава: $g_{X+Y}(s) = g_X(s) g_Y(s)$

Основни дискретни разпределения

I. Равномерно разпределение: $\{m, m+1, m+2, \dots, s = m+n-1\}$ общо $n \in \mathbb{Z}$ числа

Казваме, че X е равномерно разпределена върху $I \coloneqq \{m, \dots, s\}$, ако:

$$\mathbb{P}(X = k) = \frac{1}{n} \text{ за } k \in I$$

$$\mathbb{E}X = \frac{1}{n}.m + \frac{1}{n}.(m+1) + \cdots + \frac{1}{n}.(m+n-1) = \frac{1}{n}\big(n.m + (1+2+\cdots+n-1)\big) =$$
$$= m + \frac{(n-1)n}{2n} = m + \frac{n-1}{2} = \frac{2m-m+m+n-1}{2} = \frac{m+s}{2}$$

Забелязваме, че: $X = m-1+Y$, където Y е равномерно върху $\{1,2,\dots,n\}$

$$\mathbb{E}Y = \frac{1}{n}(1+2+\cdots+n) = \frac{1}{n}.\frac{n(n+1)}{2} = \frac{n+1}{2}$$

$$\mathbb{E}X = \mathbb{E}[m-1+Y] = m-1+\mathbb{E}Y = m-1+\frac{n+1}{2}$$

$$\mathbb{E}Y^2 = \frac{1}{n}.1^2 + \frac{1}{n}.2^2 + \cdots + \frac{1}{n}.n^2 = \frac{1}{n}(1^2+2^2+\cdots+n^2) = \frac{1}{n}.\frac{n(n+1)(2n+1)}{6} =$$
$$= \frac{(n+1)(2n+1)}{6}$$

$$\mathbb{D}Y = \mathbb{E}Y^2 - (\mathbb{E}Y)^2 = \frac{(n+1)(2n+1)}{6} - \left(\frac{n+1}{2}\right)^2 = \frac{2(n+1)(2n+1) - 3(n+1)^2}{12} =$$
$$= \frac{(n+1)(4n+2-3n-3)}{12} = \frac{(n+1)(n-1)}{12} = \frac{n^2-1}{12}$$

$$\mathbb{D}X = \mathbb{D}[m-1+Y] = \mathbb{D}Y = \frac{n^2-1}{12}, \qquad \text{тъй като } \mathbb{D}(X+c) = \mathbb{D}X$$

Пример: *хвърляне на правилна (честна) монета, правилен зар; теглене на карти от правилно тесте, т.е. всички възможни елементарни изходи са равновероятни*

II. Разпределение на Бернули: $X \sim Ber(p), p \in [0,1]$

X	0	1
$\mathbb{P}$	$1-p$	p

Полагаме: $q \coloneqq 1-p$

$$\mathbb{E}X = 0.(1-p) + 1.p = p$$

$$g_X(s) = \mathbb{E}s^X = s^0.(1-p) + s^1.p = 1-p+sp = ps+q$$

$$\mathbb{D}X = g_X''(1) + g_X'(1) - \big(g_X'(1)\big)^2 = 0 + p - p^2 = p.(1-p) = pq$$

Имаме случаен експеримент с вероятност за успех p и вероятност за неуспех $1-p \coloneqq q$.

Пример: хвърляне на монета, където да се падне ези приемаме за успех или обратно

III. Биномно разпределение: $X \sim Bin(n,p), n \in \mathbb{N}_0, p \in [0,1]$

$$X = \sum_{i=1}^{n} X_i,$$

където X_i са бернулиево разпределени случайни величини.

X отразява броя успехи измежду n случайни експеримента (еднородни, независими).

$$X = \sum_{j=1}^{n} X_j \rightarrow \text{независими в съвкупност бернулиеви случайни величини}$$

Знаем, че $g_{X_j}(s) = ps + q$. От твърдение следва, че:

$$g_X(s) = \prod_{j=1}^{n} g_{X_j}(s) = \prod_{j=1}^{n} (ps+q) = (ps+q)^n = \sum_{k=0}^{\infty} s^k \binom{n}{k} p^k q^{n-k}$$

$$\mathbb{E}X = \mathbb{E}\sum_{j=1}^{n} X_j = \sum_{j=1}^{n} \mathbb{E}X_j = \sum_{j=1}^{n} p = np$$

$$\mathbb{D}X = \mathbb{D}\sum_{j=1}^{n} X_j = \sum_{j=1}^{n} \mathbb{D}X_j = \sum_{j=1}^{n} pq = npq,$$

тъй като X_j са независими в съвкупност случайни величини.

$$\mathbb{P}(X=k) = \frac{1}{k!} g_X^{(k)}(0), 0 \le k \le n$$

$$g_X^{(k)}(0) = \frac{\partial^k}{\partial s^k}(q+ps)^n = np(n-1)p \dots (n-k+1)p(ps+q)^{n-k}|_{s=0} =$$
$$= n(n-1)\dots(n-k+1)p^k q^{n-k}$$

$$\mathbb{P}(X=k) = \frac{n(n-1)\dots(n-k+1)p^k q^{n-k}}{k!} = \frac{n!}{k!\,(n-k)!} p^k q^{n-k} = \binom{n}{k} p^k q^{n-k}$$

Пример: хвърляне на монета 10 пъти, където търсим каква е вероятността ези да се падне 6 пъти: $X \sim Bin(10,0.5)$; $\mathbb{P}(X=6) = ?$

IV. Геометрично разпределение: $X \sim Ge(p)$

$$X = \min_{k \ge 1} \left\{ \sum_{j=1}^{k} X_j = 1 \right\} - 1$$

т.е. най-малкото k, за което сумата $\sum_{j=1}^{k} X_j$ става единица, като от нея вадим единица, за да извадим последната стъпка (успеха). Това е броят неуспехи до първи успех.

$$\mathbb{P}(X=k)=\mathbb{P}(X_1=0,X_2=0,\dots,X_k=0,X_{k+1}=1)=q.q\dots q.p=q^k p,\qquad k\geq 0$$

$$g_X(s)=\mathbb{E}s^X=\sum_{k=0}^{\infty}s^k\mathbb{P}(X=k)=\sum_{k=0}^{\infty}s^k q^k p=p\sum_{k=0}^{\infty}(qs)^k=p\frac{1}{1-qs}=\frac{p}{1-qs},$$

тъй като $q<1, s<1$, т.е. $qs<1$.

$$\mathbb{E}X=g_X'(1)=p\frac{\partial}{\partial s}\left(\frac{1}{1-qs}\right)=p.(-1).\frac{\frac{\partial}{\partial s}(1-qs)}{(1-qs)^2}=-p.\frac{-q}{(1-qs)^2}=\frac{pq}{(1-qs)^2}|_{s=1}=$$
$$=\frac{pq}{(1-q)^2}=\frac{pq}{p^2}=\frac{q}{p}$$

$$\mathbb{D}X=g_X''(1)+g_X'(1)-\left(g_X'(1)\right)^2=\frac{\partial}{\partial s}\left(\frac{pq}{(1-qs)^2}\right)+\frac{q}{p}-\left(\frac{q}{p}\right)^2=$$
$$=pq.(-2).\frac{\frac{\partial}{\partial s}(1-qs)}{(1-qs)^3}+\frac{q}{p}-\frac{q^2}{p^2}=-2pq.\frac{-q}{(1-qs)^3}|_{s=1}+\frac{q}{p}-\frac{q^2}{p^2}=$$
$$=\frac{2pq^2}{(1-q)^3}+\frac{q}{p}-\frac{q^2}{p^2}=\frac{2q^2}{p^2}+\frac{q}{p}-\frac{q^2}{p^2}=\frac{q^2}{p^2}+\frac{qp}{p^2}=\frac{q^2+q(1-q)}{p^2}=\frac{q}{p^2}$$

Пример: Брой тури при хвърляне на монета до първо ези. В този случай хвърлянето на ези считаме за успех, а на тура – за неуспех. Така: $X\sim Ge(p=0.5)$.

Твърдение: Ако $X\sim Ge(p)$ и $m\geq 0$, то тогава:

$$\mathbb{P}(X\geq m+n|X\geq m)=\mathbb{P}(X\geq n)=q^n, n\geq 0.\ \text{(безпаметност)}$$

V. Отрицателно биномно разпределение: $X\sim NB(r,p), p\in(0,1), r\geq 1$

$$X=\min_{k\geq 0}\left\{\sum_{i=1}^{k}X_i=r\right\}-r\rightarrow \text{брой неуспехи до } r-\text{ти успех}$$

Това разпределение се конструира от няколко геометрични случайни величини, т.е. тук ще моделираме броя на неуспехите, но не до първия успех, а до r-тия успех:

$$X=\sum_{i=1}^{r}Y_i,\ \text{където } Y_i\sim Ge(p) \text{ са независими в съвкупност}$$

$$g_X(s)=\prod_{j=1}^{r}g_{Y_j}(s)=\prod_{j=1}^{r}\frac{p}{1-qs}=\left(\frac{p}{1-qs}\right)^r$$

$$\mathbb{E}X=\mathbb{E}\sum_{i=1}^{r}X_i=\sum_{i=1}^{r}\mathbb{E}X_i=\sum_{i=1}^{r}\frac{q}{p}=\frac{rq}{p}$$

$$\mathbb{D}X = \mathbb{D}\sum_{i=1}^{r} X_i = \sum_{i=1}^{r} \mathbb{D}X_i = \sum_{i=1}^{r} \frac{q}{p^2} = \frac{rq}{p^2}$$

$$\mathbb{P}(X = k) = \binom{r+k-1}{k} p^r q^k$$

Пример: Брой тури при хвърляне на монета до шесто ези. В този случай хвърлянето на ези считаме за успех, а на тура – за неуспех. Така: $X \sim NB(6, 0.5)$.

VI. Поасоново разпределение: $X \sim Poi(\lambda), \lambda > 0$

Казваме, че X е поасоново разпределена случайна величина с параметър $\lambda > 0$ и пишем $X \sim Poi(\lambda)$, ако:

$$\mathbb{P}(X = k) = \frac{\lambda^k}{k!} e^{-\lambda}, k \geq 0$$

Разпределението е дефинирано добре за всеки възможен елементарен изход, защото:

$$1 \overset{?}{=} \sum_{k=0}^{\infty} \mathbb{P}(X = k) = \sum_{k=0}^{\infty} \frac{\lambda^k}{k!} e^{-\lambda} = e^{-\lambda} \sum_{k=0}^{\infty} \frac{\lambda^k}{k!} = e^{-\lambda} e^{\lambda} = 1$$

$$g_X(s) = \mathbb{E}s^X = \sum_{k=0}^{\infty} s^k \mathbb{P}(X = k) =$$

$$= \sum_{k=0}^{\infty} s^k . \frac{\lambda^k}{k!} e^{-\lambda} = e^{-\lambda} \sum_{k=0}^{\infty} \frac{s^k \lambda^k}{k!} = e^{-\lambda} \sum_{k=0}^{\infty} \frac{(s\lambda)^k}{k!} = e^{-\lambda} e^{s\lambda} = e^{\lambda(s-1)} \text{ за } |s| \leq 1$$

$$\mathbb{E}X = g'_X(1) = (e^{\lambda(s-1)})'|_{s=1} = \lambda . e^{\lambda(s-1)}|_{s=1} = \lambda$$

$$\mathbb{D}X = g''_X(1) + g'_X(1) - \left(g'_X(1)\right)^2 = \left(\lambda e^{\lambda(s-1)}\right)'|_{s=1} + g'_X(1) - \left(g'_X(1)\right)^2 =$$
$$= \lambda^2 e^{\lambda(s-1)}|_{s=1} + g'_X(1) - \left(g'_X(1)\right)^2 = \lambda^2 + \lambda - \lambda^2 = \lambda$$

Пример 1: брой клиенти за единица време

Пример 2: брой постъпили заявки към сървър за единица време

Теорема (Поасон): Нека p_n е редица от реални числа от интервала $[0,1]$, такива че редицата np_n схожда до крайна граница λ. Тогава:

$$\lim_{n\to\infty} \binom{n}{k} p_n^k (1 - p_n)^{n-k} = e^{-\lambda} . \frac{\lambda^k}{k!}$$

Рецепта: $\lambda = np \leq 20$ и $n \geq 100$ (тогава се твърди, че приближението е добро)

VII. Хипергеометрично разпределение: $X \sim HG(N, M, n), N, M, n \in \mathbb{Z}^+$

$N \rightarrow$ брой обекти (общо)

$M \rightarrow$ маркирани (дефектни) (област от N)

$n \rightarrow$ общ брой изтеглени

***X* измерва броя на маркираните от изтеглените n**

$$\mathbb{P}(X = k) = \frac{\binom{M}{k}\binom{N-M}{n-k}}{\binom{N}{n}}, \quad N - M \geq n - k$$

$$\mathbb{E}X = n.\frac{M}{N}$$

$$\mathbb{D}X = n.\frac{M}{N}.\frac{N-M}{N}.\frac{N-n}{N-1}$$

Основни непрекъснати разпределения

Дефиниция (непрекъсната случайна величина): Нека X е случайна величина във вероятностно пространство V ($X: \Omega \rightarrow \mathbb{R}$). Тогава X е непрекъсната случайна величина, ако X може да приема неизброимо много стойности.

Дефиниция (непрекъсната случайна величина): X е абсолютно непрекъсната (непрекъсната) случайна величина, ако $\exists\, f_X: \mathbb{R} \rightarrow [0, \infty)$ (плътност на X), такава че:

a. $f_X(x) \geq 0, \forall x \in \mathbb{R}$
b. $\int_{-\infty}^{\infty} f_X(x)dx = 1$
c. за $\forall -\infty < a < b < \infty\ \mathbb{P}\big(X \in (a, b)\big) = \int_a^b f_X(x)dx$

Дефиниция (математическо очакване на НСВ): $\mathbb{E}X = \int_{-\infty}^{\infty} x f_X(x)dx$

Дефиниция (дисперсия на НСВ): $\mathbb{D}X = \mathbb{E}(X - \mathbb{E}X)^2 = \int_{-\infty}^{\infty}(x - \mathbb{E}X)^2\, f_X(x)dx$

I. Равномерно разпределение: $X \sim U(a, b),\ a, b \in \mathbb{R},\ a < b$, ако

$$f_X(x) = \begin{cases} \dfrac{1}{b-a}, & x \in (a, b) \\ 0, & \text{иначе} \end{cases}$$

$$\int_a^b \frac{1}{b-a}\,dx = \frac{b-a}{b-a} = 1 \rightarrow \text{дефинира плътност}$$

$$\mathbb{E}X = \int_a^b x.\frac{1}{b-a}\,dx =$$

$$= \frac{1}{b-a}.\int_a^b x dx = \frac{1}{b-a}.\frac{x^2}{2}|_a^b = \frac{1}{b-a}.\left(\frac{b^2}{2} - \frac{a^2}{2}\right) = \frac{b^2-a^2}{2(b-a)} = \frac{(b+a)(b-a)}{2(b-a)}$$

$$= \frac{a+b}{2}$$

$$\mathbb{D}X = \mathbb{E}X^2 - (\mathbb{E}X)^2 =$$

$$= \int_a^b x^2.\frac{1}{b-a}\,dx - \left(\frac{a+b}{2}\right)^2 = \frac{1}{b-a}.\frac{x^3}{3}|_a^b - \frac{(a+b)^2}{4} =$$

$$= \frac{1}{b-a}.\left(\frac{b^3}{3} - \frac{a^3}{3}\right) - \frac{(a+b)^2}{4} = \frac{(b-a)(b^2+ab+a^2)}{3(b-a)} - \frac{a^2+2ab+b^2}{4} =$$

$$= \frac{4b^2+4ab+4a^2-3a^2-6ab-3b^2}{12} = \frac{b^2-2ab+a^2}{12} = \frac{(b-a)^2}{12}$$

функция на разпределение $\leftarrow F_x(x) = \mathbb{P}(X < x) = \begin{cases} 0, & x \le a \\ \int_a^x \frac{1}{b-a}\,dy = \frac{x-a}{b-a}, & x \in (a,b] \\ 1, & x > b \end{cases}$

Пример: Влаковете на метрото пътуват на интервали от 10 минути. Случаен пътник пристига на определена спирка по случаен начин. Тогава с X означаваме „броя минути чакане до следващия влак“. В този случай: $X \sim U(0,10)$ и например вероятността човек да чака по-малко от 5 минути е: $\mathbb{P}(X < 5) = \frac{1}{10-0}(5-0) = 0.5$.

II. Нормално разпределение: $X \sim N(\mu, \sigma^2), \mu \in \mathbb{R}, \sigma^2 > 0$, ако:

$$f_X(x) = \frac{1}{\sqrt{2\pi}\sigma} e^{-\frac{(x-\mu)^2}{2\sigma^2}}, \quad x \in (-\infty, \infty)$$

$\mu = 0, \sigma^2 = 1, Z \sim N(0,1) \rightarrow$ стандартно нормално разпределение

Ако $X \sim N(\mu, \sigma^2)$, то $Y = \frac{X-\mu}{\sigma} \sim N(0,1)$.

$$\mathbb{E}X = \mu, \mathbb{D}X = \sigma^2$$

Пример: средната месечна заплата в дадена държава е нормално разпределена случайна величина $X \sim N(1000, 120)$

III. Експоненциално разпределение: $X \sim Exp(\lambda), \lambda > 0$

$$f_X(x) = \begin{cases} \lambda e^{-\lambda x}, & x > 0 \\ 0, & \text{иначе} \end{cases}$$

$$F_X(x) = \int_0^x \lambda e^{-\lambda y} dy = -\int_0^x e^{-\lambda y} d(-\lambda y) = -e^{-\lambda y}|_0^x = 1 - e^{-\lambda x}$$

$$\mathbb{E}X = \int_0^\infty \lambda x e^{-\lambda x} dx = \frac{1}{\lambda}$$

$$\mathbb{D}X = \mathbb{E}X^2 - (\mathbb{E}X)^2 = \frac{1}{\lambda^2}$$

Безпаметност: $\mathbb{P}(X \geq x + y | X \geq x) = \mathbb{P}(X \geq y) \rightarrow$ непрекъснатия еквивалент на геометричното разпределение

Пример: времето за безотказна работа на двигател е експоненциално разпределено с параметър $\lambda = \frac{1}{3}$